# 序言

著名意大利物理学家、近现代科学之父伽利略曾说：“宇宙这本书是用数学的语言写成的”，是他开启了用数学方法研究物理问题的先河。在人类发展的历史长河中，数学发挥着不可替代的作用，是人类学习和研究科学技术必不可少的基本工具。数学理论的掌握和突破，往往预示着重大自然科学理论的诞生。比如，正是因为牛顿具有极强的数学天赋并与莱布尼茨共同发明了微积分，才率先发现并证明了万有引力定律；正是因为麦克斯韦有扎实的数学基础，才取得了巨大的电磁学成就并预言了电磁波的存在；而爱因斯坦要研究相对论，也必须先掌握黎曼数学。此外很多数学理论也是众多商业模式的基础，比如在金融、保险、博彩等行业，概率论、博弈论的数学思想大行其道。

当然，计算机更是数学理论发展的必然结果，在大学早期，计算机系本来就是数学系的一个专业分支方向。都说学好计算机的前提是学好数学，因为计算机与数学是息息相关的。那么究竟哪些数学知识和理论，与计算机科学与技术紧密相关呢？本书要为大家一一揭开。

## 计算机与数学的关系

1936 年，英国数学家和计算机科学之父图灵（1912-1954，如图 0-1 所示）向伦敦权威的数学杂志投了一篇论文，题为“论数字计算在决断难题中的应用”。在这篇开创性的论文中，图灵给“可计算性”下了一个严格的数学定义，并提出著名的“图灵机”的设想，从理论上证明了，计算机确实是可以造成出来的。

图 0-1 计算机科学之父图灵

如图 0-2 所示，图灵构造出的这台假想机器，由以下几个部分组成:

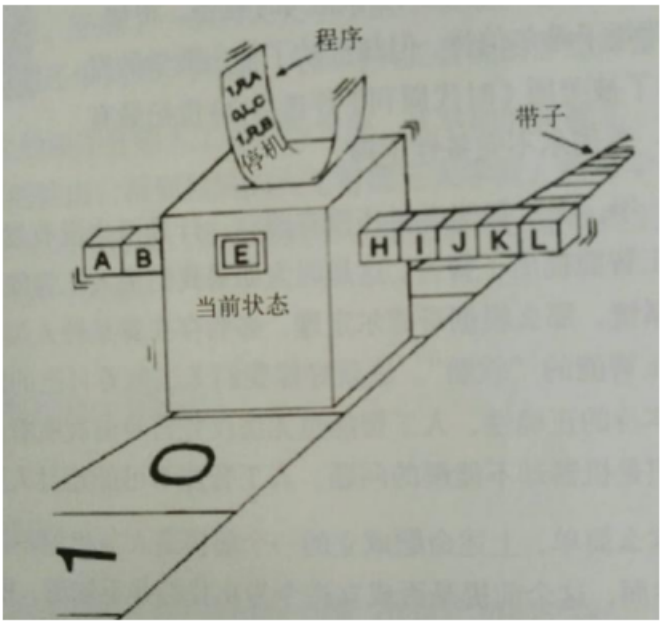



图 0-2 图灵机

（1）一条无限长的纸带 TAPE。纸带被划分为一个接一个的小格子，每个格子上包含一个来自有限字母表的符号，字母表中有一个特殊的符号 表示空白。纸带上的格子从左到右依此被编号为 0, 1, 2, ...，纸带的右端可以无限伸展。

（2）一个读写头 HEAD。该读写头可以在纸带上左右移动，它能读出当前所指的格子上的符号，并能改变当前格子上的符号。

（3）一套控制规则 TABLE。它根据当前机器所处的状态以及当前读写头所指的格子上的符号来确定读写头下一步的动作，并改变状态寄存器的值，令机器进入一个新的状态。

（4）一个状态寄存器。它用来保存图灵机当前所处的状态。图灵机的所有可能状态的数目是有限的，并且有一个特殊的状态，称为停机状态。

既然图灵从数学理论上证明了计算机是可以造出来的，那么究竟是谁最终造出了第一台现代计算机呢？这个光荣的任务交给了冯·诺依曼（1903-1957，如图 0-3 所示）。

图 0-3 现代计算机之父冯·诺依曼

冯·诺依曼领导造出了现代意义的第一台计算机。20 世纪 40 年代，冯·诺依曼在参与世界上第一台计算机-ENIAC（Electronic Numerical Integrator And Computer，即电子数字积

分计算机，如图 0-4 所示）的研制小组工作时，发现 ENIAC 有两个致命的缺陷：一是采用十进制运算，逻辑元件多，结构复杂，可靠性低；二是没有内部存储器，操纵运算的指令分散存贮在许多电路部件内，这些运算部件如同一副积木，解题时必须像搭积木一样用人工把大量运算部件搭配成各种解题的布局，每算一题都要搭配一次，非常麻烦且费时。

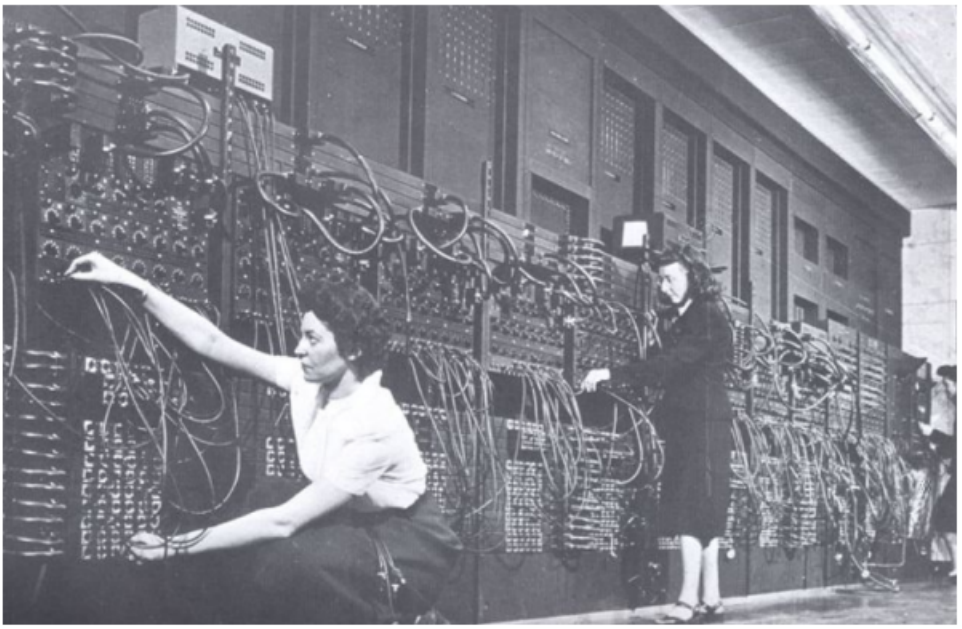


图 0-4 全球首台电子计算机 ENIAC

针对这两个问题，冯·诺依曼和其他合作者一起进行了半年多时间的改革性研究，取得了令人满意的成果。但是，由于 ENIAC 的制造已接近尾声，因此它未能采用冯·诺依曼的改进意见。

冯·诺依曼的研究成果得到了 ENIAC 研制小组专家的重视和青睐，他们在 ENIAC 尚未竣工之前，就着手计划一个结构全新的电子计算机-EDVAC（Electronic Discrete Variable Automatic Computer，离散变量自动电子计算机）方案。1945 年 6 月底，由冯·诺依曼执笔写出了 EDVAC 计划草案。在这个方案中，冯·诺依曼提出了在计算机中采用二进制算法和设置内存贮器的理论，并明确规定了电子计算机必须由运算器、控制器、存储器、输入设备和输出设备等五大部分构成的基本结构形式。他认为，计算机采用二进制算法和内部存储器后，指令和数据便可以一起存放在内部存储器中，并可作同样处理，这样，不仅可以使计算机的结构大大简化，而且为实现运算控制自动化和提高运算速度提供了良好的条件。EDVAC 于 1952 年建成，它的运算速度与 ENIAC 相似，而使用的电子管却只有 5900 多个，比 ENIAC 少得多。EDVAC 的诞生，使计算机技术出现了一个新的飞跃。它奠定了现代电子计算机的基本结构，标志着电子计算机时代的真正开始。

因此图灵从理论上证明了计算机是可以造出来的，被称为计算机科学之父，而冯·诺依曼又被称为现代计算机之父，以此纪念他造出了具有现代意义的第一台计算机。

从计算机的诞生背景，我们可以看出，可计算理论，二进制都是计算机必不可少的数学理论基础。

## 学习计算机基础阶段与数学

程序员必须要学好数学。一旦你对计算机感兴趣，想投身计算机领域，成为一名优秀的程序员，那么你首先要考虑的就是**自己的数学基础如何**？因为这决定了你在以后的职业生涯

中的潜力，即能学多深，能走多远。

比较幸运的是，学习计算机基础阶段，我们并不需要特别高深的数学理论，只要学过中学数学（当然你必须保证你学得还不错），就完全可以以此为基础，按计划推进计算机基础阶段的学习了。

在我们一开始学习计算机编程技术的时候，往往要从一门编程语言（比如 C/C++，Java，Python 等）开始学习，如图 0-5 所示。这个时候，我们要学习计算机中数的表示，因此必然会首先学习二进制，然后学习整数，实数等在计算机中的表示方法。从这个时候开始，计算机知识和数学就开始关联起来。要学好二进制以及数据在计算机中的表示，我们必须要知道什么是指数运算，对数运算，什么是数列和数列求和公式等等这些中学数学就包含的数学理论基础。

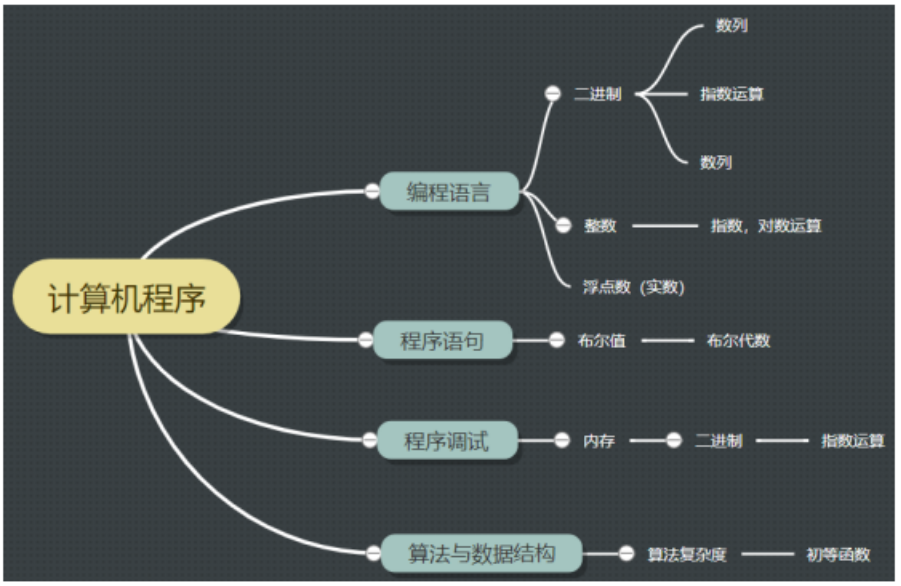



图 0-5 计算机程序入门与数学关系

在编程语言里，我们还得学习程序语句。程序语句由顺序语句，分支语句和循环语句组成。程序在执行语句的时候，会判断各种条件是否成立，因此涉及了“与（&&）”，“或（||）”，“非（！）”的逻辑运算，逻辑运算的结果就是一个布尔值，也就是“真”和“假”（即 true 和 false 或者 1 和 0）。这里用到的就是布尔逻辑代数的理论基础。

在调试编程语言编写的程序的时候，我们往往会涉及到程序执行期间计算机内存数据的查看与分析。要深入理解计算机中的内存概念，就必须涉及计算机操作系统中的实模式，保护模式和分页机制。要理解这些理论，就必须要懂数学中的二进制与指数运算规则。

在编程语言里，我们会编写以函数为基础的各种算法代码。函数也是一个非常重要的数学概念。在研究算法的复杂度方面，我们必须要掌握数学中的各种基本函数，比如幂函数，指数函数和对数函数等。比较各种算法的时间和空间复杂度，就是基于这些基本的数学函数相关知识。

## 学习计算机中、高级阶段与数学

随着我们学习的深入，我们可能会选择计算机领域的不同专业方向。现在比较热门的专业方向包括信息安全、云计算、大数据、机器学习、深度学习、人工智能、区块链等。这些领域就需要用到更高深的数学知识了。

**1. 信息安全**

在信息安全方面，数学最重要的应用领域就是信息加解密领域。加解密算法是计算机信

息安全的基础。如果加解密算法出问题了，基于加解密算法建立起来的整个安全体系就会立即土崩瓦解。所以在2018年号称已经被证明的有关素数的“黎曼猜想”也引起了很多计算机专家的广泛关注和担心，因为这会直接动摇基于素数理论建立起来的非对称加解密算法RSA算法。另一方面，椭圆曲线非对称加密也是一个强大的加解密算法。然而它却比RSA算法需要更高深的椭圆曲线等相关的数学知识。

区块链技术里面也用到了大量的加解密算法。在整个交易从发起到验证过程中，信任完全是依赖非对称加密算法进行保证。

**2.人工智能**

人工智能是计算机领域另一个非常重要的方向。人工智能主要包括五大核心技术，即计算机视觉，语音识别，自然语言处理，机器学习和机器人技术。这些技术让计算机学会了如何看，如何听，如何读，如何学习等人类才具备的能力。人工智能领域的各项技术对数学的要求比任何其他计算机领域都高。如果没有扎实的数学基础，学习人工智能技术将是寸步难行。这些数学理论基础包括但不限于如下几个重要数学分支：

**（1）高等数学。**

首先，高等数学是理工科基础中的基础，一切理工科领域复杂的问题和理论，都会用到。其中高等数学中的极限是微积分的基础，微积分是概率论的基础，概率论又是机器学习算法和人工智能的基础。

**（2）概率论与数理统计。**

人工智能要解决各种不确定问题，这需要数学为其提供不确定推理的基础，概率理论则是实现不确定推理的数学基础。概率论、随机过程、数理统计构成了概率理论知识体系，为人工智能处理各种不确定问题奠定了基础。在概率论的基础上，出现了条件概率及贝叶斯定理，奠定了大多数人工智能系统中不确定推理的现代方法基础。

人工智能的研究中，数理统计同样不可或缺。数理统计是基于大数据的机器学习的理论基础。基础的统计理论有助于对机器学习的算法和数据挖掘的结果做出解释，只有做出合理的解读，数据的价值才能够体现。数理统计的任务是根据可观察的样本反过来推断总体的性质；推断的工具是统计量，统计量是样本的函数，是个随机变量；参数估计通过随机抽取的样本来估计总体分布的未知参数。

**（3）可计算理论。**

可计算理论是人工智能的重要理论基础和工具，为了回答是否存在不可判定的问题，数理逻辑学家提出了关于算法的定义。可以被计算，就是要找到一个解决问题的算法。在不可计算性以外，如果解决一个问题需要的计算时间随着实例规模呈指数级增长，则该问题被称为不可计算的，对这个问题的研究产生了计算复杂性。可计算性和计算复杂性为人工智能判断问题求解可能性奠定了数学基础。

**（4）线性代数。**

线性代数里提供了完备的向量与矩阵理论基础，从量子力学到图像处理都离不开向量和矩阵的使用。大量的计算数据都是存放在向量或者矩阵之中的。在向量和矩阵背后，线性代数的核心意义在于提供了一种看待世界的抽象视角，即万事万物都可以被抽象成某些特征的组合，并在由预置规则定义的框架之下以静态和动态的方式加以观察。向量的实质是n维线性空间中的静止点；线性变换描述了向量或者作为参考系的坐标系的变化，可以用矩阵表示；矩阵的特征值和特征向量描述了变化的速度与方向。

**（5）最优化理论。**

人工智能的目标就是最优化，即在复杂环境与多体交互中做出最优决策。几乎所有的人工智能问题最后都会归结为一个优化问题的求解，因而最优化理论是人工智能必备的基础知识。最优化理论研究的问题是判定给定目标函数的最大值（最小值）是否存在，并找到令目

标函数取到最大值 (最小值) 的数值。如果把给定的目标函数看成一座山脉，最优化的过程就是判断山脉顶峰的位置并找到到达顶峰路径的过程。

通常情况下，最优化问题是在无约束情况下求解给定目标函数的最小值；在线性搜索中，确定寻找最小值时的搜索方向需要使用目标函数的一阶导数和二阶导数；置信域算法的思想是先确定搜索步长，再确定搜索方向；以人工神经网络为代表的启发式算法是另外一类重要的优化方法。

本书将详细介绍与计算机理论息息相关的数学基础理论知识。从计算机中最基本的数的表示到高深复杂的人工智能，这里面用到了哪些数学理论和方法，解决了什么问题都是本书重点讲解的地方。同时，本书的主要目标是为程序员学习计算机理论扫清数学基础障碍和提供学习方向，因此注重的是一种数学素养的培养，而不是通常大家遇到的以面向各种考试（比如高考和考研）为目的的学习场景，最重要的是不会搞各种题海战术。当然，数学本身是一门非常注重逻辑性和抽象性的学科，本身的很多理论是非常晦涩难懂的，证明过程也会比较复杂，因此试图把高深复杂的数学理论结合计算机问题，以通俗易懂的语言表述出来，是本书的目标之一。

为了降低学习难度，本书中部分定理，性质或者结论并没有给出详细证明和推导过程，这并不意味着它们的证明和推导不重要，因此对学有余力的同学，我们推荐大家自行找到相关参考资料或书籍进一步学习研究，做到知其然并知其所以然。

本书适合作为跨专业学生自学计算机技术的数学参考用书，适合作为学习信息安全技术的数学参考用书，也适合作为青少年学习编程技术的数学参考用书，或作为普及数学理论知识提高数学素养的数学科普读物。

# 第 1 章 数与二进制

数是数学和计算机中的基础。数是数学中的重要研究对象，数学里有专门一个数论分支来研究数，而计算机科学中为了处理数据和信息，首先要解决的就是数的存储与表示方法。

本章我们将从数的分类开始讲起，由于计算机体系结构是基于二进制进行设计和实现的，所以紧接着我们将给大家详细介绍包括二进制在内的几种常见数的进制及相互转换方法，最后再进入计算机领域，详细的介绍数在计算机中的编码方法和存储方式。

人类很早就发现了十进制记数法，然后古印度人发明了 1, 2, 3, 4, 5, 6, 7, 8, 9, 0（0 最开始是以实心圆点表示）十个数字，如图 1-1 所示。公元 8 世纪印度数字传入阿拉伯，然后传入欧洲（欧洲人误以为是阿拉伯人发明的，因此称为阿拉伯数字，实际上中国的指南针，造纸术，印刷术和火药也是通过阿拉伯人传入欧洲世界的），0 稍晚传入，对欧洲近代科学进步扮演重要角色。

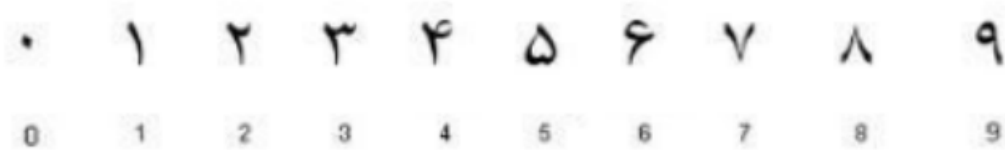



图 1-1 古印度人发明的数字与现代数字对照

与其他数字相比，古印度人对“0”的发明意义非常重大，因为 0 作为“无”的概念非常抽象，一般古人很难单独抽象出这个概念来。“0”在数的表示中起到了占位符的作用，比如 2020 中的 0，虽然它的个位和百位没有值，但是为了表示 2020 这个数，必须使用 0 占位，如果不用 0 占位，那么为了表示 2020，会非常麻烦。数字“0”大大简化了加减乘除运算，与此相对应的罗马数字就没有 0，比如要表示 1999，就必须要用 MCMXCIX，非常复杂，而且要用罗马数字来进行乘除运算，也是非常困难的。大多数民族都能发明十进制或者十二进制或者六十进制，但却很少有发明 0 的。因为从 1 到 9 都是看得见摸得着的，而且人有 10 个手指头。但如何把无这个概念抽象成 0，确实需要一定的创造性。但是，当 0 在中世纪一开始进入欧洲的时候，欧洲教会的势力非常强大，他们千方百计地阻止 0 的传播，甚至有人为了传播 0 而被处死。

当然，黑暗终将过去，科学终将迎来春天。随着勾股定理的发现和圆周率 π 的探讨研究，人类发现了无理数，引起了第一次数学危机。后来数学不断发展，数的大家庭不断添加新成员，产生了自然数，整数，奇数，偶数，合数，素数，有理数，无理数，实数，虚数和复数等概念。

既然有了数字，那么人类如何把数字保存起来呢？这也是在摸索中不断前进的。我们的祖先最开始是把包含数字在内的文字刻在死去的动物骨头上，我们把它叫做甲骨文，后来刻在竹简上；而在国外，也有刻在泥板上，纸草上，羊皮卷上，甚至靠绳子打结来计数的。最后我们的古人发明了纸，文字和数字才得以方便的和长久的保存。现在有了计算机，数字以及数据终于找到了最完美的归宿，以电磁的形式保存在了计算机的内存或者磁盘里，实现了无纸化与信息化。

那么计算机是如何表示和保存数的呢？答案是**使用二进制和相应的编码**，比如整数就是以二进制补码的形式存储的，浮点数也有自己的存储方式，我们接下来会专门论述。

# 1.1 数的分类

人类对数的认识，范围是不断扩大的。从最开始的自然数，到最终的复数概念的建立，这其中经历了漫长而曲折的历史岁月，甚至引起过数学危机，其中 16、17 世纪欧洲大多数数学家还不承认负数是数，数字“0”在一开始传播的时候也阻力重重。下面我们就依次对这些数的分类进行一个简要的介绍和回顾。

**整数(integer)**：像 0，1，2，3，-10，10 等这样的数。分为正整数，0 和负整数。

**自然数（natural number）**：非负整数， 即用数码 0，1，2，3，4，5，……所表示的数，也就是除负整数外的所有整数，通常也被称为自然数。注意，在集合论里，0 算自然数；在数论中，0 一般不算自然数。

**奇数（odd number）和偶数（even number）**：奇数，又称单数。在整数中，不能被 2 整除的数是奇数，能被 2 整除的数是偶数。奇数可用 2k+1 表示，偶数可用 2k 表示，这里 k 是整数。

**素数（也叫质数，prime number ）**：一个大于 1 的自然数，除了 1 和它本身外，不能被其他自然数整除，换句话说就是该数除了 1 和它本身以外不再有其他的因数，比如 2，3，5，7，11 等就是素数。根据算术基本定理，每一个比 1 大的整数，要么本身是一个素数，要么可以写成一系列素数的乘积;而且如果不考虑这些素数在乘积中的顺序，那么写出来的形式是唯一的。最小的素数是 2。偶数和奇数等都可以用公式来分别表示，但是截至目前为止，人们尚未找到一个公式来表示所有素数。2016 年 1 月，发现世界上迄今为止最大的素数，长达 2233 万位，如果用普通字号将它打印出来长度将超过 65 公里。

注意，虽然 1 符合素数的定义，即只能被 1 和它自身整除，但 1 却不是素数。那么为什么 1 不能是素数呢？那是因为整数有一个唯一分解定理，即整数分解质因数具有唯一性，把一个大于 1 的整数分解质因数，不考虑质因数的顺序，那么它的形式是唯一的。比如 6=2×3，只能分解成 2 和 3 相乘的形式。如果 1 是素数，则整数分解的形式就不唯一了，因为可以乘若干个 1。比如 6 可以写成 6=2×3×1，也可以写成 6=2×3×1×1 等等。所以规定 1 不是素数。

**合数（composite number）**：自然数中除了能被 1 和本身整除外，还能被其他的数整除的数，比如 4 和 9 就是合数，因为它们除了能被 1 和自身整除之外，还能被 2 和 3 整除。

**分数（fraction）**：分数是一个整数 p 和一个正整数 q 的不等于整数的比，记为 p/q。分数。一个分数要么是有限小数，要么是无限循环小数，像 π 等这样的无限不循环小数，不能用分数代替。

**有理数（rational number ）**：有理数为整数和分数的统称。正整数和正分数合称为正有理数，负整数和负分数合称为负有理数。因此有理数集中的数可分为正有理数、负有理数和零。由于任何一个整数或分数都可以化为十进制循环小数，反之，每一个十进制循环小数也能化为整数或分数，因此，有理数也可以定义为十进制循环小数。

**无理数（irrational number）**：即非有理数之实数，不能写作两整数之比。若将它写成小数形式，小数点之后的数字有无限多个，并且不会循环。 常见的无理数有大部分的平方根比如$\sqrt{2}$，还有 π 和 e 等。

**有关无理数的小故事：**

无理数诞生之前，古希腊的毕达哥拉斯学派认为宇宙万物都可归结于整数或者整数之比（即有理数），也就是万物皆数（整数）。而无理数最早由毕达哥拉斯学派弟子希帕索斯发现。他研究一个边长为 1 的等边直角三角形，如图 1-2 所示，根据勾股定理，其斜边长应该是"2 的平方根"。如果毕达哥拉斯学派的断言是正确的，那么直边和斜边应该是可公约或者可公度的（存在公约数），即存在一个有理数（即整数之比），恰好等于$\sqrt{2}$。

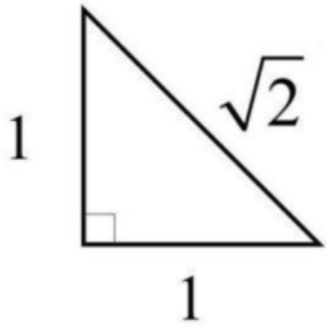



图 1-2 等边直角三角形

那么什么是可公度呢？比如比较两条长度分别为 a 和 b 的两条线段，再找一条小线段 d（当这条小线段当做尺子），如果 a 是 d 的 n 倍，b 是 d 的 m 倍（其中 n 和 m 都是整数），那么 d 为 a 和 b 的共同度量单位并说 a 和 b 是可公约或公度的。

具体来说，假设有两条线段 a 和 b 长度分别为 1.732 米和 2.414 米，选择第三条线段 d 为 1 毫米，那么 a 是 d 的 1732 倍，b 是 d 的 2414 倍。因此只要尺子足够小，总可以把两条线段同时量尽。

希帕索斯证明发现这是一个矛盾的结论，并将结论告诉给了他的老师毕达哥拉斯。新的发现由于违背了毕达哥拉斯的理念，因此他坚决反对这一新的发现，并下令禁止传播这一新的发现。希帕索斯最终将这一数学发现透露给了外人，而被毕达哥拉斯学派的人投进了大海，处以“淹死”的惩罚。毕达哥拉斯深信任意数均可用整数及分数表示，不相信无理数的存在。

这就是数学史上的第一次数学危机，发生于大约公元前 400 年左右的古希腊时期。

公元前 3 世纪，毕达哥拉斯学派的欧多克斯通过在几何学中引入新的不可通约量这个概念来解决希帕索斯发现的矛盾。他指出几何线段先天就存在着"可通约"和"不可通约"的限制，这为无理数找到了存在的基础。

直到 1872 年，德国数学家戴德金(Dedekind）通过“刀砍直线”来对有理数进行"分割"来定义无理数，并把实数理论建立在严格的分析基础上，揭开了无理数的神秘面纱。

戴德金认为既然直线是连续的，那么如果对直线砍一刀，必然砍在某个点 A 上。点不在左边就在右边，不会两边都有，也不会两边都没有。于是可以把数集分成 A，B 两个集合。A 中的任意数都比 B 中的任意数都小。那么要么 A 中有个最大数，B 中有个最小数，二者必居其一，仅居其一。如图 1-3 所示，刀砍在了 2 这个点上，并且 2 位于 A 这个集合上。

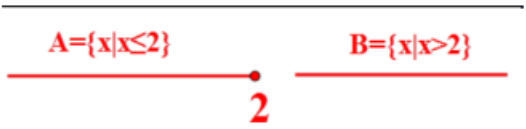



图 1-3 刀砍直线上 2 这个点

同样，我们可以把所有的有理数分成 S 和 T 集合，其中 S 集合中包含负有理数和平方不超过 2 的正有理数；T 集合包含所有平方超过 2 的正有理数。那么 S 中没有最大数，T 中没有最小数（实际上，在 S 中这个最大的数是$\sqrt{2}$，但它不属于有理数）。因此通过对有理数的一次分割，我们发现有理数不足以填充数轴，在数轴上留下了许多空洞，是不连续的。

$$S = \{x \in R | x \leqslant 0 \text{ or } x^2 \leqslant 2\},$$
$$T = \{x \in R | x > 0 \text{ and } x^2 > 2\},$$

因此，我们可以对有理数进行分割，把有理数分为 A，B 两个集合，A 中每个有理数都比 B 中每个数小，那么 A，B 是有理数的一个分割。分割确定了 A，B 之间的位置。这个位置可能是一个有理点（A 的最大值或 B 的最小值），不留缝隙；也可能有缝隙，那么这个缝隙就用无理数来填充，如图 1-4 所示。

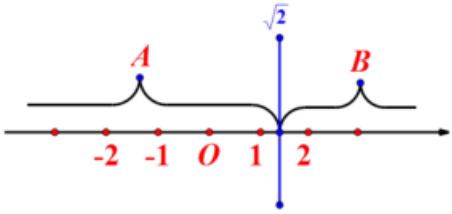



图 1-4 有理数分割产生空隙

因此有理数的一个分割就产生一个实数，带缝隙的分割叫无理数，不带缝隙的分割叫有理数。

戴德金的工作结合康托尔的集合论对实数集的研究，才最终真正结束了自古希腊时代就延续至今的数学史上的第一次危机。

与古希腊相比，世界上其他民族也曾经遇到过无理数的情形，但是对于无理数的反应却不一样，并没有认为这是一种大不了的事，更没有什么危机可言，而是持“居然还有这种数？好吧，那就接受”这种心态，并且侧重于求无理数的近似，并予以应用。而古希腊人费尽心思去思考，发展出一套研究无理数的方法，并引发了一场危机。第一次数学危机，让古希腊科学发展走上了推理证明，逻辑论证之路，成为西方科学发展的先驱和摇篮。

**代数数（algebraic number ）和超越数（ transcendental number）：**我们把满足代数方程 $x^n+a_1x^{n-1}+a_2x^{n-2}+\ldots+a_{n-1}x+a_n=0$（系数都是有理数）的实数根（包括 $\sqrt{2}$ 这种无理数）叫做代数数，实数集里除了代数数的其他数叫做超越数。π 是无理数，这是人尽皆知的。后来数学家们证明了 e 和 π 都是超越数，这意味着它们都不是上面代数方程的根。

**实数（real number）：**实数可以分为有理数和无理数两类，或代数数和超越数两类，或正实数，负实数和零三类。有理数可以分成整数和分数，而整数可以分为正整数、零和负整数。分数可以分为正分数和负分数。无理数可以分为正无理数和负无理数。

实数集合通常用字母 R 或 $R^n$ 表示。而 $R^n$ 表示 n 维实数空间。如图 1-5 所示，实数集和数轴上的点是一一对应的。

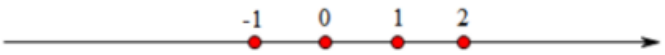



图 1-5 数轴

**虚数（imaginary number）与复数（complex number）：**在数学中，虚数就是形如 a+bi 的数，其中 a,b 是实数，且 $b\neq0$, $i^2=-1$，其中 i 为虚数单位。

虚数单位 i 的指数运算具有周期性：$i^1=i$, $i^2=-1$, $i^3=-i$, $i^4=1$, $i^5=i$, ...。所以 i 的计算以 4 为周期，有 $i^{4n}=1$, $i^{4n+1}=i$, $i^{4n+2}=-1$, $i^{4n+3}=-i$，其中 n 为整数。

**有关虚数小故事：**

虚数这个名词是 17 世纪著名数学家笛卡尔创立，因为当时的观念认为这是真实不存在的数字。后来发现虚数 a+bi 的实部 a 可对应平面上的横轴，虚部 b 可以对应平面上的纵轴，这样虚数 a+bi 可与平面内的点(a,b)对应。

16 世纪意大利数学家卡尔丹等人在研究一元三次方程求解的问题时，就遇到了负数开

平方的问题，但卡尔丹并不承认负数开根号是一个数（即后来的“复数”），因此错过了虚数的发现，但他是第一个使用了$\sqrt{-15}$这样的符号和运算的人。同时另一位数学家邦贝利在研究三次方程 $x^3=15x+4$ 的解时，代用“卡尔丹公式”得到方程的根为 $2\pm\sqrt{-1}$ 表示，邦贝利无法理解$\sqrt{-1}$，也不知道这就是一个虚数，但他给出了$\sqrt{-1}$的运算法则：

$$\sqrt{-1}*\sqrt{-1}=-1 \qquad (1.1)$$

17 世纪著名数学家、解析几何的创始人之一笛卡尔把$\sqrt{-1}$取了一个消极的名字**“虚数”**，意为不可思议的、不存在的、虚无的数，然后一直延用至今。

虚数单位“i”首先为瑞士数学家欧拉所创用，直到德国数学家高斯提倡后才普遍使用。欧拉找到了有关 i，e 和 π 的关系公式，即：

$$e^{\pi i}+1=0 \qquad (1.2)$$

这个公式揭露了 i，e、π、0 和 1 的关系，被誉为最美数学公式。

可以将虚数 bi 添加到实数 a 以形成形如 a + bi 的复数，复数就是指复杂的数。高斯第一个引进术语“复数”并记作 a+bi。复数被定义为二元有序实数对(a, b)，记为 z=a+bi，这里 a 和 b 是实数，i 是虚数单位。在复数 a+bi 中，a 称为复数的实部，b 称为复数的虚部，i 称为虚数单位。当虚部等于零时，这个复数就是实数；当虚部不等于零时，这个复数称为虚数，虚数的实部如果等于零，则称为纯虚数。由上可知，复数集包含了实数集，是实数集的扩张。

两个实部相等，虚部互为相反数的复数互为共轭复数（当虚部不等于 0 时也叫共轭虚数）。复数 z 的共轭复数记作 $\bar{z}$（z 上加一横）。根据定义，若复数 z=x+yi(x，y∈R)，则 $\bar{z}$=x−yi。将复数的实部与虚部的平方和的正的平方根称为该复数的模，记作｜z｜或者 r，即 r=|z|=$\sqrt{x^2+y^2}$。

在复变函数（以复数作为自变量和因变量的函数）中，自变量 z 可以写成 $z=r\times(\cos\theta+i\sin\theta)$，r 是 z 的模，即 r = |z|；θ 是 z 的辐角，记作 **Arg（z）**。在-π到π间的辐角称为辐角主值，记作 **arg（z）**（arg 的第一个字母小写）。共轭复数，复数的模，辐角在复平面中的关系如图 1-6 所示。

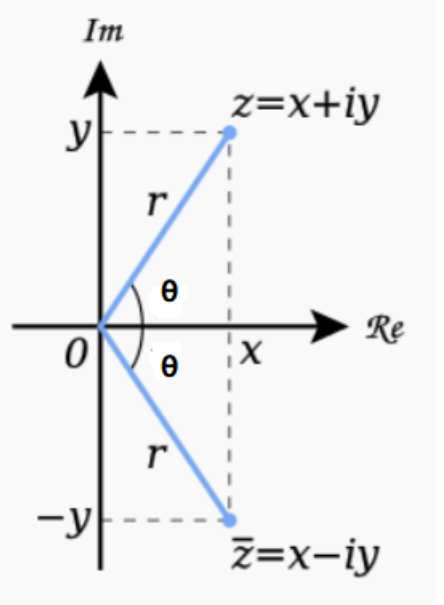



图 1-6 复平面中共轭复数、复数的模与辐角

量子力学里的薛定谔方程解出来的粒子运动的波函数，就是一个复函数，其中模 $r^2$ 表示量子出现在某个位置的概率。

实际上，虚数是想象中的虚构出来的数，现实中所有通过实验测量出来的数据都是通过实数描述的，比如物体的长度，重量，速度，位置等数据。

**有关虚数开根号的几何意义**

虚数开根号的几何意义，数学家们探索了 100 多年，其中包括牛顿，莱布尼茨，笛卡尔，甚至是欧拉，也没能解开虚数的真正意义。直到 1799 年由挪威的工程师韦塞尔（Caspar Wessel，1745-1818）在他的论文《论方向的解析表示：一个尝试》中正式把虚数当作二维数，表示成 a+bi 的形式，并提出了重要的复平面的概念，即直角坐标系中的横轴为实数轴，纵轴为虚数轴，(a, b)对应了实数轴和虚数轴的坐标。在虚数还没发现之前，单条数轴，足以描述所有的实数。虚数的发现，使得数轴不够用了，有了复平面之后，现在所有复数都能在复平面里找到。至此，虚数正式进入数学舞台，为 50 年后的黎曼创立复变函数打下基础。

现已经严格证明，一切数都能在复平面中找到，数的范围不会再超过复数的范围。

## 1.2 数的进制与转换

整数的值可以通过不同的进制来表示。进制是人为定义的带进位的计数方法，与此相对应的是不带进位的计数方法，比如原始的结绳计数，唱票时常用的“正”字计数等。

常用的进制计数方法包括十进制，十六进制，八进制和二进制等。

整数的十进制的表示方法为：0, 1, 2, 3, …, 9, 10, ..., 逢 10 就进位。

十六进制的表示方法为：0, 1, 2, 3, …, 9, a, b, c, d, e, f, 10, ...，逢 16 进位。十六进制中，除了 0 到 9 的数字字符，还引入了 a, b, c, d, e, f 等 6 个字母且不区分大小写，其中 a 对应的十进制数值为 10，b 为 11，c 为 12，d 为 13，e 为 14，f 为 15。十六进制以 0x 或 0X 开头，比如 0x21af 或 0X21AF，字母不区分大小写。

八进制的表示方法为：0, 1, 2, 3, 4, 5, 6, 7, 10, ...，逢 8 进位，八进制以 0 开头，比如 0123。

二进制的表示方法为：0, 1, 10, ...，逢 2 进位。

以十进制整数 100 为例子，它对应的十六进制数为 0x64，八进制数为 0144，二进制数为 01100100。

程序里会经常使用十六进制，因为十六进制里，1 个字符占 4 位，2 个字符代表 1 个字节（比如 0x1a，其中字符 1 和 a 各占 4 位，0x1a 共占 8 位即 1 个字节），这对于分析数据在内存中的存储非常方便；而八进制在描述计算机里的文件“读”、“写”和“执行”权限(用三位表示，分别为 rwx，即 read/write/execute 的缩写)是非常方便的。

实际上，除了上面在计算机里常用的进制，你也可以定义任意的进制，比如三进制（0, 1, 2，10，到 3 进位），五进制（0, 1, 2, 3, 4, 10，到 5 进位），七进制（0, 1, 2, 3, 4, 5, 6, 10，到 7 进位），甚至是三十六进制（0, 1, 2, 3, 4, 5, 6, 7, 8, 9, a, b, c,….z, 10，到 36 进位）。

对于任何一种 X 进制，数中每一位置上的数字在运算时都是逢 X 进一位。十进制是逢十进一，十六进制是逢十六进一，二进制就是逢二进一，以此类推，X 进制就是逢 X 进位。

科学家们在设计计算机的时候最终选择了二进制。二进制是逢 2 进位的进位制。其中 0、1 是基本算符，比如二进制中整数从 0 开始，从小到大依次为 0（对应 10 进制中的 0）,1（对应 10 进制中的 1）,10（对应 10 进制中的 2）,11（对应 10 进制中的 3）,100（对应 10 进制中的 4),101(对应 10 进制中的 5),110(对应 10 进制中的 6),111(对应 10 进制中的 7), ……。因为它只使用 0、1 两个数字符号就可以表示所有的数，非常简单方便，易于用电子方式实现，所以现代计算机之父冯洛伊曼最终在计算机里抛弃了十进制，直接使用二进制作为数的表示方式和计算机体系结构的实现基础，使得基于二进制设计实现的计算机大获全胜，成为

现代计算机的主流体系结构。

**有关二进制发明的小故事**

二进制是由伟大的德国数学家莱布尼茨发明的（他也是微积分的发明人之一）。据中国作家胡阳、李长铎编写的《莱布尼茨二进制与伏羲八卦图考》一文给出的比较可信的材料表明，莱布尼茨的二进制至少在某种程度上受到了中国古代易经里的伏羲八卦图的启发。

伏羲是中国古代三皇之一，他根据日月山川的变化，造出伏羲八卦图（也叫先天八卦图）。后来，周文王姬发根据先天八卦图，做出了新的八卦图，人称后天八卦（或文王六十四卦图）。如图 1-7 所示，在八卦图里，太极生两仪，两仪就是阴爻（yáo）、阳爻（yáo），符号分别为“- -”和“—”，简称“阴”和“阳”，可以对应二进制数字 0 和 1。两仪生四象，四象中的阴爻和阳爻组合可以分别对应二进制数 00、01、10、11，这组二进制数换算成十进制数分别是 0、1、2、3。四象生八卦，八卦分别指：坤（kūn）艮（gèn）坎（kǎn）巽（xùn）震（zhèn）离（lí）兑（duì）乾（qián）。八卦中阴爻和阳爻组合对应的二进制数是：000、001、010、011、100、101、110、111，同样把这组二进制数换算成十进制数分别是：0、1、2、3、4、5、6、7。

伏羲八卦（先天八卦）

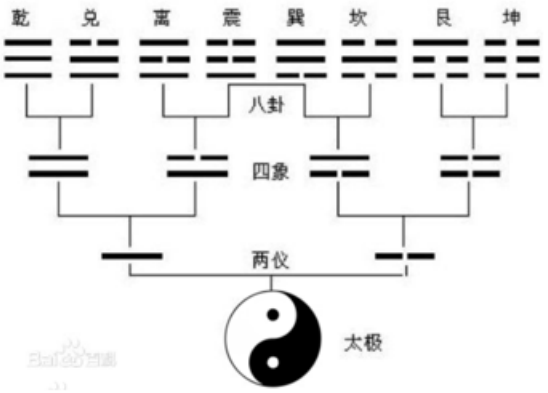



图 1-7 伏羲八卦与演进

可见伏羲八卦图与二进制的思想完全一致。在莱布尼茨的二进制中，通过 0 与 1 引申，就可以表示一切数字，在易经八卦图中，通过阴阳引申，就可以表示宇宙万物的原理。如果把阴爻看作 0，把阳爻看作 1，所有的卦象于是也就可以看成 0 和 1 的组合。伏羲图的 8 个卦象，也正好可以看作二进制算术从 0 到 7 的数字，文王六十四卦图的 64 个卦象，可以看作二进制中 0 到 63 的数字。

当 1703 年莱布尼茨看到八卦图的时候，发现自己的二进制体系与八卦图十分一致，并

高兴的说：“可以让我加入中国籍了吧”。几天后，他就写了论文《二进位算术的阐述—关于只用 0 和 1 兼论其用处及伏羲氏所用数字的意义》，发表在法国《皇家科学院院刊》上。此外，莱布尼茨还曾给中国当时的清朝皇帝康熙写信，建议康熙仿照巴黎科学院和柏林科学院，组建北京科学院。

计算机最终会将数据以二进制的形式存储在内存或者磁盘中。但是，程序员在编写程序的时候，还是会根据不同需求而采取不同的进制形式来表示数据。所以就涉及到了一个进制转换的问题。接下来，我们将给大家介绍进制的转换方法。当然，我们只介绍常见的十进制与二进制之间，二进制与十六进制之间，十六进制与十进制之间的转换方法，而其他进制之间的转换方法可以举一反三，触类旁通。

## 1.2.1 十进制与二进制转换

我们从小到大，最先开始接触的就是十进制，然后有了时间观念之后，事实上开始接触了十二进制和六十进制。我们又知道了，计算机里使用的是二进制，那么，如何把十进制和二进制对应起来呢？这里就涉及到了一个进制转化问题。

把十进制转化为二进制的常规做法是将一个正数分别取它的整数和小数两部分，然后对这两部分分别转换，对整数部分除以 2 取余，小数乘以 2 取整。

例如，对于 22.8125，首先取得整数部分 22，然后整数部分除以 2，用得到的商继续除以 2，直到商变为 0 为止，然后将对应的所有余数逆序排列就得到了对应的二进制，也就是下面的计算过程：

| | | |
|---|---|---|
| 22 / 2 | 商 11 | 余 0 |
| 11/2 | 商 5 | 余 1 |
| 5 /2 | 商 2 | 余 1 |
| 2 /2 | 商 1 | 余 0 |
| 1 /2 | 商 0 | 余 1 |

那么 22 对应的二进制是 **10110**。

再来计算小数部分。取小数部分 0.8125，小数乘以 2，取整，小数部分继续乘以 2，取整，直到小数部分变为 0 为止，然后将所得的整数顺序排列，也就是下面的计算过程：

| | |
|---|---|
| 0.8125*2=1.625 | 取整 1，小数部分是 0.625 |
| 0.625*2=1.25 | 取整 1，小数部分是 0.25 |
| 0.25*2=0.5 | 取整 0，小数部分是 0.5 |
| 0.5*2=1.0 | 取整 1，小数部分是 0，结束 |

所以 0.8125 的二进制是 0.1101。

将该数整数部分和小数部分的二进制组合之后，得到最终 22.8125 的二进制为 10110.1101。

这是一种传统的十进制到二进制的转化方法，它的一个不足就是需要涉及到大量的乘除运算。而一般来说，人更擅长于加减运算，而不是乘除运算。因此有更好的转换方法，即所谓的“表格法”。首先我们按照二进制的每位对应的值画出如表 1-1：

| ..... | 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | 0.5 | 0.25 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|

表 1-1 二进制转化表格

在表格里分别写出在二进制里每个位表示的数值。比如，在表 1-1 里，第 0 位表示 1（$2^0$），

第 1 位表示 2（$2^1$），第 2 位表示 4（$2^2$），第 3 位表示 8（$2^3$），.....，-1 位为 0.5（$2^{-1}$），-2 位为 0.25（$2^{-2}$），-3 位为 0.125($2^{-3}$)，第 N 位表示的值为 $2^N$。

对于一个正整数 X，要把它转化为二进制，首先在表格中找到比 X 小的最大数，假设 X 为 100，那么在表 1-1 里，比 100 小的最大数就是 64，所以，100 在二进制中，此位为 1，比此位更高的位数必须为 0（否则就超过 100 了）；然后，用 100 减去 64，还剩 36，从此位右边表格里继续找小于等于剩下值 36 的最大数。一直到 100 最后变为 0。

比如，以 100.25 为例子。如图 1-8 所示：

| | | | | | 1 | 1 | 0 | 0 | 1 | 0 | 0 | 0 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ..... | 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | 0.5 | 0.25 |

图 1-8 利用表格法将 100.25 转化二进制

先看整数部分。首先比 100 小的最大数是 64，因此，该位记为 1。100-64=36，那么小于等于 36 的最大数是 32，因此，32 所在的位为 1。36-32=4，所以，小于等于 4 的最大数为 4，所以 4 所在的位为 1。4-4=0，因此转换结束，除了被记为 1 的位，其他位全部为 0。于是，100 的二进制为“1100100”，如果 100 是一个 8 位的整数，1100100 共 7 位，那么左边不足 8 位的地方用 0 填充，最终为 01100100。

再看小数部分。0.25，比 0.5 要小，所以 0.5 位应该记为 0，而 0.25 位恰好相等，所以 0.25 位记为 1。 因此，100.25 对应的二进制位 **01100100.01**。

同样地，把二进制转化为十进制，也可以用表格法，分别将二进制中为 1 的位在表格中对应的值相加，就可以得到对应的十进制。比如，对于二进制 1100101.11，其中各位对应的值如图 1-9 所示：

| | | | | | 1 | 1 | 0 | 0 | 1 | 0 | 1 | 1 | 1 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ..... | 1024 | 512 | 256 | 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 | 0.5 | 0.25 |

图 1-9 利用表格法，将 1100101.11 转化为十进制

所以，1100101.11 对应的十进制值为 64+32+4+1+0.5+0.25=101.75。

除此之外，二进制转换为十进制也可以直接使用公式。转化公式为：

$$a_n a_{n-1}\ldots a_0 a_{-1}\ldots = a_n*2^n + a_{n-1}*2^{n-1} + \ldots + a_0*2^0 + a_{-1}*2^{-1} + \ldots$$

因此，1100101.11 也可以通过上述公式完成二进制到十进制的转换，即：

$$1100101.11 = 1*2^6+1*2^5+0*2^4+0*2^3+1*2^2+0*2^1+1*2^0+1*2^{-1}+1*2^{-2}=101.75$$

## 1.2.2 二进制与十六进制转换

那么二进制如何转化为十六进制呢？实际上也很简单。把对应的二进制数从右往左，按每四位为一组进行分组，左边剩下的数字不足 4 位时，用 0 补齐 4 位（这里假设所处理的二进制都是正整数，因此可以用 0 补齐），然后将二进制中的四 4 位变为十六进制中的 1 位（因为十六进制共有 16 个字符（0-F），恰好需要二进制中的 4 位来表示，所以十六进制中的 1 个字符对应了二进制中的 4 个字符），比如对于以下的二进制数：

01110111101

根据转换方法，对该二进制数从右往左按照 4 位为一组分组之后为：

011 1011 1101

左边 011 为 3 位，不足 4 位，因此增加 1 位 0 补齐 4 位，最后为：

0011 1011 1101

然后结合表 1-2 对应关系，将每一组中的 4 位变 1 位即为 0x3bd。

| 二进制 | 十六进制 | 十进制 |
|---|---|---|
| 0000 | 0 | 0 |
| 0001 | 1 | 1 |
| 0010 | 2 | 2 |
| 0011 | 3 | 3 |
| 0100 | 4 | 4 |
| 0101 | 5 | 5 |
| 0110 | 6 | 6 |
| 0111 | 7 | 7 |
| 1000 | 8 | 8 |
| 1001 | 9 | 9 |
| 1010 | A | 10 |
| 1011 | B | 11 |
| 1100 | C | 12 |
| 1101 | D | 13 |
| 1110 | E | 14 |
| 1111 | F | 15 |

表 1-2 二进制，十六进制与十进制对应关系

十六进制在转换为二进制的时候，正好和二进制转化为十六进制相反。转化规则为**十六进制中的一位变为二进制中的四位**。

比如 0x3bd，根据一位变四位规则，通过表 1-2 中一位十六进制数与四位二进制数的对应关系，那么 3 变为 0011，b 变为 1011，d 变为 1101，所以 0x3bd 对应的二进制为 0011 1011 1101。

那么，为什么十六进制中的一位对应着二进制中的四位呢？这是因为十六进制中共有 0-9，a-f 共 16 个字符。由于二进制中一位只能表示 2 个有效信息，所以要完整表示这 16 个字符信息，恰好需要二进制中的四位，因为 $2^4=16$。

## 1.2.3 十六进制与十进制转换

一个任意 X 进制（比如 8 进制，16 进制）的数 $a_na_{n-1}...a_1$，转化为十进制的公式为：

$$a_na_{n-1}...a_1=a_n*X^{(n-1)}+a_{n-1}*X^{(n-2)}+...+a_2*X+a_1$$

因此，将十六进制转化为十进制，可以直接使用转化公式：

$$a_na_{n-1}...a_1=a_n*16^{(n-1)}+a_{n-1}*16^{(n-2)}+...a_2*16+a_1*1$$

比如将十六进制数 0x1b 和 0x40 转化为十进制，利用上面的公式如下：

$0x1b=1*16^1+11*16^0=16+11=27$；

$0x40=4*16^1+0*16^0=64$

实际上，十进制转化为任意 X 进制，可以用该十进制数除以 X，并依次用商除以 X 直到商为 0，然后将得到的所有余数按序逆置，就是 X 进制对应的十进制数。

以十六进制为例，十进制整数直接转化为十六进制，只需从该数除以 16 开始，并依次用商除以 16 直到商为 0，然后得到的所有余数逆置，就是十六进制。比如十进制的整数 100，转化为对应的十六进制，计算过程为：

第一次计算商和余数：

100/16 = 6　　100%16=4

第二次计算商和余数：

6/16 = 0　　6%16=6

此时，商已经变为 0，把余数 4，6 按序逆置排列得 64，得到 100 的十六进制是 0x64。

但这种方法，还是不大方便，毕竟如果数大了，除法不是那么好计算的。一个更为方便的方法，是先把十进制数转化为二进制数，然后再把二进制数转化为十六进制数。

比如，100 对应的二进制我们很快就能通过 1.2.1 中的表格法求出来：

1100100

把这个二进制数从右往左按照 4 位为一组进行划分，不足 4 位用 0 补齐，则有：

0110 0100

然后再将每一组的数转化为十六进制为 6 和 4，所以对应的十六进制为 0x64。

关于进制的转化我们就介绍到这里。下面是某知名互联网公司关于整数进制转化的笔试题目。现在我们来尝试解决这些问题。

题目 1：八进制的 256，转化成七进制数是多少？

解：先把八进制的 256 转化为十进制，得：

$2*8^2+5*8+6=174$

再把十进制 174 转化为七进制：

174/7 =24　　174%7=6

24/7=3　　24%7=3

3/7=0　　3%7=3

此时商为 0，把所有余数逆置，即得七进制为 336。

题目 2：15*4=112 成立，则系统采用的是几进制？

解：设系统采用的是 x 进制，那么左边 15*4 转化为十进制有：

(x+5)*4

右边 112 转化为十进制有：

$x^2+x+2$

左右相等，得关于 x 的一元二次方程：

$(x+5)*4=x^2+x+2$

化简为：

$x^2-3x-18=0$

解此一元二次方程（如果不会解方程，请参考本书 3.5 节），有：

x=6

所以，系统采用的是 6 进制。

题目 3：假如 36 进制的规则为“0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ”。比如常见 10 进制与 36 进制对应关系为： 1=“1”，10=“A”，20=“K”，35=“Z”，36=

“10”。 那么 10 进制 128 对应的 36 进制为多少？

解：把 10 进制的 128 转化为 36 进制的方法为用 128 除以 36 取商和余，商继续除以 36，直到商为 0，这时把所有余数按序逆置，计算步骤如下：

第一步：128/36=3　　128%36=20(即 K）

第二步：3/36=0　　3%36=3

此时商为 0 了，把所有余数 K，3 按序逆置排列，那么 128 对应的 36 进制为 3K。

## 1.3 数的编码与存储

在正式讲数的编码和存储之前，我们先来理解一下位（Bit）和字节（Byte）的概念。数据一般是存放在内存或磁盘等存储设备上，存储的大小以字节为单位。如图 1-10 所示，在计算机里，最基本的存储单位为字节（Byte，常说的大 B），1 个字节包含 8 位（bit，常说的小 b）即：

1Byte=8Bit

计算机的数据就是以一个字节一个字节的形式存储在内存或磁盘等存储设备中。

1 个 bit 只能存放一个二进制值，要么是 0，要么是 1，因此只能存储 2 个信息，即 0 和 1；而 1 个 byte 等于 8 个 bit，所以能存储 $2^8$（256）个信息。一个字节的整数的取值范围如下：

- 有符号数：【-128，127】
- 无符号数：【0，255】

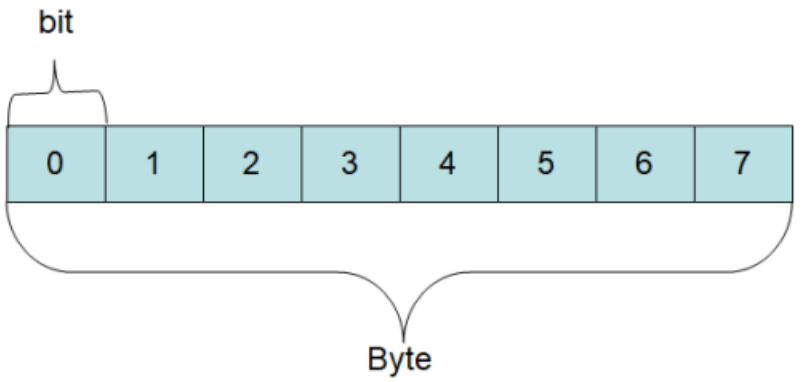



图 1-10 位与字节

内存或者磁盘等外存的大小单位有 B，KB，MB，GB，TB，PB，EB，ZB，YB，BB 来表示。其中他们的大小关系如下：

- 1KB (Kilobyte 千字节)=1024B
- 1MB (Megabyte 兆字节 简称“兆”)=1024KB
- 1GB (Gigabyte 吉字节 又称“千兆”)=1024MB
- 1TB (Trillionbyte 万亿字节 太字节)=1024GB，其中 1024=$2^{10}$
- 1PB（Petabyte 千万亿字节 拍字节）=1024TB
- 1EB（Exabyte 百亿亿字节 艾字节）=1024PB
- 1ZB (Zettabyte 十万亿亿字节 泽字节)= 1024 EB
- 1YB (Yottabyte 一亿亿亿字节 尧字节)= 1024 ZB
- 1BB (Brontobyte 一千亿亿亿字节)= 1024 YB

大家知道，整数包括负整数，零，和正整数。目前在计算机里，一个整数通常用 1 个字

节（8 位），2 个字节（16 位），4 个字节（32 位）或者 8 个字节（64 位）来存储。当然，也由一些其他方式来存储整数的形式，比如为了节约存储空间而以不固定长度（变长）存储整数。

计算机中的整数分为有符号整数和无符号整数。

有符号整数可以表示正整数，也可以表示负整数，它的最高位表示符号，即最高位为 0，表示正整数，最高位为 1，表示负整数。

无符号数表示非负整数，整个位数都用来表示整数的值。

以 1 个字节表示的整数为例子。对于二进制数 10011100，如果它表示的是一个有符号数，那么由于它的最高位（左边第一位）为 1，所以它一定是一个负整数；而对于二进制数 01100100，如果它表示的是一个有符号数，那么由于它的最高位（左边第一位）为 0，所以它一定是一个正整数。而如果它们都是无符号数，那么它们都是正整数。

如果用 N 位来存储和表示整数，那么有符号整数的范围为$[-2^{N-1}, (2^{N-1})-1]$；无符号整数的表示范围为$[0, 2^N-1]$。

比如，用 8 位来表示有符号整数，由于第 8 位用于表示了符号，因此，整数的表示范围为[10000000, 01111111]，也就是十进制的[-128，127]；如果是表示无符号整数，则表示范围为[00000000, 11111111]，也就是十进制的[0，255]。

在理解了位与字节的概念之后，我们现在开始介绍数的编码与存储方法。数虽然可以使用二进制来表示，但是计算机却不能直接存储数对应的二进制值，否则会遇到一些计算问题（具体问题请参考 1.3.1），所以，我们还必须首先对数进行科学的编码，然后通过数对应的编码来存储整数。

接下来，我们首先介绍整数的编码，然后介绍整数的存储方法，最后介绍浮点数的编码格式与存储方法。

### 1.3.1 整数的编码

整数在计算机里是按照对应的编码进行存储的。整数的编码分为原码、反码、和补码。计算里采用的是补码的存储方式。它们的定义如下：

- 原码：在数值前面增加了一位符号位（即最高位为符号位），该位为 0 表示正数，该位为 1 表示负数，其余位表示数值的大小。以一个字节的整数为例，1 的原码为 00000001，-1 的原码为 10000001。
- 反码：正整数的反码等同于对应的原码，而负整数的反码是对其绝对值逐位求反。以一个字节的整数为例，1 的反码为 00000001， -1 的反码为 11111110。
- 补码：正数的补码与原码相同（通过本书 1.2.1 节讲的表格法可方便求出），负数的补码就是对该负数的绝对值取反加 1，0 的补码是 0。

因为计算机是以补码来存储整数的，所以补码就显得很重要。那么如何计算整数的补码呢？下面以一个字节的整数为例子来说明补码的求法。

1.计算十进制整数 100 的补码

由于 100 是正整数，所以 100 的补码，直接按照前面 1.2.1 节介绍的表格法，将 100 转化为二进制即可，结果为 01100100。

2. 计算 0 的补码

根据定义，0 的补码是 0，用一个字节表示为 00000000。

3. 计算-100 的补码

由于-100 是负数，所以先求它的绝对值即 100 的二进制 01100100 ，再对绝对值的二进制按位取反得 10011011，再加 1，即 10011011+1 ，那么结果为 10011100 。

4. 计算 1 的补码

由于 1 是正整数，那么 1 的补码就是 00000001。

5. 计算-1 的补码

由于-1 是负整数，先求它的绝对值 1 的二进制是 00000001，然后再按位取反并加 1 得 11111110+1，所以结果为 11111111。

6. 计算 127 的补码

由于 127 是正整数，所以直接通过表格法，即可求出它的补码为 01111111 。

7. 计算-128 的补码

由于-128 是负数，所以先求它的绝对值 128 的补码为 10000000，再按位取反加 1 得 01111111+1，结果为 10000000 。

在计算机系统中，整数值一律用补码来表示和存储。

上面我们介绍了 1 个字节的整数补码的求法，如果需要扩充为 2 个字节或者 4 个字节或者 8 个字节的补码，那么左边比 1 个字节多出的位数一律按照符号位去填充即可。

比如将 1 个字节的 100 的二进制 01100100 扩充为 2 个字节，左边多出了 8 个位，因为最高位符号位是 0，因此就用 0 去填充多出的 8 个位，那么结果就是 **00000000** 01100100；-100 的二进制 10011100 扩充为 2 个字节，左边多出了 8 个位，因为最高位符号位为 1，因此就用 1 去填充多出的 8 个位，那么结果就是 **11111111** 10011100。

知道了一个数的补码，如何知道它对应的值呢？其实也很简单。首先是看补码的符号位，确定这个数是正数还是负数。如果是正数，那么直接将它按照本书 1.2.2 节的方法转化为十进制即可；如果是 0，那么它的值也是 0；如果是负数，那么对这个补码取反再加 1，就是它本身的绝对值了。

对于 8 位整数来说，补码的表示范围为[-128，127]。 大家应该记住一些常见的补码的表示，这些数包括但不局限于下面表 1-3 中列出的数：

| 数值 | 补码 |
|---|---|
| -128 | 10000000 |
| -1 | 11111111 |
| 0 | 00000000 |
| 127 | 01111111 |

表 1-3 常见整数补码

那么有了原码和反码，计算机为什么最后却采用补码来表示和存储整数呢？

来看看它们的运算情况就明白了。 以字长为 8 位（即 1 个字节的整数）为例 ，我们来看看在原码、反码和补码编码情况下计算算术减法“1-1”的情况。其中原码的运算方式为：

$$1 - 1 = 1 + (-1) = (00000001) + (10000001) = (10000010) = -2$$

但是，数学规则告诉我们，1-1 的正确结果应该是 0，而不是-2，所以原码这个计算结果显然是不正确的。因此原码无法满足运算要求，再来看反码的减法运算：

$$1 - 1 = 1 + (-1) = (00000001) + (11111110) = (11111111) = (-0)$$

结果有问题。反码的问题出现在(+0)和(-0)上，因为在数学概念里零是没有正负之分的。所以反码在表示零的地方出了问题，居然有一个-0，这是不对的。再来看补码的加减运算：

$$1 - 1 = 1 + (-1) = (00000001) + (11111111) = (00000000) = 0$$

结果正确。

$$1 - 2 = 1 + (-2) = (00000001) + (11111110) = (11111111) = (-1)$$

结果正确。

所以通过上面的例子我们可以看到，原码符号位无法参加运算，反码有负零的问题。因此才最终采用了补码。通过补码的运算，可以看出补码的设计目的有：

第一，使符号位能与有效值部分一起参加运算，从而简化运算规则。

第二，使减法运算转换为加法运算，进一步简化计算机中运算器的线路设计（实际上乘除法也都可以最终转化为加法运算，比如 8*5 可以转化为 5 个 8 相加，8/5 可以用 8 反复减 5 直到减不够为止，减法完成的次数就是商，减完不足剩下的数就是余数）。

此外，在补码中用-128 代替了-0，所以没有+0 和-0 之分，符合常理，所以在 1 个字节的整数中，补码的表示范围为[ -128, 127]共 256 个。

## 1.3.2 整数的存储方式

整数的存储分为高位优先存储（big-endian）和低位优先存储（little-endian）。高位优先存储是指把整数的高位优先存在低地址。 低位优先存储是指把整数的低位首先存在低地址。

假设一个 4 字节十进制整数值为 25000，它的补码为 0x000061a8，其中最左边的 00 是最高字节，然后依次为 00 次高字节，61 为次低字节，a8 为最低字节。那么在内存中可用 4 个字节来存放这个整数。根据高位优先和低位优先的规定，图 1-11 是这一个整数的低位优先和高位优先的存储实例：

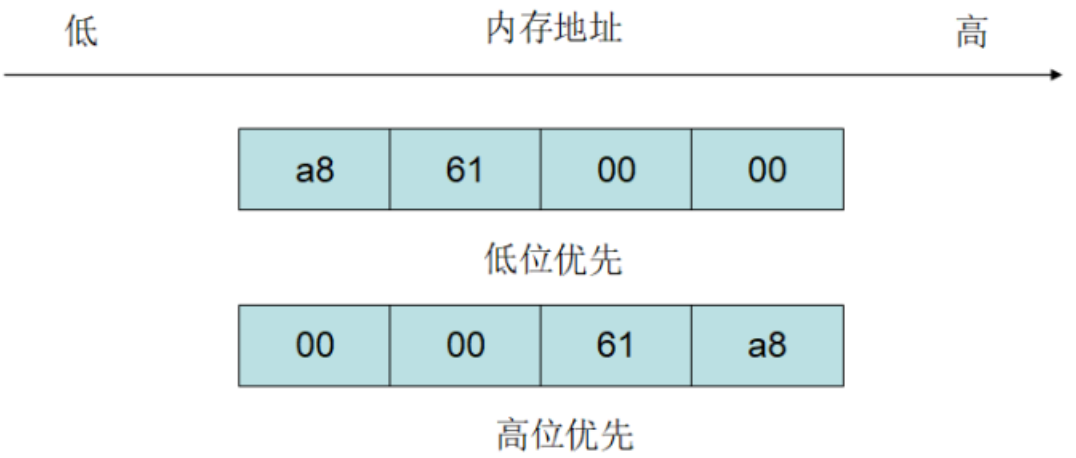



图 1-11 整数的存储方式

如图 1-9 可以看出，内存地址从左往右增加，在低位优先的系统中，会将整数 0x000061a8 低位的一个字节 a8 优先存放在内存中的低地址，而在高位优先的系统中，正好相反，会优先把高位的 00 存放在低地址。

实际的计算机系统中，X86 计算机体系结构是一个按照低位优先存储整数的系统；而 SPARC 等计算机体系结构则是一个按照高位优先存储整数的系统。在网络上传输一个整数，也是采用的高位优先的方法来传输的。

## 1.3.3 浮点数格式与存储

计算机中的数分为整数与实数。对于实数，绝大多数现代的计算机系统采纳了所谓的浮

点数表达方式。 这种表达方式利用科学计数法来表达实数，即用一个尾数 M（Mantissa），一个基数 B（Base），一个指数 e（exponent）或阶码 E(E=e+127 或者 e+1023)以及一个表示正负的符号(Sign)来表达实数。

比如 123.45，用十进制科学计数法可以表达为 $1.2345 \times 10^2$，其中 1.2345 为尾数，10 为基数，2 为指数。浮点数利用指数 e 达到了浮动小数点的效果，从而可以灵活地表达更大范围的实数。

同理，对于一个二进制的小数比如 1011.01，用科学计数法也可以表示为 $1.01101*2^3$，其中 1.1101 为尾数，2 为基数，3 为指数。

**1. 浮点数的存储方法**

计算机中是用有限的连续字节保存浮点数的。保存这些浮点数当然必须有特定的格式， C/C++中的浮点数类型 float 和 double 采纳了 IEEE 754 标准中所定义的单精度 32 位浮点数和双精度 64 位浮点数的格式。

如图 1-12 所示，在 IEEE 标准中，浮点数是将特定长度的连续字节的所有二进制位分割为特定宽度的符号域，指数域和尾数域三个域，其中保存的值分别用于表示给定二进制浮点数中的符号，指数和尾数。这样，通过尾数和可以调节的指数（所以称为"浮点"）就可以表达给定的数值了。

float 单精度实数的存储结构：

| S（31） | E（阶码，30~23，共 8 位） | M（尾数，22~0，共 23 位） |
|---|---|---|

double 双精度的存储结构：

| S（63） | E（阶码，52~62，共 11 位） | M（尾数，51~0，共 52 位） |
|---|---|---|

图 1-12 浮点数存储格式

根据国际标准 IEEE 754，任意一个二进制浮点数 V 可以表示成下面的形式：

$V = (-1)^s \times M \times 2^E$

（1）$(-1)^s$ 表示符号位，当 s=0，V 为正数；当 s=1，V 为负数。

（2）M 表示有效数字，大于等于 1，小于 2，但整数部分的 1 不变，因此可以省略，不用存储。

（3）$2^E$ 表示指数位。

比如，对于十进制的 5.25 对应的二进制为 101.01，写成科学计数法为 $1.0101*2^2$。所以，它的符号位 S 为 0，尾数 M 为 1.0101，省略掉小数点前面的 1 则为 0101，指数 E 为 2。

而 $-5.25=-101.01=-1.0101*2^2$。所以 S 为 1，M 为 1.0101，E 为 2。

对于 32 位的单精度数来说，从低位到高位，尾数 M 共用 23 位来表示，阶码 E 共用 8 位来表示，而符号位用最高位 1 位来表示，0 表示正，1 表示负。对于 64 位的双精度数来说，从低位到高位，尾数 M 共用 52 位来表示，阶码 E 共用 11 位来表示，而符号位用最高位 1 位来表示，0 表示正，1 表示负。

IEEE 754 对有效数字 M 和指数 E，还有一些特别规定。前面说过，M 可以写成 1.xxxxxx 的形式，其中 xxxxxx 表示小数部分。IEEE 754 规定，在计算机内部保存 M 时，默认这个数的第一位总是 1，因此可以被舍去，只保存后面的 xxxxxx 部分。比如保存 1.0101 的时候，只保存 0101，等到读取的时候，再把第一位的 1 加上去。这样做的目的，是节省 1 位有效数字。以 32 位浮点数为例，留给 M 只有 23 位，将第一位的 1 舍去以后而读取的时候再把

舍去的 1 加上，实际上相当于可以保存 24 位有效数字。

对于阶码 E，首先，E 为一个无符号整数（unsigned int），这意味着，如果 E 为 8 位，它的取值范围为 0~255；如果 E 为 11 位，它的取值范围为 0~2047。然而科学计数法中的指数 e 是可以出现负数的，所以 IEEE 754 规定，阶码 E 的真实值必须等于指数 e 再加上一个中间数，对于 8 位的 E，这个中间数是 127；对于 11 位的 E，这个中间数是 1023。 比如，$2^2$ 的 e 是 2，所以保存成 float 32 位浮点数时，必须保存成解码 E=2+127=129，即 10000001。

此外，E 还需要考虑下面 3 种情况：

（1）E 不全为 0 或不全为 1。这时，浮点数就采用上面的规则表示，即解码 E 的计算值减去 127（或 1023），得到真实值即指数 e，再将有效数字 M 前加上第一位的 1 得到实际的浮点数。

（2）E 全为 0。这时，浮点数的指数 e 等于 1-127（单精度浮点数）或者 1-1023（双精度浮点数），有效数字 M 不再补加舍弃掉的第一位的 1，而是还原为 0.xxxxxx 的小数。这样做是为了表示±0，以及接近于 0 的很小的数字。这也是我们在 C 语言等编程语言里，不能直接将浮点数与 0 进行比较的原因：

```
float f = 0.01f;

if(fabs(f) < 1e-6){// 如果 f 的绝对值小于 1e-6（或者一个足够小的数）,就可以认为 f 等于 0 了
    printf("f equal 0\n");
}
```

（3），E 全为 1。这时，如果有效数字 M 全为 0，表示±∞（正负取决于符号位 s）；如果有效数字 M 不全为 0，表示这个数不是一个数（NaN）。

**2. 浮点数的转换方法**

有时候我们遇到浮点数，可能需要在二进制和浮点数之间进行转化。浮点数的转换方法可以分为如下 2 种情况：

（1）给出一个浮点数，计算对应的二进制。

比如给定一个浮点数 7.25，如何计算它对应的单精度和双精度的二进制呢？

首先，十进制浮点数 7.25 对应的二进制为 111.01(二进制，十进制和十六进制转化方法请参考本书 1.2 节)。用二进制的科学计数法为 $1.1101*2^2$。所以，按照浮点数的存储结构，得出符号位为 0，表示正数；阶码 E 在单精度时为 2+127=129，双精度时为 2+1023=1025；小数部分 M 舍弃 1.1101 小数点前面的 1 后为 1101（小数本来为 1.1101，因为所有的小数中，小数点前面的值都是 1，所以不用存储，只需要存小数点后面的数，在读取的时候，再自动把前面的 1 加上即可）。所以，7.25 对应的单精度二进制位为 0 10000001 11010000000000000000000; 双精度的二进制位为 0 10000000001 1101000000000000000000000000000000000000000000000000。

（2）给出一个浮点数的二进制，计算对应的十进制值。

如果给出了一个浮点数的二进制，如何计算它对应的十进制，其实就是（1）中的逆运算。分别求出浮点数二进制中对应的符号位，指数 e 和小数 M 这三个部分，就可以了。比如，给定一个单精度浮点数的二进制存储为：

0 10000001 1101 00000000000000000000;

那么按照单精度浮点数的存储格式，将这个二进制划分为符号位，阶码和尾数三个部分。这三个部分对应的符号位为 0，表示正数；阶码 E 为 129 因此指数 e 为 129-127=2；尾数为 1101，补上 1 之后为 1.1101。所以上面的二进制对应的二进制科学计数法为 $1.1101*2^2$，也就是 111.01，再转化为十进制即 7.25。

## 1.4 基于整数处理能力的计算机系统分类方法

有了前面二进制和数的基础，那么我们现在很容易理解计算机系统中常常被提到的 16 位，32 位，64 位，128 位系统了。

16 位系统，就是指计算机一次能处理的最大整数为 16 位（即 2 个字节），因此在计算机里的寄存器存储整数的长度最多只能达到 2 个字节。32 位系统是指计算机中一次能处理的最大整数为 32 位（即 4 个字节），因此计算机里的寄存器存储整数的长度最多能达到 4 个字节。64 位系统是指计算机中一次能处理的最大整数为 64 位（即 8 个字节），因此计算机里的寄存器存储整数的长度最多能达到 8 个字节。128 位系统则是用 128 位的整数来表示一个数据，这已经是一个非常大的天文数字了。

现实中，16 位系统一般用于早期的计算机，现在已经被淘汰。目前主流的操作系统（比如 Windows，Linux，MacOS 等）一般都是 32 位或者 64 位的，而且 32 位也会很快被淘汰，64 位将成为主流。而 128 位操作系统至今没有出现，但网络协议中的 IPV6 协议和文件系统中的 ZFS 文件系统是 128 位的。实际上 128 位系统是很恐怖的一个系统级别。比如说：

（1）与 IPv4 相比，IPv6 具有更大的地址空间。IPv4 中规定 IP 地址长度为 32 位（即一个 IP 地址可以用一个 4 字节的整数表示），因此最大地址个数为 $2^{32}$，这个数字并不是很大，也就 42 亿个左右；而 IPv6 中 IP 地址的长度为 128 位（即一个 IP 地址可以用 128 位的整数表示），那么最大地址个数高达 $2^{128}$。与有 42 亿个左右 IP 地址的 IPv4 相比，IPv6 理论上可提供的 IP 地址数量达 2 的 128 次方，几乎可以为全世界的每一粒沙子分配一个 IP 地址。

（2）ZFS 是之前 SUN 公司（后来被 Oracle 收购）在 Solaris 操作系统中推出的世界上第一个 128 位的文件系统，这意味着它能存储 1800 亿亿($18.4 \times 10^{18}$)倍于当前 64 位文件系统的数据。ZFS 的设计如此超前以至于这个极限就当前实际可能永远无法遇到（所以该文件系统用 26 个英文字母中最后一个字母 Z 来表示，意思就是最后一个文件系统）。ZFS 项目的创始人 Jeff Bonwick 曾说：“要填满一个 128 位的文件系统，将耗尽地球上所有存储设备。除非你拥有煮沸整个海洋的能量，不然你不可能将其填满。(Populating 128-bit file systems would exceed the quantum limits of earth-based storage. You couldn't fill a 128-bit storage pool without boiling the oceans.)”

作为对这些数字的感性认识，假设每秒钟创建 1000 个新文件，达到 ZFS 文件数极限需要大约 9000 年。在辩解填满 ZFS 与煮沸海洋的关系时，ZFS128 位文件系统的创始人写到：一个完全的 128 位存储池将包含 $2^{128}$ 个块 $=2^{137}$ 字节 $=2^{140}$ 位；应此，保存这些数据位至少需要($2^{140}$ 位) / ($10^{31}$ 位/公斤) = 1360 亿公斤的物质。

# 第 2 章 数学和计算机的理论基础：集合论

研究集合的理论叫集合论。集合论是在 19 世纪末由德国数学家康托尔（Cantor，1845－1918，参见图 2-1）创立。计算机中的人工智能理论以概率论为重要基础，概率论又以微积分为基础，而严格的微积分基础被归结为实数理论，实数理论又以自然数理论和集合论为基础。自然数理论又建立在集合论和逻辑之上。数学中的群、环、拓扑空间，或者自然数、实数、函数等都是集合。从这个意义上说，集合论是现代数学和计算机理论的基础。

图 2-1 德国数学家、集合论创始人康托尔

“集合”是我们进入中学之后学习的第一个数学概念。集合论曾引起第三次数学危机，让无数数学家感到惶恐，正所谓基础不牢，地动山摇。

康托尔也因为集合论的提出，遭受了包括来自他在柏林大学的导师在内的各种攻击和不公正待遇，甚至因此无法获得柏林大学教授职位，不得不在哈雷大学领着微薄的薪水。这也让我们感受到了，犹如日心说的提出一样，任何革命性的新事物的出现总是会受到各种守旧势力的攻击和打压。

学过中学集合论的读者可能会感到不解，集合论的知识非常简单和易于理解，为什么它会引起了第三次数学危机，为什么它会受到那么多数学家的反对和攻击？

那是因为我们中学学的集合论仅仅是集合论的皮毛，是相当的初级部分。康托尔对集合论的研究主要体现在了无穷集合部分。也就是在这一部分，犹如打开了潘多拉魔盒一样，引起了数学界对待集合论截然不同的看法，甚至发生了互相攻击。

在开始正式学习集合论之前，我们先来提出和思考这样几个问题，这些问题也是古代科学家们感到困惑的问题。带着这样的问题，有利于我们提高对学习集合论的兴趣和保持注意力。

问题 1：给定一个自然数 n，就能找到对应的偶数 2n，因此二者是一一对应的关系，即给一个自然数，总能找到一个偶数，因此二者应该是一样多。但直觉告诉我们，如果把自然数作为一个整体，那么偶数只是自然数的一部分，因此整体肯定大于局部。那么全体自然数和全体正偶数，究竟谁包含的数更多？

问题 2：把两个同心圆上的点用公共半径连接起来，这样就构成了两个圆上的点之间的一一对应关系，如图 2-2 所示，对于大圆上的任意一点，通过公共半径，总能找到小圆上的一点与它对应；同样，对于小圆上的任何一点，通过公共半径，总可找到大圆上的一点与它对应。通过分析，我们发现大小圆上的点应该一样多。然而，常识会认为大小圆上的点不可能是一样多的，大圆上的点应该要多于小圆上的点。那么大小圆上的点，究竟是谁更多？

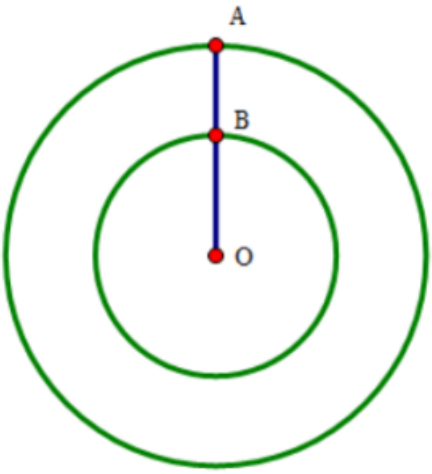



图 2-2 公共半径连接同心圆

问题 3：伽利略在《关于新科学的对话》一书中曾注意到**所有自然数与自然数的平方可以建立一个一一对应关系**，比如：

$$1\rightarrow 1,\ 2\rightarrow 4,\ 3\rightarrow 9,\ 4\rightarrow 16,\ 5\rightarrow 25,\ \ldots,\ n\rightarrow n^2,\ \ldots$$

因此意味着平方数与自然数一样多。然而，常识告诉我们，自然数比平方数要多许多。此即伽利略悖论。那么究竟自然数和平方数谁更多？

问题 4：希腊善走的神阿里基斯和乌龟进行百米赛跑，阿里基斯的速度却是乌龟的 10 倍，阿里基斯却追不上乌龟，因为乌龟先走 10 米，阿里基斯开始追，可当阿里基斯跑完 10 米，乌龟却已爬完了 1/10 米，当阿里基斯跑完 1/10 米，乌龟却已爬完了 1/100 米，……，就这样阿里基斯永远追不上乌龟吗？

现在，我们开始学习集合论，并尝试给大家揭开上面 4 个问题的答案。

# 2.1 集合论基础

在提出上面 4 个问题之后，我们现在来正式从基础开始来学习集合论相关知识，并力图从集合理论上解决上面提出的问题。

## 2.1.1 集合定义

那么什么是集合呢？所谓集合，通常是指按照某种特征或规律结合起来的事物的总体。例如中国所有的城市，自然数的全体以及直线上所有点的总体等等。一般地，我们把研究对象统称为元素（element），把一些元素组成的总体叫集合（set）。元素互不相同（不重复）。两个集合的元素一样，那么两个集合相等。

通常用大写 A, B, C 等表示集合；用小写 a, b, c 等表示元素。元素 a 属于集合 A 表示为：$a\in A$；a 不属于集合 A 表示为：$a\notin A$。

比如，设集合 A 为中国所有城市的集合，那么设元素 a=“成都”，那么 $a\in A$;设 b=“洛

杉矶”， 则 b∉A。

## 2.1.2 集合的表示

当我们使用集合的时候，首先需要通过一些方式来表示集合。集合表示方法主要包括列举法，描述法和图像法等。

- 列举法：把集合中的所有元素一一列举出来,写在大括号内,这种表示集合的方法叫做列举法。比如，集合 A={0,1,2,3,4,5,6,7,8,9}。列举集合的时候，集合与元素顺序无关，个数有限。列举法常用于表示有限集合。
- 描述法：把集合中元素的公共属性用文字、符号或式子等描述出来。比如，集合 B={x ∈R|10<x<20} 或者 B={x|10<x<20}；集合 C={a+bi | a,b∈R}。描述法常用于表示无限集合。
- 图像法：又称韦恩图法，是一种利用二维平面上的点集表示集合的方法。一般用平面上的矩形或圆形表示一个集合，这种表示方法非常直观。具体例子请参考本书 2.1.4 节。

数学理论中常见集合表示有：

- 非负整数集（自然数集）：N（其中在集合论包含元素 0，数论不包含元素 0）；
- 正整数集：$N^*$、$N^+$；
- 整数集：Z；
- 有理数集：Q；
- 实数集：R；
- 复数集：C。

## 2.1.3 集合关系

集合与集合之间可能存在着一定的关系，最常见的关系包括相等，子集。现在我们来简单介绍一下这些关系的概念。

- 全集：全集含有所研究问题中涉及的所有元素，一般用 E 或 U 表示，任意集合都可能是全集，若研究实数，则 R 就是全集。
- 子集：如果集合 A 的任意一个元素都是集合 B 的元素，那么集合 A 称为集合 B 的子集，记作 A⊆B。
- 相等：当 A⊆B，B⊆A 时，也就是如果集合 A 是集合 B 的子集并且集合 B 也是集合 A 的子集，那么集合 A 和集合 B 相等，记作 A=B。
- 真子集：如果集合 A 是 B 的子集，且 A≠B，则集合 A 是 B 的真子集，记作 A⊂B 或 A⊊ B， A ⫋ B。
- 空集：不包含任何元素的集合叫作空集，记为Φ。空集是任何集合的子集。

例如，对于集合{a, b}，它共有 4 个子集：

Φ, {a}, {b}, {a, b}

其中包含 3 个真子集：

Φ, {a}, {b}

3 个非空子集：

{a}, {b}, {a,b}

2 个非空真子集：

{a}, {b}

实际上，含有 n 个元素的集合共有 $2^n$ 个子集（乘法原理可证明，参见本书 11.1.1 节）；$2^n$-1 个真子集（集合的子集中去掉集合本身）；$2^n$-1 个非空子集（集合的子集中去掉空集）；$2^n$-2 个非空真子集（集合的子集中去掉集合本身和空集）。

为了理解空集的概念，我们来看下面这个选择题。

对于空集用符号⌀表示，则：

A．⌀∈{⌀}，⌀⊊{⌀}

B．⌀∉{⌀}，⌀⊊{⌀}

C．⌀∈{⌀}，⌀⊈{⌀}

D．⌀∉{⌀}，⌀⊈{⌀}

因为⌀是集合{⌀}中的一个元素，所以⌀∈{⌀}，而且⌀是任何非空集合的真子集，所以⌀⊊{⌀}。因此正确答案为A。注意集合论中下面两种表示方法的区别：

- {a} ⊆A：{a}是 A 的子集，表示的是集合与集合的关系；
- a∈A： 元素 a 属于集合 A，表示的是元素与集合的关系。

## 2.1.4 集合运算

与数与数之间可以进行加减乘除运算类似，集合与集合之间也可以进行相关的运算，这些运算主要包括交集运算，并集运算，补集运算，差集运算，幂集运算等。设有集合 A={1, 2, 5, 0, 3, 11}，B={2, 4, 5, 9, 8, 13}，全集 U={1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15}，现在我们来看下各种集合运算的定义和计算过程。

**1.交集运算**

集合 A 与 B 的交集运算记为 A∩B，元素 x 属于 A ∩B 当且仅当 x 属于 A 且 x 属于 B，交集运算的韦恩图表示如图 2-3 所示。

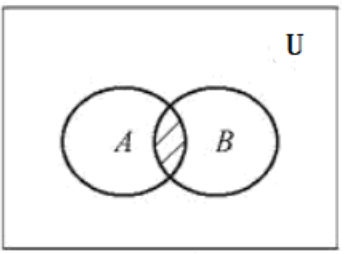



图 2-3 交集运算

根据上面给出的集合数据和定义，A∩B={2, 5}，也就是说，A∩B 包含的元素其实就是集合 A 和集合 B 的公共元素。

**2.并集运算**

集合 A 与 B 的并集运算记为 A∪B， 元素 x 是 A∪B 的元素，当且仅当 x 是 A 的元素，或 x 是 B 的元素。并集运算的韦恩图表示如图 2-4 所示。

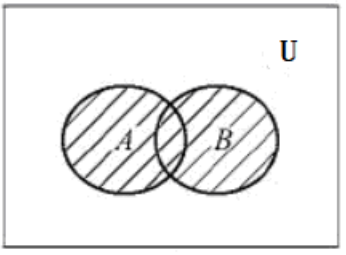


图 2-4 并集运算

根据上面给出的集合数据和定义，A∪B={1, 2, 3, 4, 5, 8, 9, 11, 13}，也就是说，A∪B中的元素就是A集合和B集合中的元素合并去重之后的所有元素。

**3.补集运算**

集合A的补集运算记为$\complement_U A$或者$A^c$，由全集U中所有不属于A的元素组成的集合。补集运算的韦恩图表示如图2-5所示。

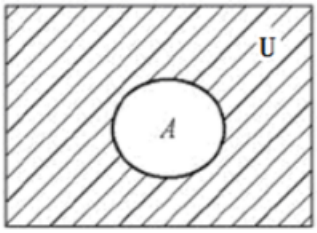



图 2-5 补集运算

根据上面给出的集合数据和定义，$\complement_U A$={4, 6, 7, 8, 9, 10, 12, 13, 14, 15}，也就是全集U中去掉所有A中的元素构成的集合。

**4.差集运算**

集合A与B的差集运算记为A-B或A\B ，由所有属于A但不属于B的元素x组成的集合。差集运算的韦恩图表示如图2-5所示。

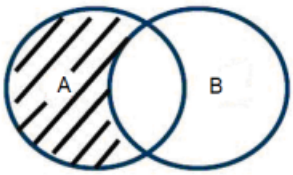



图 2-6 差集运算

根据上面给出的集合数据和定义，A\B ={1, 0, 3, 11}。

**5.幂集运算**

集合A的幂集是由集合A一切子集构成的集合，记为P(A)。假设集合A的元素个数为n，那么幂集的元素个数为$2^n$。比如设集合A={1, 2}，那么A的幂集为{Φ , {1}, {2}, {1, 2}}。

**6.笛卡尔积**

设A，B为集合，用A中元素为第一元素，B中元素为第二元素构成有序对，所有这样的有序对组成的集合叫做A与B的笛卡尔积，记作A×B。比如设集合A={a, b}，集合B={c, d}则两个集合的笛卡尔积为{(a, c), (a, d), (b, c), (b, d)}。

我们可以把集合A所包含的元素个数称为集合的基数，记为card（A）或|A|，在计算A∪B的元素个数的时候，有如下关系：

$$card(A \cup B)=card(A)+card(B)-card(A \cap B)$$

这是一个非常有用的公式，在后面练习计算集合元素个数的时候，我们会用到它。

此外，集合运算也有一些常用的运算性质，我们把这些性质总结如下：

1. $A \cap A = A$
2. 交换律：$A \cap B = B \cap A$
3. 集合律：$A \cap B \cap C = A \cap (B \cap C)$
4. $A \cap \phi = \phi \cap A = \phi$
5. 分配律：

$A \cap (B \cup C) = (A \cap B) \cup (A \cap C)$

$A \cup (B \cap C) = (A \cup B) \cap (A \cup C)$

6. A ∪ (A ∩ B) = A

7. A ∩ (A ∪ B) = A

为了加深对集合基本运算的理解，现在我们来看几个具体的问题以及对问题的求解过程。

问题 1：设集合 A={x|1≤x<4},B={x|2x-8≥8-2x}，那么怎么求 A∪B，A∩B 呢？

我们可以通过解不等式，求得集合 B 为{x|x≥4}，如果把集合 A{x|1≤x<4}和 B{x|x≥4}表示在数轴上，如图 2-7 所示：

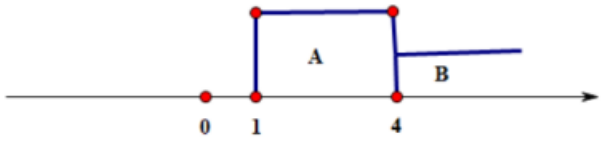



图 2-7 在数轴上表示集合 A 与 B

那么结合图 2-7，根据并集与交集的定义有：

A∪B={x|x≥1}

A∩B=ϕ

问题 2：设已知集合 A={1, 4}，集合 B 满足 A∪B={1, 4}，则集合 B 有多少个？

因为 A∪B=A，所以，所以 A 的子集都可以满足这个条件。由于 n 个元素的集合共有 $2^n$ 个子集，所以满足条件的集合为 $2^2$=4。它们是ϕ，{1}，{4}，{1, 4}。

问题 3：学校先举办了一次田径运动会，某班有 9 名同学参赛，又举办了一次球类运动会，这个班有 12 名同学参赛，两次运动会都参赛的有 5 人。两次运动会中，这个班共有多少同学参赛？

设参加田径运动会的同学组成集合 A，参加球类运动会的同学组成集合 B。那么有：card(A)=9, card(B)=12, card(A∩B)=5，而参加 2 次比赛的同学可以表示为 card(A∪B)。

由集合公式 card(A∪B)=card(A)+card(B)-card(A∩B)，代入上述已知条件：

$$card(A\cup B)=9+12-5=16$$

所以，这个班共有 16 人参加比赛。

问题 4：某班有学生 50 人，其中体育爱好者 42 人，音乐爱好者 35 人，还有 7 人既不爱好体育也不爱好音乐，则该班既爱好体育又爱好音乐的人数为多少人？

设体育爱好者组成集合 A，音乐爱好者组成集合 B，则有 card(A)=42, card(B)=35，因为 50 人中有 7 人既不爱好音乐，也不爱好体育，那么有 50-7=43 人要么爱好音乐要么爱好体育，也就是 card(A∪B)=43，求的是既爱好体育又爱好音乐的人，记为 card(A∩B)。那么有：

$$card(A\cup B)=card(A)+card(B)-card(A\cap B)$$

代入上述已知条件有：

$$43=42+35-card(A\cap B)$$

于是有：

$$card(A\cap B)=34$$

即既爱好音乐又爱好体育的有 34 人。

问题 5：设集合 A={x|0≦x<3 且 x∈N}的真子集的个数是？

根据已知条件求出集合 A={0, 1, 2}，包含 3 个元素，于是真子集个数为：$2^n-1=2^3-1=7$。

## 2.2 无穷集合

学完集合论基础之后，我们尚不能解决开始提出的那 4 个问题，而我们中学的集合论知识也到此为止了。

不过在本书，我们打算带领大家把集合论的知识进一步拓展，走向无穷，进入无限的王国。因为康托尔的不朽的功绩正是朝向无穷的冒险冒进。

### 2.2.1 潜无穷和实无穷

在讨论无穷集合的时候，有两个概念比较重要，即“潜无穷”和“实无穷”。我们应该能够意识到自然数是无穷无尽的，因此自然数构成的集合应该是一个无穷集合。但在人们传统的观念里，一定是“整体大于局部”。我们潜意识就觉得自然数的数目一定大于偶数的数目，也大于平方数的数目，但是自然数与偶数和平方数的一一对应也暗示着它们的数目实际上是一样多，这就与传统观念矛盾了。因此，在伽利略遇到这个问题的时候，就拒绝或者回避自然数全体的存在，即否认实无穷的存在。所以，这里的实无穷就是把自然数看成是无限的，而且把无限的自然数全体看成一个存在的整体。

**1.实无穷**

所谓实无穷，是指把无限的整体本身作为一个现成的单位，是已经构造完成了的东西，换言之，即是把无限对象看成为可以自我完成的过程或无穷整体。

实际上，亚里斯多德等都只承认潜无穷。所谓潜无穷，就是把无限看作永远在延伸着的，一种变化着成长着被不断产生出来的东西来解释。它永远处在构造中，永远完成不了，是潜在的,而不是实在。把无限看作为永远在延伸着的不断在创造着的永远完成不了的过程。照此观点，自然数不能构成一个集合，因为这个集合是永远也完成不了的，它不能构成一个实在的整体，而是永远都在构造之中。比如构成一条直线的点有无穷个，并且这条直线永远延伸着，不会有终结的一天。

实无穷论者认为，无穷是一个现实的、完成的、存在的整体。潜无穷论者认为,无穷并不是已完成的,而是就其发展来说是无穷的，无穷只是潜在的。

微积分的理论基础极限论使用的潜无穷（∞）的观点（参考本书 10.1 节），随着极限论的发展，潜无穷逐渐取代了实无穷的优势。

**2.潜无穷**

潜无穷是把无限数量放入无限时间中，即所谓“一尺之捶，日截其半，万世不竭”；实无穷是把无限数量放入有限时间中，实无穷可以被数尽。如同有一个点在数轴上从 0 点向 1 点移动，这个点一定会在有限时间内经过 0 与 1 之间所有的点，数完 0 与 1 之间的无穷多的点，即所谓“一尺之捶，时进其距，穷其无穷”。

支持实无穷派的主要观点主要有以下两个：

首先，物质不是无限可分的，存在着最小的单位；

第二，无穷大是可以构造出来的。例如：他们认为“全体自然数”是存在的，因为每个自然数都是可以数到的，既然每个自然数都存在，那么“全体自然数”当然是存在的。

而潜无穷派则与之相反，他们认为：

首先，物质是无限可分的；

第二，无穷大是无法构造出来的。与实无穷派相对应，他们认为“全体自然数”是不存在的，因为自然数是数不完的，这表明自然数的产生是个无穷无尽的过程，只有这个过程结束了，才能得到自然数全体，但这个过程无法结束，因而无法得到自然数的全体。

康托尔的集合论就是建立在实无穷的基础之上的，他将无穷集合看成一个完成了的实体。康托尔突破了“整体一定大于部分”的传统观念，接受“部分能等于整体”，以“可以通过一一对应的方法来比较两个集合的元素多少；实无限是一个确实的概念”为前提，创立了影响深远的集合论。

## 2.2.2 集合等势

如果规定集合A中包含的元素个数比集合B中包含元素个数多，集合A就大于集合B，那么如何比较两个集合的大小呢？对于有限集合，只需要数出每个集合中的元素个数即可比较大小。但是对于无限集合是无法数出集合中的元素的个数的，所以无法用这种方法比较集合的大小。康托尔给两个集合数目进行大小比较定义了如下规则：

对于集合A和B，如果从A到B上的元素存在**一一对应**的关系，那么A与B等势或者说具有相同的基数。

根据上面的规则，无论集合是有限的还是无限的，都可以进行大小比较了。

比如，自然数与正偶数之间，存在一一对应关系 $f:n\rightarrow 2n$；自然数与平方数之间存在一一对应关系 $f:n\rightarrow n^2$，因此自然数与正偶数、平方数之间就是等势的，即具有相同的基数。

那么整数和自然数是等势的吗？这里只需要找到一个一一对应的关系即可，比如把自然数和整数这两个集合写成如下形式：

$N=\{0, 1, 2, 3,\ldots\ldots\}$

$Z=\{0, 1, -1, 2, -2,\ldots\ldots\}$

于是，可以找到两个集合之间的一一对应关系：

$$Z(i)=\begin{cases} i & \text{当 } i=0 \text{ 时} \\ (i+1)/2 & \text{当 } i \text{ 属于}\{\text{正奇数}\}\text{时} \\ -i/2 & \text{当 } i \text{ 属于}\{\text{正偶数}\}\text{时} \end{cases}$$

具体来说：

当 i=0 时，Z(0)=N(0)=0；

当 i=1（正奇数）时，Z(1)=(N(1)+1)/2=(1+1)/2=1；

当 i=2（正偶数）时，Z(2)=-(N(2))/2=-2/2=-1。

因此我们建立了自然数和整数之间一一对应的关系，所以整数和自然数是等势的。

## 2.2.3 可数集与不可数集

我们把能与自然数集建立一一对应关系的集合（即与自然数集等势的集合），比如正偶数，整数以及平方数的集合，叫做可数集。已经证明**在无穷集合的等级上，可数集是最小的**。

那么有哪些比可数集更大的集合呢？

我们知道数分为自然数，整数，有理数（整数和分数），实数（有理数和无理数如π和$\sqrt{2}$）等组成。其中自然数和整数已经确认是可数集了。所以接下来看看有理数和实数是不

是可数集呢？

首先考查一下有理数。

**1. 有理数集合的等级**

在数轴上，自然数是比较稀疏的，两个连续自然数之间还有很大的间距，而有理数却显得密密麻麻。因此我们直觉上肯定会认为有理数集比自然数集要大，因此是在可数集之上的一个新等级。但康托尔给出的答案是**否**。也就是说，有理数集与自然数集等势，也是可数集。

既然有理数集和自然数集是等势的，就应该存在一个自然数集到有理数集的一一对应关系。那么这个关系存在吗？康托尔用对角线法则证明了有理数集与自然数集等势。所谓的对角线法则，就是将所有正分数按照如图 2-8 所示排列：

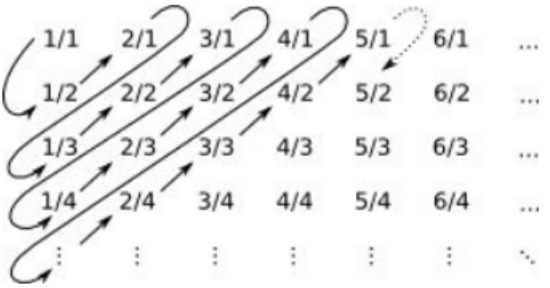



图 2-8 有理数的一种排列

第一行依大小次序包括所有以 1 为分母的正分数（即全体正整数）；

第二行依大小次序包括所有以 2 为分母的正分数；

第三行依大小次序包括所有以 3 为分母的正分数；

…

第 N 行依大小次序包括所有以 N 为分母的正分数；

显然，图 2-8 包含了每一个正有理数。按照图中箭头的方向依次重新遍历，略去已经出现过的数，就得到了全体有理数的一个无穷序列$\{q_1, q_2, q_3, \ldots\}$，把 0 和负有理数加进去，得新序列$\{0, -q_1, q_1, -q_2, q_2, \ldots\}$就是一个包含所有有理数的集合。

康托尔上面用对角线法给出的图形本身就是一个函数，即自然数和有理数之间的一个一一对应关系。虽然算出这个函数的具体表达式比较困难，但是，如图 2-9 所示，知乎上曾有网友给出过这个函数的表达式：

$$-\frac{2\left(\left\lfloor\frac{1}{2}\left(\sqrt{8n-7}-1\right)\right\rfloor+2\right)}{\left\lceil\frac{1}{2}\left(1-\sqrt{8n-7}\right)\right\rceil+(-1)^{\left\lfloor\frac{1}{2}\left(\sqrt{8n-7}-1\right)\right\rfloor}\left(\left\lfloor\frac{1}{2}\left(\sqrt{8n-7}-1\right)\right\rfloor\left(\left\lfloor\frac{1}{2}\left(\sqrt{8n-7}-1\right)\right\rfloor+2\right)-2n+2\right)-2}-1$$

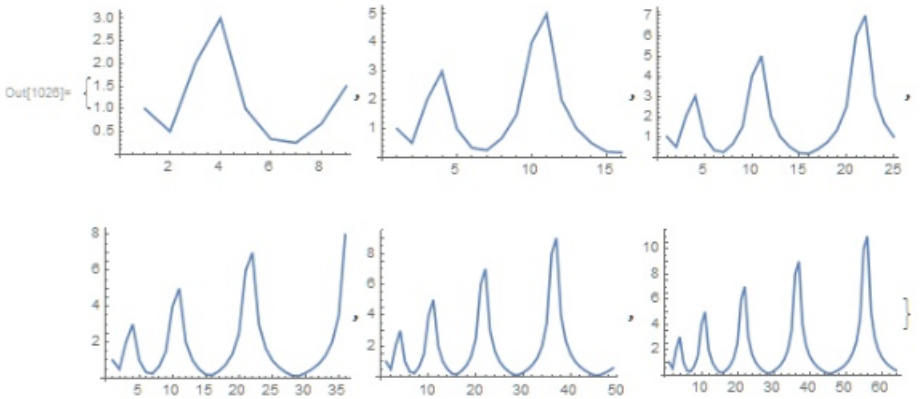



图 2-9 对角线法的函数表达式

可见，有理数集确实是可数集，和自然数集等势。

那么，是不是就意味着所有的无穷集合都是可数集了呢？再来看看实数集。

**2. 实数集的集合等级**

自然数集和实数集能一一对应吗？也就是说，实数集是可数集吗？康托尔证明了**实数集不是可数集**。他给出的证明方法中，最有名的是对角线证法。现在我们来介绍下这个著名的证明方法。

康托尔以数轴区间(0, 1]上的点为目标，只要证明了这个区间上的点不能与自然数集一一对应，那么实数集就当然也不可能和自然数集一一对应了。

康托尔用反证法证明如下：

假设(0, 1]是可数集，那么可以把它里面的所有实数排成一个序列：$a_1, a_2, a_3, \ldots, a_n, \ldots$。现在将每个这样的实数写成十进制小数形式如图 2-10 所示：

$$
\begin{aligned}
a_1 &= 0.a_{11}a_{12}a_{13}a_{14}\cdots \\
a_2 &= 0.a_{21}a_{22}a_{23}a_{24}\cdots \\
a_3 &= 0.a_{31}a_{32}a_{33}a_{34}\cdots \\
&\cdots\cdots \\
a_n &= 0.a_{n1}a_{n2}a_{n3}a_{n4}\cdots \\
&\cdots\cdots
\end{aligned}
$$

图 2-10 实数区间 (0,1]的一个排列

其中，有理数也可以写成无穷小数，比如 1/2=0.4999999999…。现在构造一个实数 $b=0.b_1b_2b_3\ldots b_n\ldots$，并使它的第 n 位数字 $b_n \neq a_{nn}$。例如，如果 $a_{33}=6$，那么 $b_3=5$；$a_{77}=8$，$b_7=1$，这样构造一个 $b_n$ 是很容易的。

通过上面方法构造出来的 $b_n$ 必然与上面的 $a_1\ldots a_n\ldots$中的所有数都不相同，因为至少 $b_n$ 与 $a_n$ 在对角线上的第 n 位是不同的。于是，假设自然数集与区间(0, 1]上的实数一一对应时，b 被遗漏了。所以假设错误。因此区间(0, 1]上的实数是不可数的，那么整个实数集也是不可数的。

**对角线方法与计算机起源小故事**

对角线方法，于 1891 年由康托尔在一篇 4 页的论文中发表，此后对角线方法被奥地利数学家哥德尔(1906-1978)用来解决一致性问题时构造系统内不可证命题，然后图灵又再次使用了哥德尔的方法构造出了不可判定命题并引发了关于图灵机的构想。从这里，我们可以看到**康托尔的工作与计算机的起源存在着内在联系**。

实数集是不可数的，我们称之为不可数集，并记为 C。不可数集具有比可数集更高的等级。同样可以证明**任何长度不相同的线段中的点乃至于整个数轴的点都是相同的，都是不可数集。**

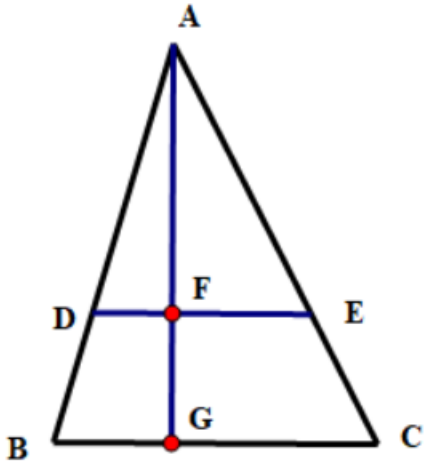



图 2-11 长度不相同的两条线段上的点一一对应

如图 2-11 所示，对于长度不同的两条线段 DE 和 BC，我们可以过构造三角形 ABC 并从它的顶点 A 做任意直线与 DE 和 BC 相交于点 F 和 G，那么 DE 和 BC 上的点就是一一对应的。由于存在一一对应的关系，那么 DE 和 BC 上的点就是一样多的。

同样的，在前面图 2-2 中所示，同心圆中大圆小圆上的点一一对应，因此都是一样多的，而且都是不可数集。这就告诉我们，虽然有理数和无理数在数轴上都是密密麻麻的，但是无理数却比有理数要多得多，因为有理数是可数的，而无理数是不可数的。

**通过有理数集是可数集，实数集是不可数集，让我们认识到了无穷集合也是具有不同等级的。**

一般人凭直觉会认为代数数（比如 1，2，$\sqrt{2}$ 等），要比超越数（比如 e，π 等）多。但是康托尔通过存在性证明了，代数数是可数集，由于实数集是不可数集，所以超越数是不可数集，也就是说尽管我们知道的超越数很少，但实际上超越数却比代数数多很多，虽然我们并不知道这些超越数具体是什么，等于多少，这就是所谓的存在性证明。

**3. N 维空间点的集合等级**

数轴上的所有点，也就是对应的实数集是不可数的，那么 n 维空间的点是什么样的呢？

实际上，区间(0, 1)与单位正方形也可以建立一一对应关系。如图 2-12 所示，在区间(0, 1)任取一点 0.827102372932…，然后将这个无穷小数（任意一点都可以表示为无穷小数）的奇数位和偶数位取出来，得到 2 个新小数，比如 0.870323…和 0.212792…。以这两个数为横坐标和纵坐标的点恰好落在了这个单位正方形内；反过来，取单位正方向内的任意一点，也可以把它的横坐标和纵坐标按照奇偶位结合在一起，得到了单位区间上的点。比如(0.17246…, 0.39232…)，对应单位区间上的点为 0.1379224362…。

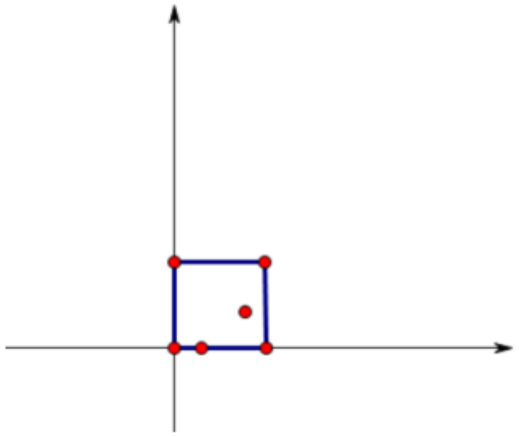


图 2-12 区间(0, 1)与单位正方形建立一一对应关系

同样的，单位正方形里的点与整个平面上的点也是一样多的，这与单位线段上的点与整个数轴上的点数量一样多类似。实际上，整个 n 维空间也可以和直线建立一一对应的关系。因此 n 维空间的点集都是不可数集。

回到本章开始的的阿基里斯追乌龟的问题。过去传统思维认为，阿基里斯和乌龟在跑步过程中，同一点两者都不会停两次，即每个时刻只能对应一个点，它们跑的时刻是相同的，因此跑过的点也同样多。但是，阿基里斯要追上乌龟，必须跑过比乌龟更多的点，从而超过乌龟。因此阿基里斯绝不可能追上乌龟。现在，集合论告诉我们，不等长的两线段具有相同的点，所以阿基里斯可以追上乌龟。

## 2.2.4 超限基数与序数

通过前面的内容，我们已经知道，自然数集，整数集和有理数集是可数集；实数集是不可数集。那么，还能找到比不可数集更大的基数吗？答案是**能**。

康托尔从两个角度找到了比不可数集更大基数的集合，这两个角度分别是**幂集**和**无穷序数**。

**1. 超限基数**

首先是幂集的基数。幂集的概念相信大家已经清楚了。一个含有 n 个元素的有限集合，它的幂集是 $2^n$ 个元素。比如包含 2 个元素的集合{a,b}，那么它的幂集就是{ ϕ,{a},{b},{a,b}}，共 $2^2=4$ 个元素。对于有限集合来说，显然它的幂集元素数目要大于原集合。那么对于无穷集合是什么情况呢？康托尔证明了**对于任意集合，它的幂集基数总是大于它的原集基数**。这就是康托尔定理。

我们把可数集的基数称为可数基数，并记为$\aleph_0$（阿列夫零，$\aleph$是希伯来字母表的第一个字母），于是可数集幂集基数可记为$\aleph_1=2^{\aleph_0}$，根据康托尔定理有$\aleph_0<\aleph_1$。通过反复使用康托尔定理，可以得出 $2^{\aleph_0}$的幂集基数为$\aleph_2=2^{2^{\aleph_0}}$。于是基数之间的大小不等式链可以记为：

$$\aleph_0<2^{\aleph_0}<2^{2^{\aleph_0}}<\cdots$$

我们把这些基数叫做超限基数，把自然数叫做有限基数。超限基数是与有限基数相对的一类基数，可数基数、不可数基数统称超限基数。将所有超限基数从小到大排列出来，得到超限基数序列：

$$\aleph_0,\aleph_1,\aleph_2,\ldots,\aleph_\omega,\aleph_{\omega+1},\ldots,\aleph_\alpha,\aleph_{\alpha+1},\ldots$$

这里$\aleph_0$是可数基数，也是最小的超限基数。当$\alpha>0$时，$\aleph_\alpha$都是不可数基数。

基数考虑的是集合中元素的多少，没有考虑元素间可能出现的次序。于是，康托尔还从序数的角度建立了无穷的序数理论。

**2. 超穷序数**

自然数以及自然数全体都是一种序数。康托尔把自然数序列 1, 2, 3, …从 1 开始，并通过相继加 1 产生有穷序数的过程概括为“第一生成原则”（或延伸原则）。由第一生成原则得到的全体有穷序数集称为第一数类。第一数类中显然没有最大数，于是康托尔通过第二生成原则（穷竭原则），从 1, 2, 3, ...出发，通过穷竭，引出一个新数ω，它大于原集合中的任意数，也不再是潜无穷观念中的∞，而是一个确实存在的数，我们把它叫做超穷序数。

通过以上 2 个原则反复应用，就形成如下的无穷序列：

0, 1, 2, ..., ω, ω+1, ω+2,... ω+ω(ω*2), ω*2+1,... ω*ω(ω²), ω²+1,... ω³, ω³+1,..., ω^ω, ω^ω+1,... ω₁,... ω₂,... ω₃,...

0, 1, 2, …是第一“数类”，它是可数集，基数为$\aleph_0$。

从ω开始的无穷序列的全体，即 ω, ω+1, ω+2,... ω+ω(ω*2), ω*2+1,... ω*ω(ω²), ω²+1,... ω³, ω³+1,..., ω^ω, ω^ω+1,...构成了第二“数类”，基数为$\aleph_1$。

第二“数类”也不存在最大元素，因此康托尔引入第三生成原则（限制原则），保证新“数类”的基数是“大于且是第一个大于”前一个“数类”的基数。所以：

$\omega_1$, …是第三“数类”，基数为$\aleph_2$，$\omega_1$大于第二数类中的任何数。

$\omega_2$, …是第四“数类”，基数为$\aleph_3$，$\omega_2$大于第三数类中的任何数。、

...

归纳起来，也就是下面的序列：

第一“数类”：0, 1, 2, ...

第二“数类”：ω, ω+1, ω+2,... ω+ω(ω*2), ω*2+1,... ω*ω(ω²), ω²+1,... ω³, ω³+1,..., ω^ω, ω^ω+1,...

第三“数类”：$\omega_1$,...

第四“数类”：$\omega_2$,...

第五“数类”：$\omega_3$,...

就这样，康托尔从超限基数和超穷序数两个角度一起刻画了无穷集合的完整视图，建立起了集合论的牢固基础。但当时在数学界，世界各地的保守势力无法接受实无穷和比较无穷的大小，因此都竭力反对。法国著名数学家庞加莱甚至宣称，总有一天，康托尔的集合论将被当做一种被征服了的疾病。但最终人们还是接受了康托尔的集合论。

康托尔从幂集和无穷序数两个角度构造越来越大的基数，得到了两种无穷基数序列：

$$\aleph_0 < 2^{\aleph_0} < 2^{2^{\aleph_0}} < \cdots$$

$$\aleph_0,\ \aleph_1,\ \aleph_2,\ \cdots$$

第二行的不等式中$\aleph_1$是紧跟在$\aleph_0$之后第一个超穷基数，但是第一行的不等式中对于$2^{\aleph_0}$我们只知道它比$\aleph_0$大，但不知道是不是第一个比$\aleph_0$大的基数。因此康托尔提出了一个假设：$2^{\aleph_0}=\aleph_1$，即在$\aleph_1$与$2^{\aleph_0}$之间没有其它超穷基数。

后来康托尔证明了$2^{\aleph_0}=C$（即$2^{\aleph_0}$是不可数集，与实数集等势），于是上面的假设又等价于在可数集基数$\aleph_0$与不可数集基数$2^{\aleph_0}$之间不存在其它无穷基数。

由于实数集又叫连续统，所以此假设被称为**连续统假设**，它又被称为希尔伯特第一问题，在 1900 年第二届国际数学家大会上，大卫·希尔伯特把康托尔的连续统假设列入 20 世纪有待解决的 23 个重要数学问题之首。1938 年哥德尔证明了连续统假设和世界公认的 ZFC 公理系统不矛盾。1963 年美国数学家保罗·寇恩证明连续假设和 ZFC 公理系统是彼此

独立的。因此，连续统假设不能在 ZFC 公理系统内证明其正确性与否。

## 2.3 罗素悖论与第三次数学危机

所谓悖论，是指表面上同一命题或推理中隐含着两个对立的结论，而这两个结论都能自圆其说。最古老和著名的悖论当属公元前 4 世纪由古希腊哲学家欧布里德提出的“说谎者悖论”，即**“我现在说的是一句假话”**。为什么这句话是悖论呢？因为如果这句话是假话，那么得出这句话就是真话，与假话矛盾；如果这句话是真话，那么得出这句话是假话，与真话矛盾。所以，这是一个悖论。

悖论的抽象公式可以理解为**如果事件 A 发生，则推导出非 A；而非 A 发生则推导出 A。即 A 为真当且仅当 A 为假。**

当集合论作为数学的基础，重新严格建造起了整个数学大厦的时候，突然爆发出了一个严重的悖论，号称绝对严密的数学因此出现自相矛盾和裂缝，因此引起第三次数学危机。这个悖论就是“罗素悖论”。

现在我们来看看什么是集合论中的罗素悖论，这个悖论曾经引起数学界极大的危机，产生了巨大的影响。

### 2.3.1 罗素悖论

1901 年英国数学家罗素（Bertrand Arthur William Russell，1872 年—1970 年，见图 2-13）发现了集合论存在一个悖论。这个悖论可以用集合语言描述如下：

设集合 A 为所有属于自身的集合组成的集合，即 $A=\{x|x\in x\}$，其中 x 是集合并且属于自己（集合是可以属于自己的）。

设集合 B 为所有不属于自身的集合组成的集合，即 $B=\{x|x\notin x\}$。

那么集合 $B\in A$ 还是 $B\in B$ 呢？

如果集合 $B\in A$，根据集合 A 的定义可以知道 $B\in B$，所以 B 是 B 中的一个元素，根据集合 B 的定义可以知道 $B\notin B$，此时产生了矛盾。

如果集合 $B\in B$，则符合集合 A 的定义，因此 $B\in A$。但是很显然 $A\cap B=\phi$，因此如果 $B\in A$，那么 $B\notin B$，此时产生了矛盾。

由于这个集合论悖论是罗素发现的，因此被后人称之为罗素悖论。

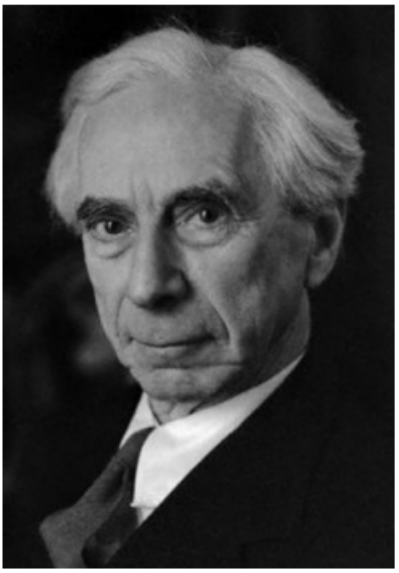


图 2-13 罗素

罗素悖论的一个生活中通俗的例子是所谓的“理发师悖论”。这个悖论是这样的：

某村子里的理发师宣布：他给所有不给自己理发的人理发。问理发师是否可以给自己理发？

假如理发师可以给自己理发，由于他给所有不给自己理发的人理发，因此理发师不能给自己理发，矛盾；假如理发师不可以给自己理发，由于理发师给所有不给自己理发的人理发，所以理发师可以给自己理发，矛盾。

幸好罗素悖论提出得比较晚，这时大家已经普遍接受集合论了，不然集合论有可能因为这个悖论而被扼杀。

实际上，这些悖论的根源，很多是由于在定义对象时，借助了一个整体来定义属于这个整体的某个部分，从而造成了循环定义或“反身自指”。

比如，理发师是村里所有村民集合中的一个成员。在定义理发师的时候，对应的就是村里所有人中，那个给所有“不给自己刮胡子的人”刮胡子的那个成员。这里，理发师就是属于这个村里所有人这个整体的一个部分，因此构成了循环定义。

## 2.3.2 公理集合论

罗素悖论提出之后，立即引起了数学界的大地震。许多基于集合论建立起来的数学理论和专著被迫延期发表，不得不直面集合论中这样一个根本问题。于是数学家们开始分析和解决悖论。

数学家们一致认为，解决集合悖论最有效的方法就是**公理化**。

所谓公理，是指依据人类理性的不证自明的基本事实，经过人类长期反复实践的考验，不需要再加证明的基本命题。

公理系统就是把一个科学理论公理化，用公理方法研究它，每一科学理论都是由一系列的概念和命题组成的体系。公理化的实现就是：

- 从其诸多概念中挑选出一组初始概念，该理论中的其余概念，都由初始概念通过定义引入，称为导出概念；
- 从其一系列命题中挑选出一组公理，而其余的命题，都应用逻辑规则从公理推演出来，称为定理。应用逻辑规则从公理推演定理的过程称为一个证明，每一定理都是

经由证明而予以肯定的。由初始概念、导出概念、公理以及定理构成的演绎体系，称为公理系统。初始概念和公理是公理系统的出发点。

例如，欧几里德《几何原本》中就规定了五条公设（以现代观点来看，公设也是公理）：

- 过相异两点能作且仅能作一条直线（直线公理）
- 一条有限直线可以继续延长。
- 以任意点为心及任意长为半径可以画圆（圆公理）。
- 凡直角都相等（角公理）。
- 若两条直线都与第三条直线相交，并且在同一边的内角之和小于两个直角，则这两条直线在这一边必定相交。

平面几何中的一切定理都可由这些公设推导而得。

公理系统要满足某些一般要求，包括系统的一致性（无矛盾性）、完全性，以及公理的独立性。

德国数学家策梅罗建立了第一个集合论公理系统。在该公理系统中，策梅罗给出了 7 条公理，并把集合当做不加定义的符合公理规定性质的原始概念。后经过弗兰克尔的完善和补充，形成了 ZF 公理系统。这 7 条公理分别为：

(1)外延公理：一个集合完全由它的元素所决定。如果两个集合含有的元素相同，则它们是相等的。

(2)分离公理：“对任意集合 X 和任意对 X 的元素有定义的逻辑谓词 P(x)，存在集合 Y，使 x∈Y 当且仅当 x∈X 而且 P(x)为真”

也就是说：若 A 是一个集合，那么可以断定，B={x∈A|P（x）}也是一个集合。

(3)无序对公理：对任意集合 X 和 Y，存在集合 Z，使得 X,Y 是它仅有的元素。

也就是说：我们可以用一个集合 Z={X,Y}来表示任给的两个集合 X 和 Y，称之为 X 与 Y 的无序对。

(4)并集公理：如果 S 是一集合，则 S 的并仍然是一集合。

(5)幂集公理：如果 S 是一集合，则 S 的幂集合仍然是一集合，换言之，一集合 S 的所有子集合仍然组成一集合。

(6)无穷公理：存在归纳集。（存在一个集合，空集是其元素，且对其任意元素 x，x+=x ∪ {x}也是其元素）

上面就是 ZF 公理系统，如果再加上下面的选择公理（Axiom of Choice，简称 AC），则叫做 ZFC 公理系统：

(7)选择公理：设 C 为一个由非空集合所组成的集合。那么，我们可以从每一个在 C 中的集合中，都选择一个元素和其所在的集合配成有序对来组成一个新的集合。可以通俗化理解即如果在前面放了放置了几堆苹果，那么我们可以在每堆中选取一个苹果，再把它们放在新的一堆内。

选择公理是一条有争议的公理，甚至会导致“巴拿赫-塔斯基”悖论，但是去掉选择公理会导致更多的悖论。

实际上，康托尔集合论出现悖论是因为不能把太大的集合看成是一种真正的集合，太大集合的引入是由于康托尔集合论中构造集合时使用的概括原则造成的，即所有满足某种性质的元素可以一起构成集合。后来冯洛伊曼（现代计算机之父）提出了集合和类的概念进行区别，引入类作为不定义概念，类分为集合和真类，规定真类不能作为类的元素，排除了“所有集合的集合”，因此对 ZF 系统进行了扩充，后经贝耐斯，哥德尔改进，成为 NBG 公理系统

此时，集合论是一个完全抽象的公理化理论，在这种理论里，集合概念不加定义，是所

有满足上述公理条件的对象，集合受到了恰当限制，使之不太大，避免了所有“对象”，剔除了由概括原则引出悖论的不合理部分，消除了罗素悖论。第三次数学危机也从形式上得到了解决，但正如公元前爆发的由无理数引起的第一次数学危机，直到近现代才由戴德金和康托尔最终解决一样，不少人认为第三次数学危机到目前为止，并没有圆满的解决。

公理化集合论推出之后，我们把 1908 年以前由康托尔创立的集合论称为“朴素集合论”。实际上，微积分的基础是实数理论，实数理论在自然数理论和集合论的基础上发展起来，其中自然数理论又被后来的数学家建立在集合论与逻辑之上。因此，集合论是数学的基础，整个现代数学构建在集合论的基础之上。

## 2.4 计算机中的集合

集合论的学习到此我们就告一段落了，回到计算机数学的话题上来，看看计算机里有哪些集合理论的应用或者体现呢？

在计算机里，很多对象比如文件，进程，线程，指令，运算符，网址 URL，网页，图片，日志，样本等都是可以组成对应的集合。我们往往把计算机程序中要处理的重要数据放在一个或几个集合里，然后进行遍历和计算。比如：

在机器学习领域，我们需要将日志，图片，网页，视频等数据分成训练集和验证集，利用机器学习算法，训练出模型，用于在线实时判断。

在爬虫领域，我们需要将网页上所有的 URL 链接放在一个集合里，根据这个集合中的 URL 来抓取对应的网页数据，进行分析并抽取有用数据。

在计算机安全领域，我们需要把文件，进程等放在一个集合里，用于扫描分析和对比。

在汇编编程语言里，所有的汇编指令组成了一个指令集。

在关系数据库里，一张二维表就是一组数据的集合。

在可计算理论里，P 问题与 NP 问题也是被当做集合来研究的。

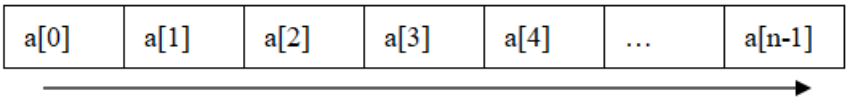
每个编程语言里，都提供了专门用来存放集合元素的容器，比如数组，链表，队列，栈，树，hash 表，map 等。通过容器对集合中的这些元素进行增加，查询，修改，和删除和相应的集合运算。

计算机里专门研究这些容器的学科被称为“数据结构(data structure)”。下面我们对这些数据结构进行一个简要的介绍。

### 2.4.1 数组

数组，就是一组相同类型的变量连续的存储在一段内存，并共用一个名字即数组名。每个变量使用下标来区别，并被称作数组的元素。数组的下标是从 0 开始的。因为每个元素的类型相同，因此各个元素在内存中存放的长度也一样，即它们占用的内存空间是等长的，而数组名，就是这段内存的首地址。

如图 2-14 所示，存放数组的内存是连续的，因此可以随机存取（即根据下标直接获取对应元素）。数组根据存放的数据的类型，可分为数值数组、字符数组、指针数组、结构数组等各种类别，而按照数组的维度，又可以分为一维数组，二维数组，三维数组等。如图 2-14 所示就是计算机内存中保存的一个包含 n 个元素的数组 a[n]，需要注意的是数组下标是从 0 开始计算的，所以最后一个元素为 a[n-1]。




内存增长方向

图 2-14 数组内存布局

## 2.4.2 链表

链表通常由一连串节点组成，如图 2-15 所示，每个节点包含任意的实例数据（data fields）和一个用来指向下一个节点地址的指针(next 指针)，链表最后一个节点的 next 指针为空（NULL），表示链表的结束。

使用链表结构可以克服数组需要预先知道数据大小的缺点，链表结构可以充分利用计算机内存空间，实现灵活的内存动态管理。但是和数组中每个元素的内存空间是连续的不同，链表中每个结点的存储空间是离散的，因此失去了数组随机读取的优点。同时，链表由于增加了结点的指针域，空间开销比较大。链表在插入的时候可以达到 O(1)的算法复杂度，但是查找一个节点或者访问特定编号的节点则需要 O(n）的算法复杂度。

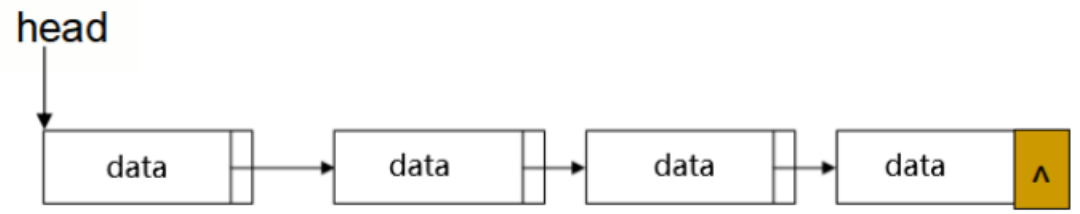



图 2-15 链表结构

我们把数组，链表叫做线性表，以此区别后面我们将要提到的树或图等非线性结构。

## 2.4.3 队列

队列是一种只允许前端（front）进行删除操作，而在后端（rear）进行插入操作的数据结构。在队列这种数据结构中，最先插入的元素将是最先被删除的元素；最后插入的元素将是最后被删除的元素，因此队列又称为“先进先出”（FIFO—first in first out）的数据结构。往队列插入数据叫作入队，从队列删除数据叫作出队。

操作系统的进程调度就是用的队列，银行排队也是队列。

如图 2-16 和 2-17 所示，队列可以基于链表和数组来构建。

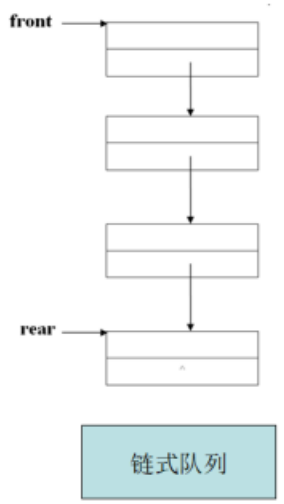



图 2-16 基于链表的队列

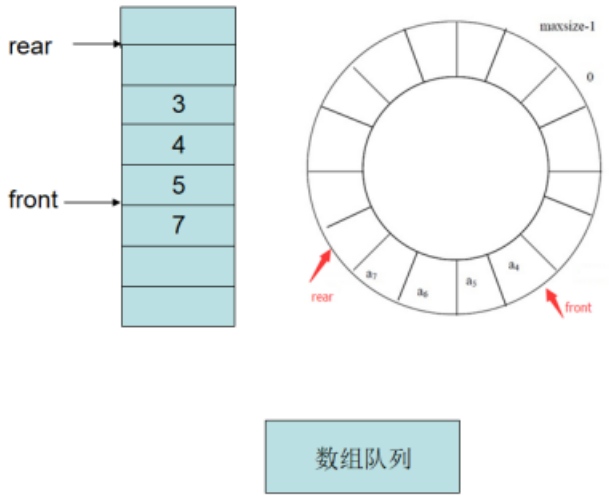



图 2-17 基于数组的队列和循环队列

## 2.4.5 栈

栈是允许在同一端进行插入和删除操作的特殊数据结构。被允许进行插入和删除操作的一端称为栈顶(top)，另一端为栈底(bottom)；栈底固定，而栈顶浮动，一般通过栈顶指针来标示当前栈顶位置；栈中元素个数为零时称为空栈。插入数据一般称为进栈（PUSH），删除栈中数据则称为退栈（POP）。

栈的一个典型特点就是后进先出。关于队列和栈的区别，有一个非常形象的比喻，比如人吃了吐就是栈，而吃了拉就是队列。实际生活中的子弹夹，程序设计里的函数调用过程都是栈的典型例子。

如图 2-18 所示，栈也可以基于链表和数组来构造。

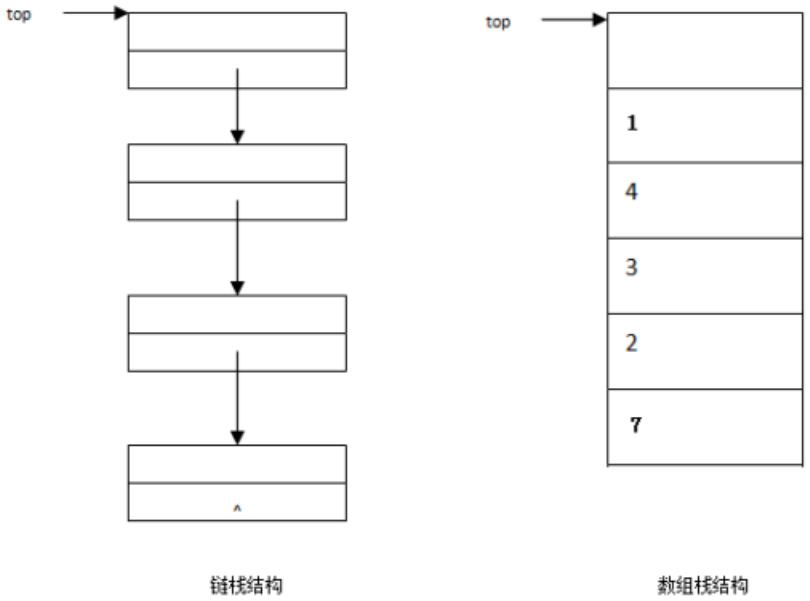



图 2-18 基于链表和数组的栈结构

## 2.4.6 树

树是由一个或多个结点组成的有限集合，其中满足如下条件：

- 必有一个特定的称为根(ROOT)的结点；
- 剩下的结点被分成 n >=0 个互不相交的集合 $T_1$、$T_2$、.....$T_n$，而且， 这些集合的每一个又都是树。
- 树 $T_1$、$T_2$、.....$T_n$ 被称作根的子树(Subtree)。

如图 2-19 所示，二叉树是每个结点至多只有两棵子树，并且二叉树的子树有左右之分，左右次序不能改变。

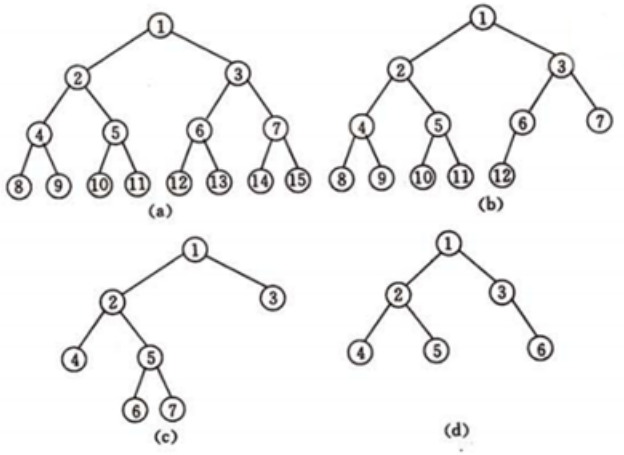



图 2-19 树的形态

## 2.4.7 Hash 表

哈希表（Hash table，也叫散列表），是根据关键字(key)直接访问数据(value)的数据结构。它通过把关键字映射到表中一个位置来访问记录，以加快查找的速度。这个映射函数叫做哈希函数，存放记录的数组叫做哈希表。

如果多个不同的 key，通过 hash 函数计算出来的记录位置一样，就叫 hash 冲突，解决 hash 冲突的方法很多，比如常见的链式法，如图 2-20 所示，将冲突的所有记录放在同一个链表里，为了性能优化，也可以将链表换成红黑树等。

Hash 表的查找速度是 O（1）的，所以查找速度非常快，适用于一些对查找速度要求较高的地方。

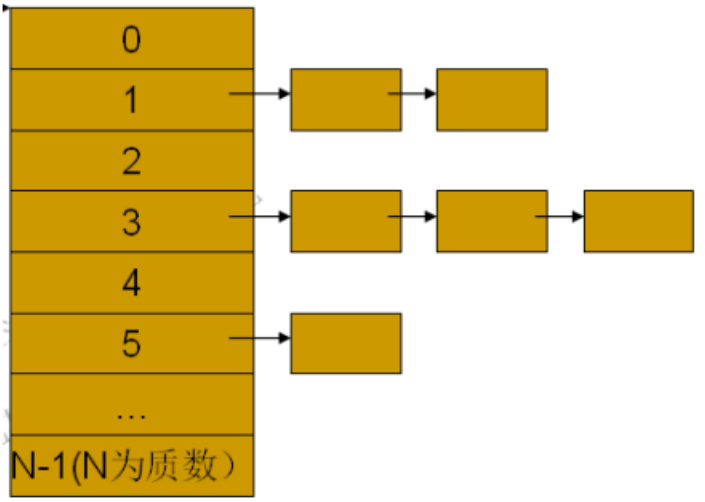



图 2-20 hash 表结构

## 2.4.8 Map 表

Map 表又叫字典结构，是用来存储不重复 key 的键值对 key-value 数据（比如“学号-学生成绩、身高等数据”）的一种高效数据结构。Map 表的底层可以是红黑树（树的一种）或者 hash 表。很多编程语言，比如 C++，Python，Go 等都提供了 Map 功能。

具有了集合论的基础知识，我们才能更好的理解计算机中这些数据结构，并对它们进行相关的运算，包括并集，交集，补集，差集等等。当然，在这里介绍的这些计算机中专门的数据结构，我们只是从数学集合论的角度来讨论它们的，我们并没有对它们进行更深入的介绍，如果大家对数据结构有兴趣，可以找到专门的书籍进行深入学习。

# 三，数学与程序的主角：函数

无论在数学领域还是计算机领域，函数都是一个重要的基本概念。高等数学的主要目标之一就是研究各种函数的性质和关系。计算机程序里的函数也是实现各种复杂算法的基础。在机器学习和人工智能领域，我们通过数据集训练而得到的模型本质上就是一个简单或者复杂的函数关系，而要找到这个函数关系，我们也需要与函数相关的最优化理论来求函数的最大值或者最小值以确保我们得到的模型最优。

因此，掌握好函数的基本理论，是我们进一步学习高等数学和计算机理论尤其是人工智能理论的关键。

函数的概念并不是一开始就有的，是经过历史上的数学家和物理学家先后提出并不断完善而形成的，尤其是后来集合论的诞生，使得函数的定义最终建立在集合论的基础之上。

17 世纪伽利略(1564-1642)在《两门新科学》一书中，几乎全部包含函数或称为变量关系的这一概念，此时他用文字和比例的语言表达函数的关系。

1637 年前后法国数学家笛卡尔(1596-1650)在他的解析几何中，已注意到一个变量对另一个变量的依赖关系，但因当时尚未意识到要提炼函数概念，因此直到 17 世纪后期牛顿、莱布尼兹建立微积分时还没有人明确函数的一般意义，大部分函数是被当作曲线来研究的。

1673 年，德国数学家莱布尼茨（1646-1716）首次使用“function”（函数）表示“幂” 。与此同时，牛顿在微积分的讨论中，使用 “流量”来表示变量间的关系。

1755，瑞士数学家欧拉(1707-1783) 把函数定义为“如果某些变量，以某一种方式依赖于另一些变量，即当后面这些变量变化时，前面这些变量也随着变化，我们把前面的变量称为后面变量的函数。”

1821 年柯西（1789-1857 ）完成《分析教程》,基于函数理论提出极限定义，，把微积分建立在极限基础之上。德国数学家魏尔斯特拉斯（1815-1897）提出了极限的“ε－N， ε－δ， ε－X”抽象定义，给微积分提供了严格的理论基础，解决了由无穷小引起的第二次数学危机。

1822 年法国数学家傅里叶（1768-1830）发现某些函数可以用曲线表示，也可以用一个式子表示，或用多个式子表示，从而结束了函数概念是否以唯一一个式子表示的争论，把对函数的认识又推进了一个新层次。

1837 年德国数学家狄利克雷(1805-1859) 认为怎样去建立 x 与 y 之间的关系无关紧要，他拓广了函数概念，指出：“对于在某区间上的每一个确定的 x 值，y 都有一个确定的值，那么 y 叫做 x 的函数。”这个定义避免了函数定义中对依赖关系的描述，以清晰的方式被所有数学家接受。这就是人们常说的经典函数定义。

等到康托尔创立的集合论在数学中占有重要地位之后，美国数学家维布伦(Veblen，1880－1960)用“集合”和“对应”的概念给出了近代函数定义，通过集合概念把函数的对应关系、定义域及值域进一步具体化了，且打破了“变量是数”的局限，其实变量可以是数，也可以是任何其它对象。

中国清代数学家李善兰（1811-1882）在 1859 年与英国传教士合译数学专著时，首次将“function”翻译为“函数”。

## 3.1 函数定义与性质

本节我们将给大家介绍函数的定义以及函数相关的一些基本性质，比如单调性、奇偶性、周期性等。

## 3.1.1 函数定义

一般地，给定非空数集A和B，按照某个对应法则f，使得A中**任一元素x**，都有B中**唯一确定**的元素y与之对应，那么从集合A到集合B的这个对应，叫做从集合A到集合B的一个函数。记作：

$$y=f(x), x\in A$$

其中x称为自变量，y称为因变量，集合A叫做函数的定义域，记为D，集合$\{y \mid y=f(x), x\in A\}$叫做函数的值域，记为C。**定义域，值域，对应法则**称为函数的三要素，是判断两个函数是否相等的标准。

函数一般书写为$y=f(x)$，$x\in D$。若省略函数的定义域，则定义域默认为使函数有意义的一切实数所组成的集合，比如分母不能为0，二次根号下的表达式需要大于等于0等。

以$h(t)=1/2gt^2$为例，这是物理学中计算自由落体运动的公式，它就是一个函数，其中t为时间，g为重力加速度，h为在t时刻的物体下落的位移，对每一个$t\geqslant 0$，都有唯一的一个$h\geqslant 0$与之相对应。在这里，t是自变量，h为因变量（注意，函数自变量和因变量取名不一定是x和y），因此它的定义域$D=\{t \mid t\geqslant 0\}$，值域$C=\{y \mid y=1/2gt^2, t\in D\}$。

学了函数的定义之后，我们现在来看这样一个问题。如图3-1所示，集合A和B的对应关系，谁是函数？

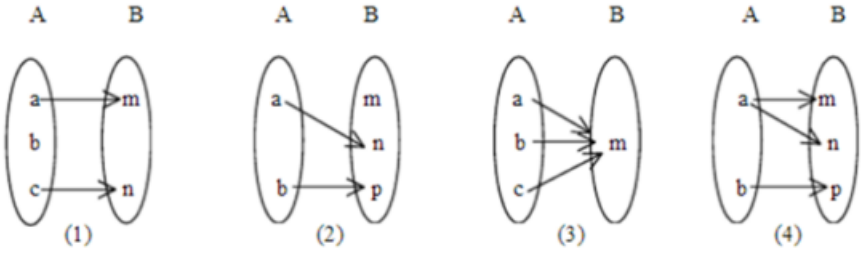



图3-1 集合A，B对应关系

在图3-1中，（1）里集合A中的元素b在集合B中没有元素与之对应，（4）里集合A中的元素a在B中存在多个元素与之相对应，都不符合函数的定义。(2)和(3)的集合A中的每一个元素，在集合B中都有唯一的一个元素与之相对应，符合函数的定义，所以是函数关系。

从图3-1中可以看出，集合A和集合B中的元素如果是一对一或多对一，那么就是函数； 如果是一对多或多对多，就不是函数。

在研究函数性质时候，我们常用到区间的概念，下面是一些常用的区间的表示方法：

- $a\leqslant x\leqslant b$ $[a, b]$ 闭区间
- $a<x<b$ $(a, b)$ 开区间
- $a\leqslant x<b$ $[a, b)$ 半开半闭区间
- $a<x\leqslant b$ $(a, b]$
- 实数R $(-\infty, +\infty)$
- $x>a$ $(a, +\infty)$
- $x\geqslant a$ $[a, +\infty)$
- $x<a$ $(-\infty, a)$
- $x\leqslant a$ $(-\infty, a]$

以a为中心的任意开区间，我们称为a的邻域，记为U(a)；以a为中心，$\delta$为半径的开

区间，我们称为 a 的 δ 邻域，记为 U(a, δ)；以 a 为中心，δ 为半径的开区间并且去掉 a 所在的点，我们称为 a 的 δ 去心邻域，记为 $\mathring{U}(a,\delta)$。根据邻域的定义有：

$$U(a,\delta)=\{x\mid a-\delta<x<a+\delta\}=\{x\mid|x-a|<\delta\}$$

$$\mathring{U}(a,\delta)=\{x\mid 0<|x-a|<\delta\}$$

邻域 U(a, δ)在数轴上可表示为图 3-2 所示：

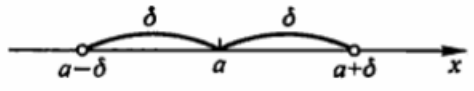



图 3-2 领域标示方法

我们在研究函数性质的时候，需要借助与函数的各种不同的表示方法。函数的表示方法主要有 3 种：

- **解析法**：这是最常见的表示方法，即用数学表达式来表达函数关系。比如：
  f(x)=x;
  f(x)=sinx;
  $f(x)=(x-1)^2$
- **图像法**：在坐标系中用图示的方法描述函数，函数关系通过图像一目了然，比如图 3-3 所示 f(x)=x 和 $g(x)=x^2$ 的函数图像：

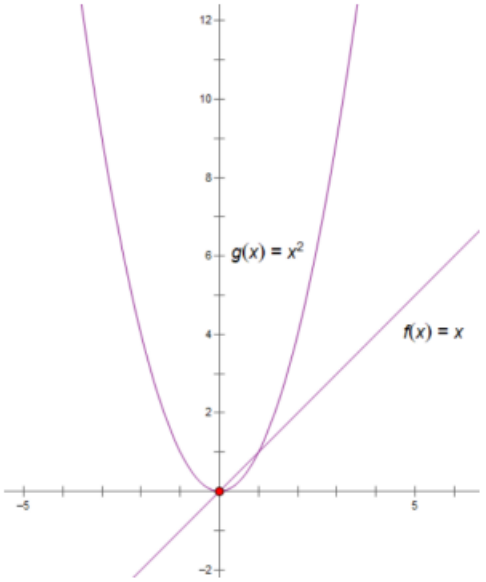



图 3-3 函数的图像表示方法

一般要求常用基本函数图像的形状要记住。在 3.2 节中，我们会给出这些基本函数的函数图像。

- **列表法**：一些函数如果没有解析法，用图像也不方便，那么可以用表格表示（比如一天 24 小时每隔一个小时的温度表），甚至是用对画角线的方法来表示（参考本书 2.2.3 节）。

## 3.1.3 函数单调性

通过研究函数在区间上的单调性可以看出函数在区间上函数值的增减变化趋势。现在我

们给出函数单调性相关的定义。

一般地，设函数 f(x)的定义域为 I，如果对于定义域 I 内某个区间 D 上的任意两个自变量值 $x_1$，$x_2$，当 $x_1<x_2$ 时，都有 $f(x_1)<f(x_2)$，那么就说函数 f(x)在区间 D 上是**增函数**；如果对于定义域 I 内某个区间 D 上的任意两个自变量值 $x_1$，$x_2$，当 $x_1<x_2$ 时，都有 $f(x_1)>f(x_2)$，那么就说函数 f(x)在区间 D 上是**减函数**；如果函数 y=f(x)在区间 D 上是增函数或减函数，那么就说函数 y=f(x)在这一区间具有严格的单调性，区间 D 叫做 y=f(x)的**单调（递增或递减）区间**。

现在我们来考查下面 2 个函数的单调性：

- f(x)=x
- $f(x)=x^2$

对于函数 f(x)=x，定义域为 I=(-∞, +∞)，令 $x_1<x_2$，且 $x_1,x_2\in I$，则有：

$$f(x_2)-f(x_1)=x_2-x_1>0,$$

所以，函数 f(x)=x 在定义域上是单调递增函数。

对于函数 $f(x)=x^2$，定义域为 I=(-∞, +∞)，令 $x_1<x_2$，且 $x_1,x_2\in I$，则：

$$f(x_2)-f(x_1)=x_2^2-x_1^2=(x_2+x_1)*(x_2-x_1)$$

因为 $x_2>x_1$，所以 $x_2-x_1>0$，所以当 $x_2, x_1\in(-\infty, 0]$时，$x_1+x_2<0$，$f(x_2)-f(x_1)<0$，当 $x_1$，$x_2\in[0, +\infty)$时，$x_1+x_2>0$, $f(x_2)-f(x_1)>0$。因此函数在(-∞, 0]单调递减；在[0, +∞)单调递增。

除了通过上面的方法判断函数的单调性，还可以根据函数图像直接识别它的单调区间，比如图 3-4 所示：

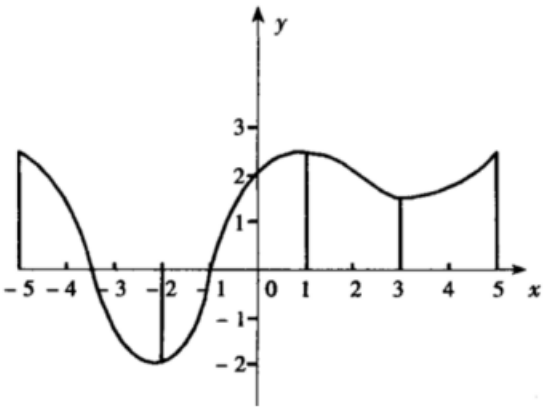



图 3-4 函数的单调性

函数 f(x)在区间[-5, -2]，[1, 3]上是单调递减的，因为随着自变量 x 的增加，因变量 y 会减小，因此在该区间函数图像呈下降图形；在区间[-2, 1]，[3, 5]上是单调递增的，因为随着自变量 x 的增加，因变量 y 也会增加，因此在该区间函数图像呈上升图形。

此外，对于函数定义域 I 中任意 x，如果都有 f(x) ≤Max，则称 Max 为最大值；如果都有 f(x) ≥Min，则称 Min 为最小值。最大值和最小值简称最值。

现在我们试着求函数 $f(x)=x^2$ 在[4, 8]之间的最值。

首先，在函数 $f(x)=x^2$ 定义域 I=(-∞, +∞)内，因为 $x^2\geq 0$，所以函数有最小值 Min=0，但无最大值。

然后通过 $f(x)=x^2$ 的图形很容易知道，函数在区间[4, 8]上是单调递增函数，所以函数在 x=4 处取得最小值 $Min=4^2=16$；在 x=8 处取得最大值 $Max=8^2=64$。

函数的最大值就是函数图像的最高点；最小值就是函数图像的最低点。比如在图 3-4 中，函数将在最低点 x=-2 处取得最小值，在最高点 x=1 处取得最大值。当然，函数也可能没有

最大值或者最小值。比如 f(x)=x，在定义域 I=(-∞, +∞)内就没有最大值和最小值，这从图 3-3 中 f(x)=x 的图像就可以看出。

### 3.1.4 函数奇偶性

函数的奇偶性是函数的一个重要特征。通过函数的奇偶性研究，我们可以很容易掌握函数的图像对称性，以及在高等数学中求微积分也可以通过函数的奇偶性来简化计算。下面我们给出函数奇偶性的定义。

对于定义域 I 中的任意 x，如果都有 f(-x)=-f(x)，则称函数为**奇函数**；对于定义域 I 中的任意 x，如果都有 f(-x)=f(x)，则称函数为**偶函数**。

从定义可以看出，奇函数和偶函数的定义域，都必须关于坐标原点对称，比如[-3,3], (-∞, +∞)等。如果不对称，比如[3,5],或者(-3,3]，如果取 x=5，或者 x=3，那么根据奇偶函数的定义，求 f(-x)时 f(-5)和 f(-3)就不在定义域内而没有意义了，那么无论是 f(-x)=f(x)或者 f(-x)=-f(x)就不成立了。

下面来考查两个函数的奇偶性：

- f(x)=x

因为 f(x)定义域为(-∞, +∞)，定义域关于原点对称，且 f(-x)=(-x)=-f(x)，所以是奇函数。

- f(x)=|x|

因为 f(x)定义域为(-∞, +∞)，定义域关于原点对称，且 f(-x)=|-x|=|x|=f(x)，所以是偶函数。

同理可证明 $f(x)=x^2$ 在定义域上也是偶函数。

在函数图像上，奇偶函数有明显的特征。如图 3-5 所示，奇函数的函数图像是关于原点对称的，而偶函数的图像是关于 Y 轴对称的。

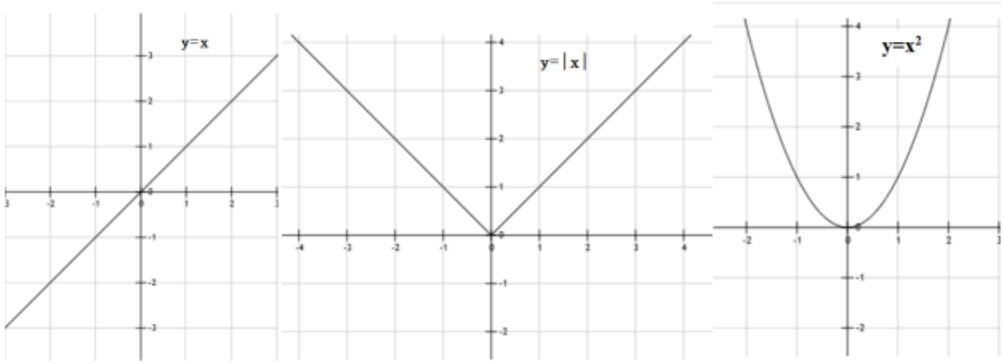



图 3-5 奇偶函数的对称性

### 3.1.5 函数周期性

对于有的函数，函数值会出现周期性的变化，也就是函数值会按照一定的规律重复出现，这就是函数的周期性。

对于函数 y=f（x），如果存在一个不为零的常数 T，使得当 x 取定义域内的每一个值时，f（x+T）=f（x）都成立，那么就把函数 y=f（x）叫做周期函数，不为零的常数 T 叫做这个函数的周期。事实上，如果 T 是函数的周期，那么任何一个常数 kT（k∈Z，且 k≠0）都是

它的周期。并且周期函数 f（x）的周期 T 是与 x 无关的非零常数，且周期函数不一定有最小正周期。

比如狄利克雷函数（Q 表示有理数集，$Q^c$ 表示无理数集）：

$$D(x)=\begin{cases}1, x\in Q\\ 0, x\in Q^c\end{cases}$$

任何正有理数 r 都是 D(x)的周期，即 D(x+r)=D(x)=1，但是因为没有最小正有理数，所以该函数没有最小正周期。

**狄利克雷简介**

狄利克雷函数是以德国数学家狄利克雷（1805-1859）命名的函数。狄利克雷是科隆大学博士。历任柏林大学和格廷根大学教授。柏林科学院院士。是解析数论的创始人。对函数论、位势论和三角级数论都有重要贡献。主要著作有《数论讲义》、《定积分》等。

一个最小正周期为 2π 的函数 y=sinx 图像如图 3-6 所示：

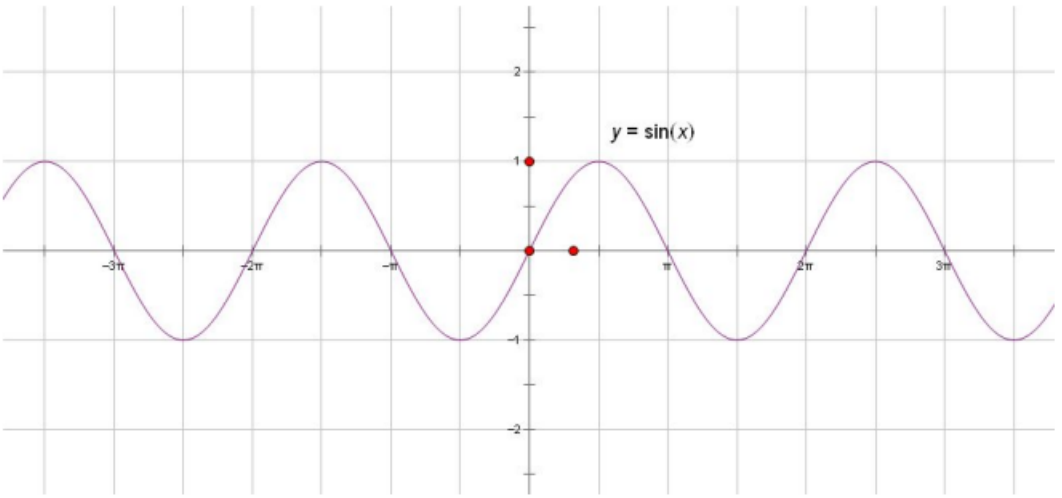



图 3-6 sinx 的函数图像

## 3.2 函数与反函数

函数是自变量 x 到因变量 y 的对应法则，反过来，如果因变量 y 到 x 也有相应的对应法则，即对于每一个 y，都有唯一的一个 x 与它对应，那么这个函数就存在反函数。

一般地，设函数 y=f(x)(x∈A)的值域是 C，根据这个函数中 x, y 的关系，用 y 把 x 表示出，得到 x= g(y). 若对于 y 在 C 中的任何一个值，通过 x= g(y)，x 在 A 中都有唯一的值和它对应，那么，x= g(y)就表示 y 是自变量，x 是因变量，x 是 y 的函数，由于大家一般习惯用 x 表示自变量，y 表示因变量，所以 x=g(y)(y∈C)可改写为 y=g(x)(x∈C)，并将之称为函数 y=f(x)(x∈A)的反函数，记作 $y=f^{-1}(x)$ ，并且它的定义域、值域分别是函数 y=f(x)的值域、定义域。

如图 3-7 所示，函数存在反函数的条件是函数的定义域与值域是一一对应，而多对一就不存在反函数了，因为反过来之后，就存在一个 y，有多个 x 与之相对应了。

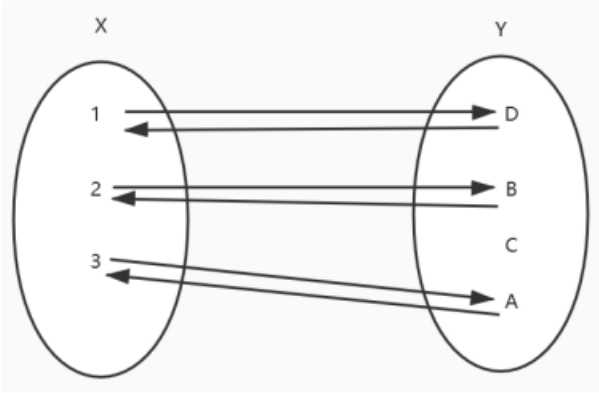



图 3-7 定义域与值域一一对应

现在我们来试着求如下两个函数的反函数。

- 求函数 y=3x-1 的反函数

由于 y=3x-1;那么 x=(y+1)/3，交换函数自变量与因变量为传统写法，所以反函数为：y=(x+1)/3

- 求函数 $y=x^2$, $x\in[0,+\infty)$的反函数

由于 $y=x^2$，那么 $x=\sqrt{y}$ ，交换函数自变量与因变量为传统写法，所以反函数为：

$$y=\sqrt{x}\ ,x\in[0,+\infty)$$

反函数的一些常见性质有：

- 原函数与反函数图象关于直线 y=x 对称，比如如图 3-8 所示，$y=2^x$ 与 $y=\log_2 x$ 互为反函数，那么它们的图像关于 y=x 对称：

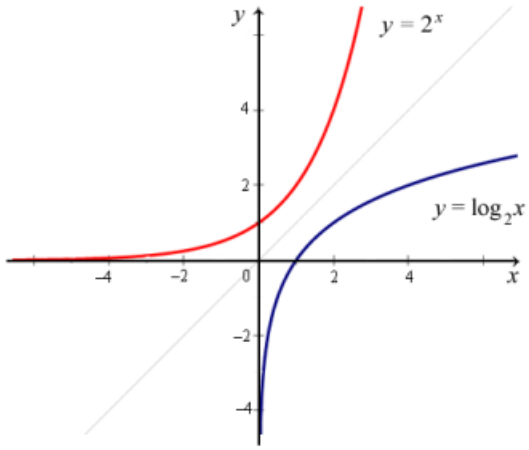



图 3-8 原函数与反函数对称性

- 一个函数与它的反函数在相应区间上单调性一致；严格增（减）的函数一定有严格增（减）的反函数。
- 绝大部分偶函数不存在反函数，因为偶函数大部分为多对一对应（由于偶函数关于 Y 轴对称，所以会存在 2 个以上的自变量对应同一个函数值，即多对一）。只有当函数 y=f(x)， 定义域是{0} 且 f(x)=C，C 为常数，则函数 f(x)是偶函数且有反函数，其反函数的定义域是{C}，值域为{0} 。奇函数不一定存在反函数，被与 y 轴垂直的直线截时能过 2 个及以上点即没有反函数（多对一了）。
- 若一个奇函数存在反函数，则它的反函数也是奇函数。

## 3.3 复合函数

如图 3-9 所示，设函数 u=g(x）的定义域为 X，值域为 Y1，函数 y=f(u)的定义域为 Y2，值域为 Z，Y1⊂Y2，对于 X 内的任意一个 x 经过 Y1 中的 u 有唯一确定的 Z 中的 y 值与之对应，则变量 x 与 y 之间通过变量 u 形成的一种函数关系，这种函数称为复合函数(composite function)，记为：y=f.g(x)=f[g(x)]，其中 x 称为自变量，u 为中间变量，y 为因变量。

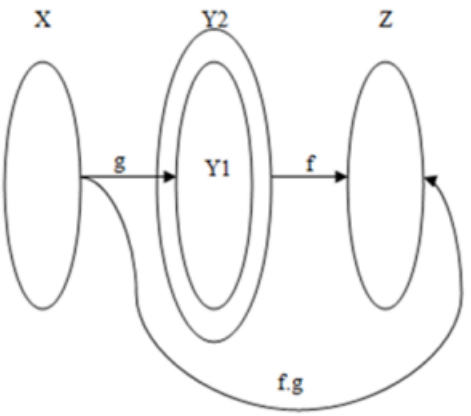



图 3-9 复合函数

比如 y=sin(1/x)就是 y=sinu 和 u=1/x 复合而成，而 $y=\sin(x^2)$就是 y=sinu 和 $u=x^2$ 复合而成。

## 3.4 基本初等函数

我们把**常数函数，幂函数，指数函数，对数函数，三角函数，反三角函数**这六类统称为基本初等函数。基本初等函数经过有限次的加、减、乘、除四则混合运算和函数复合构成的函数叫初等函数，而大部分的分段函数则是非初等函数。

我们首先给大家介绍基本初等函数。

### 3.4.1 常数函数

首先来看最简单的常数函数。

常数函数，顾名思义，就是指函数值不发生改变为常量的函数，即 f(x)=C，C 为常数。更一般地，对一个函数 f: A→B，如果对定义域 A 内所有的自变量 $x_1$ 和 $x_2$，都有 $f(x_1)=f(x_2)$，那么，f 是一个常数函数。

常数函数在坐标系里是一条平行于 X 轴或与 X 轴重合的直线。

- y=C (C≠0)， x∈R。图像如图 3-10 所示，平行于 X 轴：

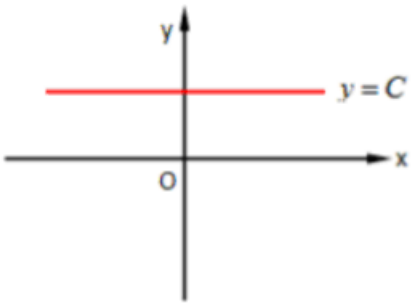



图 3-10 常数函数

- y=C (C=0)， x∈R。图像如图 3-11 所示，与 X 轴重合：

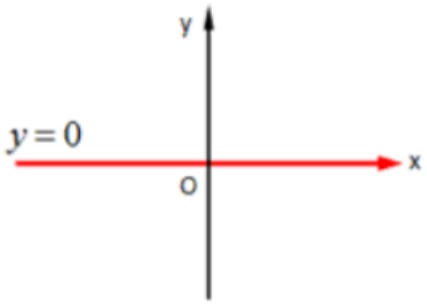



图 3-11 常数函数，C=0

## 3.4.2 指数函数

一般地，我们把 n 个相同的因数 a 相乘的积记为 $a^n$。这种求几个相同因数的积的运算叫做乘方，乘方的结果叫做幂。在 $a^n$ 中， a 叫做底数， n 叫做指数。$a^n$ 读作“a 的 n 次方”或“a 的 n 次幂”。

**1.指数运算**

首先，我们来看看指数运算相关的性质：

- 同底数幂相乘，底数不变，指数相加：

$$a^m.a^n = a^{m+n}$$ （m,n 都是有理数）

比如：$2^5.2^3=2^{5+3}=2^8$。

- 幂的乘方，底数不变，指数相乘：

$$(a^m)^n = a^{mn}$$ （m,n 都是有理数）

比如：$(2^3)^2=2^{3*2}=2^6$。

- 积的乘方，等于把积的每一个因式分别乘方，再把所得的幂相乘：

$$(ab)^n = a^n b^n$$

比如：$(2.3)^5=2^5.3^5$。

- 分式乘方，分子分母各自乘方：

$$\left(\frac{b}{a}\right)^n = \frac{b^n}{a^n}(a \neq 0)$$

比如：（2/3）$^5$=2$^5$/3$^5$。

- 同底数幂相除，底数不变，指数相减：

$$a^m \div a^n = a^{m-n}(a \neq 0, n \in N^*)$$

比如：2$^5$÷2$^3$=2$^{5-3}$=2$^2$。

- 任何不等于零的数的-p（p 是正整数）次幂，等于这个数的 p 次幂的倒数：

$$a^{-p} = \frac{1}{a^p}(a \neq 0, p \in N^*)$$

比如：2$^{-5}$=1/2$^5$

- 任何不等于零的数的零次幂都等于 1：

$$a^0 = 1(a \neq 0)$$

- 0 的正分数指数幂等于 0,0 的负分数指数幂没有意义，即：

**$a^0$=1(a≠0)，$0^a$=0(a>0)， $1^a$=1， $0^0$（为 1 或者无意义）**

- 指数部分为分数的运算规则

规则 1：$\mathbf{a^{\frac{m}{n}} = \sqrt[n]{a^m}(a > 0, m, n \in N^*, n > 1)}$

比如：$\mathbf{2^{\frac{2}{3}} = \sqrt[3]{2^2}}$

规则 2：$\mathbf{a^{-\frac{m}{n}} = \frac{1}{a^{\frac{m}{n}}} = \frac{1}{\sqrt[n]{a^m}}(a > 0, m, n \in N^*, n > 1)}$

比如：$\mathbf{2^{-\frac{2}{3}} = \frac{1}{2^{\frac{2}{3}}} = \frac{1}{\sqrt[3]{2^2}}}$

规则 3：$a^r a^s = a^{r+s}(a > 0, r, s \in Q)$

比如 $\mathbf{2^{\frac{1}{2}}.2^{\frac{2}{3}} = 2^{\frac{1}{2}+\frac{2}{3}} = 2^{\frac{7}{6}}}$

规则 4：$(a^r)^s = a^{rs}(a > 0, r, s \in Q)$

比如：$\mathbf{(2^{\frac{1}{2}})^{\frac{1}{3}} = 2^{\frac{1}{2}\cdot\frac{1}{3}} = 2^{\frac{1}{6}}}$

规则 5：$(ab)^r = a^r b^r(a > 0, b > 0, r \in Q)$

比如：$\mathbf{(2.3)^{\frac{1}{2}} = 2^{\frac{1}{2}} 3^{\frac{1}{2}}}$

一般地，无理数指数幂 $a^{\alpha}$($a>0$, $\alpha$ 是无理数）是一个确定的实数。有理数指数幂的运算性质同样适用于无理数的指数幂。

**2. n 次方根**

- 一般地，如果 $x^n=a$，那么 x 叫做 a 的 n 次方根，其中 $n>1$，且 $n\in N^*$。
- 当 n 是奇数时，正数的 n 次方根是一个正数，负数的 n 次方根是一个负数。这时，a 的 n 次方根用符号 $\sqrt[n]{a}$ 表示。当 n 是偶数时，正数的 n 次方根有两个，这两个数互为相反数，正的用 $\sqrt[n]{a}$ 表示，负的用 $-\sqrt[n]{a}$ 表示。负数没有偶次方根。
- 0 的任何次方根都是 0

$$\sqrt[n]{0}=0$$

- $(\sqrt[n]{a})^n=a$
- 当 n 为奇数时，$\sqrt[n]{a^n}=a$；当 n 为偶数时，$\sqrt[n]{a^n}=|a|$

**3. 指数函数**

一般地，我们把函数 $y=a^x(a>0,a\neq 1)$叫做指数函数（exponential function），其中 x 是自变量，定义域为 R。根据底数 a 的取值不同（即 $a>1$ 和 $0<a<1$ 两种情况下），指数函数的性质也不同。现在，我们来学习一下指数函数的性质，包括它的定义域，值域，单调性和图像等。

- $y=a^x$ ($a>1$)

此时，指数函数的定义域：$x\in R$；值域：$(0,+\infty)$；共点性：过点(0,1)；单调性：在 R 上是增函数；函数图像如图 3-12 所示：

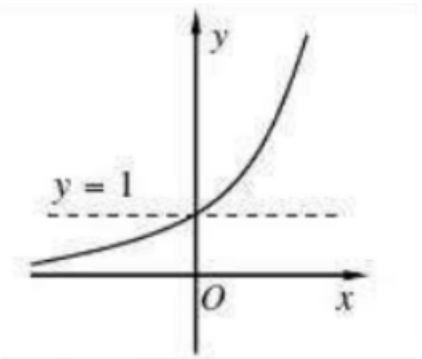



图 3-12 指数函数图像($a>1$)

- $y=a^x(0<a<1)$

此时，指数函数的定义域：$x\in R$；值域：$(0,+\infty)$；共点性：过点(0,1)；单调性：在 R 上是减函数；函数图像如图 3-13 所示：

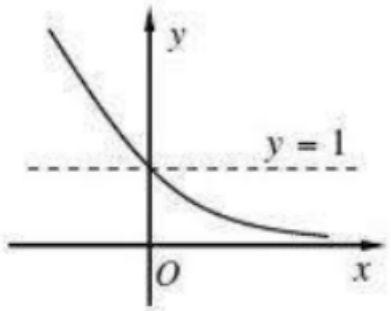



图 3-13 指数函数图像（0<a<1)

由于 $y=a^x(a>1)$在定义域内是增函数，而 $y=a^x(0<a<1)$在定义域内是减函数，所以当 a 取 0.99 和 1.01 的时候，下面两个值的计算结果会体现出巨大的差异：

$$(0.99)^{365}=0.025517964452291210028750334589 96$$

$$(1.01)^{365}=37.783434332887158877616604796498$$

这也告诉我们，同样是一年 365 天，多努力一点（0.01）和少努力一点（0.01），其结果区别非常大。

## 3.4.3 对数函数

一般地，如果 $a^x=N(a>0,且\ a\neq1)$，那么 x 叫做以 a 为底 N 的对数（logarithm），记为：

$$x=\log_a N$$

其中 a 叫做对数的底数，N 叫做真数。以 10 为底的对数叫做常用对数，并把 $\log_{10}N$ 记为 lgN。在科学技术中常使用无理数 e=2.71828...为底数的对数，以 e 为底的对数称为自然对数，并把 $\log_e N$ 记为 lnN。对数运算的一些常用性质包括：

- 如果 $a^x=N$，那么 $x=\log_a N$；如果 $x=\log_a N$，那么 $a^x=N$（a>0, 且 a≠1 ）
- 负数和零没有对数（对数函数与指数函数互为反函数，指数函数的值域中的值都是>0，那么对数函数的定义域中的值也是>0 的）
- $\log_a 1=0$, $\log_a a=1$（a>0, 且 a≠1 ）

**1.对数运算**

现在来看看对数的运算规则，如果 a>0，且 a≠1，M>0，N>0，那么对数有如下运算规则：

- 乘法规则：$log_a(M.N) = log_aM + log_aN$

比如：$\log_2(3.7)=\log_2 3+\log_2 7$

- 除法规则：$log_a \frac{M}{N} = log_aM - log_aN$

比如：$\log_2(3/7)=\log_2 3-\log_2 7$

- 幂次规则：$log_aM^n = nlog_aM(n \in R)$

比如：$\log_2 3^7=7\log_2 3$

- 换底公式：$log_a b = \frac{log_c b}{log_c a}(a>0, a\neq 1; c>0, c\neq 1; b>0)$

比如：$\log_2 3=\log_e 3/\log_e 7=\ln 3/\ln 7$

**有关对数发明小故事**

对数发明之前，人们对三角运算中将三角函数的积化为三角函数的和或差的方法已很熟悉，而且德国数学家斯蒂弗尔（M.Stifel，约 1487-1567）在《综合算术》（1544 年）中阐述了如下所示的一种对应关系：

$$[r^0, r^1, r^2, r^3, r^4] \rightarrow [0, 1, 2, 3, 4]$$

该关系可被归纳为$(r^n \rightarrow n)$。

对数是由苏格兰数学家纳皮尔（J.Napier,1550-1617）发明的，对数的发明改进并简化了天文、航海、工程、贸易中的数字计算，大大减少了计算量，誉为"延长了天文学家寿命的发现"。恩格斯曾经把对数的发明和解析几何，微积分并列为 17 世纪数学的三大成就，伽利略也说过："给我空间、时间及对数，我就可以创造一个宇宙"。纳皮尔在 1614 年出版的《奇妙的对数定律说明书》中，借助了运动学和几何术语阐述了对数方法。

将对数加以改造使之广泛流传的是纳皮尔的朋友布里格斯（H.Briggs，1561-1631），他通过研究《奇妙的对数定律说明书》，感到其中的对数用起来很不方便，于是与纳皮尔商定，使 1 的对数为 0，10 的对数为 1，这样就得到了以 10 为底的常用对数。由于我们的数系是十进制，因此它在数值上计算具有优越性。1624 年，布里格斯出版了《对数算术》，公布了以 10 为底包含 1~20000 及 90000~100000 的 14 位常用对数表。

纳皮尔在讨论对数概念时，并没有使用指数与对数的互逆关系，因为当时还没有明确的指数概念，指数符号也是 20 多年后 1637 年由法国数学家笛卡尔开始使用，直到 18 世纪，才由瑞士数学家欧拉发现了指数与对数的互逆关系，并首先使用 $y=a^x$ 来定义对数。对数的发明先于指数，成为数学史上的珍闻（这就好比汽车先于自行车发明一样）。

纳皮尔画了两条线段，设 AB 是一条定线段（长度为 $10^7$），CD 是给定的射线，令点 P 从 A 出发，沿 AB 变速运动，速度跟它与 B 的距离成比例地递减。同时，令点 Q 从 C 出发，沿 CD 作匀速运动，速度等于 P 出发时的值，纳皮尔发现此时 P、Q 运动距离有种对应关系，他就把可变动的距离 CQ 称为距离 PB 的对数，如图 3-14 所示：

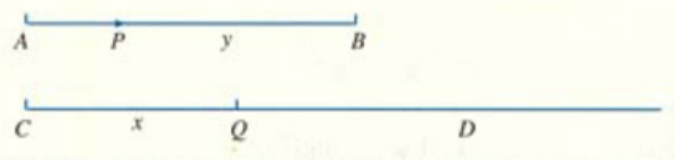



图 3-14 P、Q 运动距离对应关系

设 CQ 的长度为 x，PB 的长度为 y，那么纳皮尔发现 x 和 y 之间有如下关系：

$$y = 10^7\left(\frac{1}{e}\right)^{\frac{x}{10^7}}$$

$$x = 10^7 log_{\frac{1}{e}}\frac{y}{10^7}$$

此时，x 和 y 之间就构成了上述等式的对数关系。

**2.对数函数**

一般地，我们把函数 $y=\log_a x(a>0,且\ a\neq 1)$叫做对数函数（logarithmic function），其中 x 是自变量，定义域为$(0,+\infty)$。根据底数 a 的取值不同，对数函数的性质也不同。现在我们来分别总结一下对数函数的性质：

- $y=\log_a x(a>1)$

当 a>1 时，对数函数的定义域：$(0,+\infty)$；值域：R；单调性：增函数；共点性：过(1,0)点；函数图像如图 3-15 所示：

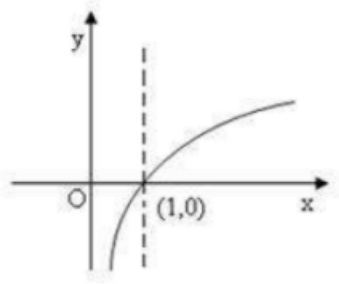



图 3-15 对数函数图像（a>1）

- $y=\log_a x(0<a<1)$

当 0<a<1 时，对数函数的定义域：$(0,+\infty)$；值域：R；单调性：减函数；共点性：过(1,0)点；函数图像如图 3-16 所示：

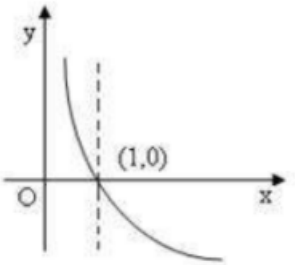



图 3-16 对数函数图像(0<a<1)

实际上，对数函数与指数函数互为反函数。

## 3.4.4 幂函数

一般地，函数 $y=x^a$ 叫做幂函数（power function），其中 x 是自变量，a 是常数。一些常见的幂函数：$y=x$, $y=x^2$, $y=x^3$, $y=x^{1/2}$, $y=x^{-1}$ 等等。它们的函数图像如图 3-17 所示：

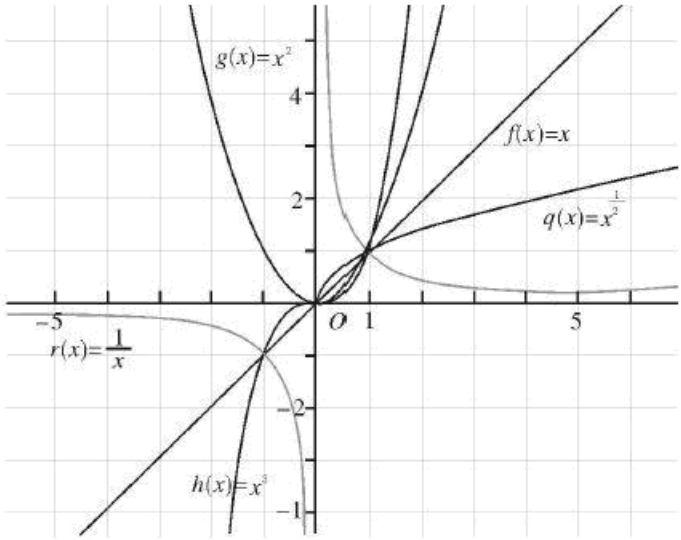



图 3-17 幂函数图像

幂函数的一些常见的性质如表 3-1 所示：

| | $y=x$ | $y=x^2$ | $y=x^3$ | $y=x^{1/2}$ | $y=x^{-1}$ |
|---|---|---|---|---|---|
| 定义域 | R | R | R | $[0,+\infty)$ | $x\neq 0$ |
| 值域 | R | $[0,+\infty)$ | R | $[0,+\infty)$ | $y\neq 0$ |
| 奇偶性 | 奇 | 偶 | 奇 | ✗ | 奇 |
| 单调性 | 增 | $(-\infty,0]$减<br>$[0,+\infty)$增 | 增 | 增 | $(-\infty,0)$减<br>$(0,+\infty)$减 |
| 定点 | (1,1) | | | | |

表 3-1 常用幂函数常见性质

幂函数和指数函数，可以看成是由式子 $a^b$ $(a>0, a\neq 1)$而来，比如：

- b 为自变量，就是指数函数：$y=a^x$；
- a 为自变量，就是幂函数：$y=x^b$。

## 3.4.5 三角函数

三角函数是非常重要的基本初等函数，无论是在几何学，物理学，天文学还是在计算机科学领域都有着非常重要的应用。

**1. 有关三角学历史**

古埃及人已有三角学知识，三角法主要是适应测量上的需要而产生的。例如，建筑金字塔，整理尼罗河泛滥后的耕地，以及通商航海，观测天象的需要。希腊的自然哲学家泰勒斯的相似理论，可以认为是三角学的萌芽，但历史上都认为希腊的天文学家喜帕恰斯是三角学的创始者。他著有三角学 12 卷，并作成弦表。古希腊门纳劳斯(Menelaus of Alexandria，公元 100 年左右)著《球面学》，提出了三角学的基础问题和基本概念，特别是提出了球面三角学的门纳劳斯定理;50 年后，另一个古希腊学者，“地心说”集大成者托勒密(Ptolemy)著《天文学大成》，初步发展了三角学。

印度人从天文、测量的角度，曾研究过三角学，在公元 6 世纪，有阿耶波多也曾作出正弦表。

德国的 J.雷格蒙塔努斯曾研究过天文学与三角学。在他的《论三角》一书中，有仿印度

人的正弦表作成的非常精密的正、余弦表。他对天文、航海、测量方面都有很大的贡献。

16 世纪法国著名数学家 F.韦达的《应用于三角形的数学法则》，是他对三角法研究的第一本书，其中包括他对解直角三角形、斜三角形的一些贡献，例如有正切定理。17 世纪法国数学家棣莫弗也研究过三角问题。他曾发现有名的棣莫弗定理。从 17 世纪后半期到 18 世纪，牛顿和伯努利曾发现各种三角级数。

近代三角学是从欧拉的《无穷分析引论》开始的。他定义了单位圆，并以函数线与半径的比值定义三角函数，他还创用小写拉丁字母 a、b、c 表示三角形三条边，大写拉丁字母 A、B、C 表示三角形三个角，从而简化了三角公式，使三角学从研究三角形解法进一步转化为研究三角函数及其应用，成为一个比较完整的数学分支学科。

而由于上述诸人及 19 世纪许多数学家的努力，形成了现代的三角函数符号和三角学的完整的理论。

**2. 角度制与弧度制**

在学习三角函数前，我们需要先学习一下什么叫做角度。如图 3-18 所示，在平面内一条射线绕着端点 O 从一个位置旋转到另一个位置形成了一个角 α，如果按逆时针方向旋转形成的角叫做**正角**，按顺时针方向旋转形成的角叫做**负角**。如果一条射线没有作任何旋转，便形成了一个**零角**。**任意角包括正角、负角和零角**。

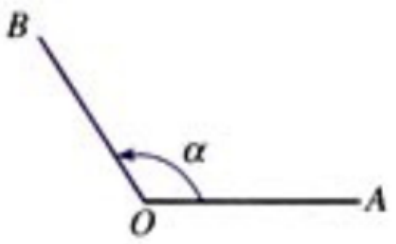



图 3-18 角 α

在直角坐标系中，使角的顶点与原点重合，角的始边与 x 轴的正半轴重合，那么角的终边在第几象限，这个角就是第几**象限角**。

一般地，所有与角 α 终边相同的角，连同角 α 在内，可构成一个集合：S={β | β = α +k*360°，k∈Z}，即任一与角 α 终边相同的角，都可以表示成角 α 与整数个**周角（360°）**的和。

用度（符号记为“°”）作为单位来度量角的单位制叫角度制。1 度(1°)的角等于周角的 1/360。比度小的单位是分和秒。1 度等于 60 分(60')，1 分等于 60 秒(60")。

另一种度量角的单位制叫弧度制（弧度制最早由欧拉明确提出）。如图 3-19 所示，如果半径为 r 的圆的圆心角 α 所对弧的长为 l：

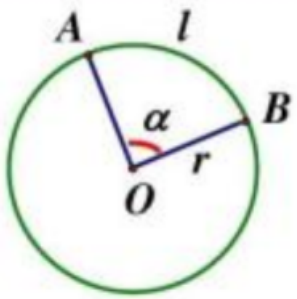



图 3-19 弧度制

那么角 α 的弧度数定义为：

$$\alpha = \frac{l}{r}$$

角 α 的弧度数绝对值是：| α |=l/r。如图 3-20 所示，我们把长度等于半径长的弧（即 l=r）所对的圆心角叫做 1 弧度。因为根据弧度的定义， α =l/r=r/r=1(rad)。

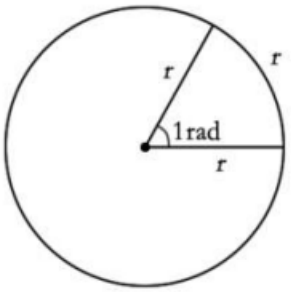



图 3-20 弧度制中 1 rad 的定义

一般地，正角的弧度数是一个正数。负角的弧度数是一个负数。零角的弧度数是 0。

现在我们来试着求一下 90° 的弧度值。

设圆的半径为 r，那么圆心角 90° 对应的弧长为：2πr/360*90，那么根据弧度的定义有：

α =l/r=2πr/360*90/r=π/2

一些常见角度制与弧度制切换关系如下：

- 90° =π/2
- 180° =π
- 360° =2π
- 1° =π/180≈0.01745rad
- 1rad=（180/π）°

根据弧度的定义，设扇形的半径为 R，扇形对应圆心角的弧度值为 α，那么使用弧度计算的扇形弧长 l 公式：

**l= α R**

扇形面积 S 的公式：

**S=1/2lR=1/2 α $R^2$**

三角学以研究平面三角形和球面三角形的**边和角的关系**为基础，达到测量上的应用为目的的一门学科。同时还研究三角函数的性质以及它的应用。

**3. 正弦、余弦、正切与余切**

如图 3-21 所示，设直角三角形中的直角边和斜边分别为 a，b，c，对应的角度为 α，那么正弦，余弦，正切的定义为：

- 正弦：sin α =b/c
- 余弦：cos α =a/c
- 正切：tan α =b/a
- 余切：cot α =a/b

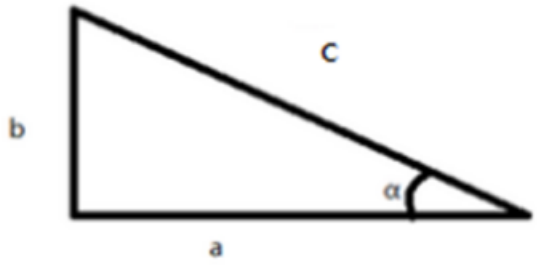



图 3-21 直角三角形三条边与正弦、余弦、正切和余切定义

- $\tan\alpha = \sin\alpha / \cos\alpha$

根据定义和勾股定理，很容易证明等式：$\sin^2 a + \cos^2 a = 1$。根据定义计算出的一些常见角度的正弦，余弦，正切，余切值如下：

- $\sin0^\circ = 0$　　$\cos0^\circ = 1$
- $\sin30^\circ = 1/2$　　$\cos30^\circ = \sqrt{3}/2$
- $\sin60^\circ = \sqrt{3}/2$　　$\cos60^\circ = 1/2$
- $\sin45^\circ = \sqrt{2}/2$　　$\cos45^\circ = \sqrt{}2/2$
- $\sin90^\circ = 1$　　$\cos90^\circ = 0$
- $\tan0^\circ = 0$
- $\tan30^\circ = \sqrt{3}/3$
- $\tan60^\circ = \sqrt{3}$
- $\tan45^\circ = 1$
- $\tan90^\circ$ 无意义

一些常用的三角公式如下：

- 诱导公式：

$$\sin(\alpha + k\cdot 2\pi) = \sin\alpha,$$
$$\cos(\alpha + k\cdot 2\pi) = \cos\alpha,$$
$$\tan(\alpha + k\cdot 2\pi) = \tan\alpha,$$
其中 $k \in \mathbf{Z}$.

$$\sin(\pi + \alpha) = -\sin\alpha,$$
$$\cos(\pi + \alpha) = -\cos\alpha,$$
$$\tan(\pi + \alpha) = \tan\alpha.$$

$$\sin(-\alpha) = -\sin\alpha,$$
$$\cos(-\alpha) = \cos\alpha,$$
$$\tan(-\alpha) = -\tan\alpha.$$

$$\sin(\pi - \alpha) = \sin\alpha,$$
$$\cos(\pi - \alpha) = -\cos\alpha,$$
$$\tan(\pi - \alpha) = -\tan\alpha.$$

$$\sin\left(\frac{\pi}{2} - \alpha\right) = \cos\alpha,$$
$$\cos\left(\frac{\pi}{2} - \alpha\right) = \sin\alpha.$$

$$\sin\left(\frac{\pi}{2} + \alpha\right) = \cos\alpha,$$
$$\cos\left(\frac{\pi}{2} + \alpha\right) = -\sin\alpha.$$

- 角度和差公式：

sin(a+b)=sinacosb+cosasinb
sin(a-b)=sinacosb-sinbcosa
cos(a+b)=cosacosb-sinasinb
cos(a-b)=cosacosb+sinasinb

- 倍角公式：(角度为 2x 与 x，x 与 x/2 即成 2 倍关系时，正弦余弦值之间的关系)

$\sin2x=2\sin x\cos x$

$\cos2x=\cos^2x-\sin^2x=2\cos^2x-1=1-2\sin^2x$

$\sin^2x=(1-\cos2x)/2$

$\cos^2x=(1+\cos2x)/2$

- 和差化积：

$\sin x+\sin y=2\sin[(x+y)/2]\cos[(x-y)/2]$

$\sin x-\sin y=2\cos[(x+y)/2]\sin[(x-y)/2]$

$\cos x+\cos y=2\cos[(x+y)/2]\cos[(x-y)/2]$

$\cos x-\cos y=-2\sin[(x+y)/2]\sin[(x-y)/2]$

- 积化和差：

$\sin x\sin y=-1/2[\cos(x+y)-\cos(x-y)]$

$\cos x\cos y=1/2[\cos(x+y)+\cos(x-y)]$

$\sin x\cos y=1/2[\sin(x+y)+\sin(x-y)]$

$\cos x\sin y=1/2[\sin(x+y)-\sin(x-y)]$

只要不是为了参加各种考试，那么这些三角公式都不用死记硬背，需要用到的时候，直接查表即可。

**4. 常用三角函数**

将正弦，余弦，正切，余切中的角度当做自变量，就构成了三角正弦函数 y=sinx，三角余弦函数 y=cosx，三角正切函数 y=tanx 和三角余切函数 y=cotx。下面我们来研究下这些函数的性质。

- **正弦函数 y=sinx**

定义域：R

值域：[-1，1]

单调性：$[-\pi/2+2k\pi, \pi/2+2k\pi]$单调递增，在$[\pi/2+2k\pi, 3\pi/2+2k\pi]$单调递减，$k\in Z$

周期性：周期函数，$2k\pi$（$k\in Z$，且$k\neq0$）都是它的周期，最小正周期是 $2\pi$

奇偶性：奇函数，即 sin(-x)=-sinx

函数图像如图 3-22 所示：

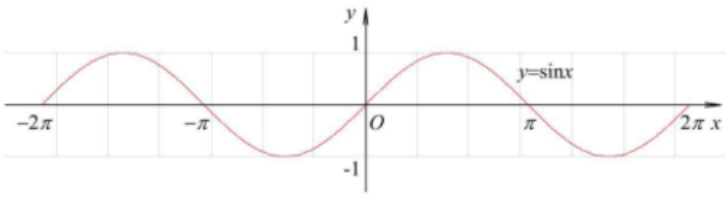



图 3-22 正弦函数图像

- **余弦函数 y=cosx**

定义域：R

值域：[-1，1]

单调性：$[-\pi+2k\pi, 2k\pi]$单调递增，在$[2k\pi, \pi+2k\pi]$单调递减，$k\in Z$

周期性：是周期函数，$2k\pi$（$k\in Z$，且$k\neq0$）都是它的周期，最小正周期是 $2\pi$

奇偶性：偶函数，即 cos(-x)=cosx

函数图像如图 3-23 所示：

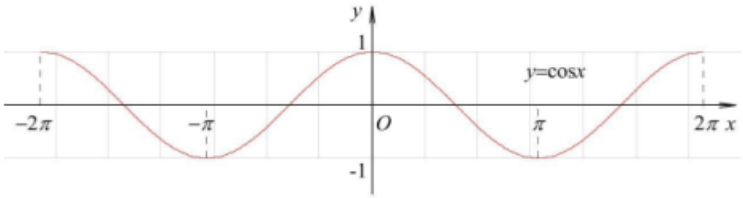



图 3-23 余弦函数图像

- **正切函数 y=tanx**

定义域：(-π/2+kπ，π/2+kπ)，k∈Z

值域：(-∞，+∞)

单调性：(-π/2+kπ，π/2+kπ)单调递增，k∈Z

周期性：周期函数，kπ（k∈Z，且k≠0）都是它的周期，最小正周期是π

奇偶性：奇函数，即 tan(-x)=-tanx

函数图像如图 3-24 所示：

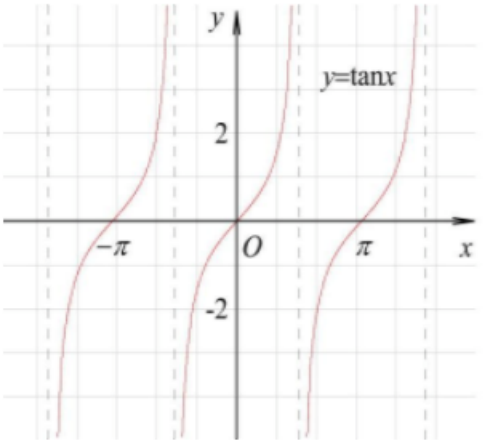



图 3-24 正切函数图像

- **余切函数 y=cotx**

定义域：(+kπ，π+kπ)，k∈Z

值域：(-∞，+∞)

单调性：(kπ，π+kπ)单调递减，k∈Z

周期性：是周期函数，kπ（k∈Z，且k≠0）都是它的周期，最小正周期是π

奇偶性：奇函数，即 cotx(-x)=-cotx

函数图像如图 3-25 所示：

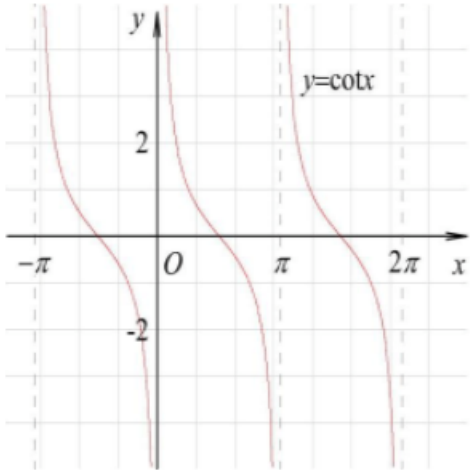



图 3-25 余切函数图像

对于三角函数 y=Asin(ωx+φ)（其中 A>0，ω>0）的图像，如图 3-26 所示，可以看作用下面的方法得到：先画出函数 y=sinx 的图像；再把正弦曲线向左平移|φ|个单位长度，得到函数 y=sin(x+φ)的图像（x+φ为向左，x-φ为向右）；然后使曲线上各点的横坐标变为原来的 1/ω倍，得到函数 y=sin(ωx+φ)的图像；最后把曲线上各点的纵坐标变为原来的 A 倍，这时的曲线就是函数 y=Asin(ωx+φ)的图像。

比如函数：y=3sin(2x+π/6)，就是把 y=sinx 向左移动π/6 个单位长度，再将横坐标变为原来的 1/2 倍，最后把曲线上各点的纵坐标变为原来的 3 倍而来。

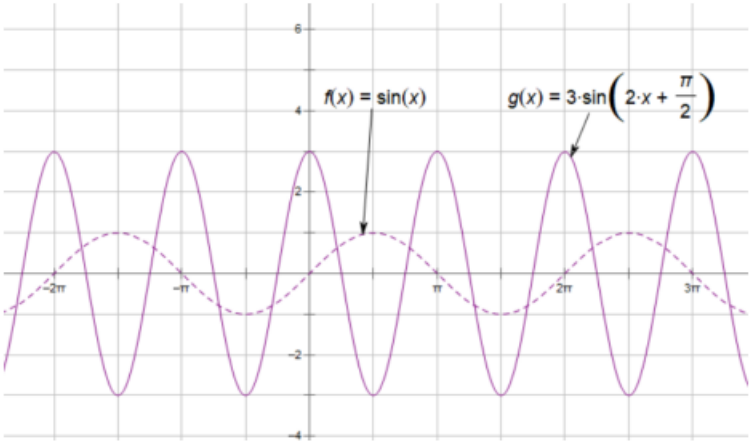



图 3-26 三角函数 y=Asin(ωx+φ)（其中 A>0,ω>0）的图像变换

y=Asin(ωx+φ)，x∈[0,+∞)，其中 A>0，ω>0，是物理中描述简谐运动的物理量，A 是振幅（离开平衡位置的最大距离），周期 T=2π/ω，频率 f=1/T=ω/2π，ωx+φ称为相位，φ为初相。

## 3.4.6 反三角函数

反三角函数是三角函数的反函数，即根据三角函数值域中的实数值，求出对应的角度值。因为三角函数具有周期性，所以在整个定义域上，是多对一的函数关系，所以在三角函数的整个定义域是没有反函数的。为限制反三角函数为单值函数，将反正弦函数的值 y（原三角函数的自变量 x）限在-π/2≤y≤π/2，记为 y=arcsin x；反余弦函数的值 y（原三角函数的

自变量 x）限在 0≤y≤π，记为 y=arccosx；反正切函数的值 y（原三角函数的自变量 x）限在-π/2<y<π/2，记为 y=arctan x；反余切函数的值 y（原三角函数的自变量 x）限在 0<y<π，记为 y=arccotx。

**1. 反正弦函数：y=arcsinx**

- 定义域：[-1, 1]
- 值域：[- π/2， π/2]
- 单调性：定义域内单调递增
- 函数图像如图 3-27 所示：

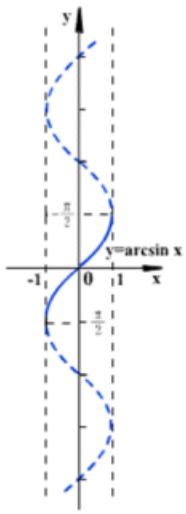



图 3-27 反正弦函数图像

**2. 反余弦函数: y=arccosx**

- 定义域：[-1, 1]
- 值域：[- π， 0]
- 单调性：定义域内单调递减
- 函数图像如图 3-28 所示：

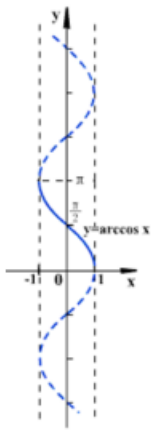
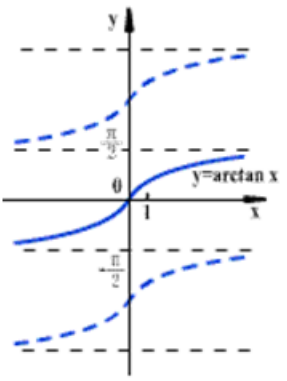



图 3-28 反余弦函数

**3. 反正切函数:y=arctanx**

- 定义域：(-∞, +∞)

- 值域：(- π/2， π/2)
- 单调性：定义域内单调递增
- 函数图像如图 3-29 所示：




1

图 3-29 反正切函数

**4.反余切函数:y=arccotx**

- 定义域：(-∞, +∞)
- 值域：(π， 0)
- 单调性：定义域内单调递减
- 函数图像如图 3-30 所示：

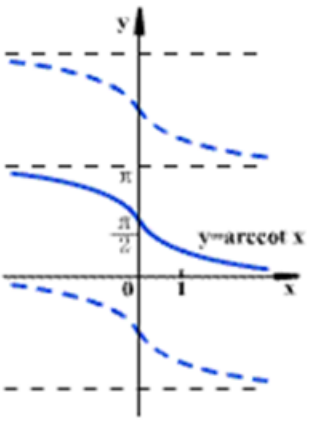



图 3-30 反余切函数

一些常见的反三角函数的值如下：

- arcsin1= π /2
- arcsin$\sqrt{2}$ /2= π /4
- arcsin1/2= π /6
- arcsin$\sqrt{3}$ /2= π /3
- arcsin0 = 0
- arccos1=0
- arccos$\sqrt{2}$ /2= π /4
- arccos1/2= π /3
- arccos$\sqrt{3}$ /2= π /6

- arccos0 = π /2
- arctan1 = π /4
- arctan$\sqrt{3}$ = π /3
- arctan$\sqrt{3}$ /3= π /6
- arccot1 = π /4
- arccot$\sqrt{3}$ = π /6
- arccot$\sqrt{3}$ /3= π /3

以上这些函数，包括常数函数，指数函数，对数函数，幂函数，三角函数，反三角函数统称为基本初等函数。由基本初等函数经过有限次的加减乘除四则运算与函数复合所得到的函数就是初等函数。

一个初等函数，除了可以用初等解析式表示以外，往往还有其他表示形式，比如三角函数 y=sinx 可以用无穷级数表示。初等函数是最先被研究的一类函数，它与人类的生产和生活密切相关，并且应用广泛。人们还编制了各种函数表，如平方表、开方表、对数表、三角函数表等。

学习这些初等函数，实际上是为以后进一步学习微积分打下基础。正如 18 世纪伟大的数学家欧拉曾经指出，微积分研究的主要对象就是函数。

## 3.5 函数与解方程

方程是含有未知数(unknown)的等式。它把问题中的未知数与已知数的联系用等式形式表示出来。在研究问题时，用字母 x, y, z 等表示未知数，列出方程，然后求出未知数。

### 3.5.1 一元一次与一元二次方程

一元一次方程与一元二次方程是最基础和最简单的方程形式，也是我们最初接触的解方程形式。现在我们来逐一介绍一元一次和一元二次方程的解法。

**1.一元一次方程**

只含有一个未知数（unknown，即“元”）x，未知数 x 的指数都是 1（次），这样的方程叫做一元一次方程。

解一元一次方程的方法是：移项，合并，系数化为 1。

丢番图(Diophantus，约公元 246-330 年)是古希腊亚历山大学后期的重要学者和数学家。丢番图是代数学的创始人之一，对算术理论有深入研究，他完全脱离了几何形式，在希腊数学中独树一帜。丢番图最著名的就是在他墓志铭上的丢番图方程：

坟中安葬着丢番图，多么令人惊讶，他忠实地记录了所经历的道路。上帝给予的童年占六分之一，又过了十二分之一，两颊长胡，再过七分之一，点燃结婚的蜡烛，五年之后天赐贵子，可怜迟到的宁馨儿，享年仅及其父之半，便进入坟墓，悲伤只有用数论研究去弥补，又过四年，他也走完了人生的旅途。问他多少岁？

解：设丢番图的年龄为x岁，那么根据题意有：

$$x/6+x/12+x/7+5+x/2+4=x$$

移项（将未知数移到等式一边，常数移到等式另一边）：

$$x-x/6-x/12-x/7-x/2=9$$

通分，合并有：

$$84x/84-14x/84-7x/84-12x/84-42x/84=9$$

即：

$$9x/84=9$$

将x的系数化为1：

$$x=84$$

所以丢番图活了84岁，这在古代已经是非常高龄了。

**2.一元二次方程**

一般地，只含有一个未知数（一元），并且未知数的最高次数是2（二次）的方程，叫做一元二次方程（quadratic equation in one unknown）。任何一个关于x的一元二次方程，经过整理，都能化成这样的形式：$ax^2+bx+c=0(a\neq0)$。

一元二次方程的解法有：公式法，配方法，和因式分解法等。

**公式法**：当$\Delta=b^2-4ac\geqslant0$时，方程的求根公式为：

$$x_1=\frac{-b+\sqrt{b^2-4ac}}{2a},x_2=\frac{-b-\sqrt{b^2-4ac}}{2a}$$

**配方法**：主要是利用了$a^2-2ab+b^2=(a-b)^2$和$a^2+2ab+b^2=(a+b)^2$这两个平方和公式。比如：

对于方程：

$$x^2-8x-9=0$$

配方有：

$$x^2-2*4*x+4^2-25=0$$

即：

$$(x-4)^2=25$$

$$x-4=5 \text{ 或者 } x-4=-5$$

因此：

$$x_1=9 \text{ 或者 } x_2=-1$$

**因式分解法**：又叫十字交叉法，具体做法为：对于$mx^2+px+q$形式的多项式，如果$a\times b=m$, $c\times d=q$且$ad+bc=p$，则多项式可因式分解为$(ax+c)(bx+d)$。比如：

对于方程：

$$3x^2-x-4=0$$

因为：

$$m=3=1(a)\times3(b),\ q=-4=1(c)\times-4(d),\ ad+bc=1\times-4+3\times1=-1(p)$$

所以有：

$$(x+1)(3x-4)=0,\ \text{所以有 } x_1=-1,x_2=4/3$$

配方法需要先配方，再降次；通过配方法可以推出求根公式，公式法直接利用求根公式；因式分解法要先使方程一边为两个一次因式相乘，另一边为0，再分别使各一次因式等于0。配方法，公式法适用于所有一元二次方程，因式分解法只能适用于某些符合条件的一元二次方程。

## 3.5.2 函数与方程

对于函数 y=f(x)，我们把使 f(x)=0 的实数 x 叫做函数 y=f(x)的零点。这样，函数的零点就是方程 f(x)=0 的实数根，即函数 y=f(x)与 x 轴的交点横坐标。所以：

方程 f(x)=0 有实数根，等价于函数 y=f(x)的图像与 x 轴有交点，也等价于函数 y=f(x)有零点。

比如 f(x)= $x^2$-3x+2 与 x 轴的交点与 $x^2$-3x+2=0 的根的关系如图 3-31 所示。

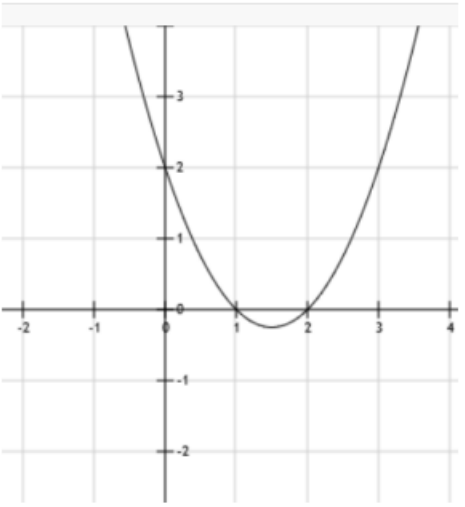



图 3-31 函数与方程根的关系

函数 f(x)与 x 轴的两个交点横坐标 $x_1$=1, $x_2$=2 就是函数的零点，也就是对应方程的两个根。

一般地，如果函数 y=f(x)在区间[a, b]上的图象是连续不断的一条曲线，并且有 f(a)*f(b)<0，那么，函数 y=f(x)在区间(a, b)内有零点，即存在 ζ ∈(a, b)，使得 f(ζ)=0。这个 ζ 也是方程 f(x)=0 的根。这就是所谓的零点定理。零点定理的几何意义如图 3-32 所示：

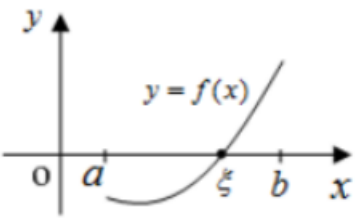



图 3-32 零点定理几何意义

现在我们试着求函数 f(x)=lnx+2x-6 的零点个数

因为：

- f(1)=-4<0
- f(3)=ln3+6-6=ln3>0
- f(1)*f(3)<0

所以函数在区间(1, 3)之间有一个零点，又因为函数 f(x)为增函数（lnx 和 2x 都是增函数，它们之和也必然为增函数），所以函数只有 1 个零点。

## 3.5.3 高次方程

只含有一个未知数，并且未知数的最高次数为 3 的整式方程叫做一元三次方程（英文名：cubic equation of one unknown）。一元三次方程的标准形式是 $ax^3+bx^2+cx+d=0$（a，b，c，d 为常数，x 为未知数，且 $a\neq0$）。一元三次方程的公式解法有卡尔丹公式法与盛金公式法。两种公式法都可以解标准型的一元三次方程。由于用卡尔丹公式解题存在复杂性，相比之下，盛金公式解题更为直观，效率更高。这里我们并不展开讲解这两种解法，有兴趣的读者可以找到相关资料进一步学习和了解。

**1.有关三次、四次方程求解的小故事**

很久以前，人们就解决了一元一次方程与一元二次方程的求解问题。然而对一元三次方程的求解却使众多的数学家们陷入了困境，许多人的努力都以失败而告终。

1494 年，意大利数学家帕西奥利（1445-1509 年），对三次方程进行过艰辛的探索后作出极其悲观的结论，他认为在当时的数学中，求解三次方程，犹如化圆为方问题一样，是根本不可能的。这种失败的悲叹声，却成为 16 世纪意大利数学家迎接挑战的号角。

费罗在帕西奥利作出悲观结论不久，大约在 1500 年左右，得到了 $x^3+mx=n$ 这样一类缺项三次方程的求解公式。他并没有发表自己的成果，而对自己的解法绝对保密。最后到费罗临终前，他将自己的方法传给了两个人：他的女婿和他的一个学生费奥。女婿不学无术不久就将此抛之脑后了，费奥独得费罗秘技而以之炫耀于世。

1534 年塔塔利亚（塔塔利亚意为“口吃者”，出生于 1500 年。他 12 岁那年，被入侵的法国兵砍伤了头部和舌头，从此说话结结巴巴，真名反倒少有人叫了，他自学成才，成了数学家）宣称自己已得到了形如 $x^3+mx^2=n$ 这类没有一次项的三次方程的求解方法。

不久，费奥和塔塔利亚在米兰挑战解三次方程。最后塔塔利亚以 30:0 的战绩获胜。最后 1541 年，塔塔利亚完全解决了三次方程求解的问题，但依然没有将自己的成果很快发表。

后来，一位米兰教书兼行医的卡尔丹在向塔塔利亚再三请求并立誓永不泄密的情况下，获得了塔塔利亚的解法。但不久，在 1545 年，卡尔丹违背誓言，在《大术》一书中将三次方程解法公诸于众。《大术》一书并非完全抄袭之作，其中也包含着他自己独特的创造，比如他注明了解法的来源，并推广到了一般方法，并给予了几何证明。

一元四次方程，卡尔丹没有给出解决方法，但他的学生费拉里发现了四次方程的一般解法。费拉里 18 岁时接替卡尔丹在米兰讲学。卡尔丹这种失信大大激怒了塔塔利亚。1546 年他在《各式各样的问题与发明》一书中严斥卡尔丹的失信行为，于是一场争吵无可避免地发生了。卡尔丹在这场公开辩论中自己避不出席而是派遣了费拉里出马。在这场公开的辩论中，塔塔利亚先以三次方程的迅速解答取得优势，而费拉里则指对方不能解四次方程。最后客场作战的塔塔利亚以失败而告终，后者宣称了自己胜利。由于卡尔丹最早发表了求解三次方程的方法，因而数学上三次方程的解法至今仍被称为“卡尔丹公式”，塔塔利亚之名反而湮没无闻了。

关于三次、四次方程的求根公式，要涉及复数概念，复数是指能写成如下形式的数 a+bi，这里 a 和 b 是实数，i 是虚数单位(即-1 开根)。 由意大利米兰学者卡尔丹在十六世纪首次引入，经过达朗贝尔、棣莫弗、欧拉、高斯等人的工作，逐渐为数学家所接受。

**2.高次方程求解**

一元三次、四次方程求根公式找到后，人们在努力寻找一元五次方程求根公式，三百年过去了，但没有人成功，这些经过尝试而没有得到结果的人当中，不乏有大数学家。

后来，年轻的挪威数学家阿贝尔于 1824 年所证明， n 次方程($n\geqslant5$)没有公式解（即没有求根公式）。不过，对这个问题的研究，并没结束，因为人们发现有些 n 次方程($n\geqslant5$)可

有求根公式。那么又是什么样的一元 n 次方程才没有求根公式呢?不久，这一问题在 19 世纪上半叶，被法国天才数学家伽罗瓦利用他创造的全新的数学方法所证明，并促使新的数学分支"群论"的诞生。

对于 5 次以上的一元高次方程，由于没有了求根公式，所以无法得到方程的精确解，但是可以通过一些方法来获取近似解。这些方法包括：二分法，牛顿法，弦截法等。

在给定精度 ε，二分法求函数 f(x)零点近似值的步骤为：

1，确定解所在区间[a, b]，使得 f(a).f(b)<0

2，求区间(a, b)的中点 $x_1=(a+b)/2$

3，计算 $f(x_1)$。若 $f(x_1)=0$，则 $x_1$ 就是函数的零点；若 $f(a).f(x_1)<0$,则解在$(a, x_1)$之间，令 $b=x_1$;若 $f(x_1).f(b)<0$，则解在$(x_1, b)$之间，令 $a=x_1$

4，判断是否达到所需精度 ε，即若$|a-b|<\varepsilon$，则得到零点近似值 a（或 b）；否则继续重复步骤 2 到步骤 4。

现在我们用二分法试着求方程 lnx+2x-6=0 的解，精确到 0.01（区间端点之差）

设函数 f(x)=lnx+2x-6，那么它的图像如图 3-33 所示：

f(2)=ln2+4-6≈4.693-6<0

f(3)=ln3+6-6=ln3>0

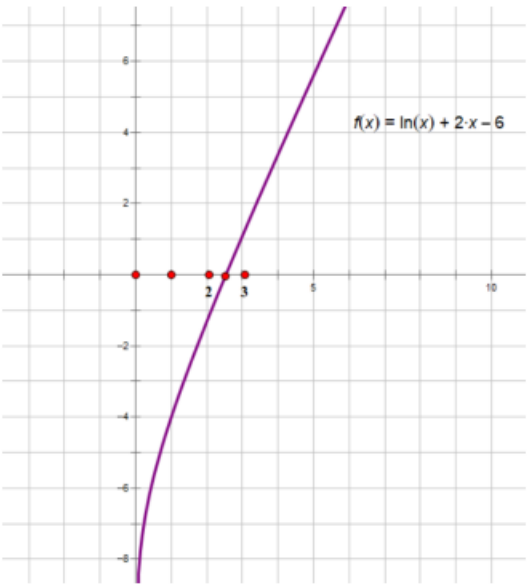



图 3-33 二分法求方程解

由于 f(2).f(3)<0，所以函数在区间[2, 3]之间必有零点。又容易看出 f(x)为单调递增函数（lnx 和 2x 都是增函数，它们之和也是增函数），所以函数在定义域内只有一个零点，也就是原方程只有一个根。

现在，从最初区间[2, 3]开始，根据二分法分别求出区间，区间的中点值和重点函数近似值如表 3-2 所示：

| 区间[a, b] | 中点的值 $x_1=(a+b)/2$ | 中点函数近似值 $f(x_1)$ |
|---|---|---|
| (2, 3) | 2.5 | -0.084 |
| (2.5, 3) | 2.75 | 0.512 |
| (2.5, 2.75) | 2.625 | 0.215 |
| (2.5, 2.625) | 2.5625 | 0.066 |
| (2.5, 2.5625) | 2.53125 | -0.009 |
| (2.53125, 2.5625) | 2.546875 | 0.029 |

| | | |
|---|---|---|
| (2.53125, 2.546875) | 2.5390625 | 0.010 |
| (2.53125, 2.5390625) | 2.53515625 | 0.001 |

表 3-2 二分法求解方程近似根区间变化

此时，由于|2.5390625-2.53125|=0.0078125<0.01，符合题目的精度要求，所以可以把 x=5.53125 作为函数在零点的近似解，也就是原方程的近似根。

牛顿法也叫切线法，需要涉及到计算函数的导数（涉及到微积分内容）。如图 3-34 所示，切线法的思想是从函数曲线上一点作切线与 x 轴相交于 $x_1$，那么 $x_1$ 将会比原来的点更靠近根。在 $x_1$ 点继续作切线，这样就会不断的靠近方程的根。

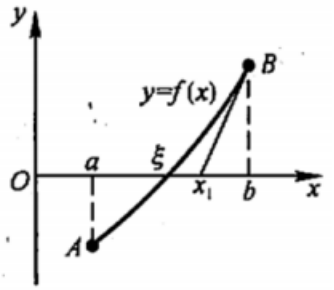



图 3-34 牛顿法解方程

有时候如果函数的导数计算比较复杂，那么也可以使用割线来代替切线，来获取根的近似解，这种方法称为割线法或者弦截法。如图 3-35 所示，通过曲线上的 2 点 $x_k$ 和 $x_{k-1}$，作一条割线，与 x 轴相交于 $x_{k+1}$，那么 $x_{k+1}$ 会比 $x_k$ 和 $x_{k-1}$ 更靠近方程的根。

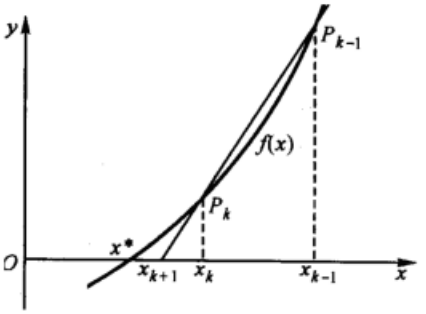



图 3-35 割线法解方程

## 3.6 函数在计算机领域的应用

初等函数的学习和掌握，无论是对于高等数学微积分的学习，还是计算机基础理论的学习，都非常重要。同时，也正是因为函数概念的建立和发展 ，才解决了由无穷小引起的第二次数学危机。

### 3.6.1 程序中的数学函数

对于一些常用的数学函数，计算机中提供了专门的库函数，在编写程序的时候直接可以调用。比如，在 C 语言里，所有的数学函数都包含在 math.h 这个头文件里。

**1.三角函数**

double sin(double);正弦

double cos(double);余弦
double tan(double);正切

**2. 反三角函数**

double asin (double);
double acos (double);
double atan (double);
double atan2 (double,double);

**3. 指数与对数**

double log (double); 以 e 为底的对数
double log10 (double);以 10 为底的对数
double pow(double x,double y);计算 x 的 y 次幂
double exp (double);求取自然数 e 的幂
double sqrt (double);开平方根

**4.取整**

double ceil (double); 取上整，返回不比 x 小的最小整数
double floor (double); 取下整，返回不比 x 大的最大整数，即高斯函数[x]

**5. 绝对值**

double fabs (double);求实型的绝对值

计算机程序设计里，也有函数这个概念，但和数学里的函数概念相比，还是有一定的区别的。

在编程语言里，把为了实现某一特定的功能的所有程序语句归纳在一起，就形成了一个函数。比如将两个数进行交换，求两个数中的最大数，将整数转换为字符串，这些功能都可以通过函数来实现。一般来说，函数只实现单一功能。如果一组代码是为了实现一个单一功能，则归纳出来，形成一个函数。下面的代码，就是在 C 语言中求两个数中最大值的函数：

```
int max(int x, int y){
    int max;
    if (x>y)
        max = x;
    else
        max = y;
    return max;
}
```

但是，编程语言里的函数，并不一定限制为一对一或者多对一这种映射关系。也有可能是一对多或者多对多。比如在 Go，Python 等编程语言里，就允许返回一对多或者多对多的函数值（程序计算结果）。

现在，当我们有了函数的基础理论知识之后，我们就可以利用函数来计算或者解释计算机理论中的一些基本常识或者结论了。这里我们分别举二进制，磁盘大小和分页内存等相关的几个简单的例子。

## 3.6.2 二进制与指数运算

我们知道，计算机的实现是基于二进制的。一个 bit 可以表示 0 或者 1。一个 byte 包含 8 个 bit，可以表示的有符号数是-128~127 之间，表示的无符号数是 0~255 之间。那么这是

怎么来的呢？

现在我们假设一个含 n 位的内存，对于无符号整数来说，n 位全部表示数值，那么它能表示的值即：

| $2^{n-1}$ | … | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|---|---|---|---|---|---|---|---|

把所有位数对应的值加起来求和，则有（等比数列求和，参考本书 5.4 节）：

$$1+2+2^2+\ldots+2^{n-1}=2^n-1$$

所以，n 位无符号数，表示的数的范围为：$[0, 2^n-1]$。

对于有符号整数来说，最高位为符号位，当符号位为 0 的时候为正数，且当符号位为 0 其它位都为 1 的时候为有符号整数能表示的最大正整数。因此有：

| 0 | $2^{n-2}$ | … | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
|---|---|---|---|---|---|---|---|

把所有位数对应的值加起来求和，则有：

$$1+2+2^2+\ldots+2^{n-2}=2^{n-1}-1$$

如果最高位为 1，其它位都为 0，那么就是最小的负数。因此有：

| 1 | 0 | … | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|

由于负数采取补码存储，它的绝对值为补码取反+1，因此它的绝对值为：

| 1 | 0 | … | 0 | 0 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|

这个数为 $2^{n-1}$。因此有符号数的取值范围为$[-2^{n-1}, 2^{n-1}-1]$。因此，根据 C 语言程序中不同类型 bit 位数（char 为 8 个比特，short 为 16 个比特，int 或 long 为 32 个比特），可以计算出，常用的整数表示范围如表 3-3 所示：

| 类型 | 有符号值范围 | 无符号值范围 |
|---|---|---|
| char | [-128,127] | [0,255] |
| short | [-32768,32767] | [0,65535] |
| int/long | [-2147483648 , 2147483647 ] | [0, 4294967295 ] |

表 3-3 整数表示范围

### 3.6.3 磁盘大小计算

在早期的计算机系统里，采用的启动方式为BIOS+MBR 方式，但这种启动方式有一个最大的限制，那就是启动盘的大小不能超过 2TB，超过 2TB 就不支持了。

为什么会有这个限制呢？现在我们通过数学计算解释下其中的原因。

一般扇区的大小为 512 字节。一个磁盘由扇区组成。在 BIOS+MBR 模式下，表示扇区的逻辑地址为 LBA(logical block address），是一个 32 位的整数。因此，能够支持的磁盘数为 $2^{32}$ 个。那么能支持的最大磁盘容量为每个扇区的大小乘以扇区的总数，即：

$$512*2^{32}=5^9*2^{32}=2^{41}=2*2^{40}=2T$$

因此，只能支持不超过 2T 的启动盘。

为了解决 BIOS+MBR 这种限制，新一代的启动系统采用了 UEFI+GPT 模式，其中的 LBA 采用了 64 位整数，那么表示扇区的总数将会比 32 位整数大很多，因此突破了 2T 的限制。

### 3.6.4 分页内存计算

现在的操作系统都是基于分页内存管理的。物理内存会被按照 4KB 为一个页面单位进行管理。在管理分页内存的时候，会用到页表机制。页表本身也是存放在以页为单位的物理内存中的。如图 3-36 所示，在 32 位系统里，一个物理地址是 32 位，所以一个 4K 的物理内存页可以存放 $4*2^{10}/4=2^{10}$ 项。页内偏移为 $4K=4*2^{10}=2^{12}$。

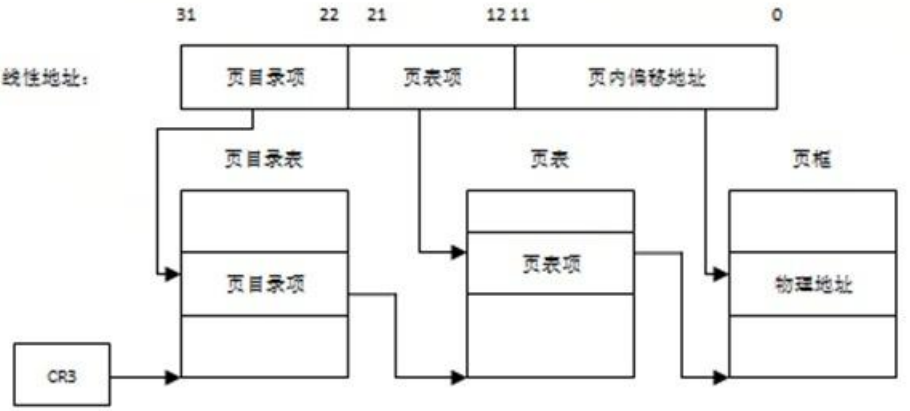



图 3-36 计算机 32 位地址映射

这就是为什么在 32 位系统里面，二级页目录寻址情况下，32 位线性地址划分为三个部分，并且这三个部分的长度保持 10:10:12 这种比例的原因。因为最低部分表示 4K 的物理页内偏移，需要 12 位。剩余 2 部分，分别表示一个物理内存页里能存放的页表项数，各需要 10 位。

在 64 位系统里，物理地址需要 8 个字节来表示了，所以一个 4K 的物理内存页，能容纳的页表项为 $4*2^{10}/8=2^{12}/2^3=2^9$，页内偏移依然为 4K，即 $4*2^{10}=2^{12}$，所以依然需要 12 位表示页内偏移。因此如图 3-37 所示，64 位的线性地址被划分为了四个部分，每个部分的长度之比为 9:9:9:9:12（共 48 位，操作系统比如 Linux 只使用了 64 位中的一部分，即 48 位）

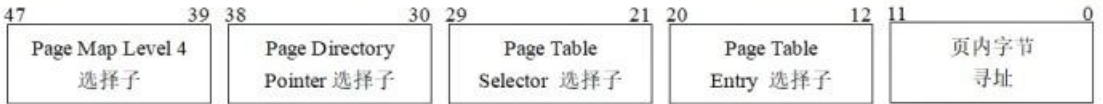


图 3-37 64 位系统虚拟地址划分

上面这些例子，其实都和指数运算或者指数函数有关。所以，如果我们没有指数运算或者指数函数的理论学习，这些结论就只能死记硬背，知其然而不知其所以然。

### 3.6.5 算法复杂度与机器学习中的函数

除此以外，初等函数在计算机中的算法复杂度，机器学习，信息论中都会用到。下面我们依次举几个这方面的例子来做一个简单的介绍。

#### 1．算法复杂度函数

在我们后面还会详细讲解的算法复杂度领域，我们就是通过初等函数来描述一个算法的复杂度的，并且根据算法复杂度的不同，有如下的不等式：

$$O(1)< O(\log n)< O(n)< O(n\log n)< O(n^2)< O(n^3)< O(2^n)< O(n!)<O(n^n)$$

即不等式越往右，算法复杂度越高，计算机执行越耗时。可见，算法复杂度也是通过初等函数来描述的。这些初等函数在直角坐标系中的图像如图 3-38 所示，复杂度越高的函数，增长速度越快，函数图像越陡。

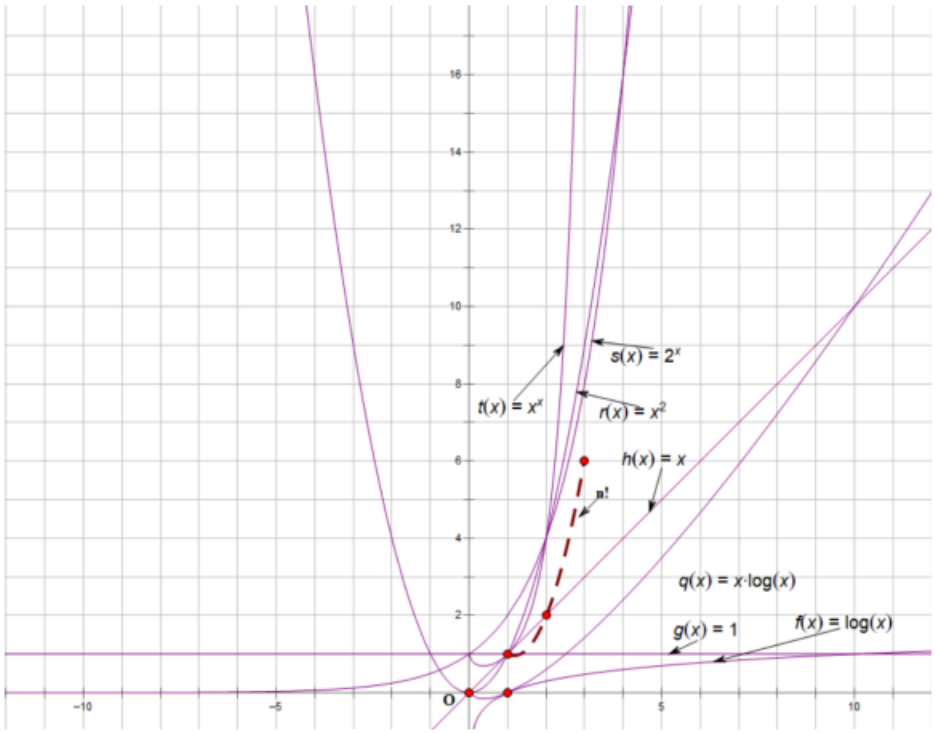



图 3-38 算法复杂度对应的函数图像

#### 2．Logloss 函数

在机器学习领域，我们会经常用函数计算和评估训练模型(training model)的效果，即 logloss（对数损失）函数。而 logloss 函数的计算方式，就是通过对数等初等函数来完成的：

$$-\frac{1}{N}\sum_{i=1}^{N}(y_i \log p_i + (1-y_i)\log(1-p_i))$$

其中，y 表示样本的真实标签(1 或-1)，p 表示模型预测为正样本的概率，N 表示样本数。

## 3．Sigmoid 函数

Sigmoid 函数，也是一个初等函数，它的解析式和函数图像如图 3-39 所示：

$$S(x) = \frac{1}{1+e^{-x}}$$

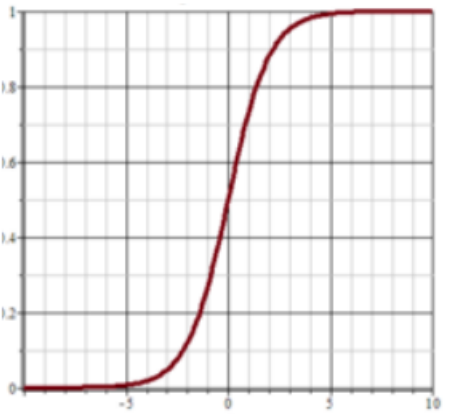


图 3-39 Sigmoid 函数

Sigmoid 函数常被用作机器学习中神经网络的阈值函数，将变量映射到[0,1]之间取值。

## 4．Relu 函数

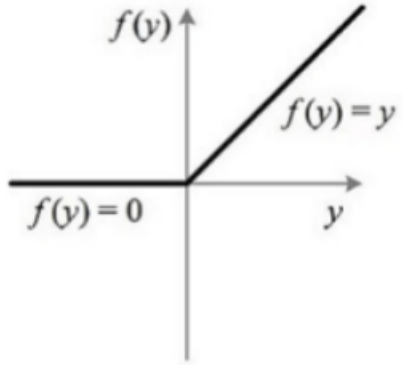
Relu 函数，即线性整流函数（Rectified Linear Unit, ReLU），又称修正线性单元，是一种人工神经网络中常用的激活函数（activation function），通常指代以斜坡函数及其变种为代表的非线性函数。它的定义如下：

$$f(x) = \max(0, x)$$

或者

$$ReLU(x) = \begin{cases} x & if\, x > 0 \\ 0 & if\, x \le 0 \end{cases}$$

函数图像如 3-40 所示：




$$ReLU(x) = \begin{cases} x & if x > 0 \\ 0 & if x \le 0 \end{cases}$$

图 3-40 Relu 函数

Relu 函数在神经网络的模型训练中能够克服梯度消失，并加快训练速度，而且这两个方面是相辅相成的，当解决了梯度消失的问题，那么训练就会变快。

只有当我们有了函数的基本概念之后，我们才能够很好的理解信息论中的信息量和信息熵等概念。现在我们来给大家简单介绍下这两个概念。

## 5. 信息量

信息量是对信息的度量，就好比秒是时间的度量一样。信息的大小跟随机事件的概率有关。越小概率的事情发生了产生的信息量越大，如中国男足进了世界杯，这是个小概率事件，如果发生了则会差生轰动效应；越大概率的事情发生产生的信息量越小，如太阳从东边升起。

因此一个具体事件的信息量应该是随着其发生概率而递减的。当越不可能的事件发生了，我们获取到的信息量就越大。越可能发生的事件发生了，我们获取到的信息量就越小。那么信息量应该和事件发生的概率有关。

设 X 是一个离散型随机变量，其取值集合为 $x$，概率分布函数为 $p(x)$, $x \in x$，则定义事件 $X=x_0$ 的信息量为：

$$I(x_0)=-\log(p(x_0))$$

从上述公式我们可以看出其中使用了对数函数来定义信息量。由于是概率，所以 $p(x_0)$ 的取值范围是[0, 1]，绘制为信息量图形如图 3-41 所示：

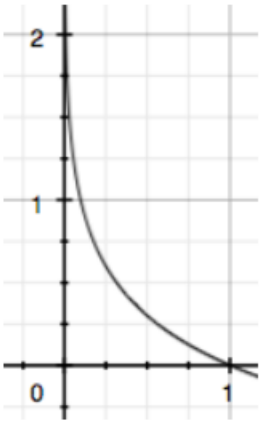


图 3-41 信息量与概率关系

从图中可以看出，概率越小，信息量越大，概率越大，信息量越小。信息量的定义里，取概率的对数的负数，是为了将信息量定义为正值。

## 6. 信息熵

1948 年，香农（Claude Elwood Shannon ）提出了“信息熵”的概念，解决了对信息的量化度量问题。信息熵这个词是香农从热力学中借用过来的。热力学中的热熵是表示分子状态混乱程度的物理量。香农用信息熵的概念来描述信源的不确定度。

在机器学习中，熵 (Entropy)，是一个非常重要的概念。信息熵常被用来作为一个系统的信息含量的量化指标，从而可以进一步用来作为系统方程优化的目标或者参数选择的判据。在决策树的生成过程中，就使用了熵来作为样本最优属性划分的判据。

那么什么是信息熵呢？

对于某个事件 $x_i$，有 n 种可能性，每一种可能性都有一个概率 $p(x_i)$，这样就可以利用信息量的定义公式，计算出某一种可能性的信息量。而信息熵就是用来表示所有信息量的期望，定义公式如下：

$$H(X) = -\sum_{i=1}^{n} p(x_i) log(p(x_i))$$

最后，初等函数也是进一步学习高等数学的重要基础。高等数学中的微积分就是基于初等函数展开的。而高等数学又是进一步学习概率论与数理统计的基础，概率论与数理统计又是学习机器学习和人工智能的基础。因此，掌握初等函数的基本理论，无论是进一步学习数学，还是学习计算机，都是非常必要的。

# 四，计算的度量：算法复杂度

本章我们来学习计算机科学中的一个重要理论，即可计算理论。为了解决一个问题，我们往往需要设计出一个具体的算法。算法是程序设计中的一个核心问题，也是进入IT名企工作的必考内容。这一章，我们将结合数学理论来学习一下算法复杂度相关的理论知识。

## 4.1 什么是算法？

算法是指解决问题的方法和步骤。

比如，把大象放冰箱，要解决这个问题，可以分为这样几步：首先打开冰箱门，然后把大象挪进去，最后关上冰箱门，这就是一个算法。在计算机里，我们往往还需要通过编程语言来实现算法的步骤。解决一个问题的算法，有好坏优劣之分，判断的标准就是算法的复杂度。

比如有一个猜数字的游戏。游戏主持人暗中选定一个范围在1到100之间的某一个数字，让游戏参与者用最少的次数猜出这个数字即可获胜。

为了猜出这个数字，你可以从数字1开始，挨个问游戏主持人：这个数字是1吗？这个数字是2吗？...这个数字是100吗？这样你最终肯定能猜出这个数字，这是一种算法。但这种算法不是最优的，如果主持人把这个数字设定为100，你得问100次才能猜出这个数字。

另一种算法是：你可以先问主持人，这个数字是否大于50？这样无论主持人回答是或者不是，都可以排除掉一半的数字(50之前的全部被排除，或者50之后的全部被排除)。假如回答不是，接着可以问是否大于25？又可以排除掉一半。这样下去，很快就会排除其它数字而最后只剩下一个数字，这个数字就是你要找的那个。这种算法明显优于第一种算法，因为它可以用更少的平均查找次数找到这个数字，它其实就是著名的二分查找算法。

## 4.2 时间复杂度与空间复杂度

对于同一个问题，也许存在多种算法可以解决它。那么究竟哪个算法是最好的呢？因此我们可以使用算法复杂度来衡量算法的好坏。算法的复杂度主要分为时间复杂度和空间复杂度。

算法的时间复杂度是指执行算法所需要的计算工作量。一个算法花费的时间与算法中语句的**执行次数**成正比，哪个算法中语句执行次数多，它花费时间就多。需要注意的是，算法复杂度并不是说某个算法计算具体需要多少时间（实际上，由于机器配置和性能的不同，即使同样的代码在不同的机器配置下，消耗的时间也可能不同），而是衡量当问题的规模（比如要处理的数据）增大时，程序需要的时间长度增长得有多快。一个算法中的语句执行次数称为语句频度或时间频度，记为T(n)，其中n为问题规模，比如数组元素的个数，链表结点的个数或者字符串中字符的个数等都和n有关系。

一般情况下，算法中**基本操作重复执行的次数**是问题规模n的某个函数。如$T(n)=4n^2+5n$，或者$8n^2+1$，那么时间复杂度都记为：$O(n^2)$，这里取的是T(n)的最高幂，并去掉系数。

常见的时间复杂度有：常数阶O(1)，对数阶O(logn)，线性阶O(n)，线性对数阶O(nlogn)，平方阶$O(n^2)$，立方阶$O(n^3)$，...，k次方阶$O(n^k)$，指数阶$O(2^n)$和阶乘O(n!)以及$O(n^n)$。随着问题规模n的不断增大，上述时间复杂度不断增大，算法的执行效率越低。

在计算时间复杂度的时候，先找出算法的基本操作，然后根据相应的各语句确定它的执

行次数 T(n)，再找出 T(n) 的同数量级（它的同数量级有：1，logn，n，nlogn ，$n^2$，$n^3$，$2^n$，n!，$n^n$），找出后，设 f(n)= 该数量级，则时间复杂度为 O(f(n)) 。比如，假设 T(n)的同数量级为 n，则 f(n)=n，那么算法复杂度为 O(n)。

下面我们举两个具体的计算算法时间复杂度的例子：

例 1：求下列程序的时间复杂度。

```
int max(int x, int y){
    return x>y?x:y;
}
```

因为函数没有循环，只有一条基本语句，所以时间复杂度：O(1)。

例 2：求下列程序的时间复杂度。

```
for (int i=1;i<=n;++i){
    for (int j=1;j<=n;++j){
        c[i][j]=0;        //该步骤属于基本操作 执行次数：n^2
        for (int k=1;k<=n;++k)
            c[i][j] += a[i][k]*b[k][j]; //该步骤属于基本操作 执行次数：n^3
    }
}
```

根据代码注释里的分析，各语句执行的次数函数为 $T(n)=n^3+n^2$，保留 T(n)的最高次数项，则算法时间复杂度为 $O(n^3)$。

一般来说，算法时间复杂度的计算的方法是看看算法里有几重循环，没有循环，O（1）；只有一重循环则时间复杂度为 O(n) ；二重循环则为 $O(n^2)$ ；依此类推。如果用到二分法则为 O(logn)，比如二分查找，一次查找，就可以去掉一半的数据。如果一个 for 循环套一个二分法，那么时间复杂度则为 O(nlogn) ，比如快速排序的算法复杂度就是 O(nlogn)。

并且根据算法复杂度的不同，有如下的不等式：

$$O(1)< O(\log n)< O(n)< O(n\log n)< O(n^2)< O(n^3)< O(2^n)< O(n!)<O(n^n)$$

即不等式越往右，算法复杂度越高，计算机执行基本操作的次数越多，越耗时。

算法的空间复杂度是指算法需要消耗的内存空间。其计算和表示方法与时间复杂度类似，一般都用复杂度的渐近性来表示。同时间复杂度相比，空间复杂度的分析要简单得多。程序在运行时候动态分配的空间，以及递归栈所需的空间等。这部分的空间大小与算法有关。而那些静态的空间，比如代码，常量以及简单的变量和数组与空间复杂度无关，或者说空间复杂度为 O(1)。

## 4.3 NP=P 问题

可计算理论中，有一个尚未证明的问题，就是 NP 问题是否等于 P 问题。这个问题是世界七大数学难题之一（另外六个是霍奇猜想、庞加莱猜想、黎曼假设、杨-米尔斯存在性和质量缺口、纳卫尔-斯托可方程、BSD 猜想。这七个问题于 2000 年由克雷(Clay)数学研究所公布，都被悬赏一百万美元，又称千年大奖问题。截至目前，庞加莱猜想已由俄罗斯数学家格里戈里·佩雷尔曼解决）。那么什么是 NP 问题，什么又是 P 问题呢？接下来我们就从头开始给大家作个详细的介绍。

### 4.3.1 多项式复杂度

要理解什么是P问题，什么是NP问题，我们要从算法的时间复杂度说起。算法的时间复杂度，一种是O(1)，O(log(n))，$O(n^a)$等，我们把它们称为多项式级的复杂度，因为这种复杂度的问题规模n出现在底数（而不是指数）的位置。比如一个算法的时间数量级为：

$$T(n)=n^3+2n^2+1$$

实际上上它的算法时间复杂度为O（$n^3$），这就是一个多项式复杂度。

另一种是$O(a^n)$、O(n!)和$O(n^n)$型复杂度，它是非多项式级的，其复杂度计算机往往不能承受。当我们在解决一个问题时，我们选择的算法通常都需要是多项式级的复杂度，非多项式级的复杂度需要的时间太多，往往会超时，除非是数据规模非常小，否则就是不可计算的。

### 4.3.2 P 问题

将算法问题，以这个算法的时间复杂度（常见的有线性、对数、多项式、指数这几种）为依据，用多项式时间复杂度将算法问题分为两大类**P 问题和非 P 问题**（P是英文单词多项式 polynomial 的第一个字母）。

P问题，就是可以在多项式时间复杂度内解决的问题。一个规模为n的问题，如果能在n的多项式时间复杂度内解决，就是p问题。

非P问题，就是不可以在多项式时间复杂度内解决的问题。对于还没有找到在多项式时间复杂度内解决算法的问题，就从验证这个问题的某个可能解出发，根据验证解的时间复杂度将非P问题分为两类NP问题，非NP问题。

### 4.3.3 NP 问题

验证一个问题的可能解是不是正确的，如果这个验证过程的时间复杂度都不是多项式时间的，那这个问题真正的解也不会是多项式时间的。所以NP问题就是指求问题的真正解不在多项式时间内，但验证这个问题的某个可能解，这个验证过程在多项式时间内。一个规模为n的问题，如果它的每一个可能的解，都能在n的多项式时间内被验证正确与否，就是NP问题。

因此NP问题是指可以在多项式的时间里验证一个解的问题。如果一个问题是NP问题，那么它也是可以在多项式的时间里猜出一个解的问题。对于NP问题来说，找一个解很困难，但验证一个解很容易。验证一个解只需要多项式的时间复杂度。如果一个问题既不是P问题，也不是NP问题，而是非NP问题，那么即使你猜到了解但是没用，因为你不能在多项式的时间里去验证这个解是否正确。

之所以要定义NP问题，是因为通常只有NP问题才可能找到多项式的算法。我们不会指望一个连多项式地验证一个解都不行的问题存在一个解决它的多项式级的算法。

那么有没有NP问题的具体例子呢？实际上，“售货员旅行”问题（也叫旅行商问题）就是一个常见的NP问题。这个问题是这样描述的：

有一个售货员需要驾驶汽车到n个城市去推销货物。他必须经过全部的n个城市。他手里有一张地图，地图指出了这n个城市之间的道路距离。试问如果他从家中出发经过全部的n个城市再回到家中，那么他应该如何取得最短的行程？

如图4-1所示，有A, B, C, D, E, F, G共7个城市以及相应的距离（单位为KM）。售货员的家在城市A，如果售货员从A出发，再回到A，那么可能的路径有：(1)A→B→C→D→E→F→G→A，共225KM，或者(2)A→G→B→C→D→F→E→A，共230KM，等等各种走法。那么如何找到这些走法中最短的路径呢？

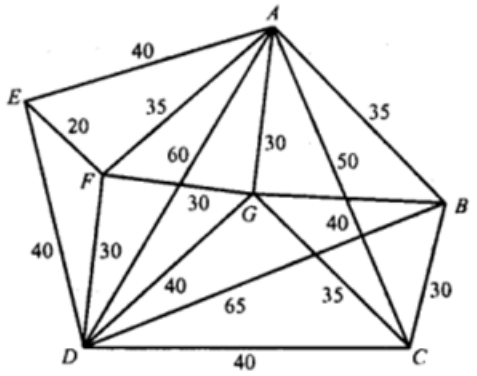



图 4-1 售货员送货问题

有一种方法是把所有的走法都挨个遍历一遍，也就是将A, B, C, D, E, F, G这7个城市进行排序，即7个城市的全排列，根据排列原理，共有6！（出发的城市是固定的，不用排序）种可能，从这些可能中找出最短距离。

那么对于n+1个城市，将会需要进行n!次遍历。由于n! $\approx\sqrt{2\pi n}\left(\frac{n}{e}\right)^n$（此公式为斯特林公式），因此，问题会随着n的增大而呈现指数级增长。比如当n=20的时候，根据公式有：

$$19!\approx 1.24*10^{17}$$

这个数字已经非常大，如果由计算机来计算，将会耗时很多时间。

所以，一个用穷举换来的非多项式级时间复杂度的超时程序不会涵盖任何有价值的算法。非NP问题就是问题的真正解不在多项式时间内，同时验证过程也不在多项式时间内。在经典计算机中，一个呈幂次上升的计算量应该可以解决的，但对一个呈指数上升的计算量在n相当大时则毫无希望。为什么这里必须说是经典计算机呢？因为如果使用的是量子计算机，那么部分NP问题，可以转化为P问题，请参考本书4.4.5一节。

NP问题可以分为NP-Complete问题、NP-Hard问题。NP中有一批互相依赖的问题，称之为NP-Complete（NP完全问题，简称为NPC问题）类。1971年Stephen A Cook发表了“The Complexity of Theorem Proving Procedures”。他把P之外的问题归纳为三大类，即NP，NP-Complete，以及NP-hard。

## 4.3.4 NP 完全问题与 NP Hard 问题

NP问题有很多种，但若所有的NP问题都能多项式归约（可以理解为转化）到问题X，那么X就是一个NP-Hard问题，进一步如果X是NP的，就称X是NP-Complete（NP完全）的，否则X就只能是NP-Hard的。事实上，研究发现，所有的NP问题都可以归约到某一些NP问题，这类问题也就是经常出现的NP complete问题。

NP-Complete问题指的是NP问题中，若干个问题之间互相关联，都是由一个根问题归约出来的，只要解决这个根问题，其他问题也解决了，那么这若干个具有归约关系的问题里

面，每个都是 NP-Complete 问题。比如有 3 个 NP 问题 A, B, C，只要把 A 问题解决了，B，C 问题也解决了，那么 A, B, C 都是 NP-Complete 问题。

不是 NP-Complete 问题的就是 NP-Hard 问题，所有的 NP 问题可以在多项式时间归约到它，但它不一定是一个 NP 问题。

为了介绍 NPC 问题，我们先引入一个概念**约化**(也称为“归约”)。简单地说，一个问题 A 可以约化为问题 B 的含义是指可以用问题 B 的解法解决问题 A，或者说，问题 A 可以“变成”问题 B。B 的时间复杂度高于或者等于 A 的时间复杂度。也就是说，问题 A 不比问题 B 难或者说问题 B 的难度大于等于问题 A，这是因为如果不这样，既然问题 A 能用问题 B 来解决，倘若 B 的时间复杂度比 A 的时间复杂度还低了，那 A 的算法就可以改进为 B 的算法，两者的时间复杂度还是相同。正如解一元二次方程比解一元一次方程难，因为解决前者的方法可以用来解决后者。约化具有一项重要的性质**传递性**。如果问题 A 可约化为问题 B，问题 B 可约化为问题 C，则问题 A 一定可约化为问题 C。

“可约化”是指的可“多项式地”约化(Polynomial-time Reducible)，即能在多项式的时间里完成约化的。约化的过程只有用多项式的时间完成才有意义。

再回想前面讲的 P 和 NP 问题，联想起约化的传递性，自然地，我们想知道，如果不断地约化上去，不断找到能“通吃”若干小 NP 问题的一个稍复杂的大 NP 问题，那么最后是否有可能找到一个时间复杂度最高，并且能“通吃”所有的 NP 问题的这样一个超级 NP 问题？答案是肯定的。也就是说，存在这样一个 NP 问题，所有的 NP 问题都可以约化成它。那么，只要解决了这个超级 NP 问题，那么所有的 NP 问题都解决了。

这种超级 NP 问题的存在难以置信，并且不只一个而是有很多个。这一类问题就是传说中的 NPC 问题，通过可约化的含义，可以看出 NPC 问题是所有 NP 问题中最复杂的问题。NPC 问题的出现使整个 NP 问题的研究得到了飞跃式的发展。

NPC 问题的定义非常简单。同时满足下面两个条件的问题就是 NPC 问题。首先，它得是一个 NP 问题；然后，所有的 NP 问题都可以约化到它。证明一个问题是 NPC 问题也很简单。先证明它至少是一个 NP 问题，再证明其中一个已知的 NPC 问题能约化到它。

既然所有的 NP 问题都能约化成 NPC 问题，那么只要任意一个 NPC 问题找到了一个多项式的算法，那么所有的 NP 问题都能用这个算法解决了，NP 也就等于 P 了。

但是，给 NPC 问题找一个多项式算法是非常具有挑战的。NPC 问题目前没有多项式的有效算法，只能用指数级甚至阶乘级复杂度的算法。因此，“正是 NPC 问题的存在，使人们相信 P≠NP”。

那么有哪些问题是 NPC 问题呢？目前，已知的 NP-Complete 问题有几百个，历史上第一个被证明为 NP-Complete 问题的是布尔逻辑的可满足性问题（SATISFIABILITY，简称为 SAT），即“布尔可满足性问题是 NP 完全的”，这就是著名的 Cook 定理（也称为 Cook-Levin 定理，以斯蒂芬库克和列昂尼德莱文命名）。因此，可以说 SAT 问题是所有这些 NPC 问题的“祖先”。

那么什么是布尔逻辑的可满足性问题呢？我们知道，布尔表达式是由布尔变量和逻辑运算符“与 AND”、“或 OR”、“非 NOT”所构成的表达式。如果对于所有布尔变量存在某个 TURE 或 FALSE 赋值，使得一个布尔表达式的值为 TRUE，则该布尔表达式是可满足的。如果不存在这样的赋值，则对于所有可能的变量赋值，布尔表达式的值都为 FALSE，则该布尔表达式是不可满足的。

例如，对于布尔公式 F= (x AND y) OR ( (NOT x) AND z)，当 x = FALSE， y = TRUE，z = TRUE 时，该布尔表达式值为 TRUE，则表达式 F 就是可满足的。

SAT 问题就是判定一个给定的合取范式（合取范式的概念，请参考本书 8.3 节）的布尔公式是否是可满足的。如果限定组成合取范式的析取范式的变量为 3 个，则对应的问题就是

3SAT 问题。3SAT 问题也是 NPC 问题，而且 SAT 问题可以规约到 3SAT 问题。

SAT 问题既是逻辑学的一个基本问题，也是当今计算机科学和人工智能研究的核心问题。工程技术、军事、工商管理、交通运输及自然科学研究中的许多重要问题，如程控电话的自动交换、大型数据库的维护、大规模集成电路的自动布线、软件自动开发、机器人动作规划等，都可转化成 SAT 问题。因此致力于寻找求解 SAT 问题的快速而有效的算法，不仅在理论研究上而且在许多应用领域都具有极其重要的意义。

与 SAT 类似的问题是逻辑电路问题。我们知道，一个逻辑电路由若干个输入，一个输出，若干“逻辑门”和密密麻麻的布线组成。每一个输入可以分别为 TRUE 或者 FALSE。在不通输入的情况下，经过逻辑门的计算，输出要么为 TRUE，要么为 FALSE。在有的逻辑电路中，无论输入是什么，输出都是 FALSE。那么，这个逻辑电路不存在使输出为 True 的一组输入。因此逻辑电路问题就是指：给定一个逻辑电路，问是否存在一种输入使输出为 True。逻辑电路问题也被称为 CIRCUIT-SAT 问题。逻辑电路问题被证明也是 NPC 问题。

除了 SAT 问题和 CIRCUIT-SAT 问题，另外一些常见的 NPC 问题包括：

- Clique 完全子图问题：在一个图中，找到最大的顶点集合，要求这几个顶点之间全连接。
- Vertex-cover 顶点覆盖问题：在一个图中，找到最小的顶点集合，要求这几个顶点所相连接的边覆盖全图。
- Ham-Cycle 哈密顿圈问题：要求寻找一条环游路线使得经过每个定点恰好一次。
- TSP（Travelling salesman problem）旅行商问题：如何找到在拜访每个地点一次后再回到起点的最短路径，也就是最优哈密顿圈问题。
- Subset-sum 子集和问题：从一个集合中找到一个子集，让子集的和满足一定条件。

### 4.3.5 NP=P 是否成立

曾经在 IT 界流传着这样一则笑话。一程序员去著名的搜索引擎公司 Google 面试，被问到这样一个问题，即在什么情况下 P=NP？然后他回答：“当 N=1 的时候”。该程序员这样作答，说明他应该不知道“P=NP”的具体含义，所以闹了一个笑话。

很显然，所有的 P 类问题都是 NP 问题。因为如果能多项式地解决一个问题，必然能多项式地验证一个问题的解。问题的关键是，是否所有的 NP 问题都是 P 问题。

如果把所有 P 类问题归为一个集合 P 中，把所有 NP 问题划进另一个集合 NP 中，那么，显然有 P 属于 NP。

现在，所有对 NP 问题的研究都集中在一个问题上，即究竟是否有 P=NP？通常大家所所的“NP 问题”，也就是要证明或推翻“P=NP”。

NP 问题是哥德尔在 1956 年寄给冯•诺依曼的一封信中首次提出的。这封信后来不幸遗失，20 世纪 80 年代又被找到。NP 问题出现在学术界是在 20 世纪 70 年代初，由斯蒂芬•库克和列昂尼德•莱文独立提出。之后卡普列出了这个领域中包括 TSP 旅行商问题在内的 21 个重要难题。计算机科学家从卡普的工作开始认识到 P/NP 问题极为重要。

NP 问题一直都是信息科学的一个巅峰，引人注目但难以解决。在信息科学研究中，这是一个耗费了很多时间和精力也没有解决的终极问题，好比物理学中的大统一理论和数学中的黎曼猜想、歌德巴赫猜想等。

对 NP 问题的研究具有十分重要的意义。现代密码学建立在 NP≠P 的假定之上。如果 NP=P，从理论上说，现代密码学体系建立起来的安全防线将被彻底攻破，这真是一个坏消息。但另外一方面，如果 NP=P，大数据和人工智能研究中的很多问题，工业和应用数学中

的很多问题，都能够快速得到精准解答，地球科学、生命科学、宇宙科学、环境科学、数学科学、物理科学，等一切学科都将获得极大推进。

但是，大家普遍认为，P=NP 不成立，也就是说，多数人相信，存在至少一个不可能有多项式级时间复杂度算法的 NP 问题，只要找到这样一个反例，那么结论就不成立了。大家如此坚信 P≠NP 是有原因的，就是在研究 NP 问题的过程中找出了一类非常特殊的 NP 问题，即 NPC 问题。正是 NPC 问题的存在，使人们相信 P≠NP。

现在被证明是 NPC 问题的有很多，任何一个 NPC 问题如果找到了多项式算法，那么所有的 NP 问题都可以完美解决了。因此说，正是因为 NPC 问题的存在，P=NP 变得难以置信。

图 4-2 给我们展示了，P≠NP 和 P=NP 时候，P，NP，NPC 和 NP-Hard 各问题集合之间的关系。

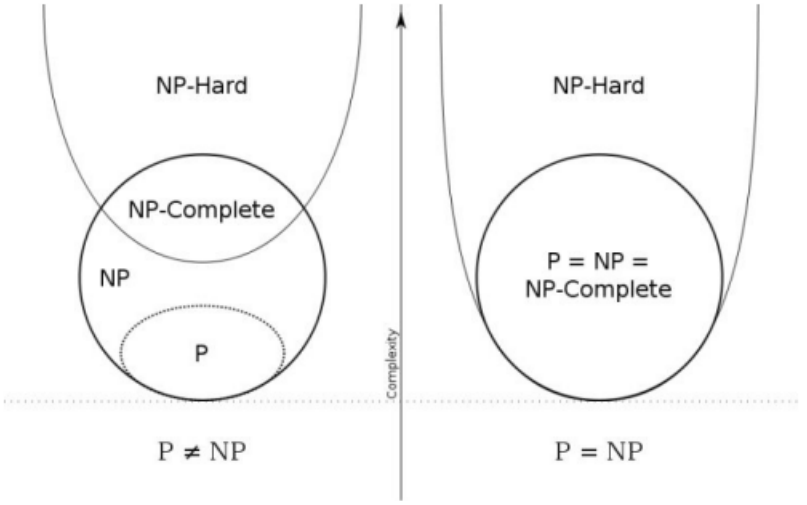



图 4-2 P≠NP 和 P=NP 时各个集合关系

2020 年 7 月出版的中国计算机学会会刊《计算机科学》发表了国防科技大学教授姜新文题为《哈密顿图判定问题的多项式时间算法》的论文，号称证明了“P=NP”问题，但尚未获得学术界的一致认可。

最后，我们根据计算的复杂度，再将各种问题做一个归纳总结。

- P ： 多项式时间解决的问题
- NP ： 非多项式时间可解决，但多项式时间可以验证解的问题
- EXP：指数时间可解决的问题，指数问题是 NP 问题
- R ：有限时间可解决的问题，如果不是 R 问题，那么这个问题将是不可计算不可解的问题，比如图灵在 1936 年提出了第一个不可解问题的实例：图灵停机问题（The Halting Problem），参考本书 8.5.3 一节。
- NP-hard： NP 问题能够多项式范围内归约到的问题，即 NP-hard 问题，如果 NP-Hard 问题又是一个 NP 问题，那么就是 NP 完全问题。

NP 完全问题是 2000 年由美国克雷数学研究所提出的悬赏一百万美元的七个世界难题中的一个。

如图 4-3 所示，在一个标示计算复杂度的坐标轴上，我们把各种问题分为 P 问题，NP 问题，EXP 问题，R 问题以及 NP-Hard 问题。

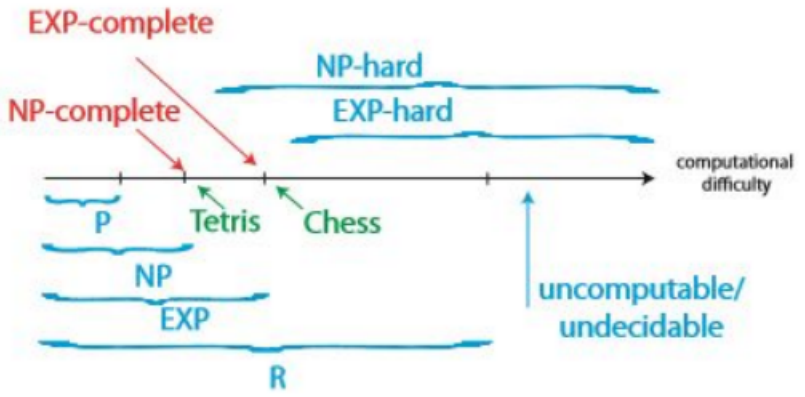



图 4-3 各种可计算问题的关系

## 4.4 量子计算

20 世纪 80 年代初期，量子计算(quantum computation) 的概念最早由美国阿贡国家实验室（Argonne National Laboratory，简称 ANL，是二次世界大战曼哈顿工程的一部分，在芝加哥大学的冶金实验室基础上发展起来。1942 年 12 月 2 日，美国科学家费米(Enrico Fermi，1901-1954)和他约 50 名的同事在芝加哥大学的壁球场里产生了世界上第一个受控核链式反应，"芝加哥一号堆"）的 P. Benioff 提出，他提出二能阶（只有两个能级，可以分别表示 0 和 1）的量子系统可以用来仿真数字计算。

### 4.4.1 量子计算机历史与现状

1981 年，美国物理学家、诺贝奖获得者费曼（Richard Phillips Feynman，1918-1988）在麻省理工学院举行的 First Conference on Physics of Computation 会议上的演讲中，勾勒出了以量子现象实现计算的愿景。费曼指出用经典计算机模拟量子力学系统存在本质的困难，当量子系统规模较大时所需的计算代价将会呈指数上升。他提出构造基于量子力学原理的计算机以克服那些困难。

1985 年，牛津大学的 D. Deutsch 提出量子图灵机（quantum Turing machine）的概念，量子计算开始具备了数学的理论基础支持。

1994 年，贝尔实验室的应用数学家 P. Shor 指出利用量子计算可以在更短的时间内将一个很大的整数分解成质因子的乘积，此即 shor 算法。此算法的提出，直接威胁到了基于大整数难以质因数分解数学难题的 RSA 加密算法的安全性。

1998 年，英国牛津大学的研究人员宣布，他们利用两个量子比特计算信息的能力取得了突破性进展。

2011 年 5 月，加拿大量子计算公司 D-Wave 正式发布了全球第一款商用型量子计算机“D-Wave One”。不过严格来说这还算不上真正意义的通用量子计算机，只是一台能用一些量子力学方法解决特殊问题的机器。通用任务方面还远不是传统硅处理器的对手，而且编程方面也需要重新学习。另外，为尽可能降低 qubit 的能级，需要利用低温超导状态下的铌产生 qubit，D-Wave 的工作温度需保持在绝对零度附近（20 mK）。

2015 年 7 月，阿里巴巴的阿里云部门和中国科学院在上海建立了一个名为阿里巴巴量子计算实验室的研究机构。这家实验室研究电子商务和数据中心的量子安全技术。

2017 年 10 月，阿里又成立了前沿与基础科学研究机构达摩院，核心研究方向之一就是量子计算。2018 年 5 月，达摩院量子实验室推出量子模拟器“太章”，基于阿里计算平台在线集群超强算力，在世界上率先成功模拟了 81(9×9)比特 40 层作为基准的随机量子电路，超过此前谷歌的 49 比特。

2017 年 1 月，D-Wave 公司推出 D-Wave 2000Q，他们声称该系统由 2000 个 qubit 构成，可以用于求解最优化、网络安全、机器学习、和采样等问题。

科技领域的一些最大参与者，比如谷歌、微软、英特尔、IBM 和阿里巴巴，都正在探索量子计算，以提高网络安全等更多方面的能力，这表明下一场大型计算竞赛已经开始。

2017 年，IBM 证明了在 50 个量子比特上计算的能力。

2018 年，谷歌展示了用 72 个量子比特信息处理。8 月，Rigetti Computing 宣布了制作 128 量子比特量子芯片的计划。

2019 年 GOOGLE 发表在《自然》杂志上的一篇论文中，声称首次获得了“量子霸权”。他们的 53 位量子计算机（名为 Sycamore）花了 200 秒来完成一项计算任务，根据谷歌的说法，这项计算任务将花费世界上最快的超级计算机 10000 年的时间。成就媲美飞机发明人莱特兄弟 1903 年首次 12 秒的飞行。但是以 IBM 公司为代表的“反方”则认为，IBM 研制的一种超级计算机在两天半内就能完成谷歌所展示的运算量，所谓实现“量子霸权”的说法误导公众。

2020 年，美国加州大学洛杉矶分校和澳大利亚新南威尔士大学两个物理研究小组将量子计算平台运行温度提至一开尔文以上。成功让硅基量子计算平台在超过 1 开尔文（1.5 开）的温度下运行。被认为是该领域一个重要里程碑，对于将现有原型扩展至更大更强的量子计算机来说，研究结果将能带来极大影响。一般来说，固态平台需要在约 0.1 开尔文（零下 273.05 摄氏度）的温度下运转，维持这样的运转需要非常昂贵的制冷技术。

现在全球都在参与量子计算的研究攻关和竞赛。量子计算机的研制目标分为三个阶段：

第一个阶段就是“量子霸权”阶段，所谓“量子霸权”是指，当量子计算机发展到 50 个比特时，计算能力将超越全球最快的传统计算机，实现“称霸”。目前美国在“量子霸权”的争夺中暂时领先，我国预计在 2020 年左右实现“量子霸权”的科学目标，缩小与美国的差距。

第二个阶段是实用化量子模拟机阶段，实现数百个量子比特相干操纵的专用型量子计算系统，应用于具有实用价值的组合优化、量子化学、机器学习等方面，指导新材料设计、药物开发等。

第三个阶段是通用可编程的量子计算机阶段，能够相干操纵数亿量子比特，实现可容错的量子计算机，能在经典密码破解、大数据搜索、气象预测、卫生制药、人工智能等方面发挥巨大作用。

尽管量子计算具有美好的前景和强大的计算能力，但是要造出一台通用的量子计算机还非常困难。一台真正通用的量子计算机可能会使用超过 10 万个量子比特，一些人估计会达到 100 万个量子比特。而截至目前，科技工作者能做到的最多的量子比特甚至不到 128 个。量子计算机一旦实现，会成为人类历史上最伟大的科学技术成就之一。量子计算机的研究任重道远，目前仍然处于探索投资烧钱研发攻关阶段。

接下来，就给大家介绍一下量子计算的原理和相关的算法。由于量子计算机的理论、制造与实现是基于量子力学理论的，所以我们先给大家介绍一下什么是量子，以及量子力学的核心观点。

### 4.4.2 什么是量子？

物理学领域，科学家把微观世界中的所有的微观粒子，比如光子，电子和原子等统称为量子。实际上，在物理学的标准模型中一共描述了 61 种（不包括假说中的引力子）基本微观粒子。其中包括 36 种夸克，12 种轻子（其中包括电子、正电子即电子的反粒子、μ子、反μ子、τ子、反τ子、电子中微子、反电子中微子、μ子中微子、反μ子中微子、τ子中微子、反τ子中微子），13 种规范玻色子（其中包括 8 种胶子，互为反粒子的 W-和 W+玻色子，Z 玻色子，光子和上帝粒子希格斯（Higgs）玻色子），共有 36+12+13=61 种基本粒子。

量子具有很多宏观世界无法解释的特性，包括量子态叠加性，量子态纠缠，量子态不可克隆，量子“波粒二象性”，以及量子相干性等。

基于量子的量子态叠加及相干性可以实现量子计算；基于量子态纠缠和不可克隆性可以实现量子通信。

### 4.4.3 量子力学核心观点

量子力学创立于 20 世纪初，是研究物质世界微观粒子运动规律的物理学分支，主要研究原子、分子、凝聚态物质，以及原子核和基本粒子的结构、性质的基础理论。量子力学不仅是现代物理学的基础理论之一，而且在许多近代技术中得到广泛应用，其中量子计算理论也是基于量子力学中的各种奇特效应的。

现在我们就为大家概括介绍一下量子力学中的一些核心观点。这对于我们理解量子计算理论是有很大的帮助的。

#### 1.概率波

在 1920 年代与 1930 年代，德布罗意和薛定谔等使用数学工具微积分，共同创建了波动力学。海森堡和玻恩等科学家使用线性代数建立了矩阵力学。后来，薛定谔证明这两种方法完全等价。

薛定谔参考威廉·哈密顿先前关于牛顿力学与光学之间的类比这方面的研究，利用微积分，从哈密顿-雅可比方程、德布罗意方程，最后求出了一个非相对论（粒子速度不太大，不包含粒子自旋）的薛定谔方程。薛定谔方程能够正确地描述波函数的量子行为。1926 年，玻恩提出概率幅的概念，成功地解释了薛定谔方程中波函数的物理意义，波恩也因此而获得了诺贝尔奖。薛定谔方程如下所示：

$$\hat{H}\Psi = i\hbar\frac{\partial}{\partial t}\Psi$$

其中，$\hat{H}$是哈密顿算子。该方程是一个将物质波的概念和波动方程相结合建立的二阶偏微分方程，可描述微观粒子的运动，每个微观系统都有一个相应的薛定谔方程式，通过解方程可得到波函数的具体形式以及对应的能量，从而了解微观系统的性质。薛定谔方程表明量子力学中，粒子以概率的方式出现，具有不确定性。

薛定谔方程名震 20 世纪物理史，被誉为“十大经典公式”之一 。薛定谔认为电子是

一种波，就像云彩一般（电子云说法的由来，如图 4-4 所示），放大来看后，就好像在空间里融化开来，变成无数振动的叠加。

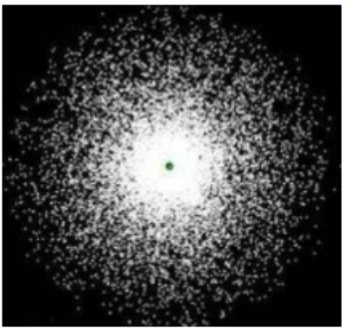


图 4-4 电子云

函数 ψ 是一个复数函数，是电子电荷在空间中的实际分布波函数。量子事件发生的概率可以用波函数来计算，其概率幅的绝对值平方就是量子事件发生的概率密度

一般来讲，波函数是空间和时间的函数，并且是复函数，即 Ψ=Ψ（x，y，z，t），其中 x，y，z 是空间坐标，t 为时间。将爱因斯坦的“鬼场”和光子存在的概率之间的关系加以推广，玻恩假定 Ψ*Ψ 乘积就是粒子的概率密度，即在时刻 t，在点（x,y,z）附近单位体积内发现粒子的概率。波函数 Ψ 的绝对值的平方因此就称为概率幅。在一维空间（只有 x 坐标）下概率密度函数如下：

$$P(x,t) = \Psi^*(x,t)\Psi(x,t) = |\psi_E(x)|^2.$$

采取概率的形式描述微观粒子的运动规律，这是微观粒子的本质属性。微观粒子构造出了大千世界，人类目前只能通过概率的形式了解微观粒子的运动，就好像上帝在掷骰子一样，只能靠猜。这使得伟大的科学家爱因斯坦在一开始也惊呼：“无论如何，我都确信，上帝不会掷骰子。”

## 2.波粒二象性

波粒二象性（wave-particle duality），是指所有的粒子或量子不仅可以部分地以粒子的术语来描述，也可以部分地用波的术语来描述。波粒二象性是微观粒子的基本属性之一。1905 年，爱因斯坦提出了光电效应的光量子解释，人们开始意识到光波同时具有波和粒子的双重性质。1924 年，德布罗意提出“物质波”假说，认为和光一样，一切微观粒子，包括电子和质子、中子，都具有波粒二象性。并且最终通过电子双逢干涉实验（电子具有波的特性）得到证实。

电子双逢干涉实验是一个让人感觉到很“诡异”的实验。实验中，用电子枪射出电子通过一条缝隙，后墙出现一条线。当开了两条缝隙时候，出现了干涉条纹现象。当两条缝开启时，电子枪单个射出电子，其间间隔足够长的时间，最后得到的电子分布依然是干涉图像，好像是先到的电子“规定”后到的电子的行为。因此不仅要问：单个电子通过双逢的时候和谁干涉呢？还是说电子同时通过了两个缝隙和自己发生了干涉？

如图 4-5 所示，更诡异的是：当用摄像机试图看着电子通过双逢的时候，干涉条纹竟然消失了。不看的时候，干涉条纹又出现了。电子居然知道我们在看它？如果在电子通过双缝之前不去做任何观测，等到电子通过了“双缝”之后，再马上打开观测设备进行观测，这种情况下，即使是电子已经通过了“双缝”，它还是会根据观测者的行为，来决定自己的状态。

即存在观测者的时候，它们是“粒子性”，而假如对它们不做观测，它们又呈现出“波动性”。

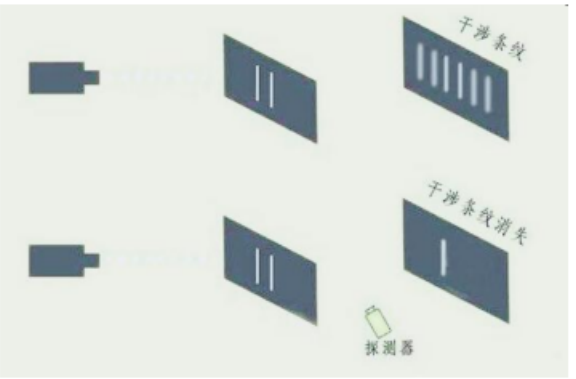



图 4- 5 电子双逢干涉实验

一个电子会同时通过两条缝隙吗？以目前大家的认知，电子当然是不能同时通过两个缝隙的。因此一个电子只可能通过自相互作用发生干涉，只是很难观测到。经典的观测装置，会让量子的波粒叠加态发生坍塌从而表现为粒子性。

波动性和粒子性这两种属性既对立又互补，一个实验中具体展示哪种属性取决于实验装置。在经典的实验中，探测光的波动性和粒子性的实验装置，对光的波动性和粒子性不能够同时展现出来。

2012 年中科大郭光灿院士领导的中科院量子信息重点实验室李传锋研究组首次设计出了量子实验装置，通过巧妙地控制测量装置，使其处于探测波动性与探测粒子性的两种对立状态的量子叠加态上，从而制备出了光子的粒子和波的叠加状态。光子可以在波动和粒子两种行为之间相干地振荡。

实验结果显示，处于波粒叠加态上的光子，既不象普通的粒子态那样没有干涉条纹，也不象普通的波动态那样表现出标准的干涉条纹，而是展现出锯齿形条纹这样一种“非波非粒，亦波亦粒”的表现形式。

## 3.叠加原理

在量子力学中，如果$\Psi_1$和$\Psi_2$是体系的可能状态，那么，它们的线性叠加

$$\Psi = c_1\Psi_1 + c_2\Psi_2 (c_1, c_2 \text{ 是常数})$$

也是这个体系的一个可能状态，这就是量子力学中的态叠加原理。

叠加原理是量子力学中的一个基本原理，假若一个量子系统的量子态可以是几种不同量子态中的任意一种，则它们的归一化线性组合(即线性组合的系数 $c_1, c_2, \ldots, c_n$ 满足 $\sum_n |c_n|^2 = 1$)也可以是其量子态，我们称这种线性组合为“叠加态”。假设组成叠加态的几种量子态相互正交（即设两种量子态分别为 $c_{i1}, c_{i2}, \ldots, c_{in}$ 和 $c_{j1}, c_{j2}, \ldots, c_{jn}$，那么 $c_{i1}c_{j1}+c_{i2}c_{j2}+\ldots+c_{in}c_{jn}=0$），则该量子系统处于其中任意量子态的概率是对应权值的绝对值平方。

由于量子态具有叠加性，因此单次测量不能确定一个量子态，因为一次测量，量子状态已经坍缩，不可能对它进行重复测量，也不能将这个量子态复制出来进行反复测量，这就是量子态的不可克隆性。它是信息理论和量子密码安全的重要基础和保障。

## 4.量子相干性

量子相干性，也叫"态之间的关联性"，是爱因斯坦和其合作者在1935年根据假想实验作出的一个预言。这个假想实验是这样的：在高能加速器中，由能量生成的一个电子和一个正电子朝着相反的方向飞行，在没有人观测时，两者都处于向右和向左自旋的叠加态，而进行观测时，如果观测到电子处于向右自旋的状态，那么正电子就一定处于向左自旋的状态。这是因为，正电子和电子本是通过能量无中生有而来，必须遵守守恒定律。这也就是说，“电子向右自旋”和“正电子向左自旋”的状态是相关联的，称作“量子相干性”。

在量子计算机中实现高效率的并行运算，就要用到量子相干性。彼此有关的量子比特串列，会作为一个整体动作。因此，只要对一个量子比特进行处理，影响就会立即传送到串列中多余的量子比特。这一特点，正是量子计算机能够进行高速运算的关键。

## 5.量子纠缠

量子纠缠描述的是两个暂时耦合（彼此相互作用）的粒子，不再耦合之后彼此之间仍旧维持的关联。假设一个零自旋中性π介子衰变成一个电子与一个正电子。这两个衰变产物各自朝着相反方向移动。电子移动到区域A，在那里的观察者甲会观测到电子沿着某特定轴向自旋；正电子移动到区域B，在那里的观察者乙也会观测到正电子沿着同样轴向自旋。

在测量之前，这两个纠缠粒子共同形成了零自旋的“纠缠态”。电子的自旋为上旋当且仅当正电子的自旋为下旋；电子的自旋为下旋当且仅当正电子的自旋为上旋。两种状况叠加在一起，每一种状况都有可能发生，不能确定到底哪种状况会发生。因此，电子与正电子纠缠在一起，形成纠缠态。

假若不做测量，则无法知道这两个粒子中任何一个粒子的自旋。对于两个粒子的自旋分别做测量，假若电子的自旋为上旋，则正电子的自旋为下旋，反之亦然；假若电子的自旋下旋，则正电子自旋为上旋，反之亦然。

## 6.测不准原理

1927年海森堡提出“测不准原理（不确定性原理）”。测不准原理指出，你不可能同时测量出一个粒子的位置和它的速度。

比如，要想测定一个量子的精确位置，那么就需要用波长尽量短的光波（因为不可能将粒子的位置确定到比光的两个波峰之间的距离更小的程度），这样的话，光波的频率就会很高，能量就会更大，对这个量子的扰动也会越大，对它的速度测量也会越不精确；如果想要精确测量一个量子的速度，那就要用波长较长的波，那就不能精确测定它的位置了。

位置的不确定性与动量的不确定性遵守不等式：$\Delta x \Delta p \geq h/(4\pi)$，其中h为普朗克常数，26届国际计量大会(CGPM)表决通过为精确数为$h=6.62607015\times10\text{-}34\ J\cdot s$。由于不少公式中常用到$h/2\pi$这个数，为避免反复写$2\pi$，引用另一个常用的量为约化普朗克常数（reduced Planck constant），有时称为狄拉克常数(Dirac constant)$\hbar=h/(2\pi)$。约化普朗克常量（又称合理化普朗克常量）是角动量的最小衡量单位。其中π为圆周率常数，约等于3.14，ħ(h上有一条斜杠)读着 "h 拔"。

不确定性原理的维持必须倚赖量子纠缠机制。例如，设想先前的一个零自旋中性π介子衰变案例，两个衰变产物各自朝着相反方向移动，分别测量电子的位置与正电子的动量，假

若量子纠缠机制不存在，则可借着守恒定律预测两个粒子各自的位置与动量（参考下面的EPR 佯谬），这违反了不确定性原理。由于量子纠缠机制的存在，粒子的位置与动量就会遵守不确定性原理。

## 7.薛定谔的猫

薛定谔的猫是奥地利著名物理学家薛定谔提出的一个思想实验，是指将一只猫关在装有少量镭和氰化物的密闭容器里，氰化物是剧毒物质。

镭的衰变存在几率，如果镭发生衰变，会触发机关打碎装有氰化物的瓶子，猫就会被毒死；如果镭不发生衰变，猫就存活。根据量子力学理论，由于放射性的镭处于衰变和没有衰变两种状态的叠加，猫就理应处于死猫和活猫的叠加状态。这只既死又活的猫就是所谓的“薛定谔猫”。但常识告诉我们，是不可能存在即死又活的猫，必须在打开箱子后才知道结果 。该实验试图从宏观尺度阐述微观尺度的量子叠加原理的问题。

根据经典物理学，在盒子里必将发生这两个结果之一，而外部观测者只有打开盒子才能知道里面的结果。在量子的微观世界里，当盒子处于关闭状态，整个系统则一直保持不确定性的波态，即猫生死叠加。猫到底是死是活必须在盒子打开后，外部观测者观测时，物质以粒子形式表现后才能确定。这项实验旨在论证量子力学对微观粒子世界超乎常理的认识和理解，通过这个思想实验，使微观不确定原理变成了宏观不确定原理

## 8.EPR 佯谬

EPR 佯谬是 E（爱因斯坦）、P（波多尔斯基）和 R（罗森）在 1935 年为论证量子力学的不完备性而提出的一个悖论（佯谬）。EPR 是这三位物理学家姓氏的首字母缩写。EPR 佯谬的具体内容如下：

假设一个零自旋中性 π 介子衰变成一个电子与一个正电子，它们处于量子纠缠态，然后把这两个衰变产物各自朝着相反方向移动至超级远的星际空间 A 点（放电子）、B 点（放正电子）。A，B 两点的距离如此之大以至于对其中一个粒子（正电子或电子）的任何操作都不会对另外一个产生干扰。

假设 A 点的位置坐标为 $x_1$，B 点的位置坐标为 $x_2$，两者之间的距离 $d=x_1-x_2$。如果在 A 点测得电子位置为 x，那么在 B 点正电子的位置为 x-d。假设测得 B 点正电子的动量为 p，则 A 点的电子动量为-p，那么就完成了对电子和正电子两个粒子位置和动量的测量。

但测不准原理却告诉我们，这是不可能的。因此意味着在测量 A 点电子时候，B 点的正电子的动量也不能精确测量了。这就意味着：要么存在超距离作用，使得在测量 A 处的电子时候，立即干扰了 B 处正电子的动量；要么一个粒子的位置和动量是可以同时精确测量的，只是量子力学不够完备。

显然爱因斯坦等人坚持认为量子力学不够完备。但波尔认为，粒子之间，无论多远，都存在着量子关联（即量子纠缠），对其中一个粒子实行局域操作，必然会导致另一个粒子状态改变，即量子力学的非局域性。

后来科学家通过相关实验，支持了波尔的理论。1997 年瑞士学者直接在 10 千米的光纤中测量到作为 EPR 对的两个光子之间的量子关联（即这两个光子是处于纠缠态的一对粒子）。2017 年 8 月，中国"墨子号"卫星与地面率先成功实现"千公里级"的星地双向量子纠缠。由于环境噪声，量子纠缠态的纠缠性会随距离的增加而变差，如何保持量子纠缠态的纯度是量子通信研究中的一大难题。

### 4.4.4 量子比特

构成经典计算机内信息的最小单位“比特（bit）”是用二进制中的 0 和 1 表示，所有的信息都是由 0 和 1 组成，保存，运算以及传递的。物理上，比特是由一个实际物理系统来实现的。比如开关中的开代表 1，关代表 0；也可以用光纤中的光脉冲，磁带中的磁化性质等来实现。如图 4-6 所示，在传统计算机里，0 和 1 是由电位的高低来表示，这种用传统比特存储和处理信息的手法称为经典信息。

相对于经典信息的基本存储单元比特，量子信息的基本存储单元称为量子比特（qbit 或 qubit）。如果我们用量子力学中光子的两个极化状态，或电子的两个自旋状态，或原子的基态和激发态来实现信息中的 0 与 1，这样的比特称为量子比特，用量子比特来存储和处理信息，则称为量子信息。

一个量子比特的状态是一个二维复数空间的向量，它的两个极化态|0>（迪拉克标记，bra（左矢向量<|）和 ket(右矢向量|>）和|1>对应于经典状态的 0 和 1。

量子比特的重要特性在于一个量子比特可以连续地，随机地存在于状态|0>和|1>的任意叠加状态上。

一个量子比特能够处在既不是|0>又不是|1>的状态上，而是处于它们之间的一个线性组合的所谓中间状态之上，即处于状态|0>和|1>的叠加态上。

$$|\Psi\rangle=\alpha|0\rangle+\beta|1\rangle$$

这里的 α 和 β 为任意复数，且必须满足归一化要求，即 α α*+β β*=1（α*和 β*分别表示 α 和 β 的共轭复数）。处于两种状态|0>和|1>叠加态的粒子系统就是量子信息的基本存储单元-量子比特。

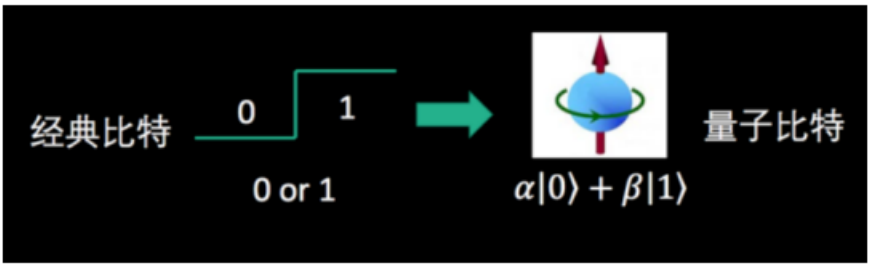



图 4-6 经典比特与量子比特

量子态的叠加性源于微观粒子“波粒二象性”的波动“相干叠加性”。量子纠缠状态指的是两个或多个量子系统之间非定域，非经典的关联，是量子系统内各子系统之间关联的力学属性。量子态可叠加的物理特性是实现量子并行计算的基础，量子态纠缠是实现信息高速的不可破译通信的理论基础，它们都是量子信息理论中特有的概念。

比如，对于十进制的 10（二进制位 1010）和 5（二进制位 0101），若用量子比特来表示，则分别可以写成：

$$|10\rangle_{10}\equiv|1010\rangle=|1\rangle\otimes|0\rangle\otimes|1\rangle\otimes|0\rangle$$

$$|5\rangle_{10}\equiv|0101\rangle=|0\rangle\otimes|1\rangle\otimes|0\rangle\otimes|1\rangle$$

取它们的叠加态就得到如下表示：

$$|1\rangle\otimes|0\rangle\otimes|1\rangle\otimes|0\rangle+|0\rangle\otimes|1\rangle\otimes|0\rangle\otimes|1\rangle$$

针对它们的叠加态可以利用量子算法同时处理十进制整数的 10 和 5。显然，状态的各

量子位就是纠缠状态，可以对这个叠加状态实施各种运算，其结果如同在同时对 10 和 5 进行计算，最后通过测量即可分别获得 10 和 5 的计算结果，实现两个数物理上的并行计算。

更进一步，如果打算同时计算一个函数 f(x)在 $x=x_1, x_2, ..., x_n$ 一系列位置上的取值，也可以取得更复杂的纠缠态。例如，设 x 和 y=f(x)为两个存储器，他们的量子态分别为|x>>和|f(x)>>，则下列纠缠态就包含了该函数整体上的信息：

$$\sum_{i=1}^{n} |x_i\rangle\rangle \otimes |f(x_i)\rangle\rangle = |x_1\rangle\rangle \otimes |f(x_1)\rangle\rangle + |x_2\rangle\rangle \otimes |f(x_2)\rangle\rangle + \cdots + |x_n\rangle\rangle \otimes |f(x_n)\rangle\rangle$$

对它尝试进行各种运算，就如同并行计算一个函数 f(x)在 $x=x_1, x_2, ..., x_n$ 一系列位置上的函数值。由此可见，量子叠加态是实现真正物理意义上并行计算的物质基础。

量子信息与经典信息最大的不同在于：经典信息中，比特只能处在一个状态，非 0 即 1。而在量子信息中，量子比特可以同时处在状态|0>和|1>中。量子比特的这一特性来自量子力学的状态叠加原理，即如果状态|0>和|1>是两个相互独立的量子态，他们的任意线性叠加|φ>=α|0>+β|1>也是某一时刻的一个量子态，而系数 α 和 β 的绝对值的平方则描述系统分别处在|0>和|1>的概率。这使得每个量子比特可表示的信息比经典比特多得多。

量子比特能利用不同的量子叠加态记录不同的信息，在同一位置上可拥有不同的信息。然而量子态是非常不稳定的，并且根据量子力学的测不准原理，任何观测都会立刻改变系统的状态，因此量子信息的实际可行性一直受到怀疑。但这恰恰保证了量子信息的绝对安全性，因为任何窃听（测量）都会被发现。

量子 qbit 的两个极化状态|0>和|1>也是二维复数列向量，它们构成二维复数空间的一对正交基底（两个夹角为 90 度，模均为一个单位的一对向量，可以用来表示其它向量）。即：

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$$

$$|\varphi\rangle = \alpha|0\rangle + \beta|1\rangle = \alpha\begin{bmatrix} 1 \\ 0 \end{bmatrix} + \beta\begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}$$

通过一个测定或观测的过程，可以把一个 qbit 的状态以概率幅的方式变换成 bit 信息。也就是通过特殊的测定，量子比特|φ>将以下列方式被转换：

- 概率$|\langle 0|\varphi\rangle|^2$ 变换成 bit 0　<x|y>表示 x 与 y 的内积
- 概率$|\langle 1|\varphi\rangle|^2$ 变换成 bit 1

其中：

- $\langle 0|\varphi\rangle = \alpha\langle 0|0\rangle + \beta\langle 0|1\rangle = \alpha$
- $\langle 1|\varphi\rangle = \alpha\langle 1|0\rangle + \beta\langle 1|1\rangle = \beta$

即|φ>以概率$|\alpha|^2$取值 bit 0；以概率$|\beta|^2$取值 bit 1。

今天的计算机，本质上都是图灵机。它读入数据，按算法处理数据，输出结果。计算机用二进制处理信息。1 个二进制数是 1 个比特（bit）。传统计算机中 1 个比特取值只能是 0 和 1，即开关电路的开或关。10 个比特能记录 1 个 10 位的二进制数。

在量子计算机里，1 个量子比特同时记录 0 和 1。10 个量子比特记录的不再是 1 个 10 位二进制数了，因为每个比特都处在 0 和 1 的叠加态，10 个量子比特记录的是 $2^{10}$ 的二进制数的叠加。传统的 10 个比特只能处理 1 个 10 位的二进制数，10 个量子比特能同时处理 $2^{10}$ 个二进制数。

### 4.4.5 量子并行计算

中国有一个寓言，叫“杨子见歧路而哭之”。这个寓言故事讲的是古代一个名叫杨朱的人听说有一只羊在道路分叉的地方走失了，不知道走哪条路去寻找，难过的哭了。

传统计算机解答问题的时候和杨子寻找丢失的羊一样，即只能是先走一条路，然后再走另外一条路，做不到两条路一起走。量子计算机则不一样了，它支持并行计算，可以像孙悟空变出很多个小孙悟空，走不同的路同时去寻找。也就是一台量子计算机，相当于千千万台台计算器，同时开工做计算题。

如图 4-7 所示，想象一串原子排列在一个磁场中，以相同的方式旋转。如果用一束激光照射在这串原子上方，激光束会影响这组原子，迅速翻转一些原子的旋转轴。通过测量进入的和离开的激光束的差异，我们就可以测量出原子旋转轴的变化，因此完成了一次复杂的量子计算，并通过测量，获得这次计算的结果。

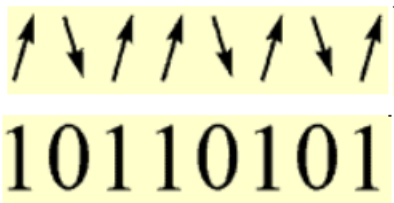



图 4-7 量子叠加态测量结果

从数学抽象上看，量子计算机执行以集合为基本运算单元的计算，普通计算机执行以元素为基本运算单元的计算，当然如果集合中只有一个元素，那么量子计算与经典计算没有区别。

以函数 y=f(x)，x∈A 为例。量子计算的输入参数是定义域 A，一步到位得到输出值域 B，即 B=f(A)；经典计算的输入参数是 x，得到输出值 y，要多次计算才能得到值域 B，即 y=f(x)，x∈A，y∈B。

量子计算机有一个待解决的问题，即输出值域 B 只能随机取出一个有效值 y。虽然通过将不希望的输出导向空集的方法，已使输出集 B 中的元素远少于输入集 A 中的元素，但当需要取出全部有效值时仍需要多次计算。

简单地说，量子计算就是利用量子态进行信息处理的方法，其实体设备称为量子计算机。量子计算机的基本原理就是通过量子力学的运用，将微晶体管压缩到原子般大小，然后在极小的面积上放入数十亿颗量子微晶体管，进而利用量子态的叠加性和相干性进行信息运算，保存和处理。

在传统计算机中，运算对象是各种比特序列，在量子计算机中，运算对象是量子比特序列，所不同的是，量子比特序列可以处在各种正交态的叠加态上。以一个 3 比特组成的序列为例，可以用 8 个三位二进制表示，即 000, 001, 010, ..., 111，分别代表 0 到 7 这 8 个数字。由这三位比特序列构成的经典存储器每次只能记录这 8 个数字中的一个。但量子存储器可以在同一时刻以量子叠加态同时记录这 8 个不同的数字，这意味着用更多的量子比特组成的存储器，其存储信息的能力将呈指数增加。4 个量子比特可同时存储 16 个不同的数字，n 个量子比特可同时存储 $2^n$ 个数字。即在相同比特位数下，量子计算机记录信息的容量是目前传统计算机的 $2^n$ 倍。

另一方面，计算机运算是由逻辑门电路做基本组件的，而量子计算机则由量子逻辑门电路构成其运算组件。量子逻辑门不仅可以将|0>态和|1>态做变换，还可以将|0>态和|1>态变为它们的任意叠加态。更为关键的是，量子计算会将存储器内的量子比特变换为纠缠态。比

如两个量子比特可构成纠缠态（|00>+|11>），其特性是它不能被分解为两个单独量子比特状态的乘积。因此，纠缠态内量子比特间具有很强的相干性或关联性，其中一个量子比特状态被改变或测量时，也决定了纠缠态内所有其他比特状态相应变化，这类特殊量子态提供了量子并行处理的可行性。量子并行处理就是对量子态每一叠加分量进行幺正变换，所有这些变换在同一时刻一次完成，并按一定的概率叠加起来得出结果。一台 32 个量子比特的计算机，其能力相当于 40 亿台传统计算机作并行运算。

不管是量子并行计算还是量子模拟计算，本质上都是利用量子纠缠态特有的相干性。但在实际系统中，量子纠缠态很难维持。在量子计算机中，由于量子比特是由原子或其他微粒子系统所构成，很容易受外部环境影响，导致量子相干性消失，称为退相干，从而使运算容易产生错误结果。要使量子计算机成为现实，最重要的问题就是克服这种退相干，其最有效办法就是在发生退相干前完成运算，或用误差修正的方法消去因退相干引起的错误。前者依赖于纠缠态的寿命，一般说来，一个量子信息由产生到消失的时间只有十亿分之一秒，而后者是同时做几种相同的运算，并不断对相应的状态做比较，发生偏差时及时修正。

虽然量子计算机实现计算的量子比特越多，运算能力就越强，而且截止目前人类也只是实现了不到 100 量子比特的计算，而通用的量子计算机需要几百万甚至上亿量子比特。但是由于量子纠缠的不稳定性，因此增加一定量的量子比特，就需要增加额外的量子比特来容错。按当前技术水平，当量子计算机的量子比特规模增大到足以进行实际模拟任务时，用来容错的物理量子位与实际发挥作用的逻辑量子位之比将高达 3000:1，这在目前显然是不现实的。

量子并行计算的能力来自于量子态的可叠加性，是量子信息理论应用的一个重要分支。量子计算机对每一个叠加分量实现的变换相当于一种经典计算，所有这些经典计算同时完成，并按一定的概率振幅叠加起来，最终给出量子计算机的输出结果，以这种方式实现的信息处理叫量子并行处理。量子并行处理大大提高了量子计算机的效率，使其可以完成经典计算机很难完成的工作，例如大自然数因式分解。

### 4.4.5 量子计算对 RSA 密码破解，Shor 算法

量子计算可以把其中的一部分 NP 问题变成 P 问题，即问题的复杂度随着比特位数的增长降为以多项式数量级上升。因此这类问题从理论上变成了可计算问题。

一个具体的例子就是大整数的因式分解。按经典计算复杂度理论，这个问题不存在有效算法，是一个数学难题，所以被利用来进行经典密钥分配。

截止目前，最快的计算机能够每秒完成 $10^{11}$ 次运算。计入对一个 60 位的大整数 $10^{60}$ 进行因式分解，需要 $10^{30}$ 次运算，那么大约需要 $3*10^{11}$ 年。

1994 年美国计算机科学家彼得·秀尔（Peter Williston Shor，如图 4-8 所示）等人提出了一种大因数分解的量子多项式算法，即 Shor 算法（也叫秀儿或者舒尔算法），引起了轰动，核心是利用数论中的一些定理，将大数因子分解转化为求某个函数的周期。

如果用量子计算机结合 Shor 量子算法，大整数的因式分解问题就变成了 P 问题，原来计算需要上万年的时间，现在利用量子计算方法，就变成了几分钟即可完成。经典计算机中的 RSA 公钥体系就是利用两个大素数的乘积难以分解实现加密的，被利用来进行经典密钥分配。量子计算的出现，使得目前基于 RSA 的密码系统的安全性荡然无存，那么对于军事，金融等行业的信息安全构成了严峻的挑战。

现在我们来介绍一下这个 Shor 算法的基本原理与实现步骤。

图 4-8 秀尔算法提出者

设整数 $N=n_1 \cdot n_2$，其中 $n_1$、$n_2$ 为两个奇素数（所谓奇素数，就是指大于 2 的素数），对 N 进行因式分解从而得到 $n_1$、$n_2$（如果用一般算法，这个问题复杂度会很高）的 Shor 算法概述如下：

1，任取整数 a，使得 $1<a<N$，计算 a 和 N 的最大公约数 gcd(a, N)；

2，若 gcd(a, N)>1，得到因子，问题解决；否则，设函数为：

$$f(x)=a^x \bmod N$$

这个函数会出现周期性的变化规律，即是周期函数。找到函数 f(x)的周期 r，使得：

$$f(x+r)=f(x);$$

3，如果 r 是奇数，则返回步骤 1，重新计算；

4，如果 r 是偶数，那么：

由于：

$$a^r \equiv 1 \bmod N$$（因为 $a^r=f(r+0)=f(0)=(a^0 \bmod N)=(1 \bmod N)=1$）

注意，这里“≡”是同余符号，表示 $a^r$ 和 1 除以 N，余数相等。

所以：

$$a^r-1 \equiv 0 \bmod N$$

即：

$$(a^{r/2}-1)(a^{r/2}+1)=tN=tn_1n_2$$ （此步利用了平方差公式：$a^2-1=(a+1)(a-1)$）

设 $t=t_1t_2$，那么：

$$(a^{r/2}-1)(a^{r/2}+1)=tN=tn_1n_2=(t_1n_1)(t_2n_2)$$

5，根据上式有两种情形，第一种情形为：

$$(a^{r/2}+1) \equiv 0 \bmod n_1, \quad (a^{r/2}-1) \equiv 0 \bmod n_2$$

解为：

$$\mathbf{n_1=gcd(a^{r/2}+1, N), \quad n_2=gcd(a^{r/2}-1, N)}$$

或者第二种情形：

$$(a^{r/2}+1) \equiv 0 \bmod n_2, \quad (a^{r/2}-1) \equiv 0 \bmod n_1$$

解为：

$$\mathbf{n_1=gcd(a^{r/2}-1, N), \quad n_2=gcd(a^{r/2}+1, N)}$$

这样我们就得到了大整数因式分解之后的 $n_1$ 和 $n_2$。这里关键是要求得通过整数 a 构造出来的函数的周期 r。只要得到了函数的周期 r，问题就转化为求 $\gcd(a^{r/2}-1, N)$和 $\gcd(a^{r/2}+1, N)$。

上面就是 Shor 算法的核心逻辑。下面我们举一个实际的例子，应用 Shor 算法来对整数

进行因式分解。

比如，我们要对 35 进行因式分解，那么应用 Shor 算法有：

1，任选一个小于 35 的整数，比如 2；

2，gcd(2,35)=1，设：$f(x)=2^x \bmod 35$，现在要求 f(x)的周期，那么:

x=0, f(x)=1

x=1 , f(x)=2

x=3, f(x)= 8

x=4 , f(x)=16

x=5 , f(x)=32

x=6 , f(x)=29

x=7 , f(x)=23

x=8 , f(x)=11

x=9 , f(x)=22

x=10 , f(x)=9

x=11 , f(x)=18

x=12 , f(x)=1

x=13 , f(x)=2

3，通过 2 中的计算，观察可以知：f(x)=f(x+12)，即 f(x)的周期 r=12，r/2=6;

4，根据 Shor 算法的第 5 步，计算 $\gcd(2^6+1, 35)$, $\gcd(2^6-1,35)$有：

$$\gcd(65, 35)=5, \quad \gcd(63, 35)=7$$

所以 5*7=35，因此，结合 Shor 算法，35 被成功因式分解为 5*7。

Shor 算法的主要计算体现在辗转相除，计算 f(x)和求 f(x)的周期上。其中辗转相除的复杂度为 $O(n^2)$，求 f(x)的周期中用到的量子傅里叶变换为 O(lgn)，所以 Shor 算法的事件复杂度为 **$O(n^2(lgn)(lglgn))$** 。

Shor 算法是一种随机算法，实验次数越多，成功概率越高。其中，获得周期 r 是关键，而我们可以使用量子计算来获取 r。通过利用量子存储以及量子并行计算，可以同时得到所有可能状态模数运算后的函数值，对函数值进行测量后利用量子纠缠得到函数值所对应的所有原像（自变量）的叠加态。最后利用量子傅里叶变换得到周期。

1994 年舒尔算法被提出以来，有许多物理实验用来验证舒尔算法的正确性。2001 年，量子计算的开拓者之一 Chuang 第一次以实验的方式实现了 Shor 算法的主要原理验证。实验中成功分解了整数 15。到目前为止，实验上，利用舒尔算法可以分解的最大整数仅为 21。换句话说量子计算机研制依然任重道远。

截止目前，Google 的最新研究表明，量子计算机能在 8 小时内破解 2048 位的 RSA 加密。2012 年物理学家用一台 4qbit 的量子计算机分解了素数 143（11*13），2014 年，用类似的设备分解了 56153。但量子因式分解在实际应用中依然困难重重。因为量子计算中噪声问题需要使用大量的额外量子比特来进行纠错。研究人员曾经估计，要因式分解 2048 位数字，那么需要 10 亿个量子比特才能可靠的完成这项工作。而截止目前最先进的量子计算机只有 70 个量子比特。但现在，Google 等公司的研究人员发现，用 2000 万个量子比特，即可在 8 小时内完成 2048 位数字的分解。他们改进了 Shor 算法中的计算方法，用一种称为幂模运算（将数字提高到某个幂然后除以另一个数找到余数）的方法来执行数学运算。

# 五，数列、程序递归和程序基本算法

数列是数学理论中的经典研究对象，也是我们后续学习高等数学极限，无穷级数等理论的基础，同时，在计算机领域中，它也与数组和程序递归等算法概念息息相关。因此，我们必须学习和了解数列相关理论知识。

## 5.1 数列的定义

那么，什么是数列呢？我们把按照一定顺序排列的一列数称为数列，数列中的每一个数叫做这个数列的项。数列中的每一项都和它的序号有关，排在第一位的数称为这个数列的第 1 项（首项），...，排在第 n 位的数称为这个数列的第 n 项。所以，数列的一般形式可以写成：

$$a_1, a_2, a_3, \ldots, a_n, \ldots$$

并简记为$\{a_n\}$。比如常见的偶数列和奇数列：

偶数列：

$$2, 4, 6, 8, 10, \ldots, 2n, \ldots$$

奇数列：

$$1, 3, 5, 7, 9, 11, \ldots, 2n+1, \ldots$$

项数有限的数列叫有穷数列，项数无限的叫做无穷数列。

如果数列$\{a_n\}$的第 n 项与序号 n 之间的关系可以用一个式子来表示，那么这个公式叫做这个数列的通项公式。知道了通项公式，就可以算出数列的各项。比如：

- $a_n=n$， 1, 2, 3, 4, 5, …
- $a_n=2n-1$ 1, 3, 5, 7, 9, …
- $a_n=2n$ 2, 4, 6, 7, 8, ⋯
- $a_n=（-1）^{(n+1)}+1$ 2, 0, 2, 0, 2, 0, ⋯
- $a_n=2^n$ 2, 4, 8, 16, 32, 64, ⋯
- $a_n=2^{n-1}$ 1, 3, 7, 15, 31, 63, ⋯

其中 n 为自然数。我们知道，2n-1 是奇数数列的通项公式，2n 是偶数数列的通项公式。目前数学家尚未找到素数数列的通项公式，而要解决这个问题，就会涉及到黎曼猜想（参考本书 6.4.5 一节）。

如果数列第 n 项都是它前面一项或者几项的式子，这种给出数列的方法叫做递推法，比如：

- $a_1=1, a_n=2a_{n-1}+1(n>1)$, 1, 3, 7, 15, 31, ⋯
- $a_1=1, a_2=3, a_n=a_{n-1}+a_{n-2}+1\ (n>2)$ 1, 3, 5, 9, 15, 25, ⋯

## 5.2 斐波那契数列

斐波那契数列，又称黄金分割数列，指的是这样一个数列：0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...,

在数学上，斐波那契数列可用递归的方法定义：

$$F(n)=\begin{cases} n & (n=0, 1) \\ F(n-1)+F(n-2) & (n>1，\ n\in N^*) \end{cases}$$

在现代物理、准晶体结构、化学、计算机等领域，斐波那契数列都有直接的应用，为此，美国数学会从 1963 起出版了以《斐波那契数列季刊》为名的一份数学杂志，用于专门刊载这方面的研究成果。

**有关斐波那契的小故事**

斐波那契数列的发明者，是意大利数学家列昂纳多·斐波那契（Leonardo Fibonacci，1170-1250，如图 5-1 所示），籍贯是比萨。他被人称作“比萨的列昂纳多”。1202 年，他撰写了《算经》（Liber Abacci）一书。他是第一个传播和研究印度和阿拉伯数学理论的欧洲人，促进了欧洲数学的发展和进步。他在书中提出了一个关于兔子繁殖的问题。当 n 趋向于无穷大时，前一项与后一项的比值越来越逼近黄金分割 0.618。

图 5-1 斐波那契

斐波那契数列除了递归通项公式，还有一个很优美的通项式如下：

$$a_n = 1/\sqrt{5}\left[\left(\frac{1+\sqrt{5}}{2}\right)^n - \left(\frac{1-\sqrt{5}}{2}\right)^n\right]$$

本章后面，我们将会再次提到斐波那契数列，并以斐波那契数列为引子，给大家介绍程序递归，动态规划等计算机基本算法思想与应用。

## 5.3 等差数列

提到等差数列，就不得不提德国数学家高斯小时候的故事。

高斯（1777-1855，如图 5-2 所示），德国著名数学家、物理学家、天文学家、大地测量学家。是近代数学奠基者之一，高斯被认为是历史上最重要的数学家之一，并享有“数学王子”之称。高斯和阿基米德、牛顿并列为世界三大数学家。他一生成就极为丰硕，以他名字“高斯”命名的成果达 110 个，属数学家中之最。他对数论、代数、统计、分析、微分几何、大地测量学、地球物理学、力学、静电学、天文学、矩阵理论和光学皆有贡献。美中不足的

是，没有一个数学分支，是他独立开创的。

图 5-2 德国数学家高斯

高斯 7 岁那年开始上学。10 岁的时候，他进入了学习数学的班级，这是一个首次创办的班，孩子们在这之前都没有听说过算术这门课。数学教师是布特纳，他对高斯的成长也起了一定作用。

一天，老师布置了一道题，1+2+3……这样从 1 一直加到 100 等于多少。高斯很快就算出了答案，起初高斯的老师布特纳并不相信高斯算出了正确答案:"你一定是算错了，回去再算算。"高斯非常坚定，说出答案就是 5050。高斯是这样算的:1+100=101，2+99=101……50+51=101。从 1 加到 100 有 50 组这样的数，所以 50*101=5050。

布特纳对他刮目相看。他特意从汉堡买了最好的算术书送给高斯，说:"你已经超过了我，我没有什么东西可以教你了。"接着，高斯与布特纳的助手巴特尔斯建立了真诚的友谊，直到巴特尔斯逝世。他们一起学习，互相帮助，高斯由此开始了真正的数学研究。

那么什么是等差数列呢？一般地，如果一个数列从第 2 项起，每一项与它的前一项的差等于同一个常数，那么这个数列就叫做等差数列（arithmetic sequence），这个常数叫做等差数列的公差（common difference），公差通常用字母 d 表示。由三个数 a，A，b 组成的等差数列可以看成最简单的等差数列，这时，A 叫做 a 与 b 的等差中项，并且根据定义有 A=a+d，A=b-d，所以 A=(a+b)/2。

根据定义，很容易得出等差数列的通项公式为：

$$a_n=a_1+（n-1）d$$

比如，对于一个公差为-3 的等差数列：8, 5, 2, …，它的第 10 项为：8+(10-1)*-3=-19。

高斯给出了 1 到 100 的求和方法，这实际上是一个等差数列求和的问题。那么等差数列如何求前 n 项和呢？现在我们来研究一下。我们用 $S_n$ 表示等比数列的前 n 项和，那么有：

$$S_n=a_1+a_2+a_3+…+a_n$$

根据高斯算法的启发，那么等差数列前 n 项和可以表示为：

$S_n=a_1+(a_1+d)+(a_1+2d)+…+(a_1+(n-1)d)$ ①

同时，也可以表示为：

$S_n=a_n+(a_n-d)+(a_n-2d)+...+(a_n-(n-1)d)$ ②

①+②得：

$2\ S_n=(a_1+a_n)+(a_1+a_n)+...+(a_1+a_n)=n(a_1+a_n)$，①和②中对应的项正负 d 抵消（比如 d 和-d，2d 和-2d 等），剩余 n 个$(a_1+a_n)$

于是有：

$$S_n = \frac{n(a_1 + a_n)}{2}$$

又因为 $a_n=a_1+(n-1)d$，代入上式，得：

$$S_n = na_1 + \frac{n(n-1)}{2}d$$

于是我们得到了两种求等差数列之和的公式形式。第一个可以用于知道第一项和最后一项的情况；另一个可以用于知道首项和公差的情况。有了等差数列的通项公式和求和公式，那么就可以很方便的去解决一些等差数列问题了。

已知 $a_1=20$，$a_n=54$，$S_n=999$，如何求 d 和 n 呢？

根据等差数列通项公式与求和公式可得方程如下：

$$\begin{cases} 20+(n-1)d=54 \\ (20+54)n/2=999 \end{cases}$$

解方程有：

$$n=27,\quad d=17/13$$

已知数列$\{a_n\}$的前 n 项和为 $S_n=n^2+1/2n$，求这个数列的通项公式。这个数列是等差数列吗？如果是，它的首项与公差分别是什么？

实际上，当 n=1 时：

$$a_1=s_1=3/2;$$

当 n>1 时：

$$a_n=s_n-s_{n-1}=(n^2+1/2n)-((n-1)^2+1/2(n-1))=2n-1/2$$

当 n=1 时，$2n-1/2=3/2=a_1$，所以此通项公式满足 $a_1$ 的值，所以数列的通项公式为：

$$a_n=2n-1/2$$

$$d=a_n-a_{n-1}=(2n-1/2)-(2(n-1)-1/2)=2$$

由于 d 为常数，所以该数列是等差数列，首项为 3/2，公差为 2。

课后思考：已知数列$\{a_n\}$的前 n 项和为 $Sn=1/4n^2+2/3n+4$。求这个数列的通项公式。并尝试着总结出等差数列前 n 项和公式的特点。

## 5.4 等比数列

除了等差数列，还有一类数列是比较常见的，那就是等比数列。一般地，如果一个数列从第 2 项起，每一项与它的前一项的比等于同一常数，那么这个数列叫做等比数列（geometric sequence），这个常数叫做等比数列的公比（common ratio），公比通常用字母 q 表示（$q\neq 0$）。如果在 a 与 b 中间插入一个数 G，使 a，G，b 成等比数列，那么 G 叫做 a 与 b 的等比中项，根据等比数列的定义有 G=aq, G=b/q，所以有 $G^2=ab$。

比如首项为 1，公比 q 为 2 的等比数列：

$$1, 2, 4, 8, 16, 32, 64, \ldots 2^n, \quad \ldots$$

首项为 1，公比 q 为 1/2 的等比数列：

$$1, 1/2, 1/4, 1/8, 1/16, 1/32, 1/64, \ldots 1/2^n, \ldots$$

根据等比数列的定义，容易得出它的通项公式：

$$a_n=a_1*q^{(n-1)},\ (q\neq 0, n>1, n\in N^*)$$

前面，我们通过分析，给出了等差数列的求和公式，那么等比数列前 n 项和公式是什么

样的呢？

对于一个公比为 q 的等比数列$\{a_n\}$，记前 n 项和为 $S_n$，即：

$$S_n=a_1+a_2+a_3+\ldots+a_n$$

根据等比数列的通项公式，等比数列的前 n 项和又可以写成：

$$S_n=a_1+a_1q+a_1q^2+a_1q^3+\ldots+a_1q^{n-1} \quad ①$$

将①式等号两边同时乘以公比 q，有：

$$qS_n=a_1q+a_1q^2+a_1q^3+a_1q^4+\ldots+a_1q^n \quad ②$$

用①等式两边分别减去②等式两边，有：

$$(1-q)S_n=a_1-a_1q^n \quad ③$$

注意：上式中①和②中间相同部分通过减法抵消掉了，因此只剩下①的第一项和②的最后一项。

当 $q\neq1$ 时，③等式两边同时除以(1-q)有：

$$S_n = a_1\frac{1-q^n}{1-q}(q \neq 1)$$

这就是当 $q\neq1$ 时，等比数列的前 n 项和公式。注意：当 q=1 时，由于该等比数列每项为定值并且等于 $a_1$，因此前 n 项和为 $na_1$。

现在我们来看看著名的国际象棋相关问题。

国际象棋起源于古印度，有八行八列 64 个格子。国王要奖励国际象棋的发明者问他有什么要求。发明者说:在第 1 个格子里放 1 颗麦粒，在第 2 个格子里放 2 颗麦粒，在第 3 个格子里放 4 颗麦粒，在第 4 个格子里放 8 颗麦粒，在第 5 个格子里放 16 颗麦粒，依次类推。国王答应了。问国王能满足满足上述要求吗?

按照国际象棋发明者的要求，棋盘格子里放的麦粒按照顺序，组成了一个 64 项，公比为 2 的等比数列：

$$1,2,4,8,16,32,64,\ldots,2^{63}$$

根据等比数列求和公式，共有麦粒：

$$S=1(1-2^{64})/(1-2)=2^{64}-1$$

$2^{64}-1 = 18446744073709551615$，这个数字是非常大的，数量级达到了 $10^{19}$，如果按照千粒麦子的质量约为 40g，那么这些麦粒的总质量超过了 7000 亿吨。所以，国王无法实现这个要求。

等比数列中，数的增长是非常快速的，比等差数列中数的增长要快很多。我们把等差数列中数的增长方式叫作算术级增长，等比数列中数的增长方式叫几何级增长。

如果纸的厚度为 0.005cm，对折 30 次，厚度为：$0.005*2^{30}$cm=5368709.12cm=53687.0912m，高达 5 万多米，而珠穆朗玛峰的高度是 8848.13 米，可见几何级增长是非常迅速的。

实际上，$2^{64}-1$ 也是现代计算机 64 位系统所能够表示的最大整数，并且取代了 32 位系统（只能表示 $2^{32}-1$=4294967295 范围内的整数，要比 $2^{64}-1$ 小得多），成为了当前主流的计算机系统。

## 5.5 程序递归

递归是一种重要的思想和算法，在数学和计算机里都有着重要的应用。在学习递归之前，我们来看看数学里面的阶乘的定义。

## 5.5.1 阶乘

对于一个正整数 n 的阶乘，记为 n！，定义为：

$$n!=\begin{cases}1 & \text{当 n=0 或 1 时}\\ n*(n-1)! & \text{当 n>1 时}\end{cases}$$

比如:

6！ =6*5!=6*5*4!=6*5*4*3!=6*5*4*3*2!=6*5*4*3*2*1!=6*5*4*3*2*1

=6*5*4*3*2=6*5*4*6=6*5*24=6*120=720。

上述就是阶乘的一个递归定义。当 n 为 0 或者 1 的时候，定义 n！的值为 1；当 n>1 时，求 n 的阶乘，只需求出 n-1 的阶乘，然后将结果乘以 n，即 n!=n*(n-1)!，而求 n-1 的阶乘，又只需求出 n-2 的阶乘，将结果乘以(n-1)，即(n-1)!=(n-1)*(n-2)!。这样依次递归，直到计算 1 和 0 的阶乘，而 1 和 0 的阶乘是递归的出口，已经从定义可以求得，于是再将结果层层上传并用于计算，最终得到结果。因此，递归可以分为“递”与“归”两个过程，“递”的过程就是不断向出口靠近的过程，到达出口拿到结果之后，再通过“归”，回到问题原来的而位置，从而获得计算结果。

比如为了计算 10!，其过程如下：

```
10*9!
10*9*8!
10*9*8*7!
10*9*8*7*6!
10*9*8*7*6*5!
10*9*8*7*6*5*4!
10*9*8*7*6*5*4*3!
10*9*8*7*6*5*4*3*2!
10*9*8*7*6*5*4*3*2*1! // 此时遇到递归出口，将结果再逐层返回上层递归
10*9*8*7*6*5*4*3*2*1
10*9*8*7*6*5*4*3*2
10*9*8*7*6*5*4*6
10*9*8*7*6*5*24
10*9*8*7*6*120
10*9*8*720
10*9*5760
10*51840
518400    // 在这里得到最终 10！的值为 518400
```

递

归

从 10 到递归出口的过程我们可以称之为“递”，从递归出口回到 10 的过程，我们可以称之为“归”。

在这个计算过程中，我们把 n!=n*(n-1)!叫作阶乘的递归式子，而把 0！=1 和 1！=1 叫作求阶乘的递归出口。在计算过程中，通过递归式嵌套，进入下一层递归，直到遇到递归出口，递归就可以结束，将结果返回上层计算。

一个递归，必须拥有递归出口，递归出口的数量可以一个或者多个，如果没有递归出口，

递归式就会陷入无穷递归，无法结束。

需要注意的是，0！=1 在连乘意义下是无法解释的，因为阶乘是连乘运算，而 0 与任何实数相乘的结果都是 0。因此用正整数阶乘的定义是无法推广或推导出 0！=1 的。其实 0!=1 是一种为了数学中一些其它公式的方便表示或者必然要求而进行的人为规定。

比如在组合数学定义中（参考本书 11.1 节），对于正整数$n$满足条件 $0 \le m \le n$的任一非负整数 $m$，$C(n,m)$ 都是有意义的，特别地在 $m=0$ 及 $m=n$ 时，有 $C(n,0)=C(n,n)=1$。但对于组合数公式 $C(n,m)=\dfrac{n!}{m!(n-m)!}$来说，在 $m=0$及 $m=n$时，都由于遇到 0 的阶乘没有定义而发生巨大尴尬。对照结论 $C(n,0)=C(n,n)=1$ 和公式 $C(n,m)=\dfrac{n!}{m!(n-m)!}$，我们顺势而为地定义“0！=1”就显得非常必要了。这样，组合数公式在 $m=0$及 $m=n$时也通行无阻，不会有任何尴尬了。

另外，在函数 $e^x$的麦克劳林级数展开式中 $e^x=\sum_{n=0}^{\infty}\dfrac{x^n}{n!}$用到了“0！=1”的定义，没有这个定义就只能麻烦地表示为 $e^x=1+\sum_{n=1}^{\infty}\dfrac{x^n}{n!}$。在伽玛函数 $\Gamma(s)=\int_0^{+\infty}t^{s-1}e^{-t}dt$来说，当 s=1 时，显然有 $\Gamma(1)=\int_0^{+\infty}e^{-t}dt=1$。当 $s$是大于 1 的正整数时，有公式 $\Gamma(s)=(s-1)!$，当 0 的阶乘被定义为 0！=1 后，公式 $\Gamma(s)=(s-1)!$对任意正整数 $s$都成立了。

## 5.5.2 斐波那契数列中的递归

前面学习数列的时候，我们已经认识了斐波那契数列。实际上，斐波那契数列的常用定义方式就是一个递归定义。我们再来看看斐波那契数列的递归定义：

$a_0=0$

$a_1=1$

$a_n=a_{n-1}+a_{n-2}$（n＞1，n∈N*）

其中 $a_0$ 和 $a_1$ 是这个递归定义的出口，而 $a_n=a_{n-1}+a_{n-2}$ 是这个递归的递归式。要求 $a_n$，只需要求出 $a_{n-1}$ 和 $a_{n-2}$，然后依次递归，直到遇到递归的出口 $a_0$，$a_1$ 结束递归，再逐层返回，最终得到计算结果。

在计算机编程里，递归的思想同样存在。在编写程序的时候，我们可以使用递归的方法来解决问题。在程序设计里，我们把某个函数直接或间接的调用自身称为递归。

和数学中定义阶乘以及斐波那契数列一样，程序中的递归首先需要有一个递归式，这个递归式规定如何将原问题划分成子问题(子问题解决的对象和原问题解决的对象性质一样，但规模小了），子问题解决了，原问题也就解决了。程序中的递归也需要包含一个或多个递归出口，即递归终止的条件，也就是最小子问题的求解，可以允许多个出口。

## 5.5.3 程序中的递归实现

现在我们来看看，如何通过程序递归的方法来计算阶乘和斐波那契数列的通项。对于 n

的阶乘来说，我们通过程序实现的例子如下：

```
int fact(unsigned int   n){
    if(n==0||n==1)
        return 1;//递归出口
    return n*fact(n-1);   //n*fact(n-1)就是递归式，在 fact（n）中，又自己调用了 fact(n-1)
}
```

实现了计算阶乘的递归程序之后，如果我们要计算 10！，那么只需要在程序里调用：

```
fact(10);
```

就可以计算出 10 的阶乘结果。

而对于斐波那契数列，我们写出计算它的通项公式的递归代码如下：

```
unsigned int feibo(unsigned int n){
    if (n < 2) {//递归的出口，递归嵌套到此处，即可逐层返回
        return n;
    } else {
        return feibo(n-1) + feibo(n-2);//递归式，自己调用自己，但输入数据变小
    }
}
```

有时候，对于同一个问题，我们可以使用递归算法解决，也可以使用循环迭代算法解决。

我们可以把递归看做是到楼顶取东西。从一楼开始爬，每爬一层之后便检查是否到楼顶了（楼顶就相当于我们递归的出口），如果没有到楼顶，就继续爬，每层楼梯看上去都一样，执行的过程都一样，但实际上，从 1 楼到 2 楼，从 2 楼到 3 楼的楼梯是两个楼梯，等到楼顶了，取了东西，我们也不能直接就跳楼回到 1 楼，还得从楼顶一层层退回来。

我们可以把循环看成是驴子拉磨。驴子拉磨的时候，无论跑多少次，都是在原地。变化的只是磨盘里磨的东西，而不是驴每圈所在的不同位置。

因为递归会消耗程序的栈空间，所以，如果栈空间比较小或者递归嵌套比较深，就会造成栈溢出。而且与循环相比，递归还会有比较频繁的进栈和出栈的操作，这样会降低程序执行的效率。因此递归有时候会被看作是洪水猛兽，并不被人推荐。

### 5.5.4 尾递归

当递归调用是整个函数体中最后执行的语句且它的返回值不属于表达式的一部分时，这个递归调用就是尾递归。

在传统的非尾递归中，典型的计算顺序就是首先执行递归调用，一直到递归出口，然后层层返回，获取递归调用的返回值并计算结果，这种计算顺序下，每次递归调用返回之前，不会得到计算结果。

在尾递归中，首先执行计算，然后执行递归调用，将当前步骤的结果传递给下一个递归步骤，因此最后一个语句采用的形式为 return recursive_func(param...)。而且任何给定递归步骤的返回值与下一个递归调用的返回值相同，也即等于递归出口处的返回值。

尾递归的好处在于将当前的计算结果一直传递到递归的出口处，不需要再向上返回了，因此不需要在栈上保存额外的变量空间来支持返回时的计算。

同样以阶乘和斐波那契数列为例子。对于阶乘的计算，传统递归的写法：

```
int fact(unsigned int   n){
    if(n==0||n==1)
```

```
        return 1;//递归出口
    return n*fact(n-1);   //此处返回值 fact(n-1)是表达式的一部分，不是尾递归
}
```

调用方式：fact(6);

执行过程：

6*fact(5)

6*5*fact(4)

6*5*4*fact(3)

6*5*4*3*fact(2)

6*5*4*3*2*fact(1)

6*5*4*3*2*1

6*5*4*3*2

6*5*4*6

6*5*24

6*120

720

而采用尾递归的写法为：

```
int fact_tail(unsigned int n,   int res){
    if(n==0||n==1)
        return res;//递归出口，此时 res 已经是计算好的值了
    return fact_tail(n-1, n*res);   //尾递归
}
```

尾递归的调用方式：

fact_tail(6, 1);

执行过程：

6, 1

5, 6

4, 30

3, 120

2, 360

1, 720

所以返回值为 720。

对于斐波那契数列，它的尾递归实现也是类似的：

```
int feibo_tail(int n,//第 n 个数的序号，同时也是递归深度
int a1,//第 n 个数的值
int a2 //第 n+1 个数的值
) {
    if (n < 2) {
        return a1;
    }
    int a3=a1+a2;
    return feibo_tail (n-1, a2, a3);
}
```

尾递归的调用方式：

feibo_tail(5, 1, 1);

执行过程：

5, 1, 1

4, 1, 2

3, 2, 3

2, 3, 5

1, 5, 8

所以，计算结果为5。

尾递归效率比较高，但尾递归算法理解起来略显困难，因为它的每一个参数的作用不如传统递归利于理解。在调用的时候，参数一般会传入递归出口处的初始值。

### 5.5.5 递归应用

递归程序设计是一个重要的程序设计思想。可以应用递归设计来解决字符串，链表，树等数据结构中一些常见的问题。尤其是树中，很多问题都可以使用递归的方法来解决。在利用递归解决问题的时候，有2个关键的地方：

首先就是要找到问题的最简子问题，也就是问题中最简单的情况，比如对于一个链表，最简单情况就是链表为空或者只有一个结点；对于一个字符串，最简单情况就是字符串为NULL或者只有一个’\0’字符；自然数最简单的情况就是0或者1；树最简单的情况就是树为NULL或者只有一个结点等，而更多的问题已经明确提出了最简子问题，比如阶乘中的0！和1！，在定义中已经给了出来。所以，最简子问题往往是很容易分析出来的。

然后是要通过分析和转换，将原问题转化为子问题。子问题和原问题是同类问题，但子问题的规模（即数据量）应该比原问题要小。比如求n!那么它的子问题就是(n-1)!，只需要把(n-1)!乘以n就是n的阶乘了，而n-1比n要小。又比如要求斐波那契数列的第n个值，只需要求出(n-1)的值和(n-2)的值，那么就可以求出n的值了，而n-1和n-2比n要小。在先序遍历树t的时候，当把根节点遍历完后，因为t->left和t->right是它的子树，也就是子问题，所以紧接着用递归遍历t->left和t->right就可以了。

先看看如何用递归解决一个字符串逆置的问题。用递归的方法将一个字符串逆置，比如“hello world”逆置成为“dlrow olleh”。

分析：最简子问题（递归出口）：str==NULL或者len==1或者len==0，这个时候，不需要逆置。

子问题转化：只要将起始地址为str+1，长度为len-2的子串（即字符串中去掉第一个字符和最后一个字符所得到的子串，比如“hello world”中去掉第一个字符’h’和最后一个字符’d’得到的子串“ello worl”）用同样的方法进行逆置之后，再将原字符串第一个字符与最后一个字符交换，即可完成字符串的逆置。

以“hello world”为例，先逆置除了最左边和最右边字符的子串：“ello worl”，然后再交换’h’和’d’。那么递归实现的算法为：

```
void reverse_str(char *str，size_t len){

    if(NULL==str || len==1||len==0)//递归出口，最简子问题
        return;
```

```
    reverse_str(str+1,len-2);//先逆置子串

    char tmp=*str;//再交换主串最左边与最右边的字符
    *str=*(str+len-1)
    *(st+len-1)=tmp;
}
```

再看一个用递归解决的台阶问题。一个台阶总共有 n 级，如果一次只可以跳 1 级或者 2 级。求总共有多少种跳法。

分析：最简子问题（递归出口）：当 n 为 1 或者 2 时，是最简子问题，即递归的出口。而当 n=1 时，只有 1 种跳法（1 次跳 1 级）；当 n=2 时，有 2 种跳法（1 次跳 1 级共跳 2 次，或者 1 次跳 2 级共跳 1 次）。

子问题转化：要到达 n 级台阶，必然分为到达 n-1 级再往上跳 1 级或到达 n-2 级再往上跳 2 级这两种情况。其实当跳到 n-2 级的时候，这种情况下，有两个选择：一是 1 级 1 级的跳，因此可以向上跳 1 级到达 n-1 级，然后又向上跳一级到达 n 级，而这种跳法已经包含在了 n-1 级的跳法之中；二是直接跳 2 级到达 n 级。

因此要求出 n 级台阶共有多少种跳法，实际上应该等于跳到 n-1 级的跳法加上跳到 n-2 级的跳法。也就是如果设 sum(n)表示跳到 n 级台阶的总数，那么有递归式：sum(n)=sum(n-1)+sum(n-2)。根据分析，我们可以给出如下计算跳法总数的代码：

```
size_t jump_sum(size_t   n){ //jump_size 函数计算台阶为 n 的跳法总数，递归版本
    if (n == 1 || n == 2)
        return n;

    return   jump_sum(n-1)+jum_sum(n-2);
}
```

在程序里，通过递归计算求解问题的时候，在递归嵌套过程中的一些临时结果数据都会保存在程序的栈空间里。递归如果嵌套过深，会造成栈溢出（为了防止栈溢出，可以跟踪递归的深度，如果超过某个深度，就返回）。而在系统内核里是不能使用递归，因为内核栈只有几 K 到几十 K，栈空间很容易被递归过程占满甚至溢出。

一般来说，递归算法的时间复杂度和对应的非递归算法差不多，但是递归的效率是相当低的，速度比迭代版本（迭代版本一般通过循环实现）慢。比如二路归并内排序、二分查找等算法的实现都用迭代而不用递归。因为递归要花费大量时间在反复的函数调用和**进栈出栈**，各种中断等机制上。更有甚者，在递归求解过程中，某些解会重复的求好几次，这是不能容忍的，这些也是引入非递归机制的原因之一。

递归也是可以转化为循环迭代求解的。比如对于上面用递归算法逆置字符串的问题，对应的循环迭代版本如下：

```
void reverse_str(char *str, size_t len){
    int len = 0;
    //检查指针的合法性
    if (NULL == str || 0 == *str|| len==0 || len==1){
        return;
    }
```

```
        //翻转字符串
        for(int i=0;i< len/2;i++){//循环逆置字符串
            char temp = str[i];
            str[i] = str[len-1-i];
            str[len-1-i] = temp;
        }
    }
```

对比递归逆置字符串和循环迭代逆置字符串两种不同版本的算法，我们可以发现，在循环迭代版本的计算过程中，少了反复的函数调用以及由函数调用引起的进栈和出栈，从而提高了程序的执行效率，也不会担心递归嵌套过深而引起的栈溢出风险。

把递归转化为循环的时候，有时候我们可能会需要在程序中自定义栈等数据结构，以此来保存递归算法中一些中间临时计算结果。有兴趣的读者，可以进一步研究非递归版本的树的遍历，快排等算法实现。

那么，是不是循环的算法一定就比递归的算法复杂度要好呢？这不是绝对的。比如要计算 $x^n$ 的问题。我们就可以通过循环和不同的递归方法来实现和对比一下：

```
int power1(int x, int n) {
    int result = 1;
    for(int i = 0; i < n; i++) {
        result *= x;
    }

    return result;
}

int power2(int x, int n) {
    if (n==0) {
        return 1;
    }

    return power2(x, n-1)*x;
}

int power3(int x, int n) {
    if (n==0) {
        return 1;
    }

    if (n%2==1) {
        return power3(x, n/2)*power3(x, n/2)*x;
    }
    return power3(x, n/2)*power3(x, n/2);
}
```

```
int power4(int x, int n) {
    if (n==0) {
        return 1;
    }

    int tmp = power4(x, n/2);
    if (n%2==1) {
        return tmp*tmp*x;
    }

    return tmp*tmp;
}
```

在上面给出的 4 个函数， power1(), power2(), power3(), power4()，都是用不同的方式实现了 $x^n$。其中 power1()用的是循环实现，而 power2(), power3(), power4()则使用了不同的递归算法。最终我们会发现，power1(), power2(), power3()的事件复杂度是 O(n)的，而 power4()是 O(logn)的。这个例子告诉我们，递归算法的性能也有高于循环的时候。

那么我们究竟是如何去计算递归算法的复杂度呢？有时候我们在面试的时候，用递归的方法解决了面试问题之后，这也是面试官容易紧接着提出的问题。所以现在我们就来详细讲解这个问题。

## 5.5.6 递归复杂度计算

非递归算法的时间复杂度是非常容易计算，但是递归算法的时间复杂度却不那么容易直接计算出来。

对于递归算法的时间复杂度就是每次递归中的操作次数之和。对于递归方程 T(n) = aT(n/b) + f(n)，也就是一个规模为 n 的问题被分成规模均为 n/b 的 a 个子问题，f(n)表示每次递归中操作的次数。递归地求解这 a 个子问题，然后通过对这 a 个子问题的解的综合，得到原问题的解。

递归复杂度计算主要有递归树，主定理等方法。现在我们就分别给大家介绍这两种方法的计算过程。

### 1. 递归树

首先我们来看看如何用递归树计算递归算法的复杂度。以下面某递归算法的递归方程为例：

$$T(n) = 2T(n/2) + n^2$$

现在我们来画递归算法对应的递归树。对于存在上述递归关系的递归算法，如图 5-2 所示，在递归树中，每一节点中都将当前的自由项 $n^2$ 留在其中，而将两个递归项 T(n/2) + T(n/2)分别摊给了他的两个子节点，如此循环。可以得到递归树的规则为：

(1) 每层的节点为 T(n) = kT(n / m) + f(n)中的 f(n)在当前的 n/m 下的值即 f(n/m)；

(2) 每个节点的分支数为 k；

(3) 每层的右侧标出当前层中所有节点的和。

那么，将递归树中每层的结点和加起来，就是整个递归算法的复杂度。

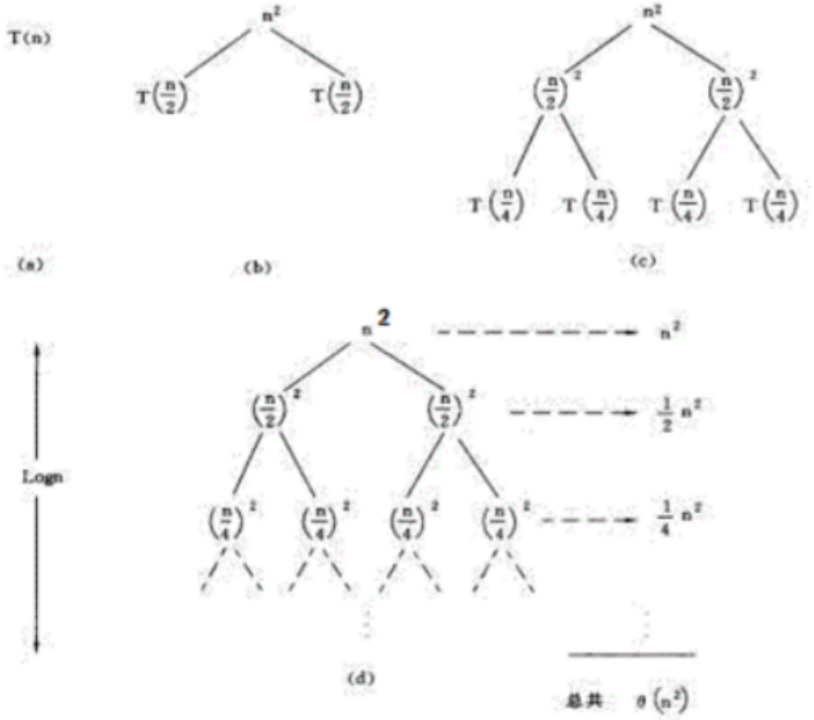



图 5-3 递归关系 $T(n) = 2T(n/2) + n^2$ 的递归树

因此，由于递归的出口是 n=1 时，假设递归树的深度为 k，那么对于上述递归算法，递归树将在 k 层有$(1/2)^{k-1}n^2=1$，因此 $k=\log_2(n)$。递归树每层结点之和分别为：

$$n^2, 1/2n^2, 1/4n^2, ..., 1/2^{(k-1)}n^2$$

所以递归的复杂度为 递归树每层结点和之和，于是有：

$$n^2+1/2n^2+1/4n^2+ ...+1/2^{(k-1)}n^2=n^2(1+1/2+1/4+...+1/2^{(k-1)})$$

根据等比数列求和公式以及 $k=\log_2(n)$有：

$$1+1/2+1/4+...+1/2^{(k-1)}=2-(1/n)。$$

所以，该递归算法的时间复杂度为 $n^2(2-(1/n))=n^2+n$，即 $O(n^2)$。

再来看一个用递归树计算递归算法复杂度的例子。假设递归算法的递归方程为：

$$T(n)=T(n/3)+T(2n/3)+n$$

那么，这个递归算法的时间复杂度是多少呢？

根据递归树的画法，我们可以画出该递归算法对应的递归树如图 5-3 所示：

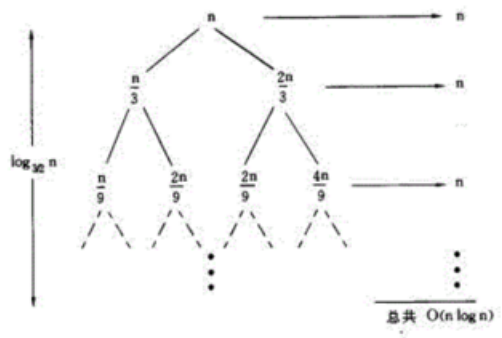


图 5-3 递归关系 T(n)=T(n/3)+T(2n/3)+n 的递归树

从图中中可以看出，递归树每层结点之和都为 n。所以，关键是需要计算该递归树有多少层。由于递归树中最右边的分支是最快趋近于 1 的，因为：

$$n \to \frac{2}{3}n \to (\frac{2}{3})^2 n \to \cdots \to 1$$

假设递归树有 k 层（其中根节点为 0 层），那么有$(2/3)^k n=1$，所以有：

$$k=\log_{3/2} n$$

因此，递归树所有结点之和为 $kn=n\log_{3/2} n$，也就是该递归算法的复杂度为 O(nlogn)。

讲到这里，相信大家对使用递归树计算递归算法复杂度已经有了一定的了解。那么现在可以尝试着使用递归树的方法计算本书 5.5.5 一节中 $x^n$ 递归算法的时间复杂度。

**斐波那契数列递归算法复杂度计算**

前面我们介绍了斐波那契数列的递归算法，斐波那契数列递归算法的时间复杂度递归方程为：

$$T(n)=T(n-1)+T(n-2)+O(1)$$

那么画出斐波那契数列递归算法的递归过程如图 5-5 所示：

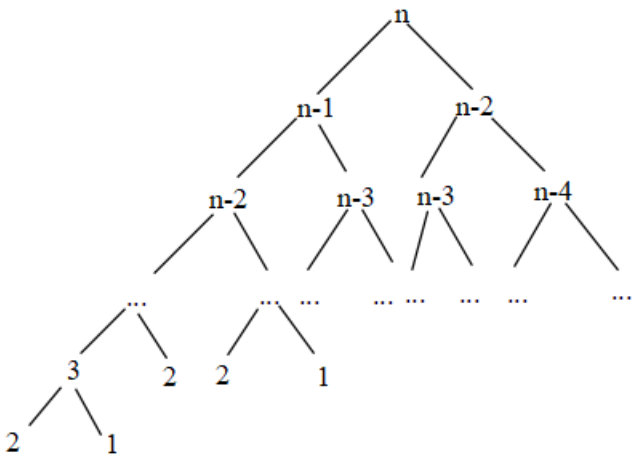



图 5-5 斐波那契数列递归过程

斐波那契数列的递归算法时间复杂度递归方程不符合 T(n) = aT(n/b) + f(n)的形式，所以我们不能通过上述递归树的画法得到它的递归树。但是，将斐波那契数列递归算法的执行过程画出之后，得到的是一颗二叉树如图 5-5 所示。虽然这颗树不一定是满二叉树，但我们可以考虑在最坏情况下，它就是一颗满二叉树，为了达到递归出口（n=1），那么树的深度就是 n，那么根据满二叉树的特性，整颗树的结点数就是 $2^n$-1。由于每个结点的复杂度是 O(1)的，那么斐波那契数列递归算法的时间复杂度即为 $O(2^n)$。

讲完递归树计算递归算法的复杂度的方法之后，我们再来介绍用主定理来计算递归算法时间复杂度的计算。

## 2. 主定理

我们知道，对于 T(n) = aT(n/b) + f(n)这种递归方程其实是分治法的时间复杂性所满足的递归关系，即一个规模为 n 的问题被分成规模均为 n/b 的 a 个子问题，f(n)是在规模为 n 时递归中的操作次数。递归地求解这 a 个子问题，然后通过对这 a 个子问题的解的综合，得到

原问题的解。

对于满足这种递归方程的递归算法，我们可以直接利用主定理提供的公式进行递归算法的复杂度计算。主定理是这样的：

设递归算法的递归方程为：

$$T(n) = aT(n/b) + f(n)$$

那么该递归算法的时间复杂度计算公式为：

$$T(n) = \begin{cases} O(n^{\log_b a}) & O(n^{\log_b a}) > O(f(n)) \\ O(f(n) * \log n) & O(n^{\log_b a}) = O(f(n)) \\ O(f(n)) & O(n^{\log_b a}) < O(f(n)) \end{cases}$$

（其中a>1, b>1均为常数，f(n)是确定正函数）

特别地，当 f(n)为常数时有：

$$T(n)=\begin{cases} O(n^{\log_b a}) & a \neq 1 \\ O(\log n) & a = 1 \end{cases}$$

当 f(n)为线性关系时，即 f(n)=cn(c 为常数），那么有：

$$T(n)=\begin{cases} O(n) & a < b \\ O(n \log n) & a = b \\ O(n^{\log_b a}) & a > b \end{cases}$$

主定理告诉我们，只需要计算出 $x=\log_b a$ 的值，然后将 $n^x$ 与 f(n)进行大小比较就可以很快利用公式确定对应的时间复杂度。现在我们就利用主定理来考虑下面两个递归方程对应算法复杂度的计算。

设某递归算法的递归方程为 $T(n) = 2T(n/2) + n^2$，那么有 a=2, b=2，所以 $\log_b a=\log_2 2=1$，由于 $n^1<n^2$，所以该递归算法的时间复杂度为 $O(n^2)$。

再来看一个问题。设某递归算法的递归方程为 $T(n) = 4T(n/2) + n$，由于 f(n)=n，是 n 的线性函数，且 a=4, b=2，$\log_b a=\log_2 4=2$。因为 a>b，所以时间复杂度为 $O(n^2)$。

我们同样可以使用主定理来计算本书 5.5.5 节计算 $x^n$ 的递归算法时间复杂度，留给读者自行练习。

我们利用主定理，可以很快计算出一些常见问题递归算法的时间复杂度，比如：

**二叉树遍历**

二叉树遍历的递归方程为 T(n)=2T (n/2)+1，其(a=2, b=2) ，f(n)=1 为常数，所以利用主定理，由于 a≠1，$\log_b a=\log_2 2=1$，所以它的时间复杂度为 O(n)。

**归并排序**

归并排序递归算法的递归方程为 T(n)=2T(n/2 )+n，其中 a=2, b=2, f(n)=n，由于 a=b，所以利用主定理公式，其时间复杂度为 O(nlogn)。

**二分搜索（折半搜索）**

二分搜索递归算法的递归方程为 T(n)=T(n/2 )+1，其中 a=1, b=2, f(n)=1，f(n)为常数，且 a=1，所以直接套用主定理公式即可立即得出它的时间复杂度为 O(log n)。

最后，我们把一些常见的递归关系及其对应的时间复杂度总结在如表 5-1 所示：

表 5-1 常见递归关系的时间复杂度

| 递归关系 | 时间复杂度 | 常见算法 |
| --- | --- | --- |
| T(n)=T(n/2)+O(1) | O(logn) | 二分查找 |
| T(n)=T(n-1)+O(1) | O(n) | 线性查找 |
| T(n)=2T(n/2)+O(1) | O(n) | 二叉树遍历 |
| T(n)=2T(n/2)+O(n) | O(nlogn) | 归并排序，快速排序 |
| T(n)=2T(n/2)+O(nlogn) | $O(n\log^2 n)$ | |
| T(n)=T(n-1)+O(n) | $O(n^2)$ | 选择排序，插入排序 |
| T(n)=2T(n-1)+O(1) | $O(2^n)$ | 汉诺塔 |
| T(n)=T(n-1)+T(n-2)+O(1) | $O(2^n)$ | 斐波那契数列递归算法 |

## 5.6 程序中的基本算法

当程序中要处理的数据量比较小时，我们只需要使用普通的循环或递归就可以快速的解决问题，但是当数据量比较大，计算比较复杂时，如果依然使用简单的循环或递归来处理就非常耗时和低效了，有时甚至计算任务都无法完成，因此必须使用一些高效的算法来解决问题。

计算机中基本的算法思想有动态规划，贪心算法，分治算法，回溯算法，分支限界算法，二分查找算法以及暴力穷举算法等。它们都有很多应用场景，最优化问题大多可以利用这些（除暴力穷举之外）算法解决。

### 5.6.1 动态规划

在 5.5.3 一节中，我们给出了计算斐波那契数的递归方法，我们用它来计算第 10 个，第 20 个斐波那契数都没有问题。但如果用此递归计算斐波那契第 100 个数，你就会发现程序会卡住，一直无法得到最终的结果。因为这种递归方法，是一种暴力递归，涉及大量重复的计算，如果仔细画出它的递归树，我们会发现它的时间复杂度高达 $O(2^n)$，所以一旦问题规模变得很大，那么计算时间就会成指数增长。因此，在本节，我们尝试用新的动态规划算法来优化计算方法从而解决此问题。

#### 5.6.1.1 什么是动态规划

动态规划（Dynamic Programming，简称 DP）就是其中一种在计算机中高效解决问题的常用算法。

动态规划的基本思想就是将待求解问题分解成若干个子问题（子阶段），先求解子问题，然后从这些子问题的解递推得到原问题的解。这就是人们常说的“大事化小，小事化了”。适合于用动态规划求解的问题，经分解得到子问题往往不是互相独立的，也就是说，下一个子问题的求解是建立在上一个子阶问题的解的基础上。动态规划所涉及的问题多数有重叠子问题，为了避免大量的重复计算，我们往往需要保存已解决的子问题的答案，而在需要时再找出已求得的答案。我们可以用一个表来记录所有已解的子问题的答案。不管该子问题以后是否被用到，只要它被计算过，就将其结果填入表中。

比如，要计算如下问题：

1+1+1+1+1+1=?

通过计算，我们可以得出并在纸上记下答案为：6。现在，我们在原问题的左边，再增加一个“1”，那么问题变为了：

1+“1+1+1+1+1+1”=?

此时，由于有了第一次的计算结果 6，我们很容易得出现在问题的答案为：1+6=7。此时，面对当前的问题，我们再也不用重复进行原来的计算，只需要将原来保存的计算结果再加 1 就可以得出新问题的计算结果。这里，将现在的问题分解为：“1+原来的子问题”，并将原来的子问题计算结果保存起来，在计算现在的问题的时候，直接调用原来子问题的答案，那么就可以快速的计算出现在问题的答案。这里体现的就是动态规划的思想。

一般来说，在我们应用动态规划来解决问题时，需要完成以下三个关键目标：

1.建立状态转移方程

先确定当前问题的子问题是什么，并找到当前问题与拆分之后的子问题之间的递推关系式以及递推式的边界也就是终止条件，对应着最简子问题的求解。只要子问题得到了解决，那么通过状态转移方程就可以解决当前问题。

2.缓存并复用子问题结果

将计算子问题得到的结果保存起来，在求解当前问题时，随时调用。

3.按顺序从小往大算

将当前问题拆分为子问题之后，子问题又可以进一步拆分。这样就从最简单的子问题开始计算，从小到大一步步递推出最终的结果。

### 5.6.1.2 动态规划求斐波那契数列值

在计算 5.2 节中的斐波那契数列的时候，我们曾经使用了递归算法。递归算法实际性能是非常差的，因为递归会造成栈空间的浪费，进栈和出栈也会造成性能的损失，并且随着递归的深入，计算的任务也会不断的翻倍，其计算复杂度，达到了 $O(2^n)$，因此当问题规模 n 增大到一定量的时候，比如 n=100 时，计算会变得非常慢，因为这是一个 NP 问题。

现在，我们也可以使用动态规划的方法来解决斐波那契数列的计算问题。

首先，建立状态转移方程，设 f(n)是斐波那契数列中第 n 个数，那么有：

f(n)=f(n-1)+f(n-2)

然后，定义一个一维数组，来保存中间子问题的计算结果。

最后，从最小开始计算，即 n 为 0 开始进行计算。

因此，斐波那契数使用动态规划计算的 Python 代码如下：

```
def feibo(n):
    dp = list(range(n+1)) #dp 缓存中间结果
    for i in range(n+1):#从小到大进行计算
        if i<2:
            dp [i]=i;
        else:
            #使用状态转移方程和子问题结果进行计算
            dp [i]=dp [i-1]+dp [i-2]
    return dp [-1] #dp  最后一个值即为所求结果

if __name__=='__main__':
```

```
    print feibo(100) # 计算第 100 个（从 0 开始）斐波那契数：354224848179261915075
```

实际上，通过进一步研究，我们会发现，要计算 feibo(n)，我们并不需要把所有的子问题结果都缓存一遍，只需要缓存 feibo(n-1)和 feibo(n-2)的结果就可以了。于是改进之后的代码如下：

```
def feibo(n):
    a=0
    b=1#a,b 缓存 n-1 和 n-2 的中间结果。初始化为 n=0,1 时候的结果
    for i in range(2, n+1):#从 n=2 开始，从小到大进行计算
        tmp=a+b
        a=b
        b=tmp
    return tmp
if __name__=='__main__':
    print feibo(100) # 计算第 100 个（从 0 开始）斐波那契数：354224848179261915075
```

而如果我们使用递归算法来求解斐波那契数列，对应的 Python 代码如下：

```
def feibo(n):
    if n<2:
        return n
    else:
        return feibo(n-1)+feibo(n-2)

if __name__=='__main__':
    print feibo(100)
```

运行两份不同的代码，你会发现，在递归版本中，由于存在大量的重复运算，复杂度高，程序一直在计算之中，很久都得不到运算结果。而利用动态规划的方法，通过缓存中间结果，避免了很多重复的运算，从而运算速度较高，很快就得到了运算结果。

以计算斐波那契第 6 个数为例子，使用递归版本从上到下展开之后的运算次数如图 5-6 所示，从图中你会发现，在计算 f(6)时，展开之后，存在大量的重复运算，比如 f(4)，f(3), f(2)等都进行了多次重复运算；而使用动态规划版本计算的次数如图 5-7 中右边从下到上粗体连接线所示，由于缓存了子问题的计算结果，相当于剪掉了递归版本中的大量分枝。二者对比发现，动态规划版本，减少了大量的运算，所以能够更快的获得计算结果。动态规划版本的时间复杂度从递归版本的 $O(2^n)$降为 O(n)，大大提高了程序执行的效率。

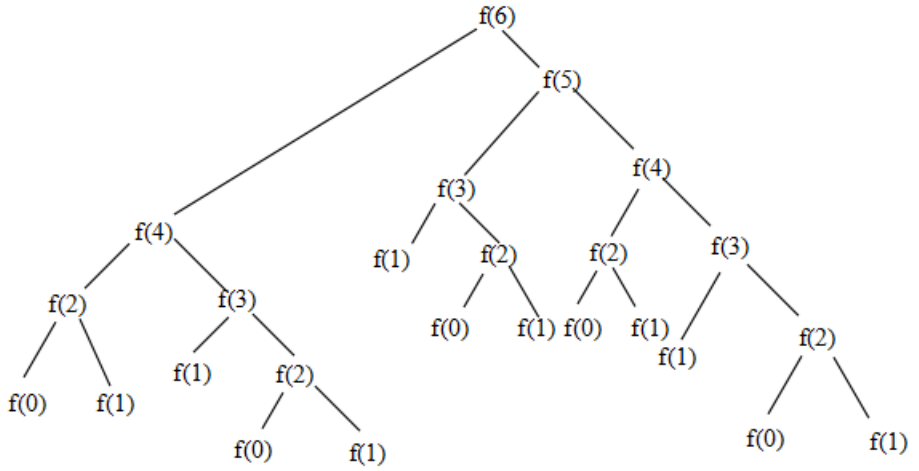



图 5-6 递归算法计算斐波那契数

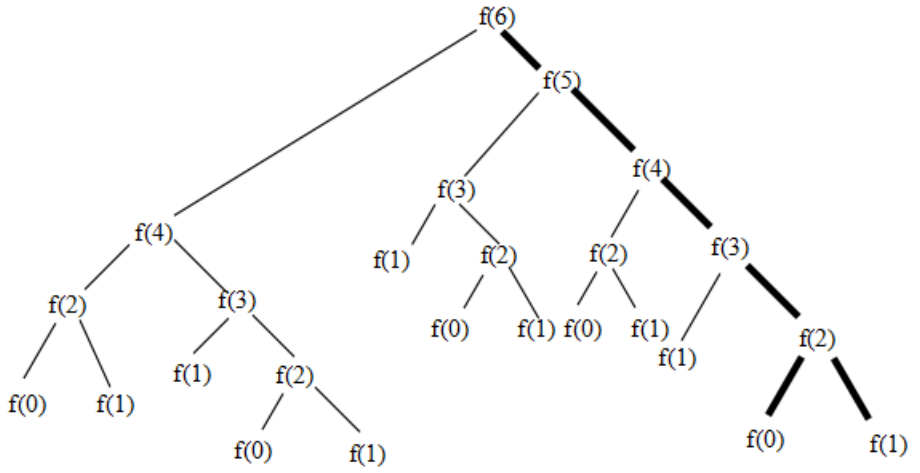



图 5-7 使用动态规划计算斐波那契数

递归算法和动态规划有很多相似之处，比如它们都需要建立一个从子问题到当前问题的递推关系。实际上，动态规划是一种对问题可以通过递归求解，并通过缓存来避免重复分支计算的优化算法。但是，递归是从上往下，从问题规模最大的函数 f(n)入手，一层一层向小规模递归，直到遇到递归出口。动态规划恰好相反，直接从最下往上，从最简单、问题规模最小的 f(0)，f(1) 和 f(2) 等开始往上计算，直到得到我们想要的答案 f(n)。所以动态规划实际上是迭代，因为它是通过循环来完成的。

递归算法自顶向下对问题进行求解时，每次产生的子问题并不总是新问题，有些子问题会被重复计算多次，这就是所谓的子问题重叠性。 动态规划算法正是利用了这种子问题的重叠性质，对每一个子问题只计算一次，然后将其计算结果保存在一个表格中，当再次需要计算已经计算过的子问题时，只是 在表格中简单地查看一下结果，从而获得较高的效率。

通过对斐波那契数的计算，相信大家已经认识到了动态规划算法的强大之处。实际上，动态规划可以用于求最大值、最小值，求各种方案总数（比如 5.5.5 节中练习 5 的上台阶方法总数）等问题的求解。

### 5.6.1.3 动态规划求方案总数

下面我们再来看看另一个有关动态规划的问题。

从位于一个 m x n 网格的左上角 （起始点如图 5-8 中，标记为“Start” ）起，每次只能向下或者向右移动一步，达到网格的右下角（在图 5-8 中标记为“Finish”）。问总共有多少条不同的路径？

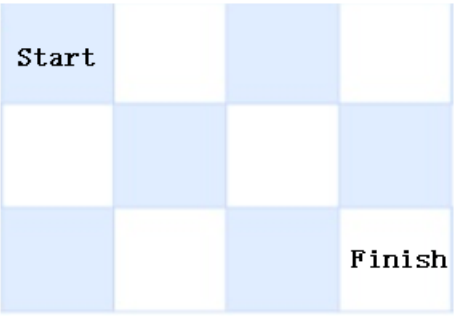



图 5-8 从 Start 到 Finish 网格走法

按照动态规划的算法思路，首先我们要找出状态转移方程。如图 5-9 所示，由于只能向下或者向右移动一步，由于再也不能通过除(i, j-1)和(i-1, j)两点之外的其他点到达(i, j)，所以网格中(i, j)所在坐标位置的路径走法应该等同于(i, j-1)和(i-1, j)两种方法之和。

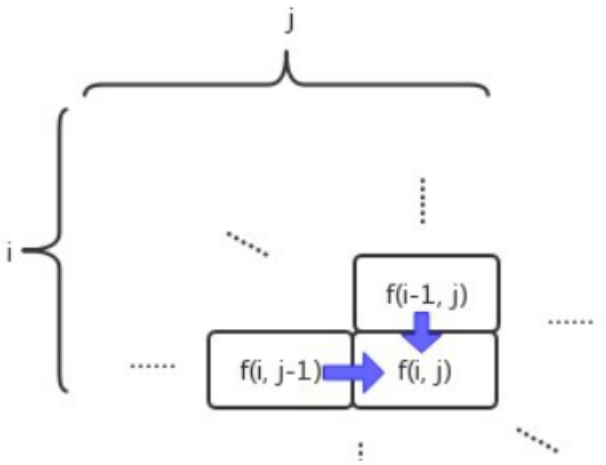



图 5-9 f(i, j)与 f(i, j-1)和 f(i-1, j)的关系

因此，状态转换方程为：

$$f(i,j)=f(i,j-1)+f(i-1,j)$$

得到此状态转换方程是最关键的一步，也是问题的难点所在。

然后，我们需要定义一个二维数组，来保存网格中每一个格子的路径总数。

最后，从网格中的第一格(1, 1)开始，从小到大计算。由于网格有两个维度，所以需两个循环，分别逐行和逐列让问题从小规模到大规模计算。

现在我们就可以写出如下 Python 代码：

```
def calc_path_count(m, n):
        dp = [[1] * n] * m#dp  为m*n 的二维数组
```

```
        for i in range(1, m):
                for j in range(1, n):
                        #状态转换方程
                        dp [i][j] = dp[i-1][j] + dp[i][j-1]
        return dp[-1][-1]#二维数组中最后一个值即为可走路径总数
if __name__ == '__main__':
        print calc_path_count(5, 9) #结果为 495
```

### 5.6.1.4 动态规划求最值

再来看一个求最大值的动态规划问题。

给你一个整数 list L，如 L=[2, -3, 3, 50]，求 L 的一个连续子序列，使其和最大，输出最大子序列的和。

例如，对于 L=[2, -3, 3, 50]， 输出 53。

首先，需要分析并建立状态转移方程。而要建立状态转移方程，就需要构造问题中的子问题以及问题和子问题之间的关系。

我们假设 $f_n$ 表示从第一个元素到第 n 个元素的最大子序列之和。现在我们尝试找到 $f_n$ 和 $f_{n-1}$ 之间的递推关系。以序列：-1, 2, -3 为例子。

$$f_3=MaxArraySum\{-1, 2, -3\}=2$$

现在当我们要求 $f_4$ 时，由于不知道 $f_3$ 中最大子序列的位置，新增加一个元素 4 时，这个元素是否能添加到前面的最大子序列中也是未知的。因为此时虽然添加了一个正的元素 4，即{-1, 2, -3, 4}，但无法利用前一个最大子序列，因为无法判断是否应该连在一起构成一组新的最大子序列，有可能被中间不在子序列中的元素（这里是-3）打断，所以也就无法构造最优解。

因此，构造的子问题是“前 N 个元素的最大子序和”在这里是行不通的。

实际上，进一步分析发现，在这里不管最大子序列是什么，一定是列表 L 中以 1 到 N 当中的某一个元素结尾，所以我们可以记 $S_n$ 为“以第 n 个元素为结尾的最大子序列和”。比如，对于：

$$-1, 2, -3$$

那么有：

$$S_3=MaxArraySum\{-1, 2, -3\}==MaxArraySum\{2, -3\}=-1$$

那么，在新增加一个元素 4 的时候，只需要判断 $S_3$ 是否大于 0，如果大于 0，对子序列和起到正增益作用，可以包含到新的子序列之中；否则，$S_3$ 起的就是负增益的作用，就应该被抛弃。那么就有：

$$S_4=max(S_3, 0)+4=0+4=4$$

因此，我们成功地得到了状态转换方程为：

$$S_n=max(S_{n-1}, 0)+L[n]$$

而很显然，当 n=0 的时候，$S_0=L[0]$。

于是，我们就可以写出如下 Python 代码：

```
def get_list_max_sub(list):

    s=range(len(list))#s 用于缓存 list 中到第 i 个元素时的最大子序列之和的值
    maxsum = list[0]
    s[0]=list[0]
```

```
    for i in range(1, len(list)):
        s[i]=max(s[i-1], 0)+list[i]#s[i]记录的是到第 i 个元素时，最大子序列之和的值
        maxsum = max(maxsum, s[i])#从列表 s 的 1-n 个中选择一个最大的值
    return maxsum


if __name__=='__main__':
    L=[2, -3, 3, 50]
    print get_list_max_sub(L)#结果为 53
```

再来看看另外一个有关最小值的问题。

给定一个包含非负整数的 m x n 网格，请找出一条从左上角到右下角的路径，使得路径上的数字总和为最小。假定：每次只能向下或者向右移动一步。

比如，输入:

```
[
  [1,3,1],
  [1,5,1],
  [4,2,1]
]
```

那么输出结果为: 7，因为路径 1→3→1→1→1 的总和最小。

分析：这是一个典型的动态规划问题。首先，我们需要建立状态转移方程。设网格为grid[m][n]，f(i, j)为从网格左上角到坐标(i, j)的路径之和最小值。由于每次只能向下或者向右移动一步，因此坐标(i, j)的位置，只能从(i-1, j)或者(i, j-1)两个位置到达，那么(i, j)处路径的最小值，应该是取(i-1, j)和(i, j-1)处最小值加上(i, j)处的值。那么 f(i, j)和 f(i-1, j)，f(i, j-1)的关系为：

$$f(i, j)=\min(f(i-1, j), f(i, j-1))+grid[i][j]$$

这就是状态转移方程。于是，我们就可以很容易写出如下用于计算网格路径之和最小值的 Python 代码：

```
def get_min_path_sum(grid, row, colum):

    dp = [[0] * colum] * row#dp 为 row*colum 的二维数组，缓存中间最小值

    for i in range(row):
        for j in range(colum):
            if i==0 and j==0:
                dp[i][j] =grid[i][j]
                continue
            tp = 1000#设置一个较大的值用于 dp[i-1][j]，dp[i][j-1]比较
            if i>0:
                tp = min(tp, dp[i-1][j])
            if j>0:
                tp = min(tp, dp[i][j-1])

            dp[i][j] = (tp+grid[i][j])
    return dp[-1][-1]
```

```
if __name__=='__main__':
    grids=[[1,3,1],[1,5,1],[4,2,1]]
    print get_min_path_sum(grids, 3,3)#结果为 7
```

最后留一道有关三角形最小路径和的思考题如下，供大家参考和练习：

给定一个三角形，找出自顶向下的最小路径和。每一步只能移动到下一行中相邻的结点上。

例如，给定三角形：

```
L=[
     [2],
    [3,4],
   [6,5,7],
  [4,1,8,3]
]
```

自顶向下的最小路径和为 11（即，2 + 3 + 5 + 1 = 11）。

提示：在建立状态转移方程的时候，需要注意三角形中两边的点只有一条路径可以到达，而中间的点，有两条路径可以到达。因此从三角顶点到任意一点(i, j)最小路径和的状态转移方程为：

$$F(i,j)=\begin{cases} F(i-1,j)+L[i][j] & j=0 \text{ 点在三角形左边边上} \\ F(i-1,j-1)+L[i][j] & j=len(L[i])-1 \text{ 点在三角形右边边上} \\ \min(F(i-1,j-1),F(i-1,j))+L[i][j] & 0<j<len(L[i])-1 \text{ 点在三角形内部} \end{cases}$$

### 5.6.1.5 动态规划解背包问题

接下来再看看著名的背包问题对应的动态规划解法。背包问题是动态规划的典型问题。它是是这样描述的：

有 N 件物品和一个容量是 V 的背包。每件物品只能使用一次。第 i 件物品的体积是 $v_i$，价值是 $w_i$。求解将哪些物品装入背包，可使这些物品的总体积不超过背包容量，且总价值最大。并输出最大价值。

分析：每个物品只有两种可能的情况，即在背包中或者不在背包中，也就是背包中的该物品数为 0 或 1，因此称为 0-1 背包问题。

设所求的问题 F(i, j)是在背包容量为 j 时，前 i 个物体所能让背包达到的最大价值。那么很显然 F(0, j)（0<=j<=V）为 0。对于每一个物品，有两种结果：能装下或者不能装下。如果包的容量比物品体积小，装不下，这时背包的最大价值和前 i-1 个物品的最大价值是一样的；如果还有足够的容量装下该物品，但是装了不一定大于当前相同体积的最优价值，所以要进行比较。

于是可以得出子问题递推式：

$$F(i,j) = \begin{cases} \max(F(i-1,j), w_i+F(i-1,j-v_i)) & v_i \leqslant j \\ F(i-1,j) & v_i > j \end{cases} \qquad i=1,2,3,\ldots,n,\ j=0,1,2,3,\ldots V$$

状态 F(i, j)的值等于状态 F(i-1, j)、状态 F(i-1, j-$w_i$)与 $v_i$ 之和 两者中的最大值。那么要求 i 个物体在一定承重背包中可取的最大价值，只需考虑 i-1 个物体在不同承重量（0，1，2， ……，W）的背包下可取的最大价值。类似地，要想知道 i-1 个物体在一定承重的背包中可取的最大价值，只需知道 i-2 个物体在不同承重量的背包中可取的最大价值。以此类推，直到所考虑的物体个数变为 1 。

所以，只要我们知道物体个数为 1 时，在不同承重量的背包中所能取到的最大价值，就可以依次求物体个数为 2, 3, ……, n 的情况下在不同承重量的背包中所能取的最大价值。

初始条件为：i = 1 当 j ≥ $v_1$ 时，F(1, j) = $w_1$； 当 0≤j <$v_1$ 时，F(1, j) = 0。

于是我们可以写出如下 Python 代码：

```
#num：物体个数
#volume：背包总体积容量
#vlist：各个物体的体积
#wlist：各个物体的价值
def calc_max_value_for_bag(num, volume, vlist,wlist):
    dp =[[0 for j in range(volume+1)] for i in range(num+1)]
    for i in range(num):
        for j in range(volume):
            if j>=vlist[i]:
                dp[i][j]=max(dp[i-1][j],dp[i-1][j-vlist[i]]+wlist[i])
            else:
                dp[i][j]=dp[i-1][j]

    return dp[-1][-1]

if __name__=='__main__':
    v=[10,3,4,5]
    w=[3,4,6,7]
    print calc_max_value_for_bag(4,10,v,w)#13
```

通过进一步分析子问题递推式，我们还发现 F(i, j)只与它的前 i-1 个物体的状态有关，因此我们完全没有必要使用一个二维的 dp 数组来存放所有的子状态，只需要存放 i-1 个物体对应的子状态即可。于是，我们改进了上述实现代码，优化了空间复杂度：

```
#num：物体个数
#volume：背包总体积容量
#vlist：各个物体的体积
#wlist：各个物体的价值
def calc_max_value_for_bag(num, volume, vlist, wlist):
    predp =[0 for j in range(volume+1)]#备份 i-1 个物体状态
    curdp =[0 for j in range(volume+1)]#根据 i-1 个物体状态计算 i 个物体状态

    for i in range(1, num+1):
        for j in range(1, volume+1):
            if vlist[i-1]<=j:
                curdp[j]=max(predp[j], wlist[i-1]+predp[j-vlist[i-1]])
```

```
                else:
                    curdp[j]=predp[j]
            predp=curdp[:]#在计算前 i+1 个物品的时候，备份前 i 个物品的解

        return curdp[-1]

if __name__=='__main__':
    v=[10,3,4,5]
    w=[3,4,6,7]
    print calc_max_value_for_bag(4,15,v,w)#17
```

上面改进版的算法，我们已经将算法的空间复杂度由原来的 O(m*n)优化为了 O(n)。但是，依然使用了两个一维的列表（数组）。那么，还有没有进一步优化的可能呢？比如改为只使用一个列表（数组）。答案是可以只使用一个列表，但是需要注意一个问题。现在我们来尝试对上面的算法继续进行改进。

首先，我们直接将上述算法去掉一个列表，只使用一个列表，得到如下的代码：

```
#num：物体个数
#volume：背包总体积容量
#vlist：各个物体的体积
#wlist：各个物体的价值
def calc_max_value_for_bag(num, volume, vlist, wlist):
    dp =[0 for j in range(volume+1)]#缓存表

    for i in range(1, num+1):
        for j in range(1, volume+1):
            if vlist[i-1]<=j:
                dp[j]=max(dp[j], wlist[i-1]+dp[j-vlist[i-1]])
            else:
                dp[j]=dp[j]
    return dp[-1]

if __name__=='__main__':
    v=[10,3,4,5]
    w=[3,4,6,7]
    print calc_max_value_for_bag(4,15,v,w)#22
```

遗憾的是，这样修改之后，我们在测试的时候，出现了错误的最大值 22。在这种情况下，22 显然是不对的，因为 22 已经是所有物品的总价值，而我们肯定无法将所有物品放进背包，因为这些物品体积之和已经超过了背包的容量。

那么问题出现在什么地方呢？问题出现在我们计算前 i 个物品在背包容量为 j 的时候能装下的最大值 dp[j]的时候，我们需要使用到备份中前 i-1 个物品在背包容量为 j-vi 的时候的值 dp[j-vlist[i-1]]。而由于我们在计算当前前 i 个物品在容量为 j 的时候的解(dp[j])的时候，j 是从小容量往大容量开始计算，因此已经覆盖了缓存中备份的前 i-1 个物品 j 在较小容量这时候的解。所以当计算前 i 个物品在 j 为较大容量的解的时候，引用的前 i-1 个物品在 j 为较小容量时候的解时出现错误引用，错误引用成了被覆盖的当前前 i 个物品在 j 为较小容量时

候的解。

为了防止前 i-1 个物品在 j 为较小容量的值被覆盖的错误，我们可以在计算前 i 个物品在容量为 j 的解的时候，让容量 j 从大到小金星计算，这样就可以避免被覆盖。于是我们给出了如下优化之后的代码：

```
def calc_max_value_for_bag2(num, volume, vlist, wlist):
    dp = [0 for i in range(volume+1)]
    for i in range(1, num + 1):
        #j 从大到小进行计算，容量 j 小于 vlist[i-1]的值，不用计算，因为放不下
        for j in range(volume, vlist[i-1]-1, -1):
                dp[j] = max(dp[j-vlist[i-1]]+wlist[i-1], dp[j])#避免了数据污染问题
    return dp[-1]
```

前面的算法，我们得到了将这些物品装入背包之后，背包能容纳的物品的最大价值。那么我们如何知道有哪些物品被放进了背包呢？我们可以利用在前面算法执行过程中生成的 dp 表，从最后一个物品(i=num)和背包容量(j=volume)开始，从大到小，判断第 i 个物品是否可以被放入背包，判断的条件为：

dp[i][j]>dp[i-1][j]

如果上述条件成立，那么第 i 个物品就可以放入背包。放入之后，背包的容量减少了 vlist[i-1]，因此容量 j 变为：

j -= vlist[i-1]

然后，继续对第 i-1 个物品利用同样的方法进行判断，直至第 1 个物品。上述逻辑实现的 Python 代码如下：

```
def calc_max_value_for_bag(num, volume, vlist, wlist):
    dp =[[0 for j in range(volume+1)] for i in range(num+1)]

    for i in range(1, num+1):
        for j in range(1, volume+1):

            if vlist[i-1]<=j:
                dp[i][j]=max(dp[i-1][j], wlist[i-1]+dp[i-1][j-vlist[i-1]])
            else:
                dp[i][j]=dp[i-1][j]

    flag = [False for i in range(num)]#记录第 i 个物品是否放入背包，默认为 False
    j = volume
    for i in range(num, 0, -1):
        if dp[i][j] > dp[i - 1][j]:
            flag[i - 1] = True #第 i 个物品放入了背包
            j -= vlist[i - 1]
    print '背包中所装物品为第:',
    for i in range(num):
        if flag[i]:
```

```
                print i, #打印放入背包的物品，物品的序号从0开始

        print '个'


        return dp[-1][-1]

if __name__=='__main__':
        v=[10, 3, 14, 5]
        w=[3, 4, 6, 7]

        print calc_max_value_for_bag(4, 15, v, w)
```

我们把 0-1 背包问题进行扩展，使每种物品的数量无限增加，便得到完全背包问题：有一个容积为 V 的背包，同时有 n 个物品，第 i(0≤i≤n）个物品均有各自的体积 $v_i$ 和价值 $w_i$，每个物品的数量均为无限个，求使用该背包最多能装的物品价值总和。

对于完全背包问题，虽然每种物品有无限件，但背包容量的固定性导致每种最多只能取某个值，再多就放不下了，这个值就是 $k=V/v_i$。也就是说，对于第 i 种物品，它可以取 0, 1, 2, ……，$V/v_i$（向下取整）件。而在 01 背包中，对于第 i 种物品，只能取 0, 1 件。所以我们可以用类似 01 背包的思路写出完全背包的基本递推式：

$$F(i, j) = \max(F(i-1, j-kv_i)+kw_i) \quad 0 \leqslant k \leqslant V/v_i, i=1, 2, 3, ..., n, \quad j=0, 1, 2, 3, ...V$$

除了 0-1 背包问题和完全背包问题之外，还有多重背包问题。多重背包问题介于 0-1 背包和完全背包之间：有容积为 V 的背包，给定 n 个物品，第 i(0≤i≤n）个物品包含体积 $v_i$、价值 $w_i$、和数量 $k_i$，求用该背包能装下的最大价值总量。

对于多重背包问题，如果所有 $k_i$ 都满足 $k_i \geq V/v_i$，那就变成了完全背包问题。因此完全背包的基本实现思路也可以应用到多重背包的基本实现。它们主要的区别在于物品的个数上界不再是 $V/v_i$ 而是 $k_i$ 与 $V/v_i$ 中较小的那个。于是我们可以得到多重背包的基本递推式如下：

$$F(i, j) = \max(F(i-1, j-kv_i)+kw_i) \quad 0 \leqslant k \leqslant \min(V/v_i, k_i), i=1, 2, 3, ..., n, \quad j=0, 1, 2, 3, ...V$$

根据递推式，也是很容易写出计算完全背包问题和多重背包问题算法对应的程序实现代码，留给大家练习。

### 5.6.1.6 动态规划解扔鸡蛋问题

现在我们用动态规划来解决一个有关从楼上扔鸡蛋的问题。这个问题是这样描述的：

有一栋从 1 到 N 共 N 层的楼，然后给你 K 个鸡蛋（K 至少为 1）。现在确定这栋楼存在楼层 0 ≤F ≤N，在这层楼将鸡蛋扔下去，鸡蛋恰好没摔碎。高于 F 的楼层都会碎，低于 F 的楼层都不会碎，没有摔碎的鸡蛋可以继续使用，摔碎的鸡蛋不能再使用（注意，这只是一个思想实验，高空抛物很危险，也是非法的）。现在问你，最坏情况下，你至少要扔几次鸡蛋，才能确定这个楼层 F 呢？

比如当 K=1，N=2 时，鸡蛋从 1 楼掉落。如果它碎了，我们肯定知道 F = 0 。否则，鸡蛋从 2 楼掉落。如果它碎了，我们肯定知道 F = 1 。如果它没碎，那么我们肯定知道 F = 2 。因此，在最坏的情况下我们需要扔 2 次以确定 F 是多少。按照此思路，实际上，当

我们手中只有一个鸡蛋的时候，如果楼层的高度为N，那么最坏的情况就是需要扔N次，才能确定F是多少。因为我们只能从第1层开始，从下往上，逐层试验（线性扫描楼层），来确定最坏情况下扔鸡蛋的次数。

要解决这个问题，最关键的地方，就是要理解这个“最坏的情况至少要扔多少次”中的“最坏”和“至少”。

首先，为了透彻理解问题，我们暂不考虑鸡蛋个数的限制，也就是假设我们手里有无数个鸡蛋，假设楼房有11层。现在我们从这11层楼房中找到鸡蛋恰好摔碎的楼层。

首先最简单的方法就是从第1层开始，从下往上，逐层开始扔。如果1楼没有碎，就去2楼扔，2楼没有碎，就去3楼扔，...。在这种策略下，最坏的情况就是扔完整个11层共11次，鸡蛋仍然没有碎，从而确认出F=11。而如果当我们扔到第1层，就发现鸡蛋碎了，这个时候确定F=0，这不是最坏的情况，这是最好的情况。

当然，这种方法，是一种对楼层的线性扫描，不是一种好的策略。题目中所要求的是在最坏的情况下至少要扔的次数。而“至少”，很显然是在提醒我们要找到一个比线性扫描更好的策略来扔鸡蛋。在依然不考虑鸡蛋数量限制的时候，我们很容易想到的方法就是二分查找算法。

如果利用二分查找算法，那么我们就不是从第一层开始扔了，而是从楼层的中间位置：（1+11）/2=6层开始扔。如果鸡蛋在6层碎了，那么说明F小于6，那么就去(1+5)/2=3层开始扔；如果鸡蛋在6层没有碎，那么说明F大于等于6，那么就去(7+11)/2=9层开始扔。

在二分查找这种策略下，最坏情况应该是试到第 11 层鸡蛋还没碎（F = 11），或者鸡蛋一直碎到第 1 层（F = 0）。而无论这两种中的哪种最坏情况，都只需要试 $\log_2 11$ 向上取整等于4 次，比刚才尝试 11 次要少，这就是所谓的至少要扔几次。

但是，二分法是在鸡蛋个数无限制的情况下才能使用的。而本问题中，鸡蛋的个数是有限制的。

比如你手中只有一个鸡蛋，面对11层楼，如果使用二分查找，万一在第6层鸡蛋摔碎了，那么试验就无法继续进行了。所以这种情况，只能使用逐层线性扫描，那么最坏的情况下至少要扔的次数为11。

那么可不可以把二分查找和线性扫描结合起来解决问题呢？也就是当鸡蛋的个数大于等于2的时候，使用二分查找，当只剩下一个鸡蛋的时候，使用线性扫描楼层的方法。

比如有一个20层的楼房，手里有2个鸡蛋。那么用二分查找从第10层开始扔，这个时候鸡蛋就碎了，那么剩下一个鸡蛋，就需要从1到9层做线性扫描。因此在最坏的情况下总共扔需要扔10次。而事实证明，这并不是最优解。

对于这种求最值的最优化问题，我们现在尝试用动态规划来解决。前面我们已经指出，利用动态规划来解决问题，最重要的就是要分析出状态转移方程和状态转移方程的终止条件。

设F(K, N)表示在鸡蛋个数为K，楼层数为N时，在最坏情况下至少要扔鸡蛋的次数。那么，假设我们在第i层(1≤i≤N）扔完鸡蛋，如果鸡蛋碎了，那么鸡蛋的个数减为K-1，扔鸡蛋的楼层区间变为[1, i-1]共i-1层，最坏情况最少得扔F(K-1, i-1）次；如果鸡蛋没有碎，那么鸡蛋的个数仍为K，而扔鸡蛋的楼层区间变为[i+1, N]共N-i层，最坏情况最少得扔F(K, N-i）次。那么在第i层时，最坏的情况的次数为:max(F(K-1, i-1), F(K, N-i))+1，因为要加上在第i楼扔的这一次。

那么可以得出状态转移方程为：

$$F(K, N)= \min(\max(F(K-1, i-1), F(K, N-i))+1) \quad 1\leqslant i\leqslant N$$

其中边界条件为：当K=1时，F(1, N)=N;当N=0时，F(K, 0)=0。

根据状态转移方程，我们很容易写出实现算法的Python代码如下：

```
def dropEgg(k, n):
    dp = [[0 for i in range(n+1)] for j in range(k+1)]

    for i in range(1, k+1):
        for j in range(1, n+1):
            if i==1:#只有一个鸡蛋
                dp[i][j]=j
            else:
                res = float('inf')
                for x in range(1, j+1):#对于j层的楼房，从其中x层扔
                    #最坏（max），至少（min）
                    res = min(res,max(dp[i][j-x], dp[i-1][x-1])+1)
                dp[i][j]=res#缓存结果

    return dp[-1][-1]

if __name__=='__main__':
    print dropEgg(2, 100)#14
```

动态规划问题一直是知名互联网公司面试时最频繁出现的算法题，因为动态规划是一种比较有难度，灵活度高，没有很明显的套路，又比较容易出的面试问题。在掌握了本书介绍的动态规划的基本思想之后，大家可以继续拓展学习，也可以参考 lintcode 上的动态规划问题的专题：https://www.lintcode.com/problem/?tag=dynamic-programming 或者 LeetCode 上的专题：https://leetcode-cn.com/tag/dynamic-programming/。

目前，动态规划的应用极其广泛，包括工程技术、经济、工业生产、军事以及自动化控制等领域，并在背包问题、生产经营问题、资金管理问题、资源分配问题、最短路径问题和复杂系统可靠性问题等中取得了显著的效果。

## 5.6.2 分治算法

分治算法的思想就是“分而治之”，就是把一个复杂的问题分成两个或更多的相同或相似的子问题，再把子问题分成更小的子问题，...，直到最后子问题可以简单的直接求解，原问题的解就是把所有子问题的解进行合并。

对于一个规模为 n 的问题，若该问题可以容易地解决（比如说规模 n 较小）则直接解决，否则将其分解为 k 个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。

分治法所能解决的问题一般具有以下几个特征：

特征 1：该问题的规模缩小到一定的程度就可以容易地解决。

特征 2：该问题可以分解为若干个规模较小的相同问题，即该问题具有最优子结构性质。满足特征 1 和特征 2，那么就可以用递归实现分治算法。

特征 3：该问题分解出的子问题的解可以合并为该问题的解；这是分治算法的关键特征。如果只满足特征 1 和 2，而不满足特征 3，那么可以考虑用动态规划或者贪心算法解决。

特征 4：该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子子问题。如果问题不满足特征 4，虽然也可以用分治算法，但由于涉及重复计算，所以效率会比较差，此时最好使用动态规划来解决。

可使用分治法求解的一些经典问题包括：二分搜索，归并排序，快速排序 ，傅立叶变换等等。

### 5.6.2.1 快速排序

快速排序的思想是：在当前无序区 R[1..H]中任取一个数据元素作为比较的“基准”(不妨记为 X，R[1])，用此基准将当前无序区划分为左右两个较小的无序区：R[1..I-1]和 R[I+1..H]，且左边的无序子区中数据元素均小于等于基准元素 X，右边的无序子区中数据元素均大于等于基准元素 X，而基准 X 则位于最终排序的位置上，即 R[1..I-1]≤X≤R[I+ 1..H](1≤I≤H)，当 R[1..I-1]和 R[I+1..H]均非空时，分别对它们重复进行上述的划分过程，直至所有无序子区中的数据元素均已排序为止。

也就是：找一个数 X 做基准（一般就是开头的数），右边比 X 小的移动到左边（移动后，右边空出一个位置），左边比 X 大的移动到右边（移动后左边空出一位置），最后空出的位置就是 X 自己的位置。

比如，对于下列的一组数：

3,6,1,2,8,4,7

如果用快速排序，那么排序过程如下：

```
def quick_sort(array, low, high):
    if low >= high:
        return array

    i=low
    j=high
    pivot = array[i]

    while i < j:
        while i < j and array[j] >= pivot:
            j -= 1
        array[i]=array[j]
        while i < j and array[i] <=pivot:
            i += 1
        array[j]=array[i]
    array[j] = pivot
    quick_sort(array, low, i-1)
    quick_sort(array, i+1, high)
    return array

if __name__=='__main__':

    arr = [13, 4, 2, 9, 7, 5,21,11]
```

```
n = len(arr)
quick_sort(arr, 0, n-1)
print arr
```

### 5.6.2.2 TOP-K 问题与 BFPRT 算法

现在我们再来研究一下如何用分治算法解决一个 TOP-K 问题。TOP-K 问题是这样描述的：

给定一个存放 n 个元素的数组和一个整数 k，1≤k≤n，要求找出这 n 个元素中第 k 大或者 K 小的元素。

对于 TOP-K 问题，我们当然可以先对所有数据进行排序，然后取其前 k 即可，但是这么做显然不是最优的，因为我们只需要前 k 大（小）的，而对其余不需要的数也进行了排序，浪费了大量多余的排序时间。

于是我们想到了快速排序，借鉴快速排序的思想，我们可以随机在数组中选择一个数作为划分值（number），然后进行快排的分区（partation）过程（将小于 number 的数放到数组左边，等于 number 的数放到数组中间，大于 number 的数放到数组右边），然后判断 k 与等于 number 区域的相对关系，如果 k 正好在等于 number 区域，那么数组第 k 小的数就是 number，如果 k 在等于 number 区域的左边，那么我们递归对左边再进行上述过程，如果 k 在等于 number 区域的右边，那我们递归对右边再进行上述过程。这种方法，在最好的情况，即每次所选的 number 正好在数组的正中间，复杂度为 O（N）；而在最坏情况的时候，即每次所选的 number 正好在数组最边上，那么时间复杂度为 O（$N^2$）。那么有没有什么方法，可以在最好和最坏情况下，时间复杂度都为 O(N)线性的呢？有，这就是 BFPRT 算法。

目前解决 TOP-K 问题最有效的算法即是 BFPRT 算法，又称为中位数的中位数算法，该算法由 Blum、Floyd、Pratt、Rivest、Tarjan 提出，最坏时间复杂度为 O(n)。在快速排序的基础上，首先通过判断主元（pivot）位置与 k 的大小使递归的规模变小，其次通过修改快速排序中主元的选取方法来降低快速排序在最坏情况下的时间复杂度。

算法的实现步骤为：

1. 数组被划分为了$\lceil N/5 \rceil$（N/5 向上取整）个小部分，每个部分的 5 个数排序需要 O(1)，所有部分排完需要 O(N/5)=O(N);

2. 取出每个小部分的中位数，一共有 $\lceil N/5 \rceil$ 个，递归调用 BFPRT 算法得到这些数中第 $\lceil N/5 \rceil/2$ 小的数（即这些数的中位数），记为 pivot； 递归出口（最小子问题）为在某次递归调用 BFPRT 算法时发现这个区域只有一个数，那么这个数就是我们要找的数。通过这种方法找出来的 pivot，就能够保证这个数最接近是整个数组排好序之后的中间位置。

如图 5-10 所示，图中每一列表示包含 5 个元素排好序之后的分组，区域 C 为每个分组的中位数（小圆表示元素，元素之间的箭头表示小于关系）。区域 C 的中间位置，就是我们通过 BFPRT 递归找到的中位数 pivot。通过大小关系，我们可以确定，区域 B 中的数是一定大于 pivot 的，区域 A 中的数是一定小于 pivot 的。而区域 A 和 B 中的数各自共有：(N/5)/2+2*（N/5)/2=3N/10。也就是说，即使在最坏的情况下，pivot 也比 3N/10 个数要大，比 3N/10 个数要小，因此 pivot 处于数组中[3N/10, 7N/10]之间，也就是说 pivot 在最坏的情况下，也

会处于数组中 3N/10 或者 N-3N/10=7N/10 的位置。

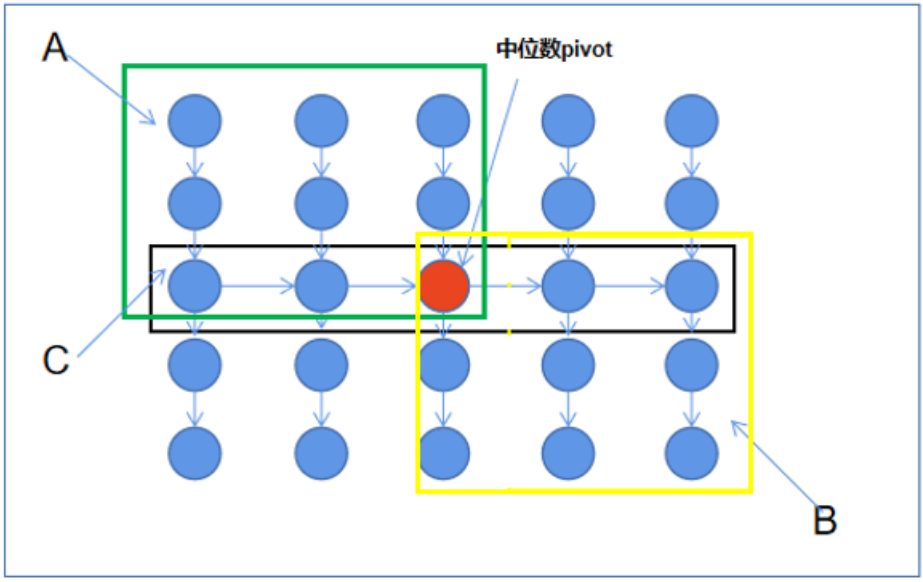



图 5-10 BFPRT 算法中位数划分

因此，以此 pivot 为划分的分区（partition）过程能够使得 pivot 在数组的靠近中间的位置。那么接下来就和快速排序一致了。

3. 以 pivot 作为比较，将整个数组划分为 <pivot , =pivot , >pivot 三个区域；

4. 判断第 K 小的数在哪个区域，如果在 = pivot 区域则直接返回 pivot ，如果在 < pivot 或 > pivot 区域，则将这个区域的数递归调用 BFPRT 算法。

所以，与快速排序相比，唯一的区别就是步骤 1 和 2 中选取 pivot 的方法不同。BFPRT 中选取的 pivot 能够非常接近整个数组的中位数，从而降低了算法复杂度。

比如对于下列一组数：

8, 3, 5, 1, 10, 7, 2, 13, 11, 27, 4, 9, 53, 6, 17, 25, 16, 19,68, 72, 93,24,37,56,21, 22,35,81

这组数共有 28 个，如果使用 BFPRT 算法，那么排序过程为：

1.先按照 5 个一组进行分组，因为 $\lceil 28/5 \rceil = 6$ ，所以共分为 6 个组：

8, 3, 5, 1, 10　7, 2, 13, 11, 27　4, 9, 53, 6, 17　25, 16, 19,68,72,　93,24,37,56,21　22,35,81

2.对这 6 组数据进行排序（任何排序算法皆可，比如插入，选择等）：

1,3,5,8,10　2,7,11,13,27　4,6,9,17,53　16,19,25,68,72　21,24,37,56,93　22,35,81

3.取出每个小组中的中位数组成一个新的数列：

5， 11， 9， 25，37，35

4.递归调用 BFPRT 获得上面数中 K=6/2=3 小的数。在递归的时候，继续对上面的数按 5 个一组进行分组：

5,11,9,25,37　35

对每个分组进行排序：

5,9,11,25,37　35

取出各自的中位数：

11, 35

继续递归找出上面数中 K=2/2=1 小的数。由于已经不足 5 个数，所以在同一个分组里，然后排序：

11,35

取出其中的中位数 11，然后 11 组成了一个中位数数组：

11

对这个中位数数组递归调用 BFPRT，由于只剩一个元素，所以 11 就是要找的中位数。因此设置 pivot=11。而检验发现，11 处于排序之后的整个数组：

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, **11**, 13, 16, 17, 19, 21, 22, 24, 25, 27, 35, 37, 53, 56, 68, 72, 81, 93

第 11 个位置，确实处于[3N/10, 7N/10]，将 N=28 代入，即[8, 19]之间。

然后将 pivot=11 对整个数组进行 partition 分区，所有小于 11 的放在 11 的左边，所有大于 11 的放在 11 的右边。于是有：

8, 3, 5, 1, 10, 7, 2, 4, 9, 6, **11**, 81, 53, 27, 17, 25, 16, 19, 68, 72, 93, 24, 37, 56, 21, 22, 35, 13

分区结束之后，pivot 也就是数 11 所在的位置为 10。所以，如果是想找数组中第 k 个数，且 k=10（从 0 开始计算），那么 11 就是这个要找的数。否则，如果 k<10，那么就从 11 左边区间递归调用 BFPTR 查找，如果 k>10，那么就从 11 右边区间递归调用 BFPTR 查找。

根据上述分析，我们给出 BFPRT 算法实现的 Python 代码和注释如下：

```
def swap(a, b):
    return b, a

def insert_sort(arr, left, right):
    for i in range(left+1, right+1):
        tmp = arr[i]
        j = i-1
        while j>=left and arr[j]>tmp:
            arr[j+1]=arr[j]
            j -= 1
        arr[j+1] = tmp

    return (right - left)/2 + left

def partition(arr, left, right, pivot_index):
    partition_index = left
    arr[pivot_index], arr[right] = swap(arr[pivot_index], arr[right])

    for i in range(left, right):
        if arr[i] < arr[right]:
            arr[partition_index], arr[i] = swap(arr[partition_index], arr[i])
            partition_index += 1
    arr[partition_index], arr[right] = swap(arr[partition_index], arr[right])

    return partition_index


def get_pivot_index(arr, left, right):
    if right - left <5:
        return insert_sort(arr, left, right)
```

```
        i = left
        sub_right = left - 1
        while i<=right:
            index = insert_sort(arr, i, min(right,i+4))
            sub_right += 1
            arr[sub_right], arr[index] = swap(arr[sub_right], arr[index])
            i += 5

        return BFPRT(arr, left, sub_right, (sub_right-left+1)/2+1)


    def BFPRT(arr, left, right, k):

        pivot_index = get_pivot_index(arr, left, right)
        partition_index = partition(arr, left, right, pivot_index)

        pos = partition_index
        if pos == k-1:
            return partition_index
        elif pos > k-1:
            return BFPRT(arr, left, partition_index-1, k)
        else:
            return BFPRT(arr, partition_index+1, right, k)

    if __name__ == '__main__':
        arr =[11, 9, 10, 1, 13, 72, 18]
        print arr

        index = BFPRT(arr, 0, len(arr)-1, 1)
        print arr

        print index, arr[index]
```

### 5.6.3 贪心算法

贪心算法是常见算法之一，它简单易行，构造贪心策略容易。贪心算法的思想就是：在对问题求解时，总是做出在当前看来是最好的选择。不从整体最优上加以考虑，所做出的仅是在某种意义上的局部最优解。

与分治算法类似，我们也是把求解的问题分成若干个子问题。然后对每个子问题求解，得到子问题的局部最优解。把子问题的解局部最优解合成原来问题的一个解。用贪心算法只能通过解局部最优解的策略来达到全局最优解，因此，一定要注意判断问题是否适合采用贪

心算法策略，找到的解是否一定是问题的最优解。因此贪心算法需要证明后才能真正运用到题目的算法中。

比如，对于下面的背包问题，可以试用贪心算法求解，但其结果却不是最优解。

有一个背包，背包承重的最大容量是 M=110。有 7 个物品，如表 5-2 所示，物品可以分割成任意大小。要求尽可能让装入背包中的物品总价值最大，但不能超过总容量。

表 5-2 背包所装物品重量与价值

| 物品 | A | B | C | D | E | F | G |
|---|---|---|---|---|---|---|---|
| 重量 | 25 | 30 | 35 | 50 | 60 | 10 | 25 |
| 价值 | 10 | 20 | 30 | 40 | 50 | 45 | 60 |

设装入背包中的第 i 个物体的价值为 pi，重量为 wi，那么目标函数： Σpi 最大。 约束条件是装入的物品总重量不超过背包容量：Σwi<=M( M=110)。

那么根据贪心算法的策略，可以每次挑选价值最大的物品装入背包，或者每次挑选所占重量最小的物品装入，或者每次选取单位重量价值 pi/wi 最大的物品。

但很遗憾的是，无论使用哪种策略，最后得到的结果都有可能不是全局最优解。我们可以举出如下反例：

假设有 3 个物品，重量分别为 18、 13、15， 价值分别为 30、20、15，背包容量为 30。如果按照单个价值最大策略挑选物品，那么将会挑选重量为 18，价值为 30 的物品。但我们很容易看出，这不是最优解，因为挑选重量分别为 13 和 15 的物品，总价值为 35，超过了价值 30，因此后者才是最优解。

如果按照重量最小为策略进行挑选，那么我们也可以举出反例，比如：有 3 个物品，重量分别为 18、 13、15， 价值分别为 40、20、15，背包容量为 30。如果按照重量最小策略挑选物品，那么将会挑选重量为 13 和 15 的物品，价值之和为 20+15=35。但我们很容易看出，这不是最优解，因为挑选重量为 18 的物品，总价值为 40，超过了价值 35，因此后者才是最优解。

如果按照单位重量价值最大策略来挑选，对应的反例与单个价值最大策略相同。这时，由于 30/18≈1.66， 20/13≈1.53，15/15=1，因此应该挑重量为 18，价值为 30 的。但我们刚才已经分析了，这不是全局最优解。

因此贪心算法无法完成对背包问题的解决。

当然，贪心算法并不是完全不可以使用，贪心策略一旦经过证明成立后，它就是一种高效的算法。

比如，对于“钱币找零”问题，就可以使用贪心算法来解决。这个问题如下：

假设 1 元、2 元、5 元、10 元、20 元、50 元、100 元的纸币分别有 c0, c1, c2, c3, c4, c5, c6 张。现在要用这些钱来支付 K 元，至少要用多少张纸币？

用贪心算法的思想，很显然，每一步尽可能用面值大的纸币即可。于是，我们给出了如下实现算法的 Python 代码：

```
#change 为要找的零钱数
#valuelist 为纸币的面值，从大到小排列
#numlist 为每个面值纸币对应的数量

def get_min_change(change, valuelist, numlist):
    num = 0
    result = []

    while change>0:
```

```
            for i in range(len(valuelist)):
                if change>=valuelist[i]:
                    count = min(change/valuelist[i], numlist[i])
                    change -= valuelist[i]*count

                    result.append(count)
                    num += count

                else:
                    result.append(0)

        return num, result


if __name__=='__main__':
        valuelist=[100, 50, 20, 10, 5, 2, 1]
        numlist = [2, 5, 3, 2, 4, 1, 9]

        print get_min_change(297, valuelist, numlist)
```

### 5.6.4 回溯算法

回溯法是一种选优搜索法，又称为试探法，按选优条件向前搜索，以达到目标。但当探索到某一步时，发现原先选择并不优或达不到目标，就退回一步重新选择，这种走不通就退回再走的技术为回溯法，而满足回溯条件的某个状态的点称为“回溯点”。

在包含问题的所有解的解空间树中，按照深度优先搜索的策略，从根结点出发深度探索解空间树。当探索到某一结点时，要先判断该结点是否包含问题的解，如果包含，就从该结点出发继续探索下去，如果该结点不包含问题的解，则逐层向其祖先结点回溯。

若用回溯法求问题的所有解时，要回溯到根，且根结点的所有可行的子树都要已被搜索遍才结束。

而若使用回溯法求任一个解时，只要搜索到问题的一个解就可以结束。

回溯法最经典的应用就是解决 N 皇后问题：

有 N 个皇后棋子，如何在 N*N 的棋盘中放置这 N 个皇后，使得任意两个皇后都不在同一条横线、纵线或者斜线上。如图 5-11 所示，这是一种 8 皇后在 8*8 的棋盘上的一种布局方法。

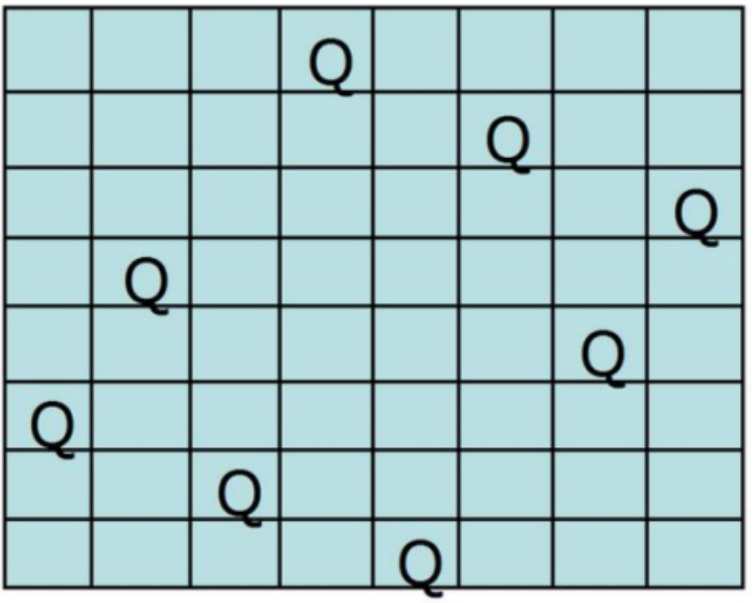


图 5-11 八皇后的一种布局（Q 表示皇后，英语皇后 Queen 的简写）

皇后的走法规则为：可以横着走，直着走或者斜着走，格数不限。因此要求皇后彼此之间不能相互攻击，等价于要求任何两个皇后都不能在同一行、同一列以及同一条斜线上。

很显然，每个皇后必须位于不同行和不同列，因此将 N 个皇后放置在 N×N 的棋盘上，一定是每一行有且仅有一个皇后，每一列有且仅有一个皇后，且任何两个皇后都不能在同一条斜线上。可以通过回溯的方式寻找可能的解。

回溯的具体做法是：使用一个数组记录每行放置的皇后的列下标，依次在每一行放置一个皇后。每次新放置的皇后都不能和已经放置的皇后之间有攻击：即新放置的皇后不能和任何一个已经放置的皇后在同一列以及同一条斜线上，并更新数组中的当前行的皇后列下标。当 N 个皇后都放置完毕，则找到一个可能的解。当找到一个可能的解之后，将数组转换成表示棋盘状态的列表，并将该棋盘状态的列表加入返回列表。

由于每个皇后必须位于不同列，因此已经放置的皇后所在的列不能放置别的皇后。第一个皇后有 N 列可以选择，第二个皇后最多有 N−1 列可以选择，第三个皇后最多有 N−2 列可以选择同时还需要排除在同一条斜线上的情况，因此所有可能的情况不会超过 N! 种。

为了降低总时间复杂度，每次放置皇后时需要快速判断每个位置是否可以放置皇后。因此，我们需要记录下放置皇后后被占用的列号，以及放置皇后位置对应的左右斜线上的点。

我们可以直接把被占用的列号存放在一个数组里，这样每次为放置皇后而在某一行选择新列号的时候，只要发现列号已经在这个数组里了，那么这一列就被排除了。

但是，如何记录被这个位置占用的斜对角线呢？

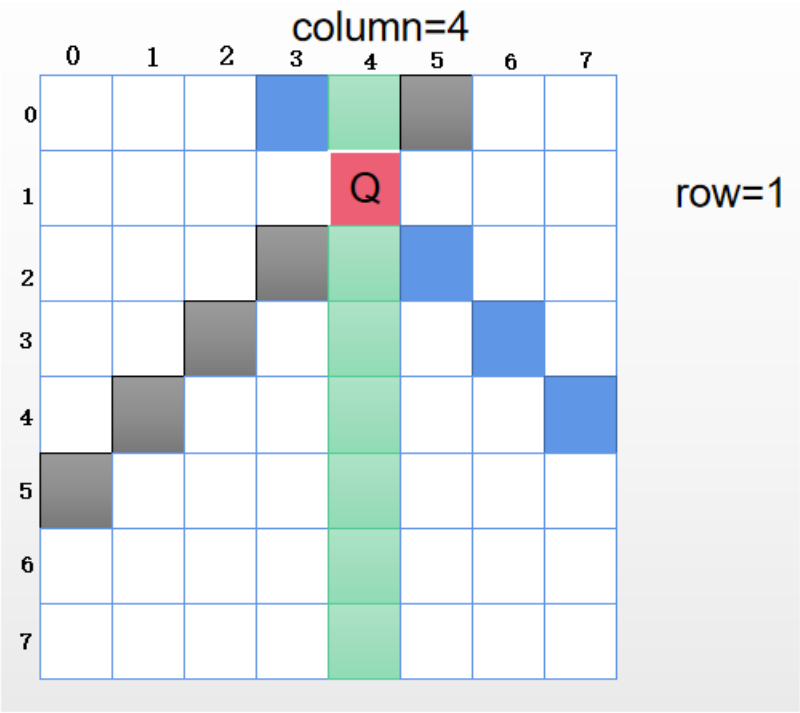



图 5-12 摆置皇后的位置对所在列和斜线位置的影响

如图 5-12 所示，当在第 1 行第 4 列的位置点（1, 4）放下皇后 Q 之后，我们发现它对应的左边斜线和右边斜线的所有位置坐标都有这样一个重要性质：左边斜线坐标点的行、列坐标之和等于当前点（1, 4）行、列坐标之和：1+4=5；右边斜线坐标点行、列坐标之差等于当前点（1, 4）行、列坐标之差：1-4=-3。因此，我们只需要把当前放皇后的点的行、列坐标之和与之差记录在两个数组里，就可以在下一次放皇后的时候排除掉那些斜线上行、列之和与之差等于这些值的点。

经过分析之后，我们便实现了 N 皇后对应的 Python 代码及注释如下：

```
def calc_n_queens(n):
    def make_board():
        board = list()
        for i in range(n):
            row[queens[i]] = "Q"#将每行对应的皇后列设置为 Q
            board.append("".join(row))#将本行放入 board 棋盘里
            row[queens[i]] = "."#清掉该行的 Q，为计算下一行准备
        return board #一个 N*N 的棋盘，里面是一个 N 皇后的布局信息

    def backtrack(row):#row 为行号
        if row == n:#row 到最后一行，棋盘完成皇后布局
            board = make_board()#生成棋盘布局
            solutions.append(board)#将本次棋盘布局放到 solutions 列表中
        else:
            for i in range(n):#对每一列 i
                if i in columns or row - i in diagright or row + i in diagleft:
                    continue
                queens[row] = i#找到 row 行可以放置皇后的列号，并记录在 queen 里
```

```
                    columns.add(i)#第 i 列被占用
                    diagright.add(row - i)#(row, i)坐标对应的右对角线，row-i 为定值
                    diagleft.add(row + i)#(row, i)坐标对应的左对角线，row+i 为定值
                    backtrack(row + 1)#递归处理下一行，体现深度优先搜索，完成一次布局
                                #深度优先搜索结束，找到一个可能的解或者无解

                    #当前列 i 已经搜索完成。回退回来（回溯）
                    #移除 i，row-i, row+i，并对 i+1，进行下一次搜索
                    columns.remove(i)
                    diagonal1.remove(row - i)
                    diagonal2.remove(row + i)

        solutions = list()
        queens = [-1] * n#用于记录每行放置皇后的列号
        columns = set()#用于记录被占用的列号
        diagleft = set()#用于记录被皇后占用的左边对角线
        diagright = set()#用于记录被皇后占用的右边对角线
        row = ["."] * n#用于棋盘每行显示，由字符.和 Q 组成
        backtrack(0)#开始从第 0 行进行深度优先搜索
        return solutions

if __name__=='__main__':
        print calc_n_queens(8)
```

接下来，我们再来看一个用回溯法解 n 个数字的全排列问题。问题描述如下：

给定一个没有重复数字的序列，返回其所有可能的全排列。示例:

输入: [1, 2, 3]

输出:

```
[
  [1, 2, 3],
  [1, 3, 2],
  [2, 1, 3],
  [2, 3, 1],
  [3, 1, 2],
  [3, 2, 1]
]
```

对于 n 个数字的全排列，我们知道共有 n！个。我们可以把这个问题看成有 n 个排列成一行的空格，我们需要从左往右依此填入题目给定的 n 个数，每个数只能使用一次，而 pos 表示当前要填入数字的位置。

因此，我们可以使用 backtrack(pos, output)函数来完成 n 个数字的排列工作。其中 pos 表示当前要填入数字的位置，output 表示已经排列好的数字。我们可以从剩余没有使用的数字里选择一个填入到 pos 的位置，然后继续递归调用 backtrack(pos+1, output)。当 pos 等于 n 的时候，那么 0 到 n-1 的位置都已经填好了数字，于是此次排列结束，output 里就存放着本次的一个完全排列。

此种方法里，有一个关键的地方就是如何记录排列中已经被使用的数字，因为数字只能

使用一次。很容易想到的就是我们可以再生成一个表（比如list），用于记录已经被使用的数字。但是这样又增加了空间复杂度。

实际上，假设这 n 个数字存放在 nums[]列表中那么，很显然在[0, pos-1]区间的数字是已经使用过了，而[pos, n-1]区间中的数字是没有使用的。于是，我们可以从[pos, n-1]区间中选择一个数字 num[i]填入到 pos 这个位置，而 pos 这个位置原来的数字则交换到 num[i]这个位置。也就是：

num[pos], num[i]=num[i], num[pos]

当 pos 这个位置填好数字之后，再递归，填下一个 pos+1 位置的数字：

backtrack(pos+1)

直到 pos 等于 n，那么所有的数字都填好了。由于需要求出所有的排列可能，因此，从 pos 位置深度优先搜索找到解之后，还需要回溯到上一层，寻找下一个解。因此必须将 pos 和 i 位置的值恢复到交换之前的状态，也就是再执行一次交换：

num[pos], num[i]=num[i], num[pos]

于是，根据上述分析，我们可以实现如下的 Python 算法代码：

```
def permute(nums):

    def backtrack(pos):
        if pos == n:#一个排列完成
            res.append(nums[:])
        else:
            for i in range(pos, n):
                nums[pos], nums[i] =    nums[i], nums[pos]
                backtrack(pos+1)#递归，深度优先
                nums[pos], nums[i] = nums[i], nums[pos]#回溯

    n = len(nums)
    res = []

    backtrack(0)

    return res

if __name__ == '__main__':
    L = [1, 2, 3, 4]

    print permute(L)
```

## 5.6.5 分支限界算法

分支限界算法类似于回溯法，也是一种在问题的解空间树 T 上搜索问题解的算法。但在一般情况下，分支限界法与回溯法的求解目标不同。回溯法的求解目标是找出 T 中满足约束条件的所有解，而分支限界法的求解目标则是找出满足约束条件的一个解，或是在满足

约束条件的解中找出使某一目标函数值达到极大或极小的解，即在某种意义下的最优解。

分支限界算法采用广度优先的策略，依次搜索 E-结点（即扩展结点）的所有分支，也就是所有相邻结点，抛弃不满足约束条件的结点，其余结点加入活结点表。然后从表中选择一个结点作为下一个 E-结点，继续搜索。

根据选择下一个 E-结点的方式不同，则会有几种不同的分支搜索方式：

1）基于队列的先进先出（FIFO）搜索

2）基于栈的后进先出（LIFO）搜索

3）基于优先队列按照优先级进行搜索

分支限界法则以广度优先或以最小耗费优先的方式搜索解空间树 T。分支限界法的搜索策略是：在扩展结点处，先生成其所有的儿子结点，然后再从当前的活结点表中选择下一个扩展对点。为了有效地选择下一扩展结点，以加速搜索的进程，在每一活结点处，计算一个用作限界的函数值，并根据这些已计算出的函数值，从当前活结点表中选择一个最有利的结点作为扩展结点，使搜索朝着解空间树上有最优解的分支推进，以便尽快地找出一个最优解。

分支限界法常以广度优先或以最小耗费优先的方式搜索问题的解空间树。问题的解空间树是表示问题解空间的一棵有序树。在搜索问题的解空间树时，分支限界法与回溯法对当前扩展结点所使用的扩展方式不同。在分支限界法中，每一个活结点只有一次机会成为扩展结点。活结点一旦成为扩展结点，就一次性产生其所有儿子结点。在这些儿子结点中，那些导致不可行解或导致非最优解的儿子结点被舍弃，其余儿子结点被子加入活结点表中。此后，从活结点表中取下一结点成为当前扩展结点，并重复上述结点扩展过程。这个过程一直持续到找到所求的解或活结点表为空时为止。而在回溯法深度优先搜索算法中，栈中的活结点的所有可行子结点被遍历后才被从栈中弹出，找出满足约束条件的所有解。分支限界法广度优先或最小消耗优先搜索队列、优先队列每个结点只有一次成为活结点的机会找出满足约束条件的一个解或特定意义下的最优解

有部分问题无论用回溯法还是分支限界法都可以得到很好的解决。现在，我们用下面这个例子来演示分支限界算法的应用：

请实现一个函数按照之字形顺序打印二叉树，即第一行按照从左到右的顺序打印，第二层按照从右到左的顺序打印，第三行再按照从左到右的顺序打印，其他行以此类推。

比如对于图 5-13 所示的二叉树：

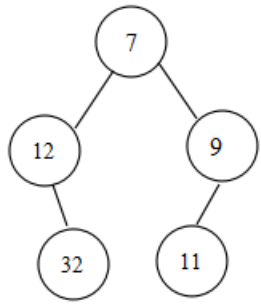



图 5-13 二叉树

那么打印的结果为：

[

[7]

[9, 12]

[32, 11]

]

问题分析：树的每一层遍历的数据放入一个队列 tmp 里，再将每一层对应的队列放入一个 res 列表里。当 res 列表（存放每一层遍历的队列 tmp）中元素个数 len(res)%2 为 0（假）时，表示目前正在遍历奇数层（因为当 len(res)等于零时，在遍历第一层，也就是奇数层）；当 len(res)%2 为 1（真）时，表示目前正在遍历偶数层。设置一个队列 deque，和用于存放每层遍历结点的队列 tmp，并用树的根节点 root 初始化 deque。当 deque 队列不为空的时候，从队列中取出一个节点，如果这个结点属于奇数层（通过上面提及的 len(res)%2 来判断），则将结点数据放入 tmp 队列的尾部；属于偶数层，则将结点数据放入 tmp 队列的头部。然后将结点的左子结点和右子结点放入 deque 队列之中。当前层所有结点遍历完之后，将 tmp 队列放入 res 列表之中。

具体的 Python 代码及算法注释如下所示：

```
import collections

class node():
    def __init__(self,v=None,l=None,r=None):
        self.val=v;
        self.left=l;
        self.right=r;

def create(root):
    a=raw_input('enter a val:')
    if a == '#':
        root=None
    else:
        root=node(v=a)
        print 'please create left tree of %s'%(a)
        root.left=create(root.left)
        print 'please create right tree of %s'%(a)
        root.right=create(root.right)
    return root

def preorder(root):          #前序遍历
    if root is None:
        return ;
    else :
        print root.val;
        preorder(root.left);
        preorder(root.right);

def levelOrder(root):
    if not root:
        return []
    res, deque = [], collections.deque([root])#root 初始化 deque
    while deque:
        tmp = collections.deque()
```

```
            for _ in range(len(deque)):#遍历同层结点
                node = deque.popleft()#取出同层一个节点
                if len(res) % 2:
                    tmp.appendleft(node.val)#偶数层结点，放入 队列头部
                else:
                    tmp.append(node.val)# 奇数层结点，放入队列尾部
                if node.left!=None:
                    deque.append(node.left)#左结点加入队列，成为下一层结点
                if node.right!=None:
                    deque.append(node.right)#右结点加入队列，成为下一层结点
            res.append(list(tmp))#将当前层结点所在的队列 tmp 加入 res 列表
        return res

    if __name__=='__main__':
        root=None
        root=create(root)
        preorder(root)
        print levelOrder(root)
```

### 5.6.6 暴力穷举算法

暴力穷举算法是利用计算机的强大计算能力来暴力穷尽问题中每一种可能的情况，从而达到求解的目的。穷举算法效率不高，可适用于一些没有明显规律可循或者没有特别的解决方案的场合。穷举算法的基本思想就是从所有可能的情况中搜索正确的答案，其执行步骤如下：

1，对于一种可能的情况，计算其结果。

2，判断结果是否满足要求，如果不满足则进行执行第 1 步来搜索下一个可能的情况；如果满足要求，则表示寻找到一个正确的答案。

现在我们就使用暴力穷举算法来解决一些实际的问题，比如鸡兔同笼问题，韩信点兵问题以及物不知数问题。

#### 5.6.6.1 暴力穷举解“鸡兔同笼”问题

中国古代著名的鸡兔同笼问题，除了可以用解方程的方法直接求解，也可以使用暴力穷举算法来求解。鸡兔同笼问题是这样描述的：

有若干只鸡和兔关在同一个笼子里，从笼子上面数，有 35 个头，从笼子下面数，有 94 只脚。问笼中各有多少只鸡和兔？

分析：设总头数为 heads，那么 heads 为鸡和兔子的总和。设鸡的总数为 i，那么 i 的所有可能取值为 0 到 heads 之间，兔子对应的总数为 heads-i。设总脚数为 feet，由于兔子有四只脚，鸡只有两只脚，那么兔子和鸡各自的总数满足的关系式为：

$$i*2+j*4=feet$$

因此，只需要通过暴力穷举鸡在 0 到 heads 之间的所有取值，并计算兔子和鸡的总数是

否满足上述关系，从而就可以确定鸡和兔子各自的总数。于是，用 Python 实现计算鸡兔同笼的暴力穷举算法如下：

```
def calc_chicken_and_rabbit(heads, feet):
    for i in range(heads+1):#i 为鸡的总数，暴力穷举 0 到 heads 所有可能
        j=heads-i #j 为兔子的总数
        if i*2+j*4==feet:#鸡和兔子满足的关系成立
            print 'chicken:%d, rabbit:%d'%(i, j)

if __name__=='__main__':
    calc_chicken_and_rabbit(35, 94)#chicken:23, rabbit:12
```

### 5.6.6.2 暴力穷举解“韩信点兵”问题

在中国古代还有一类著名的问题，被称作“韩信点兵”或者“物不知数”问题。

“韩信点兵”的问题是这样描述的：

韩信是汉朝初期著名的大将，他英勇善战，智谋超群。据说韩信的数学水平也非常高超，他在点兵的时候，为了知道有多少个兵，同时又能保住军事机密，便让士兵排队报数：

按从 1 至 5 报数，记下最末一个士兵报的数为 1；

再按从 1 至 6 报数，记下最末一个士兵报的数为 5；

再按 1 至 7 报数，记下最末一个报的数为 4；

最后按 1 至 11 报数，最末一个士兵报的数为 10。

你知道韩信至少有多少兵?

而“物不知数”也叫“孙子问题”，是在一千多年前的《孙子算经》中描述的一道算术题：

“今有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二，问物几何？”按照今天的话来说：一个数除以 3 余 2，除以 5 余 3，除以 7 余 2，求这个数。

无论是“韩信点兵”，还是《孙子算经》中的“物不知数”，它们其实是一类问题，也就是初等数论中的解同余式。“物不知数”后来被宋代的秦九韶给出了一般解法即“大衍求一术”。18、19 世纪欧拉和高斯对一次同余组进行了研究，重新独立获得了秦九韶“大衍求一术”定理，并做了严格证明。1876 年，德国马蒂生指出高斯算法和秦九韶的算法是一致的。因此被西方称为“中国剩余定理”或孙子定理。

现在我们用暴力穷举算法来解决“韩信点兵”和“物不知数”问题。

在“韩信点兵”问题中，设韩信至少拥兵总数为 x，那么由于从 1 到 5 报数，最后一个士兵报数为 1，所以 x%5=1；由于从 1 到 6 报数，最后一个士兵报数为 5，那么有 x%6=5；从 1 到 7 报数，最后一个士兵报数为 4，所以 x%7=4；从 1 到 11 报数，最后一个士兵报数为 10，所以有 x%11=10。

那么，我们可以从 x=1 开始，暴力穷举所有正整数，直到找到符合上述条件的数（当然符合这个条件的数可能不止一个，我们只需要找到最小的哪个即可），那么这个数就是韩信至少拥有的兵数。

于是，我们可以编写出如下的 Python 程序：

```
def calc_hanxin_soldiers_number():
```

```
    x = 1
    while True:
        if x%5==1 and x%6==5 and x%7==4 and x%11 == 10:
            break
        x += 1
    return x

if __name__=='__main__':
    print calc_hanxin_soldiers_number()#输出：2111
```

而“物不知数”问题的暴力穷举算法与“韩信点兵”高度类似，只是具体的约束条件不同。因此，我们直接给出对应的 Python 代码如下：

```
def calc_goods_number():

    x = 1
    while True:
        if x%3==2 and x%5==3 and x%7==2:
            break
        x += 1
    return x

if __name__=='__main__':
    print calc_goods_number()#输出 23
```

暴力穷举算法在大整数因式分解（参考本书 6.8.2 节），密码破解，区块链挖矿（参考本书 12.4 节）等领域都有实际的应用。随着待解问题数据规模的变大，暴力穷举算法的计算过程会消耗计算机大量资源，且运行过程非常缓慢，甚至无法在短时间内获得计算结果。

### 5.6.7 二分查找算法

二分查找也称折半查找（Binary Search），它是一种效率较高的查找方法，复杂度为 O(logn)。

二分查找的思想为：首先，假设表中元素是按升序排列，将表中间位置记录的关键字与查找关键字比较，如果两者相等，则查找成功；否则利用中间位置记录将表分成前、后两个子表，如果中间位置记录的关键字大于查找关键字，则进一步查找前一子表，否则进一步查找后一子表。重复以上过程，直到找到满足条件的记录，使查找成功，或直到子表不存在为止，此时查找不成功。

具体例子请参考本书 4.1 节所述。

二分查找的高效率体现在一次查找，就可以排除掉一半的数据，大大缩小了查找的范围。因此它的查找复杂度为 O(logn)。

要使用二分查找，必须满足如下 2 点要求：

1，必须采用顺序存储结构来存放要查找的数据。

2，数据必须按关键字大小有序排列。

基本的二分查找算法用 Python 实现的代码如下：

```
def bin_search(data_list, val):
    low = 0
    high = len(data_list) - 1
    while low <= high:
        mid = (low + high) //2
        if data_list[mid] == val:
            return mid
        elif data_list[mid] > val:
            high = mid - 1
        else:
            low = mid + 1
    return # val 不存在，返回 None
if __name__=='__main__':
    list=[2, 9, 8, 15, 32, 41, 55, 98]
print   bin_search(list, 32)
```

现在我们基于二分查找算法来解决一个矩阵搜索问题：

编写一个高效的算法来判断 m x n 矩阵中，是否存在一个目标值。该矩阵具有如下特性：

每行中的整数从左到右按升序排列。

每行的第一个整数大于前一行的最后一个整数。

输入:

```
matrix = [
          [1, 3, 5, 7],
          [10, 11, 16, 20],
          [23, 30, 34, 50]
         ]
target = 3
```

输出: true

分析：实际上，把矩阵里每行存放的数据按序拼接在一起，就是一个有序的数组，因此就适合使用二分查找算法。那么如何拼接呢？我们可以首先计算出矩阵中元素总数，也就是 m*n。然后，在[0, m*n-1]这个区间进行二分查找，那么该区间中间元素序号为 index=（0+m*n-1)/2，那么这个元素在矩阵中的行号 row 和列号 col 分别为：

row = index// n ， col = index% n

将该元素 a[index//n][index%n]与 target 进行比较，如果相等，那么就找到了。如果 a[index//n][index%n]<target ，那么就在 (index, m*n-1] 区间继续查找；如果 a[index//n][index%n]>target，那么就在[0, index)区间继续查找。

于是，我们实现了如下基于折半查找的矩阵搜索算法：

```
def search_matrix(matrix, target):
    m = len(matrix)
```

```
        if m==0:
            return False
        n = len(matrix[0])
        left = 0
        right = m*n-1
        while left<=right:
            pivot_index = (left+right)/2
            pivot_val = matrix[pivot_index/n][pivot_index%n]
            if target == pivot_val:
                return True
            else:
                if target<pivot_val:
                    right = pivot_index-1
                else:
                    left = pivot_index+1
        return False


if __name__=='__main__':
        matrix=[[1,3,5,7],[10,11,16,20],[23,30,34,50]]
        print search_matrix(matrix, 99)
```

# 六，数论与 RSA 加解密

数论是纯粹数学的分支之一，主要研究整数的性质。而整数的基本元素是素数（也称质数），所以数论的本质是对素数性质的研究。数论被高斯誉为“数学中的皇冠”。因此，数学家都喜欢把数论中一些悬而未决的疑难问题，叫做“皇冠上的明珠”。

公元前 300 年，古希腊数学家欧几里德证明了有无穷多个素数，公元前 250 年古希腊数学家埃拉托塞尼(Eratosthenes，约公元前 276—约公元前 195 年)发明了一种寻找素数的筛法。

像偶数和奇数这种数，我们都可以用一个通用的公式来表示，比如偶数=2n，奇数=2n+1，其中 n 为正整数。而素数目前却没有这样的通项公式。寻找一个表示所有素数的素数通项公式，或者叫素数普遍公式，是古典数论最主要的问题之一。

“数论”其实在建立初期就叫“算术”，到 20 世纪初，才正式更名为“数论”。

## 6.1 数与数的关系

在数论中，数与数之间存在一些关系，比如两个数互质。现在我们首先给大家介绍一下这些数与数的关系，为我们进一步学习数论在计算机 RSA 加解密算法中的应用打下基础。

**公约数**：如果一个自然数 a 能被自然数 b 整除，则称 a 为 b 的倍数，b 为 a 的约数。几个自然数公有的约数，叫做这几个自然数的公约数。

比如 12 和 15 都能被 3 整除，那么 3 就是 12 和 15 的公约数。

**两个数互质**：互质，也叫互素，公约数只有 1 的两个整数，叫做互质整数。公约数只有 1 的两个自然数，叫做互质自然数。8 和 10，5 和 25 不互质，因为分别有公约数 2 和 5；7 和 10 的公约数只有 1，所以互质。两个数互质有如下一些特性：

- 任意两个素数构成互质关系，比如 13 和 17。
- 一个数是素数，另一个数只要不是前者的倍数，两者就构成互质关系，比如 3 和 10。
- 如果两个数之中，较大的那个数是素数，则两者构成互质关系，比如 97 和 57。
- 1 和任意一个自然数是都是互质关系，比如 1 和 99。
- p 是大于 1 的整数，则 p 和 p-1 构成互质关系，比如 57 和 56。
- p 是大于 1 的奇数，则 p 和 p-2 构成互质关系，比如 17 和 15。

**最大公约数**：公约数中最大的一个公约数，称为这几个自然数的最大公约数（Greatest Common Divisor，缩写 GCD）。在 2、4、6 三个数中，2 就是它们的最大公约数。对于 12 和 30 的公约数有：1、2、3、6，其中 6 就是 12 和 30 的最大公约数。

**程序辗转相除法求最大公约数**

求两个数的最大公约数，可以使用辗转相除法。辗转相除法又叫欧几里德算法，具体的算法为：大数除以小数取余，余数不为零时除数做被除数，余数做除数继续相除，直到余数为零。这时除数就是最大公约数。

比如：求 26 和 39 的最大公约数。

令 a=39， b=26，a>b，那么 a 为大数做被除数，b 为小数做除数：

39%26=13，39/26=1

因为余数不为 0，所以，余数 13 做除数，除数 26 做被除数继续相除：

26%13=0，26/13=2

这个时候余数为 0，所以这个时候的除数 13，就是 26 和 39 的最大公约数。

下面我们给出用 C 语言程序实现的辗转相除求两个数最大公约数的算法：

```
int gcd(int a, int b){
    int tmp =0;
    if(a<b){//如果 a 小于 b，交换二者的值，使得 a 是最大值
        tmp=a;
        a=b;
        b=tmp;
    }
    tmp= a%b; //a 做被除数，b 做除数
    while(tmp){//余数不为零，除数做被除数，余数做除数，继续除
        a=b;
        b=tmp;
        tmp=a%b;
    }
    return b; //余数为零，除数就是最大公约数
}
```

**最小公倍数**：如果一个数既是 a 的倍数又是 b 的倍数，那么我们就把这个数叫做 a 和 b 的公倍数，如果这个数在 a, b 的所有公倍数里为最小，那这个数就是最小公倍数。

计算最小公倍数时，通常会借助最大公约数来辅助计算，即：最小公倍数等于两数之积除以其最大公约数。

## 6.2 素数无穷多证明

素数的个数是有限的还是无穷的，这曾经是数学领域一个著名的问题。希腊数学家欧几里得(约公元前 330 年-公元前 275 年，如图 6-1)早在公元前 300 年左右用反证法给出了最早的证明。现在我们来看看他是如何证明的。

图 6-1 欧几里得

命题：素数个数是无穷的。

证明：

假设素数是有限的，即存在最大的素数P，那么构造这样一个数A：

$$A=2\times3\times5\times7\times\cdots\cdots\times P+1$$

即A是从2到P所有素数的乘积再加上1。

那么，利用任何一个素数去除A，都会余1，即任何素数都无法整除A。根据素数的定义，显然A也是一个素数。并且，A比P大的多，这与假设“P是最大的素数”矛盾。

故假设不成立，素数是无穷的。

## 6.3 筛法求素数

虽然已经证明素数（质数）是无穷多的，然而人们对素数的分布却一无所知，至今尚未找到一个通项公式来表示所有素数。

那么我们如何寻找素数呢？这里给大家介绍一种快速的方法，即筛法，它是一种由希腊数学家埃拉托塞尼所提出的一种简单检定素数的算法。

筛法的主要思想就是：想要得到自然数n以内的全部素数，只需要把不大于$\sqrt{n}$的所有素数的倍数筛除，剩下的就是素数。

比如，我们要找出25以内的所有素数，那么首先列出从2开始的所有数：

2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

第1个数2是素数，那么把所有2的整数倍也就是偶数筛掉，那么数列剩下：

2 3 5 7 9 11 13 15 17 19 21 23 25

第2个数3是素数，那么把所有3的整数倍筛掉，那么数列剩下：

2 3 5 7 11 13 17 19 23 25

第3个数5是素数，那么把所有5的倍数筛掉，那么数列剩下：

2 3 5 7 11 13 17 19 23

因为$5^2=25$，所以筛法停止。那么剩下数列中的所有数，即是25以内的所有素数。

我们把素数比喻成数字中的黄金，那么筛法就好比沙中淘金一样，用筛子筛掉泥沙，留下像黄金一样的素数。

下面是一个C语言版本实现的筛法算法程序：

```
#define MAXN 40001
int sift_prime(void){
        int i=0, j=0;
        int total=0;
        int prime[MAXN];

        for (i = 2 ; i < MAXN; i++ )
            prime[i] = 1 ;//首先将数组元素的值设为1，即假设每个数都是素数

        for (i = 2 ; i*i < MAXN; i++ ){
            if (prime[i] == 1 ){
                    for (j = 2 ; i * j < MAXN; j++ )
                        prime[i * j] = 0 ; //把所有素数的整数倍设置为0，筛掉
            }
        }
        for (i = 0; i < MAXN; i++ ){
            if (prime[i] == 1 ) {//数组中元素值依然为1的数就是素数
```

```
                printf("%d ",i);
                total++;
            }
        }

        return total;
    }
```

## 6.4 数论猜想

数论里面有很多著名的猜想，这些猜想影响深远，很多理论是在基于这些猜想为正确理论的前提下发展起来，所以一旦猜想有问题，那么这些理论也就同时会被推翻。无数数学家一直不遗余力的想要努力证明这些猜想。有的猜想经过几百年才最终被证明为正确，而有的至今没有任何进展。

### 6.4.1 哥德巴赫猜想

德国数学家哥德巴赫(1690-1764)在 1742 年给欧拉的信中提出了著名的哥德巴赫猜想，这个猜想是这样描述的：

任一大于 2 的整数都可写成三个素数之和。

因现今数学界已经不使用“1 也是素数”这个约定，所以对于 3=1+1+1, 4=2+1+1 就不成立了。因此把原来的猜想修改成了：

对于任一大于 5 的整数都可写成三个素数之和。

对于 n>5：当 n 为偶数，n=2+(n-2)，2 是素数，n-2 是偶数，因此只需要把偶数 n-2 分解为两个素数的和；当 n 为奇数，n=3+(n-3)，3 是素数，n-3 是偶数，因此只需要把偶数 n-3 分解为两个素数的和。于是欧拉在回信中也提出另一等价版本，即：

任一大于 2 的偶数都可写成两个素数之和。

对于这个猜想，我们可以进行大量的验证，这个结论都是成立的。比如 4=2+2；6=3+3；8=3+5，等等。但是只有验证是不能确保命题成立的。因此要么找到反例推翻之，要么通过严谨的推导证明之。

假设 N 是偶数，虽然现在还不能证明 N 是两个素数之和，但可以证明它能够写成两个素因子个数不多的正整数（殆素数）的和，即 N=A+B，其中 A 和 B 的素因子个数都不太多，譬如说素因子个数不超过 10。用“a+b”来表示如下命题：

每个大偶数 N 都可表为 A+B，其中 A 和 B 的素因子个数分别不超过 a 和 b。

那么，哥德巴赫猜想就可以写成"1+1"，这里的 1+1 实际上是指 1 个素数加上另一个素数。历史上对于“a + b”问题的证明进展如下：

1920 年，挪威的布朗证明了“9 + 9”。

1924 年，德国的拉特马赫证明了“7 + 7”。

1932 年，英国的埃斯特曼证明了“6 + 6”。

1937 年，意大利的蕾西先后证明了“5 + 7”，“4 + 9”，“3 + 15”和“2 + 366”。

1938 年，苏联的布赫夕太勃证明了“5 + 5”。

1940 年，苏联的布赫夕太勃证明了“4 + 4”。

1956 年，中国的王元证明了“3 + 4”。稍后证明了 “3 + 3”和“2 + 3”。

1948 年，匈牙利的瑞尼证明了“1+c”，其中 c 是一很大的自然数。

1962 年，中国的潘承洞和苏联的巴尔巴恩证明了“1 + 5”，中国的王元证明了“1 + 4”。

1965 年，苏联的布赫夕太勃和小维诺格拉多夫，及意大利的朋比利证明了“1 + 3 ”。

目前取得的最大进展就是中国数学家陈景润。在 1966 年他成功证明了：每个大于 2 的偶数可以写成 1 个素数加上另外 2 个素数乘积之和。

## 6.4.2 孪生素数猜想

孪生素数就是差为 2 的素数对，例如 11 和 13。是否存在无穷多的孪生素数? 比起孪生素数猜想，有数学家提出了更一般的猜想：对所有自然数 k，存在无穷多个素数对 (p， p + 2k)。k = 1 的情况就是孪生素数猜想。素数对 (p, p + 2)称为孪生素数。数学家们相信这个猜想是成立的。

2013 年 5 月，华人数学家张益唐的论文《素数间的有界距离》在《数学年刊》上发表，破解了困扰数学界长达一个半世纪的难题，证明了孪生素猜想的弱化形势，即发现存在无穷多差小于 7000 万的素数对。这是第一次有人证明存在无穷多组间距小于定值的素数对。

除了孪生素数猜想之外，还有斐波那契数列素数猜想，即：斐波那契数列内是否存在无穷多的素数?以及梅森素数猜想：是否存在无穷多的梅森素数?

梅森素数(MersennePrimes)，命名源自 17 世纪法国数学家、法兰西科学院奠基人马林•梅森。梅森素数指形如 $2^p$-1 的正整数，其中指数 p 是素数，常记为 $M_p$。若 $M_p$ 是素数，则称为梅森素数。p=2，3，5，7 时，$M_p$ 分别为 3，7，31，127 都是素数，但 $M_{11}$=2047=23×89 不是素数，是否有无穷多个梅森素数是数论中未解决的难题之一。这种素数历来是数论研究的一项重要内容，也是当今科学探索的热点和难点之一，由于梅森素数珍奇而迷人，它被人们誉为“数论中的钻石”。

2016 年 1 月累计发现 49 个梅森素数，最大的是 $p=2^{74207281}$-1（被称为 $M_{74207281}$），此时 Mp 是一个 22338618 位数。在 2017 年 12 月底由全球合作项目“互联网梅森素数搜索”（ Great Internet Mersenne Prime Search，GIMPS）发现了目前最大的素数：$M_{77232917}$，即：$2^{77232917}$-1，是第 50 位梅森素数。它是由一位现年 51 岁在田纳西州的电气工程师 Jonathan Pace 在自己的电脑上发现的。

1996 年起，GIMPS 项目已经找到了 16 个梅森素数。GIMPS 称发现者 Pace 会得到 3000 美元的奖励。找到 1 亿位的素数会有 15 万美元的奖励。发现第一个 10 亿位的素数会有 25 万美元的奖励。发现更多的素数会帮助数学家更深入地理解，素数会在什么情况下出现。但目前他们还没有发现这个理论，也没有特定的公式，所以找素数的任务只能交由 GIMPS，通过计算机简单粗暴的力量来完成。

## 6.4.3 费马猜想

1637 年左右，费马(1601-1666，法国律师和业余数学家，如图 6-2)在阅读丢番图《算术》一书时，曾在第 11 卷第 8 命题旁写道：“将一个立方数分成两个立方数之和，或一个四次幂分成两个四次幂之和，或者一般地将一个高于二次的幂分成两个同次幂之和，这是不可能的。关于此，我确信已发现了一种美妙的证法，可惜这里空白的地方太小，写不下。”

图 6-2 费马

这就是著名的费马猜想：

方程 $x^n+y^n=z^n$，当 n 是大于 2 的整数时，没有正整数解。

这个猜想传世 358 年才最终得以解决。费马猜想也被称为费马大定理。

费马猜想提出来后，1908 年哥廷根皇家科学协会公布沃尔夫斯凯尔奖：凡在 2007 年 9 月 13 日前解决费马大定理者将获得 100000 马克奖励。沃尔夫斯凯尔是德国实业家，年轻时曾为情所困决意在午夜自杀，但在临自杀前读到库默尔论述柯西和拉梅证明费马定理的错误让他情不自禁地计算到天明，设定自杀时间过了，数学让他重生并后来成为大富豪。1908 年这位富豪死时，遗嘱将其一半遗产捐赠设立沃尔夫斯凯尔奖，奖励解决费马猜想的数学家，以谢其救命之恩。从此世界每年都会有成千上万人宣称证明了费马猜想，但全部都是错的，一些数学权威机构，不得不预写证明否定书。

历史上拉梅、柯西、勒贝格、里贝特等都给出了错误证明。1993 年 6 月英国剑桥安德鲁.怀尔斯宣称证明了该定理，但后来在审查过程中被发现此证明有严重缺陷。1994 年 10 月，怀尔斯给予了补正，至此费马大定理得证。

1996 年 3 月怀尔斯获得沃尔夫奖和 5 万美金。1997 年 6 月 27 日怀尔斯获得沃尔夫斯凯尔奖。历史竟如此巧合，就在哥廷根皇家科学协会规定期只剩下 10 年的时候沃尔夫斯凯尔当年遗愿终于实现。1998 年第 23 届国际数学家大会在柏林举行，会议颁给怀尔斯菲尔兹特别奖（已过 40 岁）。2005 年怀尔斯又荣获有“东方诺贝尔奖”之称的邵逸夫数学科学奖，奖金 100 万美金，也被授予 2016 年阿贝尔奖。

### 6.4.5 黎曼猜想

既然现在无法获得素数的通用公式，那么退而求其次，可不可以直接判断在小于某个已知数的范围内的素数个数呢？比如，小于 40000 的正整数范围以内，有多少个素数呢？

18 世纪伟大的德国数学家黎曼(1826-1866，如图 6-3)发表了一篇 8 页的论文，即《论小于已知数的素数的个数》。在这篇文章里，黎曼阐述了素数的精确分布规律，并给出了计算素数个数的公式。但是在这个公式里，有一步是不确定的，这就是这个公式里用到了一个精

心构造的函数，即 Zeta 函数（ζ(s)，s 是自变量），素数的分布频率和这个函数的特性紧密相关。

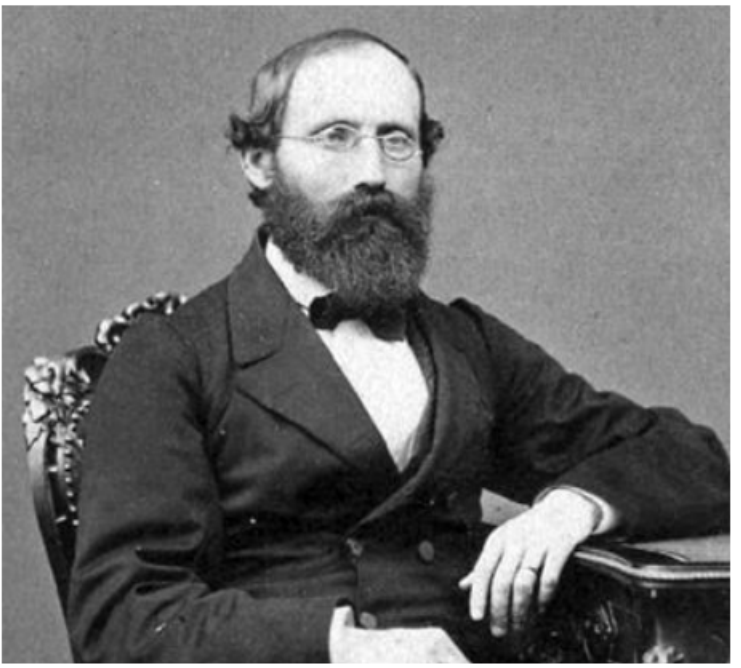


图 6-3 黎曼

Zeta 函数ζ(s)具体的定义就是对自然数 n 的负 s 次方($n^{-s}$)，n 从 1 到无穷求和。因此，ζ(s)函数就是一个如下所示的无穷级数求和：

$$\zeta(s) = \sum_{n=1}^{\infty} n^{-s} = \frac{1}{1^s} + \frac{1}{2^s} + \frac{1}{3^s} + \cdots$$

研究ζ(s)函数的重要性质之一就是对其零点的研究。零点就是使得函数的取值为零的数值集合。这些数值的取值是在复数范围内，即零点有可能为复数 a+bi（由实部 a 和虚部 b 组成）。

利用积分相关的知识，ζ(s)函数可以变形为如下表达式：

$$\zeta(s) = 2^s \pi^{s-1} \sin\frac{\pi s}{2} \Gamma(1-s)\zeta(1-s)$$

通过上式，可以看出ζ(s)函数在 s=-2n (n 为正整数），也就是在 s 为负偶数时取值为零，因为此时 $\sin\frac{\pi s}{2}$=sin(-π)=0。复平面上的这种使ζ(s) 函数取值为零的点被称为ζ(s) 函数的零点。因此 s=-2n 是ζ(s) 函数的零点，这些零点分布有序、性质简单，被称为ζ(s)函数的平凡零点 (trivial zero)。除了平凡零点，ζ(s)函数还有许多其它零点，它们的性质比平凡零点复杂，被称为非平凡零点 (non-trivial zeros)。针对非平凡零点，黎曼提出了如下三个命题：

命题 1：黎曼指出了非平凡零点的个数，且十分肯定其分布在复平面上实部大于 0 但是小于 1 的带状区域上。

命题 2：黎曼提出所有非平凡零点都几乎全部位于复平面上实部等于 1/2 的直线上。

命题 3：黎曼十分谨慎的写到：很可能所有非平凡零点都全部位于复平面上实部等于 1/2

的直线上。这条线，从此被称为临界线。

这 3 个命题将零点的实部的范围一步步的缩小和精确化。命题 3 直接指定实部等于 1/2，命题 3 就是所谓的黎曼猜想。这点已经对于开始的 1,500,000,000（15 亿）个解验证通过。说明目前它对于每一个有意义的解都成立。因此只要证明黎曼猜想的正确，那么就可以利用黎曼提出的素数分布公式来精确寻找素数了。

黎曼猜想是 21 世纪美国克雷数学所千禧年问题和 20 世纪希尔伯特 23 个数学问题之一。虽然黎曼猜想不及费马猜想和哥德巴赫猜想出名，但它在数学上的重要性要远远超过后两者，已有超过一千条数学命题以黎曼猜想（或其推广形式）的成立为前提。

据说还有人问过希尔伯特一个问题，说：“如果你沉睡了几百年，然后醒过来，你想干什么？”希尔伯特说，“我想问问有人把黎曼猜想证出来了吗？我太想知道了”。

在正式学习 RSA 加解密算法之前，我们先来学习几个重要的定理和算法，包括：扩展欧几里得算法和欧拉函数与定理，二项式定理等。

## 6.5 扩展欧几里得算法

贝祖定理（也叫裴蜀定理，一个关于最大公约数的定理，得名于法国数学家艾蒂安·贝祖）告诉我们，如果 a、b 是整数，那么一定存在整数解 x、y 使得方程 ax+by=gcd(a, b)成立。其中 gcd(a, b)表示整数 a 和 b 的最大公约数。

特别地，如果 ax+by=1 有解，那么 gcd(a, b)=1，即 a 和 b 互质。

现在的问题是，在利用贝祖定理知道方程有解的情况下，这些解是多少呢？扩展欧几里得算法（简称 exgcd）就是用来求出这种形式方程的解的算法。

我们可以使用欧几里得（辗转相除）法来求 a 和 b 的最大公约数 gcd(a, b)，用 C 语言实现的欧几里得递归算法如下所示：

```
int gcd(int a, int b){//要求 a>b
    return b==0?a:gcd(b, a%b);
}
```

在使用欧几里得算法求最大公约数的时候，当 b=0，a=gcd(a, b) 时（b=0 是在求最大公约数递归算法中的递归出口）可以容易验证 ax+by=gcd(a, b)的解为：

$$x=1, y=0$$

因为将上面的解代入方程 ax+by=gcd(a, b)有：a*1+b*0=gcd(a, b)方程成立。但这时的 a 和 b 已经不是最开始的那个 a 和 b 了，所以如果想要求出原始的解 x 和 y，就要从欧几里得算法中 b=0 开始重新回到最初始情况时候 x 和 y 的解。

在递归的算法中，如果知道了本层和上一层的关系，就可以利用这个关系，从最里层往外，一层一层计算，得到最开始的解。

假设当前在求的是 a 和 b 的最大公约数，而当前已经求出了下一个状态：b 和 a%b 的最大公约数 gcd，并且求出了一组 $x_1$ 和 $y_1$ 使得：

$$b*x_1+(a\%b)*y_1=gcd$$

又因为：a%b=a-(a/b)*b；带入上式：

$$b*x_1 + (a-(a/b)*b)*y_1= b*x_1 + a*y_1 – (a/b)*b*y_1= a*y_1 + b*(x_1 – a/b*y_1) = gcd$$

a 和 b 的最大公约数对应的解为：

$$x = y_1 , y = x_1 - a/b*y_1$$

这就是相邻两层的解的关系，从 $x_1$ 和 $y_1$ 推出上层 x 和 y 的解。那么就可以从最里层的 x=1，y=0 利用这个关系逐层往外推导，得出初始的解。

我们可以通过 C++语言实现扩展欧几里得算法如下：

```
int exgcd(int a, int b, int &x, int &y) {//这里须要满足 a>b
    if(b==0){ //递归出口，b=0，a=gcd(a, b)，此时 x=1 和 y=0 是对应方程的解
        x=1;
        y=0;
        return a;
    }
    int g=exgcd(b, a%b, x, y);//获得下一层的 x 和 y 的解
    int tmp=y;      //把 x 和 y 通过公式，转换成上一层的解
    y=x-(a/b)*y;
    x=tmp;
    return g;       //得到 a b 的最大公因数
}
```

实际上，扩展欧几里得算法其实就是利用欧几里得算法不断递归直到遇到在 b=0 时 x=1, y=0 这个解，然后再反向递归求出第一层的 x 和 y，就是所求的解。

在 RSA 算法生成私钥的过程中涉及到了扩展欧几里得算法，用来求解模的逆元。请参考本书 6.8 节。

## 6.6 欧拉函数与定理

现在我们来考虑这样一个问题：

任意给定正整数 n，请问在小于等于 n 的正整数之中，有多少个与 n 构成互质关系？（比如，在 1 到 8 之中，有多少个数与 8 构成互质关系？）

计算这个值的方法就叫做欧拉函数，以 $\phi(n)$ 表示。在 1 到 8 之中，与 8 形成互质关系的是 1、3、5、7，所以 $\phi(n)=4$。

$\phi(n)$ 的计算方法并不复杂，但是为了得到最后的计算公式，需要通过一系列的定理来推导。

**定理一**：如果 n=1，则 $\phi(1)=1$ 。因为 1 与任何数（包括自身）都构成互质关系。

**定理二**：如果 n 是素数，则 $\phi(n)=n-1$ 。因为素数与小于它的每一个数，都构成互质关系。比如 5 与 1、2、3、4 都构成互质关系。

**定理三**：

如果 n 是素数的 k 次方，即 $n=p^k$ (p 为素数，k 为大于等于 1 的整数)，则：

$$\phi(p^k)=p^k-p^{k-1}。$$

比如 $\phi(8)=\phi(2^3)=2^3-2^2=8-4=4$。这是因为只有当一个数不包含素数 p，才可能与 n 互质。而包含素数 p 的数一共有 $p^{k-1}$ 个，即 $1\times p$、$2\times p$、$3\times p$、...、$p^{k-1}\times p$，把它们去除，剩下的就是与 n 互质的数。

可以看出，上面的定理二是 k=1 时的特例。

**定理四**：如果 n 可以分解成两个互质的整数之积，$n=p_1\times p_2$，则：

$$\phi(n)=\phi(p_1p_2)=\phi(p_1)\phi(p_2)$$

即积的欧拉函数等于各个因子的欧拉函数之积。比如，$\phi(56)=\phi(8\times7)=\phi(8)\times\phi(7)=4\times6=24$。

这一条的证明要用到"中国剩余定理"：如果 a 与 $p_1$ 互质($a<p_1$)，b 与 $p_2$ 互质($b<p_2$)，c 与 $p_1p_2$ 互质($c<p_1p_2$)，则 c 与数对 (a, b) 是一一对应关系。由于 a 的值有 $\phi(p_1)$种可能，b 的值有 $\phi(p_2)$种可能，则数对 (a, b) 有 $\phi(p_1)\phi(p_2)$种可能，而 c 的值有 $\phi(p_1p_2)$种可能，所以 $\phi(p_1p_2)$ 就等于 $\phi(p_1)\phi(p_2)$。

**定理五：**

因为任意一个大于 1 的正整数，都可以写成一系列素数的积。

$$n = p_1^{k_1} * p_2^{k_2} * p_3^{k_3} * \ldots * p_r^{k_r}$$

那么有：

$$\varphi(n)=n * \left(1-\frac{1}{p_1}\right) * \left(1-\frac{1}{p_2}\right) * \ldots\ldots * \left(1-\frac{1}{p_r}\right)$$

这就是欧拉函数的通用计算公式。现在我们来证明它。因为素数之间是互质的，根据定理四有：

$$\varphi(n)=\varphi(p_1^{k_1})*\varphi(p_2^{k_2})*\varphi(p_3^{k_3})*\ldots*\varphi(p_r^{k_r}) \quad ①$$

由定理三有：

$$\varphi(p^k)=p^k-p^{k-1}=p^k\left(1-\frac{1}{p}\right) \qquad ②$$

将②代入①有：

$$\varphi(n)=p_1^{k_1} * p_2^{k_2} * p_3^{k_3} * \ldots * p_r^{k_r} * \left(1-\frac{1}{p_1}\right) * \left(1-\frac{1}{p_2}\right) * \ldots\ldots * \left(1-\frac{1}{p_r}\right) \quad ③$$

又因为已知：

$$n = p_1^{k_1} * p_2^{k_2} * p_3^{k_3} * \ldots * p_r^{k_r}$$

代入③有：

$$\varphi(n)=n * \left(1-\frac{1}{p_1}\right) * \left(1-\frac{1}{p_2}\right) * \ldots\ldots * \left(1-\frac{1}{p_r}\right)$$

因此，如果要求 φ(n)，那么我们就直接将 n 唯一分解出每一个 n 的素数，然后用通项公式就行了。

**欧拉定理：**如果两个正整数 a 和 n 互质，即(a, n)=1，那么有：

$$a^{\varphi(n)} \equiv 1(mod n)$$

其中 φ(n)为 n 的欧拉函数，≡为同余符号，如果 a≡b (mod n)，读作“a 同余于 b 模 n”，或“a 与 b 对模 n 同余”，意思为 a 和 b 除以 n 余数相同，mod 为取余运算，在计算机里一般对应%运算符。如果 a≡b (mod n)，那么有 (a-b) mod n=0，或者 a-b=kn，其中 k 为整数。

比如 9≡4 (mod 5)，那么有 9 mod 5 等于 4 mod 5 等于 4，或者(9-4) mod 5=0。

欧拉定理表明，如果两个正整数 a 和 n 互质，那么 a 的 φ(n)次方被 n 除的余数为 1，也可以理解为 a 的 φ(n)次方减去 1，可以被 n 整除。

比如，3 和 7 互质，而 7 的欧拉函数 φ(7)等于 6，所以 3 的 6 次方（729）减去 1，可以被 7 整除（728/7=104）。

欧拉定理可以简化某些运算。比如，7 和 10 互质，根据欧拉定理，那么有：

$$7^{\phi(10)} \equiv 1\,(mod\,10)$$

已知 φ(10) 等于 4，那么 7 的 4 倍数次方的个位数肯定是 1，即：

$$7^{4k} \equiv 1 \, (mod \, 10)$$

欧拉定理也是RSA加解密算法的核心理论，这个我们在本章后面会详细介绍。

欧拉定理有一个特殊情况，假设正整数a与素数p互质，因为素数p的φ(p)等于p-1，则欧拉定理可以写成：

$$a^{p-1} \equiv 1 \, (mod \, p)$$

这就是著名的费马小定理，它实际上是欧拉定理的特例。欧拉定理和威尔逊定理、孙子定理（中国剩余定理）、费马小定理并称数论四大定理。

如果两个正整数a和n互质，那么一定可以找到整数b，使得 ab-1 被n整除，或者说ab被n除的余数是1。这时，b就叫做a的**"模反元素"**。

比如，3和11互质，那么3的模反元素就是4，因为 (3×4)-1 可以被11整除。显然，模反元素不止一个， 4加减11的整数倍所得的集合 {...,-18,-7,4,15,26,...}中的元素都是3的模反元素，即如果b是a的模反元素，则 b+kn 都是a的模反元素。

欧拉定理可以用来证明模反元素必然存在。可以看到，$a^{φ(n)-1}$ 就是a的模反元素。这是因为：

$$(a*a^{φ(n)-1}-1) \bmod n=(a^{φ(n)}-1) \bmod n=0$$

## 6.7 二项式定理

二项式是指包含两项的多项式，比如a+b, a-b, x+y等。二项式定理是有关将两数之和的任意整数次幂（可推广为实数次幂）展开成和的形式的定理。

以a+b为例，首先我们来看一些最简单的情形，比如：

$(a+b)^0=1$

$(a+b)^1=a+b$

$(a+b)^2=a^2+2ab+b^2$

$(a+b)^3=a^3+3a^2b+3ab^2+b^3$

$(a+b)^4=a^4+4a^3b+6a^2b^2+4ab^3+b^4$

....

$(a+b)^n$=...//这个就是两数之和的任意整数次幂的一般形式，即二项式定理

取展开之后各项的系数刚好构成一个如图6-4所示的杨辉三角形：

```
          1
        1   1
      1   2   1
    1   3   3   1
  1   4   6   4   1
```

图6-4 杨辉三角形

公元1050年，中国北宋数学家贾宪完成了数学著作《释锁算书》（已失传），其中给出了如图6-5所示“开方作法本原图”，满足了三次以上开方的需要。

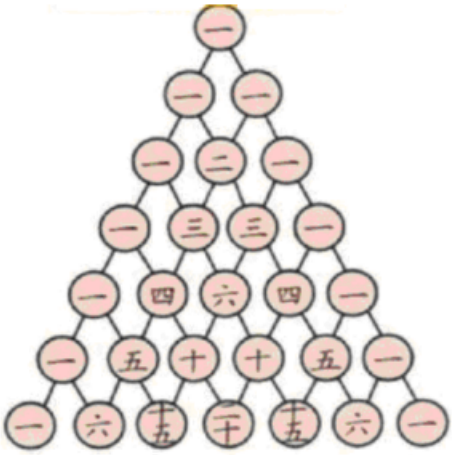



图 6-5 开方作法本原图

图 6-5 即为直到六次幂的二项式系数表。1261 年，书中的部分内容“贾宪三角”和“增乘开方法”等内容被南宋著名数学家杨辉抄录入著名的《杨辉算法》，因此数学界又称之为“杨辉三角”。

可惜中国古代的数学着重于“问题的独立应用”，没有形成“公理系统”的数学思维，所以虽然有了二项式系数的雏形，却没有进一步归纳出“二项式系数”的一般公式和定理。

1654 年，法国的帕斯卡最早建立了一般正整数次幂的二项式定理，因此算术三角形在西方至今仍以他的名字命名。1665 年，英国的牛顿将二项式定理推广到有理指数的情形。18 世纪，瑞士的欧拉和意大利的卡斯蒂隆分别采用待定系数法和“先异后同”的方法证明了实指数情形的二项式定理。

在这里，我们直接给出二项式定理的两种形式，而不给予对应的证明。首先是整数次幂的二项式定理。

设 n 为正整数，那么 n 次幂二项式定理可表述为下列公式：

$$(a+b)^n = C_n^0 a^n + C_n^1 a^{n-1} b + \ldots + C_n^k a^{n-k} b^k + \ldots + C_n^m b^n = \sum_{k=0}^{n} C_n^k a^{n-k} b^k$$

其中$C_n^k = \frac{n!}{k!(n-k)!}$ 是二项式系数，$C_n^k$ 有时候也记为$\binom{n}{k}$，它们共同组成了杨辉三角形。

比如$(1+x)^n$ 为例子，套用上面的二项式展开公式有：

$$(1+x)^n = 1 + nx + \frac{n(n-1)}{2!}x^2 + \ldots + \frac{n(n-1)\ldots(n-k+1)}{k!}x^k + .. + x^n$$

二项式定理可以推广到对任意实数次幂的展开，即牛顿的广义二项式定理：

$$(x+y)^\alpha = \sum_{k=0}^{\infty} \binom{\alpha}{k} x^{\alpha-k} y^k$$

其中$\binom{\alpha}{k} = \frac{\alpha(\alpha-1)\ldots(\alpha-k+1)}{k!} = \frac{(\alpha)_k}{k!}$ 是对应的二项式系数。

二项式定理在“组合理论”、“开高次方”、“高阶等差数列求和”和“差分法”中有着常重要的作用。牛顿以二项式定理作为基石发明了微积分。

# 6.8 RSA 加解密

现在我们分两章来学习密码学相关的理论以及对应的数学理论基础。密码学和加解密理论的实践和应用是信息安全领域的重要分支和基础设施。

互联网重要数据的传输需要加密。比如网络 TCP/IP 协议本身是明文的，不安全。如果直接使用 TCP/IP 协议来收数据，很容易被人嗅探而发生数据泄密。一些重要的数据，比如登录时候用户名和密码，比如收发的电子邮件，QQ、微信聊天内容，都需要经过加密传输。网络加密协议包括：https/tls/ssl 等。比特币，区块链都是基于加解密理论基础。

存储重要数据也需要加密。比如 Windows 文件系统 NTFS 中的 EFS 加密工具，WINRAR 的文件加密等。

此外，为了防止文件和数据被篡改，需要加密算法对数据进行数字签名，以保证数据的不可篡改性。

## 6.8.1 加密算法概述

加密算法可以分为很多种，主要包括对称加解密，非对称加解密，以及散列算法等。这些算法各有特点和应用场景，但也不能互相取代，有时候还需要相互配合使用。现在我们先给大家介绍一下这些算法的基本原理。

**1.对称加解密**

对称加解密属于单钥密码系统的加密方法，同一个密钥可以同时用作信息的加密和解密，这种加密方法称为对称加密，也称为单密钥加密。密钥是控制加密及解密过程的指令。

对称加密中，数据发送方将明文（原始数据）和加密密钥一起经过特殊加密算法处理后，使其变成复杂的加密密文发送出去。接收方收到密文后，若想解读原文，则需要使用加密密钥及相同算法的逆算法对密文进行解密，才能使其恢复成可读明文。在对称加密算法中，使用的密钥只有一个，收发信息双方都使用这个密钥对数据进行加密和解密。因此密钥的安全性就非常重要，因为只要密钥一旦泄漏，那么加密信息也就不再安全了。

对称加密算法的优点是算法公开、计算量小、加密速度快、加密效率高。缺点是密钥管理复杂，有泄漏风险。

常见的对称加密算法有：DES、3DES、TripleDes、Blowfish、IDEA、Rabbit、RC4 、RC5、RC6、AES。

**2. 非对称加解密**

非对称加密算法需要两个不同密钥来进行加密和解密，这两个秘钥分别是公开密钥（public key，简称公钥）和私有密钥（private key，简称私钥）。

如果用公钥对数据进行加密，只有对应的私钥才能解密；如果私钥对数据进行加密，那么只有对应的公钥才能解密。因为加密和解密使用的是两个不同的密钥，所以这种算法叫作非对称加密算法。现在我们来介绍一下非对称加解密的过程。

加入 A，B 两人用非对称加密通信：

1、A 生成一对密钥（公钥和私钥）并将公钥向其他方公开，私钥独自严格保密。

2、得到该公钥的 B 使用该密钥对机密信息进行加密后再发送给 A 方。

3、A 方再用自己独自保存的私钥对加密后的信息进行解密。

在传输过程中，即使攻击者截获了传输的密文，并得到了 A 的公钥，也无法破解密文，因为只有 A 的私钥才能解密密文。

同样，如果 A 要回复加密信息给 B，那么需要 B 先公布 B 的公钥给 A 用于加密，B 自

己保存 B 的私钥用于解密。

非对称加解密的优点在于密钥管理的安全性更好；缺点是加密和解密花费时间长、速度慢，只适合对少量数据进行加密。所以一般是对称加密与非对称加密相结合使用，即数据发送方用对称加密的密钥加密数据，再用非对称加密方法加密对称加密中的密钥；数据接收方在接收到密文之后，先用非对称加密算法解密对称加密的密钥，再用该密钥解密对称加密的密文。

常见的非对称加解密算法包括：RSA、Elgamal、Rabin、DH(Diffie-Hellman)、ECC。

**3. 散列算法**

散列（Hash）函数对不同长度的输入消息，产生固定长度（比如MD5 是 16 字节（128 位））的输出。这个固定长度的输出称为原输入消息的“散列”或“消息摘要”(Message digest)。一个安全的哈希函数 H 必须具有以下属性：

1）H 能够应用到大小不一的数据上。小到 1 字节的字符串，大到上 G 的文件都可以。

2）H 能够生成大小固定的输出。

3）对于任意给定的 x，H（x）的计算相对简单。

4）对于任意给定的代码 h，要发现满足 H（x）=h 的 x 在计算上是不可行的。

5) 对于任意给定的块 x，要发现满足 H（y）=H（x）而 y=x 在计算上是不可行的。

6）要发现满足 H（X）=H（y）的（X，y）对在计算上是不可行的。

散列算法的应用包括：错误矫正，文件校验（奇偶校验，CRC，无防篡改功能），数字签名等。

常见的散列算法包括：MD4、MD5、SHA-1, SHA-224, SHA-256, SHA-384 和 SHA-512 等。MD4 1990 年 输出 128 位，MD5 1991 年 输出 128 位 ，已经不安全。SHA-1 1995 年输出 160 位，已经不安全。SHA-2 包括 SHA-224、SHA-256、SHA-384，和 SHA-512，分别输出 224、256、384、512 位，目前算法安全。

一些在线网站就可以提供各种加解密算法的加解密服务，比如：https://tool.oschina.net/encrypt。

对于字符串”hello world!”，经过 MD5 散列之后：

fc3ff98e8c6a0d3087d515c0473f8677

通过 SHA-256 散列之后：

7509e5bda0c762d2bac7f90d758b5b2263fa01ccbc542ab5e3df163be08e6ca9

输入值只要改变一点点，输出就会完全改变。比如把“hello world!”改为“Hello world!”，只是将第一个字母由小写改为大写，那么：

经过 MD5 散列之后：

86fb269d190d2c85f6e0468ceca42a20

通过 SHA-256 散列之后：

c0535e4be2b79ffd93291305436bf889314e4a3faec05ecffcbb7df31ad9e51a

和修改之前的对比，你会发现 MD5 和 SHA-256 的运算结果完全变了，差之毫厘谬以千里。

**5.数字签名**

为了防止数据被人篡改，我们需要计算数据的数字签名，并把数字签名和数据一起发送。数字签名的计算过程是这样的：

首先计算数据的 HASH 值（比如 MD5 值，SHA256 值等）。

然后，用非对称加密的私钥将 HASH 值加密，然后将密文和数据一起发送。

接收方在拿到数据后，通过公钥将加密数据解密得到原始数据的 HASH 值，并计算当前数据（有可能被篡改）的 HASH 值进行比对。如果比对一致，则数据未被修改；不一致，

则数据已被篡改。

下面以一个具体例子来说明。比如有A，B两人，A要把带有数据的小纸条发给B，为了防止小纸条上的信息被人篡改，那么：

第一步：A会先用SHA256函数对自己的小纸条进行处理，得到一个固定长度的字符串（HASH值），这个字符串就等价于这张小纸条。

第二步：A使用只有自己知道的私钥，对上面固定长度的字符串进行再加密，生成一份名叫数字签名的字符串，这份数字签名能够充分证明是基于这张小纸条的。

第三步：A将带明文的小纸条、刚刚加密成功的数字签名，以及自己那把可以公布于众的公钥打包一起发给B。

第四步：当B收到这三样数据之后，首先会将明文的小纸条进行SHA256函数处理，得到一个字符串，我们将其命名为“字符串2”。然后，B使用A公布的公钥，对发过来的数字签名进行解密，得到另外一个“字符串1”。通过比对“字符串1”和“字符串2”，如果它们一致，便可充分证明：B接受到的小纸条就是A发出来的小纸条，这张小纸条在中途没有被其他人所篡改；这张小纸条确实是由A所编辑。否则，那么小纸条上的数据已经被篡改了，数据不再可信。

## 6.8.2 RSA非对称加解密算法

RSA算法是一种非对称加密算法，所谓非对称，就是指该算法需要一对密钥，使用其中一个加密，则需要用另一个才能解密。对称加密只需要一个，即加密和解密都是基于一个密码，很容易被泄漏造成不安全因素。

RSA非对称加解密基于大素数乘积难以因式分解的数学难题。所谓因式分解就是将一个整数转化成因数乘积的形式，比如12=2*6=3*4；分解质因数：12=2*2*3。计算机因式分解的过程就是在一个数的平方根内寻找能够除尽的数即为因数。

两个大素数的乘积因式分解时，除了1和其本身外，只有这两个大素数，但是分解时不知道这两个大素数，只有从最小的素数2开始，逐步排除，直到遇到这两个大素数中较小的一个，所以非常困难。假设有一个由两个素数乘积得到的1024位的整数n，其数量级为$2^{1020}$，在它的平方根范围内找因数，极限循环次数是$2^{510}$（也就是最后一个数才找到），假设在一半的时候($2^{255}$)能找到第1个因式（大素数），那么至少需要做$2^{255}$次除法。假设现在的计算机每秒钟可以进行1亿亿次($10^{16}$，约等于$2^{53}$)大整数除法运算，这应该已经超过一般的巨型机的速度了（天河一号每秒2570万亿次），那么需要$2^{255}/2^{53}=2^{202}$秒才能得出结果，约等于$2*10^{53}$年，这个时间是无法承受的。

RSA公钥加密算法是1977年由罗纳德·李维斯特（Ron Rivest）、阿迪·萨莫尔（Adi Shamir）和伦纳德·阿德曼（Leonard Adleman）一起提出的。1987年首次公布，当时他们三人都在麻省理工学院工作。RSA就是他们三人姓氏开头字母拼在一起组成的。

图 6-6 RSA 公开密钥算法的发明人

如图 6-6 所示，RSA 公开密钥算法的发明人，从左到右 Ron Rivest， Adi Shamir, Leonard Adleman。照片摄于 1978 年。

RSA 是目前最有影响力的公钥加密算法之一，它能够抵抗到目前为止已知的绝大多数密码攻击，已被 ISO 推荐为公钥数据加密标准。

RSA 的算法涉及如下 6 个参数：

**p，q，n，ϕ(n)，e，d**

其中，p，q 是两个大素数，n 是两个大素数 p、q 的积，即 n=pq（n 分解为 p，q 是数学难题），n 的二进制表示时所占用的位数，就是所谓的密钥长度（实际应用中有 1024 位，2048 位等标准），如表 6-1 所示，表中给出了不同保密级别，密钥长度和加密算法对应的保密年限。

| 保密级别 | 对称密钥长度（bit） | RSA密钥长度（bit） | ECC密钥长度（bit） | 保密年限 |
|---|---|---|---|---|
| 80 | 80 | 1024 | 160 | 2010 |
| 112 | 112 | 2048 | 224 | 2030 |
| 128 | 128 | 3072 | 256 | 2040 |
| 192 | 192 | 7680 | 384 | 2080 |
| 256 | 256 | 15360 | 512 | 2120 |

表 6-1 不通保密等级和保密年限

ϕ(n)是 n 的欧拉函数（参考本书 6.6 节），根据欧拉函数的定义有：

$$\phi(n)=\phi(pq)=(p-1)(q-1)$$

e 是任意一个整数，并且 $1<e<\varphi(n)$，且与 ϕ(n)互质 ，也就是与(p-1)(q-1)互质。

d 是 e 对于 ϕ(n)的模反元素，也就是 e*d 除以 ϕ(n)余 1，即：

$$e*d \equiv 1 \bmod(\phi(n))$$

也就是：

$$e*d-1=k\phi(n),\quad k \text{ 为整数}$$

其中 d 和 n 为已知，k 和 d 未知，要求 k 和 d，就是一个下面形式的二元一次方程求整数解：

$$ex+\varphi(n)y=1$$

其中 x 的解对应 d，y 的解对应 k。要解这个方程，需要用到贝祖定理。贝祖定理是法国数学家贝祖提出，它指出：若 a，b 是整数,且 a，b 的最大公约数 gcd(a,b)=d，那么对于任意的整数 x，y，ax+by 都一定是 d 的倍数，特别地，一定存在整数 x，y，使 ax+by=d 成立。

贝祖定理的一个重要推论是：a，b 互质的充要条件是存在整数 x，y 使 ax+by=1。

因此通过贝祖定理我们知道，上面的方程有整数解的条件就是：e 与φ(n)互质，这就是为什么我们在一开始，要求 e 与φ(n)互质的原因，这样上面的方程才会有解。该方程的具体解法可以参考本书 6.5 节扩展欧几里得算法求解。其中 x 的值就是我们要求的 d。

这样（n，e), (n，d)就是密钥对。其中(n，e)为公钥，(n，d)为私钥。

这 6 个参数中，公钥(n，e)是可以公开的，其余 4 个数字是不公开的，尤其是 d，一旦泄漏，就相当于私钥的泄漏。

那么，如果在知道公钥(n，e)的情况下，有没有可能推导出私钥中的 d 呢？

因为 ed≡1 (mod φ(n))。只有知道 e 和φ(n)，才能算出 d。由于 e 为已知，而φ(n)=(p-1)(q-1)。所以只有知道 p 和 q，才能算出φ(n)。而 n=pq，只有将 n 因数分解，才能算出 p 和 q。因此如果 n 可以被因数分解，d 就可以算出，也就意味着私钥被破解。

但大整数的因数分解，在非量子计算的情况下，是一件非常困难的事情。比如你没法对下面这个整数进行因数分解：

```
1230186684530117755130494958384962720772853569595334
7921973224521517264005072636575187452021997864693899
5647494277406384592519255732630345373154826850791702
6122142913461670429214311602220124047927473779408066535141959745985690214341 3
```

它等于这样两个素数的乘积：

```
3347807169895689878604416984821269081770479498371376
8568912431388982883793878002287614711652531743087737
814467999489
        ×
3674604366679959042824463379962795263227915816434308
7642676032283815739666511279233373417143396810270092
798736308917
```

事实上，这是截止目前人类能分解的最大整数（232 个十进制位，768 个二进制位）。

RSA 加解密的算法完全相同，用公钥加密，私钥解密；也可以用私钥加密，公钥解密（公钥加密体制中，一般用公钥加密，私钥解密）。

设 A 为明文，B 为加密后的密文，则公钥加密，私钥解密如下：

公钥加密：$B=A^e \bmod n$；

私钥解密：$A=B^d \bmod n$。

私钥加密，公钥解密如下：
私钥加密：$B=A^d \bmod n$;
公钥解密：$A=B^e \bmod n$

RSA 加解密步骤可以分为：
1，生成两个大素数：p， q;
2，计算 3 个数：n, e, d
n:n=p*q
e:e 与(p-1)*(q-1)互质
d:(e*d)mod((p-1)*(q-1))=1
3，生成公钥和私钥：(n, e):公钥，(n, d):私钥
4，加密与解密：
A:明文，B 密文
加密：$B=A^e \bmod n$;
解密：$A=B^d \bmod n$;
比如：
1，p=61, q=53，商业应用中，p，q 越大越好
2，$n = 61\times53 = 3233$，e=17, d=2753。3233 写成二进制是 12 位的 110010100001，即这个密钥就是 12 位。商业应用里 RSA 密钥一般是 1024 位，重要场合则为 2048 位。
3，公钥就是 (3233, 17)，私钥就是（3233, 2753）
4，加密：$65^{17} \pmod{3233} = 2790$
5，解密：$2790^{2753} \pmod{3233} = 65$
现在我们用数学理论来证明，公钥加密之后私钥可以得到解密信息。
A 为明文，经过公钥(n, e)加密得密文：

$$B=A^e \bmod n。$$

私钥(n, d)解密，即需要证明：

$$A=B^d \bmod (n) \qquad ①$$

因为：$B=A^e \bmod n$，因此可以写成：

$$B= A^e-kn，k 为整数 \quad ②$$

把②代入①，因此需要证明：

$$A=(A^e-kn)^d \bmod (n)$$

等同于证明：

$$A=A^{ed} \pmod n$$

因为 ed，$\phi(n)$互质，即：

$$e*d \equiv 1 \bmod(\phi(n))$$

所以：

$$e*d=h\varphi(n)+1, h 为整数$$

因此只需要证明：

$$A=A^{h\varphi(n)+1} \pmod n \quad ③$$

如果 A 与 n 互质，那么由欧拉定理有：

$$A^{\varphi(n)} \equiv 1 \pmod n$$

即

$$A^{\varphi(n)} =kn+1 \quad k 为整数 \quad ④$$

把④代入③，即只需要证明：

$$A=A（kn+1）^h（\mathrm{mod}\ n）$$

对A（kn+1）$^h$通过二项式定理展开：

$$A(kn+1)^h = A((kn)^h + h(kn)^{h-1} + \frac{h(h-1)}{2!}(kn)^{h-2} + ...h(kn) + 1)$$

很显然有：

$$A（kn+1）^h（\mathrm{mod}\ n）=A$$

所以得证。

如果A与n不互质(有公约数），那么由于：n=pq，那么必然：A=kp或者A=kq（k为整数）。由于p和q的通用性，以A=kp为例，k与q必然互质，否则，设k=r*q（r为整数），那么A=kp=r*q*p=kq，这与A=kp矛盾。因此A与q互质，由欧拉定理（费马小定理）有：

$$(kp)^{q-1} \equiv 1 \ (\mathrm{mod}\ q)$$

进一步变形整理有：

$$(kp)^{(q-1)*h*(p-1)}*kp \equiv kp(\mathrm{mod}q)$$

因为(q-1)(p-1)与ed互质，所以有：

$$ed=h(q-1)(p-1)+1$$

代入$(kp)^{(q-1)*h*(p-1)}*kp \equiv kp(\mathrm{mod}q)$，有：

$$(kp)^{ed}=kp(\mathrm{mod}q)$$

于是有：

$$(kp)^{ed}=tq+kp \quad (t, k \text{ 为整数})$$

由于q与p互质，因此为了让上式成立，t必须为p的整数倍，即：

$$t=rp \quad (r \text{ 为整数})$$

于是有：

$$(kp)^{ed}=rpq+kp$$

因为A=kp，n=pq，代入上式，有：

$$(A)^{ed}=rn+A$$

所以：

$$(A^e)^d \mathrm{mod}(n)=A$$

得证。

## 6.8.3 OpenSSL 中 RSA 算法

OpenSSL是一个开源软件库包，提供了丰富的对称和非对称密码算法库，应用程序可以使用这个包来进行加解密和安全通信。

OpenSSL一共提供了8种对称加密算法，其中7种是分组加密算法，仅有的一种流加密算法是RC4。这7种分组加密算法分别是AES、DES、Blowfish、CAST、IDEA、RC2、RC5，都支持电子密码本模式（ECB）、加密分组链接模式（CBC）、加密反馈模式（CFB）和输出反馈模式（OFB）四种常用的分组密码加密模式。其中，AES使用的加密反馈模式（CFB）和输出反馈模式（OFB）分组长度是128位，其它算法使用的则是64位。事实上，DES算法里面不仅仅是常用的DES算法，还支持三个密钥和两个密钥3DES算法。

OpenSSL一共实现了4种非对称加密算法，包括DH算法、RSA算法、DSA算法和椭

圆曲线算法（ECC）。DH 算法一般用于密钥交换。RSA 算法既可以用于密钥交换，也可以用于数字签名。DSA 算法则一般只用于数字签名。

OpenSSL 实现了 5 种信息摘要算法，分别是 MD2、MD5、MDC2、SHA（SHA1）和 RIPEMD。SHA 算法事实上包括了 SHA 和 SHA1 两种信息摘要算法。此外，OpenSSL 还实现了 DSS 标准中规定的两种信息摘要算法 DSS 和 DSS1。

现在，我们来看看如何基于 OpenSSL 来实现 RSA 的加解密算法：

**基于 OpenSSL 的 RSA 加密算法**

```
char *my_encrypt(char *str, char *path_key, char **out, int *size){
    char *p_en;
    RSA *p_rsa;
    FILE *file;
    int flen, rsa_len;
    if ((file = fopen(path_key, "r")) == NULL){
        return NULL;
    }
    if ((p_rsa = PEM_read_RSA_PUBKEY(file, NULL, NULL, NULL)) == NULL){
        return NULL;
    }
    flen = strlen(str);
    rsa_len = RSA_size(p_rsa);
    p_en = (char *)malloc(rsa_len + 1);
    memset(p_en, 0, rsa_len + 1);
    if (RSA_public_encrypt(rsa_len, (unsigned char *)str, (unsigned char*)p_en, p_rsa,
        RSA_NO_PADDING)<0){
        return NULL;
    }
    RSA_free(p_rsa);
    fclose(file);
    *out = p_en;
    *size = rsa_len;
    return p_en;
}
```

**基于 OpenSSL 的 RSA 解密算法：**

```
char *my_decrypt(char *str, char *path_key, char **out, int *size){
    char *p_de;
    RSA *p_rsa;
    FILE *file;
    int rsa_len;
    if ((file = fopen(path_key, "r")) == NULL){
        return NULL;
    }
    if ((p_rsa = PEM_read_RSAPrivateKey(file, NULL, NULL, NULL)) == NULL){
        return NULL;
    }
```

```
        rsa_len = RSA_size(p_rsa);
        p_de = (char *)malloc(rsa_len + 1);
        memset(p_de, 0, rsa_len + 1);
        if (RSA_private_decrypt(rsa_len, (unsigned char *)str, (unsigned char*)p_de, p_rsa,
            RSA_NO_PADDING)<0){
            return NULL;
        }
        RSA_free(p_rsa);
        fclose(file);

        *out = p_de;
        *size = rsa_len;
        return p_de;
    }
```

使用 openssl 命令生成密钥：openssl genrsa -out test.key 1024

使用 openssl 命令生成公钥：openssl rsa -in test.key -pubout -out test_pub.key

RSA 私钥：

```
-----BEGIN RSA PRIVATE KEY-----
MIICXgIBAAKBgQDy/E5xyt3SXid9KhLpQSgge76H134hSZ527pDveWeCHYDzGu/z
KP+VLouhM+18pjm1YcALIJn/pSsh94pHCtoNWDiyKVpmV6Hm1Rq4SUO2mnKdOs83
/xt9yTlZF9HqfgddzIMNHulanLeOO3Zx+2Rolg5fcSusCQcrPG4TH+8FFwIDAQAB
AoGBAKQDdLvQsKxIC6Wr2boC1iAfBvvQlH3JoKIs5D4RF7I3UgxGuYLLwMEk+vEk
CQDjfv6U7blVFz39ckDqQzcy9v1MP/lkMXn3MQh9aS0MDtW8XZ1illRUTySUOnof
asxW/PutJBfgoazfaZlu6qu6mULCZDmVogA7VIqvIn4Pi0FBAkEA/Abegsit7D9a
HQRhlSFwoTEcQafMCSsINpN4RKmPj5dCMhLd6jovHZ2O09UJoJZ89txUAVeRVoLB
05IZGPtXdwJBAPbQ8tAQFyYjOWmZl1rS2QmtaOvYc+Eh25t2FM3rvNfTgek/P9Pj
NCuzw+ZXoBOGnFvzVLrq2pNJOliBEUAOZ2ECQQDI52TrVrQmdt4L8Zkke5LAPFjt
Eaj5yf7zGp2uBoj1iztulVGnzRgQALCBtvrEtBIL69lfdIHcN8HxOS6eloXDAkEA
tU8zP9LCHpr7nu5cODTxiOFhVPdx0K09lMs+ZzLIS8etBDoZWj7qYTwyKXCvQAi/
rfBnitPhcxb1DHTrxiGOoQJABSc1f+gzqPQIoI7GFCgCn0vpTadf9FHb6coZQsCK
RSxgdFdug6fr6hxZ+8YxOuW6f2NFEv1W79v8DPVOVsaD7w==
-----END RSA PRIVATE KEY-----
```

RSA 公钥：

```
-----BEGIN PUBLIC KEY-----
MIGfMA0GCSqGSIb3DQEBAQUAA4GNADCBiQKBgQDy/E5xyt3SXid9KhLpQSgge76H
134hSZ527pDveWeCHYDzGu/zKP+VLouhM+18pjm1YcALIJn/pSsh94pHCtoNWDiy
KVpmV6Hm1Rq4SUO2mnKdOs83/xt9yTlZF9HqfgddzIMNHulanLeOO3Zx+2Rolg5f
cSusCQcrPG4TH+8FFwIDAQAB
-----END PUBLIC KEY-----
```

# 七，椭圆曲线与 ECC 加解密

解析几何把代数与几何紧密联系在一起，它是微积分的基础，在安全领域也有着重要应用，比如加密算法领域的椭圆曲线加密与序列号保护就和解析几何中的椭圆曲线息息相关。

解析几何是借助笛卡尔坐标系，由笛卡尔（法国数学家哲学家，1596-1650，如图 7-1）、费马（1601-1665）等数学家创立并发展，并用代数方法研究几何对象之间的关系和性质的一门几何学分支，也叫做坐标几何。

图 7-1 笛卡尔

如图 7-2 所示，二维的直角坐标系由两个互相垂直的坐标轴设定，分别称为 x 轴和 y 轴；两个坐标轴的相交点，称为原点，通常标记为 O ，既有“零”的意思，又是英语“Origin”的首字母。每一个轴都指向一个特定的方向。x 轴和 y 轴决定了一个平面，称为 xy 平面，又称为笛卡尔平面。习惯性地，x 轴水平摆放，称为横轴，通常指向右方；y 轴被竖直摆放而称为纵轴，通常指向上方。

为了方便空间图形与数的研究，我们需要建立三维空间的点 P 与有序数组(x, y, z)之间的联系，为此我们通过引进空间直角坐标系来实现。同样如图 7-2 所示，过定点 O，作三条互相垂直的数轴，它们都以 O 为原点且一般具有相同的长度单位，这三条轴分别叫做 x 轴(横轴）、y 轴(纵轴)、z 轴(竖轴)，并统称坐标轴。通常把 x 轴和 y 轴配置在水平面上，而 z 轴则是铅垂线；它们的正方向要符合右手规则，即以右手握住 z 轴，当右手的四指从正向 x 轴以 π /2 角度转向正向 y 轴时，大拇指的指向就是 z 轴的正向，这样的三条坐标轴就组成了一个空间直角坐标系（或三维笛卡尔坐标系），点 O 叫做坐标原点。

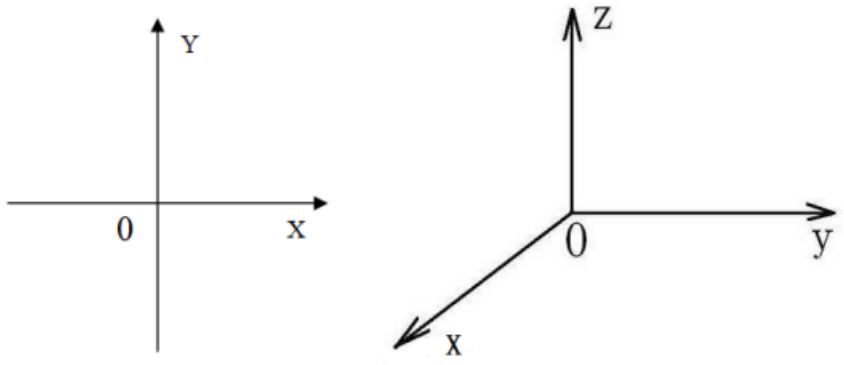



图 7-2 二维坐标系与三维坐标系

在三维笛卡尔坐标系中，三个平面，xy-平面，yz-平面，xz-平面，将三维空间分成了八个部分，称为卦限(octant) 。第 I 卦限的每一个点的三个坐标都是正值。

## 7.1 解析几何简介

十六世纪以后，由于生产和科学技术的发展，天文、力学、航海等方面都对几何学提出了新的需要。比如，德国天文学家开普勒发现行星是绕着太阳沿着椭圆轨道运行的，太阳处在这个椭圆的一个焦点上。意大利科学家伽利略发现投掷物体是沿着抛物线运动的。这些发现都涉及到圆锥曲线，要研究这些比较复杂的曲线，原先的一套方法显然已经不适应了，这就导致了解析几何的出现。

1637 年，法国的哲学家和数学家笛卡尔发表了他的著作《方法论》，这本书的后面有三篇附录，其中一篇叫《几何学》。笛卡尔的《几何学》共分三卷，第一卷讨论尺规作图;第二卷是曲线的性质;第三卷是立体和"超立体"的作图，但他实际是代数问题，探讨方程的根的性质。后世的数学家和数学史学家都把笛卡尔的《几何学》作为解析几何的起点。笛卡尔的中心思想是建立起一种"普遍"的数学，把算术、代数、几何统一起来。他设想，把任何数学问题化为一个代数问题，在把任何代数问题归结到去解一个方程式。

为了实现上述的设想，笛卡尔从天文和地理的经纬制度出发，指出平面上的点和实数对(x，y)存在对应关系，x 和 y 的不同数值可以确定平面上许多不同的点，这样就可以用代数的方法研究曲线的性质。这就是解析几何的基本思想。

具体地说，平面解析几何的基本思想有两个要点：

第一，在平面建立坐标系，一点的坐标与一组有序的实数对相对应。

第二，在平面上建立了坐标系后，平面上的一条曲线就可由带两个变数的一个代数方程来表示了。

从这里可以看到，运用坐标法不仅可以把几何问题通过代数的方法解决，而且还把变量、函数以及数和形等重要概念密切联系了起来。

据说在笛卡尔生病期间他反复思考一个问题：几何图形是直观的，而代数方程是比较抽象的，能不能把几何图形与代数方程结合起来，也就是说能不能用几何图形来表示方程呢？要想达到此目的，关键是如何把组成几何图形的点和满足方程的每一组“数”挂上钩。一次他看见屋顶角上的一只蜘蛛，拉着丝垂了下来，一会功夫，蜘蛛又顺着丝爬上去，在上边左右拉丝。蜘蛛的“表演”使笛卡尔的思路豁然开朗。他想，可以把蜘蛛看做一个点，它在屋子里可以上、下、左、右运动，能不能把蜘蛛的每个位置用一组数确定下来呢？他又想，屋子里相邻的两面墙与地面交出了三条线，如果把地面上的墙角作为起点，把交出来的三条线作为三根数轴，那么空间中任意一点的位置就可以用这三根数轴上找到有顺序的三个数。反

过来，任意给一组三个有顺序的数也可以在空间中找出一点 P 与之对应。同样道理，用一组数（x、y）可以表示平面上的一个点，平面上的一个点也可以有用一组两个有顺序的数来表示，这就是坐标系的雏形。

直角坐标系的创建，在代数和几何上架起了一座桥梁，它使几何概念用数来表示，几何图形也可以用代数形式来表示。由此笛卡尔在创立直角坐标系的基础上，创造了用代数的方法来研究几何图形的数学分支-解析几何。他设想：如果把几何图形看成是动点的运动轨迹，就可以把几何图形看成是由具有某种共同特征的点组成。

举一个例子来说，我们可以把圆看作是动点到定点距离相等的点的轨迹。如果我们再把点看作是组成几何图形的基本元素，把数看作是组成方程的解，于是代数和几何就这样合为一家人了。

解析几何的产生并不是偶然的。在笛卡尔写《几何学》以前，就有许多学者研究过用两条相交直线作为一种坐标系，也有人在研究天文、地理的时候，提出了一个点的位置可由两个坐标(经度和纬度)来确定。这些都对解析几何的创建产生了很大的影响。

解析几何在数学发展中起了推动作用。恩格斯对此曾经作过评价：“数学中的转折点是笛卡尔的变数，有了变数，运动进入了数学；有了变数，辩证法进入了数学；有了变数，微分和积分也就立刻成为必要的了。”恩格斯曾经把“解析几何、对数、和微积分”当做 17 世纪三大数学发明。

下面我们就来学习一些典型的几何图形对应的代数方程，其中包括直线、圆、椭圆、双曲线、抛物线以及椭圆曲线等。

## 7.2 直线与方程

直线是几何图形中最简单的一种图形，因此，我们首先来研究一下直线在解析几何里的直角坐标系方程。首先来学习一下直线倾斜角的定义。

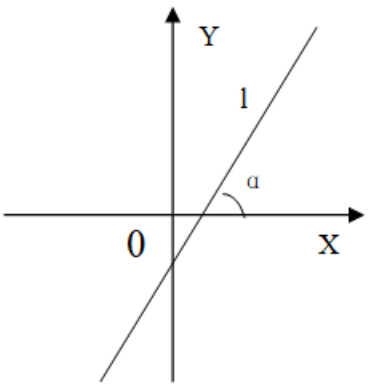



图 7-3 直线倾斜角

如图 7-3 所示，当直线 l（line 的首字母）与 x 轴相交时，我们取 x 轴作为基准，x 轴正向与直线 l 向上方向之间所成的角 α 叫做直线 l 的倾斜角。角 α 的取值范围 $0° \leqslant \alpha < 180°$。如果直线与 x 轴重合或者平行，倾斜角为 0°。倾斜角 α 的正切值 $\tan\alpha$ 叫做直线的斜率，常用小写字母 k 表示，$k = \tan\alpha$。当倾斜角为 90° 时，即当与 x 轴垂直的时候，直线的斜率为无穷大。

给定直线上的两点及坐标 $P_1(x_1, y_1), P_2(x_2, y_2)(x_1 \neq x_2)$，那么直线的斜率 k 为：

$$k=(y_2-y_1)/(x_2-x_1)$$

比如$P_1(4, 5)$，$P_2(0, 3)$，那么直线的斜率：

$$k=(5-3)/(4-0)=1/2。$$

知道了直线的斜率，那么可以确定直线与直线的平行与垂直关系。即：

如果两条直线$l_1$和$l_2$的斜率$k_1$，$k_2$相等⇔（等价于）这两条直线平行：

$$l_1 // l_2 \Leftrightarrow k_1=k_2$$

如果两条直线$l_1$和$l_2$的斜率$k_1$，$k_2$乘积为-1⇔（等价于）这两条直线互相垂直：

$$l_1 \perp l_2 \Leftrightarrow k_1k_2=-1$$

在解析几何里，每种几何图形，都对应一个方程。满足这个方程的点都在这个几何图形上，而这个几何图形的所有的点，也都满足这个方程。现在我们来介绍直线的方程，并给出几种求直线方程的形式。

**1.直线点斜式方程**

如果知道了直线的斜率k，也知道了直线过一点$p(x_0, y_0)$，那么这条直线就确定了，并且它的方程可以用点斜式方程给出：

$$y-y_0=k(x-x_0)$$

我们把这个方程叫做直线的点斜式方程，即通过一点和斜率确定的方程。如果直线经过的这一点为(0, b)，即横坐标为0，代入直线的点斜式方程，即可得出直线的斜截式方程如下：

**y=kx+b（k为斜率，我们称b为y轴截距）**

练习1：

求如下直线的方程：

（1）经过点A(3,-1)，斜率为$\sqrt{2}$

（2）经过点B(0,3)，斜率为1

（3）判断下列两组直线的位置关系：

a. $l_1$:y=1/x+3; $l_2$:y=1/2x-2

b. $l_1$:y=5/3x; $l_2$:y=-3/5x

解：

（1）根据直线的点斜式方程，有：$y-(-1)=\sqrt{2}(x-3)$，化简之后为：

$$y=\sqrt{2}x-3\sqrt{2}-1$$

（2）根据直线的斜截式方程，有：y=x+3

（3）a:$l_1$和$l_2$的斜率相等，都为1/2，所以二者是平行关系；

b:$l_1$的斜率$k_1$为5/3，$l_2$的斜率$k_2$为-3/5，二者相乘$k_1*k_2=-1$，因此二者是垂直关系。

**2.直线的两点式方程**

我们知道，两点确定一条直线。因此，如果给出了直线上的两个点的坐标，那么也肯定是可以求出直线的方程的。

通过直线上2点的坐标，求出直线的方程，我们称之为两点式方程。设直线上两点$P_1(x_1, y_1)$，$P_2(x_2, y_2)$，那么直线的两点式方程可以表示为：

$$\frac{y-y_1}{y_2-y_1}=\frac{x-x_1}{x_2-x_1}$$

如果给出的两点分别为x轴和y轴上的点，即$P_1(a, 0)$，$P_2(0, b)$，代入两点式方程，化简即可得出直线的截距方程：

$$\frac{x}{a}+\frac{y}{b}=1$$

其中，我们把 a 和 b 称为直线在 x 轴和 y 轴的截距。

**3.直线一般式方程**

通过上面的方法求出的直线方程，可以转化为如下形式：

Ax+By+C=0（A 和 B 不同时为 0）

我们把这种形式的直线方程称为直线的一般方程。对于直线的一般方程，我们可以很快求出直线的斜率 k：-A/B。当 A=0 时，直线平行于 X 轴；当 B=0 时，直线平行于 Y 轴。

比如，对于经过点 A(8,-1)，斜率为 1/4 的点，由点斜式，给出直线方程为：

y-(-1)=1/4(x-8)，即 y=(1/4)x-3

方程两边同时乘以 4：

4y=x-12；

移项即可得直线的一般方程：

x-4y-12=0

前面我们给出了直线平行和垂直的判断条件。当两条直线相交于一点，那么如何求出这一点的坐标呢？

显然，作为两条直线的交点，它应该同时满足两条直线的方程。因此可以将两条直线方程联立，如有唯一解，两条直线相交，解为交点坐标。若无解，则两条直线无公共点，两条直线平行。比如有两条直线：

$L_1$:2x-3y=7;$L_2$:4x+2y=1

联立方程：

$$\begin{cases} 2x-3y=7 & ① \\ 4x+2y=1 & ② \end{cases}$$

求解该方程有：

x=17/16，y=-13/8

所以，这两条直线的交点坐标为（17/16, -13/8）

在直角坐标系里，我们经常需要计算两点之间的距离。如图 7-4 所示，假设平面直角坐标中任意两点及坐标分别为 $P_1(x_1, y_1)$和 $P_2(x_2, y_2)$，过 $P_1$ 和 $P_2$ 分别作平行于 X 轴和 Y 轴的直线，相交于 Q，形成直角三角形: $\triangle P_1QP_2$

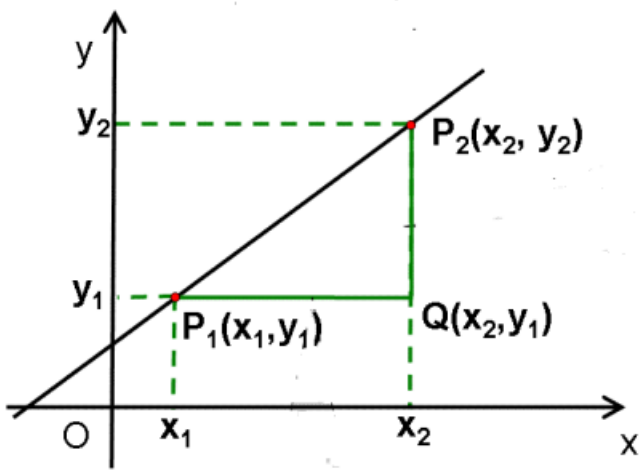



图 7-4 两点之间的距离

根据勾股定理有：$|P_1P_2|^2=|P_1Q|^2+|P_2Q|^2$，由此得到两点间的距离公式为：

$$|P_1P_2| = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

类似的，空间中两点 $P_1(x_1, y_1, z_1)$和点 $P_2(x_2, y_2, z_2)$的距离公式可表示为：

$$|P_1P_2| = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

那么，如何计算一个点到一条直线的距离呢？

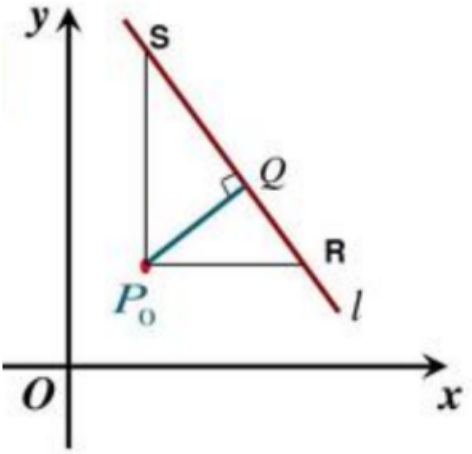



图 7-5 点到直线距离

如图 7-5 所示，设直线 l 和 l 外一点 $P_0(x_0, y_0)$，过 $P_0$ 作 l 的垂线，相交于 Q，则$|P_0Q|$就是点 $P_0$ 到直线的距离，同时过点 $P_0$ 作与 X 轴和 Y 轴的平行直线与 l 相交于点 R 和 S。易求得点 R 和点 S 的坐标，然后可求得$|P_0S|$、$|P_0R|$和$|RS|$的长度。根据三角形面积公式有$|P_0S|*|P_0R|=|RS|*|P_0Q|$，即可求出$|P_0Q|$的长度，也就是点 $P_0$ 到直线 l 的距离。按照上面的思路，计算结果如下：

$$d = \frac{|Ax_0 + By_0 + C|}{\sqrt{A^2 + B^2}}$$

有了点到直线的距离公式之后，那么也容易算出两条平行直线间的距离了。因为两条平行直线的距离，实际上等于其中一条直线上某一个点到另一条直线的距离。所以可以先确定一条直线上的点，然后利用上面的点到直线距离公式即可算出。

比如直线 $l_1$：3x-2y-1=0；$l_2$：3x-2y+1=0，求它们之间的距离。通过 $l_1$ 和 $l_2$ 的斜率相等可以判断两条直线平行。于是可以选取直线 $l_1$ 上的点 A（0，-1/2），然后计算 A 到 $l_2$ 的距离：

$$d=|3*0-2*（-1/2）+1）|/\sqrt{3^2+(-2)^2}=2\frac{\sqrt{13}}{13}$$

## 7.3 圆与方程

除了直线之外，圆是几何学中另一种基本的几何图形。那么在解析几何中，如何来表示圆的方程的呢？现在我们来学习圆的方程，首先来学习圆的定义。

## 7.3.1 圆的定义

圆可以看作是到定点的距离为定长的所有点的集合，此定点为圆心，定长为半径。因此在解析几何里，根据两点距离公式，可以得出圆的方程。

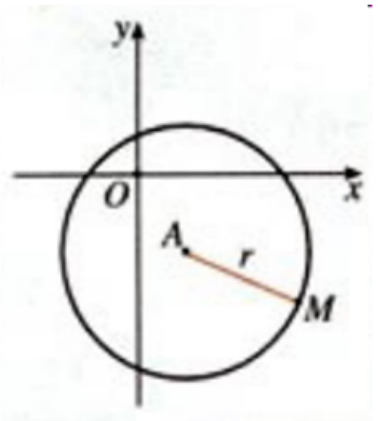



图 7-6 圆的定义

如图 7-6 所示，设圆心 A 的坐标为(a,b)，半径为 r，圆上的一点 M 的坐标为(x, y)，根据圆的定义和距离公式，可得圆的方程如下：

$$(x-a)^2+(y-b)^2=r^2$$

我们把上面的方程叫做圆的标准方程。如果圆心在原点 O(0,0)，那么圆的标准方程为：

$$x^2+y^2=r^2$$

将标准方程展开，实际是一个缺 xy 二次项的二元二次方程：

$$x^2+y^2+Dx+Ey+F=0$$

我们把上面的方程叫做圆的一般方程。给定一个缺 xy 项的二元二次方程，我们可以变通过配方法换成如下方程：

$$(x+D/2)^2+(y+E/2)^2=(D^2+E^2-4F)/4$$

通过上面的等式，我们可以得该方程为圆的一般方程的条件如下：

当 $D^2+E^2-4F>0$ 时该方程为圆的方程，其中圆心为：(-D/2, -E/2)，半径为：$\frac{1}{2}\sqrt{D^2+E^2-4F}$。

## 7.3.2 直线与圆的位置关系

圆和直线的位置关系可以分为相交，相切和相离。如图 7-7 所示，在相交的情况下，圆和直线有 2 个交点，圆心和直线的距离小于半径 r；相切的情况下，圆和直线有 1 个交点，圆心到直线的距离等于半径 r；相离的情况下，圆和直线没有交点，圆心和直线的距离大于半径 r。因此判断直线和圆的位置关系有两种方法：

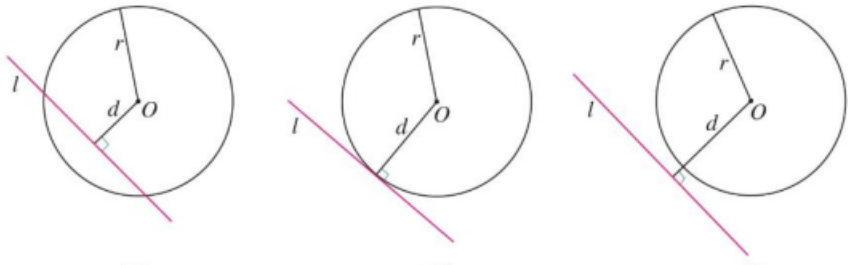



图 7-7 圆与直线关系：相交，相切和相离

方法 1：直线方程与圆的方程联立求解。如果有两个解那么两者相交，如果有一个解那么两者相切，如果无解那么两者相离。

方法 2：求圆心到直线的距离，该距离小于圆的半径那么两者相交；该距离等于圆的半径那么两者相切；该距离大于圆的半径那么两者相离。

### 7.3.3 圆与圆的位置关系

圆和圆的位置关系可分为相离、外切、相交、内切和内含五种关系，如图 7-8 所示。这五种关系同样可以通过 2 个圆方程联立求解或者通过计算两个圆圆心距离与两个圆半径之和的大小关系来判断。具体判断方法如下：

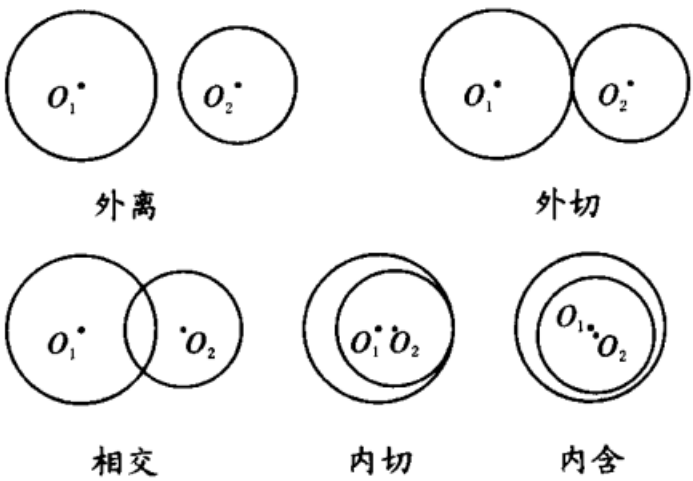



图 7-8 圆与圆的位置关系

方法 1：两个圆方程联立，求解。如果有 2 个解则两个圆相交，1 个解则两个圆相切，无解则两个圆相离。

方法 2：计算圆心到圆心的距离设为 d，如果 d 小于两个圆半径之和 $r_1+r_2$ 则两个圆相交；如果 d 等于 $r_1+r_2$ 或$|r_1-r_2|$则两个圆相切；如果 d 大于 $r_1+r_2$ 则两个圆相离。

## 7.4 椭圆的定义和方程

平面内与两定点 $F_1$、$F_2$ 的距离之和等于常数 $2a(2a>|F_1F_2|)$的动点 P 的轨迹叫做椭圆。如图 7-9 所示，$|PF_1|+|PF_2|=2a$。其中两定点 $F_1$、$F_2$ 叫做椭圆的焦点，两焦点的距离 $|F_1F_2|=2c<2a$ 叫做椭圆的焦距。P 为椭圆上的动点。

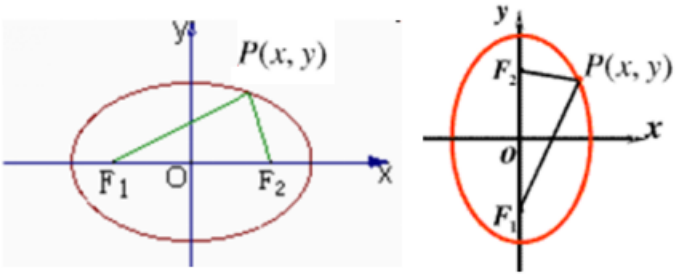



图 7-9 椭圆图形

我们把 e=c/a 称为椭圆的离心率，由于 c<a，所以椭圆的离心率 e∈(0, 1)。离心率是椭圆扁平程度的一种量度， 即椭圆轨道与理想圆环的偏离，长椭圆轨道 e 越高，椭圆越扁平，而近于圆形的轨道 e 越低，圆的离心率为 0。

椭圆的长轴为 2a，短轴为 2b。当焦点在 x 轴时，椭圆的标准方程是：

$$x^2/a^2+y^2/b^2=1，(a>b>0)$$

当焦点在 y 轴时,椭圆的标准方程是：

$$y^2/a^2+x^2/b^2=1，(a>b>0)$$

其中：a、b、c 之间的关系为：

$$a^2-c^2=b^2$$

实际上太阳系行星绕太阳的运行轨迹就是椭圆，太阳位于椭圆上的一个焦点上。物理学先驱伽利略在研究物体运动时，把行星绕日的运动轨迹当做圆，后来开普勒观察到行星绕日运动的轨迹实际上为椭圆而并非圆，但他无法解释为什么会这样，而伽利略也忽视了开普勒的观测结果，从而错过了万有引力定律的发现。后来牛顿认真研究、分析和计算了行星绕日运动为椭圆轨迹的原因，并因此发现了万有引力的存在。

## 7.5 双曲线的定义和方程

距离平面内两个定点 $F_1$ 与 $F_2$ 的距离的差的绝对值等于一个常数(这个常熟之记为 2a)的所有点的轨迹称为双曲线。如图 7-10 所示，当$|MF_1|-|MF_2|=2a$ 时，曲线仅表示焦点 $F_2$ 所对应的一支。当$|MF_1|-|MF_2|=-2a$ 时曲线仅表示焦点 $F_1$ 所对应的一支。当$|F_1F_2|=2a$ 时动点轨迹表示以 $F_1$，$F_2$ 为端点的两条射线。当$|F_1F_2|<2a$ 时动点轨迹不存在。

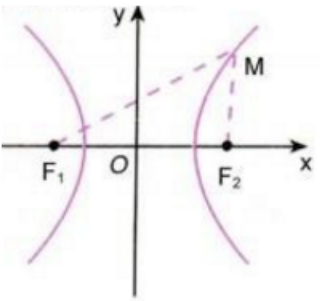



图 7-10 双曲线

根据双曲线的定义和两点之间的距离公式，可以得出焦点在 X 轴上时双曲线的方程为：

$$x^2/a^2 - y^2/b^2 = 1$$

焦点在 Y 轴上时双曲线的方程为：

$$y^2/a^2 - x^2/b^2 = 1$$

对于两个点 A(-a, 0)、A'(a, 0 )，AA'叫做双曲线的实轴且 | AA' | =2a; 对于两点 B(0, -b )、

B'(0, b )，BB'叫做双曲线的虚轴且 | BB' | =2b；对于点 F1(-c, 0)，F2(c, 0)，F1 为双曲线的左焦点，F2 为双曲线的右焦点且 | F1F2 | =2c（>2a）。

对实轴、虚轴、焦点的关系有等式：$a^2+b^2=c^2$ 成立。

双曲线的离心率 e=c/a，由于 c>a，所以双曲线的离心率大于 1。

## 7.6 抛物线定义和方程

平面内，到定点 F 与定直线 l 的距离相等的点的轨迹叫做抛物线。如图 7-11 所示，其中定点 F 叫抛物线的焦点，定直线 l 叫抛物线的准线。

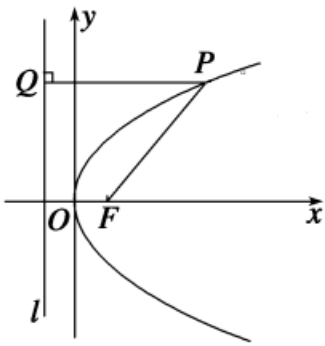



图 7-11 抛物线

如图 7-12 所示，抛物线有多种开口方向：

- 右开口抛物线：$y^2=2px$，焦点是(p/2, 0)，准线的方程是 x= -p/2，离心率 e=1，范围：x≥0；
- 左开口抛物线：$y^2= -2px$，焦点是(-p/2, 0)，准线的方程是 x=p/2，离心率 e=1，范围：x≤0；
- 上开口抛物线：$x^2=2py$，焦点是(0, p/2)，准线的方程是 y= -p/2，离心率 e=1，范围：y≥0；
- 下开口抛物线：$x^2= -2py$，焦点是(0, -p/2)，准线的方程是 y=p/2，离心率 e=1，范围：y≤0；

其中 p 为焦距，且 p>0。

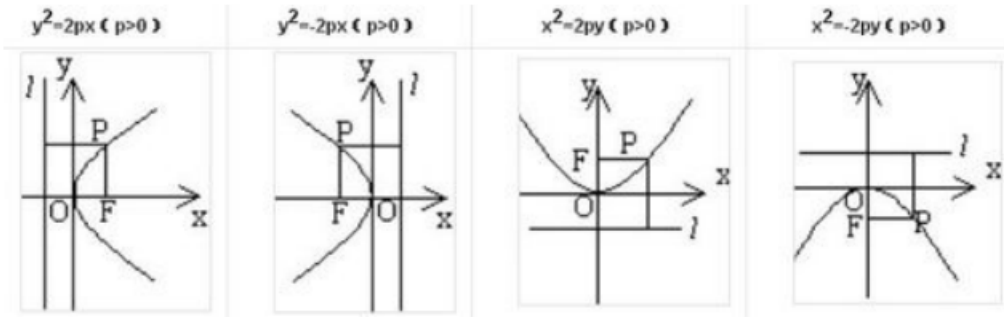



图 7-12 抛物线的开口方向

抛物线顾名思义，它是炮弹等抛射体的运行轨迹。

## 7.7 椭圆曲线加解密

对于解析几何的相关基础知识，我们就介绍到这里。现在我们来认识解析几何里一种新的曲线方程，即椭圆曲线方程，它与计算机领域的非对称加解密 ECC（Elliptic curve cryptography，椭圆曲线加解密）算法有密切的关系。由于量子计算有可能破解 RSA 加密算法，RSA 算法已经不再绝对安全，因此 ECC 算法是未来取代 RSA 算法的新的标准。

ECC 属于非对称加密算法，最初由 Koblitz 和 Miller 两人于 1985 年提出，基于一个数学难题：即椭圆曲线上的有理点构成 Abel 加法群上椭圆离散对数的计算困难。要彻底理解 ECC 的加解密原理，包括相关的数学概念，比如 Abel 加法群，离散对数等，我们得先从抽象代数基础开始学习。

### 7.7.1 抽象代数基础

抽象代数（Abstract algebra）又称近世代数（Modern algebra），它产生于十九世纪。法国数学家伽罗瓦〔Galois，1811-1832，如图 7-13 所示〕在 1832 年运用“群”的概念彻底解决了用根式求解高次代数方程的可能性问题，他指出大于等于 5 次的一元高次方程，是没有根式解的。他是第一个提出“群”的概念的数学家，一般称他为近世代数创始人。

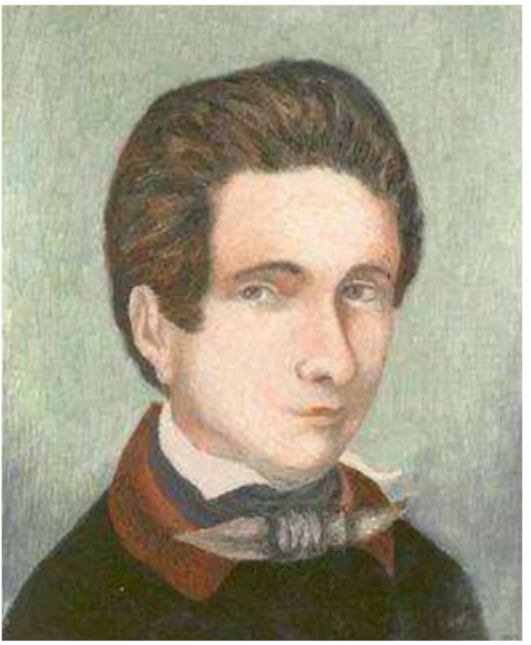


图 7-13 伽罗瓦

抽象代数包含群论、环论、伽罗瓦理论、格论、线性代数等许多分支，并与数学其它分支相结合产生了代数几何、代数数论、代数拓扑、拓扑群等新的数学学科。抽象代数也是现代计算机理论基础之一。

群(group)、环(ring)、域(field)是抽象代数中基本的代数结构。其中群是抽象代数里重点研究的对象，在抽象代数中具有重要地位。许多代数结构，包括环、域和模等可以看作是在群的基础上添加新的运算和公理而形成的。

群论是伽罗瓦发明的，他用该理论，具体来说是伽罗瓦群，解决了五次方程问题。伽罗瓦是一个极具传奇性的人物，年仅 21 岁就英年早逝于一场近乎自杀的决斗中。

**1.群**

若集合 G≠Φ，在 G 上的二元运算“ ·”（该运算称为群的乘法，其结果称为积） 满足：

- 封闭性：即 G 的任意两个元素在“ ·”下的运算结果都是该集合的一个元素，即：$\forall a,b\in G,\ a\cdot b\in G$。
- 结合律： $\forall a,b,c\in G,\ (a\cdot b)\cdot c=a\cdot(b\cdot c)$;
- 单位元（也称为幺元）： G 中存在元素 $e$，使 G 中任意元素 $a$与之相乘（包括左乘和右乘）的结果都于 $a$本身，即 $\exists e\in G$，使 $\forall a\in G$，有 $e\cdot a=a\cdot e=a$;
- 逆元： $\forall a\in G,\ \exists b\in G$，使得 $a\cdot b=b\cdot a=e$，$b$称为 $a$的逆元，记为 $a^{-1}$。

则 $(G,\cdot)$称为一个群，或乘法群。

同时，我们把满足 1 和 2 条件的集合和二元运算称作**半群**。

群一定是半群，但是半群不一定是群。

容易证明，群中的单位元和逆元都是唯一的。

群上的二元运算通常称为加法(+)或者乘法(.)，对应的群称为加法群或者乘法群。加法的幺元通常为 0，逆元为-a；乘法的幺元通常为 1，逆元为 $a^{-1}$。当集合 G 为有限集时，则群是有限的，称为有限群，G 中的元素个数称为 G 的阶。

群的简单例子如下：

G={1，-1}在普通乘法运算下是群。因为满足：

- 封闭性：1×1=1；1×-1=-1；-1×1=-1；-1×-1=1；运算结果都在集合 G 中。
- 结合律：成立，比如：(1×1)×-1=1×(1×-1)=-1。
- 单位元：1，比如 1×1=1，-1×1=-1。
- 逆元：1×1=1，-1×-1=1。所以 1 的逆元是 1，-1 的逆元是-1。

**2.Abel 群**

若一个群 $(G,\cdot)$满足交换律：对 G 的任意两个元素 $a,b$，总有 $a\bullet b=b\bullet a$。则称群 $(G,\cdot)$为阿贝尔群，也称为加法群。

设 P 为一个素数，那么 $F_p$={0,1,2,3，…，p-1}表示模 p 的整数集，那么$(F_p,+)$是一个加法群，即 Abel 群。其中单位元为 0，“+”操作定义为模 p 的整数加，即： $(a+b)\bmod p$。

群 $(\mathbb{Z},+)$， $(\mathbb{R},+)$和 $(\mathbb{R}\setminus\{0\},\times)$都是阿贝尔群。

**3.循环群**

现在我们来分别介绍乘法群和加法群的循环子群概念。

若 G 是一个阶为 n 的有限乘法群，且 g 是 G 中的一个元素，即 g∈G，那么使 $g^t$=1 的最小正整数 t 叫作 g 的阶（注意，这里是元素 g 的阶，而不是集合 G 的阶）。一般 t 都是存在的并且是 n 的一个因子（即 n 是 t 的倍数）。对 g 进行重复的乘法运算，生成的集合{$g^i$, 0≤i≤t-1}是同 G 具有相同操作的群，称为由 g 生成的 G 的循环子群。

若 G 是一个阶为 n 的有限加法群，g 是 G 中的一个元素，即 g∈G，那么使 tg=0 的最小正整数 t 称为 g 的阶（tg 表示 t 个 g 相加），且 t 为 n 的一个因子。对 g 进行重复的加法运算，生成的集合{ig, 0≤i≤t-1}是同 G 具有相同操作的群，称为由 g 生成的 G 的循环子群。

如果 G 有一个阶为 n 的元素 g，那么 G 叫做循环群，g 叫做 G 的生成元。

**4.环**

设集合 R 上定义了两个二元运算“＋”，“＊”且满足

1.（R，+）是 Abel 群，满足交换律。

2.（R，*）是半群。

3.两种运算满足分配率，a*（b+c）=a*b+a*c

则集合 R 和两个二元运算构成的代数结构叫做环。环(G, +, *)的乘法*满足交换律：a*b=b*a。 则(G, +, *)是交换环，交换环是两个代数运算+和*都满足交换律的环。如果环(G, +, *)的乘法*运算存在单位元，且非 0 元对*运算有逆元的环称作除环，除环即支持除法运算的环。

**5.域**

设(S, +, *)是代数系统，如果满足：

(1) (S，+)是交换群；

(2) (S-{0}，*)是交换群；

(3) 运算*对运算+是可分配的。

则称(S, +, *)为域。交换除环是域，因此域是一种特殊的环。

实际上，域是一个可以在其上进行加法、减法、乘法和除法运算而结果不会超出域的集合。如有理数集合、实数集合、复数集合都是域，但整数集合不是，因为使用除法得到的分数或小数已超出整数集合。

**6.有限域**

如果域 F 只包含有限个元素，则称其为有限域。有限域中元素的个数称为有限域的阶。

每个有限域的阶必为素数的幂，即有限域的阶可表示为 $p^n$（p 是素数、n 是正整数），该有限域通常称为 Galois 域(Galois Fields)，记为 GF($p^n$)。

当 n=1 时，存在有限域 GF（p），也称为素数域。在密码学中，最常用的域是阶为 p 的素数域 GF（p）或 $F_p$。

一切有限域都有加法和乘法两种运算，并必须满足以下条件：

- 封闭性：若任意两元素 a、b∈GF(p)，则有：

  $a+b\in GF(p)$ ， $a*b\in GF(p)$
- 结合律：若任意 a、b、c∈GF(p)，则有：

  $(a+b)+c=a+(b+c)$，$(a*b)c=a(b*c)$
- 交换律：若任意 a、b∈GF(p)，则有：

  $a+b=b+a$，$a*b=b*a$
- 有乘法恒等元 e 和加法恒等元 0，使任意元素 a∈GF(p)都有：

  $a+0=a$， $a*e=a$
- 任意元素 a∈GF(p)都有乘法逆元素和加法负元素(-a)，使：

  $a+(-a)=0$， $a*a^{-1}=e$

但 $0^{-1}$ 无定义。

- 乘对加分配律:任意 a、b、c∈GF(p)有：

  $a(b+c)=a*b+a*c$

有限域在密码编码学中得到了广泛的应用。

## 7.7.2 椭圆曲线加解密基础

我们这里提到的椭圆曲线，并不是前面基础部分提到的椭圆，甚至与椭圆没有任何关系，它的方程一般由如下形式给出：

$$E:y^2=x^3+ax^2+bx+c$$

如图 7-14 所示，椭圆曲线都是关于 X 轴对称的曲线：

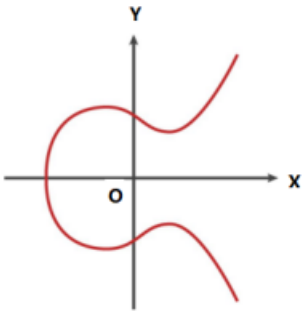



图 7-14 椭圆曲线

其中判别式为:

$$\Delta(E)=-4a^3c+a^2b^2-4b^3-27c^2+18abc\neq 0$$

限定判别式 Δ(E)不为零有特殊的意义。如果判别式 Δ(E)等于零，由三次方程判别式判定理可知，方程:

$$x^3+ax^2+bx+c=0$$

存在二重根或者三重根，曲线表现为"自相交"或者有“尖点”。比如图 7-15 中所示方程，就不是椭圆曲线方程。

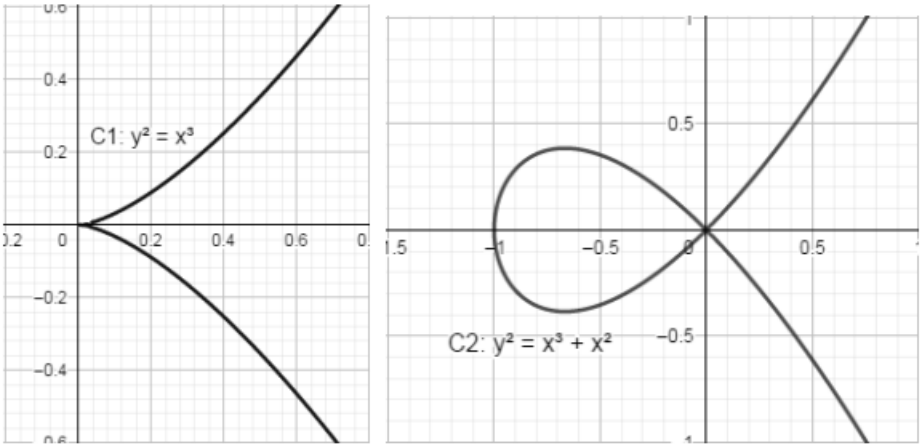



图 7-15 非椭圆曲线

在密码学中用到的椭圆曲线方程一般限定二次项系数为 0（也就是不包含二次项），即：

$$E:y^2=x^3+ax+b，\text{其中 } 4a^3+27b^2\neq 0$$

椭圆曲线上可以定义一些特殊运算规则。一般来说会定义两种运算：加法和数乘运算。加法运算是点与点之间的运算；数乘运算基于加法运算，重复的加法运算就是数乘。

**1.加法运算**

已知椭圆曲线上两个不同的点 P 和 Q，则这两个点之和 R=P+Q 可以通过如下操作得到：

过 P、Q 两点做直线 L，与椭圆曲线相交于第三点，该点关于 X 轴的对称点即是所求的 R 点。椭圆曲线的这种加法运算有比较明确的几何含义。如图 7-16 所示：

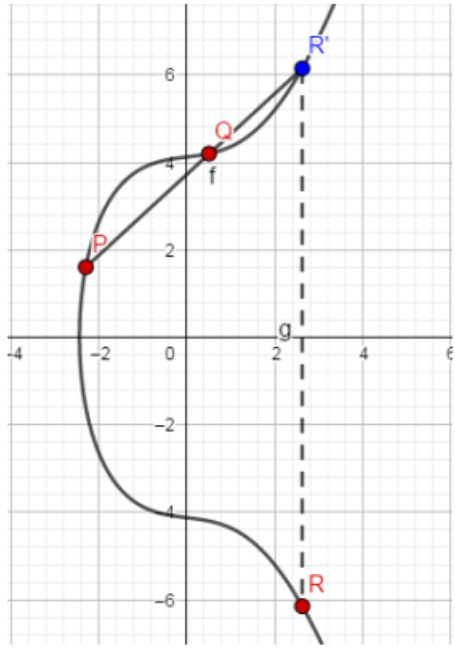



图 7-16 椭圆曲线加法运算

椭圆曲线加法运算规则如下：

- O+O=O，对任意的 P，有 P+O=P；O 看做零点，对加法运算没有实际贡献（类似于四则运算加法运算中的 0）。
- P=(x, y)的负元是关于 X 轴对称的点−P=(x,−y)（不是关于原点对称），P+(−P)=O。根据加法的定义，过 P 和−P 的直线与 X 轴垂直，该直线看起来不会与椭圆曲线再相交，但实际上可以看做它与椭圆曲线相交于无穷远点，因此也将 O 视作无穷远（∞）点。
- 计算 P 和 Q 的和，是通过做过 P 和 Q 两点的直线，与椭圆曲线相交于第三点 R'，再取该点关于 X 轴的对称点 R，以此作为 P，Q 之和，正如图 7-16 的几何图形展示的那样。
- 计算 P 点（P≠O）的两倍时，是做该点的切线，再取切线与该椭圆曲线的交点 S 关于 X 轴的对称点−S，也即是 2P=P+P=−S

容易验证，对于椭圆曲线上的点和 O 点组成的集合，以及集合上定义的二元加法运算，构成一个 Abel 群。单位元是 O 点，P(x, y)的逆元是 P(x, −y)，封闭性，结合性以及交换性也是显然满足的。

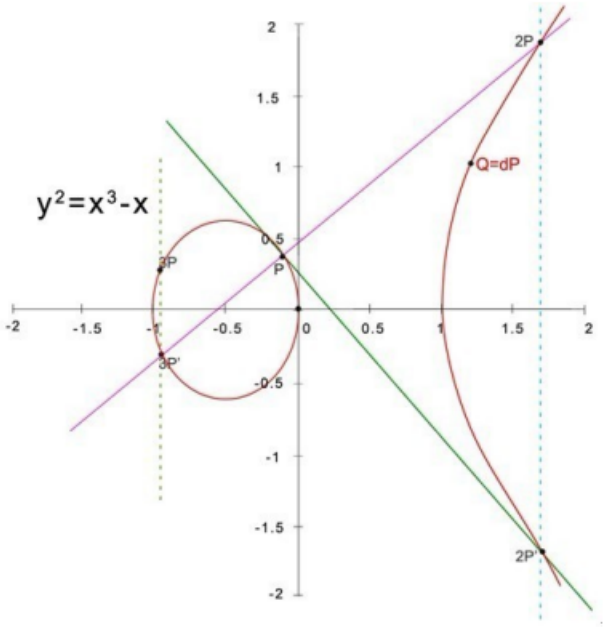



图 7-17 椭圆曲线算法原理

对于图 7-17 为方程：$y^2 = x^3 - x$ 的曲线。那么：

(1)设 P 点为基点；

(2)通过 P 点做切线，交于点 2P'点，在 2P'点做竖线，交于 2P 点，2P 点即为 P 点的 2 倍点；

(3)进一步，P 点和 2P 点之间做直线，交于 3P'点，在 3P'点做竖线，交于 3P 点，3P 点即为 P 点的 3 倍点；

(4)同理，可以计算出 P 点的 4、5、6、… 倍点；

(5)如果给定图上 Q 点是 P 的一个倍点，请问 Q 是 P 的几倍点呢？

(6)直观上理解，正向计算一个倍点是容易的，反向计算一个点是 P 的几倍点则困难的多。

在椭圆曲线算法中，将倍数 d 做为私钥，将 Q 做为公钥。当然，椭圆曲线算法还有更严格的计算过程，相比之下要复杂的多。目前由椭圆曲线公钥求解私钥的最有效算法复杂度为$O(\sqrt{p})$，其中 p 是阶数 n 的最大素因子。

**2.代数解释**

几何解释更直观，代数解释更有利于数值计算。过曲线上 $P(x_P, y_P)$和 $Q(x_Q, y_Q)$两点（P 和 Q 不互为负元）做直线，求与曲线的第三个交点的问题是很容易用代数的方法来描述的。也就是求：

$$\begin{cases} y^2=x^3+ax+b & (1) \\ y-y_P=k(x-x_P) & (2) \end{cases}$$

其中过 PQ 两点的直线斜率 $k=(y_Q-y_P)/(x_Q-x_P)$为已知。（2）与（1）联立方程，并将（2）代入（1），再利用次数对齐的方式易求第三个交点的对称点也即 P, Q 之和 $R(x_R, y_R)$为：

$$\begin{cases} x_R=k^2-x_P-x_Q \\ y_R=-y_P+k(x_P-x_R) \end{cases}$$

如果需要计算倍乘，可以让多个点自身重复相加得到。例如 P+P=2P=R，当 $y_P \neq 0$ 时，代数描述为：

$$\begin{cases} x_R=(3x_P^2+a^2y_P)^2-2x_P \\ y_R=(3x_P^2+a^2y_P)(x_P-x_R)-y_P \end{cases}$$

**3.模素数 P 的加法运算**

密码学中普遍采用的是有限域上的椭圆曲线，也即是变元和系数均在有限域中取值的椭圆曲线。使用模素数 p 的有限域 $F_p=\{0,1,2,...,p-1\}$，将模运算引入到椭圆曲线算术中，变量和系数从集合 0, 1, 2, ..., p−1 中取值而非是在实数上取值。此时讨论椭圆曲线形式如下：

$$y^2 \bmod p=(x^3+ax+b) \bmod p$$

其中$(4a^3+27b^2)\bmod p\neq 0$，mod p，变量和系数均在 $F_p$ 中取值。

将满足上式的所有非负整数对和 O 点记为集合 $E_p(a, b)$，这是一个有限的离散点集。由此可知集合中的点分布在(0, 0)到(p−1, p−1)的象限中，集合中的点有可能刚好也在椭圆曲线上，更多的可能是在椭圆曲线外。例如点(13, 7)是满足 $y^2 \bmod 23=(x^3+x+1) \bmod 23$ 的点，但是(13, 7)并不在椭圆曲线上。

实际上，集合 $E_p(a, b)$与模 p 的加法运算构成循环阿贝尔群，其生成元，阶和生成子群问题在本节后面会讨论。

对于较小的素数 p，完全可以暴力穷举找出集合 $E_p(a, b)$中的点。比如参数 a=1, b=3, p=23，$E_{23}(1, 3)$有 27 个点（包含 O 点），暴力穷举这些点分别为：

| | | |
|---|---|---|
| (0,7) | (6,15) | (15,9) |
| (0,16) | (7,10) | (15,14) |
| (2,6) | (7,13) | (19,2) |
| (2,17) | (10,1) | (19,21) |
| (4,5) | (10,22) | (21,4) |
| (4,18) | (12,8) | (21,19) |
| (5,8) | (12,15) | (22,1) |
| (5,15) | (14,1) | (22,22) |
| (6,8) | (14,22) | O |

$E_p(a, b)$上的加法规则和实数域上的加法基本一致，只是多加了模运算。而模 p 的加法没有明显的几何解释，只有代数描述。

已知 $P(x_P, y_P)$，$Q(x_Q, y_Q)$两点求解 $R(x_R, y_R)$（R=P+Q）的代数表达式为：

$$\begin{cases} x_R=(\lambda^2-x_P-x_Q)\bmod p \\ y_R=(\lambda(x_P-x_R)-y_P)\bmod p \end{cases}$$

其中λ值的确定分为两种情况：

$$\lambda=(y_Q-y_P)/(x_Q-x_P)\bmod p \qquad (P\neq Q)$$

$$\lambda=(3x_P^2+a)/(2y_P)\bmod p \qquad (P=Q)$$

现在我们考虑一个实际的例子。设椭圆曲线中 a=1，b=1，p=23，有两点 P(3，10)，Q(13，16)，求 R=P+Q 和 2P。

首先求 P+Q:

此时 P≠Q，计算$\lambda=(y_Q-y_P)/(x_Q-x_P)\bmod p=(16-10)/(13-3)\bmod 23=6\times 1/10\bmod 23$。

要计算上式首先要计算 1/10mod23。令:

$$x\equiv 1/10 \pmod{23}$$

由于 $10\equiv 10 \pmod{23}$，所以两边同时乘以 10 有：

$$10x\equiv 1 \pmod{23}$$

根据同余定理的性质有：

$$10x+23k=1$$

因为 10 和 23 互质，所以上述方程有整数解，利用扩展欧几里德算法求得：

$$x=7，k=-3$$

即：

$$1/10(\mathrm{mod}23)=7$$

所以：

$$\lambda=6\times7\mathrm{mod}23=42\mathrm{mod}23=19$$

将λ代入上面$(x_R, y_R)$的代数表达式:

$x_R=(\lambda^2-x_P-x_Q)\mathrm{mod}p=(361-3-13)\mathrm{mod}23=345\mathrm{mod}23=0$;
$y_R=(\lambda(x_P-x_R)-y_P)\mathrm{mod}p=(19\times(3-0)-10)\mathrm{mod}23=47\mathrm{mod}23=1$;
所以 R=(0, 1)。

再求 2P：
因为 2P=P+P，且 P=P，所以：
$\lambda= (3x_P^2+a)/(2y_P))\mathrm{mod}p=(3*3^2+1)/(2*10)\mathrm{mod}23=7*5^{-1} \mathrm{mod}\ 23$
设 $x=5^{-1}\mathrm{mod}23$，那么有：

$$5x\equiv1\mathrm{mod}23$$

由扩展欧几里德算法求得：

$$x=14$$

即：

$$5^{-1}\mathrm{mod}23=14$$

代入$\lambda=7*5^{-1} \mathrm{mod}\ 23$，所以：

$$\lambda=7*14\mathrm{mod}23=98\mathrm{mod}23=6$$

所以：
$x_R=(\lambda^2-x_P-x_Q)\mathrm{mod}p=(6^2-3-3)\mathrm{mod}23=7$
$y_R=(\lambda(x_P-x_R)-y_P)\mathrm{mod}p=(6(3-7)-10)\mathrm{mod}23=12$

所以，2P=(7, 12)

实际上 $E_{23}(1, 1)$中共有 28 个点（包含无穷远点 O），以 P(3, 10)开始的所有倍乘点 P, 2P, 3P, ...27P, 28P 可以暴力计算得出:

P=(3,10)
2P=(7,12)
3P=(19,5)
4P=(17,3)
5P=(9,16)
6P=(12,4)
7P=(11,3)
8P=(13,16)
9P=(0,1)
10P=(6,4)
11P=(18,20)
12P=(5,4)
13P=(1,7)
14P=(4,0)
15P=(1,16)

16P=(5,19)
17P=(18,3)
18P=(6,19)
19P=(0,22)
20P=(13,7)
21P=(11,20)
22P=(12,19)
23P=(9,7)
24P=(17,20)
25P=(19,18)
26P=(7,11)
27P=(3,13)
28P=O

容易验证，上述计算过程中 Q(13, 16)点就是 8P，P+Q=P+8P=9P=(0, 1)，与上述计算结果一致。

接下来的一个问题是子群、生成元和阶。不难得知，有限域上的椭圆曲线的点和加法运算构成一个有限 Abel 群 S。

以一个点 G 作为生成元，进行重复的加法运算，能够生成一个子群 S′。群 S′有可能与 S 相同，更有可能是 S 的一个真子群。

比如设椭圆曲线为：

$$y^2=x^3+x+1，p=23$$

以 G=(3, 10)生成的子群刚好等于 S，因为 28G=O，说明此时 S 还是一个有限循环群，群中每个元素都能由(3, 10)通过重复的加法运算得到，群中元素的个数为 28，其阶为 28(最小的使得 nG=O 成立的 n)，生成元为(3, 10)。

但是以 G=(6, 19)生成的子群是 S 的真子群，因为 14G=O14G=O，它也是一个有限循环群，因为群中每个元素都能由(6, 19)通过重复的加法运算得到，该子群的元素个数为 14，阶为 14（最小的使得 nG=O 的成立 n），生成元为(6, 19)。

拉格朗日定理告诉我们，生成的子群 S′的阶与原来的群 S 的阶存在约数关系，也就是说子群 S′的阶只能是 1, 2, 4, 7, 14, 28。群 S 的阶与子群 S′的阶的比值一般称为协因子(cofactor)：h=|S|/｜S′｜。一般会取 h=1，以保证生成子群的阶比较大。

以 G=(6, 19)生成子群过程，为方便计算，取 O=(−1, −1)：

| | |
|---|---|
| 1P=(6,19) | 15P=(6,19) |
| 2P=(13,16) | 16P=(13,16) |
| 3P=(7,11) | 17P=(7,11) |
| 4P=(5,19) | 18P=(5,19) |
| 5P=(12,4) | 19P=(12,4) |
| 6P=(17,20) | 20P=(17,20) |
| 7P=(4,0) | 21P=(4,0) |
| 8P=(17,3) | 22P=(17,3) |
| 9P=(12,19) | 23P=(12,19) |
| 10P=(5,4) | 24P=(5,4) |
| 11P=(7,12) | 25P=(7,12) |
| 12P=(13,7) | 26P=(13,7) |
| 13P=(6,4) | 27P=(6,4) |

14P=(-1,-1)　　28P=(-1,-1)

在后面的ECC加密算法过程中会有一个给定的基点G（也就是生成元）生成一个子群，然后秘钥空间在此子群取得，一般会要求保证子群的阶会尽量大，基点及其子群的阶n都是公开的信息。

**4.离散对数问题**

构造一个数学难题来保证加密信息的安全性是现代密码学中加密算法的主要思想。类似RSA算法中大数的质因子分解难题一样，椭圆曲线也有类似的数学难题。非对称加密方法根据其所依据的数学难题一般分为三类：大素数分解问题类、离散对数问题类、椭圆曲线类。有时也把椭圆曲线类归为离散对数类。

考虑Q=dP，其中Q，$P\in E_p(a,b)$，$d<p$。

对于给定的d和P，计算Q是很容易的；反过来给定Q和P，计算d是相当困难的，这就是椭圆曲线的离散对数问题。

正因为如此，可以将Q作为公钥，公开出去；d作为私钥，秘密保管，通过公钥来破解私钥十分困难。

## 7.7.3 秘钥协商

密钥协商主要是用来得到通信双方的临时会话密钥。比如依靠非对称加密算法RSA、ECC等，拿到对方公钥的一方随机产生会话密钥，利用公钥加密，并将加密后的信息传送给另一方。另一方再用手中的私钥对接受到的密文进行解密，这样通信双方都拿到会话密钥。现在我们来介绍一下使用ECC算法进行密钥协商过程。

**1.原理和流程**

利用模p有限域上椭圆曲线算术规则可以用来实现秘钥协商，假设有两个人A和B需要进行秘钥协商，那么其流程如下：

（1）A和B会共享一些椭圆曲线的参数信息：(p, a, b, G, n, h)，其中a, b确定了椭圆曲线方程，p确定了模p的有限域，G是中的生成元，用于生成子群，这里要求G的阶n应该尽量大，G的阶是使得nG=O成立的最小的整数，也即是(n−1)G所代表的点与G点的横坐标刚好相同，协因子h一般等于1。

（2）A选择小于n整数$n_A$，然后计算$P_A=n_A\times G$，将$P_A$发送给B。

（3）同理，B也选择小于n整数$n_B$，然后计算$P_B=n_B\times G$，将$P_B$发送给A。

（4）A通过如下计算得出协商的秘钥：$K_A=n_A\times P_B$；B通过如下计算得出协商的秘钥：$K_B=n_B\times P_B$，容易证明，$K_A=K_B$。

因为:

$$K_A=n_A\times P_B=n_A\times(n_B\times G)=n_B\times(n_A\times G)=n_B\times P_A=K_B$$

由此A和B计算得到相同的秘钥，达到秘钥协商的目的。

**2.实例**

例如，参数a=0, b=−4，椭圆曲线方程为$y^2=x^3-4$，p=211, G=(2, 2)，因为240G=(2, 209)，与G横坐标相同，所以n=241。

A选择$n_A$=151，$P_A$=151G=(62, 59);

B选择$n_B$=171，$P_B$=171G=(209, 153);

他们协商的秘钥 $K=151P_B=171P_A=151(209, 153)=171(62, 59)=(95, 194)=(151*171\%241)G=34G$。

### 7.7.4 椭圆曲线实现加解密

利用椭圆曲线来进行加密和解密与 RSA 算法有一定程度的类似，即每一个用户都有属于自己的公钥和私钥。私钥就是用户选定的数字 d，私钥自己保存；公钥就是由 P=dG，计算出来的点，公钥公开。

**1.原理和流程**

假设 A 与 B 进行加密通信，其中 B 手中的公钥私钥分别为： $P_B$ 为公钥，$d_B$ 为私钥且有 $P_B=d_BG$。那么其加密的流程如下：

（1）A 首先将明文消息转换（编码）为 Ep(a, b)中的 $P_m(x, y)$，然后随机选定一个正整数 k，并且利用 B 的公钥 $P_B$ 通过如下计算出密文：

$$C_m=\{kG, P_m+kP_B\}$$

因此，密文实际上是有两个点组成。

（2）B 收到密文 $C_m$，利用自己的私钥 $d_B$ 进行如下计算，可以解密得到明文 $P_m$：

$$P_m+kP_B-d_B(kG)=P_m+k(d_BG)-d_B(kG)=P_m \quad ，其中\ P_B=d_BG$$

也就是用第二个点 $P_m+kP_B$ 减去第一个点 kG 与自己的私钥 $d_B$ 之积。

**2.实例**

考虑参数 a=0, b=−4, p=199, G=(2, 2)，椭圆曲线方程为 $y^2=x^3-4$。

B 选定的私钥为 $d_B=119$，其公钥 $P_B=119G=(183, 173)$。

A 希望将消息 $P_m=(76, 66)$加密后发送给 B，于是 A 随机选定正整数 k=133，并通过 B 的公钥加密得到密文：

$$C_m=\{133(2, 2), (76, 66)+133(183, 173)\}=\{(40, 147), (180, 163)\}$$

B 收到密文消息利用自己的私钥 $n_B=119$ 进行解密：

$$P_m+kP_B-d_B(kG)=(180, 163)-119(40, 147)=(180, 163)-(98, 52)=(76, 66)=P_m$$

由此，B 顺利解密得到明文消息，A 与 B 之间完成了加密通信。

## 7.7.5 椭圆曲线在软件注册保护的应用

将公开密钥算法作为软件注册算法的好处是软件破解者（Cracker）很难通过跟踪验证算法得到注册机。下面，我们来介绍一种利用 $F_p(a, b)$椭圆曲线进行软件注册的方法。

软件作者按如下步骤和方法制作注册机（也可称为签名过程）：

(1)选择一条椭圆曲线 $E_p(a,b)$，和基点 G；

(2)选择私有密钥 d(d<n, n 为 G 的阶)，公开密钥 P=dG；

(3)产生一个随机整数 k(k<n)，计算点 R=kG，R 的坐标为$(x_r, y_r)$；

(4)计算 $r=x_r \bmod n$，如果 r=0，转到步骤 3 重复操作；

(5)计算待签名的消息 $m_0$ 的 hash 值 $e=H(m_0)$，计算方法为 SHA（Secure Hash Algorithm 安全散列算法，类似于 MD5）；

(6)计算 $s=k^{-1}(e+dr) \bmod n$，如果 s 为 0，则转到步骤 3 重复操作；

(7)将(r, s)作为消息 $m_0$ 的签名。

软件验证就是要判断消息是否已经被篡改。其验证过程如下（软件中存有椭圆曲线 $E_p(a, b)$，和基点 G，公开密钥 P）：

(1)签名(r, s)，和待验证的消息 $m_1$；假设消息没有被篡改即 $m_1=m_0$，那么 hash 值应该是不变的，即 $e=H(m_1)=H(m_0)$。

(2)计算:

$$Q=s^{-1}(e\,G+rP)$$

设 Q 的横坐标为 $x_q$， $q=x_q \bmod n$，那么如果 q=r，则签名有效，否则签名无效。

这是因为，根据签名过程有：

$$s\equiv k^{-1}(e+dr)\,(\bmod n)$$

根据同余性质有：

$$s^{-1}\equiv k(e+dr)^{-1}\,(\bmod n)$$

同时因为：

$$P=dG$$

把 $s^{-1}$ 和 P 代入 $Q=s^{-1}(e\,G+rP)$，有：

$$Q=s^{-1}(e\,G+rP)= k(e+dr)^{-1}(eG+rdG) =k(e+dr)^{-1}\,(e+dr)G = kG=R$$

其中 R 是签名过程中生成的点，因此，q 必然等于 r，否则签名无效。

破解者要想制作注册机，只能通过软件中的 $E_p(a, b)$，点 G，公开密钥 P ，并利用 P=dG 这个关系获得 d 后才可以，而求 d 是很困难的，这样就保证了算法和数据的可靠性。如表 7-1 所示，不同的加密算法和密钥强度对应的破解时间:

| RSA密钥强度 | 椭圆曲线密钥强度 | 攻破时间(年) |
|---|---|---|
| 512 | 106 | 104,已被攻破 |
| 768 | 132 | 108,已被攻破 |
| 1024 | 160 | 1011 |
| 2048 | 210 | 1020 |

表 7-1 加密算法强度对比

中国国家密码管理局于 2010 年 12 月 17 日发布了 SM2 椭圆曲线公钥密码算法，并要求对现有基于 RSA 算法的电子认证系统、密钥管理系统、应用系统进行升级改造。RSA 算法的危机在于其存在基于量子计算的亚指数算法（比如 Shor 算法），对 ECC 算法而言一般没有亚指数攻击算法。基于椭圆曲线离散对数问题，计算复杂度是指数级，求解难度较大，同等安全程度要求下，椭圆曲线密码较其他公钥算法所需密钥长度小很多。

ECC 算法的优势依然超过被广泛接受的传统 RSA 算法。随着量子计算技术的突破，许多专家担忧 RSA 和 Diffie-Hellman 背后的算法可能在五年以内被攻破。随着时间推移，ECC 可能是以后唯一选择。

## 7.7.6 OpenSSL 中的 ECC 加密算法

ECC 本身并没有真正定义任何加密和解密操作。OpenSSL 实现的 ECC 算法，包括三部分： ECC 算法(crypto/ec)、椭圆曲线数字签名算法 ECDSA (crypto/ecdsa)以及椭圆曲线密钥

交换算法 ECDH(crypto/dh)。

在 OpenSSL 开源库中，密钥数据结构定义在 openssl-1.1.0c\crypto\ec\ec_lcl.h 文件中。

```
struct ec_key_st {
    const EC_KEY_METHOD *meth;
    ENGINE *engine;
    int version;
    EC_GROUP *group; //密钥参数
    EC_POINT *pub_key;
    BIGNUM *priv_key;
    unsigned int enc_flag;
    point_conversion_form_t conv_form;
    int references;
    int flags;
    CRYPTO_EX_DATA ex_data;
    CRYPTO_RWLOCK *lock;
};
```

Openssl 中，椭圆曲线密钥生成时，首先用户需要选取一种椭圆曲线(openssl 的 crypto/ec/ec_curve.c 中内置实现了 67 种，调用 EC_get_builtin_curves 获取该列表)，然后根据选择的椭圆曲线计算密钥生成参数 group，最后根据密钥参数 group 来生公私钥。

# 八，布尔代数基础与程序应用

布尔代数是一种处理二值变量的逻辑数学，在形式逻辑中，二值表达为真（true）与假（false），在数字系统中二值表达为“开和关”，“高和低”，“1 和 0”。

1847 英国数学家乔治·布尔（George Boole，1815-1864，如图 8-1 所示）发表著作《The Mathematical Analysis of Logic》，建立布尔代数（也被称为逻辑代数）。布尔代数的诞生，为计算机的二进制数、开关逻辑元件和逻辑电路的设计铺平了道路，并最终为计算机的发明奠定了数学基础。

## 8.1 布尔代数简介

早在 17 世纪，微积分共同发明人莱布尼茨就想要发明一种通用的语言，借助它的符号和专门语法来指导推理。他认为逻辑语言应该用一些表意的符号，每一个符号代表一个简单的概念，通过各种符号的组合表达复杂的思想。他也认真地考虑过建立一种推理的代数，试图通过演算完成一切正确的推理过程。但他并没有完成这项工作。直到 2 个世纪之后，才由英国数学家乔治.布尔获得实际性进展。

乔治·布尔是皮匠的儿子，1815 年 11 月生于英格兰的林肯。由于家境贫寒，布尔不得不在养家的同时为自己能受教育而奋斗，16 岁就开始任教以维持生活，20 岁时对数学产生了浓厚的兴趣，开始广泛涉猎著名数学家牛顿、拉普拉斯、拉格朗日等人的数学名著，并写下了大量笔记。后来他发现人的逻辑思维可以用数学表达式进行表达。

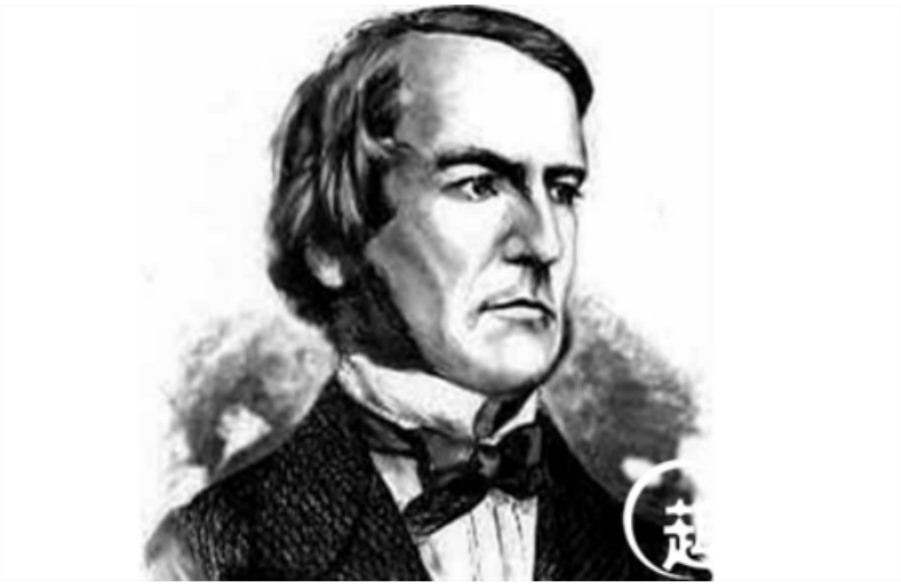


图 8-1 布尔

两千年来，哲学书都是用文字写的。比如，最著名的三段论：

- 所有人都是要死的；
- 苏格拉底是人；
- 所以，苏格拉底是要死的。

乔治·布尔认为，这种推理可以用数学表达，也就是说，哲学书完全可以用数学改写。这就是数理逻辑的起源。

1847 年，他发表了著作《The Mathematical Analysis of Logic》，在这本书中，阐述了正式的逻辑学公理。他的逻辑理论建立在两个逻辑值 0、1 和三个运算符与（ ∧ ）、或（ ∨ ）、

非（ ¬ ）的基础上。

1849 年，他被任命担任爱尔兰科克的皇后学院的数学教授。1854 年，他出版了《The Laws of Thought》，这是他最著名的著作。在这本书中布尔介绍了现在以他的名字命名的布尔代数。布尔撰写了微分方程和差分方程的课本，这些课本在英国一直使用到 19 世纪末。布尔在 1855 年结婚，他的妻子是皇后学院一位希腊文教授的侄女。1864 年，布尔在暴风雨天气中被淋湿了，但他仍坚持上课，从而引起肺炎，并最终死于肺炎。

由于布尔在符号逻辑运算中的特殊贡献，很多计算机语言中将逻辑运算称为布尔运算，并将其运算结果称为布尔值。

下面我们就正式开始学习布尔代数的基础理论知识。

## 8.2 命题的定义

在命题逻辑中，其基本成分是具有真、假意义(称为真值，也就是 0 和 1，True 和 False)的一个个陈述句，称作**命题**。只有具有或必将具有确定真值的陈述句才是命题。一切没有判断内容的句子，无所谓是非的句子都不能作为命题。

命题分为**原子命题**、**复合命题**两种类型。前者是简单陈述句，后者至少是对原子命题的否定或是几个原子命题组成的复合命题。

在命题逻辑中，规定用大写英文字母 P，Q，R，…以及 $P_1$，$P_2$，$Q_1$，$Q_2$，…等符号表示命题，但不使用 T 和 F 表示一般的命题。**T** (True 的第一个字母）和 **F**（False 的第一个字母）专门用来表示永真和永假的命题及取真(T)和取假(F)的指定，其含义即逻辑真值 1 和 0。

表示命题的符号称为命题符号。命题符号表示一个确定的命题，其真值非 T 即 F，被称为命题常量。命题符号表示任意命题，无确定真值，则称为命题变元。命题变元不是命题，以{0, 1}为变域（即变元的取值范围）。

练习 1：下面这些语句是命题吗？

（1）你去上课吗？

（2）北京是中国的首都

（3）1+101=110

（4）月球上也有生物

（5）请关掉电视！

（6）我正在说谎。

解答：

（1）是疑问句，不是命题。

（2）是有确定真值“真”的陈述句，是真命题。

（3）在二进制中是真命题，在十进制中是假命题。

（4）是必将具有确定真值的陈述句，因而是命题。

（5）是祈使句，不是命题。

（6）不是命题，而是悖论。若记此句为 A，则 “A 为真当且仅当 A 为 假”，这是矛盾；像这样形式的矛盾句称为悖论。

## 8.3 命题演算推理

命题在演算推理过程之中，需要用到 5 个逻辑联词，分别是“否定，符号为¬”，“合

取，符号为¬∧”，“析取，符号为¬∨”，“条件，符号为¬→”，和“双条件，符号为¬↔”。通过这 5 个逻辑联词，可以得到最简单的 5 种复合命题形式。下面我们先介绍这 5 个逻辑联词的真值运算规则。

**否定¬**

命题 P 的否定是命题 ¬P，读作非 P，  ¬0=1，  ¬1=0。

| $P$ | $\neg P$ |
|---|---|
| T | F |
| F | T |

**合取∧**

命题 P 与 Q 的合取是命题 P∧Q，读作 **P 与 Q，1 ∧1=1, 0 ∧1=0, 1 ∧0=0, 0 ∧0=0。**

| $P$ | $Q$ | $P \wedge Q$ |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | F |
| F | F | F |

**析取∨**

命题 P 与 Q 的析取是命题 P∨Q，读作 **P 或 Q，0 ∨0=0, 1 ∨0=1, 0 ∨1=1, 1 ∨1=1。**

| $P$ | $Q$ | $P \vee Q$ |
|---|---|---|
| T | T | T |
| T | F | T |
| F | T | T |
| F | F | F |

**范式**

我们把命题变元及其否定统称作文字，仅由有限个文字构成的析取式称为简单析取式；仅由有限个文字构成的合取式称为简单合取式。一个文字既是简单析取式，又是简单合取式。

设有命题变元 p，q，r，则简单析取式的例子如下：

$$p, q, p \vee q, p \vee \neg p \vee r, \neg p \vee q \vee \neg r$$

简单合取式的例子如下：

$$p, \neg r, \neg p \wedge r, \neg p \wedge q \wedge r, p \wedge q \wedge \neg q$$

由有限个简单合取式构成的析取式称为析取范式。由有限个简单析取式构成的合取式称为合取范式。析取范式与合取范式统称为范式。

析取范式的例子如下：

$$(\neg p \wedge q) \vee r, \neg p \vee q \vee r, p \vee \neg q \vee r$$

合取范式的例子如下：

$(p \vee q \vee r) \wedge (\neg q \vee r)$ , $\neg p \wedge q \wedge r, p \wedge \neg q \wedge r$

**条件→**

命题 P 与 Q 组成条件命题 P→Q，读作**若 P 则 Q**。 P 称前件，Q 称后件。

| P | Q | P→Q |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | T |
| F | F | T |

**双条件 ↔**

命题 P 与 Q 组成双条件命题 P↔Q，读作 **P 当且仅当 Q**。P↔Q 取真，当且仅当 P 与 Q 取相同的真值即同取真值真或同取真值假。

| P | Q | P↔Q |
|---|---|---|
| T | T | T |
| T | F | F |
| F | T | F |
| F | F | T |

从这 5 种复合命题形式出发，再经过逻辑联词各种相互组合，就可以构成更多复杂的复合命题形式，我们称之为命题公式。

**1.真值表**

真值永为真的命题公式称永真式，记为 T；真值永为假的命题公式称永假式，记为 F。对于它们所含命题变元的全部真值指派，对于每一指派都有相同的真值取值，这样的命题公式称作逻辑等值式或等值公式。命题公式 A 与 B 等值记为：A=B。命题公式满足如下运算规律：

(1) 对合律 ¬(¬P)=P

(2) 幂等律 P∧P=P；P∨P=P

(3) 结合律 (P∧Q)∧R=P∧(Q∧R)；(P∨Q)∨R=P∨(Q∨R)

(4) 交换律 P∧Q=Q∧P；P∨Q=Q∨P

(5) 分配律 P∧(Q∨R)=(P∧Q)∨(P∧R)；P∨(Q∧R)=(P∨Q)∧(P∨R)

(6) 吸收律 P∧(P∨Q)=P；P∨(P∧Q)=P

(7) 反演律 ¬(P∧Q)= ¬P∨ ¬Q；¬(P∨Q)= ¬P∧ ¬Q

(8) 壹律 P∧T=P；P∨F=P

(9) 零律 P∧F=F；P∨T=T

(10) 矛盾律 ¬(P∧ ¬P)=T 一个命题不可能既是真又是假

(11) 排中律 P∨¬P=T 一个命题要么是真要么是假

(12) 联词转化律 P→Q= ¬P∨Q；P↔ Q=(P→Q)∧(Q→P)

等价式等号左边和右边的真值相同，可用于命题的逻辑推理。

命题中的永真公式称为重(读 chong2)言式，永假公式称为矛盾式，不是矛盾式的命题公式称可满足式。A→B 是重言式，当且仅当 A⇒ B。这里 A⇒ B 读作 A 蕴涵 B。A 蕴涵 B，当且仅当 A→B 是重言式，即当且仅当 A 取真值 T 时，B 必取真值 T；或者当且仅当 B 取真值 F 时，A 必取真值 F。

一个正确的逻辑推理形式，由真的前提出发必能得出真的结论，其前提与结论之间的逻辑形式正是蕴涵式。如果推理形式是正确的，那么由推理的前提和结论构成的条件公式必定是一个重言式。检查一个推理的推理形式是否正确，只须写出相应于推理的条件命题公式，判定这个条件命题公式是否重言式即可。

下面 9 个基本蕴涵式，反映了命题逻辑推理的形式结构，体现了基本的逻辑规律，总结了人们思维推理的一些常用方法，并且是证明其他推理形式的基础和依据。

(1) 化简式：$P \wedge Q \Rightarrow P$；$P \wedge Q \Rightarrow Q$。

(2) 附加式：$P \Rightarrow P \vee Q$；$Q \Rightarrow P \vee Q$。

(3) 假言推理：$P \wedge (P \rightarrow Q) \Rightarrow Q$。

(4) 拒取式：$\neg Q \wedge (P \rightarrow Q) \Rightarrow \neg P$。

(5) 析取三段论：$\neg P \wedge (P \vee Q) \Rightarrow Q$。

(6) 假言三段论：$(P \rightarrow Q) \wedge (Q \rightarrow R) \Rightarrow P \rightarrow R$。

(7) 二难推论：$(P \vee Q) \wedge (P \rightarrow R) \wedge (Q \rightarrow S) \Rightarrow R \vee S$。

(8) 等价三段论：$(P \leftrightarrow Q) \wedge (Q \leftrightarrow R) \Rightarrow (P \leftrightarrow R)$。

(9) 合成式：$P$，$Q \Rightarrow P \wedge Q$ (此式的逻辑含义是：有前提 $P$，$Q$ 可得结论 $P \wedge Q$)

蕴涵的重要性质有：

$A \Rightarrow B$ 且 $A \Rightarrow C$，那么 $A \Rightarrow B \wedge C$

$A \Rightarrow C$ 且 $B \Rightarrow C$ 那么 $A \vee B \Rightarrow C$

**2.命题演算推理**

构造一个推理的过程实际上就是一个演绎过程。设 $S=\{H_1, H_2, \ldots, H_n\}$是作为前提的命题公式集合，C 是要证结论。从 S 推出 C 的一个演绎是命题公式的一个有限序列：$G_1$，$G_2$，…，$G_k$，其中 $G_i$或者属于 S，或者是在它之前的某些 $G_j$ $(j<i)$的逻辑结果并且 $G_k=C$。按照这一定义，在任何的推理步骤上可使用如下规则：

P 规则：前提

T 规则：引用先前步骤中得出的某些命题公式的逻辑结果，对是否是逻辑结果的判定，特别要使用基本蕴涵式和基本等价式，它们是我们已经证明过的事实，是使用 T 规则的依据。

CP 规则：如果需要论证的结论是 P→Q 的形式，则可将 P 作为附加前提使用

练习 2， 试证明下列结论。

(1)$P \vee Q$，$Q \rightarrow \neg R$，$P \rightarrow \neg R$，$(P_1 \vee P_2) \rightarrow (R \wedge S) \Rightarrow \neg P_1$

证明：

$P \vee Q$

$Q \rightarrow \neg R$

$P \rightarrow \neg R$

$\neg R$

$(P_1 \vee P_2) \rightarrow (R \wedge S)$
$\neg(P_1 \vee P_2) \vee (R \wedge S)$
$\neg(P_1 \vee P_2)$
$\neg P_1 \wedge \neg P_2$
$\neg P_1$

(2)$P \rightarrow (Q \rightarrow R)$, $\neg S \vee P$, $Q \Rightarrow S \rightarrow R$
证明:
$S \rightarrow P$
$P \rightarrow (Q \rightarrow R)$
$S \rightarrow (Q \rightarrow R)$
$Q$
$S \rightarrow R$

(3)$A \rightarrow B$, $\neg(B \vee C) \Rightarrow \neg A$
证明:
$A \rightarrow B$
$\neg A \vee B$
$\neg(B \vee C)$
$\neg B \wedge \neg C$
$\neg A$

再来看一个古代有关逻辑学家的故事。相传一逻辑学家误入一酋长国，被拘一屋。酋长对逻辑学家说：“所拘你屋有两门，一生一死；选择生门你可获得自由， 选择死门你便遭屠戮。我在每一门都派有一位守门人，他们中一人只讲真话，一人只讲假话。看你并不想死，那生的希望就由你自己确定吧。”

逻辑学家听后沉思片刻，从容地对身旁一门的守门人发问，并选得生门获得自由。逻辑学家对身旁的守门人问了什么?

逻辑学家手指身旁的门并问该门的守门人：“这门是死亡门，那个门的守门人将回答‘是’，对吗?”

设有如下命题：
$P$ 为“被问守门人只讲真话”;
$Q$ 为“另一守门人的回答是‘是’”;
$R$ 为“被问守门人的回答是‘对’”;
$S$ 为“所指之门是死亡门”。
那么可以构造出如下真值表：

| $P$ | $Q$ | $R$ | $S$ | 结论 |
|---|---|---|---|---|
| 1 | 1 | 1 | 0 | 生门 |
| 0 | 1 | 0 | 1 | 死门 |
| 1 | 0 | 0 | 1 | 死门 |
| 0 | 0 | 1 | 0 | 生门 |

所以，当被问守门人的回答是“对”，所指之门即为生门；如果被问守门人的回答是“否”，那么另一个门就是生门。

命题逻辑不能论证所有问题。比如“苏格拉底有死”的问题：凡人皆有死($P$)，苏格拉底是人($Q$)，所以苏格拉底有死($R$)。

前提 $H_1$：P

前提 $H_2$：Q

结论 C：R?

如果 $P \wedge Q \Longrightarrow R$，那么 $P \wedge Q \rightarrow R$ 是重言式，但 $P \wedge Q \rightarrow R$ 不是重言式，所以 $P$，$Q$ *不能蕴涵* $R$。这是一个显然意见的真命题，但是命题逻辑却无法给予证明。这说明了什么呢？

事实上，命题是有真假意义之一的陈述句，它作为论证的基础，只能构筑人们思维论证的一个方面。而像“苏格拉底有死”的论证，它已无力完成。原因在于这种论证的有效性并不简单依赖于作为前提的命题 $P$、$Q$ 与作为结论的命题 $R$ 之间的逻辑关系，而更依赖于这些命题作为陈述句所包含的自身组成部分之间的关系。进一步说，依赖于论证的前提和结论的内部结构。因此需要深入到陈述句内部分析。

也就是我们接着讲的“一阶谓词逻辑”可以解决此问题。

## 8.4 一阶谓词个体，谓词

命题是具有真假之一值的陈述句，主要由**主词**和**谓词**两部分组成。主词是主体，是陈述句要加以表述的事物或对象，在陈述句中是一些独立的成分，在谓词逻辑中称之为**个体**。个体可能是具体的，也可能是抽象的。刻画主词(个体)属性或揭示主词具有什么关系的词称谓词。谓词由短语和反映属性或关系的词项构成。个体是可以独立存在的对象。由个体组成的集合称**个体域**。由所有可称之为个体的对象聚合成的个体域称为**全总个体域**。如果对个体域未作说明，一律使用全总个体域。以某一个体域中的个体为变域的变元称为该个体域上的个体变元，在不致引起混淆的情况下，一般直称为个体变元。谓词表述个体、命题所具有的性质，或者几个个体、命题之间的关系。

为了使用方便，特定谓词用大写字母 A，B，C，…，$A_1$，$A_2$，…表示，谓词变元用 X，Y，Z，…，$X_1$，$X_2$，…表示；

小写字母 a，b，c，…，$a_1$，$a_2$，…表示个体常量，小写字母 x，y，z，…，$x_1$，$x_2$，…表示个体变元。

比如，我们可以将下列语句表示为一阶谓词逻辑式。

(1)中国和美国之间隔着太平洋

主体：c:中国，u：美国，p：太平洋

谓词：A（$x_1$, $x_2$, $x_3$）:$x_1$ 与 $x_2$ 之间隔着 $x_3$

A(c, u, p)

(2)如果你不吃饭，　身体就会变瘦

主体：a:你，b：身体

谓词：E(x)：x 吃饭

B($x_1$, $x_2$):$x_2$ 属于 $x_1$

D(x)：x 会变瘦

$\neg E(a) \wedge B(a, b) \rightarrow D(b)$

在上面的例子中，我就可以发现，一阶谓词已经进入命题的内部，并对其内部结构进行分析了。

**全称量词∀和存在量词∃**

在命题形式“所有的 a 都是 B”中，除了主词“a”和谓词“都是 B”之外，还有一个描述主词范围的词“所有的”。

在一阶谓词逻辑中，把主词指定为“整体”的词，比如“所有的”、“一切”、“每一个”、“任意的”等，我们称之为**全称量词，记为“ ∀ ”**。相应地，把主词指定为“部分”的词，比如“存在”、“有”、“对于一些”、“至少有”等，称之为**存在量词，记为“ ∃”**。

全称量词与存在量词统称为量词。量词是逻辑学中表示全称和存在的词，在一阶谓词逻辑中量词刻画谓词中个体的一般性与个别性。

比如：

(1)，试将“有人晚上失眠”符号化为谓词表达式。

令 M(x)表示：x 是人；F(x)表示：x 在晚上失眠。则“有人晚上失眠”可符号化为：

$$(\exists x)(M(x)\wedge F(x))$$

(2)，试将“每个四川人都爱吃辣”符号化为谓词表达式。

令 S(x)表示：x 是四川人；L(x)表示：x 爱吃辣。则原命题

可被符号化为：

$$(\forall x)(S(x)\rightarrow L(x))$$

(3)，试将“凡人皆有死，苏格拉底是人，所以苏格拉底有死” 符号化为谓词表达式。

令 M（x）：x 是人；D（x）：x 会死，s：苏格拉底。

P：$(\forall x)(M(x)\rightarrow D(x))$;

Q：M(s);

R：D(s)。

## 8.5 一阶谓词演算的推理

在一阶谓词演算推理里，有一些特有的等值式，可以用于我们的推理。现在我们把这些等值式列出如下：

- 在有限个体域D={a1,a2,...,an}中消除量词：

$(\forall x)A(x)=A(a1)\wedge A(a2)\wedge\ldots\wedge A(an)$

$(\exists x)A(x)=A(a1)\vee A(a2)\vee\ldots\vee A(an)$

- 量词否定等值式：

$\neg(\forall xA(x))=\exists x(\neg A(x))$

$\neg(\exists xA(x))=\forall x(\neg A(x))$

- 量词辖域的收缩与扩张等值式（变元x 不在B 中出现）：

$\forall x(A(x)\vee B)=(\forall xA(x))\vee B$

$\forall x(A(x)\wedge B)=(\forall xA(x))\wedge B$

$\forall x(A(x)\rightarrow B)=(\exists xA(x))\rightarrow B$

$\forall x(B\rightarrow A(x))=B\rightarrow(\forall xA(x))$

$\exists x(A(x)\vee B)=(\exists xA(x))\vee B$

$\exists x(A(x)\wedge B)=(\exists xA(x))\wedge B$

$\exists x(A(x)\rightarrow B)=(\forall xA(x))\rightarrow B$

$\exists x(B\rightarrow A(x))=B\rightarrow(\exists xA(x))$

- 量词分配等值式：

$\forall x(A(x)\wedge B(x))=(\forall xA(x))\wedge(\forall xB(x))$

$\exists x(A(x)\vee B(x))=(\exists xA(x))\vee(\exists xB(x))$

- 量词顺序变换等值式：

$\forall x\forall y(A(x,y))=\forall y\forall x(A(x,y))$

$\exists x\exists y(A(x,y))=\exists y\exists x(A(x,y))$

- 一阶逻辑中特有的推理定律：

$(\forall xA(x))\vee(\forall xB(x))\rightarrow\forall x(A(x)\vee B(x))$

$\exists x(A(x)\vee B(x))\rightarrow(\exists xA(x))\vee(\exists xB(x))$

$\forall x(A(x)\rightarrow B(x))\rightarrow(\forall xA(x))\rightarrow(\forall xB(x))$

$\forall x(A(x)\rightarrow B(x))\rightarrow(\exists xA(x))\rightarrow(\exists xB(x))$

**一阶谓词演算的推理规则**

为了消除与引入量词，一阶谓词的形式演算，在命题演算推理规则（P, T, CP）的基础上，增加下述四个规则：

(1) 全称指定——**US 规则**：$(\forall x)P(x)\Rightarrow P(t)$，其中个体符号 t 普适，消去量词。

(2) 全称推广——**UG 规则**：$P(t)\Rightarrow(\forall x)P(x)$，其中变元符号 t 普适。

(3) 存在指定——**ES 规则**：$(\exists x)P(x)\Rightarrow P(a)$，其中个体 a 专指，消去量词。

(4) 存在推广——**EG 规则**：$P(a)\Rightarrow(\exists x)P(x)$，其中个体 a 专指。

其中 U, E, G, S 代表的含义为 U：universal；E：exist；G：Generalize；S：specify

有了一阶谓词逻辑之后，那么苏格拉底有死的问题现在就可以证明了。现在我们来看看如何应用一阶谓词逻辑来证明这个问题。

练习 3：试论证“凡人皆有死，苏格拉底是人，所以苏格拉底有死”。

设：

M（x）：x 是人；

D（x）：x 会死，

s：苏格拉底。

前提：$(\forall x)(M(x)\rightarrow D(x))$，$M(s)$。

结论：$D(s)$

证明：

$G_1$：$(\forall x)(M(x)\rightarrow D(x))$ P 规则，前提引入

$G_2$：$M(s)\rightarrow D(s)$ US 规则，消去量词$(\forall x)$

$G_3$：$M(s)$ P 规则

$G_4$：$D(s)$ T 规则，对 $G_2$，$G_3$ 假言推理

练习 4：试论证:$(\forall x)(A(x)\vee B(x))$，$\neg(\exists x)B(x)\Rightarrow(\forall x)A(x)$。

证明：

| | |
|---|---|
| $\neg(\exists x)B(x)$ | P 规则 |
| $\forall x\neg B(x)$ | T 规则 |
| $(\forall x)(A(x)\vee B(x))$ | P 规则 |
| $\forall x\neg B(x)\wedge(\forall x)(A(x)\vee B(x))$ | |
| $\forall x((A(x)\vee B(x))\wedge\neg B(x))$ | |
| $\forall x(A(x)\wedge\neg B(x))$ | |
| $\forall xA(x)\wedge\forall x\neg B(x)$ | |
| $\forall xA(x)$ | |

练习 5：每一个大学生不是文科生就是理科生；有的大学生是优等生；小张不是文科生但他是优等生。因此，如果小张是大学生，他就是理科生。

分析：

首先分析对应的谓词：

P(x):x 是一个大学生；

Q(x):x 是文科生；

S(x):x 是理科生；

T(x):x 是优等生；

个体常项小张：c

前提：

$\forall x(P(x)\rightarrow(Q(x)\vee S(x)))$

$\exists x(P(x)\wedge T(x))$

$\neg Q(c)\wedge T(c)$

结论：$P(c)\rightarrow S(c)$

证明：

| | |
|---|---|
| $\forall x(P(x)\rightarrow(Q(x)\vee S(x)))$ | P |
| $P(c)\rightarrow(Q(c)\vee S(c))$ | US |
| $P(c)$ | CP |
| $Q(c)\vee S(c)$ | (2)(3)T |
| $\neg Q(c)\wedge T(c)$ | P |
| $\neg Q(c)$ | (5) |
| $S(c)$ | (4)(6)T |

练习 4：每个旅客或者坐头等舱或者坐二等舱；每个旅客当且仅当他富裕时坐头等舱；有些旅客富裕但并非所有的旅客都富裕。因此，有些旅客坐二等舱。

分析：

首先给出相关的谓词：

P(x):x 是旅客；

Q(x):x 坐头等舱；

R(x):x 坐二等舱；

S(x):x 是富裕的。

前提：

$\forall x(P(x)\rightarrow(Q(x)\vee R(x)))$

$\forall x(P(x)\rightarrow(Q(x)\leftrightarrow S(x)))$

$\exists x(P(x)\wedge S(x))$

$\neg(\forall x(P(x)\rightarrow S(x)))$

结论：

$\exists x（P(x)\wedge R(x)）$

现在来证明：

| | |
|---|---|
| $\neg(\forall x(p(x)\rightarrow S(x)))$ | P |
| $\forall x(P(x)\wedge\neg S(x))$ | (1)E |
| $P(c)\wedge\neg S(c)$ | US |
| $P(c)$ | (3)T |
| $\neg S(c)$ | (3)T |

| | |
|---|---|
| ∀x(P(x)→(Q(x)∨R(x))) | P |
| P(c)→(Q(c)∨R(c)) | US |
| Q(c)∨R(c) ( | 4)(7)T |
| ∀x(P(x)→(Q(x) ↔S(x))) | P |
| (P(c)→(Q(c) ↔S(c))) | US |
| Q(c)<->S(c) | (4)(11)T |
| Q(c)→S(c) | (11)T |
| ¬Q(c) | (12)(5)T |
| R(c) | (13)(8)T |
| P(c)∧R(c) | (4)(14) |
| ∃x(P(x)∧R(x) ) | EG |

一阶逻辑是数理逻辑的基础部分，主要包括经典命题逻辑和一阶谓词逻辑。一阶谓词逻辑包含了命题逻辑。一阶逻辑之所以是“一阶”的，是因为它所包含的谓词逻辑是一阶的。谓词就是表示对象属性的语词。对象的属性具有层次，在谓词用法中，这种层次叫做“阶”。所谓一阶谓词就是指刻画个体属性的谓词，如“红色”、“高于”等谓词都只适用于个体概念，像“鲜艳”等用来刻画“红色”这种谓词的谓词就是高阶谓词了，它们刻画的是属性的属性。

一阶逻辑足够强大，它可以形式化全部的集合论和几乎所有的数学。把量化限制于个体使它难于用于拓扑学，但它是在数学底层经典的逻辑理论，是比命题逻辑强比二阶逻辑弱的理论。

## 8.5 计算机与程序中的布尔代数

布尔代数对于计算机和程序设计都至关重要，可以说布尔代数是现代计算机的基本运行方式，它为电子计算机最终的诞生创造了理论基础。

### 8.5.1 电子计算机与布尔代数

信息论创始人香农（Claude Elwood Shannon ， 1916-2001，如图 8-2 所示）的硕士论文《继电器与开关电路的符号分析》就是数字电路的理论基础，在论文里他用布尔代数优化开关电路，里面的符号分析完全用了布尔代数体系。他把布尔代数的真假和电路系统中的开关对应起来，并且用 1 和 0 表示。

图 8-2 信息论创始人香农

香农用二进制系统表达布尔代数中的逻辑关系，用“1”代表“True”，用“0”代表“False”，并由此用二进制系统来构筑逻辑运算系统。对于电子计算机，以布尔代数为基础，任何一个机械性推理过程，都能像处理普通计算一样容易。香农把布尔代数与计算机二进制联系在了一起。

数字电路的结构通常都可以化简为布尔代数的表达式。基于逻辑输入变量，产生逻辑输出结果的电路称为逻辑门电路。如图 8-3 所示，三种基本的门电路“与”门，“或”门和“非”门（实际上，通过“与非”门和“或非”门，可以实现任何门电路）直接对应着“与”，“或”，“非”的布尔操作。

图 8-3 三种基本门电路

正是通过这些开关门电路，最终搭建出了电子计算机。而计算机就是依赖于布尔代数来构建了一些复杂的模块，例如 ALU（算术逻辑单元，arithmetic and logic unit），它就是将一切的加、减、乘、除、乘方、开方等最终转化为布尔运算。

## 8.5.2 程序设计与布尔代数

在程序设计中，布尔代数的计算方法更是无处不在。程序运行时对执行流程的控制和判断，完全依赖于布尔代数中的真值 True 和 False。

为此，每个程序设计语言中都会提供与此相关的逻辑运算和位运算。以 C 语言为例，关于逻辑运算的运算符有：

逻辑与：&&

逻辑或：||

逻辑非：！

程序执行语句主要分为 3 类：顺序执行语句，分支语句和循环语句。如图 8-4 所示，除了顺序执行语句，分支语句和循环语句都需要通过布尔真值 True 和 False 来决定语句的执行情况。

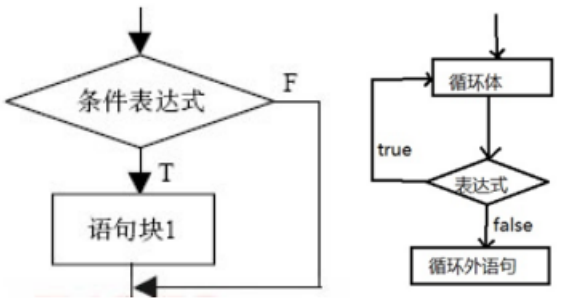



图 8-4 分支语句与循环语句

比如：

```
if (a > 0){ //a>0 为 True 的时候，执行 printf 语句，否则不执行
    printf("a > 0\n");
}
while(i>0){//i>0 为 True 的时候，执行 printf 语句，否则不执行
    printf("%d\n", i);
```

```
        i--;
    }
```

在搜索引擎或者广告行业，布尔运算也发挥着很大的应用。比如在广告行业里，我们经常会需要根据用户的特征（比如性别和年龄等，，比如化妆品的广告一般就要求投放给 25 岁到 55 岁之间的女性）去查找适合投放的广告。

查找的时候有 2 种方法。

第一种方法就是直接去广告数据库里遍历每一个广告，查看每个广告对用户的性别年龄要求。然后把这些包含关键字或者符合用户特征的广告找出来。如果广告数量庞大，那么遍历广告数据库找到符合年龄和性别要求的广告。这一种方法会相当耗时，效率低下，所以在实际应用中是不可取的。

第二种方法则是以性别和年龄为关键字，建立广告的倒排索引表。在倒排索引表中，将所有符合该投放特征的广告置 1，如表 8-1 所示：

| | ad1 | ad2 | ad3 | ad4 | ad5 | ad6 | ad7 |
|---|---|---|---|---|---|---|---|
| 性别-无 | 1 | 0 | 0 | 1 | 1 | 0 | 1 |
| 性别-女 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 性别-男 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |
| 年龄-无 | 0 | 0 | 1 | 0 | 0 | 1 | 0 |
| 年龄-0~18 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 年龄-19~30 | 1 | 1 | 1 | 1 | 0 | 0 | 1 |
| 年龄-31~40 | 1 | 0 | 0 | 0 | 1 | 0 | 1 |
| 年龄-41~56 | 0 | 1 | 0 | 1 | 0 | 0 | 0 |
| 年龄-56~100 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

表 8-1 广告倒排索引表

表中第一列为广告特征分类，第二列起，每列的取值对应了该广告对各特征的要求，即该广告如果满足该特征的投放要求，则置 1，否则置 0。比如，对应广告 ad2，要求投放的性别为男性，年龄为 19-30 和 41-56 之间，那么在 ad3 所在的列分别将对应于“性别-男”，“年龄-19-30”，“年龄 41-56”的项置 1。

如果当前的广告用户性别为女，年龄为 25 岁，那么广告数据库中有哪些广告可以用于此用户的投放呢？

我们可以通过简单的布尔计算，将表中“性别-女”，“年龄 19-30”的两行进行“与（and）”运算，即可求出所有的符合投放要求的广告：

可投放广告结果=0010010 and 1111001=0010000

根据“与（and）”计算结果，我们发现只有广告 ad3（结果中从左到右第 3 位为 1，对应着表中广告位 ad3）符合投放要求。

除了广告检索的倒排表可以设计为这种布尔检索的形式，搜索引擎也会用到倒排表。对于用户关键字的搜索请求，搜索引擎会提供对应的包含该关键字的 web 页面返回给用户。在搜索引擎后台对于关键字所在 web 文章的检索，也可以利用类似的设计来建立关键字到 web 文章的倒排索引表，以提高检索的计算速度。

### 8.5.3 图灵停机问题

图灵停机问题（halting problem）是逻辑数学中可计算性理论的一个问题。停机问题就是判断任意一个程序是否能在有限的时间之内结束运行的问题，即是否存在一个程序 P，对于任意输入的程序 w，能够判断 w 会在有限时间内结束或者死循环，也就是判断任意一个程序是否会在有限的时间之内结束运行的问题。

图灵停机问题的结论是：否，即不存在这样一个程序，它能够判断任意输入的程序在有限的时间之内结束运行。用反证法可以证明这个结论。

假设存在这样的一个程序 Halt，它能够判断任意输入的程序在有限的时间内结束运行，设这个程序的伪代码如下：

```
bool Halt(procedure, input){//procedure 为待判断的程序，input 为 procedure 的输入
    if(procedure halts on input)
        return true; //停机，返回 true
    return false;
}
```

现在我们基于 Halt 程序，实现一个新的程序：

```
bool SuperHalt(procedure){
    if( Halt(procedure, procedure) ){//程序本身也可以作为程序的输入
        while(1);          // loop forever!
        return false;    // can never get here!
    }
    return true;
}
```

现在我们来分析下程序 SuperHalt（）调用和执行过程。

设 P 为待判断是否停机的程序，当我们调用 SuperHalt(P)时，他会调用 Halt(P, P)，即 SuperHalt(P)→Halt(P, P)。

假如 Halt(P, P)判断 P 可停机，则 SuperHalt(P)会进入死循环，判断 P 不可停机。因此 SuperHalt(P)和 Halt(P，P)矛盾，即 SuperHalt(P)判断程序 P 为死循环不可停机，与 Halt(P，P)判断 P 为可停机相矛盾。

假如 Halt(P, P)判断 P 为死循环不可停机，那么 SuperHalt(P)判定 P 为可停机，两者依然矛盾。

所以，SuperHalt(P)和 Halt(P, P)总是得出矛盾的结论，因此，不存在这样的一个程序 Halt，它能够判断任意输入的程序 P 在有限的时间内结束运行。

因此，原命题被证否。

图灵的停机问题要从在 1928 年的国际数学家大会上大数学家希尔伯特提出的三个问题说起。这三个问题分别是：

问题 1：数学是完备的吗？也就是说是不是每个命题都能证明或证伪？比如 1+1=2 是可以证明的，1+1=3 就是可以证伪的。

问题 2：数学是相容的吗？也就是说数学命题会不会自相矛盾。比如知道了 1+1=2，那么你再去说 1+1=3 就矛盾了。

问题 3：数学是可判定的吗？也就是说是不是存在一个公式或者机器，可以应用于任何命题，然后自动给出命题的真假。

图灵提出的停机问题主要目的是想要证明希尔伯特三大问题中的可判定性问题**是否存**

**在一种通用的机械过程，能够判定任何数学命题的真假**。为了证明这个问题，图灵设计了覆盖了所有的“机械过程”的图灵机，如果存在一个问题，图灵机判定不了，那么就不存在这种通用的过程，原问题即可得证。通过图灵停机问题，图灵发现图灵机确实判定不了（证明见上面的反证法），所以原问题得以证否。因此这个问题后来就被称作图灵停机问题。

随着可判定性问题被证否，希尔伯特的三大问题也就全部被证否（前两个是由哥德尔证否的）。

图灵设计的图灵机，当时的目的是为了证明希尔伯特的判定问题。图灵发现，虽然这个机器不能解决所有的问题，但依然能够解决很多问题，而且是真的可以造出来的。图灵因而被称为计算机科学之父。从这一点看出，计算机的诞生从根本上来说，是离不开数学理论的发展和数学问题的解决的。

图灵停机问题也告诉我们，程序不是万能的，总有程序解决不了的问题。同样停机问题也说明了，不存在一个判定一切程序的程序，因为这个程序本身也是程序。停机问题和理发师悖论是类似的，归根结底是定义了一个集合 $S=\{x|x\notin x\}$。解决悖论的唯一方法是拒绝集合的自指（参考本书 2.3 节）。

## 8.5.4 机器学习与布尔代数

机器学习中有一种二分类问题，就是判断“是和否”，“有和无”，“正和负”的问题。比如判断一副图中是否有苹果，判断 URL 是否为恶意网址，判断电子邮件是否为垃圾邮件，判断一个人是否健康等。这类问题和布尔代数中的“真和假”，0 和 1，true 和 false 是相对应的。

在基于已有的样本训练出一个模型，然后用于对新的数据进行判断预测的时候，有这样几种可能：样本真实情况为“正”，模型判断结果也是“正”，这被称为真正 (True Positive，TP)；样本真实情况为“正”，但是模型判断结果为“负”，这被称为假负(False Negative，FN)：；样本真实情况为“负”，模型判断为“正”，这被称为假正 (False Positive，FP)；样本真实情况为“负”，模型判断为“负”，这被称为真负 (True Negative，TN)。

这些预测可能可以用下面的混淆矩阵给予说明：

| 真实情况 | 预测结果 | |
|---|---|---|
| | 正例 | 反例 |
| 正例 | *TP* (真正例) | *FN* (假反例) |
| 反例 | *FP* (假正例) | *TN* (真反例) |

基于混淆矩阵，我们可以定义一组有关模型预测结果的性能度量指标：

**准确率（Accuracy）**=所有预测正确的样本/总的样本，即（TP+TN）/(TP+TN+FP+FN)。

**精确率（Precision）**=将正类预测为正类 / 所有预测为正类，即 TP/（TP+FP），也叫查准率。针对我们预测结果而言的，它表示的是预测为正的样本中有多少是真正的正样本。针对我们预测结果而言的，它表示的是预测结果中正确的预测占总样本数的比例。

**召回率（Recall）**=将正类预测为正类 / 所有正真的正类，即 TP/（TP+FN），这叫查全率。针对我们原来的样本而言的，它表示的是样本中的正例有多少被预测正确了。

**F 值（F1 Score）** = 精确率 * 召回率 *2 / ( 精确率 + 召回率) （F 值即为精确率和召回率的调和平均值）。

为模型预测的结果定义这样几种性能指标，是有显示意义的。比如一堆西瓜，我们通过机器学习训练出了一个模型，需要从一堆西瓜里面判断是好瓜还是坏瓜。如果这一堆西瓜里坏瓜的比列很高，那么我们可以把这堆西瓜全部预测为坏瓜，准确率也会很高，但是对好瓜的识别能力几乎为0。而召回率，就可以衡量模型对正样本的检出能力，可以避免准确率的不足。比如：有 100 个西瓜，其中 90 个是坏瓜，10 个为好瓜。模型预测结果是把 90 个坏瓜和 9 个好瓜都预测为了坏瓜，把 1 个好瓜预测为好瓜。那么：

准确率=(90+1)/100=0.91

召回率=1/10=0.1

通过计算结果可以看出，虽然模型准确率很高，但是召回率很低，所以这是一个很差的模型。

另外一种情况就是：同样是 100 个西瓜，其中 90 个是坏瓜，10 个为好瓜。模型在预测的时候，倾向于把坏瓜预测为好瓜，因此 90 个坏瓜中有 70 个预测为坏瓜，其中有 20 个预测为好瓜，10 个好瓜也被预测为好瓜。那么：

准确率：(70+10)/100=0.8

召回率：10/10=1

精确率：10/30=0.33

通过计算可以知道，准确率和召回率都很不错，但是精确率却很差，这个模型也是不好的。

实际上召回率与精确率是一对矛盾的度量。

一般情况下，召回率高时，精确率往往偏低；精确率高时，召回率往往偏低。比如，为了提高召回率，可以把所有的西瓜都判断为好瓜，那么所有的好瓜都判断为好瓜了，因此召回率很高，但是精确率就会比较低了。如果为了提高精确率，可以只挑选那些最有把握的瓜判断为好瓜，那么精确率会很高，但是难免会漏掉不少好瓜，因此也就会造成召回率较低。

正是因为召回率与精确率有时候是矛盾的，因此就需要综合考虑它们，最常见的方法就是 **F 值**，通过计算 F 值来评价一个指标。F 值是 Precision 和 Recall 调和平均，综合了 P 和 R 的结果，当 F1 较高时则能说明试验方法比较有效。

# 九，线性代数基础与程序应用

线性代数是代数学的一个分支，主要处理线性关系问题。那么什么是线性关系呢？线性（linear）指量与量之间按比例、成直线的关系，在数学上可以理解为一阶导数为常数的函数（也就是变量的次数为 1）。线性关系就是指数学对象之间的关系是以一次形式来表达的。例如，平面上直线的方程是二元一次方程；空间平面的方程是三元一次方程。含有 n 个未知量的一次方程称为线性方程。关于变量是一次的函数称为线性函数。线性关系问题简称线性问题。

解线性方程组的问题是最简单的线性问题。实际上著名的“鸡兔同笼”问题就是一个简单的线性方程组求解的问题。

鸡兔同笼问题是这样描述的：有若干只鸡和兔关在同一个笼子里，从笼子上面数，有 35 个头，从笼子下面数，有 94 只脚。问笼中各有多少只鸡和兔？

一只鸡有 2 只脚，一只兔有 4 只脚，那么设鸡有 x 只，兔有 y 只，那么列出二元一次方程：

$$\begin{cases} x+y=35 \\ 2x+4y=94 \end{cases}$$

在这里，x 和 y 都是一次线性关系，解该二元一次方程得：

$$\begin{cases} x=23 \\ y=12 \end{cases}$$

所以鸡有 23 只，兔有 12 只，问题解决。

线性代数起源于对二维和三维直角坐标系的研究。一个向量是一个有方向的线段，由长度和方向同时表示。这样向量可以用来表示物理量，比如力、速度等，也可以和标量（只有数值大小，而没有方向，比如重量，温度等）做加法和乘法。

随着研究线性方程组和变量的线性变换问题的深入，行列式和矩阵在 18～19 世纪期间先后产生，为处理线性问题提供了有力的工具，从而推动了线性代数的发展。向量概念的引入，形成了向量空间的概念。凡是线性问题都可以用向量空间的观点加以讨论。因此，向量空间及其线性变换，以及与此相联系的矩阵理论，构成了线性代数的中心内容。

现代线性代数已经扩展到研究任意或无限维空间。一个维数为 n 的向量空间叫做 n 维空间。在二维和三维空间中大多数有用的结论可以扩展到这些高维空间。n 维空间中的向量（即 n 元组）用来表示数据非常有效。由于作为 n 元组，向量是 n 个元素的“有序”列表，大多数人可以在这种框架中有效地概括和操纵数据。在机器学习或人工智能里，也是以向量的方式来处理数据的特征值并利用矩阵的方法完成计算的。

线性代数在数学、物理学和技术学科中有各种重要应用，因而它在各种代数分支中占居首要地位。在计算机广泛应用的今天，计算机图形学、计算机辅助设计、密码学、虚拟现实、人工智能等技术都以线性代数为其理论和算法基础的一部分。

很多实际问题的处理，最后往往归结为线性问题，它比较容易处理。因此，线性代数在工程技术和国民经济的许多领域都有着广泛的应用，是一门基本的和重要的学科。线性是人类少数可以研究得非常透彻的数学基础性框架。进入科研领域，就会发现，只要不是线性的东西，人类其实基本都不会。如果“不学线性代数，就漏过了 95%的人类智慧”。这是因为非线性的问题极为困难，并没有足够多的通用的性质和定理用于求解非线性的具体问题。因此我们要尽可能的把非线性的问题化为线性问题再求解。微积分“以直代曲”的思想就是将整体非线性化为局部线性的一个经典的例子。“以直代曲”是人们处理很多数

学问题时一个重要的思想（参考本书 10.3.1 节）。

# 9.1 行列式

行列式（determinant）理论产生于十七世纪末，到十九世纪末，它的理论体系已基本形成。行列式的概念最早由十七世纪日本数学家关孝和（在日本被尊称为算圣）提出来的，他在 1683 年写的《解伏题之法》一书中，对行列式的概念和它的展开有了清楚的叙述。

1693 年，德国数学家莱布尼茨解方程组时将系数分离出来用以表示未知量，得到行列式原始概念，并给出了方程组的系数行列式为零的条件。

1750 年，瑞士数学家克拉默 (Gramer，1704-1752)更完整地叙述了行列式的展开法则并将它用于解线性方程组，产生了克拉默法则。

无论是在线性代数、多项式理论，还是在微积分学中，行列式作为基本的数学工具，都有着重要的应用。

## 9.1.1 行列式定义

那么什么是行列式呢？如果我们把 4 个数 $a_{11}, a_{12}, a_{21}, a_{22}$ 排成 2 行 2 列的数表如下：

$$\begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix}$$

那么表达式 $a_{11}a_{22}-a_{12}a_{21}$ 称为数表所确定的**二阶行列式**，其中数 $a_{ij}(i=1, 2; j=1, 2)$称为行列式的元素，$a_{ij}$ 的 i 和 j 分别称为行列式的行标和列标，表明该元素位于行列式的第 i 行和 j 列。行列式可以记作：

$$D=\det(a_{ij})$$

二阶行列式可以用对角线法则来计算。如图 9-1 所示，把 $a_{11}$ 和 $a_{22}$ 用实线连接，并称该实线为主对角线，$a_{12}$ 到 $a_{21}$ 用虚线连接，并称该虚线为副对角线。

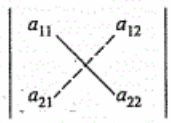



图 9-1 二阶行列式对角线法则

二阶行列式的值，就等于主对角线上的两元素乘积减去副对角线上两元素乘积的差。即：

$$\mathbf{D=\det(a_{ij})=a_{11}a_{22}-a_{12}a_{21}}$$

如果我们把 9 个数排列成 3 行 3 列的数表如下：

$$\begin{vmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{vmatrix}$$

那么所确定的行列式为**三阶行列式**，记为：

$$\mathbf{D=\det(a_{ij})=a_{11}a_{22}a_{33}+a_{12}a_{23}a_{31}+a_{13}a_{21}a_{32}-a_{11}a_{23}a_{32}-a_{12}a_{21}a_{33}-a_{13}a_{22}a_{31}}$$

三阶行列式的计算同样适用于对角线法则，如图 9-2 所示：

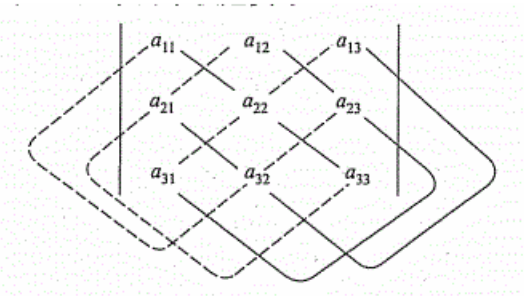



图 9-2 三阶行列式对角线法则

从主对角线开始的三条实线看作是平行于主对角线的连线，从副对角线开始的三条虚线看作是平行于副对角线的连线。三阶行列式的值就是实线上三元素的乘积取正与虚线上的三元素的乘积取负的结果。

认识了二阶和三阶行列式，那么 n 阶行列式的定义似乎就呼之欲出了。但是，n 阶（n>3）行列式的计算方法却不再适用于对角线法则。因此为了介绍 n 阶行列式，需要先了解一下排列及其逆序数。

把 n 个不同的元素排成一列，叫作这 n 个元素的全排列，简称为排列。根据组合数学知识，n 个不同元素的所有排列总数有 n！种，记为 $p_n=n!$。如果规定 n 个不同的元素在排列的时候各个元素之间有一个标准次序（比如 n 个不同的自然数，规定标准次序为从小到大），那么在这 n 个元素的任一排列中，当某一对（两个）元素的先后次序与标准次序不同时，就构成了 1 个逆序。一个排列中所有逆序之和叫作这个排列的逆序数，并且称逆序数为奇数的排列教奇排列，逆序数为偶数的排列叫偶排列。

以自然数为例子。设 $p_1p_2p_3...p_n$ 为 n 个自然数的一个排列，标准次序为从小到大。那么对于元素 $p_i(i=1, 2, 3, ..., n)$，如果比 $p_i$ 大并且排在 $p_i$ 前面的元素有 $t_i$ 个，那么 $p_i$ 的逆序数是 $t_i$。整个排列的逆序数为：

$$t=t_1+t_2+t_3+...+t_n=\sum_{i=1}^{n} t_i$$

练习 1：规定由小到大为标准次序，求排列 9，5，2，7，3 的逆序数。

分析：9 排在第一位，比 9 大并且排在 9 前面的元素为 0，因此逆序数 $t_1=0$；5 的前面比 5 大的有 9，所以 $t_2=1$；2 的前面比 2 大的有 9, 5，所以 $t_3=2$；7 的前面比 7 大的数有 9，所以 $t_4=1$；3 的前面比 3 大的数有 9, 5, 7，所以 $t_5=3$。因此逆序数：

$$t=0+1+2+1+3=7$$

有了排列和逆序数的概念，现在来正式介绍 n 阶行列式的定义。

设有 $n^2$ 个数排成 n 行 n 列的如下数表：

$$\begin{matrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ & \cdots\cdots\cdots & & \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{matrix}$$

对于 $(-1)^t a_{1p_1}a_{2p_2}\cdots a_{np_n}$，其中 $p_1p_2p_3...p_n$ 为自然数 1，2，3，...，n 的一个排列（即数表中每一行取 1 个数），它是数表中不同行不同列的 n 个数（一个排列）的乘积，并以 $(-1)^t$ 作为该乘积的符号，t 为这个排列的逆序数。因为这样的排列共有 n！个，因此所有 n！项的代数和 $\sum(-1)^t a_{1p_1}a_{2p_2}\cdots a_{np_n}$ 被称作 n 阶行列式，记为：

$$D=\det(a_{ij})=\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}=\sum(-1)^{t}a_{1p_1}a_{2p_2}\cdots a_{np_n}$$

根据此定义，求 n 阶行列式，只需要列出 1, 2, 3,..., n 的 n！个排列 $p_1p_2p_3...p_n$，然后计算 $(-1)^{t}a_{1p_1}a_{2p_2}\cdots a_{np_n}$ 之和即可。

由此定义的二阶和三阶行列式与前面通过对角线法则定义的二阶，三阶行列式是一致的。特别地，根据此定义，当 n=1 时，定义的行列式：

$$\mathbf{D=det(a_{ij})=|a|=a}$$

## 9.1.2 行列式性质

行列式有一些重要的性质，掌握这些性质，可以帮助我们简化行列式的计算。现将这些性质列举如下：

性质 1：行列式与它的转置行列式相等。所谓转置行列式，就是将行列式 D 中行的项转为列的项成为行列式 $D^T$，则行列式 $D^T$ 称为行列式 D 的转置行列式。比如 D 中第一行：

$$a_{11}, a_{12}, ..., a_{1n}$$

变为 $D^T$ 中的第一列。

$$D=\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix},\quad D^{T}=\begin{vmatrix} a_{11} & a_{21} & \cdots & a_{n1} \\ a_{12} & a_{22} & \cdots & a_{n2} \\ \vdots & \vdots & & \vdots \\ a_{1n} & a_{2n} & \cdots & a_{nn} \end{vmatrix}$$

性质 2：对换行列式的两行（列），行列式变号。性质 2 可得推论：如果行列式有两行（列）完全相同，此行列式等于零。

性质 3：行列式的某一行（列）中所有的元素都乘以同一数 k，等于用此数 k 乘此行列式。性质 3 可得推论：行列式中某一行（列）的所有元素的公因子可以提到行列式记号外面。

性质 4：行列式中如果两行（列）元素成比例，则此行列式等于零。

性质 5：若行列式的某一行（列）的元素都是两数之和，则 D 等于下列两个行列式之和：

$$D=\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1}+a'_{i1} & a_{i2}+a'_{i2} & \cdots & a_{in}+a'_{in} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}=\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}+\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a'_{i1} & a'_{i2} & \cdots & a'_{in} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix}$$

性质 6：把行列式的某一行（列）的各元素乘同一数然后加到另一行（列）对应的元素上去，行列式不变。

$$\begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{j1} & a_{j2} & \cdots & a_{jn} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} \overset{r_j+kr_i}{=\!=} \begin{vmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{j1}+ka_{i1} & a_{j2}+ka_{i2} & \cdots & a_{jn}+ka_{in} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} \quad (i \neq j)$$

对于几个典型的行列式计算方法如下：

主对角线下（上）的元素都为 0 的行列式叫上（下）三角形行列式，计算方法为：

$$D = \begin{vmatrix} a_{11} & & & \\ a_{21} & a_{22} & & 0 \\ \vdots & \vdots & \ddots & \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{vmatrix} = a_{11}a_{22}\cdots a_{nn};$$

主对角线以上和以下都为 0 的行列式叫对角行列式，计算公式为：

$$\begin{vmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{vmatrix} = \lambda_1\lambda_2\cdots\lambda_n$$

计算下面的非上（下）三角行列式，可以通过行列式行的对换方法，将此行列式转化为上（下）三角行列式。

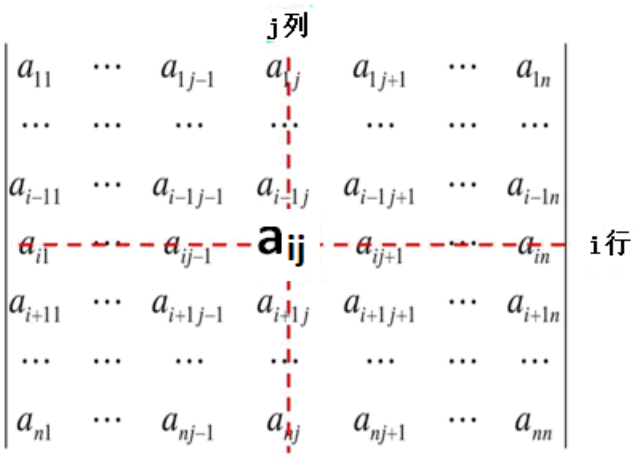
$$D = \begin{vmatrix} & & & a_{1n} \\ & & a_{2,n-1} & a_{2n} \\ & \iddots & \vdots & \vdots \\ a_{n1} & \cdots & a_{n,n-1} & a_{nn} \end{vmatrix} = (-1)^{\frac{1}{2}n(n-1)} a_{1n}\, a_{2,n-1}\cdots a_{n1}$$

$$\begin{vmatrix} & & & \lambda_1 \\ & & \lambda_2 & \\ & \iddots & & \\ \lambda_n & & & \end{vmatrix} = (-1)^{\frac{n(n-1)}{2}} \lambda_1\lambda_2\cdots\lambda_n$$

## 9.1.3 行列式按行（列）展开

高阶行列式计算比较复杂，因此能否用低阶行列式表示高阶行列式呢？答案是肯定的。首先介绍一下余子式和代数余子式的概念。

在 n 阶行列式中，如果把元素 $a_{ij}$ 所在的第 i 行，第 j 列从该行列式中划掉，那么可以得到一个 n-1 阶的行列式，这个 n-1 阶的行列式叫做元素 $a_{ij}$ 的余子式，记作 $M_{ij}$；记 $A_{ij}=(-1)^{i+j}M_{ij}$，，则 $A_{ij}$ 叫做 $a_{ij}$ 的代数余子式。




有了余子式和代数余子式的定义之后，那么我们就可以把高阶行列式转换为低阶行列式来计算了。

行列式定理：行列式等于它的任一行（列）的各元素与其对应的代数余子式乘积之和，即：

$$D=a_{i1}A_{i1}+a_{i2}A_{i2}+a_{i3}A_{i3}+\ldots+a_{in}A_{in}\ (i=1, 2, 3, ..., n)$$

或者：

$$D=a_{1j}A_{1j}+a_{2j}A_{2j}+a_{3j}A_{3j}+...+a_{nj}A_{nj}\ (j=1, 2, 3, ..., n)$$

这就是行列式按行（列）展开法则。利用这个法则，可以简化行列式的计算过程。同时，行列式中某一行（列）的元素与另一行（列）的对应元素的代数余子式乘积之和等于零。于是有：

$$\sum_{k=1}^{n} a_{ki}A_{kj}=\begin{cases}D, & 当\ i=j,\\ 0, & 当\ i\neq j\end{cases}$$

或者

$$\sum_{k=1}^{n} a_{ik}A_{jk}=\begin{cases}D, & 当\ i=j,\\ 0, & 当\ i\neq j.\end{cases}$$

# 9.2 矩阵

矩阵（Matrix）是一个按照长方阵列排列的复数或实数集合，最早来自于方程组的系数及常数所构成的方阵。矩阵的概念由 19 世纪英国数学家凯利首先提出。矩阵是高等代数中的常见工具，也常见于统计分析等应用数学学科中。

矩阵在电路学、力学、光学和量子力学中都有应用；计算机的三维动画制作、人工智能、数据分析也需要用到矩阵。

## 9.2.1 矩阵定义

由 $m\times n$ 个数 $a_{ij}$ 排成的 m 行 n 列的数表称为 m 行 n 列的矩阵，简称 $m\times n$ 矩阵，记作：

$$\mathbf{A}=\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{31} & a_{32} & \cdots & a_{3n} \\ \cdots & \cdots & & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

这 m×n 个数称为矩阵 A 的元素，简称为元，数 $a_{ij}$ 位于矩阵 A 的第 i 行第 j 列，称为矩阵 A 的(i, j)元，以数 $a_{ij}$ 为(i, j)元的矩阵可记为$(a_{ij})$或$(a_{ij})_{m\times n}$，m×n 矩阵 A 也记作 $A_{mn}$。

元素是实数的矩阵称为实矩阵，元素是复数的矩阵称为复矩阵。而行数与列数都等于 n 的矩阵称为 n 阶矩阵或 n 阶方阵。

只有一行的矩阵叫作**行矩阵**或者**行向量**：

$$\boldsymbol{A}=(a_1\ a_2\cdots\ a_n)$$

只有一列的矩阵叫作**列矩阵**或者**列向量**：

$$\boldsymbol{B}=\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix}$$

两个矩阵的行数和列数如果都分别相等，就称为同型矩阵。如果 $A=(a_{ij})$，$B=b(b_{ij})$是同型矩阵，并且它们对应的元素相等，即：

$$a_{ij}=b_{ij},\ (i=1, 2, 3, \ldots,\ \ m;\ j=1, 2, 3, \ldots, n),$$

那么就称矩阵 A 与矩阵 B 相等，记作：

$$A=B$$

也就是说，矩阵同型是矩阵相等的必要条件。元素都是零的矩阵称为零矩阵，记作 O。

对于非齐次（即方程等式右边有不为零的常数项）线性方程组：

$$\begin{cases} a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1, \\ a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2, \\ \cdots\cdots\cdots \\ a_{m1}x_1+a_{m2}x_2+\cdots+a_{mn}x_n=b_m, \end{cases}$$

可以得出如下几个矩阵：

$$\boldsymbol{A}=(a_{ij}),\boldsymbol{x}=\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix},\boldsymbol{b}=\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix},\boldsymbol{B}=\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

其中 A 称为**系数矩阵**，x 称为**未知数矩阵**，b 称为**常数项矩阵**，B 称为**增广矩阵**。后面可以看到，线性方程组的求解，可以通过这些矩阵的运算完成。

对于 n 阶方阵：

$$\boldsymbol{A}=\begin{pmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{pmatrix}.$$

如果从左上角到右下角的直线（对角线）以外的元素都是 0，那么我们把这种方阵称为**对角矩阵**，简称**对角阵**，记作：$\Lambda=\mathrm{diag}(\lambda_1, \lambda_2, \lambda_3, \ldots, \lambda_n)$(diag 是 Diagonal 对角的缩写)。

在对角阵中，如果 $\lambda_1=\lambda_2=\lambda_3=\ldots=\lambda_n=1$，它对应的 n 阶方阵：

$$\boldsymbol{E}=\begin{pmatrix}1 & 0 & \cdots & 0\\ 0 & 1 & \cdots & 0\\ \vdots & \vdots & & \vdots\\ 0 & 0 & \cdots & 1\end{pmatrix}$$

叫作 n 阶单位矩阵，简称单位阵。根据定义可知，单位阵 E 的(i, j)元 $e_{ij}$ 为：

$$e_{ij}=\begin{cases}1, \text{当 } i=j,\\ 0, \text{当 } i\neq j\end{cases}\quad (i,j=1,2,\cdots,n)$$

## 9.2.2 矩阵运算

矩阵可以进行加法，乘法，幂等运算。这些运算是处理方程求解，程序大数据处理和人工智能的基础。现在我们来学习这些运算法则。

### 1.加法运算

设有两个 m × n 的矩阵 $A=(a_{ij})$，$B=(b_{ij})$，那么矩阵 A 与 B 的和记为：

$$\boldsymbol{A}+\boldsymbol{B}=\begin{pmatrix}a_{11}+b_{11} & a_{12}+b_{12} & \cdots & a_{1n}+b_{1n}\\ a_{21}+b_{21} & a_{22}+b_{22} & \cdots & a_{2n}+b_{2n}\\ \vdots & \vdots & & \vdots\\ a_{m1}+b_{m1} & a_{m2}+b_{m2} & \cdots & a_{mn}+b_{mn}\end{pmatrix}$$

矩阵的加法运算，只适用于同型矩阵之间，并具有如下运算规律（设 A, B, C 都是 m × n 的矩阵）：

交换律：A+B=B+A

结合律：(A+B)+C=A+(B+C)

设矩阵 $A=(a_{ij})$，令 $-A=(-a_{ij})$，即把 A 中所有元素进行取反，则称-A 为矩阵 A 的负矩阵，根据负矩阵的定义有：

$$A+(-A)=O$$

因此定义矩阵的减法为：

$$A-B=A+(-B)$$

### 2.数乘运算

矩阵数乘运算即数与矩阵的乘法运算，数 λ 与矩阵 A 的乘积记为 λA，计算规则为：

$$\lambda A = A\lambda = \begin{pmatrix} \lambda a_{11} & \lambda a_{12} & \cdots & \lambda a_{1n} \\ \lambda a_{21} & \lambda a_{22} & \cdots & \lambda a_{2n} \\ \vdots & \vdots & & \vdots \\ \lambda a_{m1} & \lambda a_{m2} & \cdots & \lambda a_{mn} \end{pmatrix}.$$

矩阵的加法与数乘运算统称为矩阵的线性运算。

## 3.矩阵与矩阵乘法

设 A=($a_{ij}$)为 m × s 的矩阵，B=($b_{ij}$)是一个 s× n 的矩阵，规定它们的乘积 AB 为一个 m × n 的矩阵 C=($c_{ij}$)，其中

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + a_{i3}b_{3j} + \ldots + a_{is}b_{sj} = \sum_{k=1}^{s} a_{ik}b_{kj}$$

并记为：C=AB。

练习 2：已知矩阵 A 和 B，试计算 AB。

$$A = \begin{pmatrix} 4 & -1 & 2 & 1 \\ 1 & 1 & 0 & 3 \\ 0 & 3 & 1 & 4 \end{pmatrix} \quad 与 \quad B = \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 3 & 0 \\ -1 & 2 \end{pmatrix}$$

解：

$$C = AB = \begin{pmatrix} 4 & -1 & 2 & 1 \\ 1 & 1 & 0 & 3 \\ 0 & 3 & 1 & 4 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ 0 & 1 \\ 3 & 0 \\ -1 & 2 \end{pmatrix}$$

$$= \begin{pmatrix} 4\times1+(-1)\times0+2\times3+1\times(-1) & 4\times2+(-1)\times1+2\times0+1\times2 \\ 1\times1+1\times0+0\times3+3\times(-1) & 1\times2+1\times1+0\times0+3\times2 \\ 0\times1+3\times0+1\times3+4\times(-1) & 0\times2+3\times1+1\times0+4\times2 \end{pmatrix}$$

$$= \begin{pmatrix} 9 & 9 \\ -2 & 9 \\ -1 & 11 \end{pmatrix}.$$

此计算法则表明：乘积矩阵 C 的(i, j)元 $c_{ij}$ 就是矩阵 A 的第 i 行和矩阵 B 的第 j 列对应元素的乘积之和。一个 1× s 的行矩阵与一个 s × 1 的列矩阵的乘积是一个 1 阶方阵，也就是一个数。

$$(a_{i1}, a_{i2}, \cdots, a_{is}) \begin{pmatrix} b_{1j} \\ b_{2j} \\ \vdots \\ b_{sj} \end{pmatrix} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{is}b_{sj} = \sum_{k=1}^{s} a_{ik}b_{kj} = c_{ij}$$

同时需要注意的是，两个矩阵能进行乘法运算的前提条件就是第一个矩阵（左边矩阵）的列数与第二个矩阵（右边矩阵）的行数必须相等，否则是无法进行乘法运算的。

在进行矩阵乘法运算的时候，需要注意它们相乘的顺序。因为根据定义，AB 是 A 左乘 B 的乘积，AB 有意义，但 BA 不一定有意义，即使符合矩阵相乘的条件有意义，但其结果也不一定相等。因此，矩阵的乘法是不满足交换律的，即在一般情形下，AB≠BA。

另外，在 A≠O，B≠O，却有可能 AB=O。而如果 AB=O，不能得出 A=O 或者 B=O 的结论。

矩阵的乘法运算虽然不满足交换律，但仍然满足结合律与分配率，即：

结合律：(AB)C=A(BC)，$\lambda(AB)=(\lambda A)B=A(\lambda B)$

分配率：A(B+C)=AB+AC, (B+C)A=BA+CA

对于单位矩阵 E，有：

$$EA=AE=A$$

此外，矩阵的幂运算其实是一种特殊的矩阵乘法运算，它的定义如下：

$$A^1=A,\quad A^2=A^1A^1,\ \ldots,\ A^{k+1}=A^kA^1$$

需要注意的是，一般地，$(AB)^k\neq A^kB^k$，$(A+B)^2\neq A^2+2AB+B^2$，$(A-B)(A+B)\neq A^2-B^2$，只有当 A 与 B 可交换，即 AB=BA 时，才成立。

熟悉了矩阵乘法之后，那么多元线性方程组就可以用矩阵乘法的形式来表达了：

$$\begin{cases} a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1, \\ a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2, \\ \cdots\cdots\cdots\cdots \\ a_{m1}x_1+a_{m2}x_2+\cdots+a_{mn}x_n=b_m, \end{cases}$$

$$\boldsymbol{A}=(a_{ij}),\boldsymbol{x}=\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix},\boldsymbol{b}=\begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{pmatrix},\boldsymbol{B}=\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} & b_1 \\ a_{21} & a_{22} & \cdots & a_{2n} & b_2 \\ \vdots & \vdots & & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} & b_m \end{pmatrix}$$

则有：

$$\mathbf{A_{m\times n}x_{n\times 1}=b_{m\times 1}}$$

可简写为：

$$\mathbf{Ax=b}$$

## 9.2.3 矩阵转置

设矩阵 A 为 m×n 型，如果把它的行换成同序数的列（比如第 2 行换成第 2 列），得到一个 n×m 的新矩阵，该新矩阵称为 A 的转置矩阵，记作 $A^T$。比如：

$$\boldsymbol{A}=\begin{pmatrix} 1 & 2 & 0 \\ 3 & -1 & 1 \end{pmatrix}$$

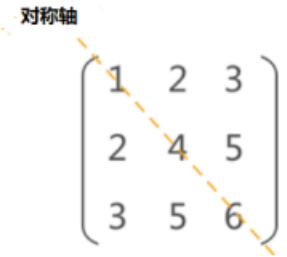
把第一行变为第一列，第二行变为第二列，得到对应的转置矩阵为：

$$A^T=\begin{pmatrix}1 & 3\\ 2 & -1\\ 0 & 1\end{pmatrix}.$$

矩阵的转置也是矩阵的一种运算，它满足如下几个规律：

- $(A^T)^T=A$
- $(A+B)^T=A^T+B^T$
- $(\lambda A)^T=\lambda A^T$
- $(AB)^T=B^TA^T$

设 A 为 n 阶方阵，如果 $A^T=A$，即 $a_{ij}=a_{ji}$，（i, j=1, 2, 3, ..., n），那么称 A 为对称矩阵。在对称矩阵中，其元素以对角线为对称轴对应相等。一个具体的对称矩阵例子如下所示：




## 9.2.4 方正行列式与伴随矩阵

由 n 阶方阵 A 的元素所构成的行列式，称为方阵 A 的行列式，记作 detA 或者|A|（这里不是绝对值的意思）。方阵 A 的行列式是一个数值，而 n 阶方阵则是一个 $n^2$ 的数表。

方阵的行列式，有如下性质：

- $|A^T|=|A|$
- $|\lambda A|=\lambda|A|$
- $|AB|=|A||B|$

定义了方阵的行列式之后，我们来定义方阵的伴随矩阵。设 n 阶方阵 A 的行列式|A|的各个元素 $a_{ij}$ 的代数余子式 $A_{ij}$ 所构成如下的矩阵：

$$A^*=\begin{pmatrix}A_{11} & A_{21} & \cdots & A_{n1}\\ A_{12} & A_{22} & \cdots & A_{n2}\\ \vdots & \vdots & & \vdots\\ A_{1n} & A_{2n} & \cdots & A_{nn}\end{pmatrix}.$$

则称 A*为矩阵 A 的伴随矩阵，A 和 A*有如下关系：

$$AA^*=A^*A=|A|E$$

因为根据代数余子式的性质，设 $A=(a_{ij})$，$AA^*=(b_{ij})$，则：

$$b_{ij}=a_{i1}A_{j1}+a_{i2}A_{j2}+\cdots+a_{in}A_{jn}=\begin{cases}|A|, & i=j,\\ 0, & i\neq j,\end{cases}$$

因此：

$$AA^*=\begin{pmatrix} |A| & & & \\ & |A| & & \\ & & \ddots & \\ & & & |A| \end{pmatrix}=|A|E$$

## 9.2.5 逆矩阵

对于 n 阶方阵 A，如果有一个 n 阶方阵 B，使得：

$$AB=BA=E$$

则称 A 可逆，并把 B 称为 A 的逆矩阵。A 的逆矩阵记作 $A^{-1}$，即若 AB=BA=E，则 $B=A^{-1}$。

容易证明，如果一个矩阵 A 可逆，那么它的逆矩阵是唯一的。因为若 B，C 都是 A 的逆矩阵，即 AC=BA=E，那么：

$$B=BE=B(AC)=(BA)C=EC=C$$

因此 B 和 C 是相等的，也就是逆矩阵是唯一的。

如果矩阵 A 可逆，那么$|A|\neq 0$，这是因为若 A 可逆，那么有 $AA^{-1}=E$，因此$|A||A^{-1}|=|E|=1$。所以$|A|\neq 0$。

那么如何求一个矩阵的逆矩阵呢？

实际上，若一个方阵 A，它对应的行列式$|A|\neq 0$，那么 A 就可逆，并且它的逆矩阵为：

$$\mathbf{A^{-1}=A^*/|A|}$$

也就是说，求一个$|A|\neq 0$ 的方阵的逆矩阵，只需要求出它的伴随矩阵 A*，再用伴随矩阵除以|A|就是这个矩阵的逆矩阵了。

这是因为：

$$AA^*=A^*A=|A|E，\text{因此}(AA^*)/|A|=(A^*/|A|)A=E。$$

当|A|=0 时，A 被称为奇异矩阵，否则称为非奇异矩阵。A 是可逆矩阵的充分必要条件是$|A|\neq 0$，即 A 是非奇异矩阵。

练习 3：求下列矩阵的逆矩阵。

$$A=\begin{pmatrix} 1 & 2 & 3 \\ 2 & 2 & 1 \\ 3 & 4 & 3 \end{pmatrix}$$

解：

首先:

$$|A|=1*2*3+2*1*3+3*2*4-3*2*3-1*4*1-3*2*2=2$$

由于$|A|\neq 0$，所以矩阵 A 可逆。由于 $A^{-1}=A^*/|A|$，所以要求 $A^{-1}$，只需要再求出 A 的伴随矩阵 A*，问题即可解决。

根据伴随矩阵的定义，有：

$$A^* = \begin{pmatrix} M_{11} & -M_{21} & M_{31} \\ -M_{12} & M_{22} & -M_{32} \\ M_{13} & -M_{23} & M_{33} \end{pmatrix} = \begin{pmatrix} 2 & 6 & -4 \\ -3 & -6 & 5 \\ 2 & 2 & -2 \end{pmatrix}$$

所以有：

$$A^{-1}=\frac{1}{|A|}A^{*}=\begin{pmatrix}1 & 3 & -2\\ -\frac{3}{2} & -3 & \frac{5}{2}\\ 1 & 1 & -1\end{pmatrix}$$

实际上，只要 AB=E，那么就有 $B=A^{-1}$，即 B 就是 A 的逆矩阵。因为|A||B|=|E|=1，因此|A|≠0。因此 A 可逆，即 $A^{-1}$ 存在。又因为 $B=EB=(A^{-1}A)B=A^{-1}（AB）=A^{-1}E=A^{-1}$。

容易证明逆矩阵有如下运算规律：

- 若 A 可逆，则 $A^{-1}$ 也可逆，且$(A^{-1})^{-1}=A$。
- 若 A 可逆，数 λ≠0，则 λA 也可逆，且$(\lambda A)^{-1}=A^{-1}/\lambda$。
- 若 A、B 为同阶矩阵且均可逆，那么 AB 也可逆，并且$(AB)^{-1}=B^{-1}A^{-1}$。

逆矩阵的一个直接应用就是用于解 n 个 n 元线性方程组成的方程组，即克拉默法则。含有 n 个未知数 $x_1$，$x_2$，$x_3$，…，$x_n$ 的 n 个线性方程的方程组：

$$\begin{cases}a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1,\\ a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2,\\ \cdots\cdots\cdots\\ a_{n1}x_1+a_{n2}x_2+\cdots+a_{nn}x_n=b_n\end{cases}$$

设：

$$A=(a_{ij}),x=\begin{pmatrix}x_1\\ x_2\\ \vdots\\ x_n\end{pmatrix},b=\begin{pmatrix}b_1\\ b_2\\ \vdots\\ b_m\end{pmatrix}$$

把这个方程组写成矩阵方程为：

$$Ax=b$$

若|A|≠0，那么 $A^{-1}$ 存在，则有：$A^{-1}Ax=A^{-1}b$，即 $x=A^{-1}b$。因为逆矩阵是唯一的，因此 $x=A^{-1}b$ 是方程组的唯一解向量。

又因为有：$A^{-1}=A^{*}/|A|$，因此有：

$$x=A^{-1}b=A^{*}b/|A|$$

这就是逆矩阵法求得的向量 x 的解。也就是：

$$\begin{pmatrix}x_1\\ x_2\\ \vdots\\ x_n\end{pmatrix}=\frac{1}{|A|}\begin{pmatrix}A_{11} & A_{21} & \cdots & A_{n1}\\ A_{12} & A_{22} & \cdots & A_{n2}\\ \vdots & \vdots & & \vdots\\ A_{1n} & A_{2n} & \cdots & A_{nn}\end{pmatrix}\begin{pmatrix}b_1\\ b_2\\ \vdots\\ b_n\end{pmatrix}=\frac{1}{|A|}\begin{pmatrix}b_1A_{11}+b_2A_{21}+\cdots+b_nA_{n1}\\ b_1A_{12}+b_2A_{22}+\cdots+b_nA_{n2}\\ \vdots\\ b_1A_{1n}+b_2A_{2n}+\cdots+b_nA_{nn}\end{pmatrix}$$

因此：

$$x_j=(b_1A_{1j}+b_2A_{2j}+b_3A_{3j}+\ldots+b_nA_{nj})/|A|=|A_j|/|A|。$$

这里的 $A_j$ 是把系数矩阵 A 中的第 j 列的元素用方程组右端的常数项代替后所得到的 n 阶矩阵，即：

$$A_j=\begin{pmatrix}a_{11} & \cdots & a_{1,j-1} & b_1 & a_{1,j+1} & \cdots & a_{1n}\\ \vdots & & \vdots & \vdots & \vdots & & \vdots\\ a_{n1} & \cdots & a_{n,j-1} & b_n & a_{n,j+1} & \cdots & a_{nn}\end{pmatrix}.$$

这就是所谓的克拉默法求得的线性方程组的解。

练习 4：用克拉默法求解下列方程组。

$$\begin{cases} 2x_1+x_2-5x_3+x_4=8 \\ x_1-3x_2-6x_4=9 \\ 2x_2-x_3+2x_4=-5 \\ x_1+4x_2-7x_3+6x_4=0 \end{cases}$$

解：方程组的系数行列式为：

$$|A|=\begin{vmatrix} 2 & 1 & -5 & 1 \\ 1 & -3 & 0 & -6 \\ 0 & 0 & -1 & 2 \\ 1 & 4 & -7 & 6 \end{vmatrix}=27$$

由于|A|≠0，所以，可以使用克拉默方法求解。将系数矩阵 A 中的 1-4 列的元素用方程组右端的常数项：

$$\begin{vmatrix} 8 \\ 9 \\ -5 \\ 0 \end{vmatrix}$$

代替后所得到的 4 阶矩阵 $A_1$-$A_4$ 分别为：

$$A_1=\begin{vmatrix} 8 & 1 & -5 & 1 \\ 9 & -3 & 0 & -6 \\ -5 & 0 & -1 & 2 \\ 0 & 4 & -7 & 6 \end{vmatrix}=81$$

$$A_2=\begin{vmatrix} 2 & 8 & -5 & 1 \\ 1 & 9 & 0 & -6 \\ 0 & -5 & -1 & 2 \\ 1 & 0 & -7 & 6 \end{vmatrix}=-108$$

$$A_3=\begin{vmatrix} 2 & 1 & 8 & 1 \\ 1 & -3 & 9 & -6 \\ 0 & 0 & -5 & 2 \\ 1 & 4 & 0 & 6 \end{vmatrix}=-27$$

$$A_4=\begin{vmatrix} 2 & 1 & -5 & 8 \\ 1 & -3 & 0 & 9 \\ 0 & 0 & -1 & -5 \\ 1 & 4 & -7 & 0 \end{vmatrix}=27$$

所以，方程组的解为：

$$x_1=A_1/|A|=3,\quad x_2=A_2/|A|=-4,\quad x_3=A_3/|A|=-1,\quad x_4=A_4/|A|=1$$

需要注意的是，克拉默法求线性方程组的解只适用于系数矩阵的行列式不为零时的方程组，至于方程组的系数行列式为零等其他情形，将在本书 9.2.8 节讨论。

## 9.2.6 矩阵初等变换

给定一个矩阵，我们可以对矩阵进行一些常见的变换。矩阵有 3 种初等行变换，分别是：

- 对换两行（对换 i，j 两行，记作 $r_i \leftrightarrow r_j$）
- 以数 $k \neq 0$ 乘某一行中的所有元（第 i 行乘 k，记作 $r_i \times k$）
- 把某一行所有元的 k 倍加到另一行对应的元上去（第 j 行的 k 倍加到第 i 行上，记作 $r_i + kr_j$）

上面这三种初等变换叫作初等行变换。把定义中的“行”换成“列”，即得矩阵的 3 种初等列变换。与初等行变换类似，矩阵的初等列变换记作：

- $\boldsymbol{c_i \leftrightarrow c_j}$　对换 i，j 两列
- $\boldsymbol{c_i \times k}$　第 i 列乘以 k
- $\boldsymbol{c_i + kc_j}$　第 j 列乘以 k，加到第 i 列上

上面初等行变换中的字母 r 是 row（行）的缩写，初等列表换中的 c 是 column（列）的缩写。

矩阵的初等行变换与初等列变换，统称为初等变换。如果矩阵 A 经过有限次初等行变换变成矩阵 B，就称矩阵 A 与 B 行等价，记作 $\boldsymbol{A \overset{r}{\sim} B}$；如果矩阵 A 经过有限次初等列变换成矩阵 B，就称矩阵 A 与 B 列等价，记作 $\boldsymbol{A \overset{c}{\sim} B}$；如果矩阵 A 经过有限次初等变换变成矩阵 B，就称矩阵 A 与 B 等价，记作 $\boldsymbol{A \sim B}$。

矩阵之间的等价关系具有如下性质：

- 反身行：A~A
- 对称性：若 A~B，则 B~A
- 传递性：若 A~B，B~C，则 A~C

现在我们试着对下列矩阵 B 进行初等行变换：

$$\boldsymbol{B}=\begin{pmatrix}2 & -1 & -1 & 1 & 2\\ 1 & 1 & -2 & 1 & 4\\ 4 & -6 & 2 & -2 & 4\\ 3 & 6 & -9 & 7 & 9\end{pmatrix}$$

$$\overset{\substack{r_1\leftrightarrow r_2\\ r_3\div 2}}{\sim}\begin{pmatrix}1 & 1 & -2 & 1 & 4\\ 2 & -1 & -1 & 1 & 2\\ 2 & -3 & 1 & -1 & 2\\ 3 & 6 & -9 & 7 & 9\end{pmatrix}=\boldsymbol{B}_1$$

$$\overset{\substack{r_2-r_3\\ r_3-2r_1\\ r_4-3r_1}}{\sim}\begin{pmatrix}1 & 1 & -2 & 1 & 4\\ 0 & 2 & -2 & 2 & 0\\ 0 & -5 & 5 & -3 & -6\\ 0 & 3 & -3 & 4 & -3\end{pmatrix}=\boldsymbol{B}_2$$

$$\overset{\substack{r_2\div 2\\ r_3+5r_2\\ r_4-3r_2}}{\sim}\begin{pmatrix}1&1&-2&1&4\\0&1&-1&1&0\\0&0&0&2&-6\\0&0&0&1&-3\end{pmatrix}=\boldsymbol{B}_3$$

$$\overset{\substack{r_3\leftrightarrow r_4\\ r_4-2r_3}}{\sim}\begin{pmatrix}1&1&-2&1&4\\0&1&-1&1&0\\0&0&0&1&-3\\0&0&0&0&0\end{pmatrix}=\boldsymbol{B}_4$$

$$\overset{\substack{r_1-r_2\\ r_2-r_3}}{\sim}\begin{pmatrix}1&0&-1&0&4\\0&1&-1&0&3\\0&0&0&1&-3\\0&0&0&0&0\end{pmatrix}=\boldsymbol{B}_5$$

经过一些列初等行变换之后，在矩阵 $B_4$ 和 $B_5$ 中可以看出：可以画一条从第一行某元左方的竖线开始到最后一列某元下方的横线结束的阶梯线，阶梯线的左下方的元全为 0；每段竖线的高度为一行，竖线的右方的第一个元为非零元，称为该非零行的首非零元。我们把这具有种特点的矩阵称为**行阶梯形矩阵**。在这种矩阵里，非零行在零行上面；非零行的首非零元所在列在上一行（如果存在）的首非零元所在列的右边。

若 A 是行阶梯形矩阵，并且非零行的首非零元为 1，首非零元所在列的其他元均为 0，则称 A 为**行最简形矩阵**。

可以看出，$B_4$ 和 $B_5$ 都是行阶梯形矩阵，并且 $B_5$ 也是行最简形矩阵。

实际上，对于任何非零矩阵 A，总可以经过有限次初等行变换把它变为行阶梯形矩阵和行最简形矩阵。这是一种非常重要的矩阵运算，在解线性方程组的时候，只需要把对应的增广矩阵化为行最简形矩阵。

对行最简形矩阵 $B_5$ 进一步进行初等列变换，可以得到一种形状更简单的矩阵，即**矩阵的标准形**：

$$\boldsymbol{B}_5=\begin{pmatrix}1&0&-1&0&4\\0&1&-1&0&3\\0&0&0&1&-3\\0&0&0&0&0\end{pmatrix}\overset{\substack{c_3\leftrightarrow c_4\\ c_4+c_1+c_2\\ c_5-4c_1-3c_2+3c_3}}{\sim}\begin{pmatrix}1&0&0&0&0\\0&1&0&0&0\\0&0&1&0&0\\0&0&0&0&0\end{pmatrix}=\boldsymbol{F}$$

F 称为矩阵 B 的标准形，其特点为：F 的左上角是一个单位矩阵，其余元全为 0。

对于矩阵 Am×n，总可以经过初等变换把它化为标准形：

$$\boldsymbol{F}=\begin{pmatrix}\boldsymbol{E}_r&\boldsymbol{O}\\\boldsymbol{O}&\boldsymbol{O}\end{pmatrix}_{m\times n}$$

在标准形 F 中，m，n，r 是完全确定的，其中 r 就是行阶梯形矩阵中非零行的行数。在所有与 A 等价的矩阵的集合中，标准形 F 是最简单的矩阵。

在介绍矩阵的初等变换性质前，我们先来学习一下初等矩阵的概念。

由单位矩阵 E 经过一次初等变换得到的矩阵称为**初等矩阵**，三种初等变换对应的初等矩阵如下：

1，把单位矩阵中的第 i，j 两行（或两列）对换，得初等矩阵：

$$E(i,j)=\begin{pmatrix} 1 & & & & & & & \\ & \ddots & & & & & & \\ & & 1 & & & & & \\ & & & 0 & \cdots & 1 & & \\ & & & & 1 & & & \\ & & & \vdots & \ddots & \vdots & & \\ & & & & & 1 & & \\ & & & 1 & \cdots & 0 & & \\ & & & & & & 1 & \\ & & & & & & & \ddots \\ & & & & & & & & 1 \end{pmatrix}\begin{matrix} \\ \\ \\ \leftarrow 第\,i\,行 \\ \\ \\ \\ \leftarrow 第\,j\,行 \\ \\ \\ \end{matrix},$$

而矩阵 $A_{m\times n}$ 中的第 i 行和第 j 行进行初等行变换（$r_i \leftrightarrow r_j$），等同于把 A 左乘一个 m 阶初等矩阵 Em(i,j)，即：

$$E_m(i,j)A=\begin{pmatrix} a_{11} & a_{12} & \cdots a_{1n} \\ \vdots & \vdots & \vdots \\ a_{j1} & a_{j2} & \cdots a_{jn} \\ \vdots & \vdots & \vdots \\ a_{i1} & a_{i2} & \cdots a_{in} \\ \vdots & \vdots & \vdots \\ a_{m1} & a_{m2} & \cdots a_{mn} \end{pmatrix}\begin{matrix} \\ \\ \leftarrow 第\,i\,行 \\ \\ \leftarrow 第\,j\,行 \\ \\ \\ \end{matrix}.$$

类似的，矩阵 $A_{m\times n}$ 中的第 i 列和第 j 列进行初等列变换，等同于把 A 右乘一个 n 阶初等矩阵 $E_n(i,j)$。

2，以数 $k\neq 0$ 乘单位矩阵第 i 行（列），得初等矩阵：

$$E(i(k))=\begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & & & \\ & & & k & & \\ & & & & 1 & \\ & & & & & \ddots \\ & & & & & & 1 \end{pmatrix}\begin{matrix} \\ \\ \\ \leftarrow 第\,i\,行 \\ \\ \\ \\ \end{matrix},$$

矩阵 $A_{m\times n}$ 中第 i 行乘以数 k 的初等变换（$r_i\times k$）等同于以 $E_m(i(k))$左乘以矩阵 A；第 i 列乘以数 k 的初等变换，等同于 $E_n(i(k))$右乘以矩阵 A。

3，以 k 乘单位矩阵的第 j 行（列）加到第 i 行（列）上，得初等矩阵：

$$E(ij(k))=\begin{pmatrix} 1 & & & & & \\ & \ddots & & & & \\ & & 1 & \cdots & k & \\ & & & \ddots & \vdots & \\ & & & & 1 & \\ & & & & & \ddots \\ & & & & & & 1 \end{pmatrix}\begin{matrix} \\ \\ \leftarrow 第\,i\,行 \\ \\ \leftarrow 第\,j\,行 \\ \\ \\ \end{matrix}.$$

矩阵 $A_{m\times n}$ 中第 j 行乘以数 k 加到第 i 行的初等变换（$r_i+kr_j$）等同于以 $E_m(ij(k))$左乘以矩阵 A；第 j 列乘以数 k 加到第 i 列的初等变换，等同于 $E_n(ij(k))$右乘以矩阵 A。

通过上面三类初等矩阵的定义和分析，那么对 Am×n 进行一次初等行变换，相当于在 A

的左边乘相应的 m 阶初等矩阵；对 A 进行一次初等列变换，相当于在 A 的右边乘相应的 n 阶初等矩阵。这样，我们就把矩阵的初等变换与矩阵的乘法联系在一起了。

设 A 和 B 为 m×n 的矩阵，那么有：

$A \overset{r}{\sim} B$ 的充分必要条件为：存在 m 阶可逆矩阵 P，使 PA=B；

$A \overset{c}{\sim} B$ 的充分必要条件为：存在 n 阶可逆矩阵 Q，使 AQ=B；

$A \sim B$ 的充分必要条件为：存在 m 阶和 n 阶可逆矩阵 P 和 Q，使 PAQ=B。

方阵 A 可逆的充分必要条件是：$A \overset{r}{\sim} E$，即 A 和 E 行等价。

这是因为：A 可逆⇔存在可逆矩阵，使 PA=E⇔$A \overset{r}{\sim} E$（⇔表示等价于）。

如果$A \overset{r}{\sim} B$，那么存在可逆矩阵 P，使得 PA=B。那么，如何计算出这个可逆矩阵 P 呢？由：

$$PA=B，PE=P⇔P(A,E)=(B,P) ⇔(A,E) \overset{r}{\sim} (B,P)$$

可知，对矩阵(A, E)作初等行变换，那么把 A 变为 B 时，E 就变为 P，即为我们要求的可逆矩阵 P；如果 B 为 E，那么由于 PA=B=E 可知，P 就是 A 的可逆矩阵，即 $A^{-1}=P$，这也给我们提供了求可逆矩阵的一种方法。

练习 5：设 $A=\begin{pmatrix} 0 & -2 & 1 \\ 3 & 0 & -2 \\ -2 & 3 & 0 \end{pmatrix}$，证明 A 可逆，并求 $A^{-1}$。

解：初等行变换把(A, E)化成(F, P)，其中 F 为 A 的行最简形矩阵。如果 F=E，那么 $A \overset{r}{\sim} E$，因此 A 可逆，并且由于 PA=E，所以 $P=A^{-1}$，即 P 就是 A 的逆矩阵。因此对(A, E)有：

$$(A,E)=\begin{pmatrix} 0 & -2 & 1 & 1 & 0 & 0 \\ 3 & 0 & -2 & 0 & 1 & 0 \\ -2 & 3 & 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\overset{\substack{r_3\times 3 \\ r_3+2r_2 \\ r_1\leftrightarrow r_2}}{\sim}\begin{pmatrix} 3 & 0 & -2 & 0 & 1 & 0 \\ 0 & -2 & 1 & 1 & 0 & 0 \\ 0 & 9 & -4 & 0 & 2 & 3 \end{pmatrix}\overset{r_3\times 2}{\underset{r_3+9r_2}{\sim}}\begin{pmatrix} 3 & 0 & -2 & 0 & 1 & 0 \\ 0 & -2 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 9 & 4 & 6 \end{pmatrix}$$

$$\overset{\substack{r_1+2r_3 \\ r_2-r_3}}{\sim}\begin{pmatrix} 3 & 0 & 0 & 18 & 9 & 12 \\ 0 & -2 & 0 & -8 & -4 & -6 \\ 0 & 0 & 1 & 9 & 4 & 6 \end{pmatrix}\overset{r_1\div 3}{\underset{r_2\div(-2)}{\sim}}\begin{pmatrix} 1 & 0 & 0 & 6 & 3 & 4 \\ 0 & 1 & 0 & 4 & 2 & 3 \\ 0 & 0 & 1 & 9 & 4 & 6 \end{pmatrix}$$

因为 F=E，所以 $A \overset{r}{\sim} E$，因此 A 可逆，并且，$A^{-1}=\begin{pmatrix} 6 & 3 & 4 \\ 4 & 2 & 3 \\ 9 & 4 & 6 \end{pmatrix}$.

## 9.2.7 矩阵的秩

在矩阵 $A_{m\times n}$ 中，任意选择 k 行和 k 列（k≤m，k≤n），位于这些行列交叉处的 $k^2$ 个元

素，不改变这些元素在 A 中的位置次序而得的 k 阶行列式，称为矩阵 A 的 k 阶子式。

比如，对于如下矩阵：

$$\begin{pmatrix} 1 & 1 & -2 & 1 & 4 \\ 0 & 1 & -1 & 1 & 0 \\ 0 & 0 & 0 & 1 & -3 \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

取第 1, 2, 3 行和第 1, 2, 4 列得到一个 3 阶非零子式：

$$\begin{vmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{vmatrix}$$

而它的任一 4 阶子式都将含有零行而为 0。因此，此矩阵的最高阶数是 3。

因此，矩阵 A 中，如果有一个不等于 0 的 r 阶子式 D，且所有 r+1 阶子式（如果存在）全等于 0，那么 D 称为矩阵 A 的最高阶非零子式，数 r 称为矩阵 A 的秩，记作 R(A)。

我们规定零矩阵的秩等于 0。

对于 n 阶方阵 A，当|A|≠0 时，R(A)等于 n；当|A|=0 时，R(A)小于 n。因为|A|≠0 时矩阵 A 可逆，因此逆矩阵的秩等于矩阵的阶数，逆矩阵又称为满秩矩阵，不可逆矩阵的秩小于矩阵的阶数，不可逆矩阵（奇异矩阵）又称为降秩矩阵。

矩阵的秩有如下一些性质：

- $0 \leqslant R(A_{m\times n}) \leqslant \min\{m, n\}$
- $R(A^T)=R(A)$
- 若 A~B，则 R(A)=R(B)
- 若 P, Q 可逆，则 R(PAQ)=R(A)
- $\max\{R(A), R(B)\} \leqslant R(A, B) \leqslant R(A)+R(B)$
- $R(A+B) \leqslant R(A)+R(B)$
- $R(AB) \leqslant \min\{R(A), R(B)\}$
- $A_{m\times n}B_{n\times l}=O$，则 $R(A)+R(B) \leqslant n$
- 若 AB=O，且 A 为满秩矩阵，则 B=O

练习 6：设有矩阵 A 和 b，求 R(A)和 R(A, b)。

$$\boldsymbol{A}=\begin{pmatrix} 1 & -2 & 2 & -1 \\ 2 & -4 & 8 & 0 \\ -2 & 4 & -2 & 3 \\ 3 & -6 & 0 & -6 \end{pmatrix}, \boldsymbol{b}=\begin{pmatrix} 1 \\ 2 \\ 3 \\ 4 \end{pmatrix}$$

解：设 B=(A, b)，对 B 进行初等行变换为行阶梯形矩阵：

$$B=\begin{pmatrix}1&-2&2&-1&1\\2&-4&8&0&2\\-2&4&-2&3&3\\3&-6&0&-6&4\end{pmatrix}\overset{\substack{r_2-2r_1\\r_3+2r_1\\r_4-3r_1}}{\sim}\begin{pmatrix}1&-2&2&-1&1\\0&0&4&2&0\\0&0&2&1&5\\0&0&-6&-3&1\end{pmatrix}$$

$$\overset{\substack{r_2\div 2\\r_3-r_2\\r_4+3r_2}}{\sim}\begin{pmatrix}1&-2&2&-1&1\\0&0&2&1&0\\0&0&0&0&5\\0&0&0&0&1\end{pmatrix}\overset{\substack{r_3\div 5\\r_4-r_3}}{\sim}\begin{pmatrix}1&-2&2&-1&1\\0&0&2&1&0\\0&0&0&0&1\\0&0&0&0&0\end{pmatrix},$$

则有 R(A)=2, R(A, b)=R(B)=3。

## 9.2.8 线性方程组解

现在我们来讨论线性方程组的解。设有 n 个未知数 m 个方程的线性方程组如下：

$$\begin{cases}a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=b_1,\\a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=b_2,\\\cdots\cdots\cdots\cdots\\a_{m1}x_1+a_{m2}x_2+\cdots+a_{mn}x_n=b_m,\end{cases}$$

上述方程组可以写成向量方程：

$$Ax=b$$

其中 $A=(a_{ij}),x=\begin{pmatrix}x_1\\x_2\\\vdots\\x_n\end{pmatrix},b=\begin{pmatrix}b_1\\b_2\\\vdots\\b_m\end{pmatrix}$。

如果该线性方程组有解，则称它是相容的；如果无解，则称它是不相容的。那么如何判断一个线性方程组有解和无解的情况呢？可以利用系数矩阵 A 和增广矩阵 B=(A, b)的秩来分析线性方程组的解的情况。对于 Ax=b，如果：

- R(A)<R(A,b)⇔无解
- R(A)=R(A,b)=n⇔有唯一解
- R(A)=R(A,b)<n⇔有无限多解

练习 7：解下列齐次线性方程组。

$$\begin{cases}x_1+2x_2+2x_3+x_4=0\\2x_1+x_2-2x_3-2x_4=0\\x_1-x_2-4x_3-3x_4=0\end{cases}$$

解：对系数矩阵 A 进行初等行变换为最简形矩阵，过程为：

$$A=\begin{pmatrix}1&2&2&1\\2&1&-2&-2\\1&-1&-4&-3\end{pmatrix}\overset{\substack{r_2-2r_1\\r_3-r_1}}{\sim}\begin{pmatrix}1&2&2&1\\0&-3&-6&-4\\0&-3&-6&-4\end{pmatrix}$$

$$\overset{\substack{r_3-r_2\\r_2\div(-3)}}{\sim}\begin{pmatrix}1&2&2&1\\0&1&2&\frac{4}{3}\\0&0&0&0\end{pmatrix}\overset{r_1-2r_2}{\sim}\begin{pmatrix}1&0&-2&-\frac{5}{3}\\0&1&2&\frac{4}{3}\\0&0&0&0\end{pmatrix}.$$

R(A)=2<4(这里 n 的值为 4)，有无限多个解。根据最简形矩阵，有如下同解方程组：

$$\begin{cases}x_1-2x_3-5/3x_4=0\\x_2+2x_3+4/3x_4=0\end{cases}$$

于是有：

$$\begin{cases}x_1=2x_3+5/3x_4\\x_2=-2x_3-4/3x_4\end{cases}$$

由于 $x_3$ 和 $x_4$ 可以自由取值，所以令 $x_3=c_1$，$x_4=c_2$，则有解的参数形式：

$$\begin{cases}x_1=2c_1+\frac{5}{3}c_2,\\x_2=-2c_1-\frac{4}{3}c_2,\\x_3=c_1,\\x_4=c_2,\end{cases}$$

其中 $c_1$ 和 $c_2$ 是任意实数。或者把解写成向量形式：

$$\begin{pmatrix}x_1\\x_2\\x_3\\x_4\end{pmatrix}=\begin{pmatrix}2c_1+\frac{5}{3}c_2\\-2c_1-\frac{4}{3}c_2\\c_1\\c_2\end{pmatrix}=c_1\begin{pmatrix}2\\-2\\1\\0\end{pmatrix}+c_2\begin{pmatrix}\frac{5}{3}\\-\frac{4}{3}\\0\\1\end{pmatrix}.$$

练习 8：求解非齐次线性方程组：

$$\begin{cases}x_1+x_2-3x_3-x_4=1\\3x_1-x_2-3x_3+4x_4=4\\x_1+5x_2-9x_3-8x_4=0\end{cases}$$

解：对增广矩阵 B 做初等行变换为最简形矩阵，过程如下：

$$B=\begin{pmatrix}1&1&-3&-1&1\\3&-1&-3&4&4\\1&5&-9&-8&0\end{pmatrix}\overset{\substack{r_2-3r_1\\r_3-r_1}}{\sim}\begin{pmatrix}1&1&-3&-1&1\\0&-4&6&7&1\\0&4&-6&-7&-1\end{pmatrix}$$

$$\overset{\substack{r_3+r_2\\r_2\div(-4)}}{\sim}\begin{pmatrix}1&1&-3&-1&1\\0&1&-\frac{3}{2}&-\frac{7}{4}&-\frac{1}{4}\\0&0&0&0&0\end{pmatrix}\overset{r_1-r_2}{\sim}\begin{pmatrix}1&0&-\frac{3}{2}&\frac{3}{4}&\frac{5}{4}\\0&1&-\frac{3}{2}&-\frac{7}{4}&-\frac{1}{4}\\0&0&0&0&0\end{pmatrix}$$

于是有：

$$\begin{cases}x_1=3/2x_3-3/4x_4+5/4\\x_2=3/2x_3+7/4x_4-1/4\\x_3=\quad x_3\\x_4=\qquad\quad x_4\end{cases}$$

最终的解（向量形式）：

$$\begin{pmatrix}x_1\\x_2\\x_3\\x_4\end{pmatrix}=c_1\begin{pmatrix}\frac{3}{2}\\\frac{3}{2}\\1\\0\end{pmatrix}+c_2\begin{pmatrix}-\frac{3}{4}\\\frac{7}{4}\\0\\1\end{pmatrix}+\begin{pmatrix}\frac{5}{4}\\-\frac{1}{4}\\0\\0\end{pmatrix}\quad(c_1,c_2\in\mathbb{R})$$

## 9.3 向量

在数学中，向量（vector，也称为欧几里得向量、几何向量、矢量），指具有大小和方向的量。它可以形象化地表示为带箭头的线段。箭头所指代表向量的方向；线段长度代表向量的大小。“向量”一词来自力学、解析几何中的有向线段。最先使用有向线段表示向量的是英国大科学家牛顿。与向量对应的量叫做数量（物理学中称标量），数量（或标量）只有大小，没有方向。

向量，最初被应用于物理学。很多物理量如力、速度、位移以及电场强度、磁感应强度等都是向量。大约公元前 350 年前，古希腊著名学者亚里士多德就知道了力可以表示成向量，两个力的组合作用可用著名的平行四边形法则来得到。

几何向量的概念在线性代数中经过抽象化，得到更一般的向量概念。此处向量定义为向量空间的元素。

在计算机领域，比如机器学习，人工智能等方向，向量都有重要的应用。

### 9.3.1 向量，向量组

在线性代数里，我们是这样定义向量的：n 个有次序的数 $a_1$，$a_2$，$a_3$，…，$a_n$ 所组成的数组称为 n 维向量，这 n 个数称为该向量的 n 个分量，第 i 个数 $a_i$ 称为第 i 个分量。

分量全为实数的向量称为实向量，分量都为复数的向量，称为复向量。

n 维向量写成一行，称为行向量；写成一列，称为列向量。比如：

行向量：

$$(a_1, a_2, \cdots, a_n)$$

列向量：

$$\begin{pmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{pmatrix}$$

行向量其实就是 $a_{1\times n}$ 的行矩阵，列向量其实就是 $a_{n\times 1}$ 的列矩阵。因此无论是行向量还是列向量，都可以按照矩阵的运算规则进行运算。

行向量和列向量是 2 个不同的向量。一般我们用 a，b，α，β 等表示列向量，用 $a^T$，$b^T$，$\alpha^T$，$\beta^T$ 表示行向量。

n 阶单位矩阵 $E=(e_1, e_2, e_3, \ldots e_n)$ 的列向量叫作 n 维单位坐标向量。

当向量的维度 $n\leqslant 3$ 时，如图 9-3 所示，可以用有向线段（既有大小又有方向）作为向量的几何形象：

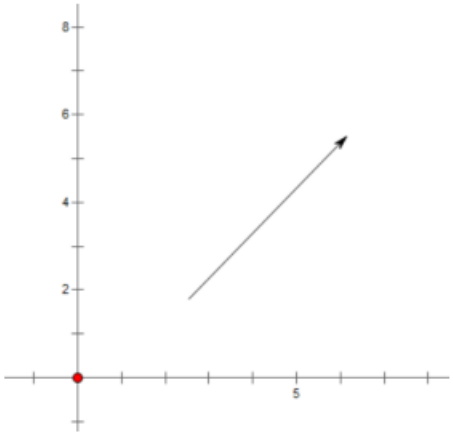



图 9-3 向量的几何形象

当 n>3 时，n 维向量就不再有这种几何形象了。

n 维向量的全体所组成的集合叫作 n 维向量空间，记作：

$$\mathbb{R}^n = \{\boldsymbol{x} = (x_1, x_2, \cdots, x_n)^T \mid x_1, x_2, \cdots, x_n \in \mathbb{R}\}$$

若干个同维数的列向量（或行向量）组成的集合叫作向量组。

一个 m× n 矩阵 A 可以看成是 m 个 n 维行向量或者 n 个 m 维列向量所组成的向量组，即含有有限个向量的有序（向量与向量之间位置不变）向量组可以与矩阵一一对应。

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ a_{31} & a_{32} & \cdots & a_{3n} \\ \cdots & \cdots & & \cdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

## 9.3.2 线性相关性

对于一个给定的向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$，对于任何一组实数 $k_1$，$k_2$，$k_3$，…，$k_m$，表达式：

$$k_1a_1+k_2a_2+k_3a_3+\ldots+k_ma_m$$

称为向量组 A 的一个线性组合，$k_1$，$k_2$，$k_3$，…，$k_m$ 称为这个线性组合的系数。

给定向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 和向量 b，如果存在一组数 $\lambda_1$，$\lambda_2$，$\lambda_3$，…，$\lambda_m$，使得：

$$b= \lambda_1a_1+\lambda_2a_2+\lambda_3a_3+\ldots+\lambda_ma_m$$

那么向量 b 是向量组 A 的线性组合，并称向量 b 能由向量组 A 线性表示。向量 b 能由向量组 A 线性表示，其实就是方程组：

$$x_1\boldsymbol{a}_1+x_2\boldsymbol{a}_2+\cdots+x_m\boldsymbol{a}_m=\boldsymbol{b}$$

有解。根据本书 9.2.8 节介绍的线性方程组有解的条件，可以得出如下定理：

向量 b 能由向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性表示的充分必要条件是矩阵 A($a_1$, $a_2$, $a_3$, ..., $a_m$)的秩等于矩阵 B=($a_1$, $a_2$, $a_3$, ..., $a_m$, b)的秩。

给定两个向量组 A 和 B，若 B 中的每个向量都能由向量组 A 线性表示，则称向量组 B 能由向量组 A 线性表示，若向量组 A 与向量组 B 能互相线性表示，则称这两个向量组等价。

矩阵 A 与 B 如果行等价，即 A 经初等行变换变成 B，那么 A 的行向量组与 B 的行向量组等价；类似的，矩阵 A 与 B 如果列等价，则 A 的列向量组与 B 的列向量组等价。

向量组 B：$b_1$，$b_2$，$b_3$，…，$b_l$ 能由向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性表示含义就是矩阵方程：

$$(\boldsymbol{a}_1,\boldsymbol{a}_2,\cdots,\boldsymbol{a}_m)\boldsymbol{X}=(\boldsymbol{b}_1,\boldsymbol{b}_2,\cdots,\boldsymbol{b}_l)$$

有解。因此，根据有解的条件有：

向量组 B：$b_1$，$b_2$，$b_3$，…，$b_l$ 能由向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性表示的充分必要条件是矩阵 A=($a_1$，$a_2$，$a_3$，…，$a_m$)的秩等于矩阵(A, B)= ($a_1$，$a_2$，$a_3$，…，$a_m$，$b_1$，$b_2$，$b_3$，…，$b_l$)的秩，即 R(A)=R(A, B)。

那么很容易得出向量组 A 和 B 等价的充分必要条件为：

$$R(A)=R(B)=R(A, B)$$

如果向量组 B：$b_1$，$b_2$，$b_3$，…，$b_l$ 能由向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性表示那么 $R(B) \leqslant R(A)$，这是 B 能由 A 线性表示的必要条件。

给定一个向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$，如果存在不全为零的数 $k_1$，$k_2$，$k_3$，…，$k_m$，使得：

$$k_1a_1+k_2a_2+k_3a_3+\ldots+k_ma_m=0$$

则称向量组 A 是线性相关的，否则称它线性无关。

向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 构成矩阵 A=($a_1$，$a_2$，$a_3$，…，$a_m$)，向量组线性相关就是齐次线性方程：

$$x_1\boldsymbol{a}_1+x_2\boldsymbol{a}_2+\cdots+x_m\boldsymbol{a}_m=\boldsymbol{0}$$

即 Ax=0 有非零解。根据齐次线性方程有非零解的性质有：

向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性相关的充分必要条件是它所构成的矩阵 A=($a_1$，$a_2$，$a_3$，…，$a_m$)的秩小于向量个数 m；向量组 A 线性无关的充分必要条件是 R(A)=m。

线性相关性是向量组的一个重要性质，关于向量组的线性相关性，还有如下一些结论：

- 若向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性相关，则向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$，$a_{m+1}$ 也线性相关；反之，若向量组 B 线性无关，则向量组 A 也线性无关。
- m 个 n 维向量做成的向量组，当维数 n 小于向量个数 m 时一定线性相关，特别地，n+1 个 n 维向量一定线性相关。
- 若向量组 A：$a_1$，$a_2$，$a_3$，…，$a_m$ 线性无关，而向量组 B：$a_1$，$a_2$，$a_3$，…，$a_m$，b 线性相关，则 b 必能由向量组 A 线性表示，且表示唯一。

## 9.3.4 向量组的秩

设有向量组 A，如果在 A 中能选出 r 个向量 $a_1$，$a_2$，$a_3$，…，$a_r$，并且：

1：向量组 $A_0$：$a_1$，$a_2$，$a_3$，…，$a_r$ 是线性无关的；

2：向量组 A 中任意 r+1 个向量（如有 r+1 个向量）都线性相关。

那么称向量组 $A_0$ 是向量组 A 的一个最大线性无关向量组，并简称为最大无关组。最大无关组所含有的向量个数 r 称为向量组 A 的秩，记作 $R_A$ 或 R($a_1$，$a_2$，$a_3$，…，$a_m$)。容易证明：向量组 A 的最大无关组 $A_0$ 和其自身是等价的。

特别地，只含零向量的向量组没有最大无关组，并规定它的秩为 0。

含有有限个 n 维向量的向量组 $a_1$，$a_2$，$a_3$，…，$a_m$ 可以构成矩阵 A=( $a_1$，$a_2$，$a_3$，…，$a_m$)，实际上，向量组的秩也等于矩阵的秩，即：

矩阵的秩等于它的列向量的秩，也等于它的行向量的秩。

## 9.3.5 线性方程组的解结构

现在我们来研究线性方程组的解结构。对于线性齐次方程组：

$$\begin{cases} a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n=0, \\ a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n=0, \\ \cdots\cdots\cdots \\ a_{m1}x_1+a_{m2}x_2+\cdots+a_{mn}x_n=0, \end{cases}$$

记：

$$\boldsymbol{A}=\begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix},\ \boldsymbol{x}=\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix},$$

则上述线性齐次方程组可以改写为向量方程：

$$\mathbf{Ax=0}$$

若 $x_1=\xi_{11}$，$x_2=\xi_{21}$，$x_3=\xi_{31}$，…，$x_n=\xi_{n1}$ 是上述方程的解，则称：

$x=\xi_1=\begin{pmatrix} \xi_{11} \\ \xi_{21} \\ \vdots \\ \xi_{n1} \end{pmatrix}$ 为该方程组的解向量。

容易证明：若 $\xi_1$ 和 $\xi_2$ 是上述向量方程的解，那么 $k\xi_1$（k 为任意实数），$\xi_1+\xi_2$ 都是向量方程的解。

设上述向量方程的全体解向量所组成的集合为 S，如果能求得 S 的一个最大无关组 $S_0$：$\xi_1$，$\xi_2$，$\xi_3$，…，$\xi_t$，那么该向量方程的任一解都可由最大无关组 $S_0$ 线性表示，即：

$$x=k_1\xi_1+k_2\xi_2+k_3\xi_3+\ldots+k_t\xi_t$$

称为齐次线性方程组的通解，而 $S_0$ 称为线性方程组的基础解系。要求齐次线性方程组的通解，只需要求出它的基础解系。需要注意的是，齐次线性方程组的基础解系和通解都不是唯一的。

设 m×n 矩阵 A 的秩 R(A)=r，则 n 元齐次线性方程组 Ax=0 的解集 S 的秩 $R_s=n-r$。

练习 9：求下列齐次线性方程组的基础解系和通解。

$$\begin{cases} x_1+x_2-x_3-x_4=0, \\ 2x_1-5x_2+3x_3+2x_4=0, \\ 7x_1-7x_2+3x_3+x_4=0 \end{cases}$$

解：

$$A=\begin{pmatrix}1&1&-1&-1\\2&-5&3&2\\7&-7&3&1\end{pmatrix}\xrightarrow[r_3-7r_1]{r_2-2r_1}\begin{pmatrix}1&1&-1&-1\\0&-7&5&4\\0&-14&10&8\end{pmatrix}$$

$$\xrightarrow{r_3-2r_2}\begin{pmatrix}1&1&-1&-1\\0&-7&5&4\\0&0&0&0\end{pmatrix}\xrightarrow[r_1-r_2]{r_2\div(-7)}\begin{pmatrix}1&0&-\frac{2}{7}&-\frac{3}{7}\\0&1&-\frac{5}{7}&-\frac{4}{7}\\0&0&0&0\end{pmatrix},$$

因此有：

$$\begin{cases} x_1=\frac{2}{7}x_3+\frac{3}{7}x_4, \\ x_2=\frac{5}{7}x_3+\frac{4}{7}x_4, \end{cases}$$

其中 $x_3$ 和 $x_4$ 是自由变量，可以任意取值，从而得到不同的基础解系。比如令 $x_3$ 和 $x_4$ 分别取值(1, 0)和(0, 1)，$x_1$ 和 $x_2$ 则有(2/7, 5/7)和(3/7, 4/7)，于是基础解系为：

$$\xi_1=\begin{pmatrix}\frac{2}{7}\\\frac{5}{7}\\1\\0\end{pmatrix},\quad \xi_2=\begin{pmatrix}\frac{3}{7}\\\frac{4}{7}\\0\\1\end{pmatrix},$$

通解为：

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = c_1 \begin{pmatrix} \frac{2}{7} \\ \frac{5}{7} \\ 1 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} \frac{3}{7} \\ \frac{4}{7} \\ 0 \\ 1 \end{pmatrix} \quad (c_1, c_2 \in \mathbb{R})$$

对于非齐次线性方程组：

$$\begin{cases} a_{11}x_1 + a_{12}x_2 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + \cdots + a_{2n}x_n = b_2, \\ \cdots\cdots\cdots \\ a_{m1}x_1 + a_{m2}x_2 + \cdots + a_{mn}x_n = b_m, \end{cases}$$

与齐次方程类似，也可以写为向量方程：

**Ax=b**

容易证明：如果 $\eta_1$ 和 $\eta_2$ 是上述向量方程的解，则 $\eta_1$-$\eta_2$ 为对应的齐次线性方程组

**Ax=0**

的解。

若 η 是 Ax=b 的解，ξ 是 Ax=0 的解，那么 ξ+η 仍是方程 Ax=b 的解。

设 $\xi_1$，$\xi_2$，$\xi_3$，…，$\xi_{n-r}$ 是 Ax=0 的基础解系，η*为 Ax=b 的一个解（称为特解），那么 Ax=b 的通解为：

**$x = k_1\xi_1 + k_2\xi_2 + k_3\xi_3 + \ldots + k_{n-r}\xi_{n-r} + \eta^*$**

也就是说：非齐次线性方程组的通解=对应的齐次方程的通解+非齐次方程的一个特解。

练习 10：求解如下线性方程组。

$$\begin{cases} x_1 - x_2 - x_3 + x_4 = 0, \\ x_1 - x_2 + x_3 - 3x_4 = 1, \\ x_1 - x_2 - 2x_3 + 3x_4 = -\frac{1}{2}. \end{cases}$$

解：对该线性方程组的增广矩阵 B 进行初等行变换：

$$B = \begin{pmatrix} 1 & -1 & -1 & 1 & 0 \\ 1 & -1 & 1 & -3 & 1 \\ 1 & -1 & -2 & 3 & -\frac{1}{2} \end{pmatrix} \overset{\substack{r_2 - r_1 \\ r_3 - r_1}}{\sim} \begin{pmatrix} 1 & -1 & -1 & 1 & 0 \\ 0 & 0 & 2 & -4 & 1 \\ 0 & 0 & -1 & 2 & -\frac{1}{2} \end{pmatrix} \overset{\substack{r_1 - r_3 \\ r_2 + 2 \\ r_3 + r_2}}{\sim} \begin{pmatrix} 1 & -1 & 0 & -1 & \frac{1}{2} \\ 0 & 0 & 1 & -2 & \frac{1}{2} \\ 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

R(A)=R(B)，方程组有解，并且有：

$$\begin{cases} x_1 = x_2 + x_4 + \frac{1}{2}, \\ x_3 = 2x_4 + \frac{1}{2}. \end{cases}$$

$x_2$ 和 $x_4$ 是自由变量，取 $x_2=x_4=0$，则有 $x_1=x_3=1/2$。于是该非齐次线性方程组的一个解为：

$$\eta^* = \begin{pmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \\ 0 \end{pmatrix},$$

该非齐次线性方程对应的齐次线性方程为：

$$\begin{cases} x_1 = x_2 + x_4 \\ x_3 = 2x_4 \end{cases}$$

令 $x_2$=1 和 0，$x_4$=0 和 1，则 $x_1$=1 和 1，$x_3$=0 和 2，于是齐次线性方程组的基础解系为：

$$\xi_1 = \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix}, \xi_2 = \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \end{pmatrix}$$

所以，该非齐次线性方程组的通解为：

$$\begin{pmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{pmatrix} = c_1 \begin{pmatrix} 1 \\ 1 \\ 0 \\ 0 \end{pmatrix} + c_2 \begin{pmatrix} 1 \\ 0 \\ 2 \\ 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} \\ 0 \\ \frac{1}{2} \\ 0 \end{pmatrix} \quad (c_1, c_2 \in \mathbb{R})$$

## 9.3.6 向量空间

设 V 为 n 维向量的集合，如果集合 V 非空，且集合 V 对于向量的加法和数乘运算封闭，那么就称集合 V 为向量空间。这里的封闭，是指集合 V 中可以进行向量的加法及数乘运算，也就是说，如果 a∈V，b∈V，λ∈R，则 a+b∈V，λa∈V，即 a+b 和 λa 的结果都不会超出集合 V 的范围。

容易证明：n 维向量的全体 $R^n$ 就是一个向量空间，n 元齐次线性方程组的解集也是一个向量空间。

而对于如下集合：

$$V = \{x = (1, x_2, \cdots, x_n)^T \mid x_2, \cdots, x_n \in \mathbb{R}\}$$

就不是一个向量空间，因为若 $a=(1, a_2, a_3, \ldots, a_n)^T \in V$，而 $2a=(2, 2a_2, 2a_3, \ldots, 2a_n)^T \notin V$。

非齐次线性方程组 Ax=b 的解集 S，也不是向量空间。因为假设 S 为空（无解）则不是向量空间；假设 S 非空（有解），若 η∈S，则 A（2η）=2b≠b，所以 2η∉ V。

一般地，由向量组 $a_1$，$a_2$，$a_3$，…，$a_m$ 可以生成向量空间，记为：

$$L=\{x=\lambda_1 a_1+\lambda_2 a_2+\lambda_3 a_3+\ldots+\lambda_m a_m \mid \lambda_1, \lambda_2, \lambda_3, \ldots, \lambda_m \in R\}$$

设有向量空间 $V_1$ 和 $V_2$，若 $V_1 \subseteq V_2$，就称 $V_1$ 是 $V_2$ 的子空间。比如，任何有 n 维向量

组成的向量空间，都是$R^n$的子空间，即$V \subseteq R^n$。

设 V 为向量空间，如果有 r 个向量$a_1$，$a_2$，$a_3$，…，$a_r \in V$，且满足：

1：$a_1$，$a_2$，$a_3$，…，$a_r$线性无关；

2：V 中任何一向量都可由$a_1$，$a_2$，$a_3$，…，$a_r$线性表示。

则称$a_1$，$a_2$，$a_3$，…，$a_r$为向量空间 V 的一个基，r 称为向量空间的维数，并称 V 为 r 维向量空间。

如果向量空间 V 没有基，维数记为 0，只包含一个零向量。

设有向量组$a_1$，$a_2$，$a_3$，…，$a_m$生成的向量空间：

$$L=\{x=\lambda_1 a_1+\lambda_2 a_2+\lambda_3 a_3+\ldots+\lambda_m a_m \mid \lambda_1,\ \lambda_2,\ \lambda_3,\ \ldots,\ \lambda_m \in R\}$$

那么向量空间 L 与向量组$a_1$，$a_2$，$a_3$，…，$a_m$等价，因此向量组$a_1$，$a_2$，$a_3$，…，$a_m$的最大无关组就是 L 的一个基，向量组$a_1$，$a_2$，$a_3$，…，$a_m$的秩就是 L 的维数。

设向量空间的一个基为$a_1$，$a_2$，$a_3$，…，$a_r$，则 V 可表示为：

$$V=\{x=\lambda_1 a_1+\lambda_2 a_2+\lambda_3 a_3+\ldots+\lambda_r a_r \mid \lambda_1,\ \lambda_2,\ \lambda_3,\ \ldots,\ \lambda_r \in R\}$$

如果在向量空间 V 中取定一个基$a_1$，$a_2$，$a_3$，…，$a_r$，那么 V 中任何一向量 x 可以唯一的表示为：

$$x=\lambda_1 a_1+\lambda_2 a_2+\lambda_3 a_3+\ldots+\lambda_r a_r$$

其中$\lambda_1$，$\lambda_2$，$\lambda_3$，…，$\lambda_r$称为向量 x 在基$a_1$，$a_2$，$a_3$，…，$a_r$中的坐标。

如果在 n 维向量空间$R^n$中取单位坐标向量组$e_1$，$e_2$，$e_3$，…，$e_n$为基，则以$x_1$，$x_2$，$x_3$，…，$x_n$为分量的向量 x 可表示为：

$$x=x_1e_1+x_2e_2+x_3e_3+\ldots+x_ne_n$$

即向量在基$e_1$，$e_2$，$e_3$，…，$e_n$中的坐标就是该向量的分量。基$e_1$，$e_2$，$e_3$，…，$e_n$被称为$R^n$中的自然基。

在$R^3$中取一个基$a_1$，$a_2$，$a_3$，再取一个基$b_1$，$b_2$，$b_3$，并设$A=(a_1, a_2, a_3)$，$B=(b_1, b_2, b_3)$，那么有：

$$(a_1, a_2, a_3)=(e_1, e_2, e_3)A$$

等式两边同时右乘$A^{-1}$，于是有：

$$(e_1, e_2, e_3)=(a_1, a_2, a_3)A^{-1}$$

因此：

$$(b_1, b_2, b_3)=(e_1, e_2, e_3)B=(a_1, a_2, a_3)A^{-1}B$$

我们把式子：

$$(b_1, b_2, b_3)=(a_1, a_2, a_3)A^{-1}B$$

称为用$a_1, a_2, a_3$表示$b_1, b_2, b_3$的基变换公式，$P=A^{-1}B$称为从旧基$(a_1, a_2, a_3)$到新基$(b_1, b_2, b_3)$的过渡矩阵。

设向量 x 在旧基中的坐标分别为$y_1$，$y_2$，$y_3$，如何求它在新基中的坐标$z_1$，$z_2$，$z_3$呢？

根据定义有：

$$x=(a_1,a_2,a_3)\begin{pmatrix}y_1\\y_2\\y_3\end{pmatrix},\quad x=(b_1,b_2,b_3)\begin{pmatrix}z_1\\z_2\\z_3\end{pmatrix}$$

即：

$$A\begin{pmatrix}y_1\\y_2\\y_3\end{pmatrix}=B\begin{pmatrix}z_1\\z_2\\z_3\end{pmatrix}.$$

根据矩阵运算法则，等式两边同时左乘 $B^{-1}$，有：

$$\begin{pmatrix} z_1 \\ z_2 \\ z_3 \end{pmatrix} = \boldsymbol{B}^{-1}\boldsymbol{A}\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \boldsymbol{P}^{-1}\begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix}$$

这就是从旧坐标到新坐标的坐标变换公式，其中 P 为从旧基($a_1$, $a_2$, $a_3$)到新基($b_1$, $b_2$, $b_3$)的过渡矩阵。在进行坐标转换的时候，只需要求得过渡矩阵 P，和 $P^{-1}$，即可进行坐标的变换计算。

## 9.3.7 内积、长度和正交

在解析几何里，3 维向量 x 和 y 的数量积定义为：

$$\mathbf{x.y=|x|.|y|.\cos\theta}$$

其中|x|和|y|是向量 x 和 y 的模（长度），θ 是向量 x 和 y 的夹角。而在直角坐标系里，向量 x 和 y 的数量积计算公式为：

$$\mathbf{x.y=(x_1, x_2, x_3).(y_1, y_2, y_3)=x_1y_1+x_2y_2+x_3y_3}$$

在线性代数里，n 维向量的内积是数量积的一种推广。现在我们来看看内积的定义。

设有 n 维向量：

$$\boldsymbol{x}=\begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix},\quad \boldsymbol{y}=\begin{pmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{pmatrix},$$

令

$$\mathbf{[x, y]=x_1y_1+x_2y_2+x_3y_3+...+x_ny_n}$$

那么[x, y]称为向量 x 和 y 的内积。内积的计算结果是一个实数。如果 x 和 y 都是列向量，那么内积实际上等价于如下的矩阵运算：

$$\mathbf{[x, y]=x^Ty}$$

设 x 和 y 是 n 维向量，λ 为实数，内积有如下运算性质：

1 : [x, y]=[y, x]

2 :[λx, y]=λ[x, y]

3 :[x+y, z]=[x, z]+[y, z]

4 :当 x=0 时，[x, x]=0；当 x≠0 时，[x, x]>0

5 :$[x, y]^2 \leqslant [x, x][y, y]$，并称该不等式为施瓦茨不等式

令

$$||x||=\sqrt{[x, x]}=\sqrt{x_1^2+x_2^2+\cdots+x_n^2}$$

则称||x||为 n 维向量 x 的范数（或长度）。

向量的范数有如下性质：

1：非负性， 即当 x=0 时，||x||=0；当 x≠0 时，||x||>0

2：齐次性， 即||λx||=|λ|||x||

当||x||=1 时，x 称为单位向量；如果 a 不是一个单位向量且 a≠0，那么可以通过 a/||a||

得到一个单位向量，这个过程称为把向量 a 单位化。

根据施瓦茨不等式：

$$[x, y]^2 \leqslant [x, x][y, y]$$

当$[x, x][y, y] \neq 0$ 时有：

$$[x, y]^2/([x, x][y,y]) \leqslant 1$$

所以有：

$$-1 \leqslant \frac{[x, y]}{\|x\| \|y\|} \leqslant 1$$

这正好在一个三角余弦函数的取值范围内，因此当 $x \neq 0$，$y \neq 0$ 时，有如下定义：

$$\theta = \arccos \frac{[x, y]}{\|x\| \|y\|}$$

我们把 θ 称为 n 维向量 x 与 y 的夹角。

当$[x, y]=0$，即 $\theta = \pi/2$ 时，称向量 x 与 y 正交。根据正交定义，当 $x=0$ 时，它与任何向量都正交。

根据正交和线性无关的定义，容易证明：若 n 维向量 $a_1$，$a_2$，$a_3$，...，$a_r$ 是一组两两正交的非零向量，则 $a_1$，$a_2$，$a_3$，...，$a_r$ 线性无关。

设 n 维向量 $e_1$，$e_2$，$e_3$，...，$e_r$ 是向量空间 V（$V \subseteq R^n$）的一个基，如果 $e_1$，$e_2$，$e_3$，...，$e_r$ 两两正交，且都是单位向量，则称 $e_1$，$e_2$，$e_3$，...，$e_r$ 是 V 的一个标准正交基。

比如：

$$e_1 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \\ 0 \\ 0 \end{pmatrix}, \quad e_2 = \begin{pmatrix} \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \\ 0 \\ 0 \end{pmatrix}, \quad e_3 = \begin{pmatrix} 0 \\ 0 \\ \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}, \quad e_4 = \begin{pmatrix} 0 \\ 0 \\ \frac{1}{\sqrt{2}} \\ -\frac{1}{\sqrt{2}} \end{pmatrix}$$

标准正交基的一个优势之处在于很容易求出一个向量 a 在标准正交基里的坐标。设 $e_1$，$e_2$，$e_3$，...，$e_r$ 是 V 的一个标准正交基，那么 V 中的任一向量 a 可线性表示为：

$$\mathbf{a} = \lambda_1 \mathbf{e}_1 + \lambda_2 \mathbf{e}_2 + \lambda_3 \mathbf{e}_3 + \ldots + \lambda_r \mathbf{e}_r$$

如果用 $e_i^T$ 左乘上面等式两边，则有（因为正交，其它项都为 0 了）：

$$\mathbf{e}_i^T \mathbf{a} = \lambda_i \mathbf{e}_i^T \mathbf{e}_i = \lambda_i$$

也就是：

$$\lambda_i = [a, e_i]$$

上述公式即是向量在标准正交基中的坐标计算公式。通过这个公式，可以很方便的求出向量的坐标。

我们可以把一个向量空间的基 $a_1$，$a_2$，$a_3$，...，$a_r$ 转化为标准正交基，这个过程称为基的标准正交化。设 $a_1$，$a_2$，$a_3$，...，$a_r$ 是向量空间 V 的一个基，可以用下面的公式来完成它的标准正交化：

第一步：利用下面的公式从 $a_1$，$a_2$，$a_3$，...，$a_r$ 导出新的向量组 $b_1$，$b_2$，$b_3$，...，$b_r$

$$
\begin{aligned}
&\boldsymbol{b}_1=\boldsymbol{a}_1,\\
&\boldsymbol{b}_2=\boldsymbol{a}_2-\frac{[\boldsymbol{b}_1,\boldsymbol{a}_2]}{[\boldsymbol{b}_1,\boldsymbol{b}_1]}\boldsymbol{b}_1,\\
&\cdots\cdots\\
&\boldsymbol{b}_r=\boldsymbol{a}_r-\frac{[\boldsymbol{b}_1,\boldsymbol{a}_r]}{[\boldsymbol{b}_1,\boldsymbol{b}_1]}\boldsymbol{b}_1-\frac{[\boldsymbol{b}_2,\boldsymbol{a}_r]}{[\boldsymbol{b}_2,\boldsymbol{b}_2]}\boldsymbol{b}_2-\cdots-\frac{[\boldsymbol{b}_{r-1},\boldsymbol{a}_r]}{[\boldsymbol{b}_{r-1},\boldsymbol{b}_{r-1}]}\boldsymbol{b}_{r-1}
\end{aligned}
$$

容易证明 $b_1$，$b_2$，$b_3$，...，$b_r$ 两两正交，且与 $a_1$，$a_2$，$a_3$，...，$a_r$ 等价。这一正交化过程，称为施密特正交化。

第二步：把 $b_1$，$b_2$，$b_3$，...，$b_r$ 单位化，即：

$$
\boldsymbol{e}_1=\frac{1}{\|\boldsymbol{b}_1\|}\boldsymbol{b}_1,\ \boldsymbol{e}_2=\frac{1}{\|\boldsymbol{b}_2\|}\boldsymbol{b}_2,\ \cdots,\ \boldsymbol{e}_r=\frac{1}{\|\boldsymbol{b}_r\|}\boldsymbol{b}_r
$$

这样得到的 $e_1$，$e_2$，$e_3$，...，$e_r$ 就是 V 的一个标准正交基。

练习 11：将下面一组向量标准正交化。

$$
\boldsymbol{a}_1=\begin{pmatrix}1\\2\\-1\end{pmatrix},\boldsymbol{a}_2=\begin{pmatrix}-1\\3\\1\end{pmatrix},\boldsymbol{a}_3=\begin{pmatrix}4\\-1\\0\end{pmatrix}
$$

如果 n 阶方阵 A 满足：

$$A^TA=E$$

即 $A^{-1}=A^T$，那么称 A 为正交矩阵，简称正交阵。

根据正交阵的定义有：

$$
\begin{pmatrix}\boldsymbol{a}_1^T\\\boldsymbol{a}_2^T\\\vdots\\\boldsymbol{a}_n^T\end{pmatrix}(\boldsymbol{a}_1,\boldsymbol{a}_2,\cdots,\boldsymbol{a}_n)=\boldsymbol{E}
$$

也就是：

$$
\boldsymbol{a}_i^T\boldsymbol{a}_j=\begin{cases}1,\text{当 } i=j,\\0,\text{当 } i\neq j\end{cases}\quad(i,j=1,2,\cdots,n)
$$

这表明：方阵 A 为正交矩阵的充分必要条件是 A 的列向量和行向量都是单位向量，且两两正交。

下面的矩阵就是一个正交矩阵，它的列向量和行向量都是单位向量，且两两正交。

$$
\begin{pmatrix}\frac{1}{2}&-\frac{1}{2}&\frac{1}{2}&-\frac{1}{2}\\\frac{1}{2}&-\frac{1}{2}&-\frac{1}{2}&\frac{1}{2}\\\frac{1}{\sqrt{2}}&\frac{1}{\sqrt{2}}&0&0\\0&0&\frac{1}{\sqrt{2}}&\frac{1}{\sqrt{2}}\end{pmatrix}
$$

正交矩阵的意义在于求逆的代价非常小，计算容易。根据正交矩阵的定义，容易证明正交矩阵的如下性质：

1：如果 A 为正交矩阵，那么 $A^{-1}=A^T$ 也是正交矩阵，且|A|=1 或-1。

2：若 A 和 B 都是正交矩阵，那么 AB 也是正交矩阵

### 9.3.8 幺正矩阵

量子力学或者量子计算里，还会涉及到一个幺正矩阵的概念。那么什么是幺正矩阵呢？

首先我们从共轭转置矩阵开始讲起。

设复数域为 C，且矩阵$A \in C^{m\times n}$（$C^{m\times n}$表示元素属于复数域 C 的全体$m \times n$矩阵组成的集合），用$\overline{A}$表示以$A$中元素的共轭复数（参考本书 1.1 节中对复数的介绍）为元素组成的矩阵，按矩阵的加法和数与矩阵的数量乘法构成复数域 C 上的线性空间；则有：

$$A^H = \left(\overline{A}\right)^T$$

其中$\left(\overline{A}\right)^T$表示对$\overline{A}$取转置。$A^H$ 称为$A$ 的共轭转置矩阵。

如果一个 n 阶复数方阵 U 满足下列条件：

$U^HU=UU^H=E$, 其中 E 为 n 阶单位方阵，$U^H$ 为 U 的共轭转置矩阵，则称 U 为幺正矩阵或酉（yǒu）矩阵。

由幺正矩阵的定义可以看出，$U^{-1}=U^H$，如果 U 矩阵的元素都为实数，那么 U 矩阵就是正交矩阵。如果 U 是幺正矩阵，那么 $U^H$ 也是幺正矩阵。

### 9.3.9 线性变换

设有向量 x 和矩阵 P，用矩阵 P 左乘向量 x 得到一个新的向量 y，这一过程就是对 x 的线性变换，即：

$$y=Px$$

比如，对于一个 2×2 的矩阵 A 和一个二维列向量 x 的线性变换：

$$y=Ax=\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} x \\ y \end{bmatrix} = x\begin{bmatrix} a \\ c \end{bmatrix} + y\begin{bmatrix} b \\ d \end{bmatrix} = \begin{bmatrix} ax+by \\ cx+dy \end{bmatrix}$$

本质上，变换就是函数。输入一个向量，经过某个变换的作用之后，输出另一个向量。

在几何坐标上，变换让空间发生了扭曲。线性变换就在于，变换前后，所有的直线仍然是直线，变换前后，原点保持不变。

变换前的空间，如图 9-4 所示：

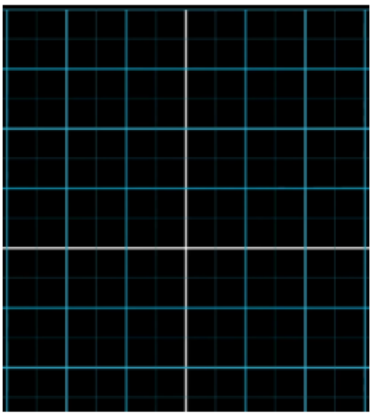


图 9-4 变换前的空间

变换后的空间，如图 9-5 所示：

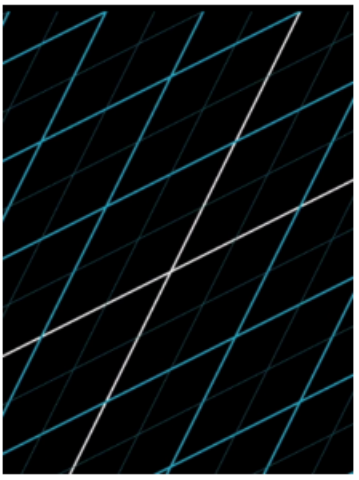


图 9-5 变换后的空间

设有向量 v=xi+yj，在某个特定线性变换的作用下，i，j 以及 v 都会因为线性变换而运动到新的位置。线性变换前后原点不变，网络线保持平行且等距分布，这使得线性变换后的向量 v 依然是变换后的 i 和 j 的线性组合，并且线性组合的系数和变换前一样。

因此，对于一个线性变换，只需要关注基向量在变换前后的变化，就可以确定空间中所有向量的变化。如果将线性变换后的基向量坐标按列组合起来，可以拼接成一个矩阵。线性变换的所有信息都包含在这个矩阵当中。

也就是说，对于线性变换，只需要知道原来的基向量在线性变换后的位置坐标，然后把它们按列组成一个矩阵，这个矩阵就是对应的线性变换矩阵。

对于上述定义中的线性变换：

$$y=Px$$

其中 P 是由原来的基向量在线性变换后的位置坐标按列组成的矩阵。这个矩阵为我们提供了一种描述线性变换的信息。线性变换作用于一个向量，对应于用线性变换矩阵左乘该向量。

若 P 为正交矩阵，x，y 为向量，则线性变换 y=Px 称为正交变换。由于：

$$\| y \| = \sqrt{y^{\mathrm{T}} y} = \sqrt{x^{\mathrm{T}} P^{\mathrm{T}} P x} = \sqrt{x^{\mathrm{T}} x} = \| x \|$$

所以，经正交变换后向量的长度保持不变，那么由向量构成的三角形的形状在正交变换

后也保持不变。这是正交变换的一个优良特性。

如果 P 为幺正矩阵，x，y 为向量，则线性变换 y=Px 称为幺正变换。幺正变化是保留内积的变换：变换之前的两个向量的内积等于其转换后的内积，即：

设 P 是幺正矩阵，x，y 是两个向量，则[x, y]=[Px, Py]。

## 9.3.10 空间的定义

在数学中，有很多不同的空间的定义，比如向量空间、内积空间、欧式空间以及希尔伯特空间等。现在我们来对这些空间做一个总结和整理。

首先，空间中会用到距离这样一个抽象概念。那么什么是距离呢？在生活中我们对距离的概念并不陌生。比如两个城市之间的距离，两个国家之间的距离。在数学理论中，距离是抽象的一个概念，除了直线距离，向量之间，函数之间，曲面之间都可以用到距离的概念。我们把距离做如下的定义：

设 X 为一个集合，一个映射 d：X×X→R。若对于任何 x, y, z 属于 X，有

1，（正定性）d(x, y）≥0，且 d(x, y)=0 当且仅当 x = y；

2，（对称性）d(x, y)=d(y, x）；

3，（三角不等式）d(x, z）≤d(x, y)+d(y, z）

则称 d 为集合 X 的一个度量（或距离）。称偶对（X，d）为一个度量空间，或者称 X 为一个对于度量 d 而言的度量空间。度量空间中最符合我们对于现实直观理解的是三维欧氏空间。这个空间中的欧几里德度量定义两点之间距离为连接这两点的线段的长度。

同样，我们还可以进一步定义范数的概念。在向量空间中，如果增加定义范数的概念，表示某点到空间零点的距离，即：

1，||x|| ≥0;

2，||ax||=|a|||x||;

3，||x+y||≤||x||+||y||。

将范数与距离比较，范数比距离多了第 2 个条件，即数乘的运算，因此这是一个强化了的距离概念。

## 9.3.11 线性空间、向量空间

向量空间亦称线性空间。设 V 是一个非空集合，P 是一个域(比如实数域 R 或者复数域 C）。若：

1.在 V 中定义了一种运算，称为加法，即对 V 中任意两个元素 α 与 β 都按某一法则对应于 V 内惟一确定的一个元素 α + β，称为 α 与 β 的和。

2.在 P 与 V 的元素间定义了一种运算，称为纯量乘法(亦称数量乘法)，即对 V 中任意元素 α 和 P 中任意元素 k，都按某一法则对应 V 内惟一确定的一个元素 kα，称为 k 与 α 的积。

3.加法与纯量乘法满足以下条件：

1) α + β = β + α，对任意 α， β ∈V.

2) α +( β + γ )=( α + β )+ γ，对任意 α， β， γ ∈V.

3) 存在一个元素 0∈V，对一切 α ∈V 有 α +0= α，元素 0 称为 V 的零元.

4) 对任一 α ∈V，都存在 β ∈V 使 α + β =0，β 称为 α 的负元素，记为- α .

5) 对 P 中单位元 1，有 1 α = α ( α ∈V).

6) 对任意 k，1∈P， α ∈V 有(kl) α =k(l α ).

7) 对任意 k，1∈P， α ∈V 有(k+l) α =k α +l α .

8) 对任意 k∈P， α ， β ∈V 有 k( α + β )=k α +k β ，

则称 V 为域 P 上的一个线性空间，或向量空间。V 中元素称为向量，V 的零元称为零向量，P 称为线性空间的基域。当 P 是实数域时，V 称为实线性空间；当 P 是复数域时，V 称为复线性空间。

在线性空间上加入范数概念，就是线性赋范空间。

## 9.3.12 内积空间与欧氏空间

内积空间是增添了一个额外的内积的线性赋范空间。内积又叫标量积，或点积。内积的加入使得我们可以计算向量的角度。这时，内积空间已经有了距离、长度、角度等概念。

设 V 是数域 P 上的线性空间，定义一个代数运算（V×V—>P），记为 (α, ß) 。如果(α, ß)满足下列条件：

1) 对称性：(α, ß) = (ß, α)；

2) 可加性：(α+ß, γ) = (α, γ) + (ß, γ)；

3) 齐次性：(kα, ß) = k(α, ß)；

4) 非负性：(α, α)≥0，当且仅当α=0 时(α, α)=0，

其中 k 是数域 P 中的任意数，α、ß、 γ 是 V 中的任意元素，则称(α, ß)为α与ß的内积，定义了内积的线性空间 V 称为内积空间，有限维的内积空间 V 为 Euclid 空间（欧式空间）。

## 9.3.13 希尔伯特空间

继续在内积空间上扩展，使得内积空间满足完备性，就形成了希尔伯特空间。完备性是指空间中的极限运算不能跑出该空间。

希尔伯特空间就类似于地球，无论你怎么走，都还在地球内。微积分中的大部分概念都可以无障碍地推广到希尔伯特空间中。希尔伯特空间成为泛函分析中最重要的和最常用的一类空间，也是公式化数学和量子力学的关键性概念之一。

总结起来就是：向量空间加上范数就是所谓的赋范空间，赋范空间加上线性结构就是所谓的线性赋范空间，线性赋范空间加上内积运算就是所谓的内积空间；有限维的内积空间就是欧氏空间，内积空间如果满足完备性就是希尔伯特空间。

## 9.3.14 张量

张量（tensor）是一个非常复杂的概念，最初由威廉·罗恩·哈密顿(1805-1865)在 1846 年引入。爱因斯坦广义相对论的发表，也是在黎曼几何和张量分析两大数学工具帮助下完成的。张量既有数学上的定义，又有物理上的意义，在计算机的深度学习里（谷歌的深度学习库就叫 TensorFlow），也有重要的应用。

在这里，我们暂时不去深究张量复杂的数学定义，物理意义等难以理解的东西，侧重从计算机的角度来理解张量的含义。

在计算机视角来看，张量是多维数组，目的是把向量、矩阵推向更高的维度。其中，如图 9-6 所示：

0 维张量就是标量，就是一个简单的数字，比如 100, 1024 等，可以看成是一个 0 维数

组。

1 维张量就是向量，可以看成是一个 1 维数组。

2 维张量就是矩阵，可以看成是一个 2 维数组。

3 维张量就是矩阵的数组，可以看成是一个 3 维数组。

以此类推，n 维张量可以看成是一个 n 维数组。所以可以看出，张量是对向量和矩阵的一般化（泛化），而向量和矩阵是张量的特例。

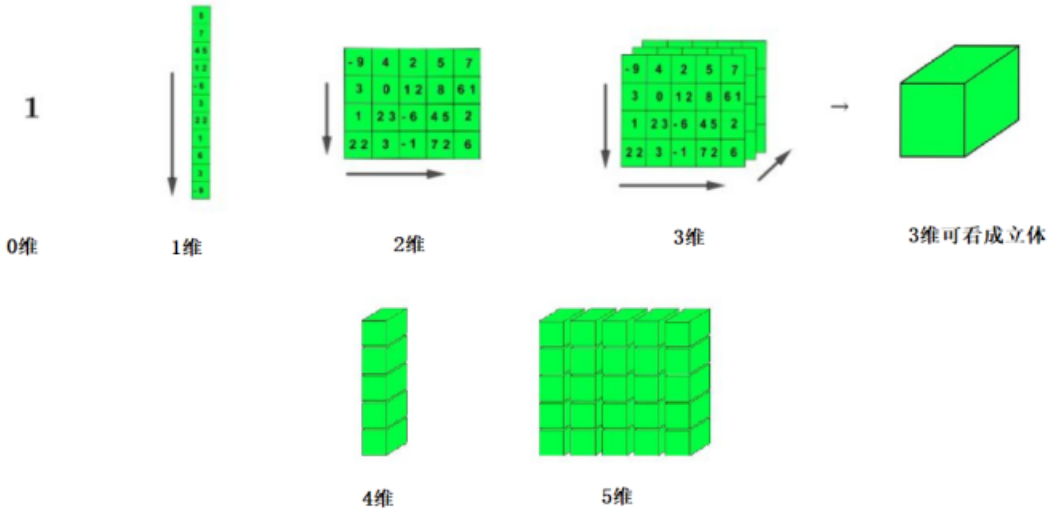



图 9-6 0-5 维张量

一张图片的数据，如图 9-7 所示，可以用一个三维张量来表示（存放在三维数组）。这是因为一张图片是由众多像素组成。像素是整个图像中不可分割的单位。这些像素决定图像在屏幕上所呈现的大小。图片的像素通常用“水平像素数(width)×垂直像素数(height)”（分辨率）的形式表示。而且一张图片分为 RGB（红绿蓝，三原色组成了所有颜色）三个通道，每个通道分别表示了红，绿，蓝三种颜色的像素。

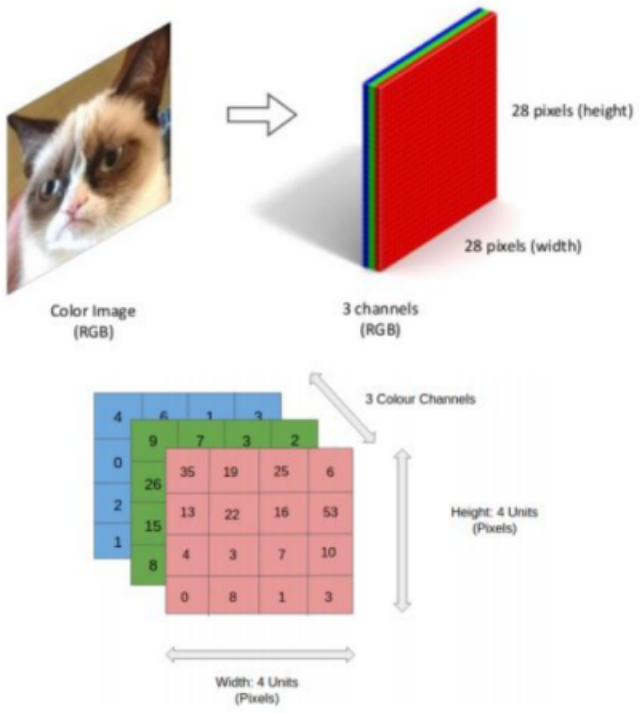



图 9-7 图片与三位张量

因此一张图片的数据，就可以定义一个三维数组（三维张量）来存放，即：PicData[channel][height][width]，其中 channel 表示颜色为红、绿、蓝的三个通道，height 和 width 表示图片的高度和宽度。

进一步说，在数学上，张量就是向量的多重线性函数，也就是多个向量到标量的映射，因此张量就是在向量上的函数（Tensor is a function which eats vectors and spits a number，意思是张量是一个函数，输入向量，输出一个数（标量））。如果限定在实数域，就是：

$$T：V_1 \times V_2 \times ... V_p \rightarrow R$$

如果把上述 $V_1$ 到 $V_p$ 中任意 p-1 个的取值固定，使得 T 变成剩下的那个向量 $V_k$ 到 R 的映射，那么这个映射就是线性映射。这就是为什么 p 阶张量的数据构造是 p 维数组的原因，因为它和 p 维数组的映射取值是一致的。

# 9.4 矩阵特征分解

正如我们前面所学，矩阵通常的表示方法，就是一个数表中的一组数，通过这一组数，我们可能无法直观的发现矩阵一些内在的性质，因此有时候需要对矩阵进行适当的拆解。矩阵的特征分解就是矩阵中应用最广的分解之一。它可以将矩阵分解成一组特征值和特征向量。

通过矩阵的特征值和特征向量，我们可以更加直观的发现矩阵的一些内在的特征和性质。

## 9.4.1 方阵特征值与特征向量

设 A 是 n 阶矩阵，如果数 $\lambda$ 和 n 维非零列向量 x 使关系式：

$$Ax = \lambda x$$

成立，那么就称 $\lambda$ 为矩阵 A 的特征值， x 为 A 对应特征值 $\lambda$ 的特征向量。

其中，$Ax=\lambda x$ 也可以写成：

$$(A-\lambda E)x=0$$

这实际上是一个包含 n 个未知数 n 个方程的齐次线性方程组，根据齐次线性方程组有非零解的充分必要条件为：

$$|A-\lambda E|=0$$

即：

$$\begin{vmatrix} a_{11}-\lambda & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22}-\lambda & \cdots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn}-\lambda \end{vmatrix}=0$$

这是一个以 $\lambda$ 为未知数的一元 n 次方程，称为矩阵 A 的特征方程。$|A-\lambda E|$ 是 $\lambda$ 的 n 次多项式，记作 $f(\lambda)$，称为矩阵 A 的特征多项式。

方阵 A 的特征值就是特征方程的解，n 阶矩阵 A 在复数范围内有 n 个特征值，设这 n 个特征值为 $\lambda_1$， $\lambda_2$， $\lambda_3$， …， $\lambda_n$，那么有：

$$\lambda_1+\lambda_2+\lambda_3+\ldots+\lambda_n=a_{11}+a_{22}+a_{33}+\ldots+a_{nn}$$

$$\lambda_1\lambda_2\lambda_3\ldots\lambda_n=|A|$$

设 $\lambda_i$ 是矩阵 A 的一个特征值，那么有：

$$(A-\lambda_i E)x=0$$

可求得非零解 $x=p_i$，那么 $p_i$ 就是 A 对应特征值 $\lambda_i$ 的特征向量。

可以证明的是：如果 $\lambda$ 是 A 的特征值，则 $\lambda^k$ 是 $A^k$ 的特征值。$\phi(\lambda)$ 是 $\phi(A)$ 的特征值。这是矩阵特征值的一个重要性质。

练习 12：求下列矩阵的特征值和特征向量。

$$A=\begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix}$$

解：A 的特征多项式为：

$$|A-\lambda E|=\begin{vmatrix} 3-\lambda & -1 \\ -1 & 3-\lambda \end{vmatrix}=(3-\lambda)^2-1=8-6\lambda+\lambda^2=(4-\lambda)(2-\lambda)$$

所以 A 的特征值为：$\lambda_1=4$， $\lambda_2=2$

当 $\lambda_1=4$ 时，代入特征方程有：

$$\begin{pmatrix} -1 & -1 \\ -1 & -1 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}=\begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

于是有 $\lambda_1$ 的特征向量 $p_1=\begin{pmatrix} -1 \\ 1 \end{pmatrix}$。

当 $\lambda_2=2$ 时，代入特征方程有：

$$\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}\begin{pmatrix} x_1 \\ x_2 \end{pmatrix}=\begin{pmatrix} 0 \\ 0 \end{pmatrix}$$

于是有 $\lambda_2$ 的特征向量 $p_2=\begin{pmatrix}1\\1\end{pmatrix}$。

设 $\lambda_1$，$\lambda_2$，$\lambda_3$，…，$\lambda_m$ 是方阵 A 的 m 个特征值，$p_1$，$p_2$，$p_3$，…，$p_m$ 依次是与之对应的特征向量，如果 $\lambda_1$，$\lambda_2$，$\lambda_3$，…，$\lambda_m$ 各不相等，则 $p_1$，$p_2$，$p_3$，…，$p_m$ 线性无关。

实际上， 对于 m(m≥2)个不同特征值的线性无关的特征向量组，合起来仍是线性无关的。

矩阵的特征分解可以给我们提供很多有关矩阵的内在性质。比如，如果矩阵是奇异的，当且仅当矩阵含有为 0 的特征值。实对称矩阵 A 对应着一个二次方程 $f(x)=x^TAx$（参见 9.4.5 一节），如果限制向量 x 的范数||x||=1，那么当 x 等于矩阵 A 的某个特征向量时，f(x)的值就是该特征向量对应的特征值，同时函数 f 的最大值是最大特征值，最小值是最小特征值。

## 9.4.2 奇异值分解

设有 m×n 的矩阵 A，如果我们把 A 分解成三个矩阵（U，D，$V^T$）的乘积：

$$A=UDV^T$$

其中 U 是一个 m×m 的矩阵，D 是一个 m×n 的矩阵，V 是一个 n×n 的矩阵，并且 U 和 V 都是单位正交矩阵（即 $UU^T=E, VV^T=E$），D 为对角矩阵，那么这就是对矩阵 A 的奇异值分解（Singular Value Decomposition，简称 SVD）。

对角矩阵 D 对角线上的元素称为矩阵 A 的奇异值，矩阵 U 的列向量为左奇异向量，矩阵 V 的列向量称为右列向量。

实际上，可以证明：A 的左奇异向量是 $AA^T$ 的特征向量，A 的右奇异向量是 $A^TA$ 的特征向量。A 的非零奇异值是 $A^TA$ 特征值的平方根，同时也是 $AA^T$ 特征值的平方根。因此可以通过 $A^TA$ 和 $AA^T$ 的特征向量和特征值来对 A 进行奇异值分解，求出左奇异向量，右奇异向量以及奇异值。

非方阵没有特征分解，但每个实数矩阵都有一个奇异值分解，因此奇异值分解的应用更加广泛。

## 9.4.3 伪逆矩阵

奇异值分解的一个非常有价值的应用在于非方阵的求逆。我们知道，对于非方矩阵或者奇异矩阵，是没有对应的逆矩阵的，但却可以有伪逆矩阵。

现在我们再来总结一下矩阵的逆，左逆，右逆和伪逆。

设有 m×n 的矩阵 A，当：

- R(A)=m=n时，此时矩阵 A 为 n 阶方阵，矩阵 A 存在可逆矩阵 $A^{-1}$，使得 $AA^{-1}=A^{-1}A=E$。
- R(A)=m，则矩阵 A 行满秩，存在右逆矩阵 $A^R=A^T(A^TA)^{-1}$，使得 $AA^R=E$。
- R(A)=n，则矩阵 A 列满秩，存在左逆矩阵 $A^L=A^T(A^TA)^{-1}$，是的 $A^LA=E$。
- R(A)<m 并且 R(A)<n，则存在伪逆，记为 $A^+$，且 $A^+=VD^+U^T$，其中矩阵 U, D, V 是对 A 进行奇异值分解之后的三个矩阵，对角矩阵 D 的伪逆矩阵是对其非零对角元素取倒数再转置而得。

那么伪逆矩阵的用处是什么呢？

设有 m×n 的矩阵 A，x 和 y 为 n 维列向量，现在我们再来看看下列线性方程组的求解问题：

$$Ax=y$$

我们已经知道：

1. 当 R(A)=R(A, y)=n 时，有唯一解。
2. 当 R(A)=R(A, y)<n 时，有无穷多个解。
3. 当 R(A)<R(A, y)时，无解。

对于上面第 2 种情况，由于有无穷多个解，$A^R$ 为矩阵 A 的右逆矩阵，方程两边同时左乘 $A^R$，那么有：

$$x=A^Ry$$

此时的 x 是所有解中距离原点最近的一个点，也就是最小范数解。

对于上面第 3 种情况，由于方程无解，$A^+$为矩阵 A 的伪逆矩阵，方程两边同时左乘 $A^+$，那么有：

$$x=A^+y$$

此时 x 虽然不是方程传统意义上的解，但却是使得 Ax 和 y 之间的范数（距离）||Ax-y|| 最小的。由此可见，伪逆矩阵可以用于求出方程的最优解。因此伪逆矩阵在最优化理论和机器学习中都有重要应用。

### 9.4.4 相似矩阵

如果两个矩阵相似，那么有时候可以通过相似矩阵来简化矩阵相关的运算复杂度。设 A，B 都是 n 阶方阵，若有可逆矩阵 P，使：

$$P^{-1}AP=B$$

则称 B 是 A 的**相似矩阵**或者 A 与 B 相似，而 $P^{-1}AP$ 称为 A 的相似变换，P 称为**相似变换矩阵**。

根据矩阵相似的定义容易证明：如果 n 阶矩阵 A 与 B 相似，则 A 与 B 的特征多项式相同，从而 A 与 B 的特征值也相同。

特别地，如果 n 阶矩阵 A 与对角矩阵：

$$\Lambda=\begin{pmatrix}\lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n\end{pmatrix}$$

相似，那么 $\lambda_1$，$\lambda_2$，$\lambda_3$，…，$\lambda_n$ 就是 A 的特征值。

设 $\phi(x)=a_0+a_1x+\ldots+a_mx^m$ 为 x 的 m 次多项式，A 为 n 阶矩阵，记：

$$\phi(A)=a_0E+a_1A+\ldots+a_mA^m$$

那么 $\phi(A)$称为矩阵 A 的 m 次多项式。

在矩阵的运算中，与相似矩阵相关的一些非常重要的公式如下：

- 若 $A=PBP^{-1}$，则 $A^n=PB^nP^{-1}$
- 若 $A=PBP^{-1}$，那么：

$$\phi(A)=P\phi(B)P^{-1}$$

- 对于对角矩阵 $\Lambda=\mathrm{diag}(\lambda_1,\lambda_2,\lambda_3,\ldots,\lambda_n)$，有：

$$\Lambda^k=\mathrm{diag}(\lambda_1^k,\lambda_2^k,\lambda_3^k,\ldots,\lambda_n^k)$$

$$\phi(\Lambda)=a_0E+a_1\Lambda+\ldots+a_m\Lambda^m$$

$$=\begin{pmatrix}\varphi(\lambda_1) & & & \\ & \varphi(\lambda_2) & & \\ & & \ddots & \\ & & & \varphi(\lambda_n)\end{pmatrix}.$$

所以，如果有 $A=P\Lambda P^{-1}$，即如果 A 与 Λ 相似，则 $A^n=P\Lambda^nP^{-1}$，$\phi(A)=P\phi(\Lambda)P^{-1}$，因此可以通过把矩阵 A 的计算转化为对角矩阵 Λ 的计算，简化运算过程。

把 n 阶矩阵 A 通过相似变换矩阵 P，使得 $P^{-1}AP=\Lambda$，称为把矩阵 A 对角化。那么相似变换矩阵 P 满足什么样的条件才能把 A 对角化呢？

因为：

$$P^{-1}AP=\Lambda$$

等式两边同时左乘 P 得：

$$AP=P\Lambda$$

把 P 用列向量表示为：$P=(p_1,p_2,p_3,\ldots,p_n)$，那么：

$$A(p_1,p_2,\cdots,p_n)=(p_1,p_2,\cdots,p_n)\begin{pmatrix}\lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n\end{pmatrix}$$

$$=(\lambda_1p_1,\ \lambda_2p_2,\ \lambda_3p_3,\ \ldots,\ \lambda_np_n)$$

所以有：

$$Ap_i=\lambda_ip_i$$

也就是说，P 的列向量就是由 A 的特征值 $\lambda_i$ 对应的特征向量 $p_i$ 所组成。那么由 A 的特征值 $\lambda_i$ 对应的特征向量 $p_i$ 所构成的 P 满足了 $AP=P\Lambda$，它是否可逆呢？如果 A 的特征值对应的特征向量 $p_1$，$p_2$，$p_3$，…，$p_n$ 线性无关，那么 P 一定可逆。而如果 A 的特征值互不相等，则对应的特征向量 $p_1$，$p_2$，$p_3$，…，$p_n$ 就是线性无关的，因此由这些特征向量构成的 p 就是可逆的，因此 A 就可以通过 $P^{-1}AP$ 对角化为 Λ，且相似变换矩阵 P 为 A 的特征值对应的特征向量 $p_1$，$p_2$，$p_3$，…，$p_n$ 所构成。

练习 13：判断下列矩阵是否可以对角化，如果能对角化，请求出可逆矩阵 P 和对角矩阵 Λ。

$$A=\begin{pmatrix}-2 & 1 & 1\\ 0 & 2 & 0\\ -4 & 1 & 3\end{pmatrix}$$

解：首先求出矩阵 A 的特征值，由特征方程 $|A-\lambda E|=0$ 有：

$$-(\lambda+1)(\lambda-2)^2=0$$

因此特征值为：

$$\lambda_1=-1,\ \lambda_2=\lambda_3=2$$

当 $\lambda_1=-1$ 时，代入 $(A+E)x=0$，解方程得对应的特征向量为：

$$p_1=\begin{pmatrix}1\\0\\1\end{pmatrix}$$

当 $\lambda_2=\lambda_3=2$ 时，代入(A+E)x=0，解方程得对应的特征向量为：

$$p_2=\begin{pmatrix}0\\1\\-1\end{pmatrix},\ p_3=\begin{pmatrix}1\\0\\4\end{pmatrix}$$

由于 $p_1$，$p_2$，$p_3$ 线性无关，所以 A 可以对角化，令

$$P=(p_1,p_2,p_3)=\begin{pmatrix}1&0&1\\0&1&0\\1&-1&4\end{pmatrix}$$

对应的对角阵为：diag(-1,2,2)，其中对角元的排列次序要与 P 中列向量的排列次序一致，即 $p_1$ 对应着 $\lambda_1$，$p_2$ 对应着 $\lambda_2$，$p_3$ 对应着 $\lambda_3$。

把 n 阶方阵对角化其实是一个较为复杂的问题，而对称矩阵却都是可以对角化的。一个 n 阶对称方阵 A，必有正交矩阵 P，使得 $P^{-1}AP=P^TAP=\Lambda$，其中 $\Lambda$ 是以 A 的 n 个特征值为对角元的对角矩阵。

练习 14：对角化下列对称矩阵。

$$A=\begin{pmatrix}0&-1&1\\-1&0&1\\1&1&0\end{pmatrix}$$

解：先求 A 的特征值。由特征方程$|A-\lambda E|=0$ 有：

$$-(\lambda-1)^2(\lambda+2)=0$$

所以，特征值为：

$$\lambda_1=-2,\ \lambda_2=\lambda_3=1。$$

当 $\lambda_1=2$ 时，代入:

$$(A+2E)x=0$$

得基础解系：

$$\xi_1=\begin{pmatrix}-1\\-1\\1\end{pmatrix}$$

单位化后，得：

$$p_1=\frac{1}{\sqrt{3}}\begin{pmatrix}-1\\-1\\1\end{pmatrix}$$

当 $\lambda_2=\lambda_3=1$ 时，代入：

$$(A-E)x=0$$

得基础解系：

$$\xi_2=\begin{pmatrix}-1\\1\\0\end{pmatrix},\xi_3=\begin{pmatrix}1\\0\\1\end{pmatrix}$$

将 $\xi_2$ 和 $\xi_3$ 正交单位化后得：

$$p_2=\frac{1}{\sqrt{2}}\begin{pmatrix}-1\\1\\0\end{pmatrix},p_3=\frac{1}{\sqrt{6}}\begin{pmatrix}1\\1\\2\end{pmatrix}$$

因此，$p_1$，$p_2$，$p_3$ 构成正交矩阵：

$$P=(p_1,p_2,p_3)=\begin{pmatrix}-\frac{1}{\sqrt{3}} & -\frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}}\\ -\frac{1}{\sqrt{3}} & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{6}}\\ \frac{1}{\sqrt{3}} & 0 & \frac{2}{\sqrt{6}}\end{pmatrix}$$

使得：

$$P^{-1}AP=P^{T}AP=\Lambda=\begin{pmatrix}-2&0&0\\0&1&0\\0&0&1\end{pmatrix}.$$

因此，得到了矩阵 A 的对角矩阵 Λ。

## 9.4.5 二次型，正定矩阵

在机器学习算法里，我们往往会研究大数据中每个对象的多个属性（比如人的年龄，性别，职业，爱好等，每个属性可以认为是一个变量）之间的二次关系，因此很有必要了解一下二次型的相关理论基础。我们把含有 n 个变量的 $x_1$，$x_2$，$x_3$，…，$x_n$ 的二次齐次函数:

$$f(x_1,x_2,x_3,\ldots,x_n)=a_{11}x_1^2+a_{22}x_2^2+a_{33}x_3^2+\ldots+a_{nn}x_n^2+2a_{12}x_1x_2+2a_{13}x_1x_3+\ldots+2a_{n-1,n}x_{n-1}x_n$$

称为二次型，为了变形方便，我们把其中的 $2a_{ij}x_ix_j$ 可以拆分成 $a_{ij}x_ix_j+a_{ji}x_jx_i$ 的和，其中 $a_{ji}=a_{ij}$。那么二次型可以改写成：

$$\begin{aligned}f=&a_{11}x_1^2+a_{12}x_1x_2+a_{13}x_1x_3+\ldots+a_{1n}x_1x_n+\\&a_{21}x_2x_1+a_{22}x_2^2+a_{23}x_2x_3+\ldots+a_{2n}x_2x_n+\\&\ldots\\&+a_{n1}x_nx_1+a_{n2}x_nx_2+a_{n3}x_nx_3+\ldots a_{nn}x_n^2\end{aligned}$$

进一步提取公因式 $x_1$，$x_2$，$x_3$，…，$x_n$ 之后，二次型可以表示为：

$$f=x_1(a_{11}x_1+a_{12}x_2+a_{13}x_3+\ldots a_{1n}x_n)+x_2(a_{21}x_1+a_{22}x_2+a_{23}x_3+\ldots a_{2n}x_n)+\ldots+x_n(a_{n1}x_1+a_{n2}x_2+\ldots+a_{nn}x_n)$$

$$=(x_1,x_2,\cdots,x_n)\begin{pmatrix}a_{11}x_1+a_{12}x_2+\cdots+a_{1n}x_n\\a_{21}x_1+a_{22}x_2+\cdots+a_{2n}x_n\\\vdots\\a_{n1}x_1+a_{n2}x_2+\cdots+a_{nn}x_n\end{pmatrix}$$

$$=(x_1,x_2,\cdots,x_n)\begin{pmatrix}a_{11}&a_{12}&\cdots&a_{1n}\\a_{21}&a_{22}&\cdots&a_{2n}\\\vdots&\vdots&&\vdots\\a_{n1}&a_{n2}&\cdots&a_{nn}\end{pmatrix}\begin{pmatrix}x_1\\x_2\\\vdots\\x_n\end{pmatrix}$$

记：

$$A=\begin{pmatrix}a_{11}&a_{12}&\cdots&a_{1n}\\a_{21}&a_{22}&\cdots&a_{2n}\\\vdots&\vdots&&\vdots\\a_{n1}&a_{n2}&\cdots&a_{nn}\end{pmatrix},\quad x=\begin{pmatrix}x_1\\x_2\\\vdots\\x_n\end{pmatrix}$$

则二次型可以记作：$f=x^TAx$，其中 A 是对称方阵。对称方阵和二次型是一一对应的，也就是对于一个给定的二次型，可以唯一确定一个对称方阵；给定一个对称方阵，可以唯一确定一个二次型。对称方阵的秩也叫做它所对应的二次型的秩。

一般我们可以通过如下的线性变换：

$$\begin{cases}x_1=c_{11}y_1+c_{12}y_2+\cdots+c_{1n}y_n,\\x_2=c_{21}y_1+c_{22}y_2+\cdots+c_{2n}y_n,\\\cdots\cdots\cdots\\x_n=c_{n1}y_1+c_{n2}y_2+\cdots+c_{nn}y_n\end{cases}$$

记 $C=(c_{ij})$，则 $x=Cy$，代入 f，将二次型转化为只含平方项的二次型：

$$f=k_1y_1^2+k_2y_2^2+\cdots+k_ny_n^2$$

我们把这种只含平方项的二次型称为标准形（或称为法式）。如果标准形中的系数 $k_1$，$k_2$，$k_3$，...，$k_n$ 取值只有 1，-1，和 0，那么称这个标准形为规范形。

要使函数 f 通过 $x=Cy$，变成标准形，那么把 $x=Cy$ 代入 $x^TAx$，就是使得：

$$(Cy)^TA(Cy)=y^TC^TACy=k_1y_1^2+k_2y_2^2+\cdots+k_ny_n^2$$

$$=(y_1,y_2,\cdots,y_n)\begin{pmatrix}k_1&&&\\&k_2&&\\&&\ddots&\\&&&k_n\end{pmatrix}\begin{pmatrix}y_1\\y_2\\\vdots\\y_n\end{pmatrix}$$

那么很明显，$C^TAC$ 需要是一个对角矩阵。由于 A 是对称矩阵，那么由对称矩阵的性质知，总存在一个正交矩阵 P，使得 $P^{-1}AP=\Lambda$，也就是 $P^TAP=\Lambda$，因此问题转化为将矩阵 A 对角化。因此有：

对于给定的一个任意二次型 $f=\sum_{i,j=1}^{n} a_{ij}x_ix_j(a_{ij}=a_{ji})$，总存在一个正交变换 x=Py，使 f 化为标准形：

$$f=\lambda_1 y_1^2+\lambda_2 y_2^2+\cdots+\lambda_n y_n^2$$

其中 $\lambda_1$，$\lambda_2$，$\lambda_n$，...，$\lambda_n$ 是 f 的矩阵 $A=(a_{ij})$的特征值，设二次型的秩为 r，则 $\lambda$ 中恰好有 r 个不为零的值，设为 $\lambda_1$，$\lambda_2$，$\lambda 3$，...，$\lambda_r$，则 $\lambda_{r+1}=\ldots=\lambda_n=0$。标准形进一步化为规范行的方法为：

令：

$$K=\begin{pmatrix} k_1 & & & \\ & k_2 & & \\ & & \ddots & \\ & & & k_n \end{pmatrix}，其中\ k_i=\begin{cases} \dfrac{1}{\sqrt{|\lambda_i|}}, & i\leqslant r \\ 1, & i>r \end{cases}$$

则 y=Kz 或 x=PKz，把 f 化为规范形：

$$\frac{\lambda_1}{|\lambda_1|}z_1^2+\cdots+\frac{\lambda_r}{|\lambda_r|}z_r^2$$

二次型的标准形不是唯一的，惯性定理指出，标准形中所含项数是确定的并且等于二次型的秩，而且在实数线性变换时，标准形中正负系数个数都不变。标准形中正系数的个数称为二次型的正惯性指数，负系数的个数称为负惯性指数。

对于二次型 $f(x)=x^TAx$，如果对任何 $x\neq 0$，都有 $f(x)>0$，则称 f 为正定二次型，并称对称方阵 A 是正定的；如果对于任何 $x\neq 0$，都有 $f(x)<0$，则称 f 为负定二次型，并称对称方阵 A 是负定的。

n 元二次型 $f(x)=x^TAx$ 为正定的充分必要条件是：它的标准形的 n 个系数全为正，即它的规范形的 n 个系数全为 1，也就是它的正惯性指数等于 n。

对于对称矩阵 A 为正定的充分必要条件是：A 的特征值全部为正。

对称矩阵 A 为负定的充分必要条件是：奇数阶主子式为负，偶数阶主子式为正。这里的主子式是指矩阵 A 对应的 n 阶行列式中，选取行号（如 1、3、5 行），再选取与行号相同的列号（1、3、5 列），则行数和列数都为 i 个的行列式即为 n 阶行列式的 i 阶主子式，也可以说由上述选取的行列交汇处的元素所组成的新的行列式 就称为“n 阶行列式的一个 i 阶主子式”。

二次型和对称矩阵的正定判断在机器学习的最优化算法牛顿法里有着实际的应用，在寻找 n 维空间极值判断时，需要利用海森矩阵的正定性进行判断，具体内容请参考本书章 10.7.3 节。

## 9.5 程序中的应用场景

计算机图形学、机器学习、密码学等技术无不以线性代数为其理论和算法基础的一部分。

Google 搜索的排序方法建模使用了马尔科夫链，涉及了 n 步转移概率矩阵，因此问题最终归结为一个矩阵方程的求解。

天气预测中的 Navier-Stokes 方程（纳维叶－斯托克斯方程，是流体力学中描述粘性牛顿流体的非线性偏微分方程，数学中最为难解的非线性方程中的一类，寻求它的精确解是非常困难的事，目前为止尚未被完全解决的方程。直至今天，大约也只有 70 多个精确解），计算量之大，除了要改进计算机硬件，算法的线性优化和近似是必须的。

机器学习，本质也是求解线性方程组，这一点可以参考本书第 11 章 11.7.6 节的具体讲解与分析。

下面我们就来具体学习一下线性代数在计算机科学中的一些应用场景，通过这些场景的学习，我们就能够认识到线性代数的重要意义。

## 9.5.1 机器学习中的应用

线性代数在机器学习中有着重要的应用。在机器学习中，各种数据的特征值，都会通过线性代数中的向量或者矩阵来进行描述和表示。现在我们来看看线性代数在机器学习中的特征矩阵，主成分分析和深度学习中的应用例子。

## 1.特征矩阵

机器学习中，需要大数据样本进行模型训练。而大数据集是一个这样的数据文件，其中每行为一条数据记录，每列为一个属性特征。比如广告点击率预估相应的机器学习算法中，用于训练的数据文件每行就是广告被用户点击的日志记录（包括广告是否被点击的标记 label，用户的 id，性别和年龄等特征）：

0,1,0,10
0,2,0,20
0,3,1,30
0,4,1,40
1,5,2,40
1,6,2,50
1,7,0,60
1,8,0,10
0,9,1,10
……

第一列为 label 值，表示广告是否被点击，广告被点击值为 1，未被点击值为 0。第二列的值为用户 id。第三列的值为用户性别，0 代表女性，1 代表男性。第四列值为用户年龄等。

这些数据，实际上是被当做矩阵来存储和运算的。每行的数据属性个数相同（也就是列数相同），因此每行的数据可以看作一个向量，每个属性就是向量的一个分量。

我们还可以对这个矩阵中的数据进行专门的编码（比如 one-hot 独热编码）进一步得到一个稀疏矩阵，以更加适应特定的机器学习算法的输入格式。在这个矩阵数据上来拟合一个监督机器学习模型，比如预估某条广告在未来的点击概率。

同样以广告点击率预估为例子。为了得到广告点击率预估模型，我们可以使用线性回归的方法（请参考本书第 11 章 11.7.6 节），对样本数据，利用最小二乘法（请参考本书第 10 章 10.7.1 节），找到一组系数，用这些系数与每个输入变量（年龄，性别，广告 id 等各种

特征组成的一个向量）相乘并将结果相加，得出最佳的输出变量预测，即广告点击的概率：

$$y = A \cdot b$$

其中，y 是输出变量（广告点击的概率，取值范围在 0 到 1 之间），A 是输入数据集矩阵，b 是所求的模型系数。

## 2.主成分分析

主成分分析（Principal Component Analysis，PCA）是机器学习中的一个常用算法。在主成分分析算法中也用到了大量的线性代数知识。主成分分析通过正交变换将一组可能存在相关性的变量转换为一组线性不相关的变量，转换后的这组变量叫主成分。

主成分分析是对于原先提出的所有变量，将重复或关系紧密的冗余变量去掉，建立尽可能少的新变量，使得这些新变量是两两不相关的，而且这些新变量在反映结果方面尽可能保持原有的信息。

在机器学习算法中，如果数据的维度过高（即相关变量太多，比如广告点击率预估算法里，预测一个广告是否被点击，就和广告自身的内容，场景，时段，形式，以及用户的年龄，性别，职业，爱好等都有关系），这就给数据的存储和计算带来过高的复杂度。因此为了简化计算，需要利用主成分分析算法进行降维处理，去掉一些次要因素。

设 x 是 $R^n$ 空间中的一个向量（相当于机器学习中的一条样本记录），通过一个编码函数 f(x)，令 c=f(x)得到一个 $R^l$ 空间中的一个向量（注意这里是小写的 L，不是 1）。如果 l 比 n 小，那么我们就是用了一个低维度的向量 c 来表示一个高维度的向量 x。同时，我们又提供一个解码函数 g，我们称对向量 c 解码的结果 g(c)为重构向量，一般来说，它和 x 是不相等的，只能是 $x \approx g(c)$。

对于解码函数的选择，一般使用的是通过矩阵乘法将编码结果重新映射回 $R^n$，即：

$$g(c)=Dc$$

其中 D 是一个 n×l（注意这里是小写的 L，不是 1）的矩阵，并规定 D 中所有列向量都有单位范数，并且彼此正交。

主成分分析算法的最终目标是要找到最优的编码函数 f(x)从而获得最优的编码，以及最优的解码矩阵 D。

那么什么是最优编码呢？判断的标准就是：输入向量 x 和重构向量 g(c)之间的距离（范数）最小。也就是使：

$$||x-g(c)||$$

取得最小值的编码，为了方便计算，我们往往使用$||x-g(c)||^2$来代替$||x-g(c)||$作为最小化函数来寻找 c 的极小值点。根据向量范数的定义有：

$$||x-g(c)||^2=(x-g(c))^T(x-g(c))=x^Tx-x^Tg(c)-g(c)^Tx+g(c)^Tg(c)$$
$$= x^Tx-2x^Tg(c)+g(c)^Tg(c)$$

因此问题转化为求 $x^Tx-2x^Tg(c)+g(c)^Tg(c)$的极小值点。由于 $x^Tx$ 与 c 无关，所以 $x^Tx$ 可以忽略。因此优化的目标为：

$$-2x^Tg(c)+g(c)^Tg(c)$$

由于：

$$g(c)=Dc$$

代入目标函数有：

$$-2x^TDc+c^TD^TDc$$

其中 D 的列向量正交并且有单位范数。因此：

$$D^TD=E$$

于是目标函数进一步化简为：

$$-2x^TDc+c^Tc$$

这个目标函数可以通过向量微积分求导获得极值点：

$$\nabla c(-2x^TDc+c^Tc)=0$$

即：

$$-2D^Tx+2c=0$$

解为：

$$c=D^Tx$$

因此目标函数将在 $c=D^Tx$ 处取得最小值。可见 x 的最优编码由如下编码函数确定：

$$f(x)=D^Tx$$

确定了解码矩阵 D，那么最优的解码函数也就确定了。现在我们再看如何确定最优的解码矩阵 D。

把 $c=D^Tx$ 代入解码函数 $g=Dc$ 有：

$$g(c)=g(D^Tx)=DD^Tx$$

上面分析可以看出，解码矩阵要参与所有降维向量的解码，因此也必须最小化每个重构向量与每个原向量的范数（误差），这将组成一个误差矩阵：

$$A=(a_{ij}),\ a_{ij}=\|x_i-DD^T(x_j)\|$$

将矩阵 A 各项元素的绝对值平方的总和称为 Frobenius 范数，即：

$$\|A\|_F \equiv \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n}|a_{ij}|^2}$$

因此问题转化为求使$\|A\|_F$取最小值的解码矩阵 D。这个问题可以通过矩阵的特征分解来求解，具体方法请参考本书 9.4 节。

## 3.深度学习

深度学习是机器学习的一种，而机器学习是实现人工智能的必经之路。深度学习源于人工神经网络的研究，它通过组合低层特征形成更加抽象的高层特征，以发现数据的分布式特征表示。研究深度学习的动机在于建立模拟人脑进行分析学习的神经网络，它模仿人脑的机制来解释数据，例如图像，声音和文本等。

神经网络的执行涉及线性代数相关概念和理论知识。比如在多个维度下，深度学习方法可以处理向量、矩阵，甚至输入和系数的张量。

线性代数是深度学习方法的核心理论，深度学习中使用了大量的矩阵表示法以及矩阵运算。比如 Google 的 TensorFlow 机器学习库，其名称中包含的 tensor 一词就是线性代数里的张量，即一个两维以上的矩阵（参考本书 9.3.14 节）。

## 9.5.2 AES 加解密中的应用

AES，即高级加密标准（Advanced Encryption Standard，缩写：AES），是美国联邦政

府采用的一种区块加密标准。这个标准用来替代原先的 DES，已成为对称密钥加密中最流行的算法之一。

AES 加密时候，如图 9-8 所示，对明文数据划分为若干个数据块分组，每个数据块分组长度必须为 128 比特(16 字节)。如果数据长度不够 16 字节的整数倍，那么剩余部分需要用额外数据填充。数据块存储在一个 4×4 的矩阵之中，在后续多轮加密过程中，会对该矩阵中的数据进行相应的复杂运算，比如列变换或者行变换。

加密的密钥长度可以是 128 比特、192 比特、256 比特中的任意一个。在加密的时候我们输入的密码（password 或者 key）并不是真正用于 AES 加密解密的密钥，而是用于生成真正密钥的“种子”。

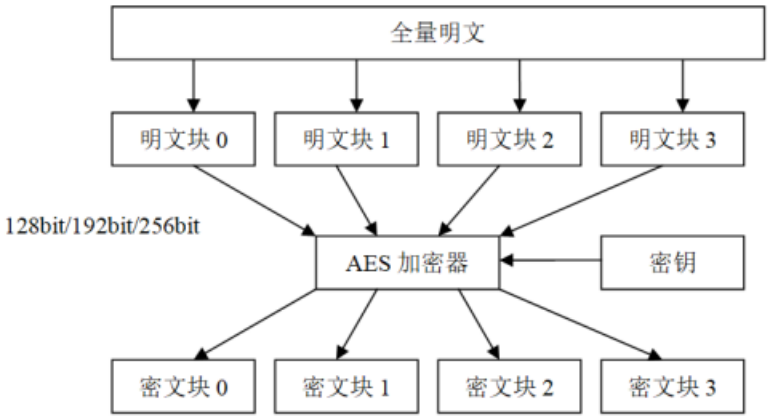



图 9-8 AES 加密

AES 在加密过程中是按照轮数执行的，那么 AES 的加密过程具体分成多少轮(round)呢？主要分为 3 个阶段，即：

- 初始轮（Initial Round） 1 次
- 普通轮（Rounds） N 次
- 最终轮（Final Round） 1 次

AES 的 Key 支持三种长度：AES128，AES192，AES256。Key 的长度决定了 AES 加密的轮数。除去初始轮，各种 Key 长度对应的轮数如下：

- AES128：10 轮
- AES192：12 轮
- AES256：14 轮

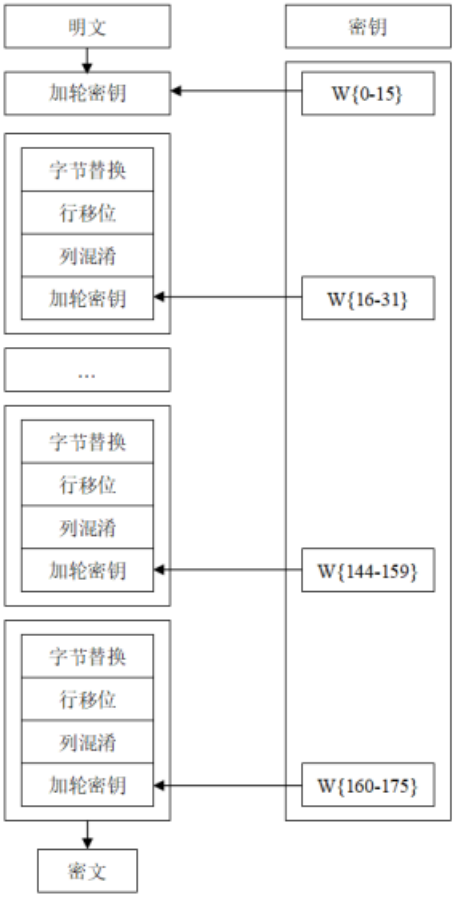



图 9-9 AES 加密流程

如图 9-9 所示，AES 加密不同阶段的 Round 有不同的处理步骤。

初始轮只有一个步骤：加轮密钥（AddRoundKey）。

普通轮有四个步骤：字节代替（SubBytes），行移位（ShiftRows），列混淆（MixColumns），加轮密钥（AddRoundKey）。

最终轮有三个步骤：字节代替（SubBytes），行移位（ShiftRows），加轮密钥（AddRoundKey）。

现在我们来看看 AES 在普通轮的四个步骤中所作的工作。

第一步：字节替代（SubBytes）

如图 9-10 所示，16 字节的明文块在每一个处理步骤中都被排列成 4×4 的矩阵。所谓字节替代，就是把明文块的每一个字节都替代成另外一个字节。替代的依据是一个被称为 S 盒（Subtitution Box）的 16×16 大小的矩阵。

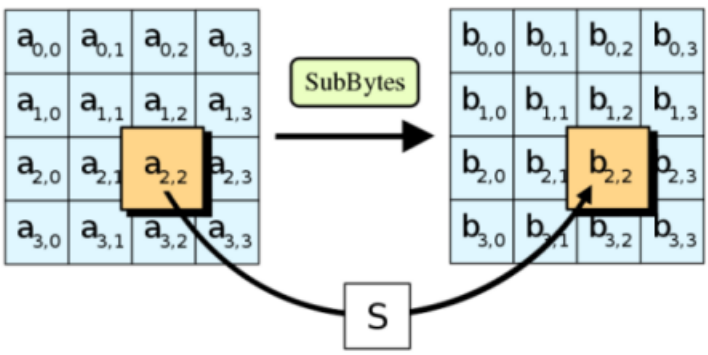



图 9-10 字节替代

假设明文块当中 a[2, 2] = 0x4A，那么以 0x4A 中的 4 和 10（即 16 进制的 A）为 S 盒的数组下标得 S[4][10]，并令输出值 b[2, 2] = S[4][10]。

第二步：行移位（ShiftRows）

行位移很简单，就是保持第一行不变，第二行循环左移 1 个字节，第三行循环左移 2 个字节，第四行循环左移 3 个字节。

第三步：列混淆（MixColumns）

这一步，输入矩阵的每一列和一个修补矩阵（fixed matrix）做矩阵相乘，得到对应的输出列，如图 9-11 所示。在解密时，对应的修补矩阵与该修补矩阵是互逆关系。

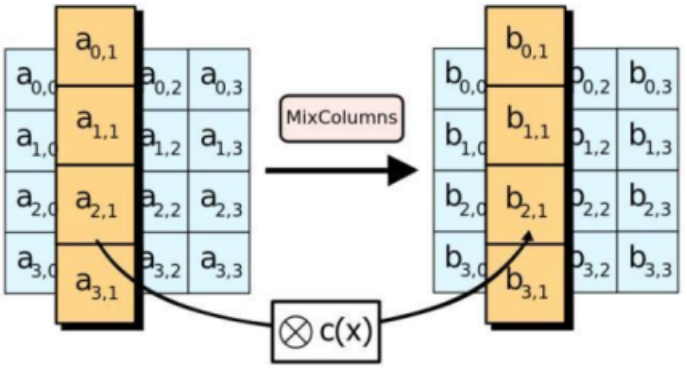



图 9-11 列混淆

第四步：加轮密钥（AddRoundKey）

这一步利用密钥进行加密，128bit 的密钥被排列成 4×4 的矩阵。输入矩阵的每一个字节 a[i, j]与密钥对应位置的字节 k[i, j]异或一次，就生成了输出值 b[i, j]。加密的每一轮所用到的密钥并不是相同的。利用扩展密钥（KeyExpansions），AES 用长度 4 * 4 *（10+1） 字节的数组 W 来存储所有轮的密钥。W{0-15}的值等同于原始密钥的值，用于为初始轮做处理。后续每一个元素 W[i]都是由 W[i-4]和 W[i-1]计算而来，直到数组 W 的所有元素都赋值完成。

因此，在 W 数组中，W{0-15}用于初始轮的处理，W{16-31}用于第 1 轮的处理，W{32-47}用于第 2 轮的处理，这样一直到 W{160-175}用于最终轮的处理。

而解密的过程和加密的过程正好相反，通过最终轮，普通轮和初始轮，把密文解密成明文。

通过上面 AES 的加密过程可以看出，线性代数中的矩阵运算在加密过程中起了非常重要的作用。这里体现了线性代数在密码学中的重要应用。

线性代数除了在机器学习，最优化理论和加解密中有大量的应用之外，还在计算机图形学，量子计算（参考本书 4.4.4 节）等领域同样有重要的应用，在此就不一一赘述。

# 十，微积分基础与程序应用

本章我们将在前面已有的数学知识基础上，从极限思想出发，给大家介绍高等数学中的导数、微分、积分以及多元微积分的基本思想和理论，最后再详细介绍微积分理论在计算机中的一些应用。

高等数学是由微积分学，代数学、几何学以及它们之间的交叉内容所形成的一门基础学科。它的主要内容包括：数列、极限、微积分、空间解析几何与线性代数、级数、常微分方程等。

从时间角度来看，16 世纪以前发展起来的各个数学学科属于初等数学的范畴，而 17 世纪（牛顿，莱布尼茨时代开始）以后建立的数学学科基本上都是高等数学的内容。

微积分由牛顿和莱布尼茨（如图 10-1）共同创立。牛顿最先研究，莱布尼茨最先（1684 年）发表。目前微积分领域主要使用的是莱布尼茨的符号系统（比如微分符号 dy/dx，积分符号∫等）。

图 10-1 牛顿(1643-1727)与莱布尼茨（1646-1716)

在数学史上，牛顿和莱布尼茨之间产生了微积分的发明权之争，由此而导致了欧洲大陆与英国的数学发展不同结果。英国因为尊崇和拥护牛顿的点主义（符号点“.”是牛顿的微积分符号之一）而故步自封，盲目排外导致其在数学领域落后欧洲大陆 100 年。18 世纪分析数学的进步主要是欧洲大陆数学家在莱布尼茨的微积分基础上取得的。欧洲大陆出现了很多杰出的数学家，比如：伯努利家族，欧拉，克莱洛，达朗贝尔，拉格朗日，拉普拉斯，勒让德（巴黎 3L）。他们都是 d 主义拥护者，d 是莱布尼茨的微积分符号之一。而英国同时期的数学家则凤毛麟角，比如：泰勒，麦克劳林。

中国清代数学家李善兰在他的《代微积拾级》（1859）一书中将 calculus 翻译为微积分，翻译的灵感来源于中国的古代成语“积微成著”。在 19 世纪把西方近代数学、物理学知识翻译为中文的传播工作中，李善兰作出了重大贡献。

高等数学是理工科基础中的基础，一切理工科领域复杂的问题和理论，都会用到。其中高等数学中的极限是微积分的基础，微积分是概率论的基础，概率论又是机器学习算法和人工智能的基础。

# 10.1 极限

极限是近代数学的一种重要思想。数学分析（以微积分和无穷级数一般理论为主要内容）就是以极限理论为基础和主要工具来研究函数的一门学科。在几乎所有的数学分析著作中，都是先介绍函数理论和极限的思想方法，然后利用极限的思想方法给出连续函数、导数、定积分、级数的敛散性、多元函数的偏导数、广义积分的敛散性、重（chong2）积分和曲线积分与曲面积分的概念。

数学分析之所以能解决许多初等数学无法解决的问题（例如求瞬时速度、曲线弧长、曲边形面积、曲面体的体积等问题），正是由于其采用了“极限”的“无限逼近”的思想方法，才能够得到无比精确的计算结果。

现在，我们就正式开始学习极限的理论基础。从极限开启我们学习高等数学之旅。

## 10.1.1 古代极限思想

我们知道，圆周率π=圆周长/圆直径，因此要计算圆周率，只需要分别求出圆的周长和直径。但是，在计算圆周率的时候，直径是直的，好测量，难点在于精确的计算圆的周长。也就是，只要认真而又耐心地精确测出圆周长，就可得出较为精确的圆周率了。

中国古代魏晋时期的刘徽（约公元 225 年-约公元 295 年，如图 10-2 所示）为了计算圆周率提出了著名的“割圆术”，在刘徽割圆术思想的基础上，祖冲之（429 年-500 年）最终完成了较为精确计算圆周率的工作，使这个难题得到了解决。

图 10-2 刘徽

实际上，中国古代从先秦时期开始，一直是取"周三径一"(即圆周周长与直径的比率为三比一)的数值来进行有关圆的计算，也就是认为圆周率π约等于 3。但用这个数值进行计算的结果，往往误差很大。正如刘徽所说，用"周三径一"计算出来的圆周长，实际上不是圆的周长而是圆内接正六边形的周长，其数值要比实际的圆周长小得多。那么可以在圆内接正六边形把圆周等分为六条弧的基础上，再继续等分，把每段弧再分割为二，做出一个圆内接正十二边形，这个正十二边形的周长不就要比正六边形的周长更接近圆周了吗?如果把圆周再继续分割，做成一个圆内接正二十四边形，那么这个正二十四边形的周长必然又比正十二边形的周长更接近圆周。这就表明，越是把圆周分割得细，误差就越少，其内接正多边形的周长就越是接近圆周。

如图 10-3 所示，如此不断地分割下去，一直到圆周无法再分割为止，也就是到了圆内接正多边形的边数无限多的时候，它的周长就与圆周"合体"而完全一致了。

刘徽在割圆术中提出的“割之弥细，所失弥少，割之又割以至于不可割，则与圆合体而无所失矣”，这可视为中国古代极限观念的佳作。

按照这样的思路，刘徽把圆内接正多边形一直算到了正 3072 边形，并由此而求得了圆周率 为 3.1415 和 3.1416 这两个近似数值。这个结果是当时世界上圆周率计算的最精确的数据。刘徽对自己创造的这个“割圆术”新方法非常自信，把它推广到有关圆形计算的各个方面，从而使汉代以来的数学发展大大向前推进了一步。

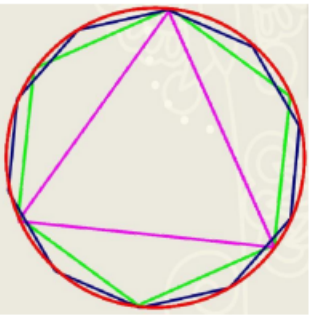
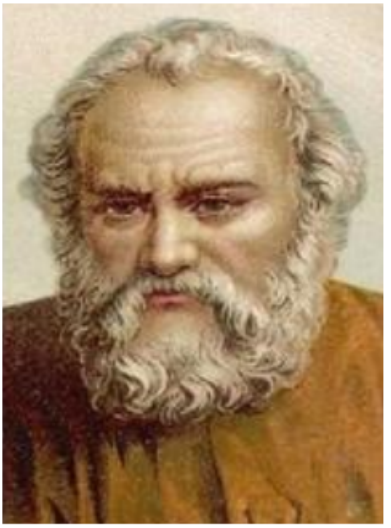


图 10-3 割圆术

到了南北朝时期，祖冲之（429 年—500 年，如图 10-4 所示）在刘徽的这一基础上继续努力，终于使圆周率精确到了小数点以后的第七位。在西方，这个成绩是由法国数学家韦达于 1593 年取得的，比祖冲之要晚了一千一百多年。祖冲之还求得了圆周率的两个分数值，一个是“约率”，另一个是“密率”，约率 π=22/7（≈3.14），密率 π=355/113（≈3.1415929），这个值，在西方是由德国的奥托和荷兰的安东尼兹在 16 世纪末才得到的，都比祖冲之晚了一千一百年。刘徽所创立的“割圆术”对中国古代数学发展做出了重大贡献。

图 10-4 祖冲之

此外，古希腊著名的科学家阿基米德(公元前 287 年-公元前 212 年，如图 10-5 所示)利用“穷竭法”（即极限的思想）算出球面积、球体积，抛物线、椭圆面积。后世的数学家依据阿基米德的数学思想加以发展，形成近代的“微积分”。而在中国，球体积的计算公式则最早由祖冲之之子祖暅根据祖暅原理（gèng，456 年-536 年）求出。



10-5 阿基米德

## 10.1.2 什么是极限

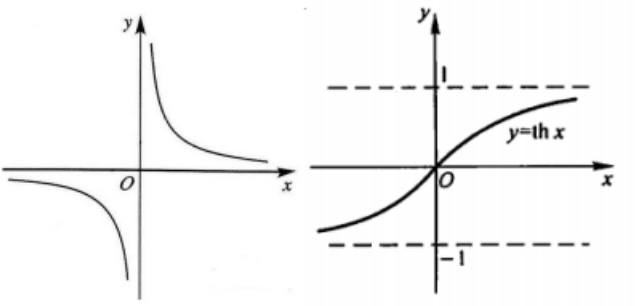
那么什么是极限呢？

广义上来说，极限就是指：无限靠近而永远不能到达（逼近）。

数学中是指：某个函数中的某个变量在变大（或者变小）的永远变化的过程中，逐渐向某一个确定的数值 A 不断地逼近而永远不能够重合到 A 的过程中， 永远靠近而不停止，有一个“不断地极为靠近 A 点的趋势”。

极限可以分为数列极限和函数极限。由于数列可以看作特殊的函数，因此本质上都是函数极限。

如图 10-6 所示，当 $x\to+\infty$ 的时候（x 趋近于正无穷），函数 y=1/x 和 y=thx 的函数值分别趋近于 0 和 1，因此 0 和 1 是当 $x\to+\infty$ 函数 y=1/x 和 y=thx 的极限。




10-6 极限无限逼近一个定值

## 10.1.3 数列极限

对于任意一个 n ∈N+， 总存在唯一的一个实数 $x_n$ 与 n 对应，并按照下标 n（不是按照 $x_n$）从小到大排列而组成的一组数，我们就称之为数列。数列如下所示：

$$x_1, x_2, x_3, \cdots, x_n, \cdots$$

数列可以看成是定义域为正整数的函数，即：$x_n=f(n), n\in N+$

对于数列，我们比较感兴趣的是，当 $n\to\infty$ 的时候，$x_n$ 的值是否趋于一个定值？这就是数列极限要研究的问题。

**数列极限定义**

$\forall\ \varepsilon>0$，$\exists$ 正整数 N，当 n>N 时，有 $|x_n-a|<\varepsilon$，那么数列 $x_n$ 的极限为 a，记为：

$$\lim_{n\to\infty} x_n = a$$

注意，上述定义中的符号“$\forall$”为全称量词表示“所有的”，“$\exists$”是存在量词，表示“存在”，请参考本书 8.4 节。

如何理解数列极限的定义呢？定义指出：不管 $\varepsilon$ 取多小，在数列中，都能找到一个 N，使得从 N+1 项开始，后面的 $x_n$ 与 a 的差都小于 $\varepsilon$。

其中的 $\varepsilon$ 具有任意性。定义中 $\varepsilon$ 的作用在于衡量数列通项 $x_n$ 与常数 a 的接近程度。$\varepsilon$ 越小，表示接近得越近；而正数 $\varepsilon$ 可以任意地变小，说明 $x_n$ 与常数 a 可以接近到任何不断地靠近的程度，要多近就多近。最后，$\varepsilon$ 一旦确定，接近程度就不变了。

其中的 N 具有相应性。一般来说，N 随 $\varepsilon$ 的变小而变大。重要的是 N 的存在性，而不在于其值的大小。

数列极限对应的几何意义在于：如图 10-7 所示，所有下标大于 N 的 $x_n$ 都落在$(a-\varepsilon, a+\varepsilon)$内；而在$(a-\varepsilon, a+\varepsilon)$之外，数列$\{x_n\}$ 中的项至多只有 N 个（有限个）。

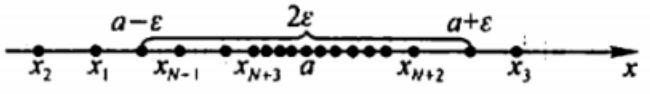



图 10-7 数列极限几何意义

如果数列的极限存在，我们就称这个数列收敛。比如下列数列：

**1, 1/2, 1/4, 1/8, …, $1/2^n$, …**

这个数列的极限存在，且为零。实际上，对于等比数列 $1, q, q^2, \ldots, q^{n-1}, \ldots$，只要它的公比 $|q|<1$，那么它的极限就是 0。这是因为根据数列极限的定义：

$\forall\ \varepsilon>0$（设 $\varepsilon<1$），因为：

$$|x_n-0|=|q^{n-1}-0|=|q|^{n-1}$$

要使 $|x_n-0|<\varepsilon$，只要使：

$$|q|^{n-1}<\varepsilon$$

两边取对数：

$$(n-1)\ln|q|<\ln\varepsilon$$

因为 $|q|<1$，所以 $\ln|q|<0$，所以有：

$$n-1>\ln\varepsilon / \ln|q|$$

即：

$$n>1+\ln\varepsilon / \ln|q|$$

取 $N=[1+\ln\varepsilon / \ln|q|]$（即向上取计算结果的整数，比如对于 3.14，那么[3.14]=4），那么当 n>N 时，就有 $|x_n-0|<\varepsilon$，因此得证该等比数列的极限是 0。

证明数列的极限的时候，我们都是这样通过任意小的 $\varepsilon$，找出一个满足定义的具体的 N，证明这个 N 是一定存在的，从而证明数列的极限。当然，如果这个 N 不存在，那么数列的极限也就不存在了。

如果数列的极限不存在，我们就称这个数列发散。比如下列数列：

**1, 2, 4, 8, 16, 32, …, $2^n$, …**

这是因为虽然该数列是等比数列，但是|q|=2>1，当n→∞的时候，$x_n$→∞。因此该数列是发散的。

数列的极限有如下几个重要的性质：

- **唯一性**：如果数列{$x_n$}收敛，那么它的极限唯一。
- **有界性**：如果数列{$x_n$}收敛，那么数列{$x_n$}一定有界（有界不一定收敛，单调有界才收敛）。
- **保号性**：如果 $\lim x_n = a$（$n \to \infty$），且 $a>0$（或 $a<0$），那么存在正整数 $N>0$，当 $n>N$ 时，都有 $x_n>0$（或 $x_n<0$）。
- **保号性推论**：如果数列{$x_n$}从某项起有 $x_n \geqslant 0$（或 $x_n \leqslant 0$），且 $\lim x_n = a$（$n \to \infty$），那么 $a \geqslant 0$（或 $a \leqslant 0$）。
- **收敛数列与其子数列间的关系**：如果数列{$x_n$}收敛于 a，那么它的任一子数列也收敛，且极限也是 a。

## 10.1.4 函数极限

在函数自变量的某个变化过程中（任意接近有限值 $x_0$ 或者无限增大∞），如果对应的函数值**无限接近于某个确定的数**，那么这个确定的数就叫做在这一变化过程中的函数极限。

如图 10-8 所示，根据自变量 x 的变化情概况，函数极限分两类：

- $x \to x_0$，即 x 趋近于某有限值，对应函数值 f(x)的变化；
- $x \to \infty$，即 x 趋近于无穷，对应函数值 f(x)的变化。

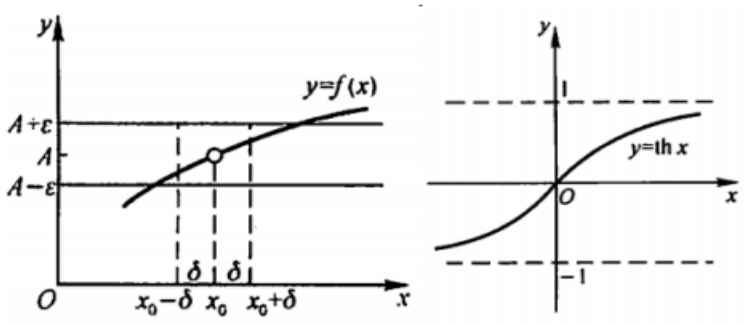



图 10-8 函数极限

**1.趋于有限值的极限**

设函数 f(x)在点 $x_0$ 的某一去心邻域内有定义，如果存在常数 A，对于任意给定的 ε（无论它多么小），总存在正数 δ，使得当 x 满足不等式 $0<|x-x_0|<\delta$，对应的函数值 f(x)都满足不等式

$$|f(x)-A|<\varepsilon$$

我们就把常数 A 称为函数 f(x)当 $x \to x_0$ 时的极限，记作：

$$\lim_{x \to x_0} f(x) = A$$

或

$$f(x) \to A(\text{当 } x \to x_0)$$

有了函数趋于有限值极限的定义之后，我们可以根据定义来证明某些函数的极限存在和取值。

练习 1：证明当 $x \to x_0$ 时 limc=c，此处 c 为常数

证明：对于任意小的正数 ε，|f(x)-A|=|c-c|=0< ε 总成立，所以可以任取 δ >0，都有 0<|x-$x_0$|< δ 时，使不等式：|f(x)-A|=|c-c|=0< ε 成立。

所以：

$$\lim c=c \text{（当 } x\to x_0 \text{ 时）}$$

练习 2：证明 $\lim x=x_0$，当 $x\to x_0$

证明：因为，|f(x)-A|=|x-$x_0$|，对于任意给定的正数 ε，要使|f(x)-A|< ε，即：

$$|x-x_0|<\varepsilon$$

只需要取 δ = ε，那么当|x-$x_0$|< δ = ε，因此 δ 存在，所以：

$$\lim x=x_0, \text{ 当 } x\to x_0$$

如果 x 仅从左侧趋于 $x_0$（记作 $x\to x_0^-$）或者 x 仅从右侧趋于 $x_0$（记作 $x\to x_0^+$）时函数的极限存在，则称为函数当 $x\to x_0$ 的左极限或右极限

左极限记作：

$$\lim_{x\to x_0^-} f(x)=A \text{或} f(x_0^-)=A$$

右极限记作：

$$\lim_{x\to x_0^+} f(x)=A \text{或} f(x_0^+)=A$$

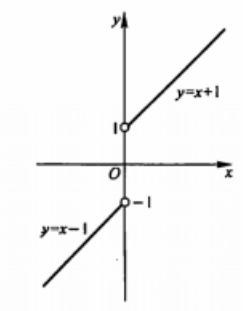
函数 f(x)当 $x\to x_0$ 时极限存在的充分必要条件是函数在 $x_0$ 的左右极限各自存在且相等，即：

$$f(x_0^-)=f(x_0^+)$$

练习 3：下列函数当 $x\to 0$ 时，极限存在吗？

$$f(x)=\begin{cases} x-1, & x<0, \\ 0, & x=0, \\ x+1 & x>0. \end{cases}$$

解：函数的图像如 10-9 所示：




10-9 分段函数图像

容易证明，函数在 x=0 这一点的左极限为：

$$\lim_{x\to 0^-} f(x)=\lim_{x\to 0^-}(x-1)=-1$$

右极限为：

$$\lim_{x\to 0^+} f(x) = \lim_{x\to 0^+} (x+1) = 1$$

由于左极限≠右极限，所以函数在 x=0 处没有极限。

**2.趋于无穷大极限**

设函数 f(x)当|x|大于某一正数时有定义。如果存在常数 A，对于任意给定的整数 ε（无论它多么小），总存在正数 X，使得当|x|>X 时，对应的函数值 f(x)都满足不等式

$$|f(x)-A|<\varepsilon$$

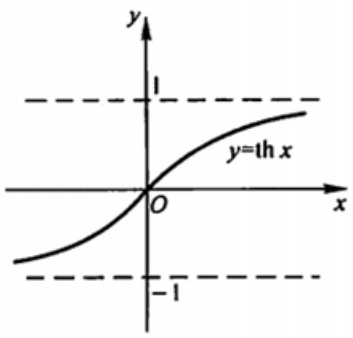
那么常数 A 就叫做函数 f(x)当 x→∞时的极限，记作：

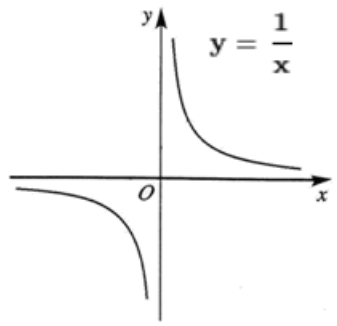
$$\lim_{x\to\infty} f(x) = A$$

或

$$f(x)\to A\ (\text{当 } x\to\infty)$$

从几何角度解释极限，如图 10-10 所示，在两个函数的图像中，随着 x→+∞时，函数值无限逼近 1 和 0。所以 1 和 0 就是函数在 x→+∞的极限。




10-10 函数极限

练习 4：证明 $\lim_{x\to\infty} \frac{1}{x} = 0$

证明：∀ ε >0，要使|f(x)-A|< ε ，即|1/x-0|=|1/x|< ε ，也就是：

$$1/|x|<\varepsilon$$

即：

$$|x|>1/\varepsilon$$

取 X=1/ ε ，那么当|x|>X 时，|1/x - 0|< ε 成立。

由于 X 存在，因此 $\lim_{x\to\infty} \frac{1}{x} = 0$ 得证。

如图 10-11 所示，通过对 f(x)=1/x 的函数图像观察，可以直观看出 $\lim_{x\to\infty} \frac{1}{x} = 0$。

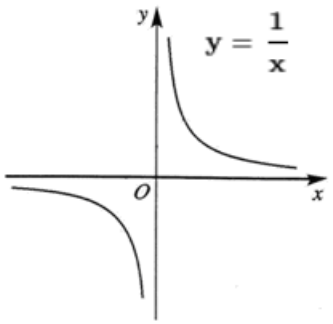



图 10-11 y=1/x 图像

同时，我们把 y=0 叫做函数 y=1/x 的水平渐近线。

与数列的极限类似，函数的极限也有如下性质：

- **唯一性**：如果函数 f(x)极限 limf(x)($x\to x_0$ 或 $x\to\infty$)存在，那么这极限唯一。
- **有界性**：如果 f(x)极限 limf(x)($x\to x_0$ 或 $x\to\infty$)存在，那么存在常数 M>0 和 δ>0，使得当 $0<|x-x_0|<\delta$ 时，有 $|f(x)|\leqslant M$。
- **保号性**：如果 f(x)极限 A=limf(x)($x\to x_0$ 或 $x\to\infty$)存在，且 A>0（或 A<0），那么存在常数 δ>0，使得当 $0<|x-x_0|<\delta$ 时，有 f(x)>0(或 f(x)<0)。
- **保号性推论**：如果在 $x_0$ 的某去心邻域内 $f(x)\geqslant 0$（或 $f(x)\leqslant 0$），而且 A=limf(x)($x\to x_0$ 或 $x\to\infty$)存在，那么 $A\geqslant 0$（或 $A\leqslant 0$）。

## 10.1.5 无穷小和无穷大

如果函数 f(x)当 $x\to x_0$（或 $x\to\infty$）时的极限为零，那么称函数 f(x)为当 $x\to x_0$（或 $x\to\infty$）时的无穷小。

以 f(x)=1/x 为例（参考图 10-10），通过图像可以看出，当 $x\to\infty$时，函数的极限为 0。因此，f(x)=1/x 是当 $x\to\infty$时的无穷小。

柯西把无穷小视为“以 0 为极限的变量”，这就正确地确立了“无穷小”概念为“似零不是零却可以人为用等于 0 处理”的办法，这就是说，在变量的变化过程中，它的值实际上不等于零，但它变化的趋向是向“零”，可以无限地接近于零。那么人们就可以用“等于 0”来处理，是不会产生错误结果的。

在自变量的同一变化过程 $x\to x_0$ （或 $x\to\infty$）中，函数 f(x)具有极限 A 的充分必要条件是 $f(x)=A+\alpha$，其中 α 是无穷小。

设函数 f(x)在 $x_0$ 的某一去心邻域内有定义（或|x|大于某一正数时有定义）。如果对于任意给定的正数 M（不论它多么大），总存在正数 δ（或正数 X），只要 x 适合不等式 $0<|x-x_0|<\delta$ （或|x|>X），对应的函数值 f(x)总满足不等式

$$|f(x)|>M,$$

则称函数 f(x)为当 $x\to x_0$ （或 $x\to\infty$）时的无穷大。如图 10-12 所示，y=1/(x-1)当 x→1 时，函数为无穷大，$y=x^2$ 当 $x\to\infty$，函数为无穷大。

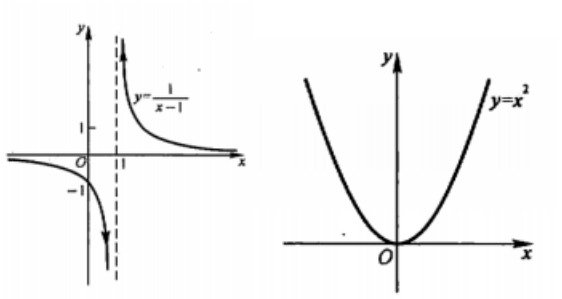



图 10-12 函数无穷大

在自变量的同一变化过程中，如果 f(x)为无穷大，则 1/f(x)为无穷小；反之，如果 f(x)为无穷小，且 f(x)≠0，则 1/f(x)为无穷大。

## 10.1.6 无穷小与第二次数学危机

恩格斯曾评价说："人类所取得的理论成就还没有什么像 17 世纪下半叶微积分的发现那样使人类整个精神达到了最高的胜利。"冯诺依曼也曾说："微积分是近代数学中最伟大的成就，对它的重要性无论作怎样的估计都不过分。"比如，在物理学领域，牛顿的万有引力定律，爱因斯坦的狭义和广义相对论、质能方程，麦克斯韦方程，薛定谔方程等都离不开以微积分为工具进行推导。

虽然 17 世纪牛顿和莱布尼茨以无穷小概念为基础建立了微积分取得了巨大成功，但后来遇到了无穷小带来的逻辑困难。比如，在物理学的"瞬时速度"概念中：

$$V=\Delta s/\Delta t(\Delta t\rightarrow 0)$$

究竟Δt（变化量）是否等于零？如果说等于零，怎么能用它去作除数呢？如果它不是零，计算和函数变形时又怎么能把包含着它的那些"微小的量"项去掉呢？

比如，在牛顿流数中研究 $x^n\rightarrow(x+o)^n$ 变化推导过程如下：

$$(x+o)^n=x^n+nx^{n-1}o+n(n-1)/2x^{n-2}o^2+\cdots$$

$$(x+o)^n-x^n=nx^{n-1}o+n(n-1)/2x^{n-2}o^2+\cdots$$

$$((x+o)^n-x^n)/o=nx^{n-1}+n(n-1)/2x^{n-2}o^1+\cdots$$

当 o 无限小的时候，保留 $nx^{n-1}$，而去掉后面部分，那么有：

$$((x+o)^n-x^n)/o=nx^{n-1}$$

在这里，o 有的时候被当作除数去除等式的两边，有的时候又被当做 0 而被丢弃掉。为此，爱尔兰主教贝克莱发表了《分析学家》专门攻击牛顿的流数术，指出他是从错误的原理出发，通过错误的抵消而获得。这被称为贝克莱悖论或无穷小悖论。当时人们不理解，想完全没有一点点误差地进行变量的计算从而导致了悖论。无穷小悖论直接导致了第二次数学危机。

18 世纪，许多数学家尝试弥补这个漏洞，但都没有真正解决这个问题。

19 世纪，法国数学家柯西在前人工作的基础上，比较完整地阐述了"极限概念"及其理论，他在《分析教程》中指出："当一个变量逐次所取的值无限趋于一个定值，最终使变量的值和该定值之差要多小就多小，这个定值就叫做所有其他值的极限值。特别地，当一个变量的绝对值无限地减小使之收敛到极限 0，就说这个变量成为无穷小。"柯西把无穷小视为"以 0 为极限的变量"，这就正确地确立了"无穷小"概念为"似零不是零去却可以人为用等于 0 处理"，这就是说，在变量的变化过程中，它的值实际上不等于零，但它变化的趋向是向"零"，可以无限地接近于零。那么人们就可以用"等于 0"来处理，是不会产生错

误结果的。柯西对极限和无穷小的定义和解释，非常的直观，通俗易懂。但作为数学科学，这种说法还不够严格。

后来，德国数学家魏尔斯特拉斯（1815-1897）提出了极限的ε－N，ε－δ，ε－X抽象定义，这个抽象定义定量地、具体地刻划了两个“无限过程”之间的联系，给微积分提供了严格的理论基础。这个定义是目前比较严格的定义，可作为科学论证的基础，因此至今仍在数学书籍中使用。

至此，第二次数学危机完美解决，时间已经过去近200年。

## 10.1.7 极限运算法则

虽然我们可以结合极限的定义来证明和确定数列或者函数的极限，但这样非常不便，更多的时候我们是通过极限的运算法则来进行极限计算的。

极限的运算法则包括四则运算法则（即极限的加减乘除）和复合函数的极限运算法则。有了这些法则之后，我们就可以利用这些运算法则计算更多函数的极限。下面我们就列出这些极限的运算法则。如无特别说明，limf(x)是指当 $x\to x_0$ 和 $x\to\infty$ 两种情况下结论同时成立。

1.两个无穷小和是无穷小。有限个无穷小和是无穷小。

2.有界函数与无穷小的乘积是无穷小。常数与无穷小的乘积是无穷小；有限个无穷小的乘积是无穷小。

3.数列或函数加减乘除的极限等于极限的加减乘除。如果 limf(x)=A, limg(x)=B，那么：

$$\lim[f(x)\pm g(x)]=\lim f(x)\pm\lim g(x)=A\pm B;$$

$$\lim[f(x)\cdot g(x)]=\lim f(x)\cdot\lim g(x)=A\cdot B;$$

（3）若又有 $B\neq 0$，则

$$\lim\frac{f(x)}{g(x)}=\frac{\lim f(x)}{\lim g(x)}=\frac{A}{B}.$$

$$\lim[cf(x)]=c\lim f(x)$$

$$\lim[f(x)]^n=[\lim f(x)]^n$$

4.如果 f(x)>g(x)，则 limf(x)>limg(x)。

5.复合函数极限运算法则：设函数y=f[g(x)]是由函数u=g(x)与函数y=f(u)复合而成，f[g(x)]在点 $x_0$ 的某去心邻域内有定义。

当 $x\to x_0$ 时：

$$\lim g(x)=u_0$$

当 $u\to u_0$ 时：

$$\lim f(u)=A$$

且存在 $\delta_0>0$，当 $x\in\mathring{U}(x_0,\delta_0)$ 时，有 $g(x)\neq u_0$，则有：

$$\lim_{x\to x_0}f[g(x)]=\lim_{u\to u_0}f(u)=A$$

运用这些极限运算法则，我们就可以很方便的求出某些函数的极限，具体运用的例子，请参考本书10.1.11节。

## 10.1.8 极限存在准则与两个重要极限

现在我们来介绍极限存在的夹逼规则，以及对应的两个重要极限。

**1.夹逼规则**

对于数列有：如果数列$\{x_n\}$，$\{y_n\}$及$\{z_n\}$满足如下条件：

从某项起，即$\exists n_0 \in N$，当$n>n_0$时，有：

$$y_n \leqslant x_n \leqslant z_n \text{ 且 } \lim y_n=\lim z_n=a,$$

那么数列$\{x_n\}$的极限存在，且

$$\lim x_n=a$$

对于函数有：如果函数 f(x), g(x)和 h(x)满足：

当$x \in U(x_0, r)$（或$|x|>M$）时，有：

$$g(x) \leqslant f(x) \leqslant h(x) \text{ 且 } \lim g(x)=\lim h(x)=A(\text{当 } x \to x_0 \text{ 或 } x \to \infty \text{时})$$

那么有：

$$\lim f(x)=A(\text{当 } x \to x_0 \text{ 或 } x \to \infty \text{时})$$

这就是极限存在的夹逼规则。

**2.单调有界数列必有极限**

利用夹逼规则，我们可以证明下面的两个重要极限（证明过程略）：

- 当$x \to 0$，$\lim \sin x/x=1$;
- 当$x \to \infty$，$\lim (1+1/x)^x=e$; $x \to 0$，$\lim (1+x)^{1/x}=e$ （e 为自然常数）

知道这两个极限之后，我们可以用它来求更多相关的函数极限。在这里，我们发现自然常数 e 作为一个无理数，它实际上就是一个极限值。我们可以想办法算出 e 的值为：2.71828...（参考本书 10.3.7 节）。

现在我们来更多的认识和了解一下这个自然常数 e。

实际上，在以前人们还没有明确的极限概念时，e 这个值就已经存在于复利计息之中了。复利计息就是利息也可以并进本金再生利息，如果计息周期（年、月、日）无限制地缩短，比如说每分钟计息一次，甚至每秒，或者每一瞬间计息一次，会发生什么状况?本利之和会无限制地加大吗?答案是不会，它的值会稳定下来，趋近一极限值，这个值就是 e。但在那时候，由于人们还没有极限的观念，因此 e 的值应该是观察出来的，而不是用严谨的证明得到的。

以 e 为底的指数函数$y=e^x$的重要方面在于它与其导数相等（参考本书 10.2 节）。在 e 被发现之前，对数在很长的一段时间里是以 10 为底的，被称为“常用对数”，也就是 lgx。常用对数简化了复杂计算，但依然显得十分烦琐，直到将 e 做为对数的底时，也就是 lnx，才使一些计算变得更加简约。

有着人类最美公式之称的“欧拉公式”为：

$$e^{i\pi}+1=0$$

欧拉将 e、π、虚数单位 i 和自然数单位 1 以及人类历史上重要的发明“0”统一在了同一个公式中，它将指数函数的定义域扩大到复数，建立了三角函数和指数函数的关系。

自然常数 e 的存在其实还不限于数学领域。大自然中太阳花的种子排列、蜘蛛网、夜空中星系的旋臂，以及如图 10-13 所示鹦鹉螺壳上的花纹都呈现螺线的形状，而螺线的方程式，是要用 e 来定义的。如果把一条链子两端固定，自然垂下，它呈现的形状若用数学式子表示的话，也需要用到 e。欧拉最早用 e 来命名和表示自然常数这个无理数的。

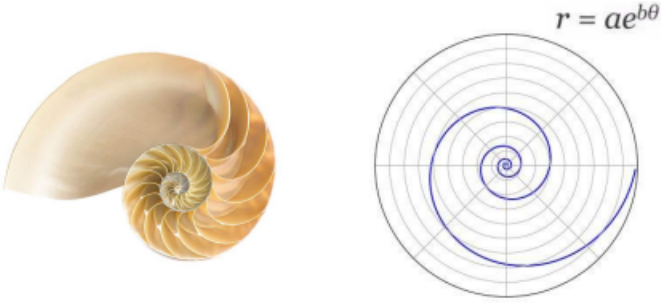



图 10-13 鹦鹉螺与螺线

注意，C 语言中的 1e-6 这种写法，e 不是自然常数 e，它表示的是 $10^{-6}$。

## 10.1.9 无穷小比较

我们已经知道，两个无穷小的和，差以及乘积都是无穷小。那么两个无穷小的商会是什么结果呢？有可能是 0，1，常数或者∞，因此需要分情况来讨论。

两个无穷小之比的极限出现不同情况，实际上是反映了不同无穷小趋于零的“快慢”程度各不相同。有的快，有的慢，有的速度相差不多。比如：

当 x→0 时：

- $\lim x^3/x=0$，那么 $x^3$ 比 x 趋于 0 快一些
- $\lim x/x^3=\infty$，那么 x 比 $x^3$ 趋于 0 慢一些
- $\lim \sin x/(2x)=1/2$，sinx 和 2x 趋近于 0 的速度差不多

于是，设 α 和 β 是两个无穷小，那么我们有关于两个无穷小比值的如下定义：

- 如果 $\lim(\beta/\alpha)=0$，就说 β 是比 α 高阶的无穷小，记作 $\beta=o(\alpha)$；
- 如果 $\lim(\beta/\alpha)=\infty$，就说 β 是比 α 低阶的无穷小；
- 如果 $\lim(\beta/\alpha)=c\neq0$，就说 β 与 α 同阶无穷小；
- 如果 $\lim(\beta/\alpha^k)=c\neq0$，$k>0$，就说 β 与 α 的 k 阶无穷小；
- 如果 $\lim(\beta/\alpha)=1$，就说 β 与 α 等价无穷小，记作 $\alpha\sim\beta$；
- β 与 α 是等价无穷小的充分必要条件为 $\beta=\alpha+o(\alpha)$；
- 设 $\alpha\sim\alpha'$，$\beta\sim\beta'$，且 $\lim(\beta'/\alpha')$ 存在，则 $\lim(\beta/\alpha)=\lim(\beta'/\alpha')$。

一些常见的等价无穷小有：sinx~x， tanx~x，arcsinx~x，$1-\cos x\sim 1/2x^2$，$(1+x)^{\alpha}-1\sim\alpha x$ 等。这些等价无穷小，常常被用于求极限中的等价变换（参考后面求极限的例子）。

## 10.1.10 函数连续性

现实世界里，温度变化，植物生长，河水流动都是连续的。这些连续的变化，体现在函数关系上即函数的连续性。

**1.连续性定义**

如图 10-14 所示，令 $\Delta y=f(x_0+\Delta x)-f(x_0)$，当 $\lim \Delta y=0$ ($\Delta x\to0$)或者 $\lim f(x)=f(x_0)$ ($x\to x_0$)，那么函数在 $x_0$ 处连续。函数在 $x_0$ 连续，顾名思义，就是在 $x_0$ 这一点没有间断，不存在间断点。

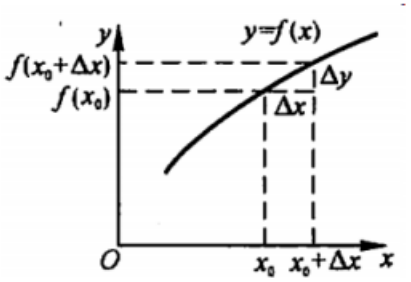



图 10-14 函数连续定义

**左连续**

若 $f(x_0^-) = f(x_0)$，那么函数在 $x_0$ 处左连续。

**右连续**

若 $f(x_0^+) = f(x_0)$，那么函数在 $x_0$ 处右连续。

函数在 $x_0$ 处连续当且仅当函数在 $x_0$ 处左连续和右连续。

**2.间断点**

如果函数在该点 $x_0$ 无定义；有定义，但极限不存在；有定义，极限存在，但 $\lim f(x) \neq f(x_0)$，那么 $x_0$ 就是该函数的一个间断点。

练习 5：试讨论函数 y=sin(1/x)的连续性。

解：函数在 x=0 处没有定义，所以，函数在 x=0 处不连续，且当 x→0 时，函数值在-1 和 1 之间变动（震荡）无限多次。

如图 10-15 所示，函数间断点可以分为两类：

- 第一类间断点：可去间断点（有极限），跳跃间断点（左右极限，但不相等）
- 第二类间断点：无穷间断点；震荡间断点。

如果一个函数在它的定义域或定义域子区间连续，那么在该区间，它的图像是一条没有间断点的连续曲线。

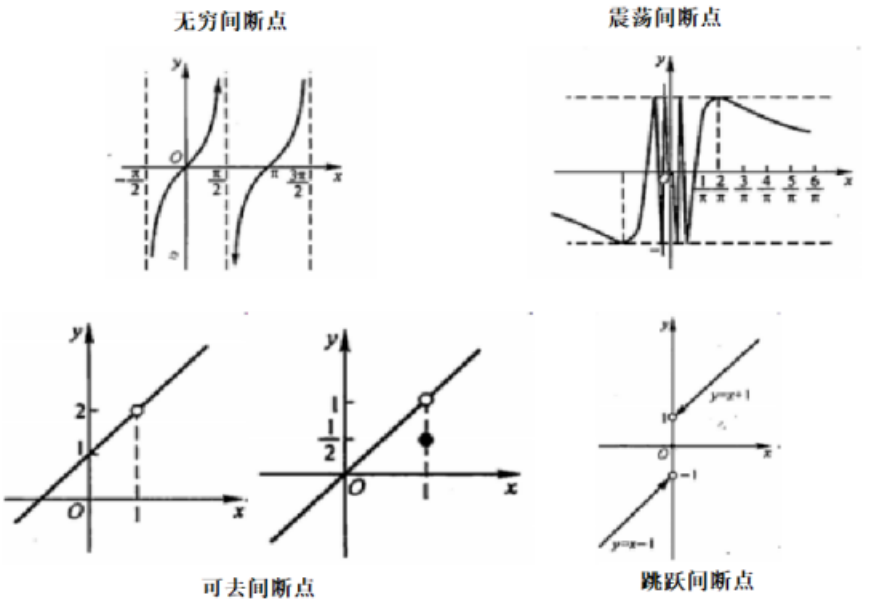



图 10-15 函数无穷间断点，震荡间断点，可去间断点和跳跃间断点

函数在点 $x_0$ 连续，那么函数的加减乘除反函数在该点也都连续；复合函数 u=g(x)和 y=f(u)分别在 $x_0$ 和 $u_0$ 连续，那么复合函数 f.g(x)也在 $x_0$ 连续。

初等函数在定义域内都是连续的，因此有：

$$\lim_{x \to x_0} f(x) = f(x_0)$$

那么对于任何初等函数，如果要求在某点 $x_0$ 的极限，只需要将该点代入函数求得该点函数的值即可。

## 10.1.11 函数连续性性质

通过对函数连续的研究，我们可以得出有关连续函数拥有一些比较重要的性质和定理，比如有界性和最大值最小值定理；零点定理；以及介值定理。现在我们来介绍一下这些定理的原理，暂时忽略对它们的证明。如果大家想进一步学习这些定理的证明，可以参考高等数学相关书籍。

**有界性与最大值最小值定理** 在闭区间上连续的函数在该区间上有界且一定能取得它的最大值和最小值，如图 10-16 所示。

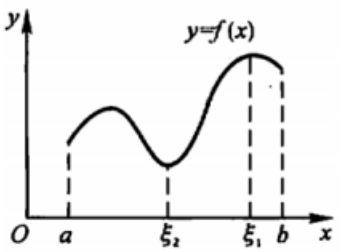



图 10-16 有界性与最大值最小值

**零点定理** 设函数 f(x)在闭区间[a,b]上连续，且 f(a)与 f(b)异号（即 f(a)·f(b)<0），那么在开区间(a,b)内至少有一点 ξ，使 f(ξ)=0，如图 10-17 所示。

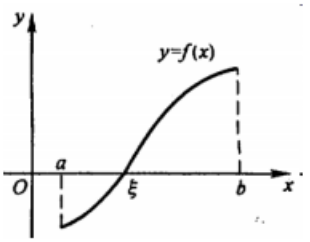


图 10-17 零点定理

**介值定理** 设函数 f(x)在闭区间[a,b]上连续，且在这区间的端点取不同的函数值 f(a)=A 及 f(b)=B,那么，对于 A 与 B 之间的任意一个数 C，在开区间(a,b)内至少有一点 ξ（有关区间中存在一点的定理，所以被称为介值定理），使得 f(ξ)=C(a<ξ<b)，如图 10-18 所示。

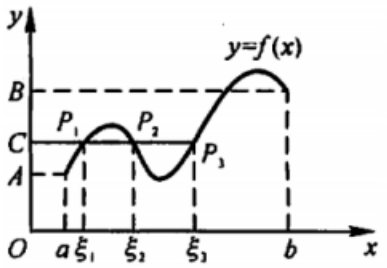


图 10-18 介值定理

## 10.1.12 求极限例子

前面我们主要利用极限的定义并通过证明的方法来求得一些函数或者数列的极限，实际上，大部分情况下，我们并不需要那么做，而是直接利用求极限的方法来求出极限。求函数或者数列的极限，是一种重要的运算技能，也是后面求导数的基础。求极限的方法很多，有的还比较复杂。现在我们举一些常见的求极限的例子和方法。

前面我们已经证明了几个常见函数的极限，比如：limc=c（当 x→$x_0$）；limx=$x_0$（当 x→$x_0$）；lim(sinx/x)=1（当 x→0）；lim $(1+1/x)^x$=e（当 x→∞）；lim$(1+x)^{1/x}$=e（当 x→0）。而且，我们也知道了，初等函数在定义域内都是连续的，因此：

$$\lim_{x \to x_0} f(x) = f(x_0)$$

基于这些结论，现在我们来练习求一些函数的极限。

练习 6：$\lim_{x \to 1}(2x-1)$

解：因为 f(x)=2x-1 为初等函数，所以当 x→1 时，lim(2x-1)=2*1-1=1

实际上，如果$f(x) = a_0x^n + a_1x^{n-1} + \ldots + a_n$是一个关于 x 的 n 次多项式，那么有如下极限计算公式：

$$\lim_{x \to x_0} f(x) = a_0x^n + a_1x^{n-1} + \ldots + a_n = f(x_0)$$

练习 7：$\lim_{x \to 3} \dfrac{x-3}{x^2-9}$

解：当 x→3 时：

$$\lim(x-3)/(x^2-9)=\lim(x-3)/(x-3)(x+3)=\lim 1/(x+3)=1/6$$

注意，在这里 x→3 时，x≠3，所以 x-3≠0，因此分子分母是可以约掉 x-3 的。

练习 8：$\lim\limits_{x\to\infty}\dfrac{sinx}{x}$

解：当 x→∞时，因为 1/x→0，sinx 为有界函数，因此：

当 x→∞时，lim(sinx/x)=0

练习 9：$\lim\limits_{x\to\infty}\dfrac{3x^3+4x^2+2}{7x^3+5x^2-3}$

解：当 x→∞时，对原有理分式的分子分母同时除以 $x^3$ 有：

$$\lim(3x^3+4x^2+2)/(7x^3+5x^2-3)=\lim(3+4/x+2/x^3)/(7+5/x-3/x^3)=3/7$$

实际上，对于如下的有理分式，当 $a_0\neq 0$，$b_0\neq 0$ 时有如下求极限的公式：

$$\lim_{x\to\infty}\frac{a_0x^m+a_1x^{m-1}+\cdots+a_m}{b_0x^n+b_1x^{n-1}+\cdots+b_n}=\begin{cases}\dfrac{a_0}{b_0}, 当\ n=m,\\ 0, 当\ n>m,\\ \infty, 当\ n<m.\end{cases}$$

练习 10：$\lim\limits_{x\to\infty}(1-\dfrac{1}{x})^x$

解：零 t=-1/x，x=-1/t 当 x→∞时，t→0。因此原式的极限转化为求下列式子的极限。当 t→0 时：

$$\lim(1+t)^{(-1/t)}=1/\lim(1+t)^{(1/t)}=1/e$$

练习 11：$\lim\limits_{x\to\ 0}(\dfrac{1-cosx}{x^2})$

解：因为当 x→0 时，1-cosx~1/2$x^2$，所以

$$\lim_{x\to\ 0}(\frac{1-cosx}{x^2})=\lim_{x\to\ 0}(\frac{\frac{1}{2}x^2}{x^2})=\frac{1}{2}$$

练习 12：$\lim\limits_{x\to\ 0}\dfrac{sinx}{x^3+3x}$

解：当 x→0 时，因为 sinx~x，所以：

limsinx/($x^3$+3x)=limx/($x^3$+3x)=lim1/($x^2$+3)=1/3

练习 13：$\lim\limits_{x\to\ 0}\dfrac{(1+x^2)^{\frac{1}{3}}-1}{cosx-1}$

解：因为当 x→0 时，$(1+x^2)^{1/3}$-1~1/3$x^2$，cosx-1~-1/2$x^2$ 所以：

$$\lim((1+x^2)^{1/3}-1)/(\cos x-1)=\lim(1/3x^2)/(-1/2x^2)=-2/3$$

## 10.2 导数

有了函数和极限理论基础之后，现在我们来学习导数这个重要的概念。导数（Derivative），也叫导函数值，又名微商，是微积分中一个非常重要的基础概念。与微积

分初期导数的定义建立在无穷小这个概念上不同，现在导数的定义完全建立在函数和极限的理论之上。

## 10.2.1 导数的概念和意义

导数可分为函数在某点的导数或者函数在整个区间上的导函数。导数在物理运动学和几何中都有明确的意义。首先我们来看函数在某点的导数定义。

**1.函数在某点导数**

设函数 y=f（x）在点 $x_0$ 的某个邻域内有定义，当自变量 x 在 $x_0$ 处有增量Δx，且 $x_0+\Delta x$ 也在该邻域内时，相应地函数值取得增量Δy=f（$x_0+\Delta x$）-f（$x_0$）；如果Δy 与Δx 之比当Δx→0 时极限存在，则称函数 y=f（x）在点 $x_0$ 处可导，并称这个极限为函数 y=f（x）在点 $x_0$ 处的导数并记作$f'(x_0)$，即：

$$f'(x_0)=\lim_{\Delta x\to 0}\frac{\Delta y}{\Delta x}=\lim_{\Delta x\to 0}\frac{f(x_0+\Delta x)-f(x_0)}{\Delta x}$$

如果Δy 与Δx 之比当Δx→0 时极限不存在，则称函数 y=f（x）在点 $x_0$ 处不可导。

因为Δx→0 时，x→$x_0$，所以，函数在某点的导数也可以定义为下面极限的形式：

$$f'(x_0)=\lim_{x\to x_0}\frac{f(x)-f(x_0)}{x-x_0}$$

**2.导函数**

如果函数在一个开区间 I 内每点处都可导，那么对于任意 x∈I，都对应着一个 f(x)确定的导数值，这种对应关系实际上就是一个新的函数。我们把这个新的函数叫作原来函数 y=f(x)的导函数，记作：

y'，f'(x)，dy/dx 或者 df(x)/dx

将函数在某点的导数定义中的 $x_0$ 用 x 去替代，就得出了导函数的极限定义形式：

$$y'=\lim_{\Delta x\to 0}\frac{f(x+\Delta x)-f(x)}{\Delta x}$$

也可以用 h 去替代Δx，那么上述导函数的极限定义形式也可以写成如下形式：

$$f'(x)=\lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$$

在导函数的定义中需要注意的是，在求极限的时候，x 是常量，Δx 或 h 才是变量。显而易见的是，函数 f(x)在点 $x_0$ 处的导数 f'($x_0$)就是导函数 f'(x)在点 $x=x_0$ 处的函数值，即：

$$f'(x_0)=f'(x)|_{x=x_0}$$

导函数 f'(x)简称导数，f'($x_0$)是 f(x)在 $x_0$ 处的导数，或者导数 f'(x)在 $x_0$ 处的值。我们可以看出，导数的定义是基于前面极限的定义基础之上的，导数实际上是函数在自变量增量Δx→0 的极限。

那么导数究竟有没有什么特别的意义呢？

实际上，导数反应了变量变化的“快慢”，即反应了因变量随自变量的变化而变化的快慢程度。这种变化的快慢，在物理运动学里体现为质点的瞬时速度，或者几何学里切线的斜率。

**质点瞬时速度**

导数是函数的局部性质。函数在某一点的导数描述了在这一点附近的变化率。导数的本质是通过极限的概念对函数进行局部的线性逼近。

在运动学中，物体的位移对于时间的导数就是物体的瞬时速度。

在直线运动中，在直线上指定一个原点，将直线视为数轴，设动点于时刻 t 在直线上的位置坐标为 s，则 s 是时间 t 的某个函数，记为：

$$s=f(t)$$

通过运动学知识，我们知道，在匀速直线运动中，动点的运动速度为：

$$v=s/t=f(t)/t$$

但如果动点不是匀速运动，那么动点在某一时刻的速度，就不能使用上述公式求得了。那么如何求得非匀速运动中动点的某一时刻($t_0$)的速度呢？

假设动点从时刻 $t_0$ 运动到 t，在这个时间间隔内，动点从位置 $s_0=f(t_0)$移动到 s=f(t)。

那么，动点在 $t-t_0$ 这个时间间隔内，运动的平均速度可以表示为：

$$\frac{s-s_0}{t-t_0}=\frac{f(t)-f(t_0)}{t-t_0}$$

当然，平均速度并不能代表动点在 $t_0$ 时刻的速度，如果时间间隔 $t-t_0$ 较短，那么平均速度可以近似说明动点在 $t_0$ 时刻的速度，但依然是不精确的。

对于动点在 $t_0$ 时刻的速度，可以通过极限和导数的方法求出，即：

$$v=\lim_{t\to t_0}\frac{f(t)-f(t_0)}{t-t_0}$$

如果这个极限（即函数 f(t)在 $t_0$ 的导数）存在，那么该极限值 v 称为动点在时刻 $t_0$ 的瞬时速度。

导数在物理运动学的运用，科学的定义和解释了质点的瞬时速度。这也解决了古代著名的芝诺悖论中关于飞矢不动的悖论。

芝诺悖论（Zeno's paradox）是古希腊数学家芝诺（Zeno of Elea）提出的 4 个关于运动的不可分性的哲学悖论，包括二分法悖论，阿基里斯追不上乌龟，飞矢不动悖论，运动场悖论。这些悖论由于被记录在亚里士多德的《物理学》一书中而为后人所知。其中的飞矢不动悖论指出：任何东西占据一个与自身相等的处所时是静止的，飞着的箭在任何一个瞬间总是占据与自身相等的处所，所以也是静止的。射出去的箭，它的飞行时间细分成无数个短的不能再短的时间点，那么在这一个个时间点上，这只箭一直都是静止的，因而这支箭的飞行过程实际上只是它不断的停驻在某一点上，而不是从一个位置飞到另一个位置。

现在我们知道了，时间和空间都是连续的，通过微积分可以求出物体的瞬时速度。物理既然有速度，那么就是运动而不是静止的。因此飞矢不动不成立。

**切线的斜率**

导数在几何学的领域的含义就是函数曲线在某点的切线的斜率。首先，为大家介绍一下什么是切线。

我们已经知道，圆的切线就是和圆只有一个交点的直线。但是这种定义方法并不适合一般的曲线切线定义。比如抛物线 $y=x^2$，在 O 点可以画出很多和它只有一个交点的直线，但只有 y=0（即 x 轴）是符合切线定义的。

现在，我们先来研究一下过曲线某点的切线的定义。

如图 10-19 所示，设函数 y=f(x)的曲线为 C，设 $M(x_0, y_0)$是曲线 C 上的一个点。在点 M 外另取 C 上的一个点 N(x, y)，作割线 MN。当点 N 沿曲线 C 趋于点 M 时，如果割线 MN 绕点 M 旋转而趋于极限位置 MT，直线 MT 就称为曲线 C 在点 M 处的切线。

只要求出切线 MT 的斜率，那么根据直线的点斜式方程，就可以确定过 M 点的切线方程了。

要求切线 MT 的斜率，先来研究一下割线 MN 的斜率。根据直线斜率定义，割线 MN 的斜率为：

$$\tan\varphi=\frac{y-y_0}{x-x_0}=\frac{f(x)-f(x_0)}{x-x_0}$$

其中 φ 为割线 MN 的倾角。当点 N 沿曲线 C 趋于点 M 时，$x\to x_0$。当 $x\to x_0$ 时，如果下式的极限存在，即：

$$k=\lim_{x\to x_0}\frac{f(x)-f(x_0)}{x-x_0}$$

那么极限 k 就是割线斜率的极限，也就是切线 MT 的斜率。因此，$k=\tan\alpha$，其中 α 是切线 MT 的倾角。

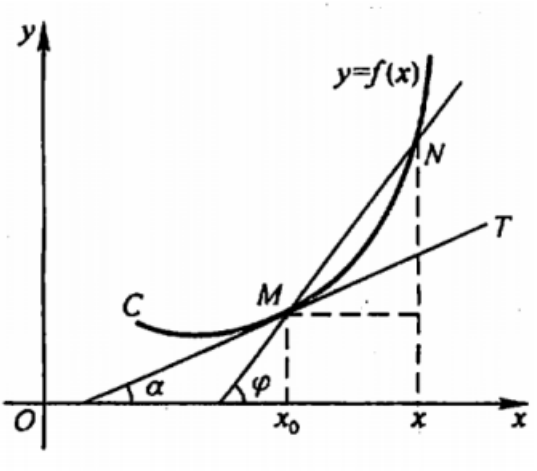



图 10-19 过点 M 的切线

## 10.2.2 根据定义求导数

现在，我们通过一些简单的练习，来演示一下如何通过导数的定义来求函数的导数。并把这些练习的结论作为更多函数导数求法的基础。

**1.根据定义求导数**

练习 1：求函数 f(x)=C（C 为常数）的导数

解：$f'(x)=\lim(f(x+\Delta x)-f(x))/\Delta x$(当 $\Delta x\to 0$ 时)$=\lim(C-C)/\Delta x$(当 $\Delta x\to 0$ 时)$=0$

也就是说，常数的导数为零，这是可以理解的，因为常数是没有变化的，而导数反应的就是函数因变量随自变量的变化而变化的快慢程度，常数是不变的，所以导数就应该为零。

练习 2：求函数 $f(x)=x^{\mu}$（$\mu\in R$)的导数。

解：因为：

$$(f(x+\Delta x)-f(x))/(\Delta x)=((x+\Delta x)^{\mu}-x^{\mu})/(\Delta x)=x^{\mu-1}(((1+\Delta x/x)^{\mu}-1)/(\Delta x/x))$$

当 $\Delta x\to 0$ 时：

$$\lim(((1+\Delta x/x)^{\mu}-1)/(\Delta x/x))=\lim\mu(\Delta x/x)/(\Delta x/x)=\mu$$

因此

$$f'(x)=\lim x^{\mu-1}(((1+\Delta x/x)^{\mu}-1)/(\Delta x/x))= \mu x^{\mu-1}\ （当\Delta x\to 0\ 时）$$

比如，当 $\mu=1/2$ 时，$y=x^{1/2}$ 的导数为：

$$y'=1/2x^{-1/2}$$

当 $\mu=2$ 时，$y=x^2$ 的导数为：

$$y'=2x$$

练习 3：求函数 $f(x)=a^x(a>0, a\neq 1)$的导数。

解：因为

$$(f(x+\Delta x)-f(x))/(\Delta x)=(a^{x+\Delta x}-a^x)/\Delta x=a^x(a^{\Delta x}-1)/\Delta x$$

因此

$$f'(x)=\lim a^x(a^{\Delta x}-1)/\Delta x=a^x\lim(a^{\Delta x}-1)/\Delta x=a^x\ln a\ （当\Delta x\to 0\ 时）$$

注意：

$$\lim(a^{\Delta x}-1)/\Delta x=\ln a\ （当\Delta x\to 0\ 时）$$

也就是说:

$$(a^x)'= a^x\ln a。$$

当 a=e 的时候，lne=1，所以

$$(e^x)'=e^x\ln e=e^x$$

也就是说，以 e 为底的指数函数的导数不变，就是它自己。

**2.左导数和右导数**

上面我们介绍了导数的定义，它是下列式子的一个极限：

$$f'(x)=\lim_{h\to 0}\frac{f(x+h)-f(x)}{h}$$

极限存在的充分必要条件就是左右极限存在且相等。因此我们把上述式子的左极限：

$$f'_-(x_0)=\lim_{h\to 0^-}\frac{f(x_0+h)-f(x_0)}{h}$$

称为函数 f(x)在 $x_0$ 处的左导数，记作 $f'_-(x_0)$；而它的右极限：

$$f'_+(x_0)=\lim_{h\to 0^+}\frac{f(x_0+h)-f(x_0)}{h}$$

称为函数 f(x)在 $x_0$ 处的右导数，记作 $f'_+(x_0)$。函数 f(x)在 $x_0$ 处可导的充分必要条件是：左导数和右导数存在且相等。

练习 4：求函数 f(x)=|x|在 x=0 处的导数。

解：当 $\Delta x\to 0$ 时

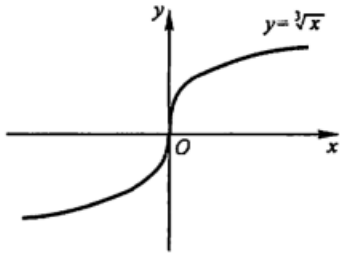
$$\lim(f(0+\Delta x)-f(0))/\Delta x=\lim|\Delta x|/\Delta x$$

当 $\Delta x<0$ 时，即 $\Delta x\to 0^-$，$\lim|\Delta x|/\Delta x=\lim -\Delta x/\Delta x=-1$，即 $f'_-(0)=-1$

当 $\Delta x>0$ 时，即 $\Delta x\to 0^+$，$\lim|\Delta x|/\Delta x=\lim \Delta x/\Delta x=1$，即 $f'_+(0)=1$

因为 $f'_-(0)\neq f'_+(0)$，因此函数 f(x)=|x|在 x=0 处不可导。

函数在某点可导，那么在此点必定连续（也就是说不连续一定不可导）；但连续不一定可导。不是所有的函数都有导数，一个函数也不一定在所有的点上都有导数。如图 10-20 所示的两个函数 $y=x^{1/3}$ 和 y=|x|，在 x=0 处虽然连续，但却不可导。



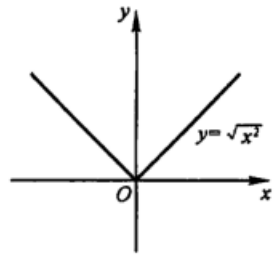



图 10-20 函数连续但不可导

因为：函数 $y=x^{1/3}$ 在 x=0 处的导数为+∞（切线垂直于 x 轴）；而函数 y=|x|，在 x=0 处的左右导数分别为-1 和 1，左右导数虽然存在但不相等。

如果一个函数 f(x)在开区间(a,b)内可导，且 $f'_+(a)$和 $f'_-(b)$都存在，那么函数 f(x)在闭区间[a, b]上可导。

## 10.2.3 和、差、积、商求导法则

通过导数的定义，我们求出了基本初等函数的导数。基于导数的定义和基本初等函数的导数，我们很容易证明如下关于导数和、差、积、商求导公式，这些公式如下：

$$[u(x)\pm v(x)]'=u'(x)\pm v'(x);$$

$$[u(x)v(x)]'=u'(x)v(x)+u(x)v'(x);$$

$$\left[\frac{u(x)}{v(x)}\right]'=\frac{u'(x)v(x)-u(x)v'(x)}{v^2(x)}\quad(v(x)\neq 0)$$

这些求导公式可以简记为：

$$(u\pm v)'=u'\pm v'$$

$$(uv)'=u'v+uv'$$

$$\left(\frac{u}{v}\right)'=\frac{u'v-uv'}{v^2}$$

练习 5：$y=2x^3-5x^2+3x-7$，求 y'。

解：

$$y'=2*3x^2-5*2x+3=6x^2-10x+3。$$

练习 6：$y=e^x(\sin x+\cos x)$，求 y'。

解：根据导数乘法运算法则，有：

$$y'=(e^x)'(\sin x+\cos x)+e^x(\sin x+\cos x)'=e^x(\sin x+\cos x)+e^x(\cos x-\sin x)=2e^x\cos x$$

练习 7：y=tanx，求 y'。

解：y=tanx=sinx/cosx，因此根据商的求导公式有：

$$y'=(\sin x/\cos x)'=((\sin x)'\cos x-\sin x(\cos x)')/\cos^2 x=(\cos^2 x+\sin^2 x)/\cos^2 x=1/\cos^2 x=\sec^2 x$$

类似的，设y=secx，那么有：y'=secxtanx。

## 10.2.4 反函数求导

如果函数x=f(y)在区间$I_y$内单调可导且f'(y) ≠0，那么它的反函数y=$f^{-1}$(x)在区间$I_x$={x|x=f(y), y∈$I_y$}内也可导。因为函数y=$f^{-1}$(x)连续，根据连续定义有 lim Δy=0(当Δx→0)即当Δx→0时，Δy→0。

因此有：

$$[f^{-1}(x)]' = \lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} = \lim_{\Delta y \to 0} \frac{1}{\frac{\Delta x}{\Delta y}} = \frac{1}{f'(y)}$$

也就是说，反函数的导数等于直接函数导数的倒数。

练习8：设x=siny，y∈[-π/2，π/2]，求y=arcsinx的导数。

解：

y'=1/x'=1/siny'=1/cosy

再把y用x来表示。因为：

$\cos^2 y+\sin^2 y=1$，且cosy>0

所以

$$\cos y = \sqrt{1-\sin^2 y} = \sqrt{1-x^2}$$

因此

$$(\arcsin x)' = \frac{1}{\sqrt{1-x^2}}$$

利用类似的反函数求导的方法，可以求出其它反三角函数的导数。

练习9：求y=$\log_a$x的导数。

解：因为y=$\log_a$x是x=$a^y$的反函数，因此y'=1/x'=1/($a^y$)'=1/($a^y$lna)

又因为y=$\log_a$x代入有：

y'=1/(xlna)

特别地，当a=e时有：

lnx'=1/x

## 10.2.4 复合函数求导

如果u=g(x)在点x可导，而y=f(u)在点u=g(x)可导，则复合函数y=f[g(x)]在点x可导，且其导数为：

dy/dx=f'(u).g'(x)或 dy/dx=dy/du.du/x

练习10：$y=e^{x^3}$，求dy/dx。

解：$y=e^{x^3}$可以看作是$u=x^3$, $y=e^u$复合而成。因此：

dy/dx=dy/du.du/dx=($e^u$)'.u'=$e^{x^3}$.$3x^2$=$3x^2e^{x^3}$

练习11：y=lnsinx，求dy/dx。

解：y=lnsinx 可以看作是 y=lnu 和 u=sinx 复合而成。因此根据复合函数求导有：

$$dy/dx=dy/du.du/dx=lnu'.u'=(1/sinx).cosx=ctgx$$

## 10.2.5 高阶导数

一般函数 y=f(x)的导数 y'=f'(x)依然是 x 的函数，因此可以继续求导。因此，y'=f'(x)的导数叫做函数 y=f(x)的二阶导数，记作 y"或者 $d^2y/dx^2$，即：

$$y''=(y')'或\ d^2y/dx^2=d(dy/dx)/dx$$

同时，我们把 f'(x)叫做函数 y=f(x)的一阶导数。

类似的，二阶导数的导数叫做三阶导数，三阶导数的导数叫做四阶导数，与此类推，n-1 阶导数的导数叫做 n 阶导数。1 到 n 阶导数可以记作：

$$y', y'', y''', y^{(4)},...y^{(n)}$$

或者：

$$dy/dx, d^2y/dx^2, d^3y/dx^3, ...d^ny/dx^n$$

由于牛顿和莱布尼茨共同创立了微积分，当时牛顿对于导数采用点(.)来表示，被称为点主义，如图 10-21 所示。

$\dot{y}$ $\ddot{y}$ $\dddot{y}$

图 10-21 点主义

而莱布尼茨使用 d 来表示，被称为 d 主义，如图 10-22 所示：

$\frac{dy}{dx}$ $\frac{d^2y}{dx^2}$

图 10-22 d 主义表示方法

函数 y=f(x)具有 n 阶导数，则称函数 f(x) n 阶可导。二阶及二阶以上的导数统称为高阶导数。

练习 12：求 y=ax+b 的二阶导数。

解：y'=a, y''=(y')'=(a)'=0。

练习 13：求 $y=e^x$ 的 n 阶导数。

解：$y'=e^x$, $y''=(y')'=(e^x)'=e^x$, $y'''=(y'')'=(e^x)'=e^x$, …, $y^{(n)}=e^x$。

从练习 13 可以看出，指数函数 $e^x$ 的 n 阶导数是不变的。

**2.基本初等函数的导数**

通过上面的练习，我们已经得出了一些常见函数的导数。现在我们把这些函数的导数总结如下。

| | |
|---|---|
| $y=C$（C为常数） | $y'=0$ |
| $y=a^x$<br>$y=e^x$ | $y'=a^x\ln a$<br>$y'=e^x$ |
| $y=x^n$ | $y'=nx^{n-1}$ |
| $y=\log_a x$<br>$y=\ln x$ | $y'=\frac{1}{x\ln a}$<br>$y'=\frac{1}{x}$ |
| $y=\sin x$ | $y'=\cos x$ |
| $y=\cos x$ | $y'=-\sin x$ |
| $y=\tan x$ | $y'=\sec^2 x$ |
| $y=\cot x$ | $y'=-\csc^2 x$ |
| $y=\sec x$ | $y'=\sec x\tan x$ |
| $y=\csc x$ | $y'=-\csc x\cot x$ |
| $y=\arcsin x$ | $y'=\frac{1}{\sqrt{1-x^2}}$ |
| $y=\arccos x$ | $y'=-\frac{1}{\sqrt{1-x^2}}$ |
| $y=\arctan x$ | $y'=\frac{1}{1+x^2}$ |
| $y=\operatorname{arccot} x$ | $y'=-\frac{1}{1+x^2}$ |
| $y=\operatorname{sh} x$ | $y'=\operatorname{ch} x$ |

表 10-1 函数导数表

表 10-1 是常见函数导数表，记住求导公式的口诀：

- 常为零，幂降次；
- 对倒数（e 为底时直接倒数，a 为底时乘以 1/lna）；
- 指不变（特别的，自然对数的指数函数完全不变，一般的指数函数须乘以 lna）；
- 正变余，余变正；
- 切割方（切函数是相应割函数（切函数的倒数）的平方）；
- 割乘切，反分式。

## 10.2.6 多元函数偏导数

含有一个自变量的函数叫一元函数；含有多个自变量的函数叫多元函数。以二元函数 z=f(x, y)为例（从二元函数很容易推广到多元函数），如果只有自变量 x 变化，而自变量 y 固定（看作常量），这时它就等同于 x 的一元函数，求函数对 x 的导数，就称为二元函数 z=f(x, y)对于 x 的偏导数，记作：

$$\frac{\partial z}{\partial x},\ \frac{\partial f}{\partial x},\ z_x \quad 或 \quad f_x(x,y)$$

其中∂ 符号是由法国数学家拉格朗日创造的偏导符号，读着 round 或者 partial，意为偏导的意思。

类似的，如果只有自变量 y 变化，而自变量 x 固定（看作常量），这时它就等同于 y 的一元函数，求函数对 y 的导数，就称为二元函数 z=f(x, y)对于 y 的偏导数，记作：

$$\frac{\partial z}{\partial y},\ \frac{\partial f}{\partial y},\ z_y \quad 或 \quad f_y(x,y)$$

练习 14：求 $z=x^2+3xy+y^2$ 在点(1, 2)处的偏导数。

解：把 y 看作常量，则：

$$\frac{\partial z}{\partial x}=2x+3y$$

把 x 看作常量，则：

$$\frac{\partial z}{\partial y}=3x+2y$$

于是：

$$\left.\frac{\partial z}{\partial x}\right|_{\substack{x=1\\y=2}}=2*1+3*2=8$$

$$\left.\frac{\partial z}{\partial y}\right|_{\substack{x=1\\y=2}}=3*1+2*2=7$$

练习 15：求 $z=x^2\sin 2y$ 的偏导数。

解：把 y 当做常量，则：

$$\frac{\partial z}{\partial x}=2x\sin 2y$$

把 x 当做常量，则：

$$\frac{\partial z}{\partial y}=2x^2\cos 2y$$

设二元函数 z=f(x, y)在定义域 D 内有偏导数：

$$\frac{\partial z}{\partial x}=f_x(x,y),\ \frac{\partial z}{\partial y}=f_y(x,y)$$

那么 $f_x(x, y)$, $f_y(x, y)$都是 x，y 的函数，如果它们的偏导数也存在，则称它们是函数 z=f(x, y)的二阶偏导数，记为：

$$\frac{\partial}{\partial x}\left(\frac{\partial z}{\partial x}\right)=\frac{\partial^2 z}{\partial x^2}=f_{xx}(x,y),\ \frac{\partial}{\partial y}\left(\frac{\partial z}{\partial x}\right)=\frac{\partial^2 z}{\partial x\partial y}=f_{xy}(x,y)$$

$$\frac{\partial}{\partial x}\left(\frac{\partial z}{\partial y}\right)=\frac{\partial^2 z}{\partial y\partial x}=f_{yx}(x,y),\ \frac{\partial}{\partial y}\left(\frac{\partial z}{\partial y}\right)=\frac{\partial^2 z}{\partial y^2}=f_{yy}(x,y)$$

并且容易证明如果二阶偏导数$\frac{\partial^2 z}{\partial y\partial x}$ 和$\frac{\partial^2 z}{\partial x\partial y}$在定义域内连续，那么有：

$$\frac{\partial^2 z}{\partial y\partial x}=\frac{\partial^2 z}{\partial x\partial y}$$

## 10.2.7 方向导数

事实上，多元函数偏导数反映的是函数沿坐标轴方向的变化率，而方向导数反应了函数 f(x, y)在 $P_0$ 点沿某一方向的变化率。现在我们来看看方向导数的定义。

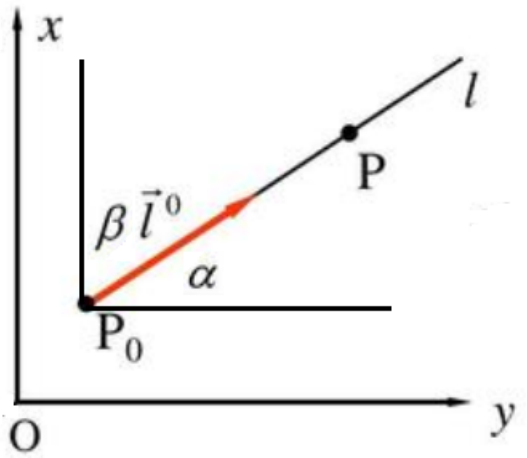



图 10-23 方向导数

如图 10-23 所示，设 α 、β 分别为直线 l 与 x 轴正方向、y 轴正方向的夹角。α 、β 共同表示直线 l 的方向。假设在直线 l 上的点 $P_0(x_0, y_0)$，函数 z=f(x, y)在 $P_0$ 的某个邻域 $U(P_0)$ 内有定义，当从 $P_0$ 点沿直线 l 正方向移动距离 t 达到点 P(x, y)时，二元函数自变量 x 增加为 $x_0+t\cos\alpha$，自变量 y 增加为 $y_0+t\cos\beta$。我们把函数沿该直线 l 方向的方向导数定义为：

$$\frac{\partial f}{\partial l}\Big|_{(x_0,y_0)} = \lim_{t\to 0}\frac{f(x_0+t\cos\alpha, y_0+t\cos\beta)-f(x_0,y_0)}{t}$$

设函数 z=f(x, y)在点(x, y)的某邻域内有定义，如果函数在点(x, y)的全增量：

$$\Delta z=f(x+\Delta x, y+\Delta y)-f(x, y)$$

可表示为：

$$\Delta z=A\Delta x+B\Delta y+o(\rho)$$

其中 A, B 不依赖与 Δx，Δy 而仅与 x，y 有关，$\rho=\sqrt{(\Delta x)^2+(\Delta y)^2}$

则称函数 z=f(x, y)在点(x, y)可微分，而 $A\Delta x+B\Delta y$ 称为函数 z=f(x, y)在点(x, y)的全微分，记为 dz。

如果函数 z=f(x, y)在点(x, y)可微分，则该函数在点(x, y)的偏导数$\frac{\partial z}{\partial x}$、$\frac{\partial z}{\partial y}$必定存在，且全微分为：

$$dz=\frac{\partial z}{\partial x}\Delta x+\frac{\partial z}{\partial y}\Delta y$$

如果函数 z=f(x, y)的偏导数$\frac{\partial z}{\partial x}$、$\frac{\partial z}{\partial y}$在点(x, y)连续，则函数在该点可微分。

事实上，如果函数 f(x, y)在点 $P_0(x_0, y_0)$可微分，那么函数在该点沿任一方向 l（方向余弦为 cosα，cosβ，其中 α，β 分别是直线 l 与 x 轴和 y 轴正向方向的夹角，$0\leqslant\alpha\leqslant\pi$，$0\leqslant\beta\leqslant\pi$，$0\leqslant\gamma\leqslant\pi$）的方向导数存在，且有：

$$\frac{\partial f}{\partial l}\Big|_{(x_0,y_0)} = f_x(x_0,y_0)\cos\alpha+f_y(x_0,y_0)\cos\beta$$

### 10.2.8 梯度

设二元函数 z=f(x, y)在定义域 D（属于 x0y 平面）内具有一阶连续偏导数，则对于每一个点 $P_0(x_0, y_0)\in D$，都可以定出一个向量：

$$f_x(x_0, y_0)i+f_y(x_0, y_0)j$$

这向量称为函数 f(x, y)在点 $P_0(x_0, y_0)$的梯度，记作：gradf$(x_0, y_0)$或$\nabla f(x_0, y_0)$，即：

$$\text{gradf}(x_0, y_0)=\nabla f(x_0, y_0)=f_x(x_0, y_0)i+f_y(x_0, y_0)j$$

其中$\nabla=\frac{\partial}{\partial x}i+\frac{\partial}{\partial y}j$，称为二维向量的微分算子（Nabla 算子），比如，$\nabla f=\frac{\partial f}{\partial x}i+\frac{\partial f}{\partial y}j$。

如果函数 f(x, y)在点 $P_0(x_0, y_0)$可微分，$e_l=(\cos\alpha, \cos\beta)$是与方向 l 同方向的单位向量，则：

$$\left.\frac{\partial f}{\partial l}\right|_{(x_0,y_0)}=f_x(x_0, y_0)\cos\alpha+f_y(x_0,y_0)\cos\beta=\text{gradf}(x_0,y_0).e_l=|\text{gradf}(x_0, y_0)|\cos\theta$$

其中 θ 为向量 $e_l$ 与梯度 gradf$(x_0, y_0)$的夹角。可以看出，函数在某点的方向导数，是函数在这一点的梯度与这个方向的单位向量的数量积。

当 θ=0 时，cos θ=1，即 $e_l$ 与梯度 gradf$(x_0, y_0)$同向时，函数 f(x, y)在这个方向的方向导数达到最大值，因此函数在这个方向增加最快，即：

$$\left.\frac{\partial f}{\partial l}\right|_{(x_0,y_0)}=|\mathbf{grad}\ f(x_0, y_0)|$$

因此函数 f(x, y)在一点的梯度 grand*f* 是一个这样的向量，它的方向是函数在这点的方向导数取得最大值的方向，它的模就等于方向导数的最大值。

当 θ=π 时，cos θ=-1，即 $e_l$ 与梯度 gradf$(x_0, y_0)$的方向相反时，函数 f(x, y)在这个方向的方向导数达到最小值，函数 f(x, y)在这个方向减少最快，即：

$$\left.\frac{\partial f}{\partial l}\right|_{(x_0,y_0)}=-|\mathbf{grad}\ f(x_0, y_0)|$$

当 θ=π/2 时，cos θ=0，即 $e_l$ 与梯度 gradf$(x_0, y_0)$的方向正交时，函数的变化率为零，即：

$$\left.\frac{\partial f}{\partial l}\right|_{(x_0,y_0)}=|\mathbf{grad}\ f(x_0, y_0)|\cos\theta=0$$

思考题：求函数 $f(x, y)=1/2(x^2+y^2)$在点(1, 1)处增长最快和最慢的方向以及沿这个方向的方向导数。

## 10.3 微分

在学完导数的概念之后，现在我们来学习微分。微分和导数联系密切，微分和我们后面将要学习的积分互为逆运算关系。

牛顿和莱布尼茨将微分及积分两个貌似不相关的问题，透过“微积分基本定理”或“牛顿—莱布尼茨公式”联系起来，说明求积分基本上是求微分之逆，求微分也是求积分之逆。这是微积分理论中的基石，是微积分发展一个重要的里程碑。

## 10.3.1 微分定义

在正式介绍微分概念之前，我们先来研究一个最常见的幂函数 $y=x^2$。当自变量 x 从 $x_0$ 增加到 $x_0+\Delta x$ 时，函数值的增加量：

$$\Delta y=(x_0+\Delta x)^2-x_0^2=2x_0\Delta x+(\Delta x)^2$$

这里可以看出，函数值的增量 $\Delta y$ 由自变量增量 $\Delta x$ 线性部分 $2x_0\Delta x$ 和 2 次项部分 $(\Delta x)^2$ 组成，且当 $\Delta x\to 0$ 时，$(\Delta x)^2$ 是 $\Delta x$ 的高阶无穷小。

设函数 y=f(x)在某区间内有定义，$x_0$ 及 $x_0+\Delta x$ 在这区间内，如果增量：

$$\Delta y=f(x_0+\Delta x)-f(x_0)$$

可表示为：

$$\Delta y=A\Delta x+o(\Delta x)$$

其中 A 是不依赖于 $\Delta x$ 的常数，那么称函数 y=f(x)在点 $x_0$ 是可微的，而 $A\Delta x$ 叫做函数 y=f(x)在点 $x_0$ 相应于自变量增量 $\Delta x$ 的微分，记作 dy，即：

$$dy=A\Delta x$$

那么 A 的值是如何确定的呢？微分和导数有什么关系呢？下面我们来分析这两个问题。

因为如果函数可微，即：

$$\Delta y=A\Delta x+o(\Delta x)$$

两边同时除以Δx，所以有：

$$\Delta y/\Delta x=A+o(\Delta x)/\Delta x$$

当 $\Delta x\to 0$ 时：

$$A=\lim \Delta y/\Delta x=f'(x_0)$$

因此，如果函数可微，那么一定可导，且：

$$A=f'(x_0)。$$

另外一方面，如果函数 y=f(x)在点 $x_0$ 可导，即：

$$\lim \Delta y/\Delta=f'(x_0)(当\ x_0\to 0\ 时)$$

那么根据极限与无穷小的关系有：

$$\Delta y/\Delta x=f'(x_0)+\alpha\ （其中当\Delta x\to 0\ 时，\ \alpha\to 0）$$

因此：

$$\Delta y=f'(x_0)\Delta x+\alpha\Delta x$$

由于 $f'(x_0)$是常数，$\alpha\ \Delta x=o(\Delta x)$，因此 $\Delta y=A\Delta x+o(\Delta x)$, 即函数 f(x)在 $x_0$ 也是可微的。

因此函数 f(x)在点 $x_0$ 可微的充分必要条件是函数 f(x)在点 $x_0$ 可导，而且微分与导数的关系是：

$$dy=f'(x_0)\Delta x$$

当 $x_0\to 0$ 时，由于 $\lim \Delta y/dy=\lim \Delta y/(f'(x_0)\Delta x)=1/f'(x_0)\lim \Delta y/\Delta x=f'(x_0)/f'(x_0)=1$，所以 dy 与 $\Delta y$ 是等价无穷小。

因此有：

$$\Delta y=dy+o(\Delta y)$$

这里，dy 是 $\Delta y$ 的线性主部（当 $x_0\to 0$ 时）。当 $f'(x_0)\neq 0$ 时，$\Delta y\approx dy$，误差为 $o(\Delta y)$。

通常把自变量 x 的增量 $\Delta x$ 称为自变量的微分，记作 dx，即 $dx=\Delta x$（因为自变量在 x 轴上，它的增量都是线性的），于是函数 y=f(x)的微分通常记作：

$$dy=f'(x)dx$$

函数的微分 dy 与自变量的微分 dx 之商等于该函数的导数（所以也叫做微商），即：

$$dy/dx=f'(x)$$

如图 10-24 所示，当|Δx|很小时，|Δy-dy|比|Δx|小得多，局部用切线段|MP|近似代替曲线段|MN|，非线性函数的局部线性化，这是微分学的基本思想方法。

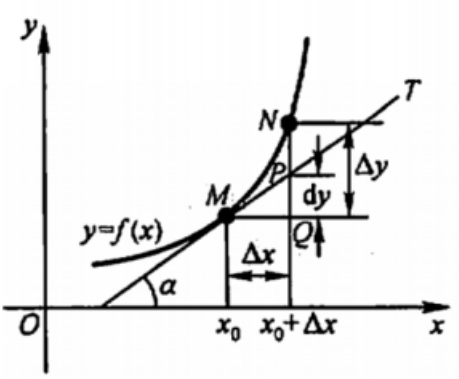



图 10-24 微分以直代曲

微分概念是在解决直与曲的矛盾中产生的，在微小局部可以用直线去近似替代曲线，它的直接应用就是函数的线性化。微分具有双重意义：它表示一个微小的量，因此就可以把线性函数的数值计算结果作为函数的数值近似值，这就是运用微分方法进行近似计算的基本思想。

通过微分与导数之间的关系，即：

$$dy=f'(x)dx$$

因此，只要求出函数的导数，那么很容易得出函数的微分。

## 10.3.2 微分的应用-近似计算

微分一个重要的应用就是自然科学上的近似计算。

当 $\Delta x\to 0$ 时，因为：

$$\Delta y\approx dy=f'(x_0)\Delta x$$

又因为：

$$\Delta y=f(x_0+\Delta x)-f(x_0)$$

所以：

$$f(x_0+\Delta x)-f(x_0)\approx f'(x_0)\Delta x$$

令 $x=x_0+\Delta x$，那么 $\Delta x=x-x_0$，代入上式有：

$$f(x)-f(x_0)\approx f'(x_0)(x-x_0)$$

因此：

$$f(x)\approx f(x_0)+f'(x_0)(x-x_0)$$

也就是说，我们可以用关于 x 的线性函数 $f(x_0)+f'(x_0)(x-x_0)$来近似的表达函数 f(x)。若令 $x_0=0$，那么上式又可以写为：

$$f(x)\approx f(0)+f'(0)x$$

比如，对于 sinx 来说，(sinx)'=cosx，sin(0)=0，cos(0)=1，代入上式有：

$$\sin x\approx x$$

类似的，我们可以得到工程上常用的如下几个近似表达式：

- $(1+x)^{1/n} \approx 1+(1/n)x$
- $\sin x \approx x$（x 为弧度制数值）
- $\tan x \approx x$（x 为弧度制数值）
- $e^x \approx 1+x$
- $\ln(1+x) \approx x$

练习 16：求 sin30° 30′ 的值。

解：30° 30′ =π/6+0.5°/180*π=π/6+π/360。

设 $x_0=\pi/6$，$\Delta x=\pi/360$，$\sin x_0=\sin \pi/6$，$\sin'(x0)=\cos x_0=\cos \pi/6$

因此：

$$\sin(x_0+\Delta x) \approx \sin(x_0)+\cos(x_0)\Delta x$$

即：

$$\sin(\pi/6+\pi/360) \approx \sin(\pi/6)+\cos \pi/6*\pi/360=0.5076$$

练习 17：计算 $\sqrt{1.05}$ 的近似值。

解：$\sqrt{1.05}$ 可以看成是函数 $f(x)=(1+x)^{1/2}$ 当 x=0.05 时的值。

因为：

$$(1+x)^{1/2} \approx 1+1/2x=1+1/2*0.05=1.025$$

而直接开根号有：

$$\sqrt{1.05}=1.02470$$

其误差 1.025-1.02470<0.001，这在一般工程应用上已经足够精确了。

### 10.3.3 微分中值定理

微分中值定理包括罗尔定理，拉格朗日中值定理，柯西中值定理，以及泰勒中值定理等，它们的关系如图 10-25 所示，通过费马引理，可以推出罗尔定理，而通过罗尔定理可以推导出拉格朗日中值定理和柯西中值定理。拉格朗日中值定理可以用于函数图形研究。而柯西中值定理可以用于洛必达法则求极限和泰勒中值定理的证明。

由于这些定理都是关于在区间(a, b)中至少存在一点 ξ 的定理，所以称为中值定理。

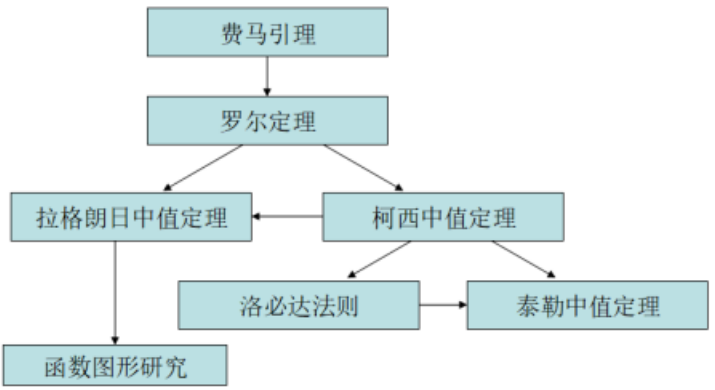



图 10-25 微分中值定理

现在我们简要介绍一下这些中值定理的原理，而暂时忽略对它们的证明。关于这些定理的证明，大家可以找到专门的高等数学书籍来进一步参考和学习。

**1.罗尔定理**

如图 10-26 所示，如果函数 f(x)满足：

- 在闭区间[a, b]上连续；
- 在开区间(a, b)内可导；
- 在区间端点处的函数值相等，即 f(a)=f(b)；

那么在(a, b)内至少有一点 ξ （a<ξ<b），使得 f'(ξ)=0。

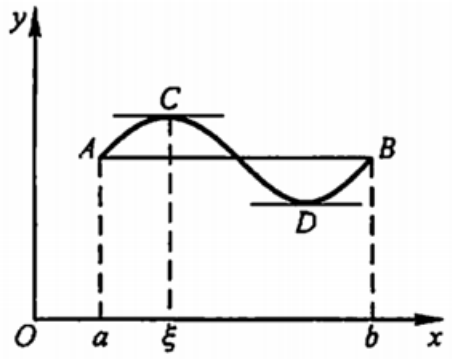



图 10-26 罗尔定理

**2.拉格朗日中值定理**

如图 10-27 所示，如果函数 f(x)满足：

- 在闭区间[a, b]上连续；
- 在开区间(a, b)内可导；

那么在(a, b)内至少有一点 ξ (a<ξ<b)，使等式 f(b)-f(a)=f'(ξ)(b-a)。

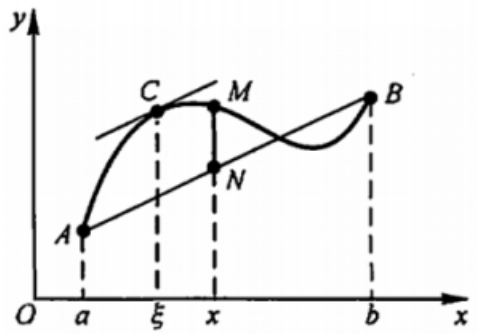



图 10-27 拉格朗日中值定理

**3.柯西中值定理**

如果函数 f(x)及 F(x)满足：

- 在闭区间[a, b]上连续；
- 在开区间(a, b)内可导；
- 对任一 x∈(a, b)，F'(x)≠0；

那么在(a, b)内至少有一点 ξ，使得等式：

$$\frac{f(b)-f(a)}{F(b)-F(a)}=\frac{f'(\xi)}{F'(\xi)}$$

成立。

## 10.3.4 函数单调性判断

设函数 y=f(x)在[a, b]上连续，在(a, b)内可导：

- 如果在(a, b)内 f'(x)>0，那么函数 y=f(x)在[a, b]上单调增加；
- 如果在(a, b)内 f'(x)<0，那么函数 y=f(x)在[a, b]上单调减少。

此定理可以通过拉格朗日中值定理证明。

练习 18：讨论函数 $y=e^x-x-1$ 的单调性。

解：对函数 y 求导有：$y'=e^x-1$，所以：

- 当 $e^x-1>0$，即 $x>0$ 时，$y'>0$，所以函数单调递增；
- 当 $e^x-1<0$，即 $x<0$ 时，$y'<0$，所以函数单调递减。

## 10.3.5 曲线凹凸性

函数的图像除了单调性之外，还有凹凸性的区别。如图 10-28 所示，即使同样的单调递增区间，函数的图像有可能是凹的（曲线 ADB 所代表的函数图像），也有可能是凸的（曲线 ACB 代表的函数图像）。

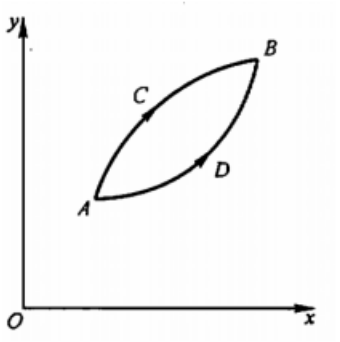



图 10-28 函数的凹凸性

如图 10-29 所示，设 f(x)在区间 I 上连续，如果对 I 上任意两点 $x_1$，$x_2$ 恒有 $f((x_1+x_2)/2)<(f(x_1)+f(x_2))/2$ 那么称 f(x)在 I 上的图形是向上凹的；如果恒有 $f((x_1+x_2)/2)>(f(x_1)+f(x_2))/2$,那么称 f(x)在 I 上的图形是向上凸的。

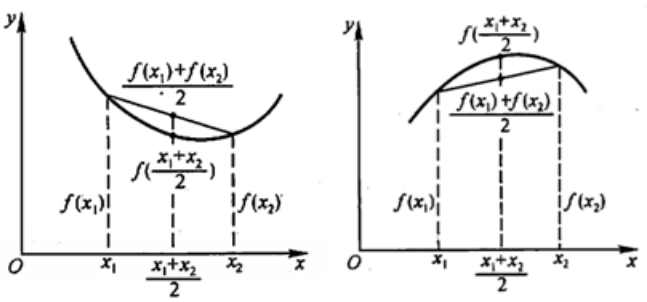



图 10-29 函数凹凸性定义

设 f(x)在[a, b]上连续，在(a, b)内具有一阶和二阶导数，那么：

- 若在(a, b)内 f''(x)>0，则 f(x)在[a, b]上的图形是凹的；
- 若在(a, b)内 f''(x)<0，则 f(x)在[a, b]上的图形是凸的；

此定理同样可以通过拉格朗日中值定理给予证明。一阶导数表示函数随自变量变化的规

律，在函数曲线上表示的是某个点的斜率；二阶导数表示的导数的变化规律，如果函数是一条曲线，且曲线存在二阶导数，那么二阶导数表示的是曲线的曲率，曲率越大，曲线越是弯曲。

练习 19：判定曲线 $y=x^3$ 的凹凸性。

解：$y'=3x^2, y''=6x$。

因此，当 $x>0$ 时，$y''>0$，因此函数 $y=x^3$ 是凹的；当 $x<0$ 时，$y''<0$，因此函数 $y=x^3$ 是凸的。

设 $y=f(x)$在区间 I 上连续，$x_0$ 是 I 内的点（非端点），如果 $y=f(x)$的曲线经过点$(x_0, f(x0))$时，曲线的凹凸性发生了改变（凸变凹或者凹变凸），那么就称点$(x_0, f(x0))$为曲线的拐点。

求函数拐点的具体步骤为：

1.求 f''(x);

2.令 f''(x)=0，解出在区间 I（定义域，连续）的实根，并求出 I 内不存在的点；

3.对于 2 中的每个实根或不存在的点 $x_0$，检查 f''(x)在 $x_0$ 左右两侧附近的符号，那么当两侧符号相反时，点$(x_0, f(x_0))$是拐点，相同时，不是拐点。

练习 20：求曲线 $y=2x^3+3x^2-12x+14$ 的拐点。

解：由于：$y'=6x^2+6x-12$，$y''=12x+6$

令 $y''=0$，即：

$$12x+6=0$$

得 $x=-1/2$。当 $x>-1/2$ 时，$y''>0$；当 $x<-1/2$ 时，$y''<0$。

因此 $x=-1/2$ 是函数曲线的拐点。

## 10.3.6 一元函数极值和最值

设函数 f(x)在点 $x_0$ 的某邻域 $U(x_0)$内有定义，如果对于去心邻域 $\mathring{U}(x_0)$ 内的任一 x 有

$$f(x)<f(x_0)\ （或\ f(x)>f(x_0)）$$

那么就称 $f(x_0)$是函数 f(x)的一个极大值（或极小值）。

函数的极大值与极小值统称为函数的极值。极值中的最大的便是最大值；最小的便是最小值。

如何取得函数的极值呢？

设函数 f(x)在 $x_0$ 处可导并取得极值，那么有 $f'(x_0)=0$。函数导数为零的点，通常被称为驻点。如图 10-30 所示，极值点一定是驻点（图中 a 极大值点和 b 极小值点），但驻点不一定是极值点（图中 c 和 d，我们称之为鞍点）。

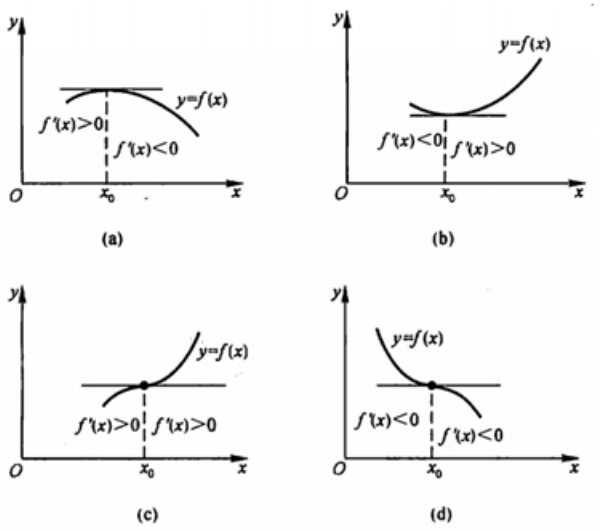



图 10-30 函数极值判断

我们可以通过一阶导数或者二阶导数来判断函数的极值和最值。

**一阶导数判断方法**

设函数 f(x)在 $x_0$ 处连续，且在 $x_0$ 的某去心邻域$\mathring{U}(x_0,\delta)$内可导。那么：

- 若 $x\in(x_0-\delta,x_0)$时 $f'(x)>0$，$x\in(x_0,x_0+\delta)$时，$f'(x)<0$，则 f(x)在 $x_0$ 处取得极大值。
- 若 $x\in(x_0-\delta,x_0)$时 $f'(x)<0$，$x\in(x_0,x_0+\delta)$时，$f'(x)>0$，则 f(x)在 $x_0$ 处取得极小值。
- 若 $x\in\mathring{U}(x_0,\delta)$时 f'(x)的符号保持不变，则 f(x)在 $x_0$ 处没有极值。

**二阶导数判断方法**

设函数 f(x)在 $x_0$ 处具有二阶导数且 $f'(x_0)=0$，$f''(x_0)\neq0$，那么：

- 当 $f''(x_0)<0$ 时，函数 f(x)在 $x_0$ 处取得极大值；
- 当 $f''(x_0)>0$ 时，函数 f(x)在 $x_0$ 处取得极小值。

**求极值步骤**

1.首先，求出 f(x)一阶和二阶导数 f'(x)和 f''(x)；

2.其次，求出 f(x)的全部驻点与不可导点；

3.然后，考察 f''(x)的符号以确定该点是否为极值点；如果是极值点，进一步确定是极大值点还是极小值点；

4.紧接着，求出各极值点的函数值，就得函数 f(x)的全部极值；

5.最后，极值中的最大的便是最大值；最小的便是最小值。

练习 21：求函数 $f(x)=(x^2-1)^3+1$ 的极值。

解：求函数的一阶和二阶导数：

$$y'=6x(x^2-1)^2,\ y''=6(x^2-1)(5x^2-1)$$

令 y'=0 求得函数的驻点：

$$6x(x^2-1)^2=0,\ x_1=0,\ x_2=-1,\ x_3=1$$

f''(0)=6>0，因此 f(x)在 x=0 时取极大值。

f''(-1)=f''(1)=0，无法判断极值问题。

对于点 x=-1，在 x 取-1 左侧的值时 f'(x)<0，取-1 右侧的值时 f'(x)<0，所在 f(x)在 x=-1 处没有极值。在 x=1 处同样也没有极值。

**多元函数极值和最值**

对于多元函数的极值和最值的求法，依然可以通过函数的偏导数来求出。首先，我们给出多元函数极值存在的必要和充分条件。

**必要条件**：设函数 $z=f(x, y)$在点$(x_0, y_0)$具有偏导数，且在点$(x_0, y_0)$处有极值，则有：

$$f_x(x_0, y_0)=0, f_y(x_0, y_0)=0$$

**充分条件**：设函数 $z=f(x, y)$在点$(x_0, y_0)$的某邻域内连续且有一阶及二阶连续偏导数，且 $f_x(x_0, y_0)=0, f_y(x_0, y_0)=0$，令：

$$f_{xx}(x_0, y_0)=A, f_{xy}(x_0, y_0)=B, f_{yy}(x_0, y_0)=C$$

则 $f(x, y)$在$(x_0, y_0)$处是否取得极值的条件为：

- $AC-B^2>0$ 时具有极值，且当 $A<0$ 时有极大值，当 $A>0$ 时有极小值；
- $AC-B^2<0$ 时没有极值；
- $AC-B^2=0$ 时可能有极值也可能没有极值。

练习 22：求函数 $f(x, y)=x^3-y^3+3x^2+3y^2-9x$ 的极值。

解：求出函数的一阶偏导：

$f_x(x, y)=3x^2+6x-9$

$f_y(x, y)=-3y^2+6y$

联立方程：

$$\begin{cases} f_x(x, y)=3x^2+6x-9=0, \\ f_y(x, y)=-3y^2+6y=0, \end{cases}$$

求解得驻点有:

$$(1, 0), (1, 2), (-3, 0), (-3, 2)$$

求该函数的二阶偏导有：

$f_{xx}(x, y)=6x+6$

$f_{xy}(x, y)=0$

$f_{yy}(x, y)=-6y+6$

在点$(1, 0)$处，$A=f_{xx}(1, 0)=12$，$B=f_{xy}(x, y)=0$，$C=f_{yy}(x, y)=6$，$AC-B2=12*6-0=72>0$，并且 $A>0$，所以在该点极值存在，且取极小值，$f(1, 0)=-5$。

在点$(1, 2)$处，$A=f_{xx}(1, 2)=12$，$B=f_{xy}(1, 2)=0$，$C=f_{yy}(1, 2)=-6$，$AC-B2=12*(-6)-0=-72<0$，所以在该点没有极值。

在点$(-3, 0)$处，$A=f_{xx}(-3, 0)=-12$，$B=f_{xy}(-3, 0)=0$，$C=f_{yy}(-3, 0)=6$，$AC-B2=-12*(6)-0=-72<0$，所以在该点没有极值。

在点$(-3, 2)$处，$A=f_{xx}(-3, 2)=-12$，$B=f_{xy}((-3, 2)=0$，$C=f_{yy}((-3, 2)=-6$，$AC-B2=-12*(-6)-0=72>0$，并且 $A<0$，所以在该点极值存在，且取极大值，$f(-3, 2)=31$。

**条件极值与拉格朗日乘数法**

如果在求函数极值的时候，除了限制自变量在函数的定义域内，没有任何其他限制条件，那么我们把这种极值叫作无条件极值。但有时候我们对函数的自变量还需要一些附加条件，在这些附加条件下求得的极值我们称为条件极值。

我们如何求函数的件极值呢？这里我们以一个具体的例子，给大家介绍一个求条件极值的方法，即拉格朗日乘数法。

求函数 $z=f(x, y)$在附加条件 $\phi(x, y)=0$ 下的可能极值点，做拉格朗日函数：

$$L(x,y)=f(x,y)+\lambda\varphi(x,y)$$

其中 λ 为参数，求其对 x 与 y 的一阶偏导数，并使之为零，与 φ(x, y)=0 联立方程：

$$\begin{cases} f_x(x,y)+\lambda\varphi_x(x,y)=0, \\ f_y(x,y)+\lambda\varphi_y(x,y)=0, \\ \varphi(x,y)=0. \end{cases}$$

解方程得到 x，y 及 λ 的值，这样得到的(x, y)就是函数 f(x, y)在附加条件 φ(x, y)=0 下的可能极值点。

练习 23：求表面积为 $a^2$ 而体积为最大的长方体的体积。

解：设长方体的长宽高分别为 x，y，z，则有：

表面积：$2(xy+xz+yz)=a^2$；

体积：V=xyz;

那么问题就是在：

$$\varphi(x, y, z)=2(xy+xz+yz)-a^2=0$$

的条件下，求函数：

$$V=xyz$$

的极大值。因此，作拉格朗日函数：

$$L(x, y, z)=xyz+\lambda(2xy+2xz+2yz-a^2)$$

对拉格朗日函数求 x，y，z 的偏导，并令其为零，联立方程组有：

$$\begin{cases} 2(xy+xz+yz)-a^2=0 \\ yz+2\lambda(y+z)=0 \\ xz+2\lambda(x+z)=0 \\ xy+2\lambda(y+x)=0 \end{cases}$$

解方程组，有：

$$x=y=z=\frac{\sqrt{6}}{6}a$$

这是唯一的一个极值点，而问题本身可知最大值一定是存在的，因此当表面积为 $a^2$ 的长方体中，边长为 $\frac{\sqrt{6}}{6}a$ 的正方体的体积为最大，最大值为 $\frac{\sqrt{6}}{36}a^3$。

## 10.3.7 泰勒公式

泰勒公式，也称泰勒展开式，是用一个函数在某点的信息（即函数在这一点的函数值信息，导数值信息等），描述其附近取值的公式。在已知函数在某一点的各阶导数值的情况下，泰勒公式可以利用这些导数值来做系数，构建一个多项式近似函数。

为什么要用多项式函数取近似一个函数呢？因为多项式，由于它本身的运算仅是有限项加减法和乘法，显得简单容易，所以在数值计算方面，多项式是人们最愿意使用的工具。如果一个非常复杂函数(比如 $\sqrt{x}$ , lgx, sinx, arcsinx 等)，想求其某点的值，直接求可能无法实现，这时候可以使用泰勒公式的多项式去逼近这个函数并求得函数在该点的近似值，这是泰勒公式的应用之一。泰勒公式在机器学习中主要应用于梯度迭代。

泰勒公式由英国数学家布鲁克·泰勒（1685-1731，如图 10-31 所示）提出。他在 1712 年的一封信里首次叙述了这个公式。拉格朗日在 1797 年之前，最先提出了带有余项的现在形式的泰勒定理。

图 10-31 泰勒

**1.一元函数泰勒展开**

现在我们来研究如何将一元函数 f(x)展开为一个多项式函数。现在的问题就是：设 f(x)在 $x_0$ 处具有 n 阶导数，那么如何找出一个在 $x_0$ 处与函数 f(x)近似的关于$(x-x_0)$的 n 次多项式 $P_n(x)$呢？记 $P_n(x)$为：

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)^2 + ... + a_n(x - x_0)^n$$

这时 $f(x) \approx P_n(x)$，并记两者的误差为 $R_n(x)=f(x)-P_n(x)$，那么 $R_n(x)$是当 $x \to x_0$ 时比$(x-x_0)^n$高阶的无穷小。

假设 $P_n(x)$在 $x_0$ 处的函数值及它的直到 n 阶导数在 $x_0$ 处的值与 $f(x_0), f'(x_0), \ldots, f^{(n)}(x_0)$相等，即：

$$P_n(x_0)=f(x_0),\quad P'_n(x_0)=f'(x_0), P''_n(x_0)=f''(x_0), \ldots, P_n^{(n)}(x_0)=f^{(n)}(x_0)$$

再对 $P_n(x)$求各阶导数有：

$P_n(x_0)=a_0=f(x_0)$

$P_n'(x_0)=a_1=f'(x_0)$

$P_n''(x_0)=2!a_2=f''(x_0)$

...

$P_n^{(n)}(x_0)=n!a_n=f^{(n)}(x_0)$

进一步就可以确定 $P_n(x)$的各项系数：

$$a_0=f(x_0),\ a_1=f'(x_0),\ a_2=f''(x_0)/2!,\ \ldots,\ a_n=f^{(n)}(x_0)/n!$$

将确定的系数代入 $P_n(x)$，有:

$$P_n(x)= f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)(x - x_0)^2}{2!} + \cdots + \frac{f^{(n)}(x_0)(x - x_0)^n}{n!}$$

那么 $f(x)=P_n(x)+R_n(x)$，即：

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)(x - x_0)^2}{2!} + \cdots + \frac{f^{(n)}(x_0)(x - x_0)^n}{n!} + R_n(x)$$

对于上式，容易证明 $R_n(x)=f(x)-P_n(x)$是比$(x-x_0)^n$ 高阶的无穷小。并且：

$$R_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}(x - x_0)^{n+1}$$，ξ 在 $x_0$ 与 x 之间

我们称 $R_n(x)$为拉格朗日余项。$P_n(x)$与 $f(x)$的误差为$|R_n(x)|$。

对于泰勒公式，如果令 $x_0=0$，那么就可以得到麦克劳林公式：

$$f(x)=f(0)+f'(0)x+\frac{f''(0)}{2!}x^2+\ldots+\frac{f^{(n)}(0)}{n!}x^n+R_n(x)$$

其中拉格朗日余项 $R_n(x)=\frac{f^{(n+1)}(\theta x)}{(n+1)!}x^{n+1}$，$(0<\theta<1)$。

对于一些常见的函数 $e^x$，sinx，cosx，ln(1+x)，$(1+x)^\alpha$ 和 1/(1-x)展开为麦克劳林公式的结果如下：

$$e^x=1+x+\frac{x^2}{2!}+\cdots+\frac{x^n}{n!}+0(x^n)$$

$$\sin x=x-\frac{x^3}{3!}+\frac{x^5}{5!}+\cdots+(-1)^{m-1}\frac{x^{2m-1}}{(2m-1)!}+0(x^{2m})$$

$$\cos x=1-\frac{x^2}{2!}+\frac{x^4}{4!}+\cdots+(-1)^m\frac{x^{2m}}{(2m)!}+0(x^{2m+1})$$

$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}+\cdots+(-1)^{n-1}\frac{x^n}{n}+0(x^n)$$

$$(1+x)^\alpha=1+\alpha x+\frac{\alpha(\alpha-1)}{2!}x^2+\cdots+\frac{\alpha(\alpha-1)\cdots(\alpha-n+1)}{n!}+0(x^n)$$

$$\frac{1}{1-x}=1+x+x^2+\cdots+x^n+0(x^n)$$

根据 $e^x$ 的麦克劳林展开公式有：

$$e^x\approx1+x+\frac{x^2}{2!}+\cdots+\frac{x^n}{n!}$$

令 x=1，则有无理数 e 的近似表达式：

$$e\approx1+1+\frac{1}{2!}+\ldots+\frac{1}{n!}$$

当 n=10 时，算得 e 的值约等于：2.718282，其误差=$|R_n(x)|=|e^\theta/(n+1)!|<e/(n+1)!<3/(n+1)!$，因此误差不超过 $10^{-6}$，这样的精度，已经足够满足一般的应用精度了。可见，e 虽然为无理数，但我们可以通过麦克劳林公式求出 e 的近似值。

**2.多元函数泰勒展开**

一元函数的泰勒公式比较简单，而多元函数的泰勒展开就稍微复杂一些。

首先来看二元函数的泰勒展开。

设 z=f(x, y)在点$(x_0, y_0)$的某一邻域内连续且有 n+1 阶连续偏导数，$(x_0+\Delta x，y_0+\Delta y)$为此邻域内任一点，则有：

$$f(x_0+\Delta x,y_0+\Delta y)=$$

$$f(x_0,y_0)+(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})f(x_0,y_0)+\frac{1}{2!}(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})^2 f(x_0,y_0)+\dots+$$

$$\frac{1}{n!}(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})^n f(x_0,y_0)+\frac{1}{(n+1)!}(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})^{n+1} f(x_0+\theta\Delta x,y_0+\theta\Delta y)$$

其中:

$(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})f(x_0,y_0)$ 表示 $\Delta x f_x(x_0,y_0)+\Delta y f_y(x_0,y_0)$;

$(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y})^2 f(x_0,y_0)$ 表示 $\Delta x^2 f_{xx}(x_0,y_0)+2\Delta x\Delta y f_{xy}(x_0,y_0)+\Delta y^2 f_{yy}(x_0,y_0)$。

一般地，记号：

$$\left(\Delta x\frac{\partial}{\partial x}+\Delta y\frac{\partial}{\partial y}\right)^m f(x_0,y_0)=\sum_{p=0}^{m}C_m^p\Delta x^p\Delta y^{m-p}\frac{\partial^m f}{\partial x^p\partial y^{m-p}}|_{x_0,y_0}$$

注意，这里的 m 表示的不是一般意义的 m 次方。

有了二元函数的泰勒展开之后，那么对于 n 元函数 $f(x_1,x_2,\dots,x_n)$ 的泰勒展开则与此类似，其展开形式如下：

$$f(x_1+\Delta x_1,x_2+\Delta x_2,\dots,x_n+\Delta x_n)=$$

$$\sum_{p=0}^{n}\frac{1}{p!}\left(\Delta x_1\frac{\partial}{\partial x_1}+\Delta x_2\frac{\partial}{\partial x_2}+\dots+\Delta x_n\frac{\partial}{\partial x_n}\right)^p f(x_1,x_2,\dots,x_n)+$$

$$\frac{1}{(n+1)!}\left(\Delta x_1\frac{\partial}{\partial x_1}+\Delta x_2\frac{\partial}{\partial x_2}+\dots+\Delta x_n\frac{\partial}{\partial x_n}\right)^{n+1} f(x_1+\theta\Delta x_1,x_2+\theta\Delta x_2,\dots,x$$

其中 $0<\theta<1$。

## 10.3.8 雅可比（Jacobian）矩阵

机器学习里经常需要用到对向量求导，其实就是把向量对应位置的元素(此时是一个函数）进行分别求导。现在我们首先来看一下向量函数的定义。

在之前我们学习的函数，包括一元函数和多元函数，都是 R→R（一元）或者 $R^n$→R（多元）的映射，也就是函数的值是一个一维的标量，我们可以称这种函数为实函数。而向量函数则是一个 n 维的向量空间 $R^n$ 到 m 维的向量空间 $R^m$ 的一个映射。

设 F：$R^n$→$R^m$ 是一个从 n 维欧氏空间映射到到 m 维欧氏空间的函数。这个函数由 m 个实函数组成：

$$y_1(x_1,\cdots,x_n),\cdots,y_m(x_1,\cdots,x_n)$$

即：

$$F:(x_1,x_2,x_3,\dots,x_n)\rightarrow(y_1(x_1,x_2,x_3,\dots,x_n),\dots,y_m(x_1,x_2,x_3,\dots,x_n))$$

函数的值是一个以每个实数函数值为分量的向量。向量函数中的每个函数的偏导数(如果存在)可以组成一个 m 行 n 列的矩阵，这个矩阵就是所谓的雅可比（Jacobian）矩阵：

$$\begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \cdots & \frac{\partial y_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial y_m}{\partial x_1} & \cdots & \frac{\partial y_m}{\partial x_n} \end{bmatrix}.$$

记为：

$$J_F(x_1,\ldots,x_n) \text{ 或者} \frac{\partial(y_1,\ldots,y_m)}{\partial(x_1,\ldots,x_n)}$$

比如，定义 $R^4$ 的 F 函数为：

$y_1=x_1$

$y_2=5x_3$

$y_3=4x_2^2-2x_3$

$y_4=x_3\sin x_1$

则雅可比矩阵 $J_F(x_1, x_2, x_3)=$ $\begin{bmatrix} \frac{\partial y_1}{\partial x_1} & \frac{\partial y_1}{\partial x_2} & \frac{\partial y_1}{\partial x_3} \\ \frac{\partial y_2}{\partial x_1} & \frac{\partial y_2}{\partial x_2} & \frac{\partial y_2}{\partial x_3} \\ \frac{\partial y_3}{\partial x_1} & \frac{\partial y_3}{\partial x_2} & \frac{\partial y_3}{\partial x_3} \\ \frac{\partial y_4}{\partial x_1} & \frac{\partial y_4}{\partial x_2} & \frac{\partial y_4}{\partial x_3} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 5 \\ 0 & 8x_2 & -2 \\ x_3\cos x_1 & 0 & \sin x_1 \end{bmatrix}$

单变量情况，如果 $f:R\to R$ ，当 $\Delta x\to 0$ 时，

$$f(x+\Delta x)\approx f(x)+f'(x)\Delta x$$

在向量函数中，一样成立，即：

$$F(x+\Delta x)\approx F(x)+F'(x)\Delta x$$

其中 $F'(x)$就是函数在一点 x 的导数，也就是雅可比矩阵 $J_F(x)$。雅可比矩阵的重要性在于它体现了一个多变数向量函数的最佳线性逼近，从而允许我们通过线性函数估计一个向量函数 F。

## 10.3.9 海森(Hessian)矩阵

海森矩阵是由多元函数的二阶偏导数构成的一个方阵。设有 n 元实函数 $f(x_1, x_2, x_3, \ldots, x_n)$，那么它的海森矩阵为：

$$A=\begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

由于二阶偏导函数$\frac{\partial^2 z}{\partial y \partial x} = \frac{\partial^2 z}{\partial x \partial y}$，所以海森矩阵是一个对称矩阵。

海森矩阵描述了函数的局部曲率。可以利用海森矩阵来判定多元函数的极值，因此它常用于牛顿法解决优化问题，在机器学习领域有着广泛的应用。在优化问题里，由于目标函数往往很复杂，常将目标函数在某点邻域展开成泰勒多项式来逼近原函数，此时函数在某点泰勒展开式的矩阵形式中会涉及到海森矩阵。

我们知道多元函数的梯度向量的各个分量为函数 f 的一阶偏导数，以二元函数为例：

$$\text{gradf（x，y）}=\nabla f(x,y)=\left\{\frac{\partial f}{\partial x},\frac{\partial f}{\partial y}\right\}=f_x(x,y)\bar{i}+f_y(x,y)\bar{j}$$

那么求出这一组一阶偏导函数的雅可比矩阵其实就是该函数 f 的海森矩阵。以上述二元函数为例，求 $f_x(x, y)$和 $f_y(x, y)$的雅可比矩阵为：

$$\begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} \end{pmatrix}$$

它就是二元函数 f(x, y)的海森矩阵。那么海森矩阵是如何求多元函数的极值的呢？

设 n 元实函数$f(x_1,x_2,\cdots,x_n)$在点$M_0(a_1,a_2,...,a_n)$的邻域内有二阶连续偏导，若有：

$$\left.\frac{\partial f}{\partial x_j}\right|_{(a_1,a_2,...,a_n)=0},j=1,2,...,n$$

并且：

$$A=\begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

则：

- 当 A 正定矩阵时，$f(x_1,x_2,\cdots,x_n)$在$M_0(a_1,a_2,...,a_n)$处是极小值；
- 当 A 负定矩阵时，$f(x_1,x_2,\cdots,x_n)$在$M_0(a_1,a_2,...,a_n)$处是极大值；
- 当 A 不定矩阵时，$M_0(a_1,a_2,...,a_n)$不是极值点。
- 当 A 为半正定矩阵或半负定矩阵时，$M_0(a_1,a_2,...,a_n)$是“可疑”极值点，尚需要利用其他方法来判定。

## 10.4 不定积分

积分可以分为不定积分和定积分。根据牛顿-莱布尼茨公式（10.5 节介绍），许多函数

的定积分的计算可以简便地通过求不定积分来进行。现在我们先来学习什么是不定积分。

### 10.4.1 原函数与导数

在学习积分之前，我们先来了解原函数的概念，以及原函数与导数之间的关系。

如果在区间 I 上，可导函数 F(x)的导函数为 f(x)，即对任一 x∈I，都有：

F'(x)=f(x)或 dF(x)=f(x)dx，

那么函数 F(x)就称为 f(x)（或 f(x)dx）在区间 I 上的**原函数**。比如因为(sinx)'=cosx，所以 sinx 是 cosx 的一个原函数。连续函数一定有原函数。

如果 F(x)是 f(x)的一个原函数，那么 F(x)+C 也是 f(x)的一个原函数（其中 C 为常数），而且 f(x)的所有原函数，都只差一个常数。也就是，设 F'(x)=f(x)，那么有：

[F(x)+C]'=f(x)

### 10.4.2 不定积分

在区间 I 上，函数 f(x)的带有任意常数项的原函数称为 f(x)或 f(x)dx 在区间 I 上的不定积分，记作：

$$\int f(x)\mathrm{d}x$$

其中$\int$称为积分号（该符号是莱布尼茨根据求和单词“Sum”的首字母 S 演变而来），f(x)称为被积函数，f(x)dx 为被积表达式，x 称为基本变量。如果 F(x)是 f(x)在区间 I 上的一个原函数，那么 F(x)+C 就是 f(x)的不定积分，即：

$$\int f(x)\mathrm{d}x = F(x) + C$$

不定积分要解决的问题是求一个可导函数，使它的导函数等于已知函数。

根据不定积分的定义，可以得出它有如下性质：

- $\frac{\mathrm{d}}{\mathrm{d}x}\left[\int f(x)\mathrm{d}x\right] = f(x)$
- $\int F'(x)\mathrm{d}x = F(x) + C$
- $\mathrm{d}\left[\int f(x)\mathrm{d}x\right] = f(x)\mathrm{d}x$

由此可见，微分运算（记号 d）与积分运算（记号$\int$）互为逆运算。这个规律实际上是由牛顿最先发现和提出的。当$\int$和 d 在一起的时候，或抵消，或抵消后差一个常数。

既然积分运算是微分运算的逆运算，那么就可以从导数公式得到相应的积分公式。比如因为(sinx)'=cosx，那么$\int$cosxdx=sinx+C。

基本积分表总结如下：

① $\int k\mathrm{d}x = kx + C$（$k$ 是常数），

② $\int x^{\mu}\mathrm{d}x = \frac{x^{\mu+1}}{\mu+1} + C$（$\mu \neq -1$），

③ $\int \frac{\mathrm{d}x}{x} = \ln|x| + C$，

④ $\int \frac{\mathrm{d}x}{1+x^2} = \arctan x + C$，

⑤ $\int \frac{\mathrm{d}x}{\sqrt{1-x^2}} = \arcsin x + C$，

⑥ $\int \cos x\mathrm{d}x = \sin x + C$，

⑦ $\int \sin x\mathrm{d}x = -\cos x + C$，

⑧ $\int \frac{\mathrm{d}x}{\cos^2 x} = \int \sec^2 x\mathrm{d}x = \tan x + C$，

⑨ $\int \frac{\mathrm{d}x}{\sin^2 x} = \int \csc^2 x\mathrm{d}x = -\cot x + C$，

⑩ $\int \sec x\tan x\mathrm{d}x = \sec x + C$，

⑪ $\int \csc x\cot x\mathrm{d}x = -\csc x + C$，

⑫ $\int \mathrm{e}^x\mathrm{d}x = \mathrm{e}^x + C$，

⑬ $\int a^x\mathrm{d}x = \frac{a^x}{\ln a} + C$.

$$\int [f(x)+g(x)]\mathrm{d}x = \int f(x)\mathrm{d}x + \int g(x)\mathrm{d}x.$$

$$\int kf(x)\mathrm{d}x = k\int f(x)\mathrm{d}x.$$

⑯ $\int \tan x\mathrm{d}x = -\ln|\cos x| + C$，

⑰ $\int \cot x\mathrm{d}x = \ln|\sin x| + C$，

⑱ $\int \sec x\mathrm{d}x = \ln|\sec x + \tan x| + C$，

⑲ $\int \csc x\mathrm{d}x = \ln|\csc x - \cot x| + C$，

⑳ $\int \frac{\mathrm{d}x}{a^2+x^2} = \frac{1}{a}\arctan\frac{x}{a} + C$，

㉑ $\int \frac{\mathrm{d}x}{x^2-a^2} = \frac{1}{2a}\ln\left|\frac{x-a}{x+a}\right| + C$，

㉒ $\int \frac{\mathrm{d}x}{\sqrt{a^2-x^2}} = \arcsin\frac{x}{a} + C$，

㉓ $\int \frac{\mathrm{d}x}{\sqrt{x^2+a^2}} = \ln(x+\sqrt{x^2+a^2}) + C$，

㉔ $\int \frac{\mathrm{d}x}{\sqrt{x^2-a^2}} = \ln|x+\sqrt{x^2-a^2}| + C$.

实际上，积分计算比导数计算更加的复杂，有的函数的积分运算甚至是十分困难的。除了一些基本的初等函数的积分公式，还有一些专门用于积分运算的方法，比如换元积分法和分部积分法，限于篇幅这里就不具体介绍了，有兴趣的读者请参考对应的专业书籍。

此外，为了积分方便，还把常用的积分公式或者积分类型做成积分表，方便直接从积分表中查到结果。

# 10.5 定积分

定积分是积分的一种，是函数 f(x)在区间[a, b]上积分和的极限。若定积分存在，则它是一个具体的数值，不定积分是一个函数表达式，它们在数学上有一个计算关系，即牛顿-莱布尼茨公式。

## 10.5.1 定积分概念与定义

**1.曲边梯形面积问题**

如图 10-32 所示，设 y=f(x)在区间[a, b]上非负、连续。由直线 x=a，x=b，y=0 及曲线 y=f(x)所围成的图形称为曲边梯形，其中 y=f(x)曲线部分称为曲边。

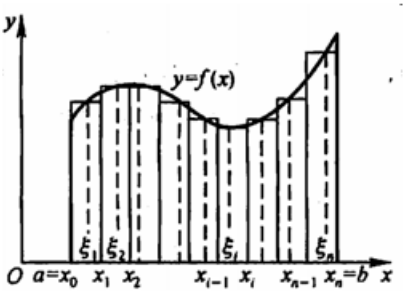



图 10-32 曲边梯形面积

现在的问题是，如何计算这个曲边梯形的面积呢？我们通过已知的数学知识知道，矩形的面积公式为：底×高，这是因为矩形的的高是固定不变的。但是现在的曲边梯形中，有一条边是曲线段，也就是曲边梯形中的高（即函数 f(x)的值）是变化的，因此曲边梯形面积的计算再也不适合于矩形的面积公式了。传统的初等数学理论，是无法解决曲边梯形的面积计算的。

但是，有了极限以及微积分的理论之后，这个问题得到了很好的解决。

由于曲边梯形的高 f(x)在区间[a, b]上是连续变化的，在很小一段区间上它的变化很小，我们可以近似的把它看做不变。因此，如果把曲边梯形的底即区间[a, b]插入 n 个点 $x_1$，$x_2$，$x_3$，…，$x_n$，那么有：

$$a = x_0 < x_1 < x_2 < \cdots < x_{n-1} < x_n = b$$

将其划分为很多小区间：

$$[x_0, x_1], [x_1, x_2], \cdots, [x_{n-1}, x_n]$$

这 n 个小区间的长度为：

$$\Delta x_1 = x_1 - x_0, \Delta x_2 = x_2 - x_1, \ldots, \Delta x_n = x_n - x_{n-1}$$

在每个小区间上用这个小区间内某一点处的高来近似代替同一个小区间上的窄曲边梯形的变高，那么每个窄曲边梯形就可以近似的看成窄矩形。于是就可以用所有这些窄矩形面积之和作为曲边梯形面积的近似值。

经过每一个分点作平行于 y 轴的直线段，把曲边梯形分成 n 个窄曲边梯形。在每个小区间$[x_{i-1}, x_i]$上任取一点$\xi_i$，以$[x_{i-1}, x_i]$为底，$f(\xi_i)$为高的窄矩形近似替代第 i 个窄曲边梯形(i=1,2,3,...,n)，然后将这样得到的 n 个窄矩形面积之和作为所求曲边梯形面积 A 的近似值，即：

$$A \approx f(\xi_1)\Delta x_1 + f(\xi_2)\Delta x_2 + \cdots + f(\xi_n)\Delta x_n = \sum_{i=1}^{n} f(\xi_i)\Delta x_i .$$

如果把区间[a,b]无限细分下去，也就是使每个小区间的长度都趋于零，求出此时所有窄矩形面积之和的极限，如果存在，那么这个极限就是这个曲边梯形的面积。

因此，要使每个小区间的长度都趋于零，记$\lambda=\max\{\Delta x_1, \Delta x_2, \Delta x_3, \ldots, \Delta x_n\}$，只需要$\lambda \to 0$，就能保证所有的区间长度都趋近于零。

于是曲边梯形的面积为：

$$A = \lim_{\lambda \to 0} \sum_{i=1}^{n} f(\xi_i)\Delta x_i$$

上面我们讨论的是 f(x) ≥0 的情况。如果在[a, b]区间上，f(x) ≤0，那么曲边梯形位于 x 轴下方，A 的值为负。如果函数值部分大于零部分小于零，如图 10-33 所示，即函数 f(x)的图形有部分在 x 轴上方，有部分在 x 轴下方，那么 A 应该为 x 轴上方图形部分面积减去 x 轴下方部分面积之差。

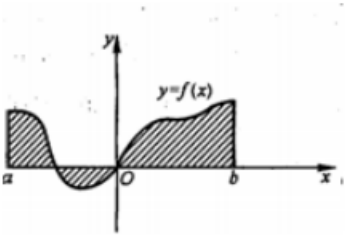



图 10-33 函数值有正有负时的曲边梯形

抛开曲边梯形问题的具体意义，事实上很多问题都可以抽象为同一个问题求解。为此我们抽象出这一类问题的定积分定义如下：

$$\int_a^b f(x)\mathrm{d}x = I = \lim_{\lambda \to 0} \sum_{i=1}^{n} f(\xi_i)\Delta x_i$$

设函数 f(x)在[a, b]上有界，在[a, b]中任意插入若干个分点：

$$a=x_0<x_1<x_2<x_3<\ldots<x_{n-1}<x_n=b,$$

把区间[a, b]分成 n 个小区间

$$[x_0, x_1], [x_1, x_2], [x_2, x_3], \ldots, [x_{n-1}, x_n]$$

各个小区间的长度依次为：

$$\Delta x_1=x_1-x_0, \Delta x_2=x_2-x_1, \Delta x_3=x_3-x2, \quad \ldots, \quad \Delta x_n=x_n-x_{n-1}$$

在每个小区间$[x_{i-1}, x_i]$上任取一点$\xi_i(x_{i-1} \leqslant \xi_i \leqslant x_i)$，作函数值$f(\xi_i)$与小区间长度$\Delta x_i$的乘积$f(\xi_i)\Delta x_i(i=1, 2, 3, \ldots, n)$，并求和：

$$S = \sum_{i=1}^{n} f(\xi_i)\Delta x_i$$

记 $\lambda=\max\{\Delta x_1, \Delta x_2, \Delta x_3, \cdots, \Delta x_n\}$，只要当 $\lambda \to 0$ 时，和 S 总趋于确定的极限 I，那么称这个极限 I 为函数 f(x)在区间[a,b]上的定积分（简称积分），记作：

$$\int_a^b f(x)\mathrm{d}x$$

即：

$$\int_a^b f(x)\mathrm{d}x = I = \lim_{\lambda \to 0} \sum_{i=1}^{n} f(\xi_i)\Delta x_i$$

其中 f(x)叫做被积函数，f(x)dx 叫做被积表达式，x 叫做积分变量，a 叫做积分下限，b 叫做积分上限，[a, b]叫做积分区间。

设 f(x)在区间[a, b]上连续，则 f(x)在[a, b]上可积。

设 f(x)在区间[a, b]上有界，且只有有限个间断点，则 f(x)在[a, b]上可积。

根据定积分的定义，我们很容易得到定积分的如下性质：

- 当 a=b 时，$\int_a^b f(x)\mathrm{d}x = 0$
- 当 a>b 时，$\int_a^b f(x)\mathrm{d}x = -\int_b^a f(x)\mathrm{d}x$
- $\int_a^b [f(x) \pm g(x)]\mathrm{d}x = \int_a^b f(x)\mathrm{d}x \pm \int_a^b g(x)\mathrm{d}x$
- $\int_a^b kf(x)\mathrm{d}x = k\int_a^b f(x)\mathrm{d}x$ （$k$ 是常数）
- $\int_a^b f(x)\mathrm{d}x = \int_a^c f(x)\mathrm{d}x + \int_c^b f(x)\mathrm{d}x$
- $\int_a^b 1\mathrm{d}x = \int_a^b \mathrm{d}x = b - a$
- 如果在区间[a, b]上有 f(x) $\geqslant$0，则 $\int_a^b f(x)\mathrm{d}x \geqslant 0 \quad (a<b)$
- 如果在区间[a, b]上有 f(x) $\leqslant$g(x)，则 $\int_a^b f(x)\mathrm{d}x \leqslant \int_a^b g(x)\mathrm{d}x \quad (a<b)$
- $\left|\int_a^b f(x)\mathrm{d}x\right| \leqslant \int_a^b |f(x)|\mathrm{d}x \quad (a<b)$
- 设 M 和 m 是函数 f(x)在区间[a, b]上的最大值和最小值，则有：

$$m(b-a) \leqslant \int_a^b f(x)\mathrm{d}x \leqslant M(b-a) \quad (a<b)$$

- 积分中值定理：如果函数 f(x)在积分区间[a, b]上连续，则在[a, b]上至少存在一个点 ξ，使下式成立：

$$\int_a^b f(x)\mathrm{d}x = f(\xi)(b-a) \quad (a \leqslant \xi \leqslant b)$$

- 若 f(x)在[-a, a]上连续且为偶函数，则：

$$\int_{-a}^{a} f(x)\mathrm{d}x = 2\int_0^a f(x)\mathrm{d}x$$

- 若 f(x)在[-a, a]上连续且为奇函数，则：

$$\int_{-a}^{a} f(x)\mathrm{d}x = 0$$

## 10.5.2 牛顿-莱布尼茨公式（微积分基本公式）

定积分的计算，可以通过定义以求极限的方式进行，但这样太麻烦。那么有没有其他的计算方法呢？有。这就是牛顿莱布尼茨公式，也被称为微积分基本公式。这个公式表述如下：

如果 F(x)是连续函数 f(x)在区间[a, b]上的一个原函数，则：

$$\int_{a}^{b} f(x)\mathrm{d}x = F(b) - F(a)$$

这个公式揭示了定积分与被积函数的原函数或不定积分之间的联系。这个公式告诉我们：一个连续函数在区间[a, b]上的定积分等于它的任一个原函数在区间[a, b]上的增量。这个公式给定积分提供了一个有效而简便的计算方法，大大简化了定积分的计算流程。

练习 24：计算定积分$\int_{0}^{1} x^2\mathrm{d}x$。

解：由牛顿莱布尼茨公式有：$\int_{0}^{1} x^2\mathrm{d}x=\left[\frac{x^3}{3}\right]_{0}^{1}$=$1^3/3-0^3/3=1/3$

练习 25：计算定积分$\int_{-2}^{-1} \frac{\mathrm{d}x}{x}$。

解：由牛顿莱布尼茨公式有：$\int_{-2}^{-1} \frac{\mathrm{d}x}{x}=\left[ln|x|\right]_{-2}^{-1}$=ln|-1|-ln|-2|=ln1/2=-ln2

练习 26：求 y=sinx 在[0, π]之间与 x 轴围成的图形面积，如图 10-33 所示。

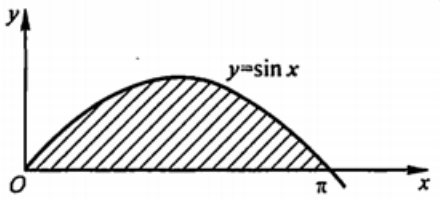



图 10-33 y=sinx 在[0, π]之间与 x 轴围成的图形

解：S=$\int_{0}^{\pi} \sin x\mathrm{d}x=\left[-cosx\right]_{0}^{\pi}$=-(-1)-(-1)=2

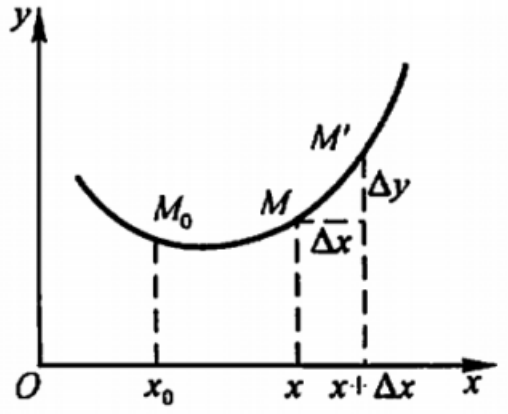
一个函数（不一定连续），可以存在不定积分，而不存在定积分；也可以存在定积分，而不存在不定积分。一个连续函数，一定存在定积分和不定积分；若只有有限个间断点，则定积分存在；若有跳跃间断点，则原函数一定不存在，即不定积分一定不存在。

牛顿-莱布尼茨公式是计算定积分的基本方法，除此以外，求定积分还有很多技巧和方法。比如分部积分法，换元法等。这里我们就不一一介绍了。有兴趣的读者，可以参考相应的数学书籍，进一步学习。

## 10.5.3 弧微分

设函数 f(x)在区间(a, b)内具有连续导数。在曲线 y=f(x)上取固定点 $M_0(x_0, y_0)$作为度量弧

长的基点，并规定依 x 增大的方向作为曲线的正向。对曲线上任一点 M(x, y)，规定有向弧段 $M_0M$ 的值 s 为：s 的绝对值等于这弧段的长度，当有向弧段 $M_0M$ 的方向与曲线的正向一致时，s>0，相反时 s<0。那么 s 显然与 x 存在函数关系，记为：s=s(x)。




设 x, x+Δx 为(a, b)内两个相邻的点，这两个点在曲线 y=f(x)上对应的点为 M, M'。设对应于 x 的增量Δx，弧 s 的增量为Δs，那么有：

$$\Delta s = \overset{\frown}{M_0M'} - \overset{\frown}{M_0M} = \overset{\frown}{MM'}$$

于是有：

$$\left(\frac{\Delta s}{\Delta x}\right)^2 = \left(\frac{\overset{\frown}{MM'}}{\Delta x}\right)^2 = \left(\frac{\overset{\frown}{MM'}}{|MM'|}\right)^2 \cdot \frac{|MM'|^2}{(\Delta x)^2}$$

$$= \left(\frac{\overset{\frown}{MM'}}{|MM'|}\right)^2 \cdot \frac{(\Delta x)^2 + (\Delta y)^2}{(\Delta x)^2}$$

$$= \left(\frac{\overset{\frown}{MM'}}{|MM'|}\right)^2 \left[1 + \left(\frac{\Delta y}{\Delta x}\right)^2\right]$$

所以有：

$$\frac{\Delta s}{\Delta x} = \pm\sqrt{\left(\frac{\overset{\frown}{MM'}}{|MM'|}\right)^2 \cdot \left[1 + \left(\frac{\Delta y}{\Delta x}\right)^2\right]}$$

令Δx->0 取极限，由于Δx->0 时，M'->M，这时弧的长度与弦的长度之比的极限等于 1，即：

$$\lim_{M' \to M} \frac{|\overset{\frown}{MM'}|}{|MM'|} = 1$$

又因为：

$$\lim_{\Delta x \to 0} \frac{\Delta y}{\Delta x} = y'$$

所以：

$$\frac{\mathrm{d}s}{\mathrm{d}x} = \pm\sqrt{1 + y'^2}$$

由于 s=s(x)随着 x 的增加而增大，所以显然是单调增加函数，从而根号前取正号，所以有：

$$\mathrm{d}s = \sqrt{1 + y'^2}\,\mathrm{d}x$$

此即为弧微分公式。

## 10.5.4 反常积分

在计算定积分的时候，可能会遇到积分区间为无穷区间或者被积函数为无界函数的积分的情况。我们把这种情况的积分称为反常积分。下面我们来分别讨论这两种情况的积分计算方法。

**1.无穷限的反常积分**

设函数 f(x)在区间[a, +∞)上连续，去 t>a，如果极限

$$\lim_{t \to +\infty} \int_a^t f(x)dx$$

存在，则称此极限为函数 f(x)在无穷区间[a, +∞)上的反常积分，记作：$\int_a^{+\infty} f(x)dx$，即：

$$\int_a^{+\infty} f(x)dx = \lim_{t \to +\infty} \int_a^t f(x)dx$$

设 F(x)是 f(x)在对应区间的原函数，根据牛顿莱布尼茨公式有：

$$\int_a^{+\infty} f(x)dx = [F(x)]_a^{+\infty} = \lim_{x \to +\infty} F(x) - F(a)$$

此时我们称反常积分收敛；如果上述极限不存在，则函数 f(x)在无穷区间[a, +∞)上的反常积分发散。

函数 f(x)在区间(-∞, a]和(-∞, +∞)上的反常积分与此类似，即：

在(-∞, a]上的反常积分定义为：

$$\int_{-\infty}^{b} f(x)dx = \lim_{t\to-\infty}\int_{t}^{b} f(x)dx = [F(x)]_{-\infty}^{b}$$

在(-∞, +∞)的反常积分定义为：

$$\int_{-\infty}^{+\infty} f(x)dx = \int_{-\infty}^{0} f(x)dx + \int_{0}^{+\infty} f(x)dx = \lim_{t\to-\infty}\int_{t}^{0} f(x)dx + \lim_{t\to+\infty}\int_{0}^{t} f(x)dx$$

若在(-∞, +∞)内有 F'(x)=f(x)，则当 F(-∞)与 F(+∞)都存在，则：

$$\int_{-\infty}^{+\infty} f(x)dx = [F(x)]_{+\infty}^{-\infty}$$

练习 27：计算反常积分 $\int_{+\infty}^{-\infty}\frac{dx}{1+x^2}$

解：$\int_{+\infty}^{-\infty}\frac{dx}{1+x^2} = [arctanx]_{-\infty}^{+\infty} = \lim_{x\to+\infty} arctanx - \lim_{x\to-\infty} arctanx = \pi$

此题结果表明位于函数 f(x)=1/(1+x²)下方和 x 轴上方的阴影虽然左右无限延伸，但面积有极限值 π，如图 10-35 所示。

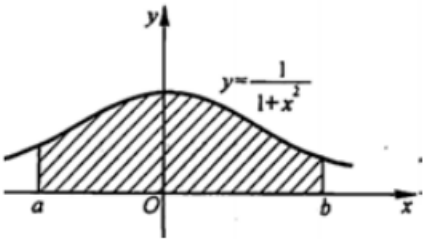



图 10-35 反常积分

练习 28：讨论反常积分 $\int_{a}^{+\infty}\frac{dx}{x^p}(a>0)$ 的收敛性。

解：当 p=1 时，有：

$\int_{a}^{+\infty}\frac{dx}{x^p} = \int_{a}^{+\infty}\frac{dx}{x} = [lnx]_{a}^{+\infty} = +\infty$，该反常积分发散。

当 p≠1 时有：

$$\int_{a}^{+\infty}\frac{dx}{x^p} = \left[\frac{x^{1-p}}{1-p}\right]_{a}^{+\infty} = \begin{cases} +\infty, p<1 \\ \frac{a^{1-p}}{p-1}, p>1 \end{cases}$$

即当 p<1 时，该反常积分发散；当 p>1 时，该反常积分收敛。

**2.无届函数的反常积分（瑕积分）**

如果函数 f(x)在点 a 的任一邻域内都无界，那么点 a 称为函数 f(x)的瑕点。无界函数的反常积分又叫做瑕积分。

设函数 f(x)在(a, b]上连续，点 a 为 f(x)的瑕点。取 t>a，如果极限：

$$\lim_{t\to a^+}\int_t^b f(x)dx$$

存在，则称此极限为函数 f(x)在(a, b]上的反常积分，记作$\int_a^b f(x)dx$，即：

$$\int_a^b f(x)dx=\lim_{t\to a^+}\int_t^b f(x)dx$$

此时称反常积分收敛；如果极限不存在，则称反常积分发散。

设 F'(x)=f(x)（即 F(x)是 f(x)的一个原函数），那么根据牛顿莱布尼茨公式有：

$$\int_a^b f(x)dx=F(b)-\lim_{x\to a^+}F(x)=F(b)-F(a^+)$$

设函数 f(x)在[a, b)上连续，点 b 为 f(x)的瑕点，则类似的瑕积分定义为：

$$\int_a^b f(x)dx=\lim_{t\to b^-}\int_a^t f(x)dx=F(b^-)-F(a)$$

设函数 f(x)在[a,b]上除点 c(a<c<b)外连续，点 c 为 f(x)的瑕点，则类似的瑕积分定义为：

$$\int_a^b f(x)dx=\int_a^c f(x)dx+\int_c^b f(x)dx=\lim_{t\to c^-}\int_a^t f(x)dx+\lim_{t\to c^+}\int_t^b f(x)dx$$

练习 29：计算反常积分$\int_0^a\frac{dx}{\sqrt{a^2-x^2}}(a>0)$。

解：因为在 x=a 处，函数无界，所以该点为函数的瑕点。于是该瑕积分计算如下：

$$\int_0^a\frac{dx}{\sqrt{a^2-x^2}}=\left[arcsin\frac{x}{a}\right]_0^a=\lim_{x\to a^-}arcsin\frac{x}{a}-0=\frac{\pi}{2}$$

练习 30：讨论反常积分$\int_{-1}^1\frac{dx}{x^2}$的收敛性。

解：因为当 x=0 时，函数无界，所以该点为函数的瑕点。由于：

$$\int_{-1}^0\frac{dx}{x^2}=\left[-\frac{1}{x}\right]_{-1}^0=\lim_{x\to 0^-}(-\frac{1}{x})-1=+\infty$$

所以该反常积分发散。如果忽略了 x=0 的瑕点，直接套用定积分公式，就可能得到错误的结果。

## 10.5.5 定积分应用

定积分在几何学和物理学中都有重要的应用。这里我们以几何学中的应用为例子，给大家介绍一下定积分解决实际问题的强大作用。

## 1.定积分元素法

如果某一实际问题中的所求量 U（比如曲边梯形的面积）符合如下条件：

- U 是一个与变量 x 的变化区间[a, b]有关的量；
- U 对于区间[a, b]具有**可分可加性**，即如果把区间[a, b]分成许多部分区间，则 U 相应的分成许多部分量 $\Delta U_i$ ，而 U 等于所有部分量之和；
- 部分量 $\Delta U_i$ 的近似值可表示为 $f(\xi_i)\ \Delta x_i$;那么可以考虑用定积分来表达这个量 U。

写出这个量 U 的积分表达式步骤：

首先，选取变量，确定区间。根据问题的具体情况，选取一个变量例如 x 为积分变量，并确定它的变化区间[a, b]；

然后，划分区间，在[x, x+dx] 确定 $\Delta U=\mathbf{f(x)dx}$。把区间[a, b]分成 n 个小区间，取其中任一小区间并记作[x, x+dx]，求出相应于这个小区间的部分量 ΔU 的近似值。如果 ΔU 能近似地表示为[a, b]上的一个连续函数在 x 处的值 f(x)与 dx 的乘积，就把 **f(x)dx 称为变量 U 的元素**且记作 dU，即：

$$dU=f(x)dx;$$

最后，以所求量 U 的元素 f(x)dx 为被积表达式，在区间[a, b]上作定积分，得：

$$U=\int_a^b f(x)dx$$

这就是所求量 U 的积分表达式。

## 2.应用举例

练习 31：计算由两条抛物线 $y^2=x$、$y=x^2$ 所围成的图形的面积，如图 10-36 所示。

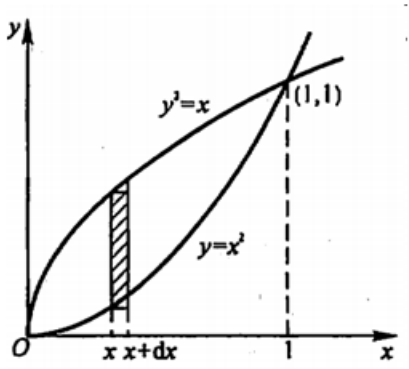



图 10-36　$y^2=x$、$y=x^2$ 所围成的图形

解：将 $y^2=x$、$y=x^2$ 联立方程组并求解，得出它们的交点为(0, 0)和(1, 1)两点。

以 x 为积分变量，变化区间为[0, 1]。在[0, 1]上的任一小区间[x, x+dx]的窄条面积近似

于高$\sqrt{x}$-x²，底为 dx 的窄矩形面积，从而面积元素为：

dS=($\sqrt{x}$-x²)dx

于是，以($\sqrt{x}$-x²)dx 为被积表达式，在闭区间[0, 1]上作定积分，所求面积为：

$$S=\int_0^1 (\sqrt{x}-x^2)\mathrm{d}x=\left[\frac{2}{3}x^{3/2}-\frac{x^3}{3}\right]_0^1=1/3$$

练习 32：求椭圆 $\frac{x^2}{a^2}+\frac{y^2}{b^2}=1$ 所围成的图形面积，如图 10-37 所示。

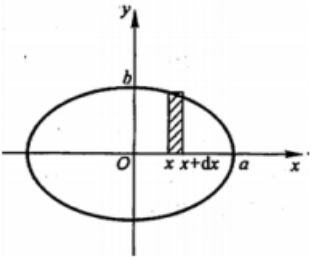



图 10-37 椭圆面积

解：由于椭圆关于 x 轴和 y 轴对称，所以设椭圆在第一象限面积为 A，那么整个椭圆的面积 S=4A。

在第一象限，根据元素法，选 x 为积分变量，积分区间为[0, a]，于是有：

$$A=\int_0^a y\mathrm{d}x,\quad S=4A=4\int_0^a y\mathrm{d}x$$

在这里，y 是 x 的因变量，如果直接通过椭圆方程，求出 y 的表达式，会比较复杂。因此考虑用椭圆的参数方程（换元法）来计算该定积分。

利用椭圆的参数方程：

$$\begin{cases} x=a\cos t, \\ y=b\sin t \end{cases} \quad \left(0\leqslant t\leqslant\frac{\pi}{2}\right)$$

即 x=acost, y=bsint, dx=-asintdt，当 x 从 0 变化到 a 时，t 从 π/2 变化到 0。则：

$$S=4\int_0^a y\mathrm{d}x=4\int_{\pi/2}^0 b\sin t(-a\sin t)\mathrm{d}t=4ab\int_0^{\pi/2}\sin^2 t\,\mathrm{d}t$$

$$=4ab[(t-\sin t\cos t)/2]_0^{\pi/2}=\pi ab。$$

如果 a=b，那么 S=πa²，即为圆的面积公式，其中 a 为圆的半径。

练习 33：计算由椭圆 $\frac{x^2}{a^2}+\frac{y^2}{b^2}=1$ 所围成的图形绕 x 轴旋转一周而成的旋转体（旋转椭球体）的体积，如图 10-38 所示。

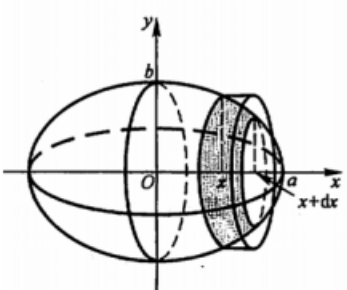


图 10-38 旋转椭球体体积

解：旋转椭球体可以看成是椭圆 $\frac{x^2}{a^2}+\frac{y^2}{b^2}=1$ 在 x 轴上半部分绕 x 轴旋转而成。设 x 为积分变量，积分区间为[-a, a]。取旋转椭球体在积分区间上的任一小区间[x, x+dx]的薄片体积，近似与一个以 $\frac{b}{a}\sqrt{a^2-x^2}$ 为底半径，dx 为高的扁圆柱体。根据圆柱体的体积公式（底面积乘以高）可得体积元素：

$$dV=\frac{\pi b^2}{a^2}(a^2-x^2)dx$$

于是该旋转椭球体的体积可表示为如下的定积分：

$$V=\int_{-a}^{a}\pi\frac{b^2}{a^2}(a^2-x^2)dx=\pi\frac{b^2}{a^2}\left[a^2x-\frac{x^3}{3}\right]_{-a}^{a}=4/3\,\pi\,ab^2$$

如果 a=b，旋转椭球体即为球体，于是可得球体的体积公式为 $4/3\,\pi\,a^3$，其中 a 为球体的半径。

## 3.球体体积计算中的祖暅原理

在微积分尚未发展起来时，古人对计算球体积的问题经历了长期的摸索。

古希腊阿基米德利用“穷竭法”（即极限的思想）算出了球面积、球体积，抛物线、椭圆面积，后世的数学家依据阿基米德的数学思想加以发展，形成近代的“微积分”。

我国古代对球的面积和体积计算，则经过了更加漫长的时间探索，直至最终求出。魏晋时期著名数学家刘徽（约 225 年—约 295 年）研究球体积时创造了一个新的立体图形，即“牟合方盖”，如图 10-39 所示。

牟合方盖是指在一个立方体内作两个互相垂直的内切圆柱，这两个圆柱相交部分即为牟合方盖，它恰好把立方体的内切球包含在内并且同它相切。

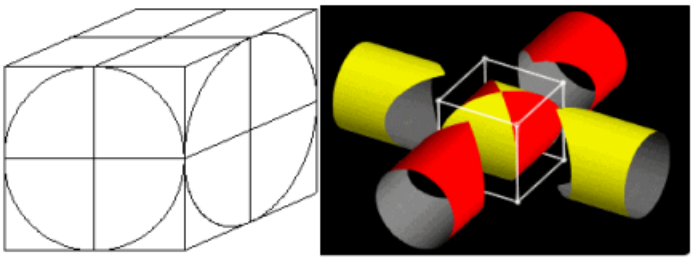


图 10-39 牟合方盖

如果用同一水平面去截他们，如图 10-40 所示，就得到一个圆（球的截面）和圆的外切正方形（牟合方盖的截面），二者面积之比为 π：4。因此球的体积和牟合方盖的体积之比也应该为 π：4，因此只要求出了牟合方盖的体积，那么球的体积也就求出了。但刘徽没有求出牟合方盖的体积。

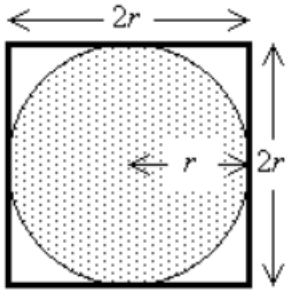



图 10-40 圆与外切正方形

南北朝时期，由祖冲之的儿子祖暅（456 年—536 年）最终求出了球的体积公式。祖冲之父子总结了刘徽的有关工作，提出“幂势既同则积不容异”，即等高的两立体，若其任意高处的水平截面积相等，则这两立体体积相等，这就是著名的祖暅原理（或刘祖原理）。祖暅应用这个原理，求出了牟合方盖的体积，从而利用刘徽的“球的体积和牟合方盖的体积之比 π：4”的结论，得到了正确的球体积公式。

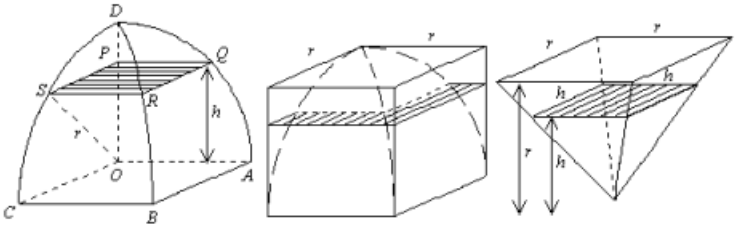



图 10-41 牟合方盖整体的八分之一

如图 10-41 所示，祖暅取牟合方盖整体的八分之一，并研究它与它的外切正方体之间所围成的体积。很显然，立方体的体积等于牟合方盖体积与牟合方盖与它的外切正方体之间所围成的体积之和。立方体体积已知，如果求出了牟合方盖与它的外切正方体之间所围成的体积，那么牟合方盖的体积就可以求出了。

如果用水平面去截取他们，可知截面正方形的面积等于左图牟合方盖的截面面积加上中图阴影面积之和。而容易证明中图阴影面积又等于倒立方锥的相同高度截面的面积。因此，根据祖暅原理，牟合方盖与它的外切正方体之间所围成的体积应该等于倒立方锥的体积，而倒立方锥的体积为 $1/3r^3$，于是：

$$V_{牟合方盖}/8=（V_{正方体}-V_{倒方锥}）=(r^3-1/3r^3)=2/3r^3$$

$$V_{牟合方盖}=8*2/3r^3=16/3r^3$$

因为：

$$V_{圆}：V_{牟合方盖}=π：4$$

所以：

$$V_{圆}=π/4\,V_{牟合方盖}$$

因此，圆的体积为：

$$V_{圆}=π/4*16/3r^3=4/3\,πr^3$$

祖暅原理在西方直到十七世纪才由意大利数学家卡瓦列利(Bonaventura Cavalieri)发现，比祖暅晚一千一百多年。

## 4.平面曲线弧长

设 A，B 是曲线弧的两个端点，如图 10-42 所示，在弧 $\overset{\frown}{AB}$ 上依次取分点 $A=M_0, M_1, M_2, \ldots,$

$M_{i-1}, M_i, \dots, M_{n-1}, M_n=B$，依次连接相邻的分点得一折线。当分点的数目无限增加且每个小段 $\overset{\frown}{M_{i-1}M_i}$ 都缩向一点时，如果此折线的长 $\sum_{i=1}^{n} |M_{i-1}M_i|$ 的极限存在，那么曲线弧 $\overset{\frown}{AB}$ 是可求长的，并称此极限为曲线弧 $\overset{\frown}{AB}$ 的弧长。

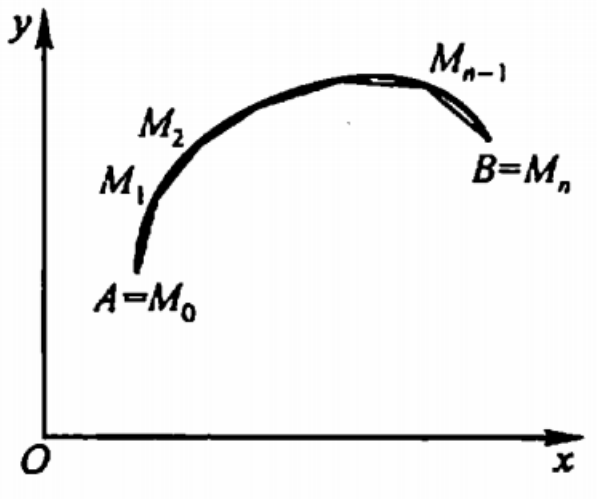



图 10-42 平面曲线求弧长

如果一条曲线弧是光滑的，也就是它上面每一个点都具有切线，且切线随切点的移动而连续转动，那么这条曲线弧是可求长的。

当曲线由直角坐标方程：

$$y=f(x) \quad (a\leqslant x\leqslant b)$$

给出，其中 f(x)在区间[a, b]上有一阶连续导数，由弧微分即弧长元素：

$$ds=\sqrt{1+y'^2}\,dx$$

那么有弧长为：

$$s=\int_a^b \sqrt{1+y'^2}\,dx$$

## 5.质能方程的推导

爱因斯坦的狭义相对论完成了两个重要的统一，即：时间和空间的统一；质量和能量的统一。首先是对时间和空间的统一，时间和空间不是孤立的和绝对的，而是相互关联和相互影响的，因此统称为时空。其次是质量和能量的统一，质能方程告诉我们，质量和能量在本质上是等价的。质量在某些情况下会转变为能量，如核裂变、核聚变等。反过来，能量在某些情况下也会转变为质量，例如，布莱特-惠勒正负电子实验（让两个光子通过撞击结合在一起，有可能变成物质，形成电子和正电子），宇宙大爆炸等。

质能方程即：

$$\mathbf{E=mc^2}$$

它是爱因斯坦根据他的狭义相对论推导出来的一个著名方程。方程中 E 表示能量，m

表示物体的质量，c 表示光速。质能方程，被誉为人类最美十大公式之一。质能方程如此神奇，但它却是可以通过物理学相关的定义和微积分进行直接推导出来的。下面，我们就给大家演示一下如何结合物理学定义和微积分来进行质能方程的推导过程。

首先，假设力F 作用在质量为m 的某物体上，沿着某一方向移动的距离为 ds，那么力 F 对物体做功导致物体动能的增长记为 $dE_k$，那么有：

$$dE_k = Fds \quad ①$$

根据物体动量和速度的定义有：

$$F = \frac{dp}{dt} \quad ②$$

$$v = \frac{ds}{dt} \quad ③$$

将②和③代入①有：

$$dE_k = Fds = \frac{dp}{dt}ds = dp\frac{ds}{dt} = vdp = vdmv = v^2dm + mvdv \quad ④$$

根据狭义相对论可知，如果物体在静止的时候质量为 $m_0$，那么当物体以速度 v 运动的时候，物体的质量 m 满足爱因斯坦质速关系如下：

$$m = \frac{m_0}{\sqrt{1 - v^2/c^2}} \quad ⑤$$

其中 c 为光速。质速关系告诉我们，当物体的速度接近光速的时候，物体的质量也将无限大。对⑤两边同时平方，并整理变形之后有：

$$m^2\left(c^2 - v^2\right) = m_0^2c^2 \quad ⑥$$

注意，在等式⑥里面，m，v 是变量（m 是因变量，v 是自变量），c 和 $m_0$ 是常量（也就是不变量）。对⑥两边同时对速度 v 求导，则有：

$$\frac{d\left(m^2\left(c^2 - v^2\right)\right)}{dv} = \frac{d(m_0c)^2}{dv} \quad ⑥$$

因为 $m_0$ 和 c 是常量，所以⑥中右边部分 $\frac{d(m_0c)^2}{dv}$=0，⑥中左边部分利用求导公式有：

$$m^2\frac{d\left(c^2 - v^2\right)}{dv} + \frac{dm^2}{dv}\left(c^2 - v^2\right) = 0 \quad ⑦$$

利用求导公式进一步展开有：

$$-2vm^2 + 2m\left(c^2 - v^2\right)\frac{dm}{dv} = 0 \quad ⑧$$

对⑧等式中约去一个 m，并整理之后有：

$$mvdv = \left(c^2 - v^2\right)dm \quad ⑨$$

将⑨代入④，从而得到：

$$dE_k = c^2 dm \quad ⑩$$

等式⑩在区间[$m_0$, m]定积分有：

$$E_k = \int_{m_0}^{m} c^2 dm = mc^2 - m_0c^2 \quad ⑪$$

而⑪就是狭义相对论中的动能公式。爱因斯坦分析动能公式之后指出，物体在静止时候的能量为 $E_0=m_0c^2$，在运动时候的能量为：$E=mc^2$。二者之差，就是物体获得的动能，即 $E_k=E-E_0$。因此他敏锐的得出了物体无论是在静止的时候还是在运动的时候，物体的能量和质量之间的质能关系，即：

$$\mathbf{E=mc^2}$$

而我们在中学物理中所学的动能公式 $E_k=1/2mv^2$ 只不过是动能公式在经典力学中的近似，这是因为对狭义相对论中的动能公式利用质速公式和泰勒级数展开有：

$$E_k = mc^2 - m_0c^2 = m_0c^2\left(\frac{1}{\sqrt{1 - v^2/c^2}} - 1\right) = m_0c^2\left(\frac{v^2}{2c^2} + \frac{3v^4}{8c^4} + \cdots\cdots\right) \approx \frac{1}{2}m_0v^2$$

# 10.6 微分方程

自然界里很多规律都是通过函数关系来描述的。但在一开始的时候，我们也许并不直接知道这个函数，也就是函数是未知的，只能根据具体问题和现象，找到函数及其导数的关系式，也就是所谓的微分方程。得到函数及其导数的微分方程之后，再通过对微分方程的研究，找出这个未知函数，这个过程就是解微分方程。

## 10.6.1 微分方程基本概念

一般地，我们把表示未知函数、未知函数的导数与自变量的关系的方程叫做微分方程。在微分方程中，未知函数的最高阶导数的阶数，叫做微分方程的阶。比如：

$$y^{(4)}-4y'''+7y''-4xy'+8y=2x$$

就是一个 4 阶微分方程。一般地，n 阶微分方程的形式为：

$$F(x, y, y', \ldots, y^{(n)})=0$$

其中 $y^{(n)}$ 在微分方程中是必须出现的，而 x, y, y', ..., $y^{(n-1)}$ 可以不出现。

在解决实际问题的时候，我们可以根据已知条件，建立对应的微分方程，然后找出满足微分方程的函数，即把函数代入微分方程能使微分方程称为恒等式，我们就把这个函数叫做该微分方程的解。比如，设函数 y=f(x) 在区间 I 上有 n 阶连续导数，并且：

$$F(x, f(x), f'(x), ..., f^{(n)}(x))\equiv 0$$

那么函数 y=f(x) 就是微分方程 $F(x, y, y', ..., y^{(n)})=0$ 在区间 I 上的解。

练习 34：某曲线上任一点 P(x, y) 处切线的斜率为 2x，并且通过点(0, 2)，求该曲线的方程。

解：设曲线的方程为 y=f(x)，根据斜率与导数的关系有：

$$y'=2x \quad ①$$

且当 x=0 时，y=2。

对①两端同时积分有：

$$y = \int 2x dx$$

即：

$$y=x^2+C \quad （其中 C 为任意常数） \quad ②$$

把 x=0， y=2 代入②：

$$2=0^2+C$$

所以 C=2，于是所求曲线方程为：

$$y=x^2+2$$

如果微分方程的解中含有任意常数，且任意常数的个数与微分方程的阶数相同，而且它们相互独立，这样的解叫做微分方程的通解。通解中由于包含了任意常数，所以通解尚不能完全确定反映对应的客观规律，所以必须确定这些常数的值。而确定了通解中任意常数的值之后，就得到了微分方程的特解。

那么如何确定通解中的常数值呢？这就需要根据问题的实际情况，提出确定常数的条件。一般来说，如果微分方程是一阶的，那么用来确定任意常数的条件是：

$x=x_0$ 时，$y=y_0$

或者写成：

$$y|_{x=x_0}=y_0$$

如果微分方程是二阶的，那么用来确定任意常数的条件是：

$x=x_0$ 时，$y=y_0, y'=y'_0$

或者写成：

$$y|_{x=x_0}=y_0, y'|_{x=x_0}=y'_0$$

其中 $x_0, y_0, y'_0$ 都是给定的值。我们把这些用来确定任意常数值的条件叫做微分方程的初值条件。

解微分方程实际上是一个非常复杂的过程，某些复杂的微分方程，至今数学家仍然没有找到具体的解。下面我们来介绍一些简单的微分方程形式，以及对应的微分方程的解法。

## 10.6.2 可分离变量微分方程

一般地，如果一阶微分方程能够写成如下形式：

$$g(y)dy=f(x)dx$$

在这种形式下，微分方程一端只含 y 的函数和 dy，另一端只含 x 的函数和 dx，那么原方程就被称为可分离变量的微分方程。这种可分离变量的微分方程是最容易求解的一类微分方程。

练习 35：求微分方程：y'=2xy。

解：y'=2xy，即 dy/dx=2xy，该微分方程为可分离变量微分方程，分离变量可得：

$$dy/y=2xdx$$

两端同时积分：

$$\int \frac{dy}{y} = \int 2xdx$$

于是有：

$$\ln|y|=x^2+C_1$$

于是：

$$y = \pm e^{x^2+C_1} = \pm e^{C_1} e^{x^2}$$

由于$\pm e^{C_1}$是任意非零常数，而且 y=0 也是该微分方程的解，所以该微分方程的通解为：

$$y = Ce^{x^2}$$，其中 C 为任意常数

### 10.6.3 齐次方程

如果一阶微分方程可化成如下形式：

$$dy/dx=f(y/x)$$

那么该方程被称为齐次方程。下面我们通过练习来演示齐次微分方程的求解方法。

练习 36：求解微分方程 $(x^2+y^2)dx-xydy=0$。

解：原方程可写成：

$$dy/dx=(x^2+y^2)/(xy)$$

等式右边式子分子分母同时除以 $x^2$，有：

$$dy/dx=(1+(y/x)^2)/(y/x)$$

因此该方程为齐次方程，令 u=y/x，那么 y=xu，因此：

$$dy/dx=d(xu)/dx=u+xdu/dx$$

那么原方程变为：

$$u+xdu/dx=(1+u^2)/u=1/u+u$$

从而有：

$$udu=dx/x$$

两端积分：

$$1/2u^2+C=\ln|x|$$

用 y/x 代替 u，有：

$$1/2(y/x)^2+C=\ln|x|$$

即：

$$y^2=x^2(2\ln|x|+C)$$

## 10.6.4 可降阶高阶微分方程

对于某些高阶微分方程，我们可以通过降阶的方法，通过代换将它化成较低阶的微分方程来求解。下面我们通过一些练习来给大家直接演示此类微分方程的求解方法。

练习 37：求微分方程 $y'''=e^{2x}+\sin x$ 的通解。

解：此微分方程为 $y^{(n)}=f(x)$的类型，右边只含有自变量 x，可以把 $y^{(n-1)}$看作新的未知函数，那么 $y^{(n)}=f(x)$就是一个有关新未知函数的一阶微分方程。两边积分，就得到一个 n-1 阶的微分方程。依此继续，连续 n 次积分，就可以得到含有 n 个任意常数的通解。

所以：

$y''=1/2e^{2x}-\cos x+C$

$y'=1/4e^{2x}-\sin x+Cx+C_2$

$y=1/8e^{2x}+\cos x+Cx^2+C_2x+C_3$

练习 38：求微分方程$(1+x^2)y''=2xy'$的通解。

解：原微分方程可以整理为：

$$y''=2xy'/(1+x^2)$$

此微分方程为 $y''=f(x, y')$的类型，即方程右端不包含未知函数 y。那么，设：

$$y'=p$$

因此有：

$$y''=d(y')/dx=dp/dx$$

代入原微分方程有：

$$dp/dx=2xp/(1+x^2)$$

整理并分离变量之后有：

$$dp/p=2x/(1+x^2)dx=d(1+x^2)/(1+x^2)$$

两边积分有：

$$\ln|p|=\ln(1+x^2)+C$$

从而有：

$$p=y'=C_1(1+x^2) \quad (C_1=\pm e^C)$$

所以：

$$y=C_1x+C_2x^3+C_3 \quad (C_2=C_1/3)$$

练习 39： 求微分方程 $yy''-y'^2=0$ 的通解。

解：此微分方程属于 $y''=f(y, y')$类型，也就是方程中不明显包含 x 自变量。令：

$$y'=p$$

于是：

$$y''=dp/dx=(dp/dy)*(dy/dx)=p*dp/dy$$

将 $y''=pdp/dy$ 代入原方程，有：

$$y(pdp/dy)-p^2=0$$

在 $y\neq0$，$p\neq0$ 时，方程约去 p，并分离变量有：

$$dp/p=dy/y$$

两边积分：

$$\ln|p|=\ln|y|+C$$

即：

$$p=C_1y \text{ 或 } y'=C_1y\ (C_1=\pm e^C)$$

从而有：

$$dy/dx=C_1y$$

分离变量：

$$dy/y=C_1dx$$

两边积分：

$$\ln|y|=C_1x+C'_2$$

所以：

$$y=C_2e^{C_1x}\ （C_2=\pm e^{C'_2}\ ）$$

## 10.6.5 常系数齐次线性微分方程

我们把形如：

$$y''+py'+qy=0\ (p,q\text{ 为常数})$$

这种微分方程称为二阶常系数齐次线性微分方程，其中二阶指的是未知函数 y 的导数最高阶为 2，常系数是指 y'', y',y 的系数都是常数，齐次是指方程右边为 0，线性是指未知函数 y 及其导数 y', y''都是 1 次的。

二阶常系数齐次线性微分方程的通解步骤如下：

首先，写出微分方程的特征方程：

$$r^2+pr+q=0$$

然后，求出特征方程的两个根：$r_1$，$r_2$。

最后，根据特征方程的两个根的不同情形，按照下列表格即可写出微分方程的通解：

| $r^2+pr+q=0$ 的两个根 $r_1$ 和 $r_2$ | $y''+py'+qy=0$ 的通解 |
|---|---|
| $r_1$ 和 $r_2$ 位实根，且 $r_1\neq r_2$ | $y=C_1e^{r_1x}+C_2e^{r_2x}$ |
| $r_1$ 和 $r_2$ 位实根，且 $r_1=r_2$ | $y=(C_1+C_2x)e^{r_1x}$ |
| $r_1$ 和 $r_2$ 位一堆共轭复根 $r_{1,2}=\alpha\pm\beta i$ | $y=e^{\alpha x}(C_1cos\beta x+C_2sin\beta x)$ |

练习 40：设有一个弹簧，上端固定，下端挂一个质量为 m 的物体。如图 10-43 所示，当物体处于静止状态时，作用在物体上的重力与弹簧弹性力大小相等，方向相反。这个位置就是物体的平衡位置。设该位置为坐标原点，取 x 轴铅直向下。如果使物体具有一个初始速度 $V_0\neq0$，那么物体将离开平衡位置，并在平衡位置附近做上下振动。在振动过程中，物体的位置 x 随时间 t 变化，变化规律为函数关系：x=x(t)。

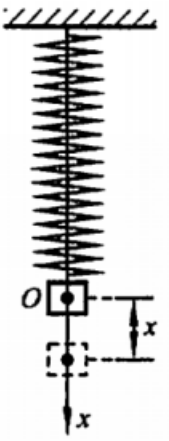



图 10-43 弹簧无阻尼自由振动

设物体只受弹性恢复力 f 的作用，且在初始时刻 t=0 的位置为 $x=x_0$，初始速度为 $dx/dt|_{t=0}=v_0$。那么，由力学知识，可以得到弹簧无阻尼自由振动的微分方程为：

$$\frac{d^2x}{dt^2}+k^2x=0$$

该微分方程的初值条件为：

$$x|_{t=0}=x_0,\ \frac{dx}{dt}|_{t=0}=v_0$$

不难发现，该方程满足二阶常系数齐次线性微分方程的形式，且它的特征方程为：

$$r^2+k^2=0$$

该特征方程的解为一堆共轭复根：

$$r=\pm ki$$

从而得到方程的通解为：

$$x=C_1\cos kt+C_2\sin kt$$

代入初值条件，可以解得：

$$C_1=x_0,\ C_2=v_0/k$$

因此，方程的特解为：

$$x=x_0\cos kt+v_0/k\sin kt$$

如果令：

$$x_0=A\sin\phi,\ v_0/k=A\cos\phi,\ 0\leqslant\phi<2\pi$$

那么根据三角函数公式，有：

$$x=x_0\cos kt+v_0/k\sin kt=A\sin\phi\cos kt+A\cos\phi\sin kt=A\sin(kt+\phi)$$

即：

$$x_0=A\sin(kt+\phi)$$

其中，因为 $x_0=A\sin\phi,\ v_0/k=A\cos\phi$，所以：

$$(A\sin\phi)^2+(A\cos\phi)^2=A^2(\sin\phi^2+\cos\phi^2)=x_0^2+(v_0/k)^2$$

即：

$$A^2=x_0^2+(v_0/k)^2$$

所以：

$$A=\sqrt{x_0^2+\frac{v_0^2}{k^2}}\ ,\ \tan\phi=kx_0/v_0$$

实际上，$x_0=A\sin(kt+\phi)$描述的是一个简谐振动，其中振幅为 A，初相为 $\phi$，周期为 $2\pi/k$，角频率为 k，其中 $k=\sqrt{\frac{c}{m}}$ (c 为弹簧的劲度系数，m 为物体的质量)，k 与初值无关，是简谐振动的固有频率。

## 10.6.6 微分方程应用举例

微分方程在天文学、几何学和物理学等领域都有着重要的应用。比如，我们可以使用微积分和微分方程来计算行星围绕恒星运行的轨迹方程，证明两点之间直线段距离最短，以及最速降线等问题。

## 行星运行轨迹

在太阳系，行星围绕着太阳进行公转运动。一开始，伽利略等科学家认为行星以太阳为中心以正圆为轨道进行着匀速公转。但是，开普勒进行天文观察之后发现，行星绕太阳运行的轨迹为椭圆，太阳正好位于椭圆上的一个焦点，并提出了著名的开普勒三大定律：

- 椭圆定律：所有行星绕太阳的轨道都是椭圆，太阳在椭圆的一个焦点上，如图 10-44 所示。
- 面积定律：行星和太阳的连线在相等的时间间隔内扫过的面积相等。
- 调和定律：所有行星绕太阳一周的周期（$T_i$）的平方与它们轨道半长轴（$a_i$）的立方成比例，即：$\frac{T_1^2}{T_2^2}=\frac{a_1^3}{a_2^3}$ 。

开普勒的发现与伽利略等科学家的传统观点发生了矛盾。而伽利略等由于无法解释其中的原因，从而无视了这些矛盾，和万有引力定律擦肩而过。当然，就连开普勒本人，也并不知道他所发现的这些定律里面蕴含着极其重大的天机即万有引力定律。

后来，牛顿研究了开普勒的观察结果和提出的定律，最终发现并提出了万有引力定律，即：

$$F_{引力}=GMm/r^2$$

万有引力定律指出物体与物体中间存在着引力，引力的大小与两个物体质量乘积成正比，与两个物体之间的距离平方成反比。牛顿利用万有引力定律并结合微积分计算，成功的解释了行星围绕太阳做椭圆轨迹运动的原因。

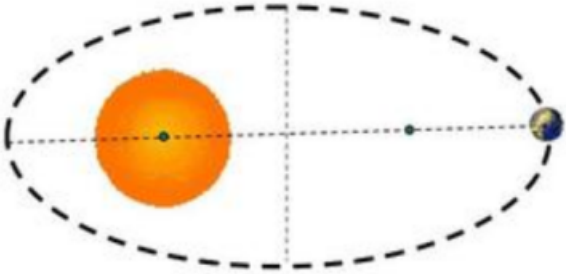


图 10-44 行星绕太阳运行椭圆轨迹

下面，我们就利用牛顿的万有引力定律和微积分理论，来给大家推导一下行星绕太阳运转的轨道，并证明此轨道为什么是一个椭圆。

首先我们研究一下质点在极坐标下的运动方程，因为使用极坐标系来研究行星绕太阳运动，会比较方便。如图 10-45 所示，我们在参考系上取一点 O 并称此点为极点，从 O 点引出一条有刻度的射线，称此射线为极轴，极点和极轴就构成了极坐标系。而质点在极坐标系中某点的位置 P，由极径 ρ 和幅角 θ 确定。其中极径 ρ 是极点到质点的距离，幅角 θ 是极径与极轴的夹角，规定幅角取逆时针方向为正。

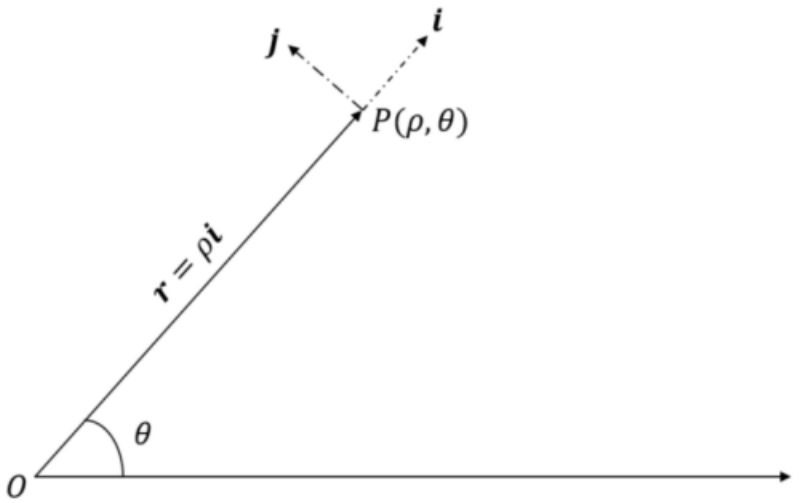



图 10-45 极坐标

由极点 O 画到动点 P 的有向线段称为动点 P 的矢径。当点 P 运动时，矢径 r 是随时间而变的变矢量，一般可表示为时间 t 的单值连续函数，它的方程称为点 P 的矢量形式的运动方程，即：r=r(t)。

我们可以将点 P 的矢径分解出两个正交的单位矢量，即：

- 径向单位矢量 i ：方向从极点指向质点。则矢径 r= ρ i，其中 ρ 表示矢径的大小，i 表示矢径的方向。
- 横向单位矢量 j ：方向与径向单位矢量垂直，且指向 θ 增加的方向。

现在我们来推导正交单位矢量对时间 t 的导数的计算。如图 10-46 所示，设初始时刻质点位置为 $P_1$，经过时间 dt，质点位置为 $P_2$，幅角变化量为 d θ 。

那么有：

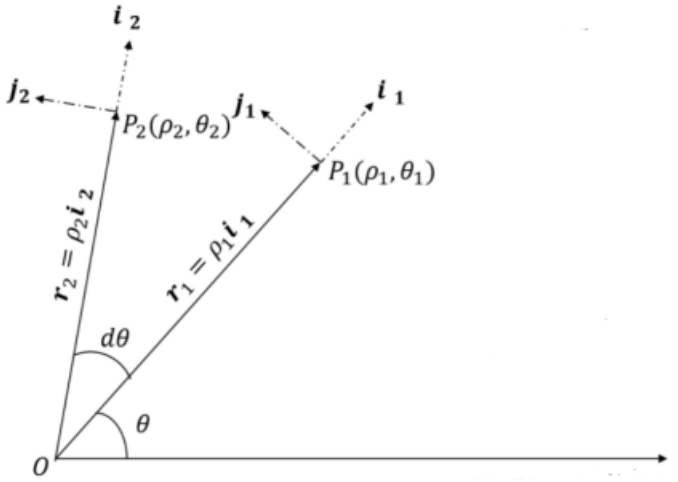



图 10-46 正交单位矢量对时间 t 的导数的计算

经过时间 dt，径向单位矢量由 $i_1$ 变化到 $i_2$，转过的角度为 d θ 。那么有：

$$d\boldsymbol{i} = \boldsymbol{i}_2 - \boldsymbol{i}_1$$

由于 i 为单位矢量，且 dt 为趋于 0 的微元。所以 di 的大小即为以 | i | 为半径，角度为 d θ 所对应的弧长，也就是单位矢量 i 从 $i_1$ 旋转到 $i_2$ 时，箭头末端所经过的路程，其大小为：

$$|d\boldsymbol{i}| = |\boldsymbol{i}| \cdot d\theta = d\theta \quad \text{（因为 i 为单位矢量，所以|i|=1)}$$

di 的方向为从 $i_1$ 的末端指向 $i_2$ 的末端，当 dt 趋于 0 时， di 的方向垂直于 i 并且指向 θ 的增加方向，所以 di 与 j 同向，因此 di 的大小和方向都确定了，那么有：

$$d\boldsymbol{i} = \boldsymbol{j}d\theta$$

因此求得 di/dt 为：

$$\frac{d\boldsymbol{i}}{dt} = \boldsymbol{j}\frac{d\theta}{dt}$$

现在我们再来研究一下 dj/dt 的求导。同理，经过时间 dt，横向单位矢量由 j1 变化到 j2 ，转过的角度为 d θ ，由于有：

$$d\boldsymbol{j} = \boldsymbol{j}_2 - \boldsymbol{j}_1$$

与 di 类似，对于 dj 有如下的关系：

$$|d\boldsymbol{j}| = |\boldsymbol{j}| \cdot d\theta = d\theta$$

当 dt 趋于 0 时， dj 的方向与径向单位矢量 i 反向，因此求得 dj/dt 为：

$$\frac{d\boldsymbol{j}}{dt} = -\boldsymbol{i}\frac{d\theta}{dt}$$

现在我们来研究质点在极坐标系中运动的速度 v 和加速度 a 的推导。因为：

$$\boldsymbol{r} = \rho\boldsymbol{i}$$

所以，根据物理学中速度 v 的定义有：

$$\boldsymbol{v}=\frac{d\boldsymbol{r}}{dt}=\frac{d(\rho\boldsymbol{i})}{dt}=\frac{d\rho}{dt}\boldsymbol{i}+\rho\frac{d\boldsymbol{i}}{dt}$$

又因为：

$$\frac{d\boldsymbol{i}}{dt}=\boldsymbol{j}\frac{d\theta}{dt}$$

所以有：

$$\boldsymbol{v}=\frac{d\rho}{dt}\boldsymbol{i}+\rho\frac{d\theta}{dt}\boldsymbol{j}$$

其中$\boldsymbol{\omega}=\frac{d\theta}{dt}$为角速度。又根据物理学中有关加速度的定义有：

$$\boldsymbol{a}=\frac{d\boldsymbol{v}}{dt}=\frac{d}{dt}(\frac{d\rho}{dt}\boldsymbol{i}+\rho\frac{d\theta}{dt}\boldsymbol{j})$$

对括号内的$\frac{d\rho}{dt}\boldsymbol{i}$根据求导公式，对时间 t 求导有：

$$\frac{d}{dt}(\frac{d\rho}{dt}\boldsymbol{i})=\frac{d^2\rho}{dt^2}\boldsymbol{i}+\frac{d\rho}{dt}\frac{d\boldsymbol{i}}{dt}=\frac{d^2\rho}{dt^2}\boldsymbol{i}+\frac{d\rho}{dt}\frac{d\theta}{dt}\boldsymbol{j}$$

对括号内的$\rho\frac{d\theta}{dt}\boldsymbol{j}$根据求导公式，对时间 t 求导有：

$$\frac{d}{dt}(\rho\frac{d\theta}{dt}\boldsymbol{j})=\frac{d\rho}{dt}\frac{d\theta}{dt}\boldsymbol{j}+\rho\frac{d^2\theta}{dt^2}\boldsymbol{j}+\rho\frac{d\theta}{dt}\frac{d\boldsymbol{j}}{dt}=\frac{d\rho}{dt}\frac{d\theta}{dt}\boldsymbol{j}+\rho\frac{d^2\theta}{dt^2}\boldsymbol{j}-\rho(\frac{d\theta}{dt})^2\boldsymbol{i}$$

整理之后，得到质点在极坐标系下的加速度公式为：

$$\boldsymbol{a}=[\frac{d^2\rho}{dt^2}-\rho(\frac{d\theta}{dt})^2]\boldsymbol{i}+[\rho\frac{d^2\theta}{dt^2}+2\frac{d\rho}{dt}\frac{d\theta}{dt}]\boldsymbol{j}$$

有了质点在极坐标系下的加速度公式之后，那么行星绕太阳运行过程中，可以把行星当做一个质点，若令$\ddot{\boldsymbol{r}}=\frac{d^2\rho}{dt^2}$，$\dot{\boldsymbol{r}}=\frac{d\rho}{dt}$，$\ddot{\boldsymbol{\theta}}=\frac{d^2\theta}{dt^2}$，$\dot{\boldsymbol{\theta}}=\frac{d\theta}{dt}$，行星的质量为 m，行星绕太阳运动。一般而言，若运动质点所受的力作用线始终通过某一个定点，则该质点所受的力是有心力。根据极坐标质点加速度公式和牛顿第二运动定律，质点在单位矢量 i 方向的 F(r) 为有心力，单位矢量 j 的方向作用力为 0，因此在平面极坐标系中，质点的运动微分方程为：

$$\begin{cases} m(\ddot{r} - r\dot{\theta}^2) = F_r = F(r) & ① \\ m(r\ddot{\theta} + 2\dot{r}\dot{\theta}) = F_\theta = 0 & ② \end{cases}$$

对于②进行积分整理，实际上等价于：

$$m\frac{1}{\mathrm{r}}\frac{d}{d_t}(r^2\dot{\theta}) = 0$$

由于$r^2\dot{\theta}$关于时间的导数为0，所以$r^2\dot{\theta}$应该为一个常数，即：

$$r^2\dot{\theta} = \mathrm{h} \quad (\mathrm{h}\text{为常数})$$

然后用$\mathrm{u} = \frac{1}{r}$代替r可得到：

$$\dot{\theta} = hu^2$$

又因为：

$$\dot{\mathrm{r}} = \frac{dr}{dt} = \frac{dr}{d\theta}\frac{d\theta}{dt} = \frac{d}{d\theta}(\frac{1}{u})\frac{d\theta}{dt} = -\frac{1}{u^2}\frac{du}{d\theta}\dot{\theta} = -h\frac{du}{d\theta}$$

$$\ddot{\mathrm{r}} = \frac{d\dot{r}}{dt} = \frac{d}{dt}(-h\frac{du}{d\theta}) = \frac{d}{d\theta}(-\frac{du}{d\theta})\dot{\theta} = -h^2u^2\frac{d^2u}{d\theta^2}$$

把$\ddot{r}$及$\dot{\theta}$的表达式代入①中就得到了如下方程：

$$\mathrm{h}^2u^2(\frac{d^2u}{d\theta^2} + u) = -\frac{F}{m}$$

这就是行星的轨道微分方程，又称比耐公式，当有心力F为负号时表示引力，为正号时为斥力。

现在来考虑太阳系的情况。令太阳的质量为M，行星的质量为m，行星与太阳之间的距离为r，则由万有引力定律可知行星和太阳之间的作用力可以写为：

$$F = -\frac{GMm}{r^2} = -\frac{k^2m}{r^2} = -mk^2u^2$$

将万有引力作为有心力，代入上面的比耐公式，有：

$$\mathrm{h}^2u^2(\frac{d^2u}{d\theta^2} + u) = k^2u^2$$

化简之后的微分方程为：

$$\frac{d^2u}{d\theta^2}+u=\frac{k^2}{h^2}$$

若令：

$$u=\xi+\frac{k^2}{h^2}$$

则上述微分方程变为：

$$\frac{\mathrm{d}^2\xi}{d\theta^2}+\xi=0$$

这个微分方程的形式与无阻尼自由振动的微分方程完全一样，所以它的解是：

$$\xi=A\cos(\theta-\theta_0)$$

而 u 为：

$$u=\xi+\frac{k^2}{h^2}=A\cos(\theta-\theta_0)+\frac{k^2}{h^2}$$

从而得到 r 为：

$$r=\frac{1}{u}=\frac{h^2/k^2}{1+A[\cos(\theta-\theta_0)]h^2/k^2}$$

式中 A 及 $\theta_0$ 是两个积分常数，如果把极轴转动一个角度，可使 $\theta_0=0$，则就简化为

$$r=\frac{h^2/k^2}{1+A\frac{h^2}{k^2}\cos\theta}$$

这就是所要求的行星运动轨道方程。把它与标准的圆锥曲线方程：

$$r=\frac{ed}{1+e\cos\theta}=\frac{p}{1+e\cos\theta}$$

进行比较，可知在平方反比引力作用下的质点的运动轨道是一条以力心为焦点的圆锥曲线。其离心率 $e=A\frac{h^2}{k^2}$，半正焦弦：$p=h^2/k^2$。因此，也就知道在万有引力作用之下的

质点的运动轨道是圆锥曲线。圆锥曲线按 e 值可分为三种类型，因此在力与距离平方成反比的引力场中质点的轨道类型有三种：

- e=$A\frac{h^2}{k^2}$<1，质点运动的轨迹为椭圆
- e=$A\frac{h^2}{k^2}$=1，质点运动的轨迹为抛物线
- e=$A\frac{h^2}{k^2}$>1，质点运动的轨迹为双曲线

这说明，行星之所以围绕太阳做椭圆轨迹运动，其离线率 e 恰好是小于 1 的。而彗星的运行轨道多为抛物线或双曲线，少数为椭圆。轨道为椭圆的彗星能定期回到太阳身边，称为周期彗星，比如哈雷彗星，它的周期为 76 年左右；轨道为抛物线或双曲线的彗星，终生只能接近太阳一次，一旦离去，就会永不复返，这种彗星被称为非周期彗星，这类彗星或许只是来自太阳系之外的过客，无意中闯进了太阳系，而后又回到宇宙深处。

## 两点之间直线段距离最短

稍微有一些数学基础的人，都知道这样一个结论，即两点之间，直线段距离最短。那么，这个结论是如何得出的呢？有没有什么证明的方法？

现在，我们结合微积分相关理论，来证明这个结论。

如图 10-47 所示，经过 A，B 两点之间有多条曲线，代表了不同的曲线方程，现在要证明的就是，其中的直线方程连接的两点得到的直线段距离是最短的。

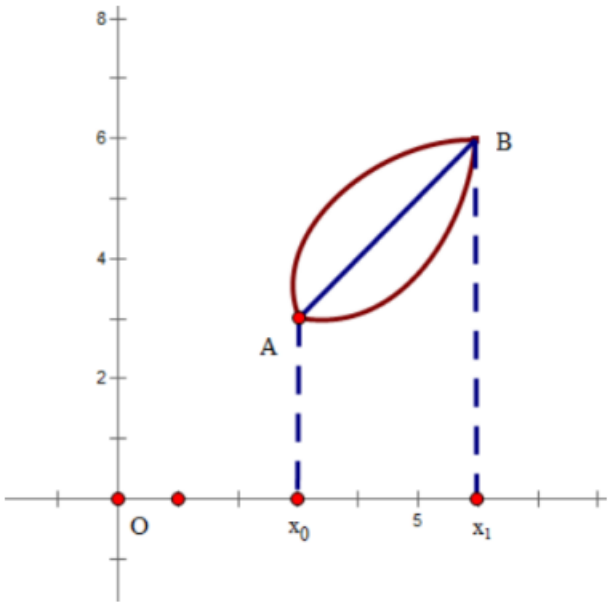



图 10-47 两点之间直线段最短

设经过 A，B 两点的任意曲线方程为 f(x)，且 A 点的横坐标为 $x_0$，B 点的横坐标为 $x_1$。那么，根据弧微分公式：

$$ds = \sqrt{1 + y'^2}$$

那么，f(x)曲线在 AB 之间的曲线段长度为：

$$S = \int_{x_0}^{x_1} \sqrt{1 + f'(x)^2}\, dx$$

现在的目标就是找到这样一个函数 f(x)，使得 S 的值最小。如果这个函数代表的是一个直线方程，那么结论就可以得证。在这里，S 是以函数为变量的函数，我们称之为泛函。而求泛函 S 的极值的方法，我们称之为变分法。1744 年欧拉发表了论文《寻求具有某种极大或极小性质的曲线的方法》，拉格朗日改进了推导并将此方法推广到多变量函数。该方法提出了用于求极值的欧拉-拉格朗日条件（E-L 条件）。泛函计算的一般形式为：

$$J[y(x)] = \int_{x_0}^{x_1} F(x, y(x), y'(x))\, dx$$

设 J[y(x)]是有关 y(x)的一个泛函，其中我们把被积函数 F(x, y(x), y'(x))称为拉格朗日函数。那么，J[y(x)]取得极值的必要条件为如下方程：

$$F_y - \frac{d}{dx} F_{y'} = 0 \text{ 或者} \frac{\partial F}{\partial y} - \frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{\partial F}{\partial y'}\right) = 0$$

式中 F 是 x , y , y ‘的已知函数并有二阶连续偏导数。上述必要条件中的方程叫做泛函的欧拉方程，也叫欧拉-拉格朗日方程。

现在，我们就可以基于欧拉方程，来证明两点之间直线段最短的结论。因为在此问题中拉格朗日函数为：

$$F = \sqrt{1 + f'(x)^2} \text{ 或者 } F = \sqrt{1 + y'^2}$$

求 F 对 y 的偏导 $F_y$：

$$F_y = 0$$

求 F 对 y’的偏导 $F_{y'}$：

$$F_{y'} = \frac{y'}{\sqrt{1+y'^2}}$$

$F_{y'}$对 x 求导$\frac{d}{dx}F_{y'}$：

$$\frac{d}{dx}F_{y'} = y''(1 + y'^2)^{-\frac{3}{2}}$$

因此，将$F_y$、$F_{y'}$、$\frac{d}{dx}F_{y'}$代入上面的欧拉方程有：

$$-y''(1 + y'^2)^{-\frac{3}{2}} = 0$$

所以有：

$$y'' = 0$$

这是一个简单的二阶微分方程，解方程有：

$$y' = C$$

所以有：

$$y = C_1 x + C_2$$

最终结果告诉我们，这很显然是一个直线方程，因此两点之间直线段最短结论得证。

## 最速降线问题

最速降线问题是指，在重力的单独作用下一质点通过两定点的用时最短的路径问题。如图 10-48 所示，小球从 A 点到达 B 点，有多条路径可走，其中就包括一条直线段。我们已经知道，经过两点之间直线段距离最短，那么是否所花时间也是最短呢？因此，我们必须找出那条真正用时最短的曲线方程。显然，这也是一个变分法求泛函极值的问题。

最速降线问题最早由伽利略提出，但伽利略并没有最终解决此问题。瑞士科学家约翰.伯努利借用了光学中费马的光程最短原理，即光线在两点间的实际路径是使所需的传播时间为极值的路径，并给出了对应的微分方程，用物理原理和方法巧妙的得到了问题的答案。约翰.伯努利得到答案之后，向同时代的科学家提出挑战，这里面就有牛顿、莱布尼兹、洛比达、以及他的哥哥雅格布·伯努利。据说牛顿在某天听到约翰·伯努利的挑战问题之后，他从铸币厂下班回家只用了一个晚上，就得到了正确的答案。

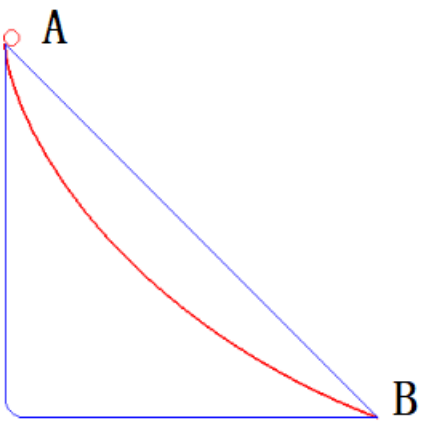



图 10-48 最速降线问题

现在，我们就用微积分以及变分法来研究下这个问题。首先，如图 10-49 所示，我们以 A 点为原点，建立对应的坐标系，其中 A 点的坐标为(0, 0)，B 点的坐标为(a, 0)。

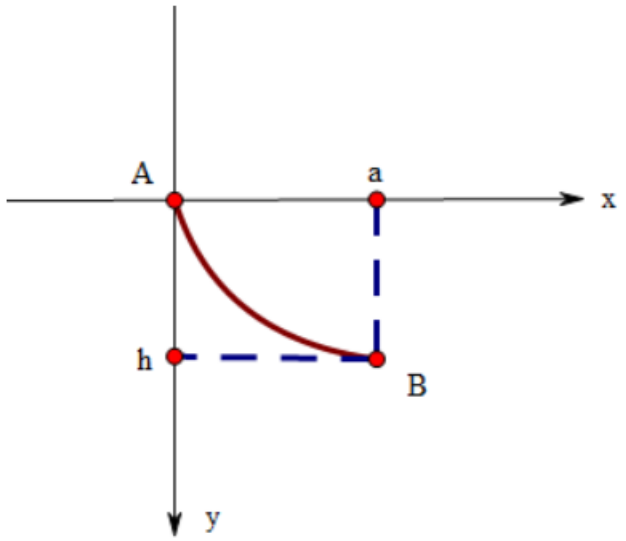



图 10-49 最速降线问题建立的坐标系

设质点到达 B 的运动轨迹对应的曲线为 y=f(x)，质量为 m，在运动过程中，下降的高度为纵坐标 y，速度为 v，那么根据能量守恒定律和动能定理有：

$$\frac{1}{2}mv^2 = mgy$$

因此质点此时的速度为：

$$v = \sqrt{2gy}$$

质点此时运动经过的曲线长度，由弧微分公式有：

$$\mathrm{d}s = \sqrt{1 + y'^{\,2}}\mathrm{d}x$$

而此时所花的时间为：

$$dt = \frac{\mathrm{d}s}{v} = \sqrt{\frac{1 + y'^{\,2}}{2gy}}\mathrm{d}x$$

那么质点从 A 点到达 B 点所花的时间为：

$$t = \int_0^a dt = \int_0^a \sqrt{\frac{1 + y'^{\,2}}{2gy}}\mathrm{d}x$$

于是，t 就是我们要求的时间，它是未知函数 y 的泛函。我们的目的就是要确定其中的未知函数 y（也就是从 A 点到达 B 点的函数曲线），使得 t 最小。这也是一个典型的二分法求解的问题，运用欧拉拉格朗日条件，其中朗格朗日函数为：

$$F(y, y') = \sqrt{\frac{1+y'^{\,2}}{2gy}} = (\frac{1 + y'^2}{2gy})^{\frac{1}{2}} \text{①}$$

对应的欧拉方程为：

$$F_y - \frac{d}{dx}F_{y'} = 0$$

也就是：

$$\frac{\partial F}{\partial y}-\frac{d}{dx}\left(\frac{\partial F}{\partial y'}\right)=0$$

同时，将$F-y'\frac{\partial F}{\partial y'}$对 x 求导，把上面的欧拉方程代入并化简有：

$$\frac{d}{dx}\left[F-y'\frac{\partial F}{\partial y'}\right]=y'\frac{\partial F}{\partial y}+y''\frac{\partial F}{\partial y'}-y''\frac{\partial F}{\partial y'}-y'\frac{d}{dx}\left(\frac{\partial F}{\partial y'}\right)=0$$

也就是$F-y'\frac{\partial F}{\partial y'}$对 x 求导导数为 0，那么有：

$$F-y'\frac{\partial F}{\partial y'}=C \quad （C 为任意常数）②$$

利用求导公式，求$\frac{\partial F}{\partial y'}$有：

$$\frac{\partial F}{\partial y'}=\frac{y'}{\sqrt{2gy}\sqrt{1+y'^{\,2}}} \quad ③$$

将①和③代入②有：

$$\sqrt{\frac{1+y'^{\,2}}{2gy}}-y'\frac{y'}{\sqrt{2gy}\sqrt{1+y'^{\,2}}}=C$$

两边平方并化简之后有：

$$y(1+y'^{\,2})=\frac{1}{2gC^2}$$

令 $2gC^2=2r$，那么解此微分方程，得到曲线的极坐标参数方程为：

$$\begin{cases}x=r(\theta-\sin\theta)\\y=r(1-cos\theta)\end{cases}$$

也就是物体沿该曲线路径从 A 点运动到 B 点所花的时间最短。实际上，这条曲线就是一条摆线。即如图 10-50 所示，摆线是一个圆沿一条直线运动时，圆边界上一定点所形成的轨迹。

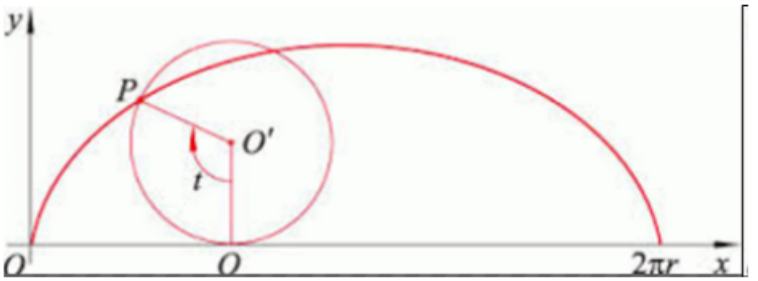



图 10-50 摆线

上面，我们利用微积分与微分方程，解决了行星绕太阳运行的椭圆轨迹问题，两点之间直线段距离最短问题，以及最速降线问题。实际上，微分方程在科学中的应用还有很多，包括薛定谔方程，麦克斯韦电磁方程，爱因斯坦广义相对论场方程，以及描述粘性不可压缩流体动量守恒的运动方程N-S 方程都和微分方程有关。而且很多方程都是二阶非线性偏微分方程，求解及其复杂，有的甚至至今无解，比如N-S 方程，因此在 2000 年的时候被美国克雷数学研究所悬赏百万美元求解。

下面我们看看这些微分方程的样子。

**薛定谔方程**

薛定谔方程（Schrödinger equation），又称薛定谔波动方程（Schrodinger wave equation），是由奥地利物理学家薛定谔提出的量子力学中的一个基本方程，如图 10-51 所示，它也是量子力学的一个基本假定。

它是将物质波的概念和波动方程相结合建立的二阶偏微分方程，可描述微观粒子的运动，每个微观系统都有一个相应的薛定谔方程式，通过解方程可得到波函数的具体形式以及对应的能量，从而了解微观系统的性质。薛定谔方程表明量子力学中，粒子以概率的方式出现，具有不确定性，宏观尺度下失效可忽略不计

$$i\hbar\frac{\partial\psi}{\partial t}=-\frac{\hbar^2}{2m}\frac{\partial^2\psi}{\partial x^2}$$

图 10-51 薛定谔方程

**麦克斯韦方程组**

麦克斯韦方程组，是英国物理学家詹姆斯·克拉克·麦克斯韦（James Clerk Maxwell，1831-1879）在 19 世纪建立的一组描述电场、磁场与电荷密度、电流密度之间关系的偏微分方程。它由四个方程组成：描述电荷如何产生电场的高斯定律、论述磁单极子不存在的高斯磁定律、描述电流和时变电场怎样产生磁场的麦克斯韦-安培定律、描述时变磁场如何产生电场的法拉第感应定律。

从麦克斯韦方程组，可以推论出电磁波在真空中以光速传播，并进而做出光是电磁波的猜想。麦克斯韦方程组和洛伦兹力方程是经典电磁学的基础方程。从这些基础方程的相关理论，发展出现代的电力科技与电子科技。

$$\nabla \bullet E = \frac{\rho}{\varepsilon_0}$$

$$\nabla \bullet B = 0$$

$$\nabla \times E = -\frac{\partial B}{\partial t}$$

$$\nabla \times B = \mu_0 J + \mu_0 \varepsilon_0 \frac{\partial E}{\partial t}$$

图 10-52 麦克斯韦方程组

**N-S 方程**

纳维-斯托克斯方程（Navier-Stokes equations），描述粘性不可压缩流体动量守恒的运动方程，非线性偏微分方程，七个“千年大奖问题”之一，如图 10-53 所示。粘性流体的运动方程首先由 Navier 在 1827 年提出，只考虑了不可压缩流体的流动。Poisson 在 1831 年提出可压缩流体的运动方程。Saint-Venant 在 1845 年，Stokes 在 1845 年独立提出粘性系数为一常数的形式，都称为 Navier-Stokes 方程，简称 N-S 方程。

$$\rho \frac{dV}{dt} = \rho g - \nabla p + \mu \nabla^2 V$$

图 10-53 N-S 方程

**爱因斯坦广义相对论引力场方程**

爱因斯坦引力场方程是广义相对论用以定量描述引力、时空和物质的统一性的方程，如图 10-54 所示。在宇宙学研究中具有重要作用。

$$R_{\mu\nu} - \frac{1}{2} R g_{\mu\nu} + \Lambda g_{\mu\nu} = \frac{8\pi G}{c^4} T_{\mu\nu}$$

图 10-54 爱因斯坦引力场方程

1915 年，爱因斯坦引力场方程发表了出来，整个广义相对论的动力学终于完成。1915 年后，广义相对论的发展多集中在解开场方程式上。但因为场方程式是一个非线性偏微分方程，很难得出解来，只有少数的解被解出来。其中最著名的解就是史瓦西解，它是第一个得出的场方程的球对称引力场严格解，这也是史瓦西黑洞的基础。

史瓦西（Karl Schwarzschild，1873-1916）是德国天文学家和物理学家，当时世界正处于一次世界大战期间，他正在德军服役。正是在俄国战场前线，史瓦西得到了引力场方程的第一个精确解，并在 1915 年 12 月 22 日将结果寄给了爱因斯坦。爱因斯坦对史瓦西的结果极为赞赏，特别是之前爱因斯坦本人只得到了引力场方程的近似解，并以此对水星的近日点进动进行了解释。

## 10.7 计算机中应用

我们知道，高等数学是理工科基础中的基础，一切理工科领域复杂的问题和理论，都会用到。其中高等数学中的极限是微积分的基础，微积分是概率论的基础，概率论又是机器学习算法和人工智能的基础。

现在，我们就给大家介绍高等数学在计算机领域的几个具体的应用。高等数学在计算机领域，最典型的应用就是一类优化问题的求解，比如在机器学习里，我们往往需要求出目标函数的误差最小，损失最小等从而寻找最优的参数建立模型。

大多数机器学习和深度学习都涉及到优化问题，即求函数 f(x)的最小值或最大值问题。通常是求函数 f(x)的最小值问题，而最大值问题可以通过-f(x)转化为最小值问题。所求最大值或最小值的函数称为目标函数。在求目标函数的最小值问题时，我们又将目标函数称为代价函数（cost function），损失函数（loss function）或者误差函数（error function）。

损失函数（loss function）也就是为了评估模型拟合的好坏，通常用损失函数来度量拟合的程度。损失函数极小化，意味着拟合程度最好，对应的模型参数即为最优参数。在线性回归中，损失函数通常为样本输出和假设函数的差取平方。比如对于 m 个样本$(x_i,y_i)(i=1,2,\cdots m)$,采用线性回归，损失函数为：

$$J(\theta_0,\theta_1)=\sum_{i=1}^{m}(h_\theta(x_i)-y_i)^2$$

其中 $x_i$ 表示第 i 个样本特征，$y_i$ 表示第 i 个样本对应的输出，$h_\theta(x_i)$为假设函数。

机器学习的本质是建立优化模型，通过优化方法，不断迭代参数向量，找到使目标函数最优的参数向量。最终建立模型。

### 10.7.1 最小二乘法

在各种工程问题里，我们如果拥有一组实验数据，这些数据包含了两个变量之间的内在函数关系。我们需要通过一些方法（比如最小二乘法）来分析这些实验数据，得到这两个变量函数关系的近似表达式。

最小二乘法（又称最小平方法），它通过最小化误差的平方和（名字中的“二乘”指的就是平方）最小来寻找数据的最佳函数匹配。利用最小二乘法可以简便地求得未知的数据，并使得这些求得的数据与实际数据之间误差的平方和为最小。

以线性函数为例子。假如有一组实验数据，包含了 y 和 x 的内在函数关系，我们把这组实验数据以 x 为横坐标，y 为纵坐标，描述在直角坐标里得到这组数据对应的点。如果这些点的连线大致接近于一条直线，如图 10-55 所示，于是可以认为 y=f(x)是线性函数，并设函数关系为：

$$y=ax+b$$

其中 a 和 b 是未知常数。现在我们就是要通过最小二乘法来选择确定最合适的常数 a 和 b。

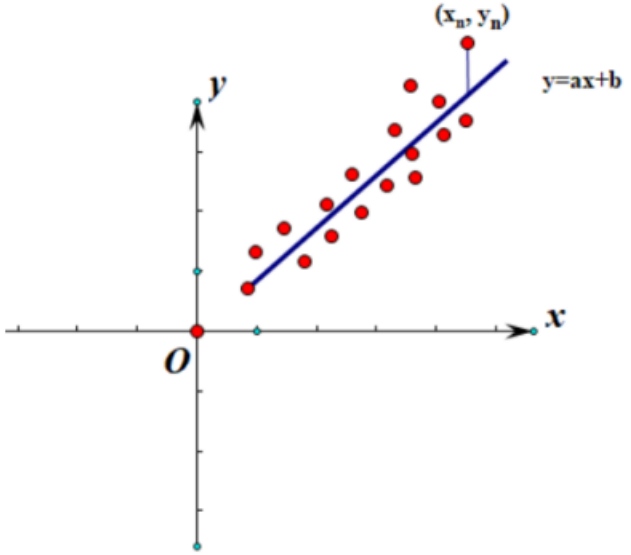



图 10-55 试验数据对应的点

最理想的情况就是选择的 a 和 b 使得直线 y=ax+b 能经过图 10-55 中所有标出的点，但由于这些点本来就不在同一条直线上，所以这实际上是不可能的。

因此，我们只能要求选择的 a 和 b 使得 f(x)=ax+b 在对应的 $x_0, x_1, x_2, ..., x_n$ 处的函数值 $f(x_i)$与实验数据 $y_0, y_1, y_2, ..., y_n$ 相差很小，也就是误差很小，即：

$$y_i-f(x_i)=y_i-(ax_i+b)(i=0, 1, 2, ..., n)$$

上面所有这些差值都很小。但是，在这里，由于误差 $y_i-f(x_i)$有正负，误差之和可能会被抵消。如果用误差的绝对值求和，使得其和最小，虽然可行但由于绝对值记号使得后续计算和分析不便，因此就用误差的平方来代表误差，也就是使:

$$\sum_{i=1}^{n}(y_i-(ax_i+b))^2$$

最小来保证每个偏差的绝对值都很小。

我们把这种根据偏差的平方和为最小的条件来选择常数 a，b 的方法叫做最小二乘法。

对于一组实验数据，$y_i$ 和 $x_i$ 都是已知数，要使上式取得最小值，实际上就是对一个关于 a 和 b 的二元函数：

$$M=M(a,b)=\sum_{i=1}^{n}(y_i-(ax_i+b))^2$$

求极值的问题。因此，根据多元函数求极值的方法，可以通过 M 的偏导数 $M_a(a,b)$和 $M_b(a,b)$联立方程求得，即：

$$\begin{cases}M_a(a,b)=0\\M_b(a,b)=0\end{cases}$$

即：

$$\begin{cases} \frac{\partial M}{\partial a} = -2\sum[y_i - (ax_i + b)]x_i = 0 \\ \frac{\partial M}{\partial b} = -2\sum[y_i - (ax_i + b)] = 0; \end{cases}$$

整理之后：

$$\begin{cases} \sum x_i[y_i - (ax_i + b)] = 0 \\ \sum[y_i - (ax_i + b)] = 0 \end{cases}$$

在这个方程中，$x_i$ 和 $y_i$ 都是试验数据，因此是已知数，a 和 b 是我们要求的未知数。因此代入数据，通过解方程，即可求得 a 和 b 的值。所求的 a 和 b 的值，就是使得误差的平方和最小的值。

通过最小二乘法，我们最后确定了 a 和 b 的取值，最终确定了两个变量 x 和 y 的线性关系：

$$y=ax+b$$

最小二乘法在工程中应用非常广泛，比如最小二乘法线性拟合(拟合就是把平面上一系列的点，用一条光滑的函数曲线连接起来。因为这条曲线有多种可能，从而有各种拟合方法)，最小二乘法多项式拟合，机器学习中线性回归的最小二乘法，系统辨识中的最小二乘辨识法，参数估计中的最小二乘法等。

根据中心极限定理，函数计算结果与试验结果的误差的分布是正态分布，因此最小二乘法可以由正态分布和极大似然估计推导出来，高斯做了相关的推导和证明工作，为此奠定了最小二乘法的重要地位。

法国科学家勒让德曾与高斯关于谁最早创立最小二乘法原理发生过争执。高斯使用的最小二乘法的方法发表于 1809 年他的著作《天体运动论》中。勒让德于 1806 年独立发明最小二乘法。

## 10.7.2 梯度下降算法

导数用于求函数的最小值非常有用。在前面的章节，我们详细介绍了如何利用一阶和二阶导数求函数极值问题。在求函数最小值的时候，我们可以将 x 沿着导数的反方向移动一小步来减小 f(x)。我们将这种方法称为梯度下降法。

对于一元函数来说，比如 $f(x)=1/2x^2$，它的导数为 f'(x)=x，在图 10-56 中，我们可以看到，当导数 f'(x)为正的时候，往 x 轴负半轴方向移动 x；当导数 f'(x)为负的时候，往 x 轴正半轴方向移动 x，这样就能快速达到函数的极小值点，这时 f'(x)=0。我们称 f'(x)=0 的点为函数的临界点或者驻点。驻点可能是极大值点，极小值点或者鞍点。

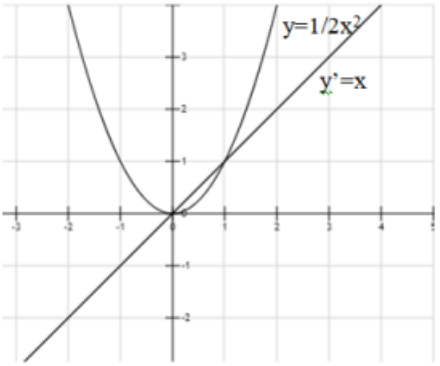



图 10-56 梯度下降法快速到极值

对于多元函数来说，即 $f:R^n \rightarrow R$，比如 $z=f(x,y)$，我们可以使用多元函数里的偏导数来获取函数的极大值或者极小值。在前面 9.2 一节里，我们已经指出，多元函数在梯度的反方向方向导数达到最小值，函数 $f(x, y)$在这个方向减少最快。我们把梯度所指方向称为上坡，梯度相反方向称为下坡。在梯度相反方向移动函数的自变量可以快速减小函数值，沿着这个方向搜索，最终达到极值点。我们把这种方法称为梯度下降法(gradient descent)或最速下降法。

在梯度下降法里，自变量 x 从一点移动到新点的距离可以设定为：

$$x_1=x_0-\varepsilon \nabla_x f$$

其中$\nabla_x f$ 为梯度在 x 轴的分量，$\varepsilon$ 称为学习率(learning rate)，是一个确定步长大小的一个正数，通常会选择一个小常数。

学习速率是指导我们该如何通过损失函数的梯度调整网络权重的超参数。通常所说的调参，就是调节此类超参数。如图 10-57 所示，学习率越低，损失函数的变化速度就越慢。虽然使用低学习率可以确保我们不会错过任何局部极小值，但也意味着我们将花费更长的时间来进行收敛，特别是在被困在函数图形中一个类似于平坦的高原地区的情况下。

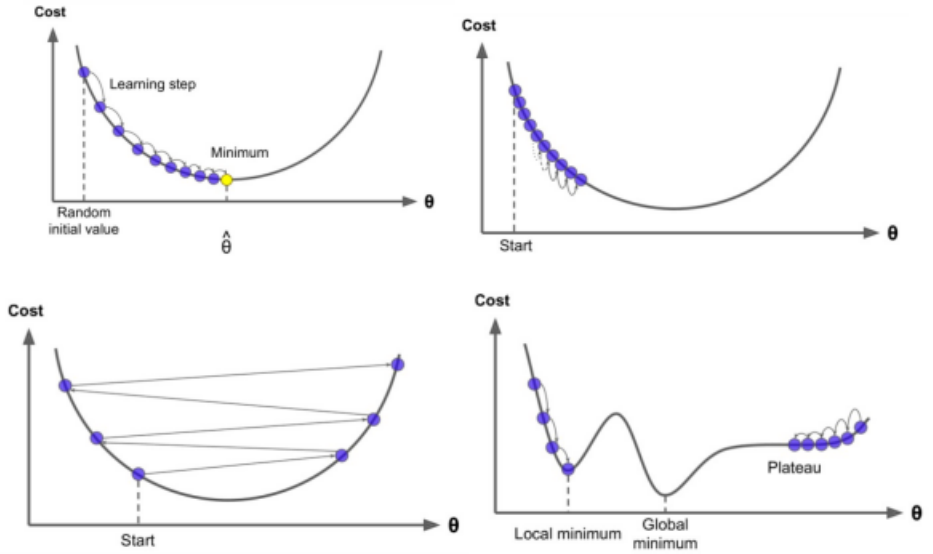



图 10-57 学习率与收敛

在选择学习率的时候，我们可以通过计算，选择使方向导数为零或者能使目标函数 $f(x_0-\varepsilon \nabla_x f)$产生最小值的 $\varepsilon$ 。

梯度下降法，将会在梯度各个分量为 0 的时候收敛（即多元函数求极值里的必要条件，各偏导数为零）。

梯度下降法也会面临一些问题，比如会陷入局部最优（Local Minimum）；有一些微分

是 0 的点不是极值点而是鞍点。实际应用过程中，我们也并不会真的更新到微分值等于 0 的点，可能只是小于一个比较小的值（比如 $10^{-6}$）就不更新参数了，而我们无从得知这个点是不是一个比较优的值，可能只是处于平坦的高原地区。梯度下降法，在越接近最优值时，步长应该不断减小，否则会在最优值附近来回震荡（即“之字形”地下降），因此在靠近极小值时收敛速度会减慢。

梯度下降法又可以进一步分为批量梯度下降和随机梯度下降。批量梯度下降会最小化所有训练样本的损失函数，使得最终求解的是全局的最优解，即求解的参数是使得风险函数最小，但是对于大规模样本问题效率低下。随机梯度下降会最小化每条样本的损失函数，虽然不是每次迭代得到的损失函数都向着全局最优方向， 但是大的整体的方向是向全局最优解的，最终的结果往往是在全局最优解附近，适用于大规模训练样本情况。

### 10.7.3 牛顿法

由于不是所有的方程，都有求根公式（比如 5 次以上的方程就没有求根公式了）或者有精确的解，因此有时候只能得到方程的近似解。牛顿法是 17 世纪由牛顿提出用于求解方程近似根的方法。用牛顿迭代法解非线性方程，是把非线性方程 f(x)=0 线性化的一种近似方法。把 f(x)在点 $x_0$ 的某邻域内展开成泰勒级数：

$$f(x)=f(x_0)+f'(x_0)(x-x_0)+\frac{f''(x_0)(x-x_0)^2}{2!}+\cdots+\frac{f^{(n)}(x_0)(x-x_0)^n}{n!}+R_n(x)$$

取其线性部分（即泰勒展开的前两项，最高次数为 1，所以叫线性部分），并令其等于 0，即

$$f(x_0)+f'(x_0)(x-x_0)=0$$

以此作为非线性方程

$$f(x)=0$$

的近似方程，若:

$$f'(x_0)\neq 0$$

则其解为:

$$x_1=x_0-\frac{f(x_0)}{f'(x_0)}$$

这样，得到牛顿迭代法的一个迭代关系式:

$$x_{n+1}=x_n-\frac{f(x_n)}{f'(x_n)}$$

已经证明，如果函数是连续的，并且待求的零点是孤立的，那么在零点周围存在一个区域，只要初始值位于这个邻近区域内，那么牛顿法必定收敛。

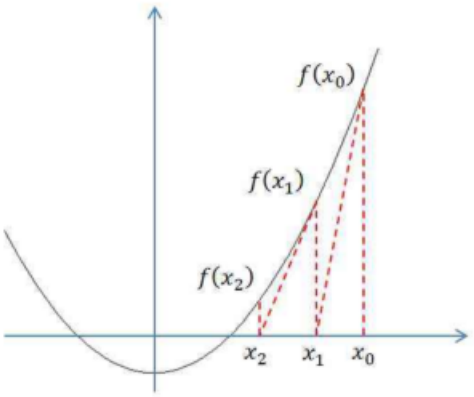



图 10-58 牛顿法求方程近似解

如图 10-58 所示，从函数的几何图形上来看，在纵坐标与 f'(x)同号的那个端点($x_0$)作切线，这切线与 x 轴的交点的横坐标 $x_1$ 就比 $x_0$ 更接近方程的根 ξ，继续在 f(x)上在点 $x_1$ 处作切线，与 x 轴相交，交点横坐标为 $x_2$，那么 $x_2$ 就比 $x_1$ 更接近方程的根。

我们知道，为了优化目标函数 f，需要求目标函数 f 的极值，那么可以转化为求解函数 f 的导数对应方程 f=0 的问题（因为函数的极值是在导数为零的驻点取得），这样就可以把优化问题看成方程求解问题（f=0）。剩下的问题就和第一部分提到的牛顿法求解很相似了。

这次为了求解 f=0 的根，把 f（x）的泰勒展开，展开到 2 阶形式：

$$f(x) = f(x_0) + f'(x_0)(x - x_0) + \frac{f''(x_0)(x - x_0)^2}{2!}$$

那么令 f'(x)=0，则有：

$$f'(x_0)+f''(x_0)(x-x_0)=0$$

求解有：

$$x=x_0-f'(x_0)/f''(x_0)$$

得出迭代公式：

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)},\ n = 0, 1, \ldots$$

上面我们讨论的是一元函数的牛顿法求函数极值得方法。那么在多元函数情况下，牛顿法是如何优化目标函数的呢？

在多元函数情况下，一个点的二阶偏导数有多个（即该函数的海森矩阵里的每一个元素都是该点的二阶偏导数），与一元函数的类似的牛顿迭代公式为：

$$\mathbf{x}_{n+1} = \mathbf{x}_n - [Hf(\mathbf{x}_n)]^{-1}\nabla f(\mathbf{x}_n),\ n \geq 0$$

其中 H 是多元函数 f 的海森矩阵，即移动的距离为点 xn 减去海森矩阵的逆矩阵乘以 f 的梯度。在这里海森矩阵的逆起到了学习率的作用，随着迭代的进行，海森矩阵的逆会变小，从而搜索的步长也会逐渐变小。

牛顿法由于可以利用到曲线本身的信息（即二阶导数，从而提供一阶导数的变化快慢信

息），因此比梯度下降法收敛速度更快（迭代更少次数），更容易接近极值点。

这是因为梯度下降法，采用的搜索极小值的策略是沿着梯度反方向移动步长进行寻找，但在多维空间里，一个点在每个方向上的二阶偏导数很可能是不同的，而二阶导数是一阶导数变化快慢的反应，所以也就是说这个点在某些方向上导数变化很快，而在另一个方向上导数变化很慢。但梯度下降法只包含了一阶导数的信息，是无法知道导数自身的变化快慢的，因此也就不能优先探索导数长期为负的方向。另外，某些时候，在下降过程中选择的步长也必须越来越小，否则因为移动而造成越过了函数的极值点。

实际上，用一个例子，就可以很好解释为什么牛顿法比梯度下降法更容易收敛，也就是使用更少的迭代次数而达到极值点。本质上讲，牛顿法是二阶收敛，梯度下降是一阶收敛，所以牛顿法就更快。由于梯度下降法利用的是函数的一阶导数，而牛顿法用的是函数的二阶导数，也就是导数的导数，这就好比于钟表上的分针和秒针。牛顿法类似于分针，梯度下降法类似于秒针。由于等级不一样，为了到达钟表上某一个时刻点，秒针移动的次数要远大于分针。

非线性函数局部的梯度方向并不一定就是朝着最优点。比如想找一条最短的路径走到一个盆地的最底部，如图 10-59 所示，梯度下降法每次只从当前所处位置选一个坡度最大的方向走一步（但从整体上来说，这样走下去，不一定能最快到达盆底），牛顿法在选择方向时，不仅会考虑坡度是否够大，还会考虑走了一步之后，坡度是否会变得更大。所以，可以说牛顿法比梯度下降法看得更远一点，能更快地走到最底部。（牛顿法目光更加长远，所以少走弯路；相对而言，梯度下降法只考虑了局部的最优，没有全局思想）

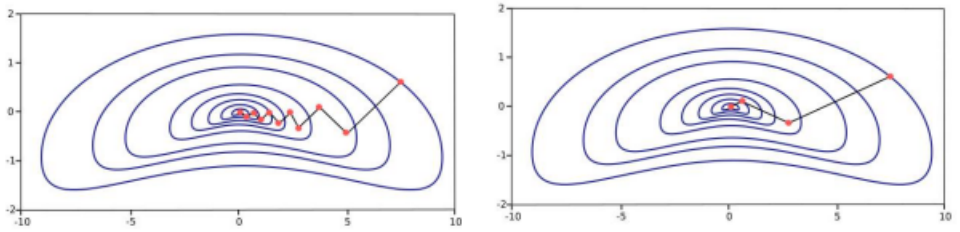



图 10-59 牛顿法与梯度下降法

牛顿迭代法对于局部二次型（海森矩阵 H 为正定的），用 $H^{-1}$ 调整梯度，牛顿法会直接跳到极小值。如果是非二次型，只要海森矩阵 H 为正定的，牛顿法也可以迭代收敛。牛顿法只适用于海森矩阵正定的情况。牛顿法在鞍点附近，是无能为力的，会导致更新朝错误方向移动，因为牛顿法的目标就是在寻找梯度（一阶导数）为零的点，这些点有可能是极值点，也有可能是鞍点。

多元函数下可以用牛顿迭代求解函数的极值，但海森矩阵引入的复杂性，使得牛顿迭代求解的难度增加了。首先，海森矩阵的元素数目是参数数量的平方，如果参数数量是 k（可能达到百万级别），那么每次迭代都需要计算 k*k 矩阵的逆，计算的代价非常大，这在很多情况下是不可接受的。

为了解决这个问题，于是有了拟牛顿法（Quasi-Newton method）。在拟牛顿法里，不再直接计算海森矩阵，而是每一步的时候使用梯度向量更新海森矩阵的近似。它使用正定矩阵来近似 Hessian 矩阵的逆，从而简化了运算的复杂度。拟牛顿法和最速下降法一样只要求每一步迭代时知道目标函数的梯度。通过测量梯度的变化，构造一个目标函数的模型使之足以产生超线性收敛性。这类方法大大优于最速下降法，尤其对于困难的问题。另外，因为拟牛顿法不需要二阶导数的信息，所以有时比牛顿法更为有效。如今，优化软件中包含了大量的拟牛顿算法用来解决无约束，约束，和大规模的优化问题。

梯度下降法和最小二乘法相比，梯度下降法需要选择步长，而最小二乘法不需要。梯度下降法是迭代求解，最小二乘法是计算解析解。如果样本量不算很大，且存在解析解，更适合于最小二乘法。如果样本量很大，最小二乘法需求一个超大逆矩阵，那么更适合使用梯度下降法。

梯度下降法和牛顿法/拟牛顿法都是迭代求解，不过梯度下降法是梯度求解，而牛顿法/拟牛顿法是用二阶导数的海森矩阵的逆矩阵或伪逆矩阵求解。一般来说使用牛顿法/拟牛顿法收敛更快（即迭代次数更少），但是每次迭代的时间比梯度下降法长。在训练大规模神经网络时，因为有上万的参数，所以梯度下降法是比较有效的。因为梯度下降算法储存的梯度算符向量规模为 n，而海森矩阵储存的规模就为 $n^2$ 了，同时梯度的计算量要远小于海森矩阵的计算量。

## 10.7.4 卷积神经网络

当学完积分学理论基础之后，我们现在就可以来学习一个广泛用于计算机人工智能领域的概念和运算：卷积（convolution）。卷积最早由大数学家欧拉发明，它与加减乘除类似，都是一种数学运算，只是这种数学运算稍微复杂一点，并且以积分运算为基础，要理解和掌握卷积，必须先要有积分的基础。

卷积最早应用在信号系统理论中，然后在很多领域都有重要的应用，比如在统计学、声学、物理学、计算机科学等领域，都起着巨大的作用。

同时，卷积也是一个比较抽象和难懂的运算，很多人第一次接触，都会感到一脸茫然，难以理解。接下来我们尝试从概念，生活中的例子，以及它在计算机领域中的应用来详细介绍相关的概念和原理。

## 1.什么是卷积？

首先我们来看一下卷积的定义。卷积是两个变量在某范围内相乘后求和的结果。卷积的定义有两种形式，一种是离散形式的定义，一种是连续形式的定义。如果卷积的变量是序列 f(n)和 g(n)（离散的），卷积的结果记为 h(n)或 f.g(n)，则有：

$$h(x) = f.g(x) = \sum_{\tau=-\infty}^{+\infty} f(\tau)g(x-\tau)$$

如果卷积的变量是函数 f(x)和 g(x)（连续的），卷积的计算结果记为 h(x)或 f.g(x)，则有:

$$h(x) = f.g(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

通过卷积的定义，可以看出，卷积的离散定义形式是 i 在(-∞, +∞)上 f(i)和 g(n-i)先相乘再相加求和；连续定义形式是 τ 在(-∞, +∞)上 f( τ )和 g(x- τ )的乘积的积分（积分也可以看作是连续求和的过程）。卷积前 f( τ )g(x- τ )是一个关于 x 和 τ 的二元函数，在卷积过程中，

去掉了 τ，所以卷积结束之后是一个 x 的一元函数。

卷积简单来说就是对两个函数的乘积求积分，如果函数不是连续的，而是离散化的，那么就是两个函数乘积后再求和然后在对这个过程不断叠加。

卷积的定义很简洁，但是它有什么含义呢？这是卷积最让人难以理解的地方，也是初学者最困惑的地方。现在我们试着通过浅显易懂的例子给大家解释一下卷积的含义。

现在我们假设有一个稳定的系统，每次外部给系统一个输入信号进行激励，系统在收到这个输入之后都会以一定的规律缓慢地对输入做出响应，当外部输入信号停止时，系统也只会按照自己内在的规律逐渐衰减到零（当然也可以一直响应而不衰减）。

比如，假设人体就是一个稳定的系统，那么假设在某个时刻，人体受到了一拳击打，就相当于对系统的一个激励，人体在收到这个激励之后就会产生疼痛感，这个疼痛从一开始的最强烈到最后会慢慢消失掉。那么我们就认为人体从疼痛开始到最后消失这个过程就是一个响应。

那么我们可以认为 f 函数是外部对系统在某个时刻的那个激励，而 g 函数就是系统对激励产生的响应衰减规律。假设 f(τ)表示在 τ 时刻带给系统的激励，x 为某一个具体的时间点，x-τ 为 τ 时刻激励发生之后，距离 x 的一个时间间隔 Δt= x-τ，而 g(Δt)=g(x-τ)则是在这个时间间隔的时候，系统响应衰减的规律，衰减成都很明显与 Δt（即 x-τ）有关，那么 f(τ)g(x-τ)相乘，就是系统在受到 τ 时刻的激励后，在 x 时刻的响应大小。

同样以人体受击打之后产生的痛感为例子。假设痛感的大小是可以度量的，那么在 τ 时刻我们给人体的一次击打，这次击打的力量可以记为 f(τ)，随之而产生痛感会从被击打那一刻开始从强到弱逐渐减小直到消失，那么在接下来的某个 x 时刻，疼痛的衰减程度设为 g(t-τ)，即衰减规律是该 x 时刻距离被击打那一刻 τ 的时间间隔 x-τ 的一个函数值。那么 f(τ)g(x-τ)就是在 x 时刻，由于 τ 时刻的一次击打而造成的疼痛的大小。

假设人体分别在 τ1，τ2，τ3 时刻受到了 3 次击打，击打的力量设为 f(τ1), f(τ2), f(τ3)，那么在 x 时刻受到的的痛感应该分别为: f(τ1)g(x-τ1), f(τ2)g(x-τ2), f(τ3)g(x-τ3)，假设痛感是可以叠加的，那么总的痛感就是 f(τ1)g(x-τ1)+f(τ2)g(x-τ2)+f(τ3)g(x-τ3)。

那么假设人体受到了无数次的击打，即：-∞, …， τ1， τ2， τ3，…∞，那么人体在某一个时刻 x 对这无数次击打总的痛感就为：

$$h(x) = f.g(x) = \sum_{\tau=-\infty}^{+\infty} f(\tau)g(x-\tau)$$

上面对应的就是卷积的离散定义形式。它的实际意义就是所有的击打，在 x 这一时刻带来的总的疼痛感的一个叠加。

假设人体受到的击打速度越来越快，从离散变成了连续，也就是人体无时无刻不间断的受到击打，且在 τ 时刻受到的击打力量记为 f(τ)，然后在 x 时刻对应的疼痛衰减程度为 g(x-τ)（x-τ 是从 τ 时刻到 x 时刻的时间间隔），那么对应的疼痛则为：f(τ) g(x-τ)，那么人体在 x 时刻对于所有时刻的击打总的疼痛感就为：

$$h(x) = f.g(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

为了让大家更加容易的理解卷积这个概念和运算，我们再给大家介绍一个生活中可以运用卷积的例子，那就是存钱。

假设某人A往银行存入100元钱，年利率是5%，按复利（即将每一年所获利息加入本金，以计算下一年的利息）计算，那么在五年之后他能拿到的钱数是$100(1+5\%)^5$。

一年之后，A又往银行中存入了100元钱，年利率仍为5%，那么这笔钱按复利计算，到了第五年，将收回的钱数是$100(1+5\%)^4$，那么与第一笔钱一起，A在每年能拿到的钱数如下表所示：

| 本金 | 第一年 | 第二年 | 第三年 | 第四年 | 第五年 |
|---|---|---|---|---|---|
| 100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ | $100\times(1.05)^3$ | $100\times(1.05)^4$ | $100\times(1.05)^5$ |
| | +100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ | $100\times(1.05)^3$ | $100\times(1.05)^4$ |

以此类似，如果小明每年都往银行中存入新的100元钱，那么这个收益表格将是这样的：

| 本金 | 第一年 | 第二年 | 第三年 | 第四年 | 第五年 |
|---|---|---|---|---|---|
| 100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ | $100\times(1.05)^3$ | $100\times(1.05)^4$ | $100\times(1.05)^5$ |
| | +100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ | $100\times(1.05)^3$ | $100\times(1.05)^4$ |
| | | +100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ | $100\times(1.05)^3$ |
| | | | +100 | $100\times(1.05)^1$ | $100\times(1.05)^2$ |
| | | | | +100 | $100\times(1.05)^1$ |
| | | | | | +100 |

通过表格可以看出，A在每年拿到的钱将等于他各年存入的钱分别计算复利之后得到的钱数的总和。如果把银行看作一个稳定的系统，那么A在第i(i从0开始)年存入银行100元钱，就是对银行的一个输入激励，银行在收到这100元钱之后，就会给出一个响应，对应的响应规则为：$(1+5\%)^{n-i}$,其中n是在第n年，n-i是第i年距离第n年的时间间隔。那么第i年投入的100元钱，在第n年的响应收益就是：$100(1+5\%)^{n-i}$。那么以第5年为例，A每年向银行存入100元钱在第5年的总收益应该为每年投入的100元带来的收益之和：

- 第0年投入100元在第5年的收益为：$100(1+5\%)^{5-0}=100(1+5\%)^5$
- 第1年投入100元在第5年的收益为：$100(1+5\%)^{5-1}=100(1+5\%)^4$
- 第2年投入100元在第5年的收益为：$100(1+5\%)^{5-2}=100(1+5\%)^3$
- 第3年投入100元在第5年的收益为：$100(1+5\%)^{5-3}=100(1+5\%)^2$
- 第4年投入100元在第5年的收益为：$100(1+5\%)^{5-4}=100(1+5\%)^1$
- 第5年投入100元在第5年的收益为：$100(1+5\%)^{5-5}=100(1+5\%)^0$

所以，第5年因为以前每年投资100元钱带来的总收益为：

$$100(1+5\%)^5+100(1+5\%)^4+100(1+5\%)^3+100(1+5\%)^2+100(1+5\%)^1+100(1+5\%)^0$$

与此类推，那么A在第n年获得的总收益应该为之前每一年投入的100元钱在第n年的收益的总和，于是有：

$$h(n)=\sum_{i=0}^{n}100(1+0.05)^{n-i}$$

设f(i)=100为A的存钱函数,g(i)为存入银行的每一笔钱的复利计算函数且$g(n-i)=(1+0.05)^{n-i}$，那么A在第n年得到的钱就是f(i)和g(n-i)的一个卷积。

如果将这个公式推广到连续的情况，也就是说，A在从0到t的这一段时间内，每时每刻都往银行里存钱，他的存钱函数为$f(\tau)\ (0\le\tau\le t)$，而银行也对他存入的每一笔钱按复利公式计算收益：$g(t-\tau)=(1+5\%)^{t-\tau}$，则A到时间t将得到的总钱数为：

$$\int_0^t f(\tau)g(t-\tau)d\tau = \int_0^t f(\tau)(1+5\%)^{t-\tau}d\tau$$

那么A在t时刻获得的钱同样是f(τ)和g(t-τ)的卷积。

在这个存钱的例子里面，如果我们将A的存款函数视为一个对银行系统激励的过程，而复利函数$g(t-\tau)$视为一个系统对信号的响应，那么二者的卷积$(f*g)(t)$就可以看做是在t时刻对系统的输出，它是过去产生的所有激励经过系统的响应后得到的结果的叠加，这也就是卷积的意义。

实际上，现实社会生产和生活中，有很多例子都可以利用卷积来进行科学计算。比如：

辐射：设某核电站事故中，某工作人员每天到抢险现场工作M分钟，接受一定剂量的辐射，辐射会自然地衰减，如此工作N天，总的辐射量。

空袭：某国部队每隔M小时对某地区或国家实行间歇性空中打击，每次打击后，其物理破坏和心理震慑作用会因为被空袭方的组织抢修心里承受度增加而随时间衰减，如此进行N天后，累积的打击总效果。

还有其他例子，如水滴石穿（石穿总是一段时间水的作用加权求和），饭店生产馒头和变素数量，长期服药的血药浓度，长期吸入污染物在人体内的积累，吸烟或喝咖啡的积累效应，多次喷洒农药的残留量等，也可以用卷积来估计。

再回到卷积的定义中来，通过关于卷积意义的学习之后，我们应该可以看出，卷积实际上表示的是在当前时间x时，过去一系列的激励带来的响应对现在的影响（响应的叠加）。

$$h(x) = f.g(x) = \int_{-\infty}^{+\infty} f(\tau)g(x-\tau)d\tau$$

比如：

当τ=x时，f(τ)=f(x)，是当前对系统的冲击，响应为g(x-τ)=g(0)；

当τ=x-1时，f(τ)=f(x-1)，是相对于x的前一秒（假设时间以秒为单位）产生的冲击，响应为g(x-τ)=g(x-(x-1))=g(1)；

当τ=x+1时，f(τ)=f(x+1)，是相对于x的未来一秒产生的冲击，响应为g(x-τ)=g(x-(x+1))=g(-1)，这个值一般为0。

与此类推。

卷积告诉我们，当下或者过去做的任何一件事都会对未来产生影响，所谓冰冻三尺非一日之寒。而未来可以是接下来一分钟，一小时，一天，甚至若干年。产生的影响可以微乎其微，也可能改变命运。那么我们可以通过卷积算出这些事件在将来任一时刻将会产生多少影响。

所以定义中的变量τ是过去或者现在对系统造成激励的时刻，而x是现在或者将来要计算所有的激励造成总的影响所对应的时刻，x-τ就是τ时刻距离x时刻的时间间隔或者距离，通过这个时间间隔代入响应函数就可以算出τ时刻在x时刻的响应g(x-τ)。

上面我们从卷积的意义出发讲解了卷积的原理。那么，卷积中的“卷”又是什么含义呢？为什么这种运算会被命名为“卷积”呢？这就需要从数学的角度来进行解释。

先来看看任意一个函数g(x)，它的图像会经过什么样的变换，得到g(a-x)（其中a为常量，x为自变量）的图像呢？

首先把g(x)沿着x轴向右平移a个单位，就可以得到函数g(x-a)。

然后把g(x-a)沿着直线x=a翻转（也叫对折），就可以得到g(a-x)的图像。

如图 10-60 所示，以 $y=1.05^x$ 为例子，把该函数图像沿着 x 轴向右平移 100 个单位，得到了 $y=1.05^{x-100}$ 的图像；然后将 $y=1.05^{x-100}$ 的图像沿着 x=100 对折翻转，就得到了 $y=1.05^{100-x}$ 的图像。

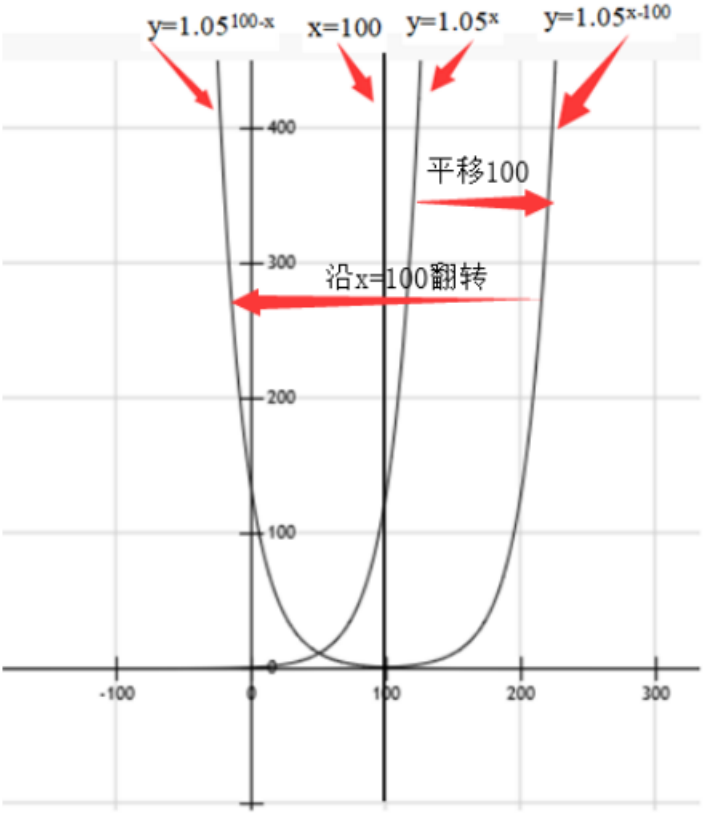



图 10-60 函数 g(x)变换到 g(a-x)

再考查卷积定义中的式子 f(τ)g(x-τ)，注意，在这个式子中τ是自变量，x 是常量，所以函数 g(x-τ)实际上也是由函数 g(τ)经过平移 x 单位然后再沿着 τ=x 这条直线翻转而来。所以可以把 g(x-τ)看作是卷，而 f(τ)g(x-τ)是积，f(τ)g(x-τ)dτ 的积分就叫做卷积。

卷积过程其实就是“滑动，加权，叠加”三步曲：

首先，第一步滑动是指对 g 函数进行平移和翻转，相当于在数轴上把 g 函数从右边褶到左边，也就是所谓的“卷”。

其次，第二步加权是指 f(τ)g(x-τ)相乘。系统在受到 f(τ)信号激励后，对系统产生的影响的大小与 f(τ)本身和因时间间隔产生的修正 g(x-τ)(修正可为衰减，或增强，或其他更复杂的形式)有关。所以 g(x-τ)相当于 f(τ)的权重，两者相乘即为τ时刻产生的激励对 x 时刻系统的影响。

最后，第三步叠加就是将系统 x 时刻前后无数多个时刻输入的激励对系统造成的影响大小叠加在一起。从数学表达式上来看，就是求和或积分。

卷积的结果不仅跟当前时刻输入信号的响应值有关，也跟过去所有时刻输入信号的响应值都有关系，考虑了对过去的所有输入的效果的累积。

## 2.图像中的卷积运算

卷积在图像处理中也有着重要的应用，而人工智能和深度学习也离不开对图像的处理，所以卷积也是人工智能和深度学习的基础。

对于一幅图像，经过卷积处理以后，输出图像将会得到模糊，边缘强化等各种效果。在图像处理中，卷积处理的结果，就是把每个像素周边的，甚至是整个图像的像素都考虑进来，对当前像素进行某种加权处理。

下面我们以一个实际例子来介绍图像中的卷积。通过本书 9.3.14 节介绍，我们知道，在计算机中，图片的像素通常用“水平像素数(width)×垂直像素数(height)”（分辨率）的形式表示。而且一张图片分为 RGB（红绿蓝，三原色组成了所有颜色）三个通道，每个通道分别表示了红，绿，蓝三种颜色的像素。图像是通过标量（3 维数组）来表示的，也就是每个颜色通道一个矩阵，共 3 个矩阵，矩阵中每个位置的元素值代表了对应的像素在该点的颜色值。

现在我们就以 3 个通道中任意一个矩阵为例子。

在机器学习中，对图像进行处理，往往包括对图像中一些特征的提取，比如边缘轮廓的提取，这就需要对图像中的一些无关信息或者噪声进行过滤去除。设图像对应的矩阵为:

$$A=\begin{bmatrix} a_{0,0} & a_{0,1} & a_{0,2} & \cdots & a_{0,n} \\ a_{1,0} & a_{1,1} & a_{1,2} & \cdots & a_{1,n} \\ a_{2,0} & a_{2,1} & a_{2,2} & \cdots & a_{2,n} \\ \cdots & \cdots & \cdots & \cdots & \cdots \\ a_{m,0} & a_{m,1} & a_{m,2} & \cdots & a_{m,n} \end{bmatrix}$$

为了对图像进行处理，可以把图像数据作为输入，传值给对应的函数来进行处理，而对图像的处理函数也可以用一个 s×s 的矩阵 g 来表示，如：

$$g=\begin{bmatrix} b_{-1,-1} & b_{-1,0} & b_{-1,1} \\ b_{0,-1} & b_{0,0} & b_{0,1} \\ b_{1,-1} & b_{1,0} & b_{1,1} \end{bmatrix}$$

在这里，g 是一个 3×3 的矩阵。 设 $a_{u,v}$ 为图像矩阵 A 中(u,v)处的一个像素点，取以 $a_{u,v}$ 为中心附近的一个 3×3 的矩阵记为 f，设:

$$f=\begin{bmatrix} a_{u-1,v-1} & a_{u-1,v} & a_{u-1,v+1} \\ a_{u,v-1} & a_{u,v} & a_{u,v+1} \\ a_{u+1,v-1} & a_{u+1,v} & a_{u+1,v+1} \end{bmatrix}$$

那么函数 f 和 g 的在（u，v）处的卷积 $f*g(u,v)$ 定义为：

$$(f*g)(u,v)=\sum_i\sum_j f(i,j)g(u-i,v-j)=\sum_i\sum_j a_{i,j}b_{u-i,v-j}$$

其中，我们把 f 称为图像输入，把 g 矩阵称为卷积核，在卷积操作中就是将卷积核与输入图像混合在一起。卷积核的输出是一幅修改后的图像，在深度学习中常被叫做特征图谱

（feature map）。每个颜色通道都会有一个特征图谱。

这个图像卷积定义可以看出，f 和 g 的卷积实际上是在矩阵中行和列（分别是 i 和 j）两个方向进行先乘积，再求和。那么具体是如何计算的呢？

首先需要将卷积核 g 进行翻转，得到一个新的矩阵 g'。翻转的过程就是先将矩阵 g 中第 i 行和 s-i+1 行交换，然后在讲矩阵中第 j 列和 s-j+1 列进行交换（当然也可以先交换列，再交换行）。对于上面的举例中的 g 是一个 3×3 的矩阵，那么就是将矩阵中的第 1 行和第 3 行交换，然后再将第 1 列和第 3 列交换，得到的一个新的矩阵：

$$g' = \begin{bmatrix} b_{1,1} & b_{1,0} & b_{1,-1} \\ b_{0,1} & b_{0,0} & b_{0,-1} \\ b_{-1,1} & b_{-1,0} & b_{-1,-1} \end{bmatrix}$$

那么计算 f 和 g 的卷积，就可以使用矩阵 f 和 g' 的内积，即：

$$f * g(u,v) = a_{u-1,v-1} \times b_{1,1} + a_{u-1,v} \times b_{1,0} + a_{u-1,v+1} \times b_{1,-1}$$

$$+a_{u,v-1} \times b_{0,1} + a_{u,v} \times b_{0,0} + a_{u,v+1} \times b_{0,-1}$$

$$+a_{u+1,v-1} \times b_{-1,1} + a_{u+1,v} \times b_{-1,0} + a_{u+1,v+1} \times b_{-1,-1}$$

可见在实际计算图像中某点和卷积核的卷积的时候，只需要把卷积核翻转，然后直接通过求矩阵内积就可以得到图形在这一点的卷积了。

以上计算的是（u,v）处的卷积，将 f 矩阵依次沿着图像矩阵行或者列的方向移动，就可以求出图像中各个位置的卷积，其输出结果是经过卷积处理后的新图像。

对于图片中的每一个像素点，计算它的邻域像素和一个卷积核（滤波矩阵）的对应位置元素的乘积，然后将所有乘积累加，作为该像素位置的输出值。卷积核依次滑过图片中的每一个像素位置，就可以输出一张分辨率不变的新图片。这个过程从数学上来说，就是两个矩阵的滑动点积（也叫滑动内积）。

如图 10-61 所示，设卷积核 g 是一个 3×3 的矩阵，经过翻转之后的矩阵为:

$$g' = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & -4 \end{bmatrix}$$

那么图片对应的矩阵为 A，卷积之后新的图片对应的矩阵为 C，那么要计算 $a_{22}$ 这一点与 g 的卷积，取 $a_{22}$ 为中心的矩阵:

$$f=\begin{bmatrix}0 & 0 & 0\\ 0 & 1 & 1\\ 0 & 1 & 2\end{bmatrix}$$

那么卷积结果就应该等于 f 与 g'的内积，即：

$$c_{22}=4*0+0*0+0*0+0*0+0*1+0*1+0*0+0*1+(-4)*2=-8$$

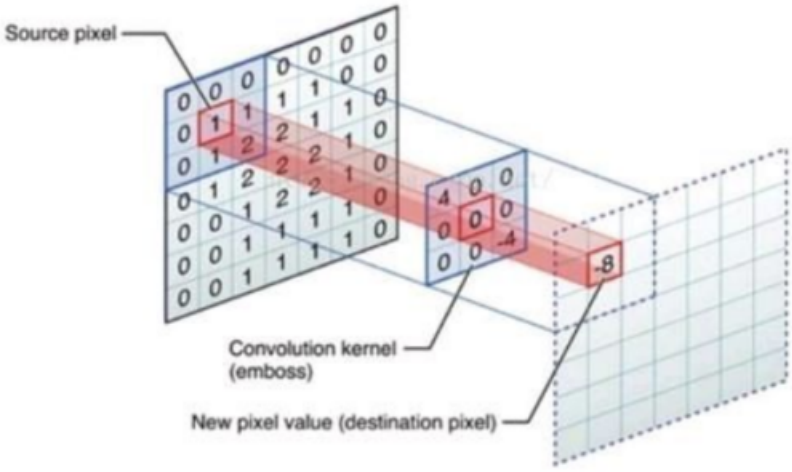



图 10-61 图像中某点和卷积核的卷积

同样，将矩阵 f 水平向右移动 1 列，f 变为新的矩阵：

$$\begin{bmatrix}0 & 0 & 0\\ 1 & 1 & 1\\ 1 & 2 & 2\end{bmatrix}$$

那么此时 f 与 g' 的内积就是图像中新的点 $a_{23}$ 与 g 的卷积，对应着新生成的图像 C 中 $c_{23}$ 处的像素值，即：

$$c_{23}=4*0+0*0+0*0+0*1+0*1+0*1+0*1+0*2+(-4)*2=-8$$

那么随着 f 的滑动，就可以完成图片 A 中所有的点与 g 的卷积，从而生成一幅分辨率与 A 相同的新图像 C。

与此类似，如果 g 矩阵不是 3x3，而是 5x5，7x7 等其他维度，那就在原始图像中取以（u，v）为中心的 5x5 或者 7x7 矩阵进行同样的卷积计算。

由此可见，图像中的卷积就是把原始图像中某点的相邻像素都加进来，与该点进行混合。相邻的区域范围大小取决于卷积核矩阵 g 的维度，维度越大，涉及的周边像素越多。而卷积核矩阵的设计，则决定了混合输出的图像与原始图像相比，究竟是变模糊还是变锐利。

卷积核被视为图像变换函数的系数矩阵，而矩阵的维度被称为感受野。如图 10-62 所示，当对一个图像使用多个不同的卷积核进行卷积时即可得到多种不同的特征图，这些特征图用于提取图像的局部特征（比如轮廓，背景等），同时保留了部分特征的空间关联性，又降低了无关信息的噪声，因此卷积核又常常被称为滤波器。

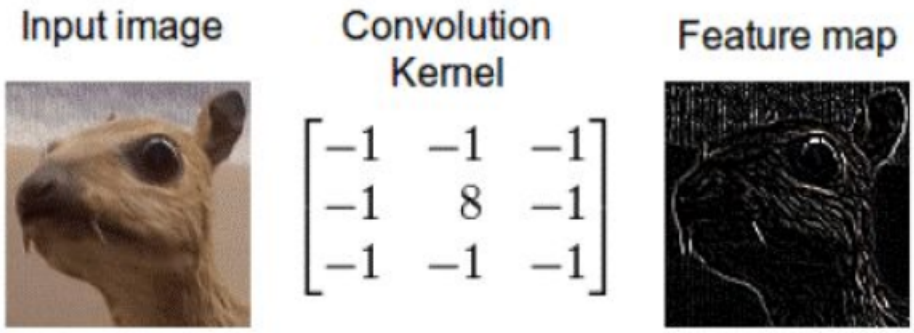



图 10-62 通过卷积提取图像局部特征

## 3.卷积神经网络

计算机科学中，卷积神经网络（CNN）是深度学习算法中的一种，近年来被广泛用到模式识别、图像处理等领域中。

在介绍卷积神经网络之前，我们先来了解一下什么是机器学习与深度学习。

**机器学习**

首先我们人类是具有学习能力的。人类通过学习，可以获得经验，然后通过经验可以进行各种判断和预测。所以人是有学习能力的。比如当我们看到晚霞，就会预测明天是个好天气；仅仅听到人说话的声音，就能判断是男是女；通过西瓜的色泽，根蒂和敲声就可以判断瓜的好坏。人类能做出这些有效的预判，完全是因为我们积累了与此相关的丰富经验，并能够利用这些经验来进行决策。

机器学习与人类的学习是类似的。只不过对于机器来说，所谓的经验通常是以数据的形式存在。我们将大数据提供给机器，机器通过模型算法（也叫学习算法，比如 11.8.3 中介绍的与广告点击率预估相关的 LR 和 FFM 模型算法，这是机器学习研究的主要内容）生成模型（模型通常是一个结果文件，大小可以达到上 GB），接下来我们就可以根据这个生成的模型结合学习算法来预测新的情况。因此，模型是机器从数据中学习得到的结果，从数据得到模型的过程叫作机器学习或者训练。

学习过程中使用的数据称为“训练数据”，其中的每一条记录称为“训练样本”，训练样本组成的集合称为“训练集”。

为了生成用于预测新情况的模型，训练样本有时候需要带上结果信息，比如在广告点击率预估机器学习中，每一条日志记录应该包含的特征信息有：什么样内容的广告，在什么显示位置，展示给了一个岁数多大，性别是男还是女的用户，以及这个用户有没有点击。那么这个有没有点击就是一个结果信息，我们把这个结果信息称为“标记（label）”。拥有了标记信息的训练样本称为“样例”。

利用模型预测的结果有可能是离散值（比如好天气与坏天气，好瓜与坏瓜），也有可能是连续值（比如广告点击概率预估的结果就是一个介于 0 到 1 之间的连续值）。预测结果为离散值的学习任务被称为“分类（classification）”，比如 logistic 二分类回归模型，softmax 多分类回归模型都属于分类任务；预测结果为连续值的学习任务被称为“回归(regression)”，比如线性回归(Linear regression)。

训练数据带有标记信息的学习任务，称为“监督学习”，比如上面介绍的分类和回归都是监督学习；训练数据不带标记信息的，称为“无监督学习”，常见的有“聚类”学习，

也就是把样本自动分成若干个组，每组称为“簇(cluster)”。每个簇可能对应了一些潜在的位置的概念，因此聚类学习可以帮助我们挖掘数据的内在规律。

现在我们以线性回归预测房屋价格为例子来介绍一下机器学习的过程。

影响房屋价格的因素很多，比如房屋的房龄，面积，地段，朝向，层高，装修，物业和周边环境，甚至包括国家购房限购政策，贷款利率等等。为了简单，我们只把房屋的房龄（以年为单位），面积（以平方为单位），和地段（距离市中心的距离，以千米为单位），作为影响房价的主要因素，现在我们通过线性回归，来探讨这些因素和房价的关系。

首先是建立房屋价格预测的线性回归模型。设房屋的售出价格为 y，房屋的面积为 $x_1$，房龄为 $x_2$，地段为 $x_3$。现在我们就是要确定 y 与 $x_1$，$x_2$，和 $x_3$ 之间的函数关系。这个关系确定之后，就得到了预测模型。就可以利用这个模型，预测新的房屋售价。线性回归，顾名思义，意味着 y 与 $x_1$，$x_2$，和 $x_3$ 是线性关系，即：

$$\hat{y} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b$$

其中 $w_1$，$w_2$ 和 $w_3$ 是上述线性函数的系数，也叫权重(weight)，b 是函数的常数项，也叫偏差（bias），$w_1$，$w_2$，$w_3$ 和 b 称为线性回归模型的参数，此时这些参数都是未知的，只要这些参数确定了，那么模型也就确定了。输出 $\hat{y}$ 是线性回归对真实价格 y 的预估。

接下来，就是模型训练。我们需要收集大量已出售房屋的数据构成我们的训练集，这些数据应该包括房屋面积，房龄和地段以及房屋出售价格。我们把这些数据用来寻找特定的模型参数值，使得模型在数据上的误差尽可能小。这个过程就被称为模型训练。

假设我们训练集中的样本数为 n，那么第 i 个样本的特征记为 $x_{1i}$，$x_{2i}$，$x_{3i}$，标签（售价）为 $y_i$，那么预测的房屋价格为：

$$\hat{y}_i = w_1 x_{1i} + w_2 x_{2i} + w_3 x_{3i} + b$$

预测与真实之间的误差使用：$\frac{1}{2}(\hat{y}_i - y_i)^2$来衡量。很显然，误差越小，模型就越精确。因此问题被转化为求误差最小值的数学问题。那么如何确定模型中未知参数，以使得预测的误差最小呢？方法就是我们在前面 10.7.1 和 10.7.2 中介绍的最小二乘法和梯度下降算法来计算。这里我们不再赘述。

在结束模型训练之后，我们用数学方法求得了最优的模型参数 $w_1$，$w_2$，$w_3$ 和 b，那么 y 与 $x_1$，$x_2$ 和 $x_3$ 之间的线性关系函数确定了。我们就可以用这个确定的线性关系函数来预测训练集之外的任意一栋房屋的价格。

实际上，模型算法里的函数系数（模型参数）是未知待确定的，而模型则是模型算法通过训练集计算出函数的系数，“模型算法”加上“模型”，就相当于给出了一个确定的已知函数。而利用模型预测新数据结果的过程就是将新的数据输入这个已知函数，该函数计算的结果就是预测的结果。

如图 10-63 所示，如果训练出来的模型在训练集上误差较高，我们称为“欠拟合(underfitting)”，这意味着模型没有很好地捕捉到数据特征，不能够很好地拟合数据；如果训练误差和测试误差差距太大，我们称为“过拟合（overfitting）”，这意味着模型把数据学习的太彻底，以至于把噪声数据的特征也学习到了，这样就会导致在后期测试的时候不能够很好地识别数据，模型泛化能力太差。

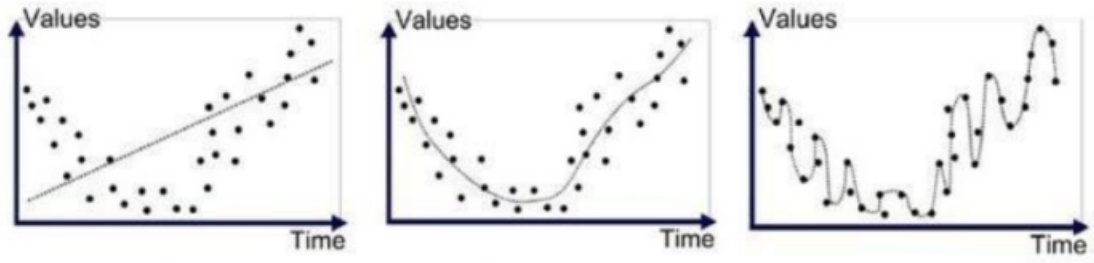



图 10-63 欠拟合，拟合和过拟合

以识别叶片的机器学习为例子，一个欠拟合的模型，由于训练误差很大，容易把一颗树识别为一片叶子；如果训练集中的叶子边缘有锯齿，如图 10-64 所示，那么一个过拟合的模型，由于学习过度，容易把锯齿作为叶子的特征，从而把一些不含锯齿的叶子判断为非叶子。

图 10-64 叶子识别中的过拟合

**深度学习**

那么什么又是深度学习呢？它和机器学习有什么关系呢？在介绍深度学习之前，我们来看看什么叫神经网络（neural networks）。

在计算机领域的神经网络，实际上是人工神经网络的简称，以区别与生物神经网络。它以人脑为模型，是一种模仿动物神经网络行为特征，进行分布式并行信息处理的机器学习模型。它是由具有适应性的简单单元组成的广泛并行互联的网络，能通过调整内部大量节点之间相互连接的关系，模拟生物神经网络对真实世界物体所作出的交互反应。

神经网络中最基本的组成是神经元。神经元与神经元相连。如图 10-65 所示，某个神经元接收到来自 n 个其它神经元传递过来的输入信号，这些信号经过带权重的连接进行传递，神经元接收到的总输入值与神经元的阈值进行比较，再通过激活函数（比如在本书 3.6 节中提到的 Sigmoid 函数）处理以产生神经元的输出。激活函数的作用就是将神经元的输出范围控制在(0, 1)区间。

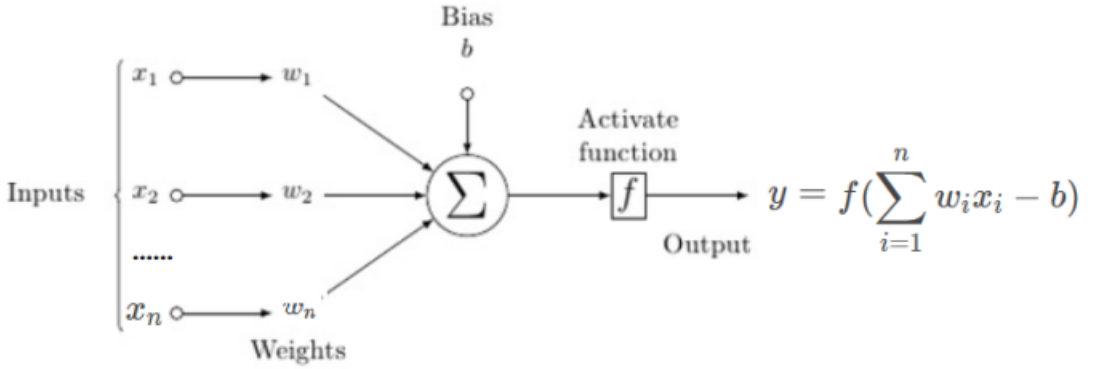



图 10-65 神经网络

许多个这样的神经元按一定的层次结构连接起来，就是神经网络。其中 $x_1$，$x_2$, …, $x_n$ 是输入，$w_1$, $w_2$, …, $w_n$ 是连接权重，简称连接权，b 是阈值。每个神经元对应了一个函数，

那么神经网络可以看作是由若干个包含参数的函数 $y = f(\sum_{i=1}^{n} w_i x_i - b)$ 相互嵌套而来。

线性回归机器学习方法实际上是单层的神经网络，如图 10-66 所示，因为只有输出层神经元进行激活函数处理。这就注定了此类方法学习能力有限，无法满足非线性相关的机器学习问题。

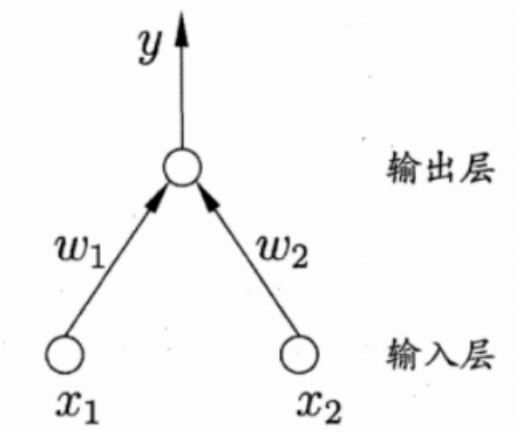



图 10-66 单层神经网络

要解决非线性问题，需要考虑使用多层神经网络。如图 10-67 所示，在多层神经网络里，每层神经元与下层神经元全互联，神经元之间不存在同层连接，也不存在跨层连接。这种神经网络通常称为“多层前馈神经网络”。

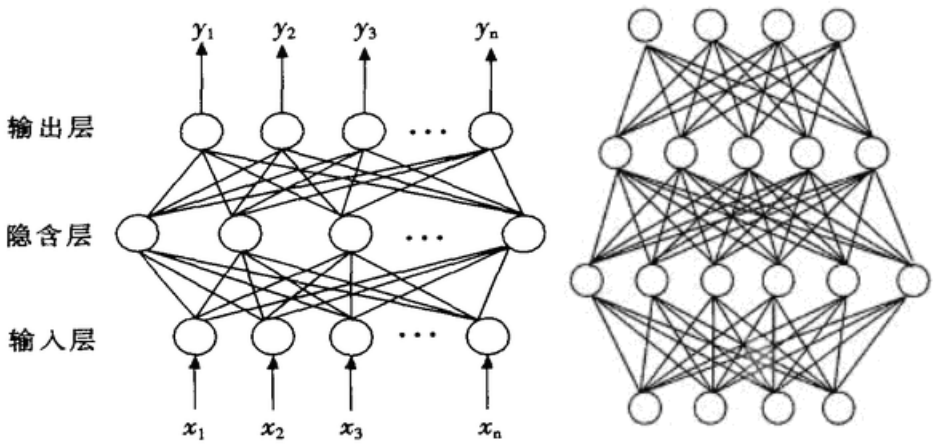



图 10-67 多层神经网络

我们把输入层与输出层之间的一层或多层神经元叫隐含层。输入层神经元只接收外界输入而不进行函数处理，隐层与输出层神经元对信号进行加工，输出层神经元将最终结果输出。

神经网络的学习，就是通过训练数据集来调整并确定神经元之间的连接权和每个功能神经元的阈值，因为这些参数确定之后，那么互相嵌套的函数就确定了。多层神经网络中最经典的学习算法称为 BP（Back-Propagation，误差逆向传播）算法。这里我们不作介绍，有兴趣者可以自行学习研究。

深度学习，作为机器学习中的一类，通常基于多层神经网络模型，并逐级表示越来越

抽象的概念。在深度学习的神经网络中，一般包含多个隐含层堆叠，每层对上一层的输出进行处理，逐层加工，从而把初始的，与输出目标之间联系不太密切的输入数据，转化成与输出目标联系更紧密的表示，也就是通过多层处理，把初始的低层特征转化为高层特征，这样使得原来仅基于输出层一层无法完成的任务变得可能，这个时候在输出层利用最简单的模型就可以完成学习任务。

**卷积神经网络**

卷积神经网络是指包含卷积层的神经网络。卷积神经网络是深度学习在计算机视觉领域取得突破的关键所在，同时也在自然语言处理，语音识别和推荐系统中有着广泛的应用。

卷积神经网络中的卷积层最常见的是二维卷积层，包含高和宽两个空间维度，因此非常适合于对图像的处理。

在二维卷积层中，存放图像数据的二维数组输入与二维卷积核数组通过特别的运算输出一个二维数组。

需要注意的是，这里的特别运算并不是典型意义上的卷积运算，而是一个所谓的“互相关”运算。它与前面讲的图像中典型的卷积运算唯一不同之处，就是卷积核在互相关运算中不需要翻转。

在互相关运算中，从输入数组左上方开始，卷积窗口（与卷积核同大小）按照从左往右，从上到下的顺序，依次滑动经过输入数组，每滑动一次，窗口中的输入子数组与卷积核数组按元素相乘并求和，得到输出数组中相应位置的元素。如图 10-68 所示就是一个二维数组与卷积核进行互相关运算的一个例子。

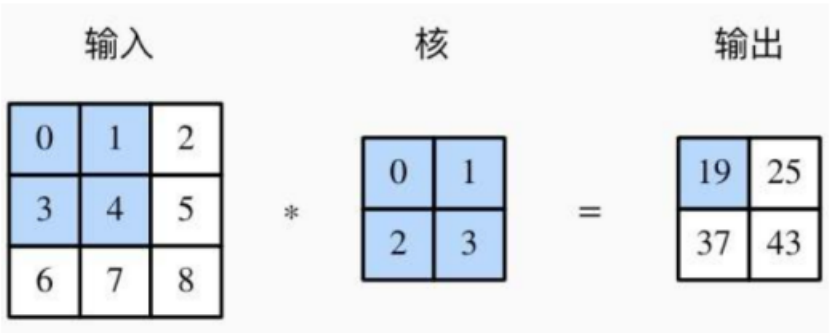



图 10-68 二维数组与卷积核进行互相关运算

二维卷积层通过将输入数组和卷积核做互相关运算，并加上一个偏差（标量）来得到输出。卷积层的模型参数包括了卷积核和偏差，也就是说卷积核和偏差是未知的，需要通过学习而得到。在训练模型时，往往是先对卷积核随机初始化，然后不断迭代得到卷积核和偏差。

我们已经知道，在实际的卷积神经网络中，我们使用的是互相关运算而不是真正的卷积运算。那么为什么这么做是可行的呢？那是因为卷积层无论使用互相关运算还是卷积运算，都不影响模型运算时的输出。

二维卷积层输出的二维数组别称为特征图。我们把在输入数组中，通过互相关运算，能影响特征图中 x 元素值的所有可能的输入值称为 x 的感受野(receptive field)。比如，上图例子中，输出特征图中第一个元素 19，它的感受野就是输入数组中所有影响 19 值的 4 个元素：0, 1, 3, 4。在卷积神经网络中，层数越深的输出特征图中单个元素的感受野会变得越广阔，因此可以捕捉输入图片上更大尺寸的特征。

在机器学习图像处理中，图像可能含有很多对我们无用的噪声数据。比如有这样一个需求：通过某一幅流行服饰的图片，自动找出款式类似的服饰。这种情况下，只需要区分衣服的样式（外形和轮廓），而衣服的颜色，商标之类的并不重要。比如女式衬衫的形状与衬衣、夹克和裤子的外观就有很大的差别。

图 10-69 卷积神经网络过滤图片噪声

如果我们通过机器学习训练出来的模型，能过滤掉这些无用的噪声数据保留有用的外形轮廓，如图 10-69 所示，那么模型就不会受到颜色、商标等相关数据的影响。而通过卷积神经网络就能轻松实现这种操作。

卷积神经网络中的卷积核参数，可以在大数据上完成训练而获取。随着卷积神经网络的训练，卷积核在图像上过滤噪声的效果会越来越好。现在只需在新的服饰数据训练集上训练从而自动找出新的卷积核即可。

卷积神经网络同样用于手写数字识别算法中。

## 10.7.5 斯特林公式

现在我们来继续研究一下阶乘。在本书 4.3.3 节中，我们曾经用到了一个用于阶乘估算的公式：

$$n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$$

这个公式其实就是斯特林公式（Stirling's approximation）。一般来说，阶乘的增长是非常快的，甚至快过指数增长（指数每次乘一个相同的数，而阶乘每次乘一个更大的数），当 n 很大的时候，n 阶乘的计算量十分大，所以一般用斯特林公式来求 n 的阶乘的近似值，而且即使在 n 很小的时候，斯特林公式的结果取值已经十分接近 n 的阶乘了。

设 N= $n! \approx \sqrt{2\pi n}\left(\frac{n}{e}\right)^n$，那么 $\lfloor lgN \rfloor + 1$ 就是 n！的位数。

实际上，更精确的斯特林公式为：

$$\lim_{n\to\infty}\frac{n!}{\sqrt{2\pi n}\left(\frac{n}{e}\right)^n}=1$$

这实际上是一个极限的证明过程，只要证明了上面的极限成立，那么上述的近似公式自然就成立了。

斯特林公式其实是从 Wallis（华里士）公式推导出来的。Wallis 公式是关于圆周率的无穷乘积的公式，公式内容如下：

$$\lim_{n\to+\infty}\left(\frac{(2n)!!}{(2n-1)!!}\right)^2\frac{1}{2n+1}=\frac{\pi}{2}$$

其中$(2n)!!=2\times4\times6\times\ldots\times(2n)$，$(2n-1)!!=1\times3\times5\times\ldots\times(2n-1)$。Wallis 公式的证明需要用到 $\sin^n x$在区间 $[0,\pi]$的积分。

令：

$$I(n)=\int_0^{\pi}sin^n x dx$$

用分部积分法，设：

$$u=sin^{n-1}x, v=cosx$$

那么：

$$\begin{aligned}I(n)&=\int_0^{\pi}\sin^n x dx=\int_0^{\pi}-u dv=-uv|_{x=0}^{x=\pi}-\int_0^{\pi}-v du\\&=-\sin^{n-1}x\cos x|_{x=0}^{x=\pi}-\int_0^{\pi}-\cos x(n-1)\sin^{n-2}x\cos x dx\\&=0-(n-1)\int_0^{\pi}-\cos^2 x\sin^{n-2}x dx, n>1\\&=(n-1)\int_0^{\pi}(1-\sin^2 x)\sin^{n-2}x dx\\&=(n-1)\int_0^{\pi}\sin^{n-2}x dx-(n-1)\int_0^{\pi}\sin^n x dx\\&=(n-1)I(n-2)-(n-1)I(n)\\&=\frac{n-1}{n}I(n-2)\end{aligned}$$

$$I(0)=\int_0^{\pi}dx=x|_0^{\pi}=\pi$$

$$I(1)=\int_0^{\pi}\sin x dx=-\cos x|_0^{\pi}=(-\cos\pi)-(-\cos 0)=-(-1)-(-1)=2$$

.....

$$I(2n)=\int_0^{\pi}\sin^{2n}x dx=\frac{2n-1}{2n}I(2n-2)=\frac{2n-1}{2n}\cdot\frac{2n-3}{2n-2}I(2n-4)$$

$$I(2n+1)=\int_0^{\pi}\sin^{2n+1}x dx=\frac{2n}{2n+1}I(2n-1)=\frac{2n}{2n+1}\cdot\frac{2n-2}{2n-1}I(2n-3)$$

由 $\sin x$的单调性推知：

$$\sin^{2k+1}x \le \sin^{2k}x \le \sin^{2k-1}x, 0 \le x \le \pi$$

由定积分的性质有：

$$I_{2k+1} < I_{2k} < I_{2k-1}$$

也就是：

$$\frac{2k!!}{(2k+1)!!} < \frac{(2k-1)!!}{2k!!}\frac{\pi}{2} < \frac{(2k-2)!!}{(2k-1)!!}$$

变形后得到：

$$1 < \frac{\frac{\pi}{2}}{\left(\frac{2k!!}{(2k-1)!!}\right)^2 \frac{1}{2k+1}} < \frac{2k+1}{2k}$$

由求极限的夹逼准则，得到

$$\lim_{k\to+\infty}\left(\frac{2^{2k}(k!)^2}{2k!}\right)^2 \frac{1}{2k+1} = \frac{\pi}{2}$$

即为 Wallis 公式得证。有了 Wallis 公式之后，现在来证明斯特林的极限公式。

令 $a_n = \frac{n!}{n^{n+\frac{1}{2}}e^{-n}}$，则：

$$\frac{a_n}{a_{n+1}} = \frac{(n+1)^{n+\frac{3}{2}}}{n^{n+\frac{1}{2}}(n+1)e} = \frac{(n+1)^{n+\frac{1}{2}}}{n^{n+\frac{1}{2}}e} = \left(1+\frac{1}{n}\right)^n \left(1+\frac{1}{n}\right)^{\frac{1}{2}} \frac{1}{e}$$

所以：

$$\frac{a_n}{a_{n+1}} > 1$$

即：

$$a_n > a_{n+1}$$

即数列$\{a_n\}$单调递减，又由积分放缩法有:

$$\ln n! > \left(n+\frac{1}{2}\right)\ln n - n$$

即:

$$n! > n^{n+\frac{1}{2}}e^{-n}$$

所以：

$$a_n > 1$$

由单调有界定理 ，$a_n$的极限存在，设这个极限为A，则：

$$A = \lim_{n\to+\infty} a_n$$

也就是：

$$A = \lim_{n\to+\infty} \frac{n!}{n^{n+\frac{1}{2}}e^{-n}}$$

利用 Wallis 公式：

$$\frac{\pi}{2} = \lim_{n\to+\infty} \frac{\left[\frac{(2n)!!}{(2n-1)!!}\right]^2}{2n+1}$$

也就是：

$$\begin{aligned}
\frac{\pi}{2} &= \lim_{n\to+\infty} \frac{\left[\frac{(2n)!!}{(2n-1)!!}\right]^2}{2n+1} \\
&= \lim_{n\to+\infty} \frac{\left[\frac{(2n)!!(2n)!!}{(2n)!}\right]^2}{2n+1} \\
&= \lim_{n\to+\infty} \frac{2^{4n}\left[\frac{(n!)^2}{(2n)!}\right]^2}{2n+1} \\
&= \lim_{n\to+\infty} \frac{2^{4n}\left[\frac{(An^{n+\frac{1}{2}}e^{-n})^2}{A(2n)^{2n+\frac{1}{2}}e^{-2n}}\right]^2}{2n+1} \\
&= \lim_{n\to+\infty} \frac{2^{4n}\left(2^{-2n-\frac{1}{2}}A\sqrt{n}\right)^2}{2n+1} \\
&= \lim_{n\to+\infty} \frac{2^{4n}A^2 2^{-4n-1} * n}{2n+1} \\
&= \frac{A^2}{4}
\end{aligned}$$

所以：

$$\frac{\pi}{2} = \frac{A^2}{4}$$

A 为未知数，解方程，有：

$$A = \sqrt{2\pi}$$

也就是：

$$\lim_{n\to+\infty} \frac{n!}{n^{n+\frac{1}{2}}e^{-n}} = \sqrt{2\pi}$$

所以：

$$\lim_{n\to+\infty} \frac{\sqrt{2\pi n}\, n^n e^{-n}}{n!} = 1$$

斯特林公式得证。

可见，当我们学习了微积分的相关理论知识之后，就可以利用微积分，来证明或推导更多的相关理论。事实上，无论是计算机领域的各种复杂算法，还是物理学中的重大理论突破，比如量子力学中的薛定谔方程，相对论中的质能方程，电磁学中的麦克斯韦方程，都是利用了微积分来推导和计算。因此，说微积分理论是一切理工科的基础，一点儿也不为过。

# 十一，概率论与数理统计基础及程序应用

概率论是一个专门研究事情发生的可能性的数学分支，但是最初概率论的起源与赌博问题有关。16 世纪，意大利的学者吉罗拉莫·卡尔达诺（Girolam Cardano，1501-1576）开始研究掷骰子等赌博中的一些简单问题。17 世纪中叶，当时的法国宫廷贵族里盛行着掷骰子游戏，游戏规则是玩家连续掷 4 次骰子，如果其中没有 6 点出现，玩家赢，如果出现一次 6 点，则庄家（相当于赌场）赢。按照这一游戏规则，从长期来看，庄家扮演赢家的角色，而玩家大部分时间是输家，因为庄家总是要靠此为生的，因此当时人们也就接受了这种现象。后来游戏规则发生了一些变化，玩家这回用 2 个骰子连续掷 24 次，如果不同时出现 2 个 6 点，玩家赢，否则庄家赢。当时人们普遍认为，2 次出现 6 点的概率是一次出现 6 点的概率的 1/6，因此 6 倍于前一种规则的次数，也既是 24 次赢或输的概率与以前是相等的。然而事实却刚好相反，从长期来看，这回庄家处于输家的状态，于是他们去请教当时的数学家帕斯卡，求助其对这种现象作出解释，这个问题的解决直接推动了概率论的产生。

随着 18、19 世纪科学的发展，人们注意到在某些生物、物理和社会现象与机会游戏之间有某种相似性，从而由机会游戏起源的概率论被应用到这些领域中，同时这也大大推动了概率论本身的发展。使概率论成为数学的一个分支的奠基人是瑞士数学家雅各布·伯努利(Jakob Bernoulli, 1654—1705，如图 11-1 所示)，他建立了概率论中第一个极限定理，即伯努利大数定律，阐明了事件的频率稳定于它的概率。

雅各布·伯努利于 1713 年出版的《猜度术》，是组合数学及概率论史的一件大事，他在这部著作中给出的伯努利数有很多应用。提出了概率论中的“伯努利定理”，这是大数定律的最早形式。伯努利家族是瑞士 17、18 世纪和牛顿、莱布尼茨齐名的一个著名的科学世家，其中的伯努利兄弟雅各布·伯努利和约翰·伯努利在科学问题上有过激烈的争论，致使双方的家庭也被卷入，以至于雅各布·伯努利死后，他的《猜度术》手稿被他的遗孀和儿子在外藏匿多年，直到 1713 年才得以出版，几乎使这部经典著作的价值受到损害。由于“大数定律”的极端重要性，1913 年 12 月彼得堡科学院曾举行庆祝大会，纪念“大数定律”诞生 200 周年。

图 11-1 雅各布·伯努利

随后棣莫弗和拉普拉斯又导出了第二个基本极限定理即中心极限定理的原始形式。拉普

拉斯在系统总结前人工作的基础上写出了《分析的概率理论》，明确给出了概率的古典定义，并在概率论中引入了更有力的分析工具，将概率论推向一个新的发展阶段。

19 世纪末，俄国数学家切比雪夫、马尔可夫、李亚普诺夫等人用分析方法建立了大数定律及中心极限定理的一般形式，科学地解释了为什么实际中遇到的许多随机变量近似服从正态分布。20 世纪初受物理学的刺激，人们开始研究随机过程。这方面柯尔莫哥洛夫、维纳、马尔可夫、辛钦、莱维及费勒等人作了杰出的贡献。

## 11.1 组合数学

组合数学首先要研究事物的安排是否存在，称存在性问题，如果这种安排存在，再研究这样的安排可以有**多少种**，称组合计数问题。

我们现在主要研究组合计数问题。这也是概率论中古典概型概率计算的基础。

### 11.1.1 加法、乘法原理

加法原理和乘法原理是组合数学中的基本原理，我们可以通过它们来统计出各种事件或者方法产生的总数。那么什么是加法原理和乘法原理呢？

**加法原理**：如果事件 A 和事件 B 是互相排斥的，而事件 A 有 p 种产生方法，事件 B 有 q 种产生方法，则事件“A **或** B”有 p ＋ q 种产生方法。

**乘法原理**：如果事件 A 与事件 B 相互独立，且事件 A 有 p 种产生方式，事件 B 有 q 种产生方式，则事件“A **与** B”有 pq 种产生方式。

现在我们通过一些练习来学习和理解加法原理和乘法原理的实际应用。

练习 1：假设北京到南京有 5 趟列车，10 班飞机，1 班客车，问旅客从北京到南京共有多少种选择方法？北京到上海必经南京，南京到上海有 10 趟列车，5 班飞机，2 班客车。那么北京到上海多少种选择方法？

解：北京到南京，设坐列车为 A 事件，有 5 种方法；坐飞机为 B 事件，有 10 种方法；坐客车为 C 事件，有 1 种方法。A，B，C 三个事件是互斥的，从北京到南京可以采用 A，B，C 三种方法中的一种，是“或”的关系，因此由加法原理有：5+10+1=16 种方法。

北京到上海必经南京，因此设北京到南京为事件 P，南京到上海为事件 Q；那么从北京到上海，需要先从北京到南京，再从南京到上海，即为事件 P 和事件 Q，是“与”的关系。由前面分析可知，事件 P 有 16 种方法，同样事件 Q 有 10+5+2=17 种方法。由乘法原理可知，从北京到上海共有：16*17=272 种方法。

练习 2：从集合 S={1，2，3，4，5，6}中选取 4 个不同的数字构成不同的四位数，问有多少种选择方法？

解：要构成四位数，只需要每次从集合中抽取一个不同的数字，连抽 4 次即可。每次抽取的可能方法种数如下：

第 1 次：6；第 2 次：5；第 3 次：4；第 4 次：3。根据乘法原来，共有 6*5*4*3=360 种方法。

练习 3：含 n 个元素的集合，有多少个子集？

解：首先我们来看 n 个元素的集合子集的构造方法。在构造 n 个元素的集合的子集时，对于每个元素有 2 种可能，即要么放入这个子集里，要么不放入这个子集里。因此，由乘法

原理共有 2*2*2…*2 种可能，即 $2^n$ 种可能。

## 11.1.2 排列与组合问题

排列与组合是组合学最基本的概念。所谓排列，就是指从给定个数的元素中取出指定个数的元素进行排序。组合则是指从给定个数的元素中仅仅取出指定个数的元素，不考虑排序。

排列组合的中心问题是研究给定要求的排列和组合可能出现的情况总数。 排列组合与古典概率论关系密切。现在我们依次来研究排列与组合问题的计算方法。

**1.无重复（n 个不同元素）排列问题**

从 n 个不同元素中有序选取 r 个元素排成一排称为 n 元集的一个 r-排列，记为：

P(n, r)

计算方法如下：

$$P(n,r) \text{ 或 } P_n^r = \frac{n!}{(n-r)!}$$

**2.无重复组合问题**

从 n 个不同元素中不考虑次序取出 r 个元素，称为 n 元集的一个 r-组合，记为：

C(n, r)

计算方法如下：

$$C(n,r) = \frac{P(n,\ r)}{r!} = \frac{n!}{r!(n-r)!} \qquad C(n,r) = C(n,n-r)$$

通过公式的定义，很容易算出：C(n, 0)=C(n, n)=1

**3.圆排列**

从 n 个有标志的球中取出 r 个放到一个位置无标号的圆周的 r 个位置上，这种排列我们称作 n 元集的 r-圆排列，记作 O(n，r )，计算公式如下：

$$O(n,r) = \frac{P(n,\ r)}{r} = \frac{n!}{r(n-r)!} \qquad O(n,n) = \frac{P(n,\ n)}{n} = \frac{n!}{n} = (n-1$$

**4.多重集**

设多重集 A= $\{n_1 \cdot a_1,\ n_2 \cdot a_2, \cdots,\ n_k \cdot a_k\}$，其中有 $n_1$ 个 $a_1$，$n_2$ 个 $a_2$，…，$n_k$ 个 $a_k$，且 $n = n_1 + n_2 + \cdots + n_k$ ，则多重集 A 中所有 $a_1$，$a_2$，…，$a_k$ 的全排列数等于：

$$\frac{n!}{n_1! \cdot n_2! \cdot \cdots \cdot n_k!}$$

练习 4：从集合 S={1，2，3，4，5，6}中选取 4 个不同的数字构成不同的四位数，问有多少种选择方法？

解：这个问题属于无重复（n 个不同元素）排列问题，从包含 6 个元素的集合中选取 4 个元素排列成一排，即 P(6, 4)=6*5*4*3=360 种排列方法。

练习 5：若从{1，2，3，4，5，6}中选取 4 个不同的数字只组成无序组，问这样的不

同组数有多少种？

解：这个问题属于无重复组合问题：从 n 个不同元素中不考虑次序取出 r 个元素，称为 n 元集的一个 r -组合，即 C(6, 4)=6!/(4!*(6-4)!)=6*5*4*3*2*1/(4*3*2*1*2*1)=15 种组合方法。

练习 6：4 个女同志和 6 个男同志一同就餐，如果没有两个女同志相邻，问有多少种就座方式？

解：男女同志就坐方式可以做如下安排：

首先 6 个男同志就坐，那么这是一个圆排列，方法有：O(6, 6)种。

然后 6 个男同志就坐后，形成 6 个空隙，由于没有 2 个女同志相邻，那么 4 个女同志可以安排在这 6 个空隙入座即符合要求，这属于一个无重复（n 个不同元素）排列问题，方法有：P(6, 4)种。

因此总共的就坐方式，由乘法原理有：O(6, 6)*P(6, 4)=6!/6*6*5*4*3=43200 种方法。

练习 7：各种面值的人民币各一张(枚)，可以组成多少种不同的面值？

解：人民币由 100 元，50 元，20 元，10 元，5 元，2 元和 1 元，5 角，2 角和 1 角，5 分，2 分和 1 分共 13 种组成。

因此从这 13 种组成中任取 1 种，2 种，3 种，…，13 种，共有：$C(13, 1)+\cdots+C(13, 13)=2^{13}-1$ 种不同的面值。

注意：$C(n, 0)+C(n, 1)+\cdots+C(n, n)=2^n$。结合前面集合子集总数的题目可以证明。

练习 8：现有 n 面彩旗，$n_1$ 面红色，$n_2$ 面蓝色，$n_3$ 面黄色，这里 $n_1+n_2+n_3=n$，把它们悬挂在商场门口，问有多少种悬挂方法？

解：这是一个多重集的问题。根据多重集的计算公式有 n 面彩旗的悬挂方法总共有：$n!/(n_1!n_2!n_3!)$种。

### 11.1.3 容斥原理

在我们需要统计某种对象的总数时，必须遵循的原则就是既没有重复，也没有遗漏。但这些对象和另外一些对象往往会有重叠部分，为了使重叠部分不被重复计算，可以先不考虑重叠的情况，把所有对象的数目先计算出来，然后再把计数时重复计算的数目排斥出去，使得计算的结果既无遗漏又无重复，这种计数的方法称为容斥原理。容斥原理可表述为：

要计算几个集合并集的大小，我们要先将所有单个集合的大小计算出来，然后减去所有两个集合相交的部分，再加回所有三个集合相交的部分，再减去所有四个集合相交的部分，...，依此类推，一直计算到所有集合相交的部分。特别地：

对于只有 2 个集合 $A_1$ 和 $A_2$ 的容斥原理公式如下：

$$|A_1 \cup A_2| = |A_1| + |A_2| - |A_1 \cap A_2|$$

对于只有 3 个集合 $A_1$，$A_2$ 和 $A_3$ 的容斥原理公式如下：

$$|A_1 \cup A_2 \cup A_3| = |A_1| + |A_2| + |A_3| - |A_1 \cap A_2| - |A_2 \cap A_3| - |A_1 \cap A_3| + |A_1 \cap A_2 \cap A_3|$$

对于 n 个集合 $A_1$，$A_2$，...，$A_n$ 的容斥原理公式如下：

$$|A_1 \cup A_2 \cup \cdots \cup A_n| = \sum_{i=1}^{n}|A_i| - \sum_{i=1}^{n}\sum_{j>i}|A_i \cap A_j| + \sum_{i=1}^{n}\sum_{j>i}\sum_{k>j}|A_i \cap A_j \cap A_k| - \cdots + (-1)^{n-1}|A_1 \cap A_2 \cap \cdots \cap A_n|$$

现在我们来利用容斥原理公式解决一些实际问题。

练习 9：求从 1 到 60 的 60 个数中能被 2 或 3 或 7 整除的数的个数。

解：能被 2 整除的数构成的集合 $A_1$ 元素个数共有：60/2；能被 3 整除的数构成的集合 $A_2$ 元素个数共有：60/3；能被 7 整除的数构成的集合 $A_3$ 元素个数共有：60/7；能同时被 2&3 整除的数 $A_1 \cap A_2$ 元素个数共有：60/(2*3)，…，能同时被 2&3&7 整除的数 $A_1 \cap A_2 \cap A_3$ 元素个数共有：60/(2*3*7)。

因此，根据三个集合的容斥原理公式有：

$|A_1 \cup A_2 \cup A_3|=|A_1|+|A_2|+|A_3|-|A_1 \cap A_2|-|A_1 \cap A_3|-|A_2 \cap A_3|+|A_1 \cap A_2 \cap A_3|=30+20+8-10-4-2+1=43$ 种。

## 11.1.4 鸽巢原理

鸽巢原理又被称为抽屉原理，在 1834 年由德国数学家狄利克雷（Dirichlet）提出，是组合数学中一个重要的原理。

鸽巢原理指出：将 n+1 个物体放入 n 个盒子中，那么至少有一个盒子包含两个或更多的物体。

应用鸽巢原理需要搞清楚以下问题：什么是“鸽子”？什么是“巢”？鸽子与巢各有多少？其中的关键往往在于巧妙的构造鸽巢。

练习 10：多少个人中至少有两个人属同一属相？

解：把 12 属相看成鸽巢，那么只需要把 13 个人放入这 12 个鸽巢，就至少有 2 个人在同一个鸽巢里。因此 13 个人中至少有 2 个人属同一属相。

练习 11：问从 n 对已婚夫妇中至少选取多少人才能保证至少选上一对夫妇？

解：可以把 n 对夫妇看作 n 个鸽巢，这 2n 个人中每个人必属于某个鸽巢。选 n+1 个人，放入 n 个鸽巢，至少保证一个鸽巢有 2 个人，即一对夫妻。

练习 12：设在正六边形内有 7 个点，则至少有两个点的距离小于或等于正六边形的边长。

解：将正六边形的中心和 6 个顶点连接形成 6 个正三角形，并看成是鸽巢，如图 11-2 所示。将 7 个点放入 6 个正三角形中，至少有 2 个点在同一个三角形内。于是这两个点的距离必然小于或等于正六边形的边长。

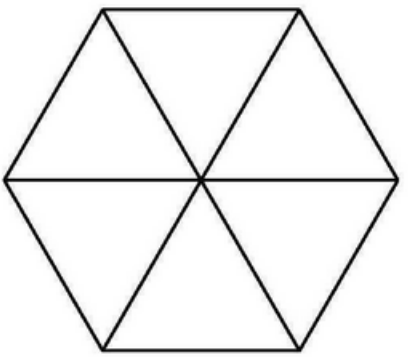


图 11-2 正六边形的中心和 6 个顶点连接形成 6 个正三角形

# 11.2 随机事件的概率

法国数学家拉普拉斯(Laplace)曾经说过:“生活中最重要的问题，其中绝大多数在实质上只是概率的问题。”在现实生活中，人们最熟悉的“抛硬币”，“抓阄”，“抽签”等等活动可以看作同一类试验，这些试验有以下三个特点：

1．可以在相同的条件下重复进行；

2．每次试验的可能结果不止一个，并且能事先明确试验的所有可能结果（比如抛硬币结果中的“正面”和“反面”）；

3．进行一次试验之前不能确定哪一个结果会出现。

具有上述三个特点的试验，称为“随机试验”，通常用 E 表示。随机试验中，所有可能结果组成的集合是已知的，称为随机试验的样本空间，记为Ω。样本空间Ω的元素，即随机试验的每个结果，称为样本点，记为ω。

一般地，在条件 S 下一定会发生的事件，叫做相对于条件 S 的**必然事件**（certain event），简称必然事件；在条件S下一定不会发生的事件，叫做相对于条件S的**不可能事件**（impossible event），简称不可能事件；必然事件和不可能事件统称为相对于条件 S 的确定事件，简称**确定事件**。在条件 S 下可能发生也可能不发生的事件，叫做相对于条件 S 的**随机事件**（random event），简称随机事件。

在相同的条件 S 下重复 n 次试验，观察某一事件 A 是否出现，称 n 次试验中事件 A 出现的次数 $n_A$ 为事件 A 出现的频数，称事件 A 出现的比例 $f_n(A)=n_A/n$ 为事件 A 出现的频率。

比如，一次抛硬币的随机实验，共进行了 100 次，设事件 A 为抛出硬币正面朝上，该事件发生了 52 次，那么 52 即为事件 A 的频数，0.52 是事件 A 的频率。

伯努利大数定律告诉我们：对于给定的随机事件 A，如果随着试验的次数的增加，事件 A 发生的频率 $f_n(A)$ 稳定在某个常数上，把这个常数记作 P(A)，称为事件 A 的概率(probability)，简称为 A 的概率。

比如抛掷硬币，如图 11-3 所示，正面朝上的概率为 0.5，即 P(正面朝上)=0.5。

图 11-3 硬币的正反面

任何事件的概率是 0~1 之间的一个数，它度量该事件发生的可能性。小概率（接近 0）事件很少发生，而大概率（接近 1）事件则经常发生。

由于事件的频数总是小于或者等于试验的次数，所以频率在 0~1 之间，从而任何事件的概率在 0~1 之间，即：$0\leqslant P(A)\leqslant 1$，其中必然事件的概率为 1，不可能事件的概率为 0。

如果事件 A 与事件 B 互斥（比如投骰子的 1 点和 2 点），则：$P(A\cup B)=P(A)+P(B)$。特别地，如果 A 与 B 事件互为对立事件（比如抛掷硬币的正面和反面事件就是对立事件），则 $A\cup B$ 为必然事件，$P(A\cup B)=1$，因此 $P(A)=1-P(B)$。

练习 13：如果买彩票中奖的概率为 1/1000，那么买 1000 张彩票中奖的概率为多少？

解：买 1000 张彩票至少有一张中奖的概率=1-买 1000 张彩票都不中奖的概率。而买 1000 张都不中奖的概率为：$(1-1/1000)^{1000}\approx 0.3677$，所以：

买 1000 张彩票中奖的概率≈1-0.3677=0.6323

通过练习 13，我们可以知道，有时候直接求某个事件的概率可能不好求，那么可以先求它的对立事件的概率，然后再用 1 减去对立事件的概率，就是该事件的概率。

练习 14：均匀骰子投 6 次，连续出现 1 点的概率是多少？

解：每次投骰子出现 1 点的概率为 1/6，那么连续投 6 次，都出现 1 点的概率为：

$$1/6*1/6*1/6*1/6*1/6*1/6=(1/6)^6\approx0.000021433$$

练习 15：17 世纪中叶，当时的法国宫廷贵族里盛行着掷骰子游戏，游戏规则是玩家连续掷 4 次骰子，如果其中没有 6 点出现，玩家赢，如果出现一次 6 点，则庄家（相当于赌场）赢。按照这一游戏规则，从长期来看谁是赢家？为了使游戏更刺激，游戏规则发生了些许变化，玩家这回用 2 个骰子连续掷 24 次，不同时出现 2 个 6 点，玩家赢，否则庄家赢，修改规则后，从长期看，谁是赢家？

解：修改规则之前，投一次骰子，不出现 6 点的概率为 5/6，连续 4 次都不出现 6 点的概率为：

$$（5/6）^4\approx0.48225$$

至少有一次 6 点出现的概率为：

$$1-0.48225\approx0.51775$$

因此，从长远看，至少有一次 6 点出现的概率大于连续 4 次都不出现 6 点的概率，庄家是赢家。

修改规则之后，2 个骰子一起投一次，不同时出现 2 个 6 点的概率为：

1-同时出现 2 个 6 点的概率=1-(1/6)(1/6)=35/36

因此，连续投 24 次，不同时出现 2 个 6 点的概率为：

$$(35/36)^{24}\approx0.5086$$

至少出现一次 2 个 6 点的概率为：

$$1-(35/36)^{24}\approx0.4914$$

因此，修改规则之后，不同时出现 2 个 6 点的概率大于至少出现一次 2 个 6 点的概率，玩家变成了赢家。

## 11.2.1 古典概型

抛硬币中“正面朝上”或“反面朝上”，抛骰子中“1 点”，“2 点”，...，“6 点”这些随机事件叫做**基本事件**。任何两个基本事件是互斥的（互不包含）；任何事件都可以表示成基本事件的和，具有这两个特点的概率模型称为古典概率模型，简称**古典概型**。

对于古典概型，任何事件的概率为：

**P(A)=(A 包含的基本事件的个数)/(基本事件的总数)**

练习 16：某种饮料每箱装 12 听，如果其中有 2 听不合格，问质检人员从中随机抽出 2 听，检测不合格产品的概率有多大？

解：先计算总的 2 听拿法：12*11/2=66

再计算有不合格的拿法：1 听不合格的拿法（先从 2 听不合格中拿一听，共有 2 种拿法，再从合格的 10 听中拿一听，共有 10 种拿法）：2*10；2 听不合格的拿法：1。

因此补合格产品的概率为：(2*10+1)/66≈0.318。

练习 17：遗传规律计算：

生男生女概率计算。性别由X和Y染色体决定。XX为女,XY为男。父亲和母亲的染色体分别为X,Y和X,X。那么共有:XX,XX,XY,XY共4中组合,其中女性组合为:XX,XX;男性组合为:XY,XY。因此生男生女的概率为:

男:2/4=50%

女:2/4=50%

## 11.2.2 几何概型

如果**试验的结果可能有无穷多**的情况(比如往一块区域内扔石子),这时就不能用古典概型来计算事件发生的概率了。在特定情形下,可以用几何概型来计算事件发生的概率。

如果每个事件发生的概率只与构成该事件区域的长度(面积或体积)成比例,则称这样的概率模型为几何概率模型,简称为**几何概型**。

在几何概型中,事件A的概率计算公式如下:

**P(A)=(构成事件A的区域长度(面积或体积))/(试验的全部结果所构成的区域长度(面积或体积))**

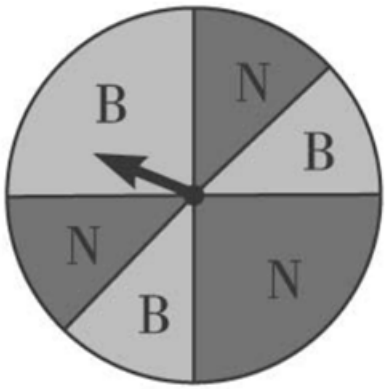



图11-4 几何概型

如图11-4所示,设事件A为指针停留在B区域,设事件A的概率为P(A),B区域的面积为$S_B$,整个面积为$S_{\square}$,那么有:

$$P(A)=S_B/S_{\square}$$

练习18:某人午觉醒来,发现表停了,他打开收音机,想听电台报时,求他等待的时间不多于10分钟的概率。

图11-5 钟表刻度

解:如图11-5所示,睡午觉可以在60分钟内的任何时刻醒来,而等待电台报时不超过10分钟,那么必须在50到60分钟这个10分钟的时间段醒来。根据几何概型,等待的时间不多于10分钟的概率为:

10/60≈0.167

## 11.2.3 条件概率

条件概率是指当事件 A 已发生的条件下事件 B 发生的概率，记为 P(B|A)。条件概率的计算方法为：

$$P(B|A)=P(AB)/P(A)$$

其中 P(AB)是指事件 A 发生且事件 B 发生的概率，A，B 之间发生没有先后顺序。P(B|A)在事件 A 发生的基础上，事件 B 发生的概率。事件 A 先发生，事件 B 再发生。由于规定了事件 A 先发生，这样一来样本空间就被缩小到了事件 A。

例题：将一枚硬币抛掷两次，设事件 A 为“至少有一次为正面”，事件 B 为“两次掷出同一面”，求 P(B|A)（即在“至少有一次为正面”的条件下，“两次掷出同一面”的概率）。

分析：此问题属于古典概型问题。试验的样本空间 S={正正，正反，反正，反反}，A={正正，正反，反正}，B={正正，反反}。因此有：

$$P(A)=3/4,\ P(AB)=(A\cap B)=1/4,\ P(B|A)=P(AB)/P(A)=(1/4)/(3/4)=1/3$$

事实上，设试验的基本事件总数为 n，A 所包含的基本事件数为 m，AB 所包含的基本事件数为 k，那么有：

$$P(B|A)=k/m=(k/n)/(m/n)=P(AB)/P(A)$$

**1.乘法定理**

由条件概率的定义，可以得到乘法定理如下：

$$P(AB)=P(B|A)*P(A)$$

P(A|B)是原本在事件 A 发生的基础上，事件 B 发生的概率。此时事件 A 发生的概率 P(A)=1。现在要在整个样本空间上求事件 A、B 同时发生的概率。由于此时的样本空间不再为事件 A，而事件 A 发生也是有概率的，所以要乘上 P(A)。乘法定理可以推广到多个事件的积事件的情况，比如：

$$P(ABC)=P(C|AB)P(B|A)P(A)$$

练习 19：设袋中装有 r 只红球，s 只白球，每次从袋中任取一只球，观察其颜色后放回，并再加入 a 只与所取颜色同色的球。若连续取 3 次，求第 1，2 次都取到红球第 3 次取到白球的概率。

分析：设 $A_i(i=1,2,3)$为取到红球，$\sim A_i$ 表示取到白球，则根据乘法定理有：

$$P(A_1A_2\sim A_3))=P(\sim A_3|A_1A_2)P(A_2|A_1)P(A_1)=s/(r+s+2a)*(r+a)/(r+s+a)*r/(r+s)$$

其中：

- $P(\sim A_3|A_1A_2)=s/(r+s+2a)$（第 1 次和第 2 次取到红球，所以共有 r+2a 个红球，s 个白球）；
- $P(A_2|A_1)=(r+a)/(r+s+a)$（第 1 次抽到红球，所以共有 r+a 个红球，s 个白球）；
- $P(A_1)=r/(r+s)$。

**2.全概率公式**

设试验 E 的样本空间为 S，A 为 E 的事件，$B_1$，$B_2$，$B_3$，...，$B_n$ 为 S 的一个划分，且 $P(B_i)>0(i=1,2,...,n)$，则：

$$P(A)=P(A|B_1)P(B_1)+P(A|B_2)P(B_2)...+P(A|B_n)P(B_n)$$

所谓 $B_1$，$B_2$，$B_3$，...，$B_n$ 为 S 的一个划分是指：

$B_i\cap B_j=\varnothing\ (i\neq j)$

$B_1 \cup B_2 \cup ... = S$

即 $B_i$ 两两互斥并且全部事件 $B_i$ 的和=整个样本空间 S。

全概率公式的意义就在于：当 P(A)不好求时，先将 P(A)碎片化，最后构成一个完整的 P(A)。因为 $A=AS=AB_1+AB_2+...+AB_n$

于是 $P(A)=P(AS)=P(AB_1+AB_2+...+AB_n)$。

又因为事件 $B_i$ 两两互斥，所以事件 $AB_i$ 也两两互斥，于是就有了 $P(AB_1+AB_2+...+AB_n)=P(AB_1)+P(AB_2)+...+P(AB_n)$

由前面的乘法公式有：

$P(AB_1)+P(AB_2)+...=P(A|B_1)P(B_1)+P(A|B_2)P(B_2)+... +P(A|B_n)P(B_n)$

最终：$P(A)=P(A|B_1)P(B_1)+P(A|B_2)P(B_2)+... +P(A|B_n)P(B_n)$

全概率公式计算事件概率，如果把事件 A 看成是“结果”，将事件 B 看成是导致结果发生的“原因”之一，那么全概率公式就是一个“原因推结果”的过程。

**3.贝叶斯公式**

由条件概率定义以及全概率公式，即可得到贝叶斯公式：

$$P(B_i \mid A) = \frac{P(B_iA)}{P(A)} = \frac{P(A \mid B_i)P(B_i)}{\sum_{j=1}^{n} P(A \mid B_j)P(B_j)}$$

贝叶斯公式的意义是：已知事件 A 这个“结果”已经发生了，反过来计算造成事件 A 发生的原因事件 $B_1$、事件 $B_2$...，$B_n$ 的概率是多少，即“因果溯源”。

练习 20：某电子设备制造厂所用的元件是由三家元件制造厂提供的。根据以往的记录有表 11-1 所示的数据：

| 元件制造厂 | 次品率 | 提供元件的份额 |
|---|---|---|
| 1 | 0.02 | 0.15 |
| 2 | 0.01 | 0.80 |
| 3 | 0.03 | 0.05 |

表 11-1 制造厂与次品率

这三家产品在仓库中是均匀混合无区别标志。那么：

1，在仓库中随机的取一件产品，求它是次品的概率。

2，若已知取到的是次品，求此次品出自第 1 家的概率。

分析：设 A 为取到次品的概率；$B_i(i=1,2,3)$为“取到次品为第 i 家提供的”。那么由题目中提供的表格数据有：

$$P(B_1)=0.15, P(B_2)=0.80, P(B_3)=0.05$$

$B_1$，$B_2$，$B_3$ 为样本空间的一个划分，且：

$$P(A|B_1)=0.02, P(A|B_2)=0.01, P(A|B_3)=0.03。$$

那么，由全概率公式：

$$P(A)=P(A|B_1)P(B_1)+P(A|B_2)P(B_2)+P(A|B_3)P(B_3)=0.02*0.15+0.01*0.8+0.03*0.05=0.0125$$

由贝叶斯公式：

$$P(B_1|A)=P(B_1A)/P(A)=P(A|B_1)P(B_1)/P(A)=0.02*0.15/0.0125=0.24$$

练习 21：根据美国的一份资料报导，在美国总的来说患肺癌的概率约为 0.1%，在人群中有 20%是吸烟者，他们换肺癌的概率约为 0.4%，求不吸烟者患肺癌的概率是多少？

分析：设 A 为“患肺癌”事件，B 为“吸烟”事件。那么根据题意有：P(A)=0.001，P(B)=0.2，P(A|B)=0.004，需求得 P(A|~B)。根据全概率公式：

$$P(A)=P(A|B)*P(B)+P(A|\sim B)*P(\sim B)$$

把已知条件代入有：

$$0.001=0.004*0.2+P(A|\sim B)*0.8$$

所以有：

$$P(A|\sim B)=0.00025$$

## 11.3 随机变量

如果将随机试验中的每一个结果，即将样本空间 S 中的每个元素 e 与实数 x 对应起来，即 S={e}，X=X(e)是定义在样本空间 S 上的实值（即值域为实数值）和单值函数（即一一对应），则称 X 为随机变量。

比如，连续将一枚硬币抛掷两次，观察硬币出现正面的次数并记为 X，那么 X 就是一个随机变量。试验的样本空间为：

S={正正，正反，反正，反反}

对其中的每一个样本点 e：

X=X(e)= { 2, e=正正 ； 1, e=正反，反正 ； 0, e=反反

其中，X=X(e)是定义在样本空间 S 上的一个实值单值函数，定义域为样本空间 S，值域为{0,1,2}。如果样本点 e 本身就是一个数（比如一个单位一天迟到人数，一个灯泡的使用寿命，某个地区一年的交通事故次数等），那么令 X=X(e)=e，那么 X 就是一个随机变量。

一般用大写字母，比如 X,Y,Z 等来表示随机变量，而用小写字母 x，y，z 等表示随机变量的实数取值。

随机变量的取值随结果而定，在试验之前不能预知，且它的取值有一定的概率。比如，对于连续两次抛掷硬币的随机试验中，观察硬币出现正面的次数的随机变量 X，那么 X≤1 的概率为：

$$P\{X\leqslant 1\}=P\{\text{正反，反正}\}=1/2$$

## 11.4 随机变量分布

一个随机变量，如果它的取值是有限个或者可列的无限多个，那么它就是离散型随机变量。比如统计抛掷硬币两次其中抛出正面的次数，它的取值只可能是 0,1,2。又比如某个地区一个月的交通事故次数，也是离散型随机变量。而如果用 T 表示灯泡的寿命，那么它可以的取值范围就是一个区间而无法一一列出，因此是非离散的随机变量。

## 11.4.1 离散随机变量的分布

设离散随机变量 X 所有可能的取值为 $x_k(k=1, 2, 3, \ldots)$，X 取各种可能值的概率为：

$$P\{X=x_k\}=p_k, k=1, 2, 3, \ldots$$

上式称为离散型随机变量 X 的分布律，也可以表示为如下的表格形式：

| $X$ | $x_1$ | $x_2$ | $\cdots$ | $x_n$ | $\cdots$ |
|---|---|---|---|---|---|
| $p_k$ | $p_1$ | $p_2$ | $\cdots$ | $p_n$ | $\cdots$ |

很显然，$p_k$ 满足如下条件：

- $p_k \geqslant 0$
- $\sum_{k=1}^{\infty} p_k = 1$

离散型随机变量，有 3 种重要的分布，分别是：0-1 分布，二项分布和泊松分布。下面分别给予介绍。

**1. 0-1 分布**

如果随机变量 X 只可能取 0 和 1 两个值，即：

$P\{x=k\}=p^k(1-p)^{1-k}, k=0, 1 (0<p<1)$

那么称 X 服从以 p 为参数的 0-1 分布。0-1 分布也可以写成：

| $X$ | 0 | 1 |
|---|---|---|
| $p_k$ | $1-p$ | $p$ |

如果一个随机试验，它的样本空间只包含两个元素（比如“抛硬币”，比如新生儿性别统计等等），即 $S=\{e_1, e_2\}$，那么总能在 S 上定义一个服从 0-1 分布的随机变量：

$$X=X(e)=\begin{cases}1, e=e_2\\ 0, e=e_1\end{cases}$$

**2. 二项分布**

如果一个随机试验 E，其结果只有两个可能：A 和 $\bar{A}$，则称 E 为伯努利（Bernoulli）试验，设 $P(A)=p(0<p<1)$，则 $P(\bar{A})=1-p$。如果将 E 独立（即各次试验结果互不影响）而又重复（即 P(A)=p 不变）地进行 n 次，则称这一组重复的独立试验为 n 重伯努利试验。

比如将硬币连抛 n 次或者将骰子连抛 n 次，都是 n 重伯努利试验。

如果用 X 表示 n 重伯努利试验中事件 A 发生的次数，则 X 是一个随机变量。如果事件 A 在 n 次试验中有 $k(0\leqslant k\leqslant n)$次发生，那么有 n-k 次试验中 A 不发生。比如，在前 k 次试验中 A 发生，后 n-k 次试验中 A 不发生，那么这种情况下的概率为：

$$\underbrace{p \cdot p \cdot \cdots \cdot p}_{k\text{个}} \cdot \underbrace{(1-p) \cdot (1-p) \cdot \cdots \cdot (1-p)}_{n-k\text{个}} = p^k(1-p)^{n-k}$$

这只是其中一种可能，根据组合数学原理，总共有$\binom{n}{k}$种排列，而且这些排列是两两互不相容的，所以 n 次试验中 A 发生的 k 次的概率为：$\binom{n}{k}p^k(1-p)^{n-k}$，记 q=1-p，那么有：

$$P\{X=k\} = \binom{n}{k}p^k q^{n-k}, k = 0,1,2,\cdots,n$$

我们称随机变量 X 服从参数为 n，p 的二项分布，并记为 X~b(n,p)。显然，0-1 分布是二项分布在 n=1 时候的特例。

练习 22：如果一个人每次射击的命中率为 0.02，独立射击 400 次，试求至少击中 2 次的概率。

分析：一次射击试验，结果要么命中，要么不命中，那么 n 次独立试验就是伯努利试验，设击中次数为 X，那么 X 服从二项分布，即 X~b(400,0.02)。X 的分布律为：

$$P\{X=k\} = \binom{400}{k}(0.02)^k(0.98)^{400-k}, k = 0,1,\cdots,400$$

那么击中次数至少 2 次的概率为：

$P\{X\geqslant 2\}=1-P\{X=0\}-P\{X=1\}=1-0.98^{400}-400(0.02)(0.98)^{399}=0.9972$

**3. 泊松分布**

如果随机变量 X 的所有可能取值为 0,1,2，...，而取各个值的概率为：

$$P\{X=k\} = \frac{\lambda^k e^{-\lambda}}{k!}, k = 0,1,2,\cdots,$$

其中 λ>0 是常数，则称 X 服从参数为 λ 的泊松分布，记为 X~π(λ)。

在实际应用中，一本书一页中的印刷错误数，某地区在一天内邮递遗失的信件数，某医院一天内的急诊病人数，某地区一个时间段内发生交通事故的次数等都服从泊松分布。

事实上，泊松分布是二项分布 n 趋于无穷时的极限，其中 λ=np。即当 n 很大，p 很小时，有：

$$\binom{n}{k}p^k(1-p)^{n-k} \approx \frac{\lambda^k e^{-\lambda}}{k!} \quad (\text{其中 } \lambda = np).$$

一般地，当 n≥20，p≤0.05 时用泊松分布作为二项分布的近似值效果最佳。

**4. 分布函数**

设 X 是一个随机变量，x 是任意实数，函数：

$$F(x)=P\{X\leqslant x\}, -\infty<x<\infty$$

称为 X 的分布函数。对于任意的实数 $x_1$，$x_2(x_1<x_2)$，有：

$$P\{x_1<X\leqslant x_2\}=P\{X\leqslant x_2\}-P\{X\leqslant x_1\}=F(x_2)-F(x_1)$$

分布函数 F(x)具有以下基本性质：

1．F(x)是一个不减函数

2．0≤F(x) ≤1

$$F(-\infty) = \lim_{x\to-\infty} F(x) = 0 \quad F(\infty) = \lim_{x\to\infty} F(x) = 1$$

3．F(x)是一个右连续函数，即 F(x+0)=F(x)

随机变量的分布函数就是一个普通的函数，通过它，我们就能够用数学分析的方法来研究随机变量。

一般地，如果离散型随机变量 X 的分布律为：

$$P\{X=x_k\}=p_k,\ k=1,2,3,\cdots$$

那么，X 的分布函数为：

$$F(x)=P\{X\leqslant x\}=\sum_{x_k<x}P\{X=x_k\}$$

练习 23：一个靶子的半径为 2m 的圆盘，设击中靶上任一同心圆盘上的点的概率与该圆盘的面积成正比，并设射击都能中靶，以 X 表示弹着点与圆心的距离。试求随机变量 X 的分布函数。

分析：

当 x<0 时，事件{X≤x}是不可能事件，即：

$$F(x)=P\{X\leqslant x\}=0 \quad x<0$$

若 0≤x≤2，根据题意有：

$$P\{0\leqslant X\leqslant x\}=kx^2，k\text{ 为常数}$$

根据题意，射击都能全中，所以当 x=2 时，概率为 1，即：

$$P\{0\leqslant X\leqslant 2\}=k2^2=4k=1$$

那么 k=1/4，所以：

$$P\{0\leqslant X\leqslant x\}=(1/4)x^2$$

所以当 x≤2 时：

$$F(x)=P\{X\leqslant x\}=P\{X<0\}+P\{0<X\leqslant x\}=0+(1/4)x^2=(1/4)x^2$$

当 x>2 时：

$$F(x)=P\{X\leqslant x\}=1$$

所以，X 的分布函数 F(x)为：

$$F(x)=\begin{cases}0, & x<0,\\ \dfrac{x^2}{4}, & 0\leqslant x<2,\\ 1, & x\geqslant 2.\end{cases}$$

## 11.4.2 连续随机变量的分布

对于非离散型随机变量，比如灯泡的寿命，它的取值是不能列举，而是在一个区间里。如果对于随机变量 X，它的分布函数 F(x)，存在非负可积函数 f(x)，对于任意实数 x 有：

$$F(x)=\int_{-\infty}^{x}f(t)dt$$

则称 X 为连续随机变量，f(x)为 X 的概率密度函数，简称概率密度。

概率密度 f(x)有如下性质：

- f(x) ≥0；

- $\int_{-\infty}^{+\infty} f(x)dx = 1$
- 对于任意的 x1，x2，(x1≤x2)，那么：

$$P\{x_1 < X \leqslant x_2\} = F(x_2) - F(x_1) = \int_{x_1}^{x_2} f(x)\mathrm{d}x$$

- 若 f(x)在点 x 连续，则有 F'(x)=f(x)

注意，对于连续型随机变量 X，它取任一指定实数值 a 的概率均为 0，即 P{X=a}=0。因此，在计算随机变量落在某一区间的概率时，可以不用区分该区间的开闭。即：

$$P\{a<X\leqslant b\}=P\{a\leqslant X\leqslant b\}=P\{a<X<b\}$$

连续型随机变量有三种重要的分布类型：均匀分布，指数分布和正态分布。

**1. 均匀分布**

若连续型随机变量 X 具有概率密度：

$$f(x)=\begin{cases}\dfrac{1}{b-a}, & a<x<b,\\ 0, & \text{其他},\end{cases}$$

那么 X 在区间[a,b]上服从均匀分布，记为 X~U(a,b)。对概率密度求积分，可以得到它的分布函数为：

$$F(x)=\begin{cases}0, & x<a,\\ \dfrac{x-a}{b-a}, & a\leqslant x<b,\\ 1, & x\geqslant b.\end{cases}$$

练习 24：设电阻值 R 是一个随机变量，均匀分布在 900Ω~1100Ω。求 R 的概率密度及 R 落在 950Ω~1050Ω的概率。

分析：R 服从[900,1100]区间上的均匀分布，因此概率密度为：

$$f(r)=\begin{cases}\dfrac{1}{1\ 100-900}, & 900<r<1100\\ 0, & \text{其他}.\end{cases}$$

R 落在 950Ω~1050Ω的概率：

$$P\{950\leqslant R\leqslant 1050\}=\int_{950}^{1\,050}\frac{1}{200}\mathrm{d}r=0.5$$

**2. 指数分布**

如果随机变量 X 的概率密度为：

$$f(x)=\begin{cases}\dfrac{1}{\theta}\mathrm{e}^{-x/\theta}, x>0\\ 0, \quad \text{其他}\end{cases}$$

其中 θ>0 为常数，那么称 X 服从参数为 θ 的指数分布。对指数分布的概率密度求积分，则可得指数分布的随机变量的分布函数为：

$$F(x)=\begin{cases}1-e^{-x/\theta}, & x>0,\\ 0, & \text{其他}.\end{cases}$$

指数分布有一个重要特性：无记忆性，即：

$$P\{X>s+t \mid X>s\}=P\{X>t\}$$

因为：

$$\begin{aligned}P\{X>s+t \mid X>s\}&=\frac{P\{(X>s+t)\cap(X>s)\}}{P\{X>s\}}\\&=\frac{P\{X>s+t\}}{P\{X>s\}}=\frac{1-F(s+t)}{1-F(s)}\\&=\frac{e^{-(s+t)/\theta}}{e^{-s/\theta}}=e^{-t/\theta}\\&=P\{X>t\}.\end{aligned}$$

比如一个灯泡的使用寿命 X 服从指数分布，那么它已经使用了 s 小时，总共能再使用至少 s+t 小时的概率，和从开始使用时它至少能使用 t 小时的概率相等，即对已经使用过的 s 小时没有记忆。

**3. 正态分布**

如果连续随机变量 X 的概率密度为：

$$f(x)=\frac{1}{\sqrt{2\pi}\sigma}e^{-\frac{(x-\mu)^2}{2\sigma^2}},-\infty<x<\infty$$

其中 μ，σ（σ>0）为常数，则称 X 服从参数为 μ，σ 的正态分布，记为：

$$X\sim N(\mu,\sigma^2)$$

正态分布概念是由德国的数学家和天文学家棣莫弗于 1733 年首次提出的，但由于德国数学家高斯率先将其应用于天文学研究，故正态分布又叫高斯分布，高斯这项工作对后世的影响极大，他使正态分布同时有了“高斯分布”的名称，后世之所以多将最小二乘法的发明权归之于他，也是出于这一工作。如图 11-6 所示，正态分布的概率密度 f(x)有如下性质：

- 曲线关于 x=μ 对称；
- 当 x=μ 时取得最大值；

$$F(\mu)=\frac{1}{\sqrt{2\pi}\sigma}$$

- x 离 μ 越远，f(x)值越小，σ 越小时，图形越尖，X 落在 μ 附近的概率越大。

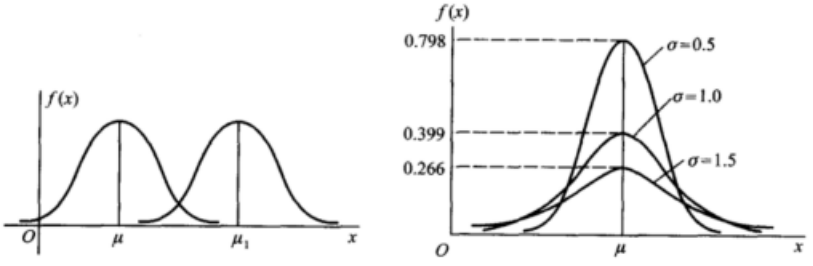



图 11-6 正态分布图

对正态分布概率密度求积分，可得正态分布随机变量 X 的分布函数为：

$$F(x)=\frac{1}{\sqrt{2\pi}\sigma}\int_{-\infty}^{x}e^{-\frac{(t-\mu)^2}{2\sigma^2}}dt$$

特别地，当μ=0，σ=1时称随机变量X服从标准正态分布，即X~N(0,1)，概率密度和分布函数分别用φ(x)和Φ(x)来表示：

$$\varphi(x)=\frac{1}{\sqrt{2\pi}}e^{-\frac{t^2}{2}}$$

$$\Phi(x)=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{x}e^{-\frac{t^2}{2}}dt$$

$$\Phi(-x)=1-\Phi(x)$$

其中，Φ(x)可以通过已经编制好的函数表查表求出。标准正态分布概率密度和分布函数的图像特征如图11-7所示：

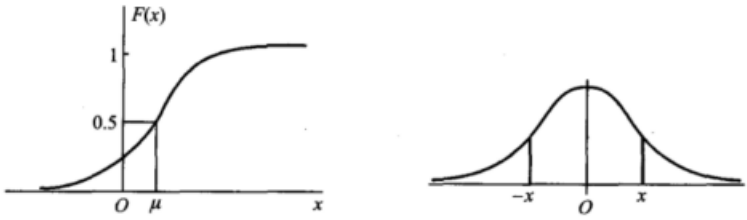



图11-7 标准正态分布概率密度和分布函数的图像特征

正态分布X~N(μ，σ²)可以通过线性变换，转化为标准的正态分布，即如果：

$$X\sim N(\mu,\sigma^2)$$

则：

$$Z=\frac{X-\mu}{\sigma}\sim N(0,1)$$

于是:

$$P\{\mu-3\sigma<X<\mu+3\sigma\}=P\{-3<(X-\mu)/\sigma<3\}=\Phi(3)-\Phi(-3)=99.74\%$$

通过上面的计算可以发现，尽管正态变量的取值范围是(-∞，+∞)，但它的值落在(μ-3σ,μ+3σ)内几乎是肯定的事，概率高达99.74%，这就是“3σ”法则。

对于标准正态分布X~N(0,1)的概率密度，如图11-8所示，若：

$$P(X>z_\alpha)=\alpha,\ 0<\alpha<1$$

则称$z_\alpha$为标准正态分布上的α分位点，且有$z_{1-\alpha}=-z_\alpha$。

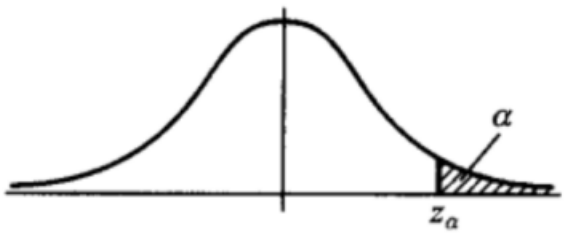



图 11-8 α 分位点

正态分布随机变量具有非常重要的作用。在现实世界中，大量随机变量都服从或近似服从正态分布，比如：人的身高，高考成绩，海洋波浪高度，零件长度误差，半导体器件中的热噪声电流电压等等。

## 11.5 随机变量数字特征

随机变量的数字特征是指能刻画随机变量某一方面特征的常数。在实际应用中，如果只研究随机变量的分布函数，概率密度和分布律往往还不够。比如一个班级中，每个学生在一次考试中的考试成绩是一个随机变量，但我们也非常关心这个班级的平均考试成绩，也关心每个学生成绩与这个平均成绩的偏离度，偏离程度越小，代表班级学生的成绩水平就越高。这里的平均成绩，以及学生成绩与平均成绩的偏离程度，就可以利用随机变量相关的数字特征数学期望和方差等来刻画。

### 11.5.1 数学期望

设离散型随机变量 X 的分布律为：$P\{X=x_k\}=p_k$, k=1, 2, 3, …。若级数:

$$\sum_{k=1}^{\infty} x_k p_k = x_1 p_1 + x_2 p_2 + x_3 p_3 + \ldots + x_n p_n + \ldots$$

收敛，则该级数的和称为随机变量 X 的数学期望，记为 E(X)，即：

$$E(X) = \sum_{k=1}^{\infty} x_k p_k$$

设连续型随机变量 X 的概率密度为 f(x)，若积分 $\int_{-\infty}^{\infty} xf(x)\mathrm{d}x$ 绝对收敛，则称积分 $\int_{-\infty}^{\infty} xf(x)\mathrm{d}x$ 的值为随机变量 X 的数学期望，记为 E(X)，即：

$$E(X) = \int_{-\infty}^{+\infty} xf(x)dx$$

数学期望有如下性质：

1，设 C 为常数，则 E(C)=C；

2，设 X 是一个随机变量，C 是常数，则 E(CX)=CE(X)；

3，设X，Y是两个独立的随机变量，则E(X+Y)=E(X)+E(Y);E(XY)=E(X)E(Y)。

数学期望又称为均值，是随机变量最基本的数学特征之一，它反映出了随机变量平均取值的大小。数学期望在现实中有着广泛的应用。下面我们通过练习来说明。

练习25：假设德国队和中国队进行乒乓球比赛。赛制有两种，一种是双方各出3人，三场两胜制，一种是双方各出5人，五场三胜制，哪一种赛制对中国队更有利？

分析：由于中国队在这项比赛中的优势，不妨设中国队中每一位队员德国队员的胜率都为60%。只需要比较两个队对应的数学期望即可。

在五场三胜制中，中国队要取得胜利，获胜的场数有3、4、5三种结果。应用二项分布可知，恰好获胜三场(即其中两场失利)对应的概率：

$$C_5^3(0.6)^3(1-0.6)^2 = 0.3456 \ ;$$

恰好获胜四场对应的概率为：

$$C_5^4(0.6)^4(1-0.6)^1 = 0.2592;$$

五场全部获胜的概率为：

$$C_5^5(0.6)^5(1-0.6)^0 = 0.07776 \text{。}$$

设随机变量为X为为该赛制下中国队在比赛中获胜的场数，则X分布律为：

| X | 3 | 4 | 5 |
|---|---|---|---|
| P | 0.3456 | 0.2592 | 0.07776 |

随机变量X 的数学期望：

E(X) = 3×0.3465 + 4×0.2592 + 5×0.07776= 2.4651。

在三场两胜制中，中国队取得胜利，，获胜的场数有2、3两种结果。对应的概率分别为：恰好获胜两场(其中有一场失利)对应的概率：

$$C_3^2(0.6)^2(1-0.6) = 0.432 \ ;$$

三场全部获胜的概率为：

$$C_3^3(0.6)^3(1-0.6)^0 = 0.216$$

设随机变量Y为该赛制下中国队在比赛中获胜的场数，则可建立Y的分布律：

| Y | 2 | 3 |
|---|---|---|
| P | 0.432 | 0.216 |

E(Y) = 2×0.432+ 3×0.216= 1.512

比较两个期望值得：E(X) > E(Y)。所以五场三胜制对中国队更有利。

练习26：某投资者有10万元，现有两种投资方案：一是购买股票，二是存人银行获取利息。买股票的收益主要取决于经济形势，假设可分三种状态：形势好、形势中等、形势不好(即经济衰退)。若形势好可获利40000元；若形势中等可获利10000元；若形势不好要损失20000元．如果是存人银行，假设年利率为8%，即可得利息8000元．又设年经济形势好、中等、不好的概率分别为30%，50%和20%。试问该投资者应选择哪一种投资方案？

分析：购买股票的收益与经济形势有关，存入银行的收益与经济形势无关．因此，要确

定选择哪一种方案，就必须通过计算这两种投资方案对应的收益期望值 E 来进行判断。由题设可知，一年中两种投资方式在不同的经济形势下对应的收益与概率如下表所示：

购买股票

| 状态 | 经济形势好 | 经济形势中等 | 经济形势不好 |
|---|---|---|---|
| 收益 | 40000 | 10000 | -20000 |
| 概率 | 0.3 | 0.5 | 0.2 |

存入银行

| 状态 | 经济形势好 | 经济形势中等 | 经济形势不好 |
|---|---|---|---|
| 收益 | 8000 | 8000 | 8000 |
| 概率 | 0．3 | 0.5 | 0.2 |

从上表可以初步看出，如果购买股票在经济形势好和经济形势中等的情况下是合算的，但如果经济形势不好，则采取存人银行的方案比较好下面通过计算加以分析.

如果购买股票，其收益的期望值：

$E_1=40000\times0.3+10000\times0.5+(-20000)\times0.2=13000$(元)；

如果存人银行，其收益的期望值：

$E_2=8000\times0.3+8000\times0.5+8000\times0.2=8000$(元)。

因此，购买股票的收益比存入银行的收益大，按期望收益最大原则，应选择购买股票，当然，股票的风险也很大。

练习 27：设连续随机变量满足均匀分布，即 X~U(a, b)，求 E(X)。

解：X 的概率密度为：

$$f(x)=\begin{cases}\dfrac{1}{b-a}, & a<x<b,\\ 0, & \text{其他},\end{cases}$$

那么，X 的数学期望为：

$$E(X)=\int_{-\infty}^{\infty}xf(x)\mathrm{d}x=\int_{a}^{b}\frac{x}{b-a}\mathrm{d}x=\frac{a+b}{2}$$

也就是说，均匀分布的数学期望位于区间(a, b)的中点。

## 11.5.2 方差

对于一个随机变量，如果只知道它的数学期望，有时候还不够。比如一个班级学生的平均成绩，即 E(X)=75，仅由这一指标还不能判定这个班级的学生成绩的好坏。因为有可能大部分的学生成绩都在 70 分到 80 分之间；也有可能有一半的学生成绩很好在 90 分以上，却

有一半的学生成绩很差，成绩大概只有 40~50 多分。因此，除了计算平均成绩之外，还需要注意学生成绩 X 与其均值 E(X)之间的偏离程度。

设 X 是一个随机变量，我们通常用 $E\{[X-E(X)]^2\}$来度量 X 与其均值 E(X)的偏离程度，若 $E\{[X-E(X)]^2\}$存在，则称 $E\{[X-E(X)]^2\}$为 X 的方差，记为 D(X)，即：

$$D(X)=E\{[X-E(X)]^2\}$$

同时，将$\sqrt{D(X)}$称为标准差或者均方差，记为 σ(X)。

随机变量 X 的方差 D(X)计算公式为：

$$D(X)=E(X^2)-[E(X)]^2$$

因为：

$$D(X)=E\{[X-E(X)]^2\}=E\{X^2-2XE(X)+[E(X)]^2\}$$
$$=E(X^2)-2E(X)E(X)+[E(X)]^2$$
$$=E(X^2)-[E(X)]^2$$

方差有如下性质：

- 设 C 为常数，则 D(C)=0;
- 设 X 是一个随机变量，C 是常数，则 $D(CX)=C^2D(X)$, D(X+C)=D(X);
- 设 X，Y 是两个独立的随机变量，则 D(X+Y)=D(X)+D(Y);E(XY)=E(X)E(Y);
- D(X)=0 的充要条件是 X 以概率 1 取常数 E(X)，即：P{X=E(X)}=1。

练习 28：设随机变量 X 服从 0-1 分布，分布律为：

$$P\{X=0\}=1-p, P\{X=1\}=p$$

求 D(X)。

解：先分别求出 E(X)和 $E(X^2)$，然后再求 D(X)，因为：

$E(X)=0*(1-p)+1*p=p$

$E(X^2)=0^2*(1-p)+1^2*p=p$

所以：

$D(X)=E(X^2)-[E(X)]^2=p-p^2=p(1-p)$

练习 29：设随机变量服从正态分布，即 $X\sim N(\mu,\sigma^2)$，求 E(X), D(X)

解：先求标准正态变量：$Z=(X-\mu)/\sigma$。

Z 的概率密度为：

$$\varphi(t)=\frac{1}{\sqrt{2\pi}}e^{-t^2/2}$$

Z 的期望和方差为：

$$E(Z)=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty}te^{-t^2/2}dt=\frac{-1}{\sqrt{2\pi}}e^{-t^2/2}\Big|_{-\infty}^{\infty}=0,$$

$$D(Z)=E(Z^2)=\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty}t^2e^{-t^2/2}dt$$

$$=\frac{-1}{\sqrt{2\pi}}te^{-t^2/2}\Big|_{-\infty}^{\infty}+\frac{1}{\sqrt{2\pi}}\int_{-\infty}^{\infty}e^{-t^2/2}dt=1.$$

而 $X=\mu+\sigma Z$，于是根据方差的性质有：

E(X)=E(μ+σZ)= μ

D(X)=D(μ+σZ)=D(σZ)= $\sigma^2$D(Z)= $\sigma^2$

### 11.5.3 协方差

前面我们主要介绍了一维随机变量，但在实际问题中，某些随机试验的结果往往需要同时用两个或两个以上的随机变量来描述。比如一个班级学生的考试，考试成绩就可能包含语文，数学等各科成绩，需要用多个随机变量来描述。

一般地，设 E 是一个随机试验，它的样本空间为 Ω={e}，设 X=X(e)和 Y=Y(e)是定义在 S 上的随机变量，由它们构成的一个向量(X, Y)叫做二维随机变量。

二维随机变量(X, Y)的性质不仅与 X 和 Y 有关，而且还依赖于这两个随机变量的相互关系，它们之间既可以是互相独立的，也可以是相互关联的。

对于二维随机变量(X, Y)，除了讨论 X 与 Y 的数学期望和方差以外，也需要讨论描述 X 与 Y 之间的相互关系和数字特征。

我们把量：

E{[X-E(X)][Y-E(Y)]}

称为随机变量 X 与 Y 的协方差，记为 Cov(X, Y)，即：

Cov(X, Y)= E{[X-E(X)][Y-E(Y)]}

而

$$\rho_{XY}=\frac{Cov(X,Y)}{\sqrt{D(X)}\sqrt{D(Y)}}$$

称为随机变量 X 与 Y 的相关系数。根据相关系数的这个定义，很容易得出如下结论：

- Cov(X,Y)=Cov(Y,X)
- Cov(X, X)=D(X)

将 Cov(X, Y)的定义展开有：

Cov(X, Y)=E(XY)-E(X)E(Y)

实际上有$|\rho_{XY}|\leqslant 1$。当$\rho_{XY}$=0 时，X 和 Y 不相关（即 X 和 Y 不存在线性关系）；当$|\rho_{XY}|$较大时，X 和 Y 线性相关度较好；$|\rho_{XY}|$较小时则 X 和 Y 线性相关性较差；而$|\rho_{XY}|\doteq 1$的充要条件是存在常数 a 和 b，使得 P{Y=a+bX}=1。

当 X 和 Y 相互独立时，因为 E(XY)=E(X)E(Y)，Cov(X, Y)=E(XY)-E(X)E(Y)=0，因此$\rho_{XY}$=0，因此 X 和 Y 不相关。但如果 X 和 Y 不相关，X 和 Y 却不一定相互独立，因为所谓的不相关只是不存在线性关系，而相互独立是指不存在一般关系（包括线性关系）。

特别地，如果(X, Y)服从二维正态分布，那么 X 和 Y 不相关与 X 和 Y 相互独立是等价的。

## 11.6 随机过程

截止目前，相信大家对随机变量这个概念已经非常熟悉了。随机变量就是用来描述一次随机试验结果可能的取值。随机变量只表示一次随机试验的结果取值，而并没有涉及到时间

维度。

如果分别在时间为$t_1$，$t_2$，$t_3$，...等不同时刻做同一个随机试验，那么这些时刻就分别对应着多个随机变量$X_1$，$X_2$，$X_3$，...等。当时间在某区间比如[0, ∞)上推移，那么就会存在一簇随机变量。

设 T 是以无限实数集，那么依赖于参数 t∈T 的一簇（无限多个）随机变量称为随机过程，记为{X(t), t∈T}。对于每一个 t∈T，X(t)就是一个随机变量，其中，T 为参数集，t 通常被看作时间，X(t)称为时刻 t 时过程的状态，X(t)所有可能的取值的全体称为随机过程的状态空间。

随机过程的一些常见实例：

(1)英国植物学家布朗注意到漂浮在液面上的微小粒子不断进行不规则的运动，这种运动叫做布朗运动。它是大量分子随机碰撞的结果。若记 X(t)为粒子在 t 时刻位于平面上的位置， X(t)是一个随机变量，布朗运动就是一个随机过程。

(2)一个醉汉在路上行走，以概率 P 前进一步，概率 1-P 后退一步，以 X(t)记他在 t 时刻时位于路上的位置，则 X(t)就是直线上的随机变量，即他在每个时刻的位置都是一个随机变量，因此醉汉在路上的行走就是一个随机过程。

(3)通信系统中的热噪声电压，热噪声是由电阻性原件中的电子因热扰动而产生端电压，它在任意确定时刻 t 的值是一随机变量 V(t)，随着时间的推移，热噪声电压表现为一簇随机变量也就是随机过程，记为{V(t), t≥0}。

随机过程作为数理统计的重要部分，在气象预报、通信工程、金融工程、人口理论和可靠性与质量控制等各个方面都有重要应用。

对随机过程{X(t), t∈T}进行一次在 T 上的全程观测试验，其结果是 t 的函数，记为 x(t)，t∈T，x(t)为随机过程的一个样本函数或样本曲线，所有不通的试验结果构成一簇样本函数。一般来说，只有通过分析样本函数才能掌握随机过程随时间变化的统计规律性。

如果随机过程在任一时刻的状态是连续型随机变量，则该随机过程称为连续型随机过程；如果随机过程在任一时刻的状态是离散型随机变量，则该随机过程称为离散型随机过程。如果 T 是有限或无限区间时，该随机过程为连续参数随机过程，简称随机过程；如果 T 是离散集合，比如 T={0,1,2,3,...}这种表示第 n 次试验，则称该随机过程为离散参数随机过程或随机序列。

有时候，也可以将连续参数随机过程转换为随机序列，比如在相同时间间隔 Δt 内，即 T={Δt, 2Δt, 3Δt, ..., nΔt,...}连续做随机试验得到的随机过程即是一个随机序列。

## 11.6.1 马尔科夫过程

马尔科夫过程描述的是随机过程在不同时刻状态之间的特殊的统计联系。如果随机过程{X(t), t∈T}在时刻 $t_0$ 所处的状态为已知的条件下，在时刻 $t>t_0$ 所处的状态的条件分布与在时刻 $t_0$ 之前所处的状态无关，那么该随机过程具有马尔科夫性，并称该随机过程为马尔科夫过程，用概率中的分布函数语言来描述马尔科夫过程即为：

设随机过程{X(t), t∈T}的状态空间为 I，如果对时间 t 的任意 n 个数值 $t_1<t_2<t_3<...<t_n$, n≥3,$t_i$∈T,在条件 $X(t_i)=x_i$，$x_i$∈I，i=1,2,3，..., n-1 下，$X(t_n)$的条件分布函数恰等于在条件 $X(t_{n-1})=x_{n-1}$ 下 $X(t_n)$的条件分布函数，即：

$P\{X(t_n)\leqslant x_n|X(t_1)=x_1, X(t_2)=x_2,, X(t_3)=x_3, ...,X(t_{n-1})<x_{n-1}\}=$

$P\{X(t_n)\leqslant x_n|X(t_{n-1})<x_{n-1}\}$, $x_n$∈R

那么该随机过程具有马尔科夫性，并称该随机过程为马尔科夫过程。

时间和状态都是离散的马尔科夫过程称为马尔科夫链，简称马氏链，记为：

$$\{X_n=X(n), n=0, 1, 2,3, ...\}$$

马氏链可看作在时间集 T={0, 1, 2, 3, ...}上对离散状态的马氏过程相继观察的结果。

设马氏链的状态空间为 $I=\{a_1, a_2, a_3, ...\}$，$a_i \in R$，记条件概率 $P_{ij}(m, m+n)=P\{X_{m+n}=a_j|X_m=a_i\}$ 为马氏链在时刻 m 处于状态 $a_i$ 条件下，在时刻 m+n 转移到状态 $a_j$ 的转移概率。当转移概率 $P_{ij}(m, m+n)$只与 i, j 及时间间距 n 有关而与 m 无关时，把它记为 $P_{ij}(n)$，即：

$$P_{ij}(m, m+n)=P_{ij}(n)$$

并称此转移概率具有平稳性，同时此马氏链是齐次或齐时的。$P_{ij}(n)= P\{X_{m+n}=a_j|X_m=a_i\}$ 称为马氏链的 n 步转移概率。$P(n)=(P_{ij}(n))$为 n 步转移概率矩阵。

当 n=1 时：

$$X_{m+1}\text{的状态}$$

$$X_m\text{的状态}\begin{array}{c} \\ a_1 \\ a_2 \\ \vdots \\ a_i \\ \vdots \end{array}\begin{array}{c} \begin{array}{cccccc} a_1 & a_2 & \cdots & a_j & \cdots \end{array} \\ \begin{bmatrix} p_{11} & p_{12} & \cdots & p_{1j} & \cdots \\ p_{21} & p_{22} & \cdots & p_{2j} & \cdots \\ \vdots & \vdots & & \vdots & \\ p_{i1} & p_{i2} & \cdots & p_{ij} & \cdots \\ \vdots & \vdots & & \vdots & \end{bmatrix} \end{array} = \boldsymbol{P}(1) \xrightarrow{\text{记成}} \boldsymbol{P}$$

在上述矩阵中，$p_{ij}$ 显示由状态 $a_i$ 一步转移到 $a_j$ 的概率。

而当 n>1 时，n 步转移概率矩阵是一步转移概率矩阵的 n 次方，即：

$$P(n)=P^n$$

## 11.6.2 平稳随机过程

如果随机过程的统计特性不随时间的推移而变化，即对于任意的 n(n=1, 2, 3, ...)，$t_1$，$t_2$，$t_3$，...，$t_n \in T$，和任意实数 h，当 $t_1+h$，$t_2+h$，$t_3+h$，...，$t_n+h \in T$ 时，n 维随机变量：

$$\mathbf{(X(t_1), X(t_2), X(t_3), ..., X(t_n))}$$

和：

$$\mathbf{(X(t_1+h), X(t_2+h), X(t_3+h), ..., \ X(t_n+h))}$$

具有相同的分布函数，则称随机过程{X(t), t∈T}具有平稳性，并称该随机过程为平稳随机过程，简称平稳过程。

在实际应用中，一般很难通过确定随机过程的分布函数进而来判断其平稳性。但如果随机过程前后的环境和主要条件不会随着时间推移而变化那么一般就认为该过程就是平稳过程。随机过程在过渡阶段，总是非平稳的。

比如：恒温条件下的热噪声电压，船舶的颠簸过程，强震阶段的地震波幅等都被认为是平稳的随机过程。

# 11.7 数理统计

前面我们学习了概率论，现在我们来学习数理统计。数理统计以概率论为基础，根据实验或观察得到数据，根据数据来研究随机现象，对被研究对象的客观规律做出种种合理的估计，并对估计进行判断和检验。

与概率论不同的是，数理统计中的随机变量的分布是未知的或者不完全知道，只能对随

机变量展开独立的观察和实验，得到一组数据，然后对数据进行统计分析，从而推断该随机变量的分布。

数理统计对于计算机中的大数据分析，数据挖掘，机器学习和人工智能都有非常紧密的联系，因此掌握数理统计的基本理论和统计分析方法，非常重要。但是非常遗憾的是，目前国内高校的数学教学，却往往更重视概率论的学习而容易轻视这一块内容。

## 11.7.1 样本

在对随机变量进行实验观察的时候，我们把全部可能的观察值称为总体，每一个可能的观察值称为个体。总体中包含的个体个数称为总体的容量，容量有限的称为有限总体，容量无限的为无限总体。

总体中的个体是随机试验的观察值，对应着随机变量 X 的值，即一个总体对应一个随机变量 X，因此总体和随机变量可以统称为总体 X。

总体的分布一般都是未知的，或者虽然知道它有某种形式但却包含着未知参数，比如指数分布中的 θ 可能就是未知参数。

从总体 X 中抽取一个个体，就是对总体的一次观察。在相同条件下对总体 X 进行 n 次重复的、独立的观察，并将结果按实验次序记为 $X_1$，$X_2$，$X_3$，…，$X_n$，那么 $X_1$，$X_2$，$X_3$，…，$X_n$ 也是相互独立的，且都是与 X 具有相同分布的随机变量。$X_1$，$X_2$，$X_3$，…，$X_n$ 称为来自总体 X 的一个简单随机样本，n 称为样本的容量。

随着 n 次观察的结束，就得到了一组实数观察值：$x_1$，$x_2$，$x_3$，…，$x_n$。这组实数观察值被称为样本值。

对总体 X 的一次观察，就是抽样。抽样分为放回抽样（即抽样后将个体重新放回总体）和不放回抽样（即抽样后将个体不放回总体）。当有限总体的容量 N 比样本容量 n 大得多的时候或者为无限总体的时候，可以采用不放回抽样。

与样本相关的函数被称为统计量，常用的统计量包括：

- 样本平均值：

$$\bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

- 样本方差：

$$S^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X}^2) = \frac{1}{n-1}(\sum_{i=1}^{n} X_i^2 - n\bar{X}^2)$$

- 样本标准差：

$$S = \sqrt{S^2} = \sqrt{\frac{1}{n-1}\sum_{i=1}^{n}(X_i - \bar{X})^2}$$

## 11.7.2 抽样分布

统计量的分布称为抽样分布，当总体的分布函数已知时，抽样分布是确定的。正态总体

的几个常用统计量分布：

## 1．卡方($\chi^2$)分布

设 $X_1$，$X_2$，$X_3$，…，$X_n$ 是来自总体 N(0, 1) 的样本，则称统计量 $\chi^2 = X_1^2 + X_2^2 + X_3^2 + ... + X_n^2$ 服从自由度为 n 的卡方分布，即 $\chi^2$ 分布，记为 $\chi^2 \sim \chi^2$ (n)，其中参数 n 称为自由度。

## 2．t 分布

设 $X \sim N(0,1), Y \sim \chi^2(n)$，且 X，Y 相互独立，则称随机变量 $t = \frac{X}{\sqrt{Y/n}}$ 服从自由度为 n 的 t 分布，记为 t~t(n)，t 分布又称为学生氏分布。

## 3．F 分布

设 $U \sim \chi^2$ ($n_1$)，$V \sim \chi^2$ ($n_2$)，且 U, V 相互独立，则称随机变量 $F = \frac{U/n_1}{V/n_2}$ 服从自由度为 ($n_1$, $n_2$) 的 F 分布，记为 F~F($n_1$, $n_2$)。

上面介绍的抽样分布在参数估计（参考本书 11.7.4 节），假设检验（参考本书 11.7.5 节）方面有着重要的应用。比如在方差已知情况下求均值是 Z 检验；方差未知求均值是 t 检验；均值方差都未知求方差是 $\chi^2$ 检验；两个正态分布样本的均值方差都未知情况下求两个总体的方差比值是 F 检验。

## 11.7.3 中心极限定理

中心极限定理指出，样本的平均值约等于总体的平均值。不管总体是什么分布，任意一个总体的样本平均值都会围绕在总体的整体平均值周围，并且呈正态分布，即给定一个任意分布的总体，每次从这些总体中随机抽取 n 个抽样，一共抽 m 次，然后把这 m 组抽样分别求出平均值，那么这些平均值的分布接近正态分布。

中心极限定理让我们能够从少量数据中得出最终的结论。比如只需要对 1000 个美国人进行电话民意调查，就能去预测美国总统大选的得票数。于是我们可以使用样本对总体进行估计，例如我们经常看到的民意调查就是这方面的经典案例。

## 11.7.4 参数估计

如果总体 X 的分布函数类型已知，但它的参数有未知的，比如泊松分布的 λ，指数分布的 θ，正态分布的参数 σ 和 μ，那么可以借助总体 X 的一个样本来估计总体未知参数的值或范围，这就是参数估计。参数估计是统计推断的一类基本问题。

### 1．点估计

对分布函数的未知参数进行值的估计，叫做点估计。比如，如果某随机变量 X 服从泊松分布，但参数 λ 未知，如何根据样本来估计 λ 的取值呢？根据 λ 泊松分布的性质，E(X)= λ。因此，可以计算出样本的均值，用样本的均值近似代替 λ，记为 $\hat{\lambda}$。即：

$$\hat{\lambda} = \bar{X} = \frac{1}{n}\sum_{i=1}^{n} X_i$$

点估计的方法常用的有最大似然估计法。最大似然估计的思想如下：

假设总体 X 属离散型，其分布律为：P{X=x}=p(x; θ)，θ ∈ Θ，其中 θ 为待估参数。又设 $x_1$，$x_2$，$x_3$，…，$x_n$ 是对应样本 $X_1$，$X_2$，$X_3$，…，$X_n$ 的一个样本值，那么 P{$X_1$=$x_1$, $X_2$=$x_2$, $X_3$=$x_3$, …, $X_n$=$x_n$}的概率为：

$$L(\theta) = L(x_1, x_2, x_3, ..., x_n; \theta) = \prod_{i=1}^{n} p(x_i; \theta), \theta \in \Theta$$

该概率随 θ 的值而变化，因此 L(θ)称为样本的似然函数。既然在实验中取得了样本值 $x_1$，$x_2$，$x_3$，…，$x_n$，那么取到这一样本值的概率 L(θ)比较大。如果 θ = θ 0∈ Θ，使得 L(θ)的取最大值，那么 θ 0 作为未知参数 θ 的估计值，较为合理。

因此，在固定样本观察值 $x_1$，$x_2$，$x_3$，…，$x_n$，在 Θ 内使得 L(θ)达到最大的取值，作为参数 θ 的估计值，记为 $\hat{\theta}$，即：

$$L(x_1, x_2, \cdots, x_n; \hat{\theta}) = \max_{\theta \in \Theta} L(x_1, x_2, \cdots, x_n; \theta)$$

该估计值被称为参数 θ 的最大似然估计值。而如何求 L(θ)的最大值，则是一个高等数学微分学中求最大值的问题，可以通过对 L(θ)求导然后解方程求得。

### 2．区间估计

对于未知参数 θ，除了可以求出它的点估计值外，还可以估计出一个范围即 $\theta_0 < \theta < \theta_1$，并希望知道这个范围包含参数 θ 真值的可信程度，我们称这种估计为区间估计。

设总体 X 的分布包含一个未知数 θ，对于给定值 α (0< α <1)，若来自 X 的样本 $X_1$，$X_2$，$X_3$，…，$X_n$ 确定的两个统计量 $\theta_0(X_1, X_2, X_3, ..., X_n)$和 $\theta_1(X_1, X_2, X_3, ..., X_n)$，如果有：

$$P\{\theta_0 < \theta < \theta_1\} \geqslant 1-\alpha,$$

则称随机区间($\theta_0$, $\theta_1$)是 θ 的置信水平为 1- α 的置信区间，$\theta_0$, $\theta_1$ 称为置信水平为 1-

α 的双侧置信区间的置信下限和置信上线，1-α 为置信水平。

多次抽样，每个样本值确定一个区间( $\theta_0$、$\theta_1$)，这些区间中包含 θ 的区间约占 100(1-α)。

设总体 X~N( μ , $\sigma^2$)， $\sigma^2$ 为已知， μ 为未知，设 $X_1$，$X_2$，$X_3$，...，$X_n$ 为来自 X 的样本，那么如何求 μ 的置信水平为 1-α 的置信区间呢？

因为有：

$$\frac{\overline{X}-\mu}{\sigma/\sqrt{n}}\sim N(0,1)$$

按照标准正态分布的上 α 分位点的定义有：

$$P\left\{\left|\frac{\overline{X}-\mu}{\sigma/\sqrt{n}}\right|<z_{\alpha/2}\right\}=1-\alpha,$$

即：

$$P\left\{\overline{X}-\frac{\sigma}{\sqrt{n}}z_{\alpha/2}<\mu<\overline{X}+\frac{\sigma}{\sqrt{n}}z_{\alpha/2}\right\}=1-\alpha.$$

于是 μ 的一个置信水平为 1-α 的置信区间为：

$$\left(\overline{X}-\frac{\sigma}{\sqrt{n}}z_{\alpha/2},\quad \overline{X}+\frac{\sigma}{\sqrt{n}}z_{\alpha/2}\right)$$

通常可以简写为：

$$\left(\bar{X}\pm\frac{\sigma}{\sqrt{n}}z_{\alpha/2}\right)$$

如果取 α =0.05，那么 1-α =0.95，如果 σ =1，n=16，那么 $z_{\alpha}/2=z_{0.025}=1.96$，代入求得置信区间为：

$$(\bar{X}\pm 0.49)$$

如果某一样本均值为 5.11，那么对应的一个区间为：(4.62, 5.6)。

若反复抽样多次，按照上面的方法算出来的所有区间中，包含 μ 的约占 95%，不包含 μ 的约占 5%。而区间(4.62, 5.6)包含 μ 的可信程度为 95%。

## 11.7.5 假设检验

假设检验是统计推断另外一类重要的问题。由于总体的分布函数完全未知或参数未知，因此需要作出一些关于总体的假设，比如假设总体服从泊松分布，又比如假设总体服从正态分布的参数 $\mu=\mu_0$。

提出假设之后，再根据试验观察的样本对所提出的假设进行检验，作出是接受还是拒绝的决策。

下面就是一个具体的假设检验的过程。

## 1．双边假设检验

设总体 X，服从正态分布，即 X~(μ, $0.015^2$)，μ 未知。现在需要通过样本观察值来判断 μ=0.5 还是 μ≠0.5。于是，提出两个互相对立的假设：

$H_0$：μ=$\mu_0$=0.5

$H_1$：μ≠$\mu_0$

现在需要做的工作，就是对接受 $H_0$ 还是拒绝 $H_0$ 做出判断。因为$\overline{X}$是 μ 的无偏估计，其观察值 $\overline{x}$ 的大小在一定程度上反应了 μ 的大小。因此假设 $H_0$ 为真，那么$|\overline{x}-\mu_0|$应该比较小，如果太大，就应该拒绝 $H_0$。

又因为$\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}\sim N(0,1)$，所以$|\overline{x}-\mu_0|$可以转化为考查$\frac{|\overline{x}-\mu_0|}{\sigma/\sqrt{n}}$的大小。于是，可以选择一个正数 k，当$\frac{|\overline{x}-\mu_0|}{\sigma/\sqrt{n}}\geqslant k$时拒绝假设 $H_0$，$\frac{|\overline{x}-\mu_0|}{\sigma/\sqrt{n}}<k$时接受假设 $H_0$。

通过一个样本来做出决策，是可能会有错误发生做出错误决策的。也就是说，当 $H_0$ 为真的时候，我们却做出了拒绝 $H_0$ 的决策。因此，需要将这个错误的概率控制在一定范围内，即通过一个较小的 α (0<α<1)，使得犯错误的概率不超过 α，即：

$$P\{\text{当 } H_0 \text{ 位真拒绝 } H_0\}\leqslant \alpha$$

取最大的概率值 α，那么当 μ=$\mu_0$ 时却拒绝 $H_0$ 的概率可表示为：

$$P_{\mu_0}\left\{\left|\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}\right|\geqslant k\right\}=\alpha$$

因为 Z=$\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}$~N(0,1)，那么由正态分布的分位点定义有：

$$k=z_{\alpha/2}$$

因此若 Z 的观察值，即$|z|=\left|\frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}}\right|\geqslant z_{\alpha/2}$，则拒绝 $H_0$；否则若$|z|=\left|\frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}}\right|\leqslant z_{\alpha/2}$，接受 H0。

假如 α 取 0.05，那么 $k=z_{0.05/2}=z_{0.025}=1.96$，已知 n=9，σ=0.015，若$\overline{x}$=0.511，那么代入$\left|\frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}}\right|$=2.2>1.96，因此拒绝 $H_0$。

通过上面的例子可以看出，确定 α 后，就可以确定 k，然后观察统计量 Z 的观察值的绝对值$|z|=\left|\frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}}\right|$和 k 做比较来做出决策。如果|z|≥k，则称$\overline{x}$与 $\mu_0$ 差异是显著的，从而拒绝 $H_0$；否则差异是不显著的，从而接受 $H_0$。实数 α 称为显著性水平，Z=$\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}$称为检验统计量。

当检验统计量取某个区域 C 中的值而拒绝原假设 $H_0$，则称 C 区域为拒绝域，拒绝域的边界点称为临界点。

## 2．单边检验

有时候如果我们只关注均值μ是否增大或者减小，那么需要做如下的检验假设：

右边检验：$H_0$：$\mu \leqslant \mu_0$，H1：$\mu > \mu_0$（$\mu_0$为最大值，在μ取值范围的右边）。

或者：

左边检验：$H_0$：$\mu \geqslant \mu_0$，$H_1$：$\mu < \mu_0$（$\mu_0$为最小值，在μ取值范围的左边）。

右边检验和左边检验统称为单边检验。先分析右边检验。设总体 X~N(μ，$\sigma^2$)，μ未知，σ已知。。给定显著水平α，那么，现在来求下面右边假设的拒绝域：

$H_0$：$\mu \leqslant \mu_0$，H1：$\mu > \mu_0$

由右边假设可得$H_0$中的全部μ要比$H_1$中的μ要小，当$H_1$为真时，那么观察值$\overline{x}$一般会偏大，因此拒绝域应该为：

$\overline{x} \geqslant k$（k为某一正常数）

P{当$H_0$为真拒绝$H_0$}=$P_{\mu \in H_0}\{\overline{X} \geqslant k\}=P_{\mu \leqslant \mu_0}\left\{\frac{X-\mu_0}{\sigma/\sqrt{n}} \geqslant \frac{k-\mu_0}{\sigma/\sqrt{n}}\right\} \leqslant P_{\mu \leqslant \mu_0}\left\{\frac{\overline{X}-\mu}{\sigma/\sqrt{n}} \geqslant \frac{k-\mu_0}{\sigma/\sqrt{n}}\right\}$（因为$\mu \leqslant \mu_0$，所以后者包含前者）。

要使P{当$H_0$为真拒绝$H_0$}$\leqslant \alpha$,那么:

$$P_{\mu \leqslant \mu_0}\left\{\frac{\overline{X}-\mu}{\sigma/\sqrt{n}} \geqslant \frac{k-\mu_0}{\sigma/\sqrt{n}}\right\} = \alpha.$$

由于$\frac{\overline{X}-\mu}{\sigma/\sqrt{n}}$~N(0, 1)，因此 $=z_\alpha$，$k=\mu_0+\frac{\sigma}{\sqrt{n}}z_\alpha$

因此拒绝域为：$\overline{x} \geqslant \mu_0+\frac{\sigma}{\sqrt{n}}z_\alpha$，即 $z=\frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}} \geqslant z_\alpha$

也就是显著水平为α时，只要$z \geqslant z_\alpha$，就可以拒绝假设$H_0$。

左边检验与右边检验类似，因此左边检验：$H_0$：$\mu \geqslant \mu_0$，$H_1$：$\mu < \mu_0$ 的拒绝域为：

$$z = \frac{\overline{x}-\mu_0}{\sigma/\sqrt{n}} \leqslant -z_\alpha$$

在正态总体中，$\sigma^2$已知，μ未知，利用统计变量Z=$\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}$来确定拒绝域的被称为Z检验法。

## 3．P值检验

学习P值检验，直接从一个例子入手。假设总体 X~N(μ，$\sigma_2$)，μ未知，$\sigma_2$=100，现有样本$x_1$，$x_2$，$x_3$，…，$x_{52}$，算得x=62.75。现假设：

$H_0$：$\mu=\mu_0$=60，$H_1$：$\mu$>60。

采用Z检验法，检验统计量为Z=$\frac{\overline{X}-\mu_0}{\sigma/\sqrt{n}}$

代入数据，得 Z 的观察值为：

$z_0=\frac{62.75-60}{10/\sqrt{52}}=1.983$

$Z\geqslant z_0$ 的概率：$P\{Z\geqslant z0\}=P\{Z\geqslant 1.983\}=1-\Phi(1.983)=0.0238$。

此概率称为 Z 检验法的右边检验的 p 值，记为：

p 值=$P\{Z\geqslant z0\}=0.0237$

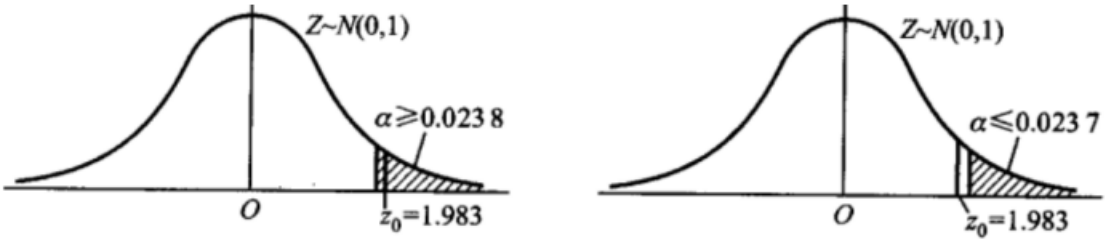



图 11-9 P 值检验

如图 11-9 所示，若显著水平 α ≥p（0.0238），则对应的临界值 $z_\alpha\leqslant 1.983$（阴影部分面积），那么 $z_0=1.983$ 落在了拒绝域内，因而拒绝 $H_0$；若 α <p=1.983，那么 $z_\alpha>1.983$，那么 $z_0=1.983$ 没有落在拒绝域内，因而接受 $H_0$。

由此可以看出，p 值是原假设 $H_0$ 可被拒绝的最小显著水平。因此，假设检验问题的 p 值是由检验统计量的样本观察值得出的原假设可被拒绝的最小显著水平。

对于任意指定的显著水平 α，若：

$p\leqslant \alpha$，则拒绝 $H_0$；

$p>\alpha$，则接受 $H_0$。

## 11.7.6 回归分析

世界是按照特定的规律运动和变化的，而变量可以用于描述这些客观规律。变量与变量之间往往存在着某些用函数来确定的关系。物理学中的定理其实就是某些变量关系的体现。比如牛顿发现的万有引力定理，就指出了两个物体之间的引力和物体的质量和距离有着确定的关系。又比如一个网站上投放的广告的点击率，就和这个广告的主题，位置，网站用户的年龄，性别，职业等都有关系。

回归分析就是研究变量之间相关关系的一种数学工具，这种关系，往往是通过一个相关函数来体现的。通过回归分析求出这个相关函数，然后可以从一个变量取得的值，通过这个相关函数，去估计另一个变量所取的值。

设随机变量 Y 与普通变量 x 之间存在着相关关系，对于 x 各个确定的值，Y 的分布函数记为 F(y|x)，通过该分布函数，我们就可以确定 Y 与 x 之间的关系。但实际情况是这样做比较复杂，所以改为寻找 Y 的数学期望 E(Y)与 x 之间的关系。若 Y 的数学期望 E(Y)存在，那么 E(Y)的值也是随 x 的取值而定，因此 E(Y)也是 x 的函数，记为 E(Y)= μ(x)，我们称 μ(x)为 Y 关于 x 的回归函数。

## 1. 一元线性回归

设 Y 关于 x 的回归函数为 μ(x)，通过样本来估计 μ(x)的问题称为求 Y 关于 x 的回归问题。若 μ(x)为线性函数，即 μ(x)=a+bx，此时估计 μ(x)的问题称为求一元线性回归问题，

即要确定线性函数中的 a 和 b 的值，因为确定了 a 和 b 的值，那么 μ(x)就可以确定了。

假设对于 x 在某个区间内的每一个取值，Y 服从正态分布，即 $Y\sim N(a+bx, \sigma^2)$，其中 a，b，$\sigma^2$ 都是和 x 无关的未知参数，记 $\varepsilon=Y-(a+bx)$，那么就相当于：

$$Y=a+bx+\varepsilon,\quad \varepsilon\sim N(0, \sigma^2)$$

我们称上面的等式为一元线性回归模型，Y 由两部分组成，一部分是 x 的线性函数 a+bx，零一部分 $\varepsilon\sim N(0, \sigma^2)$是不可控的随机误差。

那么如何来估计 μ(x)=a+bx 中的 a 和 b 的值呢？

取 x 的 n 个不全相同的值 $x_1$，$x_2$，$x_3$，…，$x_n$ 作独立试验，得到样本$(x_1, Y_1)$，$(x_2, Y_2)$，$(x_3, Y_3)$，…，$(x_n, Y_n)$，于是有：

$$Y_i=a+bx_i+\varepsilon_i,\quad \varepsilon_i\sim N(0, \sigma^2),\quad i=1, 2, 3, \ldots, n$$

通过似然估计的方法，求出使得 $Y_1$，$Y_2$，$Y_3$，…，$Y_n$ 的联合密度函数取得最大值的 a，b 就是我们的似然估计值。推导过程略，下面直接给出计算公式：

$$\left.\begin{aligned}\hat{b}&=\frac{n\sum_{i=1}^{n}x_iy_i-\left(\sum_{i=1}^{n}x_i\right)\left(\sum_{i=1}^{n}y_i\right)}{n\sum_{i=1}^{n}x_i^2-\left(\sum_{i=1}^{n}x_i\right)^2}=\frac{\sum_{i=1}^{n}(x_i-\bar{x})(y_i-\bar{y})}{\sum_{i=1}^{n}(x_i-\bar{x})^2},\\ \hat{a}&=\frac{1}{n}\sum_{i=1}^{n}y_i-\frac{\hat{b}}{n}\sum_{i=1}^{n}x_i=\bar{y}-\hat{b}\bar{x},\end{aligned}\right\}$$

其中：$\bar{x}=\frac{1}{n}\sum_{i=1}^{n}x_i$，$\bar{y}=\frac{1}{n}\sum_{i=1}^{n}y_i$

在得到 a，b 的估值后，对于给定的 x，我们就取$\hat{a}+\hat{b}x$ 作为回归函数 a+bx 的估计，称为 Y 关于 x 的经验回归函数。

而：

$$\hat{y}=\hat{a}+\hat{b}x$$

被称为 Y 关于 x 的经验回归方程，简称回归方程，图形为一条直线。

## 2. 多元线性回归

在实际问题中，随机变量 Y 可能与多个普通变量 $x_1$，$x_2$，$x_3$，…，$x_p(p>1)$有关。对于自变量 $x_1$，$x_2$，$x_3$，…，$x_p$ 的一组确定的值，Y 有它的分布。若 Y 的数学期望存在，则它是 $x_1$，$x_2$，$x_3$，…，$x_p$ 的函数，记为 $\mu(x_1, x_2, x_3, \ldots, x_p)$，它是 Y 关于 x 的回归函数。现在我们主要关注 $\mu(x_1, x_2, x_3, \ldots, x_p)$是 $x_1$，$x_2$，$x_3$，…，$x_p$ 的线性函数的情况，即多元线性回归。与一元线性模型类似，我们把：

$$Y=b_0+b_1x_1+b_2x_2+\ldots+b_px_p+\varepsilon,\quad \varepsilon\sim N(0, \sigma^2)$$

称为多元线性回归模型，其中 $b_0$，$b_1$，$b_2$，…，$b_p$，$\sigma^2$ 都是与 $x_1$，$x_2$，$x_3$，…，$x_p$ 无关的未知参数。设:

$$(x_{11}, x_{12}, x_{13}, \ldots, x_{1p}, y_1),\ \ldots,\ (x_{n1}, x_{n2}, x_{n3}, \ldots, x_{np}, y_n)$$

是一个样本，同样可以使用似然估计法来估计未知参数。

在进行多元线性回归的似然参数估计的时候，引入了如下有关样本和参数的矩阵（矩阵运算请参考线性代数部分）：

$$\boldsymbol{X}=\begin{bmatrix}1 & x_{11} & x_{12} & \cdots & x_{1p}\\ 1 & x_{21} & x_{22} & \cdots & x_{2p}\\ \vdots & \vdots & \vdots & & \vdots\\ 1 & x_{n1} & x_{n2} & \cdots & x_{np}\end{bmatrix},\boldsymbol{Y}=\begin{bmatrix}y_1\\ y_2\\ \vdots\\ y_n\end{bmatrix},\boldsymbol{B}=\begin{bmatrix}b_0\\ b_1\\ \vdots\\ b_p\end{bmatrix}$$

似然估计最终得到的解为：

$$\hat{\boldsymbol{B}}=\begin{bmatrix}\hat{b}_0\\ \hat{b}_1\\ \vdots\\ \hat{b}_p\end{bmatrix}=(\boldsymbol{X}^{\mathrm{T}}\boldsymbol{X})^{-1}\boldsymbol{X}^{\mathrm{T}}\boldsymbol{Y}$$

因此，$\mu(x_1, x_2, x_3, \ldots, x_p)$的估计方程为：

$$\hat{y}=\hat{b}_0+\hat{b}_1x_1+\cdots+\hat{b}_px_p$$

此方程称为 p 元经验线性回归方程，简称回归方程。

那么，在给定样本的情况下，我们怎么推测回归函数 $\mu(x)$的形式呢？一方面可以通过专业知识知道 $\mu(x)$的形式，另一方面如图 11-10 所示，可将没对观察值$(x_i, y_i)$在直角坐标系描点得到样本的散点图，借助散点图来粗略估计 $\mu(x)$的形式：线性关系，指数关系或者幂函数关系等。

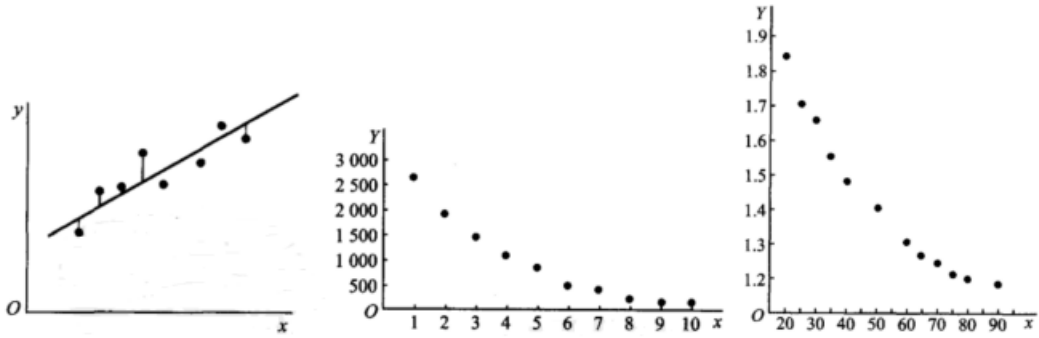



图 11-10 散点图来粗略估计 $\mu(x)$的形式

通过上面的分析，我们知道，线性回归实际上就是找到一组系数，用这些系数与每个输入变量相乘并将结果相加，得出最佳的输出变量预测。

## 11.8 概率论在计算机科学的应用

概率论在社会生产，国计民生中有着非常重要的应用。同时，在计算机信息科学领域，概率论也举足轻重，下面我们就做一些简单的介绍。

### 11.8.1 随机数与密码学

随机数是专门的随机试验的结果，可以分为真随机数和伪随机数。真随机数是使用物理现象产生的：比如掷钱币、骰子、转轮、使用电子元件的噪音、核裂变等等，这样的随机数发生器叫做物理性随机数发生器，真随机对技术要求比较高。

真正意义上的随机数（或者随机事件）在某次产生过程中是按照实验过程中表现的分布概率随机产生的，其结果是不可预测的，是不可见的。

计算机中的随机函数是按照一定算法模拟产生的，其结果是确定的，是可见的。可预见的结果其出现的概率是100%。所以用计算机随机函数所产生的“随机数”并不随机，是伪随机数。

在C语言中是通过rand函数来生成随机数的。而rand函数中有一个定义叫做种子，rand函数是通过对这个种子进行一系列的运算来模拟出一个随机数的。直接调用rand函数，并不指定种子，系统就会调用默认的种子：1，来产生随机数。因为每次重新运行时的种子都是1，运行出来的结果自然就还是这几个数。

为了避免使用默认的1作为种子，可以使用当前时间来作为随机数的生成种子：

```
srand ((unsigned)time(NULL));
int random = rand();
```

真随机具有不可预测性，而伪随机只要知道种子和算法就可以无数次的重复一个随机结果。这些“似乎”随机的数，实际上是通过一个固定的、可以重复的计算方法产生的，产生的随机数有很长的周期性。它们不真正地随机，因为它们实际上是可以计算出来的。

随机数在密码学中有着非常重要的应用，保密通信中大量运用的会话密钥的生成即需要真随机数的参与。如果一个随机数生成算法是有问题，那么会话密钥可以直接被推算出来。

密码学中大量利用伪随机数生成器的应用还有流密码，比如RC4算法，其原理是利用一个密码学安全的伪随机数生成器根据密钥产生一串密码学安全的伪随机比特列，再将消息与上述随机比特列按位异或运算。

真随机数的重要性就在于，完全没有规律。所以一般企业对产品的加密秘钥的生成必须采用真随机数生成器，杜绝被破解的可能性。

随机数的随机性检验可以分为三个标准：

- 统计学伪随机性。在给定的随机比特流样本中，1的数量大致等于0的数量，同理，“10”“01”“00”“11”四者数量大致相等。这类数字在人类“一眼看上去”是随机的。此类随机数叫作伪随机数。
- 密码学安全伪随机性。给定随机样本的一部分和随机算法，不能有效的演算出随机样本的剩余部分。此类随机数叫作密码学安全的伪随机数。
- 真随机性。随机样本不可重现。比如物理现象产生的随机数。此类随机数叫作真随机数。

### 11.8.2 信息论与机器学习中的概率论

概率论在信息论和机器学习中多有着重要而广泛的应用。信息论在信息量，信息熵等重要基础概念都是通过概率论来完成定义的。

# 1.信息量

信息量是对信息的度量，就好比秒是时间的度量一样。对于一个离散的随机变量 **x**，当观察到它的一个具体值的时候，我们接收到了多少信息呢？信息的大小跟随机事件的概率有关。越小概率的事情发生了产生的信息量越大，如中国男足进了世界杯，这是个小概率事件，如果发生了则会差生轰动效应；越大概率的事情发生产生的信息量越小，如太阳从东边升起。

因此一个具体事件的信息量应该是随着其发生概率而递减的。当越不可能的事件发生了，我们获取到的信息量就越大。越可能发生的事件发生了，我们获取到的信息量就越小。那么信息量应该和事件发生的概率有关。

设 **X** 是一个离散型随机变量，其取值集合为 $x$ ,概率分布函数为 $p(x), x \in x$ ,则定义事件 $X=x_0$ 的信息量为：

$$I(x_0)=-\log(p(x_0))$$

由于是概率，所以 $p(x_0)$ 的取值范围是[0, 1]，绘制为图形如图 11-11 所示：

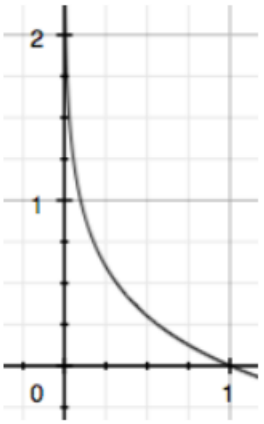



图 11-11 信息量与概率关系

从图中可以看出，概率越小，信息量越大，概率越大，信息量越小。信息量的定义里，取概率的对数的负数，是为了将信息量定义为正值。

# 2.信息熵

1948 年，香农（Claude Elwood Shannon ）提出了“信息熵”的概念，解决了对信息的量化度量问题。信息熵这个词是香农从热力学中借用过来的。热力学中的热熵是表示分子状态混乱程度的物理量。香农用信息熵的概念来描述信源的不确定度。

在机器学习中，熵 (Entropy)，是一个非常重要的概念。信息熵常被用来作为一个系统的信息含量的量化指标，从而可以进一步用来作为系统方程优化的目标或者参数选择的判据。在决策树的生成过程中，就使用了熵来作为样本最优属性划分的判据。

那么什么是信息熵呢？

对于某个事件，有 **n** 种可能性，每一种可能性都有一个概率 $p(x_i)$，这样就可以利用信息量的定义公式，计算出某一种可能性的信息量。而信息熵就是用来表示所有信息量的期望，定义公式如下：

$$H(X) = -\sum_{i=1}^{n} p(x_i) log(p(x_i))$$

比如对于 0-1 分布来说，熵的计算方法可以简化为：

$$H(X) = -\sum_{i=1}^{n} p(x_i) log(p(x_i)) \ = -p(x) log(p(x)) - (1-p(x)) log(1-p(x))$$

## 3.相对熵

相对熵又称 KL（Kullback-Leibler (KL) divergence）散度,如果对于同一个随机变量 x 有两个单独的概率分布 P(x) 和 Q(x)，可以使用 KL 散度来衡量这两个分布的差异。KL 散度的计算公式：

$$D_{KL}(p||q) = \sum_{i=1}^{n} p(x_i) log(\frac{p(x_i)}{q(x_i)})$$

在机器学习中，P 往往用来表示样本的真实分布，比如[0,0,1]表示当前样本属于第一类。Q 用来表示模型所预测的分布，比如[0.1,0.2,0.7]。

## 4.交叉熵

相对熵的计算公式展开就可以得到交叉熵，即 $D_{KL}(p||q)$ 展开如下所示：

$$D_{KL}(p||q) = \sum_{i=1}^{n} p(x_i) log(\frac{p(x_i)}{q(x_i)}) \ = \sum_{i=1}^{n} p(x_i) log(p(x_i)) - \sum_{i=1}^{n} p(x_i) log(q(x_i)) \ = -H(p(x)) + [-\sum_{i=1}^{n} p(x_i) l$$

在最后展开的式子中，前一部分-H(p(x))就是 p 的熵，后一部分就是交叉熵，即：

$$H(p,q) = -\sum_{i=1}^{n} p(x_i) log(q(x_i))$$

在机器学习中，我们需要评估标记值（label，实际的值）和预测值（predicts，模型预测的值）之间的差距，使用 KL 散度即可表示这种差距，即$D_{KL}(y||\hat{y})$，由于 KL 散度中的前一部分−H(y)不变，故在优化过程中，只需要关注交叉熵。所以一般在机器学习中直接用交叉熵作为损耗(loss)来评估模型。

交叉熵在单分类问题上是常用的标准 loss 计算方法：

$$loss = -\sum_{i=1}^{n} y_i log(\hat{y_i})$$

其中$y_i$ 是 label 的值，$\hat{y_i}$是预测的值。

比如单分类问题中，给定一张图片，识别是苹果还是梨还是香蕉。

| | 苹果 | 梨 | 香蕉 |
|---|---|---|---|

| Label | 0 | 1 | 0 |
|---|---|---|---|
| Predict | 0.2 | 0.7 | 0.1 |

那么交叉熵 loss=−(0×log(0.2)+1×log(0.7)+0×log(0.1)）=−log(0.7)

对于二项分布这种特殊的分布，熵的计算可以进行简化，交叉熵的计算也可以简化，即：

$$loss = -ylog(\hat{y}) - (1-y)log(1-\hat{y})$$

### 11.8.3 机器学习与数理统计

所谓机器学习，就是指在一个由训练样本组成的数据集上进行学习训练，得到一个模型，然后应用所得的模型，来判断和预测新的样本。

以计算广告领域的点击率预估为例。为了实现广告收益的最大化，我们一般会优先投放“广告出价*广告点击率”结果较高的广告。那么如何预测一个广告在投放之前它的点击概率呢？这就需要通过机器学习进行预估。

一个待投放的广告是否会被点击，可以通过概率来预测。而影响这个广告是否被点击的因素很多，总的来说分为三个方面：广告自身内容形式；广告展现形式和位置等场景信息；用户的画像，包括性别、年龄、职业等。

为了预测所投放广告的点击率，需要首先搜集有关广告近期的点击日志数据。每条日志里应该包含该广告是否被点击的标记（label），以及广告的信息，广告展示的场景信息，用户的画像信息等等。

然后基于这些日志进行模型训练。模型训练的过程，其实就是数理统计和回归的过程，即估计因变量点击率 p 与自变量广告内容，广告场景以及用户画像等等之间的关系。常用的模型算法包括：

#### 1. 线性回归 LR(linear regression)

LR 是一种简单高效的线性回归模型算法，视广告特征空间内特征之间彼此独立。LR 的回归模型为：

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^{n} w_i x_i .$$

其中 $\mathbf{x}_i$ 是广告内容，广告场景以及用户画像（比如年龄，性别，职业等）的特征编码，$\omega_i$ 是需要通过机器学习训练得到的于 $\mathbf{x}_i$ 相关的未知参数，$\omega_i$ 的计算方法请参考本书 10.7.1 一节的最小二乘法，$\hat{y}(\mathbf{x})$就是根据 $\omega_i$ 和 $\mathbf{x}_i$ 计算得到的预估结果，在[0, 1]之间的一个点击概率。

#### 2. FM（Factorization Machine）

LR 没有任何交叉或者组合关系，因此必须复杂到能够建模非线性关系才能够比较准确地建模复杂的内在关系。FM 通过特征的二项式组合来建模这类特征的复杂的内在关系，将二项式矩阵做矩阵分解，把高维稀疏的特征向量映射到低维连续向量空间，然后根据内积表示二项式特征关系。FFM 的回归模型如下：

$$\hat{y}(\mathbf{x}) := w_0 + \sum_{i=1}^{n} w_i x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} \langle \mathbf{v}_i, \mathbf{v}_j \rangle x_i x_j$$

对于参数矩阵 W，采用矩阵分解的方法，将每一个参数 $w_{ij}$ 分解成两个向量（称之为隐向量）的点积。这样矩阵就可以分解为 $W=V^TV$，而每个参数 $w_{ij}=(v_i, v_j)$，$v_i$ 和 $v_j$ 分别是第 i 维和第 j 维特征的隐向量，$\langle v_i, v_j \rangle$表示 $v_i$ 和 $v_j$ 的点积。

除此以外，还可以选择将 FM 于深度学习相结合的一些模型算法，比如 DNN 等。

## 11.8.4 布隆过滤器中的数学运算与推导

布隆过滤器可以用于判断一个数据元素(每个数据元素都有一个关键字 key)是否在一个集合中。该方法于 1970 年由布隆（Burton Howard Bloom）提出，因此以他的名字命名。布隆过滤器可以用在下面这些场景：

词典服务中，查询某个单词是否拼写正确（是否存在字典里）；

反垃圾邮件爬虫服务中，检查某个网页是否已经收录；

推荐服务中，检查内容是否已经被推荐过；

设计带有缓存的服务架构中，过滤无效查询等。

为了判断一个元素在集合中是否存在，传统的做法是这样的：将集合中全部的元素存在链表，树，hash 表等数据结构中，遇到一个新元素，遍历这些数据结构，将它和集合中的元素直接比较即可。问题是随着集合中元素的增加，需要的存储空间越来越大，检索速度也越来越慢。即使是 hash 表 O(1)的查找复杂度，但由于负载因子的原因，通常空间是不能被用满的，存储效率一般只有 50%左右，在数据很多时，也会占用大量的存储空间。

布隆提出的方法，改进了这种传统的做法。布隆过滤器的步骤是：

首先，分配一个长度为 n 比特(bit)的数组，每个比特位初始化为 0；

然后，定义 k 个 hash 函数，通过 hash 函数把 key 分别散列成为 k 个整数；

紧接着，当某个 key 加入集合时，用 k 个 hash 函数计算出 k 个散列值，并把数组中对应的比特位置为 1；

最后，当判断某个 key 是否在集合时，用 k 个 hash 函数计算出 k 个散列值，并查询数组中对应的比特位，如果所有的比特位都是 1，认为可能在集合中（存在一定的误判率）；只要有一个 bit 不是 1，则一定不在集合中。

比如，我们要在一个集合里存放一些名车品牌 benz（奔驰），bmw（宝马）和 audi（奥迪），然后判断这个集合里是否存在某款名车。那么，首先初始化一个 bit 数组（bit 长度为 8），如图 11-12 所示：

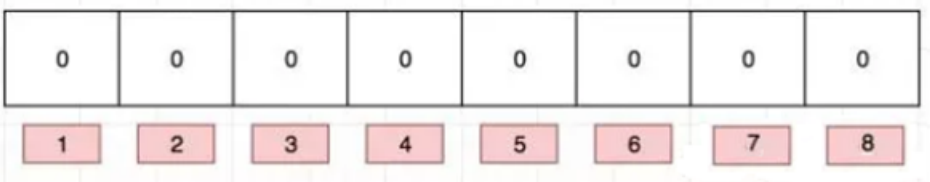



图 11-12 bit 数组

现在把 benz 放进这个 bit 数组，这里我们假设用到了 3 个 hash 函数，分别计算出来的结果为 1, 4, 7，那么 benz 计算完成之后，bit 数组的情况如下（在 1, 4, 7 这三个位置被置 1 了），如图 11-13 所示：

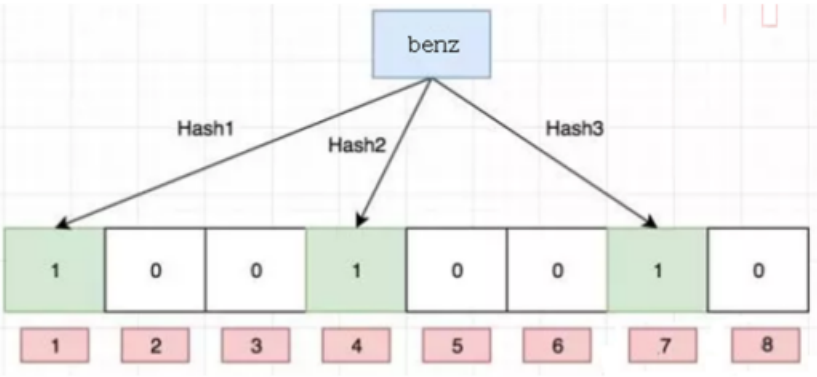



图 11-13 benz 放入数组

接着将 bmw 放进这个 bit 数组，3 个 hash 函数对 bmw 这个 key 计算出来的位置为 2，4，8，那么 bmw 计算完成后，bit 数组的情况如下（2, 4, 8 这三个位置被置 1 了），如图 11-14 所示：

图 11-14 bmw 放入数组

现在假设我们要查找 audi 在不在这个集合里，那么利用这 3 个 hash 函数，计算出来的结果是 1, 5, 7，其中我们发现在 5 这个位置，bit 数组里的值为 0，因此 audi 肯定不在这个集合里，引入如果 audi 在这个集合里，肯定会同时把 1, 5, 7 这三个位置置 1 的。

如果查 benz 在不在这个集合，那么利用 3 个 hash 函数，得到的结果是 1,4,7，在 bit 数组里，三个位置都是 1，那么就代表 benz 一定在这个集合里吗？非也。虽然刚才的初始化时，我们确实把 benz 放进了这个过滤器里。但是，如果三个位置都是 1，不代表 100%这个数据一定在集合里。那是因为 hash 函数是有 hash 冲突的。比如在上面，当把 benz 和 bmw 初始化 bit 数组的时候，都同时把 4 这个位置置 1 了，这就是 hash 冲突造成的。所以当查出 benz 在所有位置都为 1 的时候，有可能 benz 确实存在集合中，但也有可能是因为 benz 所在的这些位置由于 hash 冲突而造成的（当然这种概率会比较小，一会计算）。我们把这种结果都为 1，但数据不在集合中的现象叫作误判或者假阳（False Positive）。

布隆过滤器的优点是不需要为了查找而额外存储 key，节省空间；Hash 查找的高效率；安全，因为整个过滤器里不会保存 key，不会涉及到信息泄露的问题。缺点是存在一定的误判率，随着存入的元素数量增加，误判率随之增加。克服误判的方法可以通过设置一个白名单将这些误判的结果进行修正；只能加入数据，但不能删除，这是因为 bit 数组里的数据只能是 0 和 1，所以可以将 0 和 1 改进为一个引用计数来提供删除的功能。

现在我们通过数学的方法，来研究一下如何使得布隆过滤器中的假阳率最低。这里主要用到了概率论和导数求极值等相关的理论。

设位数组的大小为 m ， Hash 函数的个数为 k ， 数据元素个数为 n。现在我们来研究下这三者之间在什么条件下，可以使得假阳概率最低。

Hash 函数每次以等概率选择并设置位数组中的某一位，则每一位被设置 1 的概率是：$\frac{1}{m}$，那么位数组中某一特定的位在进行元素插入时的 Hash 操作中没有被置位（设置为 1）的概率是：$1-\frac{1}{m}$。

k 次 Hash 操作后该位都没有被置 “1” 的概率是：$\left(1-\frac{1}{m}\right)^k$ 。插入了 n 个元素，那么某一位仍然为 “0” 的概率是：$\left(1-\frac{1}{m}\right)^{kn}$，因而该位为 “1”的概率（假阳性）是：

$$1-\left(1-\frac{1}{m}\right)^{kn}$$

某个元素是否在集合中所需的 k 个位置都按照如上的方法设置为 “1” 的概率：

$$\left(1-\left[1-\frac{1}{m}\right]^{kn}\right)^k$$

根据极限公式：

$$\lim_{n\to\infty}(1+\frac{1}{n})^n=e$$

有：

$$[1-(1-\frac{1}{m})^{kn}]^k=[1-(1-\frac{1}{m})^{-m\frac{-kn}{m}}]^k\approx[1-e^{\frac{-kn}{m}}]^k$$

可见，假阳概率会随着 n 的增大而增大。令$a=e^{\frac{-n}{m}}$,则有布隆过滤器的假阳率函数与 k 的函数关系：

$$f(k)=(1-a^k)^k$$

要使假阳率最小，就是要求上述函数的极小值。根据微积分中最小值求法，求函数 f(k) 的导数，并求方程 f’(k)=0 的解，即是所得的最小值。为了让函数求导方便，先对函数两边取对数，有：

$$lnf(k)=kln(1-a^k)$$

然后两边同时求导有：

$$\frac{1}{f(k)}f'(k)=ln(1-a^k)-\frac{ka^klna}{1-a^k}$$

令 f’(k)=0，于是有：

$$ln(1-a^k)-\frac{ka^klna}{1-a^k}=0$$

解此方程，有：

$$k = \frac{mln2}{n} = 0.7\frac{m}{n}$$

也就是说，当$k = \frac{mln2}{n} = 0.7\frac{m}{n}$时，布隆过滤器的假阳（误判）概率最低。把该值代入 f(k)，假阳概率 p 的最小值为：

$$p = [1 - e^{\frac{-kn}{m}}]^k = (1 - e^{ln\frac{1}{2}})^{\frac{mln2}{n}} = (\frac{1}{2})^{\frac{mln2}{n}} \approx 0.6185^{\frac{m}{n}}$$

也就是：

$$p = (\frac{1}{2})^{\frac{mln2}{n}} = (\frac{1}{2})^k$$

另外，当假阳概率最小值一定的情况下，根据上面 p 的计算等式，那么位数组的长度为：

$$\mathbf{m = -\frac{nlnp}{(ln2)^2}}$$

从布隆过滤器中的参数优化推导，可以看出概率论与微积分的重要应用，它们是我们最优化布隆过滤器的重要理论基础。

# 十二，区块链中的数学与加解密算法

区块链（blockchain）起源于比特币，作为比特币的底层技术之一，与比特币一起问世。

2008 年 11 月 1 日，一位自称中本聪(Satoshi Nakamoto，日本翻译为中本哲史)的日裔美国人发表了《比特币:一种点对点的电子现金系统》一文 ，阐述了基于 P2P 网络技术、加密技术、时间戳技术、区块链技术等的电子现金系统的构架理念，这标志着比特币的诞生。

2009 年 1 月 3 日第一个序号为 0 的创世区块诞生。2009 年 1 月 9 日出现序号为 1 的区块，并与序号为 0 的创世区块相连接形成了链，标志着区块链的诞生。

如果用一句话概括区块链的话，那么区块链就是一种去中心化的分布式账本(distributed ledger)数据库。但想要更好的理解区块链技术，还得基于数学理论和加解密技术来深入分析区块链的底层原理。接下来我们就基于数学理论和加解密算法详细介绍一下区块链的内部原理以及它所解决的问题。

## 12.1 中心化及问题

一种新技术的出现，肯定是为了解决老技术中存在的一些问题。那么区块链的出现，是为了解决什么问题呢？

目前，我们的各种交易和资金往来，是通过一些权威的中间机构背书或者担保来完成的。比如银行，支付宝等。这就是交易的中心化。下面我们举几个生活中的例子：

- 转账：A 要转账 100 元给他的朋友 B，那么 A 就向自己的开户银行发送一个转账指令，银行收到指令之后，向 B 发起转账，然后 B 收到了 100 元。这个过程中的银行就是这次转账的中心或者中介。
- 购物：A 要在淘宝上从商家 B 处购买一包小饼干，那么 A 也不能直接把钱打给 B，而是需要先把钱临时存放在支付宝，然后 B 发货给 A，A 收到货之后，支付宝再把钱打到 B 的账户。这个过程中，支付宝也是这次交易的中心或者中介。
- 记账：比如一个公司，将所有的交易记录统一交给一个会计负责记账在一个账本上。账本只有一个，这也是中心化的记账方法。

此外，像滴滴打车，网络订餐等都是基于这种中心化的模式来完成交易的。

在中心化模式里，参与交易的第三方对交易起着非常重要的作用，一方面它具有一定的权威性和信任度，但如果第三方出了问题，那么将会严重影响整个交易流程。比如第三方收了交易费跑路了或者倒闭了怎么办？第三方网络瘫痪了怎么办？第三方账本丢失了怎么办？第三方的账本被篡改了怎么办？第三方的设备故障了怎么办？第三方的工作或转账效率低怎么办（比如一笔跨国转账的等待时间可能要好几天）？

像银行这种中心机构，还可能超发货币，导致通货膨胀，货币贬值，金融泡沫，进一步诱发金融危机等。

因此虽然中心化交易是目前普遍采用的机制，但也确实存在着各种现实问题。而区块链技术的发明，就正好可以通过去中心化的方法来解决这些问题。

## 12.2 去中心化与分布式记账

区块链技术的第一个典型的特征就是去中心化。在前面我们已经认识到了中心化框架下

的交易存在各种问题。所以，如果能把其中的第三方从交易中去掉，通过去中心化的方法让交易双方直接交易并能够安全的完成，这就是一个非常重要的改变。

我们依然以记账为例子。比如一个村子，在中心化时代，全村村民的交易记录，全部委托给村长一个人负责记录在一个账本上。比如：村民 A 借了 100 元给村民 B；村民 C 归还了村民 D 50 元钱，这些记录都由村长负责记账，以后谁借了谁的钱，谁还了谁的钱，就靠这个账本来查清。这种情况下是有很大的风险的，比如村长的账本某天丢失了，或者账本被人恶意修改了等等。那么全村的账目就混乱了，谁也说不清楚谁欠谁的钱了。

现在我们如果引入了去中心化的区块链之后，那么就改变了之前的方法，把记账的这个权力下发给每个村民，即每个村民都有一个账本，负责记录全村中每个交易记录。比如村民 A 同样借了 100 元给村民 B，那么村民 A 就会发布一个全村人都能听见的公告，公告的内容就是：我是村民 A，我借了 100 元钱给村民 B。然后每个村民都往自己的账本上记录下这些信息。这样全村的账务信息，在每个村民的账本上都有完整的记录。

在这种情况下，即使大部分村民的账本被丢失了，只要有一份存在，就可以还原整个村子的账务信息。同样，也不怕账本被恶意篡改。因为每个人都有一个账本，如果你只修改了自己的账本信息，但你无法修改其他村民的账本，因此把所有的账本放在一起对比，就会发现被修改的账本，那么这个账本也就无效了。

在这个过程中，可能有人会问，当某个村民将转账借款信息广播出去之后，所有村民为什么就会主动把这条信息记录在自己的账本上呢？这里会涉及到所谓的记账奖励机制。即第一个听到广播并记录下来的村民将会获得一些奖励（比如一定数量的比特币奖励，这些奖励可以是系统给的，也可以是村民在发布这条记录时给的记账酬劳）。而获奖者记录了这条信息之后，要马上告诉其他人，让别人放弃继续记录，并将这条信息加上 id 编号（id 编号在所有账本上都是从小到大递增，并保持一致编号）广播出来，那么其他人就会把这条已经被记录带上编号的信息拷贝到自己的账本上，所有的信息按照 id 编号顺序放在一起，实际上就相当于一个用于记账的链，一环扣一环。于是，通过这种奖励的机制，那么系统中肯定就会有一批人抢占第一个记账的权利。

这就是区块链去中心化机制的核心思想（当然区块链的具体实现要比这个例子更复杂一些），去中心化机制把原来处于中心的村长记账的权力和唯一的账本，改为了每个村民记账并拥有独立的账本。这种去中心化的机制，是靠着分布式网络系统来实现的。那么什么是分布式网络呢？

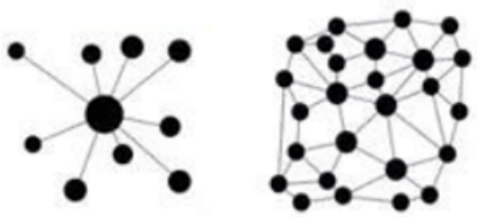


图 12-1 中心化（左）与去中心化（右）

如图 12-1 所示，图中左边是一个中心化模型。其中心位置的结点就是核心，其他结点都是靠着中心结点来完成各种交易或者记账任务的，所有的信息都是存放在这个中心结点。而图中右边就是一个点对点(p2p)的分布式网络结点图。在这个分布式网络环境中，就再也没有中心结点，每个结点都是对等的，每个结点都存放着完整的交易信息。同时在分布式网络里，每个结点也可以向所有其他结点发送广播通知。当一个结点以平等的身份加入这个分布式网络之后，就可以从离它最近的结点同步和更新所有记账数据。

需要注意的是，在中心化模式下，只要中心足够权威，比如有大公司甚至有国家出面背书，像支付宝和银行这种，那么普通人还是可以信任的。但现在这个分布式网络由于没有了

中心，所以是不安全的。也就是说，这个网络里的某些结点，有可能是坏的。这个坏的结点可以发布虚假信息，修改本地的账本数据等恶意行为。因此，光有分布式网络还不行，还需要解决这个网络信息的可靠性和安全性问题。

具体来说，通过分布式网络去中心化之后，会面临如下几个典型的安全问题：

**1.两军问题**

由于在古代没有先进的通信工具比如电话或互联网，因此两军之间只能通过信使进行信息交换。如图 12-2 所示，A、B 两支友军（Blue Army，蓝军）准备进攻位于两军之间的一座敌军（White Army，白军）驻守的城池，而这座城池位于 A、B 两只友军信使通信的必经之路，由于 A、B 友军中单军进攻成功率极小，因此需要确保进攻同时性。为了达到同时进攻的目的，A 军派出一名信使偷偷经过敌军阵地去 B 军并告诉 B 军进攻时间：比如 6 月 6 日凌晨。

信使派出去后，A 军并不知道 B 军是否收到了这个消息，因为信使有可能会被敌军捕获。这种不确定性使得 A 军不敢在 6 月 6 日凌晨发起进攻，因为有孤军奋战而导致失败的危险。如果信使最后成功达到 B 军阵地，并将进攻时间告诉了 B 军，为了让 A 军放心，B 军需要再派遣一个信使发送一个确认的信息给 A 军：“消息已收到，我会在 6 月 6 日凌晨发起攻击”。可是，这个给 A 军的确认消息也面临着被敌军截获的可能，B 军也会犹豫，如果 A 军没有收到确认信息，那么 A 军很有可能停止此次攻击。

这样一直循环下去就形成了一个无解的难题，两军无论派多少信使都不能完全保证对方与自己保持一致。

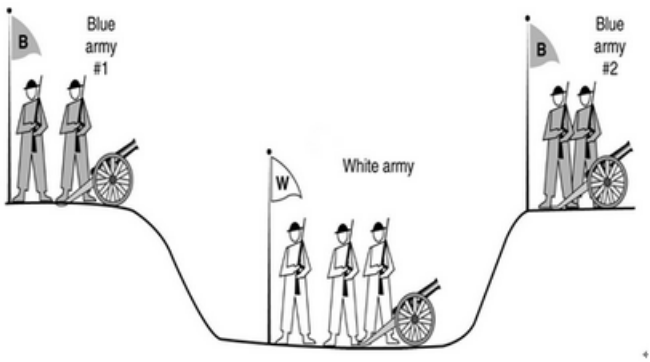



图 12-2 两军问题

两军问题的根本问题在于传递消息的信道不可靠，如果信道是可靠的，两军问题就迎刃而解。然而，并不存在这样一种信道。也就是说，在分布式计算上，试图在异步系统和不可靠的通道上达到一致性是不可能的。

**2.拜占庭将军问题**

拜占庭位于如今的土耳其的伊斯坦布尔，是古代东罗马帝国的首都。拜占庭将军问题（Byzantine failures），是由莱斯利·兰伯特提出的点对点通信中的基本问题。这个问题是这样描述的：如图 12-3 所示，拜占庭罗马帝国在军事行动中，如果要进攻一个强大的敌人，将会派出多支军队包围这个敌人。由于敌人很强大，所以单支军队很难取得胜利，于是采取将军投票的策略来决定是进攻还是撤退，也就是说如果多数人决定进攻，就发起进攻。但是军队中如果有叛徒，比如已经反水的将军故意乱投票，或者传令官叛变擅自修改军令，那怎么保证最后投票的结果真正反映了忠诚的将军的意愿呢？这就是拜占庭将军问题。

这个问题反映到信息交换领域中来，可以理解为在一个去中心的系统中，有一些节点是坏的或者恶意的，它们可能向外界广播错误的信息或者不广播信息，在这种情况下如何验证

数据传输的准确性。需要注意的是，拜占庭将军问题假定了信道是可靠的，并在这个前提下，去做一致性和容错性相关研究。如果需要考虑信道的不可靠性，这涉及到上面所讲的“两军问题”。

图 12-3 拜占庭将军问题

现在我们重新回到区块链的分布式记账中来，看看上面两个问题在区块链中的具体体现以及解决办法。

首先我们来看看两军问题在区块链中对应的问题。由于信道是不可靠的，那么分布式网络中发出的信息，有可能在传输过程中发生丢失或者延时，那么其他节点接收到的信息就会出现不一致。而我们已经知道，当有人在网络中发出交易记录广播之后，由于存在记账奖励，因此所有人都争着抢夺记账权。那么很有可能有 2 个人同时号称抢到了记账权，并给这条交易记录分配了编号（带编号的记录，我们可以称之为区块），然后再把编号和记录也就是区块广播告诉别人，是自己抢到了记账权，你们赶快来把我的这条记账记录和编号复制进你们的账本吧。这种情况下，其他人中有的可能同时听到了这两条广播，有的可能只听到了其中一条广播，有的可能先听到其中一条广播后听到了另外一条广播。那么出现这种情况怎么办呢？

在区块链里，解决这个问题有两条思路。

首先是提高抢占记账权的难度或者叫增加记账的工作量从而控制记账发生的频率，这样记账付出的工作量大了之后，记账的速度就会慢下来，比如比特币区块链一般 10 分钟左右才产生一次记账（也就是生成一个区块），这样就让这种同时抢到记账权的概率大大降低，这就是所谓的工作量共识(proof of work，简称 POW)机制，抢占记账权限生成新区块的过程我们称之为挖矿，后面 12.4 一节还会详细介绍。

在提高记账难度和工作量从而降低记账频率之后，如果还是因为各种原因出现了两个人或者多个人同时抢到了记账权力，那么就允许所有人在接收到这些记账人发出的带编号记账信息（也就是区块）的广播之后，同时把这些信息记录在自己的账本上，这样记账信息在账本上就会出现分叉，如图 12-4 所示。

图 12-4 区块链分叉

分叉的出现，就很难保持账本的一致性了，也就是每个人的账本上的信息出现了不一致性。这肯定是我们不想要的结果。那么怎么办呢？区块链引入了一个新的共识机制，即“最长链共识机制”，即只保留账本中最长的链，抛弃短链。什么意思呢？就是允许账本中的链短暂分叉，然后每个人继续监听网络中的广播，继续抢夺记账权。这样，到一定时间之后，就会在网络中出现有的链长，有的链短。链长代表了多数人的共识，那么少数服从多数，因此自然就只保留了链长的记录，那么账本信息重新达到了一致性。

如果有人想对交易记录造假，那么他就需要一直抢先生成包含假的交易记录的区块，并广播出去。这就要求造假者生产新的区块的速度，要快于系统中的其他所有用户生成新的区块的运算能力之和，然后链接在自己之前生成的假区块之后，造假才有可能成功。但由于每个用户用的算力相比系统总算力都是九牛一毛，微不足道的，这就使得工作量巨大，造假成本巨大，大到难以接受，因此造假变得不可能。这也告诉我们，在这个去中心化的点对点分布式系统中，参与的结点越多，总的运算能力越高，那么对抗作假的能力就越强，系统就越安全。

因此两军问题造成的系统不一致性就通过最长链共识得到了解决。由于区块链的最长链共识也能够避免记账记录造假，因此也同时解决了一部分拜占庭将军问题。

前面我们已经提到，现在的去中心化分布式网络是一个非安全的网络环境，那么如何保证：

- 每个结点接受到的这条交易记录在传输过程没有被其他人所篡改；
- 每个结点接受到的这条交易记录确实是由发起交易的人所创造；

这就需要我们前面讲的密码学算法来解决相关问题了。事实上，区块链里是通过 Hash 散列算法 SHA256 结合椭圆曲线 ECC 非对称加解密算法来保证信息的真实可靠性。同样以村中记账过程为例，具体来说就是如下步骤：

首先，村民 A 说了一句话：“我是村民 A，我借了 100 元给村民 B”，并将这句话用 SHA256 函数对自己的这句话进行计算，得到一个固定长度的输出字符串，记为 str1。

然后，村民 A 使用只有自己知道的 ECC 私钥，对 SHA256 运算结果的固定长度的字符串 str1 进行加密，密文就是一份数字签名，这份数字签名能够充分证明这句话就是村民 A 说的，而且说的内容也是真实可靠的。这非常类似于现实中我们的合同盖章签字一样，只有盖章和签字了的合同，合同才是真实有效的。

紧接着，村民 A 将“我是村民 A，我借了 100 元给村民 B”的明文，这条明文的数字签名，以及自己 ECC 密钥对中的公钥打包一起发给网络中的其他人。

最后，其他村民，比如村民 B 接收到村民 A 的打包数据后，首先会将打包中的明文进行 SHA256 计算，输出一个固定字符串，记为 str2（如果明文没有改动，那么 str2 应该和村民 A 最开始调用 SHA256 计算的字符串 str1 相等）。然后，村民 B 使用村民 A 公布的公钥，

对发过来的数字签名进行解密（非对称加解密的特性，私钥加密的数字公钥可以解密，公钥加密的数据，私钥可以解密），得到另外一个固定字符串，记为 str1（这个字符串就是村民 A 自己生成的 str1）。这时通过比对 str1 和 str2 的一致性，便可确定信息的真实性，即如果 str1 等于 str2，那么村民 B 接受到的信息就是村民 A 发出来的信息，这中途没有被其他人所篡改，因此可以确认该信息确实是由村民 A 所发出来的（因为是通过村民 A 的公钥解密的，在网络世界中，村民 A 的公钥就代表了村民 A）；如果 str1 不等于 str2，那么村民 B 接收到的信息就是被篡改之后的数据，是不可靠的信息，应该丢弃。

所以，我们可以看到，所谓的拜占庭将军问题，实际上是通过最长链共识以及密码学算法来解决的。因为密码学的技术保证了没有人能够篡改或者伪造交易信息，从而避免了“叛徒或奸细”的出现。用数字签名保证了不能篡改他人的交易记录，用区块链及工作量证明保证了不能在区块链中加入自己的伪造交易。通过这两点，就完成了：证明区块链的每一条交易记录都是可信的这一终极目标。

现在我们再来看看区块链中的转账交易过程。

区块链中的一个典型的应用就是比特币。那么如何在区块链上进行比特币转账操作呢？区块链上的转账操作一般分为如下几步：

1．判断余额是否足够完成交易

比如 A 想转账 100 个比特币给 B，那么和现实中的转账一样，首先 A 的账户余额里起码得大于或等于 100 个比特币才能发起转账功能。但在比特币为代表的区块链里，并没有所谓余额这个概念。因此，所谓的余额，实际上来源于每个人所拥有的各种交易记录。

也就是说，你所拥有的余额，实际上是来源于别人给你的转账。比如在 A 的全部有效交易记录里有：

- C 给 A 转账：70
- D 给 A 转账：10
- E 给 A 转账：40

那么把这 3 条交易记录涉及到的金额加在一起，就是 A 目前拥有的余额：70+10+40=120。由于大于 100，所以 A 可以给 B 转账 100。

2．判断是否需要找零

如果 A 根据交易记录计算出的余额恰好等于要转账的金额（注意，还有可能包含交易手续费用），那么就可以不用找零，否则，需要将余额中减去转账部分的金额，重新转账给 A 自己。比如 A 现在的余额是 120，转账 100 给 B，那么剩余的 20 则由 A 自己转账给自己，完成找零。

3．广播出去，让全网结点认同和备份转账记录

最后，A 把“A 转账 100 给 B”和“A 转账 20 给 A”与加密签名信息等一起广播出去，等待网络中的某个人来打包记账。在未完成记账之前，发出的交易信息会被标记为“未确认”，一旦被人完成记账加入区块链末尾之后，就会被标记为“确认”，一般某笔交易被全网广播以后，达到 6 个确认以上就成功记录在案了，且不可逆转不可撤销。若这条记录被当做其它交易的余额而被使用，那么这条交易信息会被标记为“无效”。而之前的转账记录：

- C 给 A 转账：70
- D 给 A 转账：10
- E 给 A 转账：40

将会被标记为无效记录，因此这些记录将永远不再会被追溯到。

需要注意的是，在区块链里，每个人的信息都是加密的，接收转账的地址和公钥信息也都是匿名的。没有人可以通过公钥或接收转账的地址知道这个公钥或者地址的主人是谁。这就是区块链的匿名性。

通过上面的转账过程，我们可以看出，拥有交易记录等同于拥有数字货币，区块链里记录的公共账本就是货币（The Ledger is the currency）。

## 12.3 区块与区块链

上一节，我们详细介绍了区块链的去中心化与分布式记账原理。现在，我们再来深入研究一下这些原理的底层实现。

实际上我们前面提到的，A 从自己所在的结点，把一条交易记录广播到这个去中心化的分布式网络之后，其他所有争抢记账的结点并不会马上对这一条单独的记录进行记账，而是不断搜集一段时间内（比如比特币区块链是 10 分钟之内）发布到网上的所有交易记录，然后一起打包统一记账。每个结点都在进行同样的搜集交易记录信息并打包记账工作，但最终只有一个结点能够真正完成打包记账。

这段时间中所有的交易记录，会被统一打包记录和存放在一个所谓的区块里。而区块链就是由这些区块按先后顺序链接而成。

要理解区块链的底层实现原理，首先我们需要先了解区块的组成。如图 12-5 所示，每一个区块总体上由区块头和区块体两个部分组成。在区块头包含的一些重要元素为：前一区块哈希值，随机值 Nonce 和 Merkle 树的根哈希值等。区块体部分则存放着交易记录和对应的 hash 值，并通过 Merkle 树组织在一起。

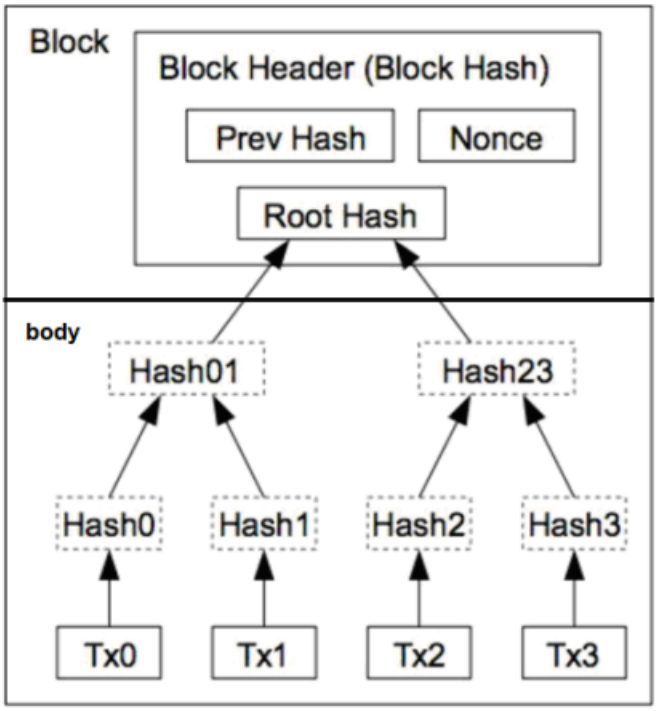



图 12-5 区块构成

所有区块组成的区块链如图 12-6 所示：

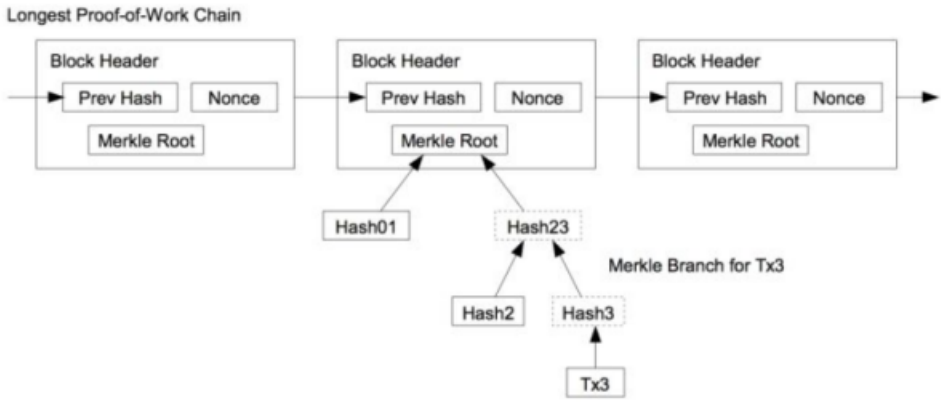



图 12-6 区块链构成

在 https://www.blockchain.com 上看到的一个具体的比特币区块信息如图 12-7 所示：

| | |
|---|---|
| Hash | 00000000000000000009a1a5608f7b6a8f4b2d9249059451a93bef7260e421b1 |
| Confirmations | 11 |
| Timestamp | 2020-02-25 14:47 |
| Height | 618898 |
| Miner | F2Pool |
| Number of Transactions | 1,977 |
| Difficulty | 15,546,745,765,529.37 |
| Merkle root | 492e2b9b16be2252d075e775ff04c78087fd5109fd81ea1a9265e011fc902a63 |
| Version | 0×3fffe000 |
| Bits | 387,062,484 |
| Weight | 2,405,907 WU |
| Size | 771,033 bytes |
| Nonce | 322,459,559 |
| Transaction Volume | 4933.82109305 BTC |
| Block Reward | 12.50000000 BTC |
| Fee Reward | 0.09156139 BTC |

图 12-7 一个具体的比特币区块信息

现在我们来详细介绍这些数据元素的含义以及它们之间的关系。

- **Hash**：上一个区块的 Hash 值。在每个区块的头部，保留着它前一个区块的 Hash 值，然后所有区块通过这种形式组成了一个区块链。而区块 hash 值的计算却非常讲究，涉及到挖矿过程，这个我们在后面讲 Nonce 的时候再一起介绍计算方法。
- **Height**：区块高度

区块高度就是在区块链中，当前区块是第几个区块，可以看做是区块的序号。Height 为 0 的区块就是第一个区块，即所谓的创世块（genesis block）。在比特币上，这个区块是由区块链的发明人中本聪于北京时间 2009 年 1 月 4 日，在芬兰赫尔辛基的一台小型服务器上创建。

- **TimeStamp**：区块创建时的时间戳。
- **Tx Count**：区块包含的交易总数，具体交易信息保存在区块体中。
- **Bits、Difficulty 和 Nonce**：Bits 为压缩格式的当前目标(target)的 hash 值，Difficulty 为区块难度系数，Nonce 为一个随机数。这三个字段用于控制记账难度和频率的参数，我们将在 12.4 一节给大家详细介绍。
- **Miner**：获得记账权的矿工，也就是负责记账的人，此区块由矿工负责打包生成。
- **Number of Transaction**:该区块包含的交易记录数。
- **Version**:区块版本号。
- **Size**：区块的大小，以字节 B 为单位。

目前区块链中的区块大小最多支持 1MB 的存储空间。最初中本聪在创建区块链的时候区块的容量是 32MB，但是他通过一个说明为”Clear up“这样毫无明确意义和目的的代码提交 Commit 中把区块容量改成了 1MB，以防止区块链体积增长过快。

由于每一个区块最大只能是 1MB，一条交易记录大概是 0.25KB，那么一个区块最多可以储存 4000 多条交易记录，如果在一个新区块产生的时段中，发生的交易请求超过 4000 条，那就肯定存不下了。我们可以算算，4000 条交易这个量大概是一个怎样的交易频率。由于每个区块的平均产生时间是 10 分钟即 600 秒，也就意味着，平均每秒的交易量不能超过 7（4000/600≈7）条。如果超过 7 条，那么就一定会出现排队等待打包的交易记录了。

这个交易频率实在很低，要知道支付宝一秒钟大约要处理上万笔交易。这一秒钟七笔交易对于全球来说，实在是太不够用了。

- **Weight**:区块重量，区块计费指标。

最大区块大小为 1 MB，限制了比特币每秒能处理的交易笔数为 7。这会限制比特币的增长潜力，并阻碍其成为一个有用的高体量的支付系统。于是区块链引入了所谓的“隔离见证(segwit，Segregated Witness 的缩写)”，它是改变数据存储方式的一种协议升级，以使得比特币区块能够处理更多的交易笔数。隔离见证去除了区块中交易记录的签名信息(或者也被称为“见证人”信息)并将其存储在基本交易块(base transaction block)之外的隔离见证块中（segwit block，当然基本交易块也属于隔离见证块的一部分），这样在区块内部就可以容纳更多的交易记录。

隔离见证引入了一个新的观念，称为“区块重量(block weight)”，它是由有签名数据（segwit 数据）和没有签名数据（基本交易块中数据）的区块尺寸的混搭，其容量被设定在 4 MB，而基本交易区块的大小则保持 1 MB。这意味着隔离见证的升级能与先前的协议兼容。那么 weight = 3*基本交易块中所有交易记录的非签名信息大小之和+所有交易记录中的签名信息大之和。

- **Transaction Volume**:区块中所有交易的总金额。
- **Block Reward**：生成一个区块（挖矿）记账人（也就是矿工）所获得的奖励，早期是 50 个比特币，现在是 12.5 个比特币。
- **FeeReward**：交易手续费，也属于对矿工的奖励，当以后生成区块的奖励趋于零之后，此为记账人的主要收益。
- **Merkle Root**：即 Merkle Tree 的根哈希值(Root Hash)。

Merkle 树是一种哈希二叉树，由一个根结点(root)、一组中间结点和一组叶结点(leaf)组成。叶结点（leaf）存储数据或其数据的哈希值，中间结点是它的两个孩子结点内容的哈希值，根结点也是由它的两个子结点内容的哈希值组成。所以 Merkle 树也称哈希树。它可以用来进行快速查找和检验大规模数据完整性。对于比特币网络来说，使用 Merkle 树来存储交易信息的目的是为了高效的查找和校验某笔交易的信息是否存在。

当 N 个数据元素经过加密（使用两次 SHA256 算法，也称 double-SHA256），至多计算 logN 次就能检查出任意元素是否在树中。

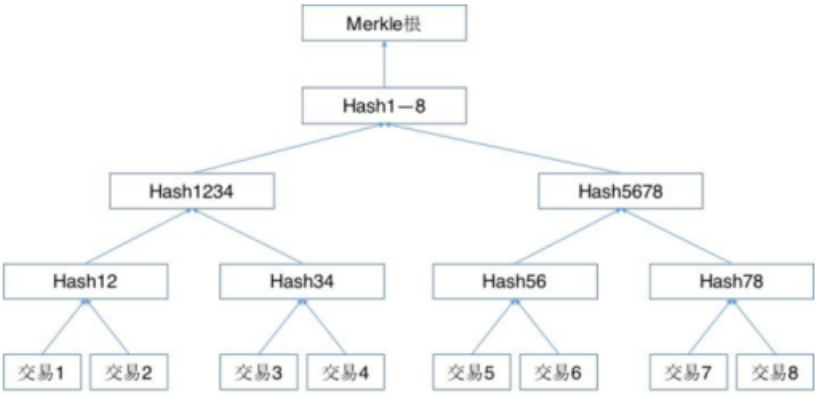



图 12-8 Merkle 树结构

如图 12-8 所示，Merkle 树中的叶子结点为交易 1 到交易 8，记录每一笔交易数据。Hash12 是交易 1 和交易 2 的哈希指针，记录着交易 1 与交易 2 共同计算出的哈希值；Hash34 则是 交易 3 和交易 4 的哈希指针，记录着交易 3 与交易 4 共同计算出的哈希值；而 Hash1234 则是 Hash12+Hash34 的哈希指针，保存的是 Hash12 与 Hash34 对应的哈希值。这样层层叠加上去，得到最后的根，即 Root Hash。Merkle Tree 的叶子结点是数据块的 Hash 值，而非叶子结点的值则是根据下面的所有叶子结点的 Hash 值组合后再计算 Hash 得出的。

通过 Merkle 树结构，底层的每一笔交易都无法篡改，因为一旦篡改，那么 Root Hash 就必然不一致。任何人就不能随意修改其中的内容，或者交换顺序。如果你这么做，意味着你需要重新计算所有的特殊数字。前面提到在区块链出现分叉的时候采用的最长链共识，即只保留最长的区块链，也就是包含的工作量最大的那一条，如果重新计算这些关键数据，会让篡改的工作量巨大，大到无法接受。

同时 Merkle 树也简化了支付校验。如果我们要校验交易 5 的有效性，那只要校验从交易 5 到 Root Hash 的分支 Hash 值即可，也就是先计算交易 5 与交易 6 的 Hash 值得到 Hash56，再结合 Hash78，计算出 Hash5678，再结合 Hash1234 计算出 Hash1-8 得到待验证的根哈希值，然后把计算出来的结果和实际存储的 Root Hash 进行对比，如果一致则表明该笔交易是有效的，否则就是无效的。

## 12.4 挖矿与奖励

在上一节我们只是简要的介绍了区块链的区块中 **Bits、Difficulty 和 Nonce** 这三个字段。在本节，我们把这三个字段放在一起作详细介绍，因为它们都是和区块链挖矿紧密相关的。

我们已经知道，为了防止在某一个时刻有 2 个或者多个人对交易记录同时记账，需要通过提高记账的难度来控制记账的频率，从而降低这种同时记账的冲突。实际上，这几个字段结合在一起的目的就是用来提高去中心化分布式网络中每个结点记账的难度的。在比特币区块中，这些字段一起能够结合当前网络总的运算能力，动态保证每 10 分钟左右才能有一次记账发生，也就是每隔 10 分钟左右，才会把这段时间内所有的交易记录打包放在一个区块中。

当一个或多个交易记录发布到网络上之后，每个结点都开始打包这些记录并开始为这些记录生成一个新的区块以此抢占记账权从而获得奖励。每个新的区块都必须要有一个 ID。区块 ID 实际上就是调用 SHA256 算法，将新的区块头部所有信息结合在一起计算得出一个 hash 值，即：

区块 ID =SHA256(区块头信息)

而每个结点抢占记账权其实就是争着计算出这个包含这段时间所有交易记录即将被创建的区块的 ID。

仅仅计算一段信息的 SHA256 哈希值并不难也很快就会计算出结果，因此如果不做任何难度限制，那么区块 ID 很容易被其中的很多结点同时计算出来，因此我们必须对区块 ID 的计算加上特别的限制，来提高各个结点算出这个值的难度。事实上区块链也正是这样做的。

首先区块头包含一个难度系数（difficulty），这个值决定了计算哈希的难度。区块链协议规定，使用一个常量除以难度系数，可以得到目标值（target），即：

$$Target = \frac{某一个常量}{Difficulty}$$

而 Bits 则为压缩格式的当前目标(target)的 hash 值。显然，难度系数越大，目标值就越小。在目标值的计算公式里，某一个常量其实就是一个如下的大整数：

**0x0000 0000 FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF FFFF**

这个十六进制常量含有 8 个前导零，代表了比特币网络最初的目标值，是一个固定不变的数。

比特币创世的时候，中本聪把 Difficulty 设置为 1，之后根据全网算力周期性的动态调整 Difficulty 难度值，来尽可能保证全网在 10 分钟运算找到符合要求的区块 ID。调整的周期约每两周一次（每隔 2016 个区块）。

计算的结果 Target 最终也会是一个二进制位中含有多个前导零的整数。当全网算力越大（通常是在结点数越多的时候），难度 Difficulty 就会调整得越大， Target 也就越小，某种意义上意味着前导零更多，那么计算难度就会越大。因为当 Target 确定之后，那么计算这个即将创建的区块 ID 哈希值必须满足如下条件：

$$SHA256\left(PrevBlock + MerkleRoot + Nonce + Bits + \ldots\right) \leq Target$$

即区块 ID 哈希值跟目标（Target）值密切相关，只有小于目标值的 ID 哈希值才是有效的，否则必须重算。由于目标值非常小，区块 ID 哈希值作为一个 256 位的值，小于该值的机会极其渺茫，256 位中前面必须有很多个前导的 0 组成，因此很有可能计算 10 亿次，才算中一次。由于当前区块 ID 哈希值由区块头唯一决定，如果要对同一个区块反复计算 ID 哈希值，就意味着，区块头中某些数据必须不停地变化，否则不可能算出不一样的值。而这个变化的部分，正是 Nonce 字段。Nonce 是一个整数随机值，各个结点争抢记账权的努力其实就是找出 Nonce 的值，使得区块头在这个 Nonce 的参与下使得它的 SHA256 哈希值（也就是区块 ID 哈希）可以小于由难度字段 Difficulty 确定的目标 target 值。由于 SHA256 算法算出来的哈希值没有任何规律，因此 Nonce 值只能靠猜。一旦猜中，那么就可以拥有了这个区块的创建资格，并将交易记录打包写入区块。区块创建出来之后，该结点就将这个区块在网络上进行广播，告诉所有其他结点这个区块我已经创建好了。其他结点在接收到这个通知之后，开始验算区块的合法性，一旦合法，那么其他节点就放弃已有的打包和计算工作，将这个区块复制到自己本地的区块链尾部。

Nonce 非常难猜，只能通过穷举法，从 0 开始，一个个试。根据定义，Nonce 是一个 32 位的有符号整数值，其中正整数个数最大可以到 21.47 亿(即 $2^{32}/2$)。如图 12-7 的比特币区块截图例子，第 618898 个区块的 Nonce 值是 322459559，那么意味着结点从 0 开始，一共计算了 3.22 亿次，才得到了一个有效的 Nonce 值，使得算出的哈希值能够满足小于等于目标值这个条件。这个过程没有任何规律可循，完全是碰运气。运气好也许一下子就找到了 Nonce。运气不好可能算完了 21.47 亿次，都没有发现 Nonce。这个时候，必须修改区块体，然后重新开始计算。

现在我们假设使用 SHA256 哈希算法计算出区块头的 256 位哈希值中前 72 位都为 0 就小于等于 target 目标值，那么猜中符合要求的 Nonce 的概率为多少呢？这实际上是一个古典概型的概率计算问题。

首先，前 72 位都为 0 的 SHA256 哈希值共有：$2^{256-72}$ 个，而 256 位哈希值共有：$2^{256}$ 个。那么算出符合前 72 位都为 0 的 SHA256 哈希值的 Nonce 概率为：

$$P=2^{256-72}/2^{256}=2^{184}/2^{256}=1/2^{72}$$

也就是：1/4722366482869645213696。它大约就是 4.7 万亿亿分之一。换句话说，就是平均要进行 4.7 万亿亿次 SHA 计算，才可能得到一个 Nonce。

我们把结点猜测 Nonce 随机值，使得通过 SHA256 计算出区块头的 hash 值小于等于 Target 目标值从而创建出新的区块的过程，形象的比喻为挖矿。而运行挖矿程序的电脑就叫“矿机”或“矿工”，一个装满矿机的房间叫“矿场”，矿场的主人就是“矿主”，如图 12-9 所示。

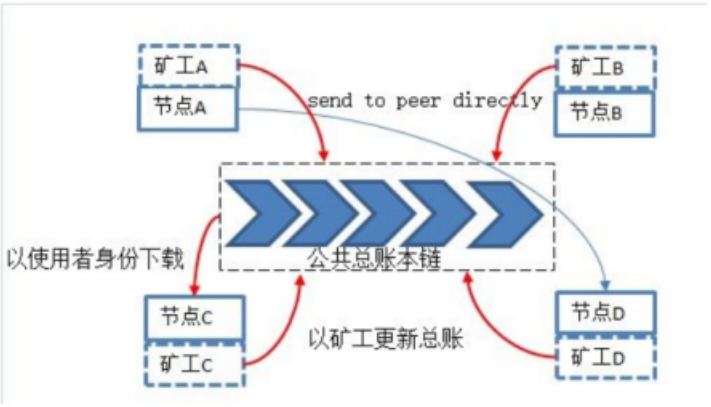



图 12-9 矿工挖矿

比特币中每十分钟左右产生一个区块，即每隔十分钟计算机就打包全网交易进入一个区块。矿工正是在打包游戏中争夺区块链记账权的人，谁能最快解开 SHA256 这个数学命题的 Nonce 值，谁就赢得了这个十分钟一个的区块的打包记账权。而这十分钟里的每一笔交易，都会被盖上一个 timestamp 时间戳。

挖矿的好处我们在前面已经提到，就是可以获得奖励。在比特币区块链里，成功挖出一个区块的好处就是可以获得比特币的奖励和交易费。

关于奖励，在比特币区块链里，中本聪设计了这样的规则，即：奖励的初始值是挖出一个区块奖励 50 比特币，然后每增加 21 万个区块后，奖励就减半，所以北京时间 2009 年 1 月 4 日中本聪自己创建的第一个创世区块，就奖励给了自己 50 比特币。按照平均 10 分钟一个区块的产出速度，大约是每隔四年会减半一次。第一次减半发生在北京时间 2012 年 11 月 29 日 7 点 24 分，第 21 万个区块诞生，比特币奖励减少为 25 个。北京时间 2016 年 7 月

10 日凌晨 0 点 46 分，第 42 万个区块诞生，这次相隔时间大约是三年零七个月，比特币奖励再次减半，成为现在的 12.5 个。大约到 2140 年，比特币的奖励就会趋于零。

于是比特币的总数可以表示为：

$$210000\times 50+210000\times 25+210000\times 12.5+...+210000\times 50/2^n+...$$

对上式提一个 210000*50 的公因式出来，比特币的总数可以表示为：

$$210000\times 50\times \left(1+\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+...\right)$$

当 n 趋于∞时，对上述求和公式求极限容易知道：

$$210000\times 50\times \left(1+\frac{1}{2}+\frac{1}{4}+\frac{1}{8}+...\right)=210000\times 50\times 2=21000000$$

因此比特币的上限是 2100 万个。这个上限避免了货币滥发和通货膨胀的风险。由于到了 2140 年，比特别的挖矿奖励就会趋于零，那么这时还会有人去挖矿吗？答案是肯定的，因为这时虽然没有了挖矿奖励，但还有交易手续费的奖励。

挖矿是一个运算量大因而耗电量也大的过程，因此矿场一般建在电力资源便宜的地方。在中国西南三省，由于拥有丰富的水电资源，因此拥有较多的矿场。如图 12-10 所示，图中展示了一个矿场的内部场景。一个位于中国四川的矿场，拥有 5000 多台矿机，平均每天耗电超过 20 万度，当地的电价是每度 3 毛，一天光是电费就 6 万多元，平均每天可以挖出大约 50 个比特币，而比特币的价格波动很大，目前大概是 1 万美元左右，所以利润还是很可观的。

图 12-10 矿场

## 12.5 双花问题与 51%攻击

区块链中有一个非常著名的双花（double spending）问题。所谓双花，就是指双重花费的意思。比如 A 手中只有 2 个比特币，那么他同时向 B 和 C 转账这 2 个比特币，那么 A 就把这 2 个比特币重复花了 2 次，这就是所谓的双花问题。

在传统中心化交易系统中因为结算速度极快而不太可能出现这种情况，但在区块链网络中可能性却比较大。在区块链里，一般在正常情况下，当 A 把仅有的 2 个比特币转账给 B，

这条交易记录被打包进区块并被确认之后，A 原来持有的这 2 个比特币所对应的交易记录就会被标记为失效，因此当 A 再把这 2 个比特比转账给 C 的时候，就会失败。但由于没有中心化的管理者，交易确认的时间会被延长，使得这种欺骗行为可能实现。

那么在区块链里，我们是如何防止双花问题的呢？我们可以举一个现实中的例子来说明。

假如 A 手中有 1 个比特币，去超市购买一台 iPhone 手机。当 A 把 1 个比特币支付给超市之后，为了达到双花的目的，他会同时把这 1 个比特币转账给自己。如果 A 的运算能力足够强大，同时让这两条交易记录（一条转账给超市，一条转账给自己）打包进入 2 个区块之中（注意，这两条记录不能打进同一个区块，否则就冲突了）。这个时候，超市如果看见转账给自己的那个区块被确认了，就发送 iPhone 给 A。此时，A 为了让转账给超市的记录作废，由于最长链共识机制，A 就必须使转账给自己这条记录的区块所在的链成为主链。但在 A 没有拿到 iPhone 手机之前，他还必须同时维护短的那条链。这样一来，A 就必须同时维护两条分叉的区块链，直到拿到 iPhone 手机，然后才能放弃短的那条链，这样短链上的区块就会作废，超市收到的比特币就会返回给 A。否则，只要其中一条链作废，双花就会失败。因为如果主链失败，A 就无法再花这 1 个比特币，短链失败 A 就无法拿到 iPhone 手机。

超市为了万无一失，一定会等到自己的区块所在的区块链足够长的时候才会发货，一般来说等到 6 个以上的区块确认，就足够安全了。这个时候 A 所维护的区块链就变成了短链。因此 A 为了废掉超市收账所对应的区块链，就必须自己加长自己的短链。而这个时候，网络中的所有矿工都在超市所在的区块链长链上挖矿，会继续增加长链的长度。所以 A 要加长自己的短链来超过长链，这也不是不可能，但就必须得拥有超强的算力，因为区块由谁生成，主要就是看谁的算力强大从而先计算出区块头对应的 hash 值。

那么这个超强的算力是多少呢？如果 A 掌握了全网 51%的算力，那么他就有 51%的概率获得下一个区块的记账权，从而维持对自己有利的长链，使得发给超市比特币的区块失效。这就是所谓的 51%攻击。

如果超市等到 5 个以上的区块确认之后才发货，那么对超市拥有的区块所在的区块链领先了 6 个区块，那么 A 如果要追赶这条区块链的长度，必须抢夺 6 次以上的记账权，即使在掌握了全网 51%的运算力的情况下，每次抢占成功的概率也只有 51%，那么根据数学中的概率论知识，6 次都抢占成功的概率为：

$$0.51^6 \approx 1.7\%$$

这个成功的概率已经很低。随着用户等待区块的增加，这个难度，幂次上升。在 7 到 8 个区块产生后，概率上来讲，就可以绝对信任这次转账记录了。

一般来说，发动 51%攻击是无意义的，因为为了达到全网 51%算力所需投入的成本，一般远远大于成功实施攻击后获得的收益。同时我们也可以看出，参与系统中的结点越多和计算能力越强，那么一般人就越难超过全网 51%的运算能力，那么该系统的安全性也就越高。

总之，通过对区块链技术的详细介绍，我们认识到我们用于支付的货币并不是一种商品，而是信用和交易构成的一套记账体系，区块链就相当于这个账本，区块就相当于这个账本的一页，区块中所承载的信息，就是这一页上记载的不可更改的交易数据。

虽然区块链解决了很多问题，但也不乏对区块链提出质疑的声音。比如区块链本质上是通过牺牲速度和 IT 资源来换取公平性。除了公平性，区块链的所有任何其他特性与传统技术相比并无任何优势。区块链的最大优势往往是它的匿名性，但这又会引起很多社会问题，比如将区块链用于贩毒或者洗钱等等。

# 参考文献


[1] 同济大学数学系. 高等数学. 高等教育出版社，2014
[2] 同济大学数学系. 工程数学线性代数. 高等教育出版社, 2014
[3] 盛骤，谢式千，潘承毅. 概率论与数理统计. 高等教育出版社，2017
[4] 段禅伦，魏仕民. 离散数学. 北京大学出版社, 2005
[5] 百度百科. https://baike.baidu.com.
[6] Wikipedia. https://www.wikipedia.org.
[7] 知乎. https://www.zhihu.com
[8] Leetcode. https://leetcode-cn.com
[9] 刘绍学，钱珮玲，章建跃. 普通高中课程标准实验教科书数学. 人民教育出版社，2007
[10] 周志华. 机器学习. 清华大学出版社, 2016
[11] 伊恩.古德费洛，约书亚.本吉奥，亚伦.库维尔. 深度学习. 人民邮电出版社, 2017
[12] 韩雪涛. 三次数学危机与数学悖论. 人民邮电出版社， 2016
[13] 李文林. 数学史概论. 高等教育出版社，2011
[14] 段钢. 加密与解密. 电子工业出版社，2018
[15] 陈汉武. 量子信息与量子计算简明教程. 东南大学出版社，2006
[16] 王晓琳，陈海军. 行星运行轨道的推导. 陇东学院，2013

# 附录一，程序员入门学习路线建议

IT 是一个生机勃勃的行业，不少年轻人选择 IT 作为自己的职业规划与选项，希望能够在 IT 行业里挖掘自己的潜力，奔一个好的前程。有一句话说：富可搞金融，穷可搞 IT。

但很多朋友发现，IT 行业入门也并非易事，不少人无法坚持就只好中途放弃。很多数程序员都是靠自学，这其中的艰辛自不必多言，他们有时会很迷茫，发现自己看了很多书，花了很多时间，依然无法写出像样的程序。这可能是学习的步骤不对。 一个循序渐进的学习步骤，能够帮助程序员朋友在最短的时间内达到最好的效果。

首先我们要有一个意识就是，无论干哪行，学哪个方面的技术，在我们下决心学习它的时候，第一个需要解决的问题就是：学习它可以分为哪几个阶段？每个阶段都有哪几本经典的好书供学习？只有在每个阶段都找到了一两本相关的经典的好书，才为我们学习这个方面的技术找准了方向，学习起来也会事半功倍。所以，在我学习程序设计的时候，我对书的要求很高，不是随便拿一本来看，看一本差书，既浪费时间，又毫无收获，而是要找口碑最好，写得非常经典的书籍来看。

假如你是一个 IT 行业零基础而且年龄小于 26 周岁的年轻人，渴望进入 IT 行业发展，那么下面我们就来介绍一个底层安全程序员的学习阶段， 并同时推荐在每个阶段相关的好的学习书籍。

学习周期：

我们把学习划分为十一个阶段。首先来看下学完这十一个阶段花费的时间，也就是学习周期。

自学模式：阶段一到阶段十应计划 1 年时间；阶段十一应计划 1 年时间，共 2 年时间

培训模式：如果参加一些系统的优质的培训可少走弯路，可以将学习周期缩减为 1 年时间

注意：在看书学习的时候，要习惯性的打开 VC，经常实践书中的理论和例子。一些重要的代码在理解了之后应该再背着写一遍（注意：不是照着抄），加深印象，当背着写不出来的时候，再回头看看书中的代码。要明白，书看懂了，离会写代码还有一段距离，在写代码的时候还会遇到很多问题。遇到问题不要怕，解决了，才是学会了。

## 阶段一，编程语言学习

要进入 IT 行业，做码农，首先就得精通一两门计算机编程语言。就好比你想成为一个伟大的作家，首先也得会一两门人类语言一样。否则，你即使能力再大，想法再多，也是无法实现的。计算机编程语言，是人类与计算机交流的主要途径。正是程序员通过编程语言，让计算机完成相应的任务。

作为程序员的入门编程语言，我们推荐选择 C 语言。曾经就职于 Amazon 和 Google 的卓越程序员 Steve Yegge 在他的文章《几种编程语言的优缺点》以及《程序员的呐喊》中指出“你必须懂 C”，因为 C 最懂冯诺依曼计算机， 而冯诺依曼计算机是当今计算机的架构标准。

C 语言是一门很好很实用的入门编程语言。C 语言运行效率高，它是做底层和安全开发

的必备语言。很多低效率的编程语言，也提供了访问 C 语言库的接口，就是因为 C 语言的执行效率高。时至今日，绝大多数的底层软件，包括操作系统（如 Windows, Linux, UNIX 等），数据库软件（如 MySQL），办公软件（如 OpenOffice），杀毒软件，网络通信协议，包括 JAVA 语言的实现，互联网，以及各种内核驱动等都是用 C 语言开发的。C 语言作为编程语言之王，是当之无愧的。截止 2016 年 9 月，C 语言在 TIOBE（http://www.tiobe.com/index.php/content/paperinfo/tpci/index.html）编程语言使用排行榜上依然占据 TOP2 的位置，仅次于排名第一的 Java 语言。

但是 C 语言的学习有一定的难度，这使得一些初学者望而生畏，学到中途就半途而废。某些高校为了减轻大家的学习负担，也放弃将 C 语言作为编程入门的教学语言，而选择更容易的语言比如 Java 作为编程语言。但是，只要坚持学习，C 语言是可以学会的。而且只要把 C 语言掌握了，学习其它语言，就变得非常容易。事实上，很多优秀的程序员只懂 C 和汇编。

C 语言的学习书籍推荐：可以选择谭浩强老师编著的《C 程序设计》，或者 C 语言创始人写的《C 程序设计语言(K&R)》，以及《一站式学习 C 编程》等。当然，最权威的参考还包括我们在周哥教 IT 推出的《周哥教 IT.C 语言深学活用》以及麦洛科菲的官网入门专区推出的《C 语言学习》http://www.mallocfree.com/basic_outline.htm#c，它包含了麦洛科菲多年 C 语言和内核安全教学的经验总结，深入全面的讲解了 C 语言编程知识，值得大家收藏学习。

在 C 语言学习过程中，需要重点掌握的地方包括：

- 整数编码与存储；
- 二进制，八进制，十六进制与转换；
- 变量的类型，作用域，存储空间，生命周期；
- 字符串的定义以及常见操作；
- 函数的传参：传值，传指针，（传引用）；
- 函数的调用约定：cdecl,stdcall,fastcall,nakedcall,(thiscall)；
- 指针，指针定义，*与&，指针加减运算，指针与数组关系，常量指针与指针常量，数组指针与指针数组，函数指针与指针函数，二级指针；
- 内存布局，分配与泄漏；
- 结构体，自然对齐与 sizeof 计算大小，结构体赋值（浅拷贝，深拷贝）；
- 位运算与应用
- 学会调试工具的使用

入门 C 之后，需要进一步学习的书籍包括《高质量 C/C++编程》--林锐博士，《C 语言编程精粹》--姜静波

# 阶段二，数据结构学习

数据结构是算法的基础。算法又是程序的核心。所以，学完 C 语言后，应该立即学习数据结构。数据结构包含：字符串，数组，链表，堆栈，队列，二叉树，排序，查找，HASH

表等等，这些都要牢固掌握。

数据结构教材推荐使用清华严蔚敏老师的《数据结构（C 语言版）》教材。除了严蔚敏的教材，还推荐一本李春葆老师的《数据结构习题与解析》 李春葆 C 语言版。这两本书结合看效果最好。此外，也同样可以参考麦洛科菲官网入门专区的《数据结构》部分。

数据结构是程序设计中的关键。因此我们一定要多花精力把数据结构学好，打下牢固的基础。不能急于求成，数据结构都没学好，就想用 VC 等开发工具，写一个不错的软件出来，当然这也是不可能的。

数据结构中需要重点掌握的包括：

- 链表常规操作：创建，遍历，查找，插入，修改，删除
- 队列：先进先出，出队，入队，队空判断，链队与数组队列实现
- 栈：后进先出，堆栈，入栈，栈空判断，链栈和数组栈实现
- 树：二叉树定义，遍历（先序，中序，后序），二叉排序树，平衡二叉树
- 排序：插入（希尔排序），选择（堆排序），交换排序（冒泡排序，快速排序）思想，实现与复杂度
- 查找：折半查找，hash 查找，平衡二叉树查找

实现与提高算法设计能力的一般方法：

http://www.mallocfree.com/interview/algo-1-advice.htm

# 阶段三，汇编语言学习

汇编是一门很有价值的课程。在调试程序，逆向和漏洞挖掘等方面都会涉及到汇编知识。有了程序设计和 C 语言的基础，我们可以紧接着学习汇编语言。与 C 语言相比，汇编语言显得晦涩难懂。汇编语言包括各种各样的寄存器，内存地址，寻址方法，8086 的常用汇编指令，再加上一些伪操作，还有 DOS 和 BIOS 中断的调用。有了这些概念和汇编基础，我们就可以看懂书上的汇编程序了。到此，你应该会对计算机的理解更深一层。学好汇编语言，对于理解计算机程序执行的底层机制，以及以后程序调试和分析，都有很大的帮助。推荐汇编学习书籍：《汇编语言》--王爽，《天书夜读》--邵坚磊。

汇编语言学习重点

- CPU 与寄存器（X86，X64）
- 汇编指令集
- (1)CISC VS RISC
- (2)传送指令：Mov/lea/push/pop
- (3)算术指令
- (4)逻辑指令
- (5)串操作指令
- (6)控制转移指令
- (7)处理机控制指令
- (8)指令机器码

- C 语句、函数调用的汇编分析
  - 循环语句（for,while,do-while）与汇编
  - 条件语句与汇编
  - 数组访问与汇编
  - 结构体访问与汇编
  - 结构体数组与汇编
  - cdecl 调用汇编
  - stdcall 调用汇编
  - fastcall 调用汇编

# 阶段四，C++与面向对象

C 是面向过程语言，C++是面向对象语言。一个程序员，是需要掌握面向对象的思想的。以把大象装入冰箱为例子，虽然同样是为了解决问题，在面向过程的程序员眼里，需要分为 3 步，第一步打开冰箱，第二步把大象放进冰箱，第三步关上冰箱门，然后每一步实现一个函数；但是在面向对象程序员眼里，首先是分析解决问题中各种不同的对象，这里涉及的对象有 3 个：人，冰箱，和大象。然后再分析各个对象的属性和行为以及它们之间的关系。找本 C++上手的书（推荐《C++ Primer》），用一个月的时间去了解 C++和面向对象语言程序设计思想，包括数据的封装，关系的继承，和多态等。C++的推荐书籍包括：《C++Primer》，《Effective C++》，《Thinking in C++》，《More Effective C++》。

C++重点学习内容：

- 封装
- 继承
- 多态
- STL 模板库
- 现代 C++语法规则

# 阶段五，多线程与进程通信

现在大多数程序，为了提高自己任务的并发，都使用了多线程。在多线程环境里，程序为了保证全局资源的一致性，而提供了各种同步与互斥机制。

多线程与多进程重点学习内容：

- 线程与进程的概念和区别：资源分为基本单位，调度基本单位
- 事件
- 互斥体
- 信号量
- 临界区
- 死锁

- 共享内存
- 管道

## 阶段六，网络协议与编程学习

现在是互联网时代，而网络协议是互联网的基础协议。因此，必须学会网络编程。在网络学习这块，最重要的就是 TCP/IP 协议的学习，SOCKET 编程的学习。学会如何使用 SOCKET 进行通信，传输数据。

推荐的书籍包括：《TCP/IP 详解 卷 1-2》 W. RICHARD STEVENS 机械工业出版社。

网络协议学习重点：

- TCP/IP 协议：协议分层，TCP 建立连接握手，断开连接，UDP 协议，IP 协议，ARP 协议等
- HTTP 协议：GET,POT,PUT,DEL 等
- SOCKET 编程：ip/port,socket,accept,recv,send,recvfrom,sendto 等
- 服务器端 IO 通信模型：比如完成端口模型（IOCP），SELECT 模型，以及 EPOLL 模型等。

## 阶段七，数据库与 sql 学习

后台程序一般都需要管理数据，比如注册的用户信息，订单信息等。这些信息一般都是存放在各种数据库中。因此，我们有必要系统的学习一下数据库。数据库的学习，最重要的就是 SQL 编程的学习，包括 insert,update,delete,select 等语句的灵活使用。学习 SQL 编程可以结合具体的 MySQL 数据库来学习。各个编程语言都提供了访问数据库和执行 SQL 语句的接口。

此外，还必须了解一些非关系数据库，比如 redis,mecached 等。这些数据库比传统的关系数据库有更高的执行效率，一般用来做后台的优化和性能的提升。

SQL 的学习推荐网站: http://www.w3school.com.cn/sql/index.asp。书籍：《sql 语句用法与讲解》，《MySQL 中文参考手册》等。

## 阶段八，界面开发学习

后台服务器端程序，可以不用界面。但如果要写客户端程序，就必然会涉及到界面的开发。作为底层程序员，界面开发可以不精通，但必须要会。界面开发就是要处理各种控件来接受数据的输入和输出用作人机交互，比如按钮，编辑框，单选，多选，列表等等，此外还需要掌握控件的消息处理机制。

在 WINDOWS 平台可以学习 MS 官方的 MFC 界面库（对 WIN32 SDK 的封装，但开发更容易了），也可以学习跨平台的 QT 界面库，以及开源的 DUILIB 库。Linux 中有 GTK 或者是 QT 界面库。当然还包括移动端的 Android 界面，IOS 界面开发等。

实际上界面开发的原理都差不多，都是在处理各种控件和消息，所以入门还是很容易的，掌握了其中一个界面库，学习新的界面库也很容易。

界面开发推荐书籍：《VC 技术内幕》，《深入浅出 MFC》，《疯狂的 Android》等。

# 阶段九，Linux 与开源项目学习

曾经 WINDOWS 是一统天下，不可一世的 WINTEL 联盟长期占据 IT 市场绝对支配地位。但是，时过境迁，随着 LINUX 不但完善和发展，尤其是在 Android 将 Linux 选为底层操作系统之后，Linux 系统得到了极大的普及和发展。现在谁也不敢小觑 Linux 了。

作为程序员，掌握 Linux 系统与开发成为必要，Linux 平台的程序设计与开发是必备技巧。很难想象一个程序员，不懂 Linux 意味着什么。

Linux 入门书籍推荐：《UNIX 环境高级编程第二版》，《Linux 程序设计（第四版）》

Linux 系统学习重点：

- Linux 各种管理命令
- LINUX 文件系统
- VIM or EMACS
- GCC
- GDB
- LINUX 开源项目：lamp/mysql,redis,memcached,curl,json 等

# 阶段十，其它语言学习

一般来说，程序员应该精通一门语言，熟悉 2 到 3 种语言，所以有必要学习一些其它语言。这里的其它语言包括 JAVA，Python 等。在学好了 C 语言之后，如果再学习 Java，python,php,C#,GO,rust 等别的语言，就非常容易了。因为语言都是触类旁通的，拿来就用了，而且 C 和 C++算是语言中最难掌握的，如果都学会了，那么其它语言都很容易的了。

学习了 JAVA，就可以进一步学习移动编程，比如 Android 应用编程（比如 《疯狂的 Android》）了，而在一开始又学习了 MFC 的消息映射的思想，学习 ANDROID 界面编程也应该很容易理解里面的事件和监听机制，Android 的界面布局与 MFC 不一样，是通过 XML 来进行独立布局的，这个需要重新理解。从长远来看，程序员起码应该掌握：C，C++，Java，Python 这几种语言，是很有必要的。

推荐书籍：《Java 语言程序设计-基础篇》，《Java 语言程序设计-进阶篇》，《简明 Python 教程》，《Python 学习手册》

语言学习多了，各种语言语法容易混淆：

各编程语言比较分析: http://www.mallocfree.com/data/bcyycmp-mallocfree.com.pdf

# 阶段十一，方向选择与底层安全学习

在有了 C 语言基础之后，我们就可以向自己最感兴趣和最擅长的领域发展了。其中安全方向是一个很好的选择，我们在这里给大家强烈推荐，理由请参考《从事 IT 安全行业前景分析报告》。对于安全专业来讲，可以分为如下几个领域，当然这些领域不是孤立的，而是互相联系，相辅相成的。

## 1，内核开发

主要分为 Windows 内核驱动开发和 Linux 内核驱动开发。所用语言主要为 C 语言，负责为系统编写.sys 驱动或者.ko 驱动。用来完成各种底层监控和管理工作。这是各类安全软件，主动防御，防火墙，自保护等必须用到的底层技术。

Windows 平台内核驱动开发的重点内容为：

- SSDT，SHADOW SSDT ,INLINE HOOK
- 各种回调框架
- SFILTER 文件过滤
- MINIFILTER 文件过滤
- DISKPERF 磁盘过滤
- TDI/NDIS/WFP 网络防火墙
- 主动防御 HIPS
- ARK 等

LINUX 内核重点内容包括：

- 内核的编译与裁剪
- 内核模块的开发与编译
- 内核内存分配
- 内核同步与互斥
- 内核中断上半部与下半部机制
- 内核 HOOK 与调试
- 基于 linux 内核的网络防火墙开发

推荐书籍：《Windows Internal》，《Windows 2000 设备驱动程序设计指南》，《Windows 驱动开发技术详解》（张帆著），《寒江独钓：Windows 内核安全编程》，《Linux 设备驱动程序第三版》，《Linux 内核设计与实现第三版》，《Linux 设备驱动开发详解》，《Linux_内核完全注释_V11_赵炯》，《深入分析 Linux 内核源码》

## 2，逆向技术

逆向技术就是使用反汇编工具，将二进制程序还原成汇编代码，然后再基于汇编代码分

析整理，得到程序的算法逻辑，甚至还原出 C 语言代码。逆向工具一般都是使用强大的 IDA Pro 工具。当然，现实中很多程序为了防止被逆向，会使用一些保护措施，比如加壳加花指令等。注意，逆向技术与开发技术是相辅相成的。并且学习逆向技术，必须以开发技术为前提。如果开发技术都没有掌握好，甚至想绕过开发技术的学习，直接学习逆向技术，迟早会让你付出代价。大家可以关注下所有招聘逆向工作的帖子里，都会对开发能力提出明确要求，要求熟练掌握 C 和相关平台的开发技术。

推荐书籍：《The IDA Pro Book》

## 3，移动安全

随着移动互联网的普及，移动安全问题越来越突出，尤其是开放平台 Android 系统，面临着日益突出和严重的安全问题，针对手机安全的威胁层出不穷。因此，移动安全是未来必须重点面对的技术领域。

推荐学习书籍：《Android 软件安全与逆向分析》《Android 应用安全防护和逆向分析》（姜维著）

移动安全学习的重要内容：

- NDK 编程
- SMALI 语言
- ARM 汇编
- HOOK
- ROOT
- 动态调试

## 4，漏洞挖掘

漏洞分析与挖掘是系统安全的一个重要分支。每当 IT 界有重大漏洞曝出，都让人心惊胆战，更何况还有很多黑客手握 0Day 而尚未曝光的各种漏洞，利用这些未公开的漏洞，可以随时入侵用户的计算机，窃取各种机密数据。

漏洞挖掘重要内容：

- 缓冲区溢出漏洞：栈溢出，堆溢出
- 任意地址写入任意数据攻击
- XSS 攻击
- SQL 注入攻击
- 竞争条件漏洞攻击
- PATH 攻击等
- FUZZ 测试
- 操作系统安全机制：Security cookie, SAFE SHE ,DEP, ASLR, SEHOP, Safe Unlink 等

- 安全编码

相关书籍推荐：《0day 安全软件漏洞分析技术》

## 5，WEB 安全与渗透测试

任何一个企业无论大小，都会有自己的网站，有的企业甚至一些重要的业务和交易完全基于 Web 展开。而 Web 面临的安全威胁是最普遍和广泛的一类安全威胁。因此，无论大中小公司（包括银行，企事业单位，电子商务），为了维护自己的 Web 服务的可靠性和安全性， 都不惜重金投入到 Web 安全的建设中去。因此，在互联网时代，Web 安全人才也有很广泛的就业前景。 Web 安全与渗透这一块的重要内容包括：

- WEB 安全基础
  如何搭建一个网站（前端 html+css+js 和后端:php,mysql,redis 等），实现注册登录功能 tcp/ip，http,https 协议，session，cookie Sql 编程 脚本语言：python ICP 备案 https 证书申请
- SQL 注入
- XSS 跨站攻击
- CSRF 攻击
- 上传漏洞
- CSRF 攻击
- 钓鱼与挂马
- 代码注入
- 中间件漏洞
- 旁注攻击
- DOS 攻击
- 权限控制
- 基于 metasploit 的 WEB 渗透技术

以上便是我们针对初学者学习 IT 编程技术的一些指导性建议。如果认真掌握了上面的知识点，很难在 IT 行业找不到一个非常不错的工作。

# 附录二，数学史概述

学习数学，应该了解数学发展的伟大历史。数学理论的每一点进步，都和历史上的数学家从实际出发，孜孜不倦，不断求真，追求真理的精神分不开的，有时候甚至付出了生命的代价。

## 一，世界数学史

起源于远古记数：大多数文明发明了 10 进制（巴比伦 60 进制，玛雅 20 进制），结绳记数、刻痕记数；尼罗河泛滥重新丈量土地、天文观测-几何；埃及莱茵德和莫斯科纸草书：单位分数 1/n，PI=3.16，讨论线性方程，无位值概念；巴比伦泥板文书：60 进制，位值概念，开方根 1.414，三项二次方程，三次方程（查表），PI=3（3 又 1/8），普林顿 322 泥板文书（公元前 1600 年）4 列 15 行表格记录了勾股数组。

公元前 6 世纪，毕达格拉斯是古希腊的哲学家和数学家（约前 582-500 年），传说发现了勾股定理，并宰了一百多头牲畜来祭祀缪（miu）斯女神。现在普遍认为在毕达格拉斯之前，已为巴比伦人所知。其实中国西周数学家商高已提出了勾股定理，比毕达格拉斯早 600 多年。毕达哥拉斯学派相信任何量都可以表示成两个整数之比，万物皆依赖于整数。但正方形的对角线和一边比是不可公度的（无理数 1.414…）。有一次他们航行在海上开会，一个学员发现后，学派惊恐不已，把这个学员抛进了大海。根号 2 等无理数的发现引起了第一次数学危机。

公元前 5 世纪，安提丰提出圆内接正多边形逼近圆面积来化圆为方（古希腊三大几何问题：化圆为方；倍立方体；三等分角（限尺规作图）），成穷竭法的始祖。

公元前 4 世纪，亚里士多德《物理学》记载了有关无限性概念的 4 个芝诺悖论：两分法运动不存在；阿基里斯追不上乌龟；飞箭静止；操场或游行队伍问题。而要理清这些悖论，需要极限、连续和无穷集合等抽象概念，当时希腊数学家是不可能解答的。亚里士多德把研究数学推理的出发点的基本原理，区分为公理和公设（一切科学的真理和某门科学第一性原理），创立逻辑学矛盾律（一个命题不能同时是真的又是假的）和排中律（一个命题或者是真的或者是假的，二者必居其一），使数学推理规律规范化和系统化，为欧几里得演绎几何体系奠定了方法论的基础。芝诺悖论和不可公度困难一起成为希腊数学追求逻辑精确性的强力激素。因此，希腊也被称为数学的故乡。

公元前 4~3 世纪，欧几里得《几何原本》：5 条公理，5 条公设，119 个定义，465 条命题，历史上第一个数学公理体系，影响深远。直到 19 世纪才提出新的几何学。

公元前 3 世纪，亚历山大学派的阿基米德利用穷竭法求出了圆的周长和面积；平衡法求出了球的体积；体现了近代极限与微积分的思想，并用穷竭法给予了证明。（我国祖冲之和祖暅在南北朝时期（公元 5,6 世纪）才正确求出球的体积）

公元前 3~2 世纪，阿波罗尼奥斯《圆锥曲线论》，从一个对顶锥得到并命名椭圆（ellipse）、双曲线（hyperbola）和抛物线（parabola），在对它们性质的研究中已经包含了近代微分几

何和射影几何的课题。

公元 2 世纪，地心说集大成者托勒密在《天文学大成》总结了古代三角学，并制定了弦表（正弦函数表）

公元 3~4 世纪，古希腊丢番图《算术》中对不定方程求解，因此不定方程又叫丢番图方程。$z^2=x^2+y^2$。引出了后来的费马大定理（n 大于 2 时，没有正整数解，1994 年被英国数学家维尔斯最终解决，已过去 300 多年），创用了一套缩写符号并用特殊符号表示未知数代替之前的文字叙述，是代数符号的起源。

公元 2~3 世纪，巴基斯坦巴克沙利村庄桦树皮上的手稿出现完整的十进制数码，零用点表示，后来（至迟不超过 9 世纪）逐渐演变为圆圈。零是伟大的发明，在数字中表示无，并且起着占位的作用，比如 1054，简化了加减乘除运算。公元 8 世纪印度数字传入阿拉伯，然后传入欧洲，零稍晚传入，至迟在 13 世纪斐波那契《算经》已经有包括零在内的完整印度数字介绍，对欧洲近代科学进步扮演重要角色。

公元 8~9 世纪，阿拉伯花拉子米《还原与对消计算概要》在 12 世纪翻译成拉丁文，在欧洲产生巨大影响。阿拉伯语 al-jabr 意为还原和移项，成为当今英语 algebra 的来源，因此也叫《代数学》，叙述一、二次方程求解问题。另一本《印度计算法》，系统介绍了印度数码和十进制记数，使印度数码在阿拉伯流行起来，并随后被翻译成拉丁文传播到欧洲，让欧洲误以为是阿拉伯数字。

5-11 世纪，罗马时期，天主教成为欧洲绝对势力，欧洲进入黑暗的宗教时期。12 世纪，开始大量翻译阿拉伯文、希腊数学书籍为拉丁文。

13 世纪，意大利斐波那契写成《算经》汇集中国，印度，希腊数学问题汇集，系统介绍了阿拉伯印度数字，修订版记载了“兔子问题”并导致了著名的斐波那契数列。

15 世纪，意大利费罗和塔塔利亚挑战解 3 次方程，后者获胜并把方法秘密传给了卡尔丹。卡尔丹违背承诺在《大法》中公开了这些解法，推广到了一般情形并给出了几何证明。其学生费拉里解决了 4 次方程。《大法》对不可约情况感到困惑（其实已经邂逅了复数）。邦贝利在教科书《代数》中引进了虚数，以解决三次方程不可约情况。卡尔丹已经认识到复根是成对出现并且三次方程有三个根，四次方程有四个根。荷兰人吉拉德在 17 世纪的《代数新发现》里进一步推断 n 次方程 n 个根，即“代数基本定理”，但没有证明。

16 世纪，韦达在《分析引论》里第一次有意识的使用系统的代数字母和符号，辅音字母表示已知量，元音字母表示未知量。笛卡尔改进为拉丁字母前几个 a、b、c、d 表示已知量，后几个 x、y、z、w 表示未知量成为今天的习惯。

17 世纪初，苏格兰纳皮尔在《奇妙的对数》中阐述了对数方法，减轻了天文学的计算工作量。同时，笛卡尔和费马分别发明了解析几何。

1637 年，法国的哲学家和数学家笛卡尔发表了他的著作《方法论》，书后有三篇附录，其中一篇叫《几何学》。笛卡尔的《几何学》实际是代数问题，探讨方程的根的性质。后世

的数学家和数学史学家都把笛卡尔的《几何学》作为解析几何的起点。笛卡尔的中心思想是建立起一种“普遍”的数学，把算术、代数、几何统一起来。他设想，把任何数学问题化为一个代数问题，再把任何代数问题归结到去解一个方程式。解析几何也是微积分发明的基础。17 世纪牛顿和莱布尼茨独立发明微积分。微积分被誉为人类精神的最高胜利。牛顿证明了积分与微分的互逆关系。牛顿莱布尼茨微积分发明之争：发明时间牛顿早于莱布尼茨；发表时间莱布尼茨（1684）早于牛顿（1687，原理）。牛顿在《原理》中为微积分披上的几何外衣使他的流数术僵硬呆板，固守牛顿的几何形式，阻碍了 18 世纪英国数学的发展；莱布尼茨的微积分符号（∫，dx）很精心并沿用至今，牛顿符号中用点.表示流数（微分），撇’表示流量（积分）已被淘汰；牛顿三大力学定律和万有引力定律，莱布尼茨二进制。解析几何，对数和微积分被恩格斯称为 17 世纪数学三大发明。

在微积分大范围应用的同时，关于微积分基础的问题也越来越严重。关键问题就是无穷小量(0/0)究竟是不是零?无穷小及其分析是否合理?由此而引起了数学界甚至哲学界长达一个半世纪的争论，造成了第二次数学危机。

18 世纪微积分进一步深入发展：泰勒，麦克劳林等发展研究了牛顿的流数术，欧陆数学家有伯努利兄弟，欧拉，克莱洛，达朗贝尔，拉格朗日，拉普拉斯，勒让德，蒙日。18 世纪，费马提出费马大定理等开始研究数论，哥德巴赫给欧拉写信，提出猜想：每个偶数是两个素数之和；每个奇数是三个素数之和。

拉格朗日指出用根式求解解 4 次以上的方程是不可能的，但未给出证明，19 世纪后被挪威年轻人阿贝尔（27 岁肺结核去世）证明，并引入了“域”这一重要的近世代数概念。后来法国数学家伽罗瓦（21 岁死于与政敌的爱情纠葛引起的决斗）给出了能用根式可解的特殊方程的充要条件，并提出了最早的群的定义，导致代数学的新生。伽罗瓦的论文曾三次向法国科学院递交：第一次被柯西丢失；第二次傅里叶病逝下落不明；第三次泊松认为不可理解打入冷宫。决斗前夜，伽罗瓦整理了数学手稿，并在遗书中坚信“总会有人发现（14 年后），解释清楚这堆东西对他们是有益的”

爱尔兰数学家哈密顿在推广复数的时候，发现了四元数：a+bi+cj+dk（丢失了乘法的交换性），麦克斯韦把四元数分为数量和向量部分。向量部分的三个分量解释成三个坐标轴的长度。美国吉布斯和英国亥维赛抛弃四元数的数量部分，提出向量：v=ai+bj+ck

线性代数起源于线性方程组求解中的行列式和矩阵，19 世纪开始独立发展，柯西，凯莱和西尔维斯特等发展了行列式，矩阵理论。

19 世纪中叶，英国布尔在 1847 和 1854 年出版的《逻辑的数学分析》和《思维规律研究》中阐述了布尔逻辑代数思想。

1840 年，俄国罗巴切夫斯基发表非欧几何的德文著作，其基本思想与高斯，波约是一致的，被称为几何学中的哥白尼，遭到群起而攻之。1854 年黎曼（不到 40 岁肺结核去世）在罗氏几何，高斯的内蕴微分几何基础上建立了更广泛的黎曼几何，欧氏几何和罗氏几何和黎曼几何对应了三维空间的曲率为 0，负和正。

1872 康托尔发表集合理论，并证明了有理数可数，实数不可数，为超穷集合理论迈出

了坚实一步。数学史上的第三次危机，由于在康托尔的一般集合理论的边缘发现悖论造成的。1897 年，福尔蒂揭示了集合论中的第一个悖论。两年后，康托尔发现了很相似的悖论。1902 年，罗素又发现了一个悖论，它除了涉及集合概念本身外不涉及别的概念。悖论问题涉及数理逻辑和集合论，但它一开始就牵涉到无穷集合，而现代数学如果脱离无穷集合将寸步难行。数学家们通过将集合的构造公理化来排除了悖论中集合的存在性，以此解决此次数学危机。

1900 年 8 月，希尔伯特在国际数学家大会上作了题为《数学问题》的演讲，提出了 23 个数学问题（包括黎曼猜想，哥德巴赫猜想，孪生素数猜想），拉开了 20 世纪数学的序幕。

2000 年 5 月 24 日美国克雷数学研究所公布数学猜想：千禧年大奖难题(Millennium Prize Problems)，又称世界七大数学难题。根据克雷数学研究所订定的规则，任何一个猜想的解答，只要发表在数学期刊上，并经过两年的验证期，解决者就会被颁发一百万美元奖金。这些难题是呼应 1900 年德国数学家大卫·希尔伯特在巴黎提出的 23 个数学问题。包括了庞加莱猜想（其中三维的情形被俄罗斯数学家格里戈里·佩雷尔曼于 2003 年左右证明。2006 年，数学界最终确认佩雷尔曼的证明解决了庞加莱猜想。Clay 数学研究所在 2010 年为此召开特别会议，为此猜想盖棺定论。佩雷尔曼拒领菲尔兹奖和千年问题奖。），霍奇猜想，黎曼猜想等。

现代数学奖：菲尔兹奖（1.5 万美元，四年一次，40 岁下），沃尔夫奖（10 万美元，每年一次，40 岁以上），阿贝尔奖（80 万美元，每年一次），千年问题奖（100 万美元），陈省身奖（25 万美元+25 万美元，四年一次）等。

## 二，中国数学史

殷商甲骨文中使用完整的十进制记数，至迟到春秋战国时代（公元前 8~3 世纪）开始严格的十进位值制筹算记数，零以空位表示，位值法是中国古代对人类文明的特殊贡献。

至迟不晚于公元前 2 世纪的《周髀算经》里记录了西周开国时期（公元前 10~8 世纪）周公与大夫商高讨论勾股测量的对话，商高提到“勾广三，股修四，径隅五”特例。卷上另一处叙述了周公后人荣方与陈子（公元前 7-6 世纪）的对话中，则包含了勾股定理的一般形式：以日下为勾，日高为股，勾股各自乘，并而开方除之，得邪至日。公元三世纪赵爽在备注《周髀算经》的时候，进行了证明。

至迟不晚于公元前 1 世纪（先秦至西汉中叶）的《九章算术》，是中国古代最重要的数学著作。记录了分数四则运算，比例算法，盈不足术（今有共买物，人出八盈三，人出七不足四。问人数、物数各几何。实际上是已知两点，求直线方程。盈不足术在中世纪阿拉伯数学著作中称为“契丹算法”，13 世纪意大利数学家斐波那契《算经》里也有一章讲“契丹算法”），方程消元法，正负术（负数的认识，印度 7 世纪，欧洲 16 世纪韦达还在回避负数），开方术，几何方面直线类型的面积和体积计算都是正确的，PI=3，球体积公式误差大。

公元 3 世纪魏晋刘徽在备注《九章算术》时利用割圆术求出了 PI=3.14，并在“开立圆术”注文中指出了一条求球体积公式的“牟合方盖”正确路径，但最终失败。

南北朝（南朝宋、齐之间，公元5、6世纪），《南齐书.祖冲之传》中记载祖冲之著《缀术》，《隋书》记载祖冲之计算出PI=3.1415926~3.1415927之间（这个圆周率值是当时世界上最先进的数学成就，直到15世纪阿拉伯数学家阿尔·卡西（al-kāshī）和16世纪法国数学家韦达（1540～1603）才得到更精确的结果。祖冲之还确定了两个分数形式的圆周率值，约率π=22/7（≈3.14），密率π=355/113（≈3.1415929），其中密率是在分母小于1000条件下圆周率的最佳近似分数。密率为祖冲之首创，直到16世纪才被德国数学家奥托（1550～1605）和荷兰工程师安托尼兹（1543～1620）重新得到。在西方数学史上，这个圆周率值常被称为安托尼兹率）。并与儿子祖暅（geng4）一起求出球体公式。祖冲之父子总结了魏晋时期著名数学家刘徽的有关工作，提出“幂势既同则积不容异”，即等高的两立体，若其任意高处的水平截面积相等，则这两立体体积相等，这就是著名的祖暅公理(或刘祖原理)。祖暅应用这个原理，解决了刘徽尚未解决的球体积公式。该原理在西方直到十七世纪才由意大利数学家卡瓦列利发现，比祖暅晚一千一百多年。

公元4世纪的《孙子算经》提到了“物不知数“（今有物不知其数，三三数之剩二，五五数之剩三，七七数之剩二，问物几何）并给出了答案和解法。孙子问题后来被宋代的秦九韶给出了一般解法”大衍求一术”，被西方称为“中国剩余定理”或孙子定理。

公元11世纪，南宋杨辉记录了北宋贾宪用基于贾宪三角的增乘开方法开高次方。

公元13世纪，1247年南宋秦九韶写成《数书九章》，将增乘开方法推广到了高次方程的一般情形，提出了“正负开方术”，此书包含了21个高次方程，次数最高的是10次方程；卷一提出的求解一次同余方程组的“大衍求一术”，与18、19世纪欧拉和高斯提出的定理一致，因此被称为“中国剩余定理”。

元末后，中国数学逐渐衰落微。原因：社会停滞和腐朽，数学发展缺少社会动力和思想刺激；废除数学考试唯八股取士数学家地位低没出路；自由探讨受束缚和禁锢；中国数学本身弱点：筹算本身有很大局限性，无法演进为纯符号代数，超过4个变量的方程组无能为力；重算法，缺演绎论证推导难以升华为现代数学；天朝帝国妄大自守，造成外界先进数学思想传播困难。近代16,17世纪西方数学蓬勃发展的时候，中国数学就明显落后了。

中国（华人）现代数学家：陈省身（现代微分几何之父，沃尔夫奖获得者，“希腊”），丘成桐（卡拉比猜想证明，菲尔兹奖和沃尔夫奖，数学界之王），华罗庚（“华氏定理”、“华氏不等式”、“华—王方法”等），陈景润（哥德巴赫猜想）等。

# 附录三，希腊字母表

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | A | α | /ˈælfə/ | alpha | 阿尔法 | 角度，系数，角加速度 |
| 2 | B | β | /ˈbiːtə/ 或 /ˈbeɪtə/ | beta | 贝塔/毕塔 | 磁通系数，角度，系数 |
| 3 | Γ | γ | /ˈgæmə/ | gamma | 伽玛/甘玛 | 电导系数，角度，比热容比 |
| 4 | Δ | δ | /ˈdeltə/ | delta | 得尔塔/岱欧塔 | 变化量，化学反应中的加热，屈光度，一元二次方程中的判别式 |
| 5 | E | ε | /ˈepsɪlɒn/ | epsilon | 埃普西龙 | 对数之基数，介电常数 |
| 6 | Z | ζ | /ˈziːtə/ | zeta | 泽塔 | 系数，方位角，阻抗，相对黏度 |
| 7 | H | η | /ˈiːtə/ | eta | 伊塔/诶塔 | 迟滞系数，效率 |
| 8 | Θ | θ | /ˈθiːtə/ | theta | 西塔 | 温度，角度 |
| 9 | I | ι | /ˈaɪəʊtə/ | iota | 埃欧塔 | 微小，一点 约塔 |
| 10 | K | κ | /ˈkæpə/ | kappa | 堪帕 | 介质常数，绝热指数 |
| 11 | Λ | λ | /ˈlæmdə/ | lambda | 兰姆达 | 波长，体积，导热系数 |
| 12 | M | μ | /mjuː/ | mu | 谬/穆 | 磁导系数，微，动摩擦系（因）数，流体动力黏度 |
| 13 | N | ν | /njuː/ | nu | 拗/奴 | 磁阻系数，流体运动粘度,光子频率，化学计量数 |
| 14 | Ξ | ξ | 希腊 /ksi/ 英美 /ˈzaɪ/ 或 /ˈsaɪ/ | xi | 可西/赛 | 随机变量，（小）区间内的一个未知特定值 |
| 15 | O | o | /əʊˈmaikrən/ 或 /ˈɑmɪˌkrɑn/ | omicron | 欧 (阿~) 米可荣 | 高阶无穷小函数 |
| 16 | ∏ | π | /paɪ/ | pi | 派 | 圆周率，π(n)表示不大于n的质数个数 |
| 17 | P | ρ | /rəʊ/ | rho | 柔/若 | 电阻系数，柱坐标和极坐标中的极径，密度 |
| 18 | ∑ | σ | /ˈsɪgmə/ | sigma | 西格玛 | 总和，表面密度，跨导，正应力 |
| 19 | T | τ | /tɔː/ 或 /taʊ/ | tau | 套/驼 | 时间常数，切应力，2π（两倍圆周率） |
| 20 | Y | υ | /ˈipsɪlɒn/ 或 /ˈʌpsɪlɒn/ | upsilon | 宇 (阿~) 普西龙 | 位移 |
| 21 | Φ | φ | /faɪ/ | phi | 弗爱/弗忆 | 磁通，角，透镜焦度，热流量 |
| 22 | X | χ | /kaɪ/ | chi | 凯/柯义 | 统计学中有卡方(χ^2)分布 |
| 23 | Ψ | ψ | /psaɪ/ | psi | 赛/普赛/普西 | 角速，介质电通量，ψ函数 |
| 24 | Ω | ω | /ˈəʊmɪgə/ 或 /oʊˈmegə/ | omega | 欧米伽/欧枚嘎 | 欧姆，角速度，交流电的电角度，化学中的质量分数 |

# 附录四，编程语言进化史

目前，在 IT 工业领域，最受企业和程序员欢迎的主流编程语言有 C，Java，Python，C++，Go，Rust，SQL，JavaScript 等等。截止 2020 年 9 月，在 TIOBE 的官网上列出的，全球排行 TOP20 的主流编程语言如图 1 所示：

| Sep 2020 | Sep 2019 | Change | Programming Language | Ratings | Change |
|---|---|---|---|---|---|
| 1 | 2 | ^ | C | 15.95% | +0.74% |
| 2 | 1 | v | Java | 13.48% | -3.18% |
| 3 | 3 | | Python | 10.47% | +0.59% |
| 4 | 4 | | C++ | 7.11% | +1.48% |
| 5 | 5 | | C# | 4.58% | +1.18% |
| 6 | 6 | | Visual Basic | 4.12% | +0.83% |
| 7 | 7 | | JavaScript | 2.54% | +0.41% |
| 8 | 9 | ^ | PHP | 2.49% | +0.62% |
| 9 | 19 | ^^ | R | 2.37% | +1.33% |
| 10 | 8 | v | SQL | 1.76% | -0.19% |
| 11 | 14 | ^ | Go | 1.46% | +0.24% |
| 12 | 16 | ^^ | Swift | 1.38% | +0.28% |
| 13 | 20 | ^^ | Perl | 1.30% | +0.26% |
| 14 | 12 | v | Assembly language | 1.30% | -0.08% |
| 15 | 15 | | Ruby | 1.24% | +0.03% |
| 16 | 18 | ^ | MATLAB | 1.10% | +0.04% |
| 17 | 11 | vv | Groovy | 0.99% | -0.52% |
| 18 | 33 | ^^ | Rust | 0.92% | +0.55% |
| 19 | 10 | vv | Objective-C | 0.85% | -0.99% |
| 20 | 24 | ^^ | Dart | 0.77% | +0.13% |

图 1 TIOBE 2020 年 9 月 TOP20 编程语言排行

这些编程语言，在程序员的手中，在各个行业领域创造出了一个又一个优秀的软件，推动了社会的信息化和科技进步。而这些众多的编程语言，就如同地球上的生命一样，是从无到有，逐渐进化而来的。

现在，我们就来看看编程语言的进化史。

## 编程语言的诞生

1801 年，法国发明家雅卡尔（Joseph Marie Jacquard）用打孔卡为一台织布机编写指令，在挂毯上织出了“hello, world”字样。他设计出人类历史上首台可设计织布机“雅卡尔织布机”，对将来发展出其他可编程机器（例如计算机）起了重要作用。

1842 年，英国诗人拜伦之女、数学家 Ada Lovelace（1815-1852，如图 2 所示）写出了世界上第一个程序，但遗憾的是当时并没有任何计算机能够用来运行她的程序。Ada Lovelace 为巴贝奇分析机写了一个计算伯努利数的算法实现，因此被后世公认为是世界上第一个程序员。实际上，分析机由于其设计思想过于先进，在当时根本没有被制造出来。作为计算机程序创始人，Ada 建立了循环和子程序概念。为计算程序拟定“算法”，写作了第一份

“程序设计流程图”。

为了纪念 Ada 对现代电脑与软件工程所产生的重大影响，美国国防部将耗费巨资、历时近 20 年研制成功的高级程序语言命名为 Ada 语言。

图 2 计算机程序创始人 Ada

1936 年， 英国数学家图灵（Alan Turing）发明了世间一切程序语言的最终形态，与通用图灵机（Universal Turing machine）等价的语言被称为图灵完备的（Turing completeness），它定义了“什么样的语言可以被称作是程序语言”。

## 编程语言发展阶段和分类

到目前为止，程序设计语言的发展经过了第一代机器语言、第二代汇编语言、第三代高级语言、以及最新的第四代语言四个阶段，每一个阶段的发展，都让程序设计效率得以提高。

机器语言是计算机能直接识别和执行的一组机器指令的集合。它编写出的程序都是由 0 和 1 构成的符号串，可读性差，还容易出错，不易交流和维护。

早期的程序设计均使用机器语言。程序员们将用 0, 1 数字编成的程序代码打在纸带或卡片上，1 打孔，0 不打孔，再将程序通过纸带机或卡片机输入计算机，进行运算。这样的机器语言由纯粹的 0 和 1 构成，十分复杂，不方便阅读和修改，也容易产生错误。

计算机科学家们很快就发现了使用机器语言带来的麻烦，于是发明了汇编语言。汇编语言的主体是汇编指令，由汇编指令，伪指令和各种运算符号组成。在汇编指令里使用“助记符号”来代替 0、1 码编程，即用助记符代替机器指令中的操作码，用地址符号或标号代替机器指令中的地址码，将机器语言变成了汇编语言。汇编语言提高了程序的可读性和程序开发效率。

汇编语言程序占用空间小，运行速度快，因此汇编语言常用来编写系统软件和过程控制软件。

但是，汇编语言和机器语言一样，都与具体的机器有关，因此我们可以将它们都称为面向机器的语言，也就是所谓的为低级语言。程序员用它们编程时，不仅要考虑算法实现，还要熟悉具体机器的内部构造，需要对硬件结构及其工作原理都十分熟悉，但这是非常困难的，

尤其是对非就计算机专业的编程人士，更是难以做到的。因此编程的效率依然有很大的提升空间。因此，科学家们又进一步发明了高级编程语言。

我们把与自然语言相近，并为计算机所接受和执行的计算机语言称为高级语言。低级语言面向的是机器，而高级语言则是面向用户的语言。无论何种机型的计算机，只要配备相应的高级语言的编译程序，用该高级语言编写的程序就可以在该机器上运行。我们平时常用的编程语言，比如 Java，C++，Go，Rust，Python 等都是高级语言。

一段高级语言 C 语言（有人也把 C 语言当做接近硬件的中级语言）代码如下：

```
int _tmain(int argc, _TCHAR* argv[])
{
    int a = 10;
    int b = 20;

    printf("%d\n",a+b);

    return 0;
}
```

对上面的代码进行反汇编，如图 3 所示：

```
int _tmain(int argc, _TCHAR* argv[])
{
000C13A0 55              push        ebp
000C13A1 8B EC           mov         ebp,esp
000C13A3 81 EC D8 00 00 00 sub         esp,0D8h
000C13A9 53              push        ebx
000C13AA 56              push        esi
000C13AB 57              push        edi
000C13AC 8D BD 28 FF FF FF lea         edi,[ebp-0D8h]
000C13B2 B9 36 00 00 00  mov         ecx,36h
000C13B7 B8 CC CC CC CC  mov         eax,0CCCCCCCCh
000C13BC F3 AB           rep stos    dword ptr es:[edi]
    int a = 10;
000C13BE C7 45 F8 0A 00 00 00 mov         dword ptr [a],0Ah
    int b = 20;
000C13C5 C7 45 EC 14 00 00 00 mov         dword ptr [b],14h

    printf("%d\n",a+b);
000C13CC 8B 45 F8        mov         eax,dword ptr [a]
000C13CF 03 45 EC        add         eax,dword ptr [b]
000C13D2 8B F4           mov         esi,esp
000C13D4 50              push        eax
000C13D5 68 3C 57 0C 00  push        offset string "%d\n" (0C573Ch)
000C13DA FF 15 BC 82 0C 00 call        dword ptr [__imp__printf (0C82BCh)]
000C13E0 83 C4 08        add         esp,8
000C13E3 3B F4           cmp         esi,esp
000C13E5 E8 5B FD FF FF  call        @ILT+320(__RTC_CheckEsp) (0C1145h)

    return 0;
000C13EA 33 C0           xor         eax,eax
}
000C13EC 5F              pop         edi
000C13ED 5E              pop         esi
000C13EE 5B              pop         ebx
000C13EF 81 C4 D8 00 00 00 add         esp,0D8h
000C13F5 3B EC           cmp         ebp,esp
000C13F7 E8 49 FD FF FF  call        @ILT+320(__RTC_CheckEsp) (0C1145h)
000C13FC 8B E5           mov         esp,ebp
000C13FE 5D              pop         ebp
000C13FF C3              ret
```

图 3 高级语言代码反汇编

对于高级语言中的：int a=10 语句，对应的汇编代码为：

mov dword ptr [a], 0Ah

而十六进制机器语言为：

C7 45 F8 0A 00 00 00

对应的二进制机器语言为：

1100 0111 0100 0101 1111 1000 0000 1010 0000 0000 0000 0000 0000 0000

因此，通过计算机编程语言的发展阶段，我们可以看出，程序员一开始使用汇编，然后使用面向过程的语言（如 Pascal 和 C），然后是面向对象语言（如 C++），随后便引入了托管机制，语言运行于受托管的执行环境上，比如 C#，Java，Go，这些语言都有自动的垃圾收集，类型安全等典型特性。

需要特别提到的是，2014 年诞生的 Rust 是一门革命性的编程语言，它拥有和 C/C++同样的高性能，却又比 C/C++安全。它既无 GC 机制，又不用手动释放内存，还不会有各种溢出漏洞，Rust 是 C/C++，Go，Java 等编程语言的直接竞争者。

我们还可以通过程序设计的不同风格，将程序设计语言分为不同的编程范式（programming paradigm）。而主要的编程范式有三种：命令式编程(Imperative)、声明式编程(Declarative)和函数式编程(Functional)。

命令式编程的主要思想是关注计算机执行的步骤，即一步一步告诉计算机先做什么再做什么。无论在 C, C++ 还是 C#, Java, Python,等编程语言里这种方式都是很常见的。比如下面的代码就是 C 语言中遍历链表时候的命令式编程语句：

```
if(h==NULL)
    return;
node *p=h;
while(p!=NULL){
    printf("%d ",p->val);
    p=p->next;
}
```

在遍历链表的时候，首先告诉计算机判断链表头结点是否为空，接着告诉计算机，从链表头结点开始，通过循环依次取出每个结点中的值并打印出来，直到链表最后一个结点遍历结束。

声明式编程是以数据结构的形式来表达程序执行的逻辑。它的主要思想是告诉计算机应该做什么，但不指定具体要怎么做。SQL，网页中的 HTML 和 CSS 属于声明式编程。这种编程的特点是它不需要创建变量用来存储数据，也不包含循环控制的代码如 for， while 等。

SQL 语句中的查询语句：

```
select * from mfusers where user_name='tom'
```

网页中的 HTML 代码：

```
<html>
<head>
<meta charset="utf-8">
<title>麦洛科菲-专注 IT 安全技术！ </title>
<meta name="keywords" content="麦洛科菲 Web 安全" />
<meta name="description" content="XSS,sqlinjection" />
<link rel="shortcut icon" href="Img/logo.ico" type="image/x-icon" />
```

```
</head>
<!--comment-->
<body>

hello html，你好，html

</body>
</html>
```

函数式编程和声明式编程思想是一致的：只关注做什么而不是怎么做。但函数式编程不仅仅局限于声明式编程。函数式编程最重要的特点是函数第一位，函数可以出现在任何地方，函数可以作为参数传递给另一个函数，函数可以作为返回值。大部分常见的编程语言都提供了对这种编程方式的支持，比如 C# 中的 LINQ ，Python、Java、Go、Rust 中的 Lambda 和闭包的概念。

Rust 中的函数编程，把函数指针作为参数传递：

```
fn add_one(x:i32)->i32 {
    x+1
}

fn myadd(f:fn(i32)->i32, arg:i32)->i32 {
    f(arg)+f(arg)
}
```

现在 C++中的 lambda 支持：

```
testFunc() {
    int a = 1, b = 2;
    auto lambda = [](int x, int y) ->int{
        b = a + 5;
        return a + b;
    };
    std::cout << lambda(a, b) << std::endl;
    std::cout << b << std::endl;
}
```

我们可以通过编程范式把大多数编程语言进行分类，包括：过程式编程（procedural programming，简称 pp），面向对象（object-oriented programming，简称 opp）编程和函数式（functional programming，简称 fp）编程。

从 pp 到 oop 再到 fp 是一个从指令式往声明式过渡的过程，其中 pp 最具有指令式特性，fp 最具有声明式特性，而 oop 是介于两者之间，并且 fp 是最靠近人类的自然语言，pp 最靠近机器的指令，比如 c 以及各种脚本都是过程式的，java 就是面向对象，haskell 就是函数式。

大多数语言并不纯粹是某一个特定的范式而是多个范式的组合，比如 Scala 就是面向对象与函数式相结合。

随着编程语言的发展，原本的边界也变得越来越模糊，现在各种主流的编程语言都受到

函数式语言的影响。因此，“多范式”程序设计语言也是一个发展趋势。

# 20 世纪 50 到 60 年代编程语言发展

在 20 世纪 50 年代到 60 年代，计算机科学家发明了汇编语言，Autocode，FLOW-MATIC，FORTRAN，IPL，Algol，COBOL，RPG，APL，Simula，SNOBOL，PL/I，JOSS，BCPL，Logo，B 等编程语言。

**汇编语言（Assembly Language）**

Kathleen Booth 于 1947 年在伦敦大学伯贝克学院（Birkbeck College, University Of London）参与 ARC2 计算机的研发工作中，就作了与发明汇编语言相关的理论工作，她被认为是世界上首个为计算机系统开发与设计汇编语言与汇编器的人。

汇编语言诞生于 20 世纪 50 年代中期，它起源于“电子离散时序自动计算机 EDSAC”。此时用户编写的程序中的指令由单字母指令码，十进制地址和终结字母组成。这基本上就是汇编语言的雏形。

第一个真正的汇编程序是“符号优化汇编程序(SOAP，Symbolic Optimal Assembly Program )系统”，它是 1955 年为 IBM650 计算机研制的。IBM650 用磁鼓作存储器，每条指令需要指出后继指令在磁鼓中的位置。

而 IBM704 计算机的符号汇编程序(SAP)是汇编程序发展中的一个重要里程碑。此后的汇编程序大都以这一系统为模型，其主要特征至今未发生本质的变化。

现在的汇编指令集可以分为以 ARM 汇编指令为代表的精简汇编指令集（RISC:reduced instruction set computer）和以 X86 汇编为代表的复杂汇编指令集（CISC:complex instruction set computer）。

**Autocode**

Autocode 由 Alick Glennie 于 1952 年在曼彻斯特大学为 Mark 1 计算机开发的编程语言，并被某些人认为是第一个编译型编程语言。Autocode 设计的主要目的就是增加编程语言的可理解性，因为之前的机器语言非常晦涩难懂。但是，它依然是一门非常依赖于机器的编程语言。

**FLOW-MATIC**

FLOW-MATIC (COBOL 语言的祖先)编程语言诞生于 1955 年。Flow-Matic 是第一个适用于商用数据处理的语言，其语法与英语语法类似。

**FORTRAN**

John Backus 和 IBM 于 1957 年发明了 FORTRAN 语言，它是世界上第一个真正的编译型计算机语言。

Fortran I 运行于 IBM 704 计算机上。Fortran I 支持：变量（变量名最多 6 个字符）、If 和 do 语句、格式化 IO。但不支持数据类型声明、单独编译子程序。Fortran II 发布于 1958 年，主要对 Fortran I 的编译器做了优化，并支持了单独编译的子程序。Fortran IV 发布于 1962 年，并在 1966 年被标准化为 Fortran 66，这也是第一个 Fortran 的标准化版本。

1978 年的 Fortran 77 是非重要的标准化版本，这时的 Fortran 语言已经大量的引进了其他计算机语言的特点，例如字符串处理、逻辑判断、带 else 子句的 if 语句等。Fortran 的设计思想基于冯诺依曼体系结构是命令式语言的鼻祖。

**IPL 与 LISP**

人工智能语言 IPL（Information Processing Language）诞生于 1956 年。IPL 相当于一种假想的计算机的机器语言，这种计算机是以列表处理为基础，这有别于冯诺依曼体系结构。IPL 的关键是证明了：基于列表处理的计算机的可行性。

1958 年夏，John McCarthy 在 IBM 研究代数表达式的微分运算时，产生了一系列对编程语言的新需求，包括：用数学函数控制流程的方法（即递归和条件表达式）；列表的自动分配；废弃列表的隐式回收。而这些新的需求，IPL 无法满足，所以需要设计新的语言。

1958 年秋，麦卡锡在 MIT 成立了人工智能小组，开始着手设计这种新的列表处理语言。这最终导致 1960 年，Lisp 的诞生。

1958 年，John McCarthy 和 Paul Graham 发明了 LISP。但 LISP 从未流行过。尽管欠缺足够的流行度，LISP 仍然被视作一门有影响力的语言，在关键的算法思想诸如递归（recursion）上尤为突出。

1960 年，Lisp 还没有来得及设计自己的语法，就被 John McCarthy 的学生实现了，因为没有语法，他的学生就以列表格式的语法分析树作为临时语法。 但没有想到的是随着 Lisp 语言广泛流传，这种语法被程序员迅速接受，并发现了其独特的魅力。

John McCarthy 希望 Lisp 的语法是数学表达式的元语言，命名为 M（Mate）表达式，类似于后面的 ML 语言的语法。但是这种想法遭到了当时 Lisp 程序员的反对，最后 Lisp 的语法固定在了 S 表达式上。

Lisp 建立在列表和 lambda 演算的基础上，它是函数式编程的鼻祖。

**Algol**

Algol 58 年诞生于 1958 年，由苏黎世会议制定。是第一个与具体机器无关的计算机语言。

随着 Fortran 的诞生，其他一些语言也在开发中，这些语言的语法格式各不相同，而且和 Fortran 一样，都与运行它们的具体计算机关联。用这些语言写的代码很难交流。于是 Algol 58 在 Fortran I 的基础做了扩展，并兼容了当时其他语言的一些特性，成为了程序设计思想的集成。

Algol 60，发表于 1960 年，在 Algol 58 的基础上做了重大改进：移入块结构、 支持两种子程序参数传递（按值传递和按名传递）、允许递归、支持本地数组（在栈上分配数组）。Algol 60 是命令式语言的一个里程碑。

Algol 在设计之初，就定下语法应当尽可能接近数学记法的目标。这使得 Algol 60 语法格式几乎影响到后来所有的计算机语言，并且 Algol 60 是很多文献表达算法的唯一正式方式。

Algol 60 是后来除 Cobol 之外所有计算机语言语法的鼻祖。

在高级编程语言的进化历史中，最重要的里程碑就是 ALGOL 算法语言的推出。ALGOL 是算法语言（ALGOrithmic Language）的简称，是在计算机发展史上首批清晰定义的高级语言，由欧美计算机学家合力在 1950 年代所开发。国际计算机学会(Association for Computing Machinery，ACM)将 ALGOL 模式列为算法描述的标准，启发 ALGOL 类现代语言 Pascal、Ada、C 语言等出现。

1950 年代末期，大量的新增计算机语言涌现，而这些新语言大都围绕单个体系结构如 UNIVAC 计算机、IBM700 系列计算机等开发，使不同系统用户间的交流变得困难。美国一些主要的计算机用户组建议研究与开发适用于与计算机无关的科学用程序设计语言。1958 年 5 月 27 日至 6 月 1 日，德国的应用数学和力学学会(Gesellschaft für Angewandte Mathematik

und Mechanik,GAMM)和 ACM 派人出席在苏黎世举行第一次设计会议，为新语言定下目标。新语言先被命名为国际代数语言(International Algebraic Language，IAL），后定名为 ALGOL，即算法语言。后来改称 Algol 58，成为 ALGOL 家族的第一个成员。

图灵奖获得者:艾伦.佩利(Alan J.Perlis)在巴黎举行的由全世界一流软件专家参加的讨论会上，发表了"算法语言 Algol 60 报告"，确定了程序设计语言 Algol 60。Algol60 语言的第一个编译器由 Edsger W. Dijkstra 来实现。1962 年，艾伦。佩利又对 Algol 60 进行了修正。

Algol 60 是程序设计语言发展史上的一个里程碑，它标志着程序设计语言成为一门独立的科学学科，并为后来软件自动化及软件可靠性的发展奠定了基础。2006 年 3 月 1 日，ACM 宣布彼得.诺尔（Peter Naur，丹麦天文学家，如图 4 所示）为 2005 年 A.M.图灵奖的获得者。这个奖项是由于诺尔在定义 Algol 60 这种程序设计语言方面的先驱性工作（他把 BNF（Bcakus-Naur-Form）范式引入 Algo60 中）而颁发给他的。Algol 60 是许多后来的程序设计语言，包括今天那些必不可少的软件工程工具的原型。

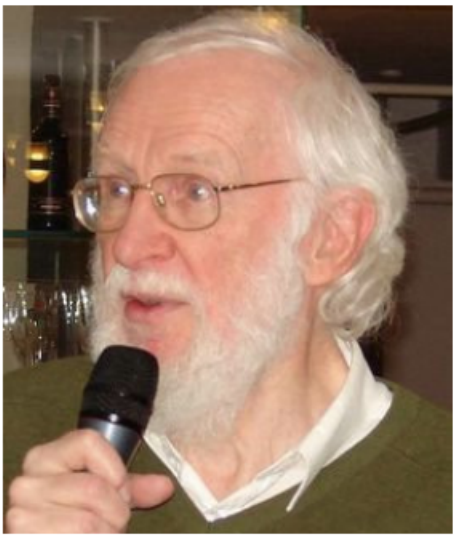


图 4 图灵奖获得者 Peter Naur

2002 年，前图灵奖得主 Edsger Dijstra 把 Algol 60 的发展描述为"一个绝对的奇迹"，标志着他所说的"计算科学"的诞生，因为 Algol 60 首次显示了自动计算能够并且应当成为学术关注的主题。 Algol 60 的发展是一群极具天赋的人才的工作成果，其中包括几位以后的图灵奖获得者。

**COBOL**

1959 年，Grace Hopper 和其他几个人一起发明了所谓的“面向范例（Boilerplate）的全大写化语言（Capitalization Of Boilerplate Oriented Language，COBOL）”。COBOL 语言以代码极其冗长和通篇大写字母的书写风格而闻名。

COBOL 语言的设计理念是：“数学程序应该用数学记法来写，数据处理程序应该用英语语句来写”。 商务应用显然属于后者，所以 COBOL 语言看起了就像是全大写的英语。COBOL 创造了一些新概念，比如宏（DEFINE）和数据结构。

COBOL 在商务领域被广泛的使用。COBOL 对后来的语言几乎没有影响。

**RPG**

RPG（Report Program Generator）编程语言诞生于 1959 年，它是应用于 IBM 的 AS/400 小型机上的高级语言，程序运行效率很高。RPG 起始是一个非常简单的面向事务处理的编程语言，用被储存在系统 IBM709 和 360 型号的 20 针打孔卡中的数据而生成报表。

**A 语言**

A 语言，即 APL 语言，它是在 1960 年左右由 IBM 的 Kenneth E．Iverson 设计的，设计的目的是把它作为描述计算机体系结构的工具。APL 最早是在名为《一种程序设计语言(A Programming Language)》的书中介绍的，这也是它名字的由来，所以我们也可以认为它就是 A 语言，与后面的 B 语言，C 语言，D 语言相呼应。

A 语言具有大量功能强大的运算符，对实现者造成了困难。最初，使用 A 语言的方法是通过 IBM 的打印终端，这些终端有特殊的打印球，提供语言所需的特殊的字符集。

A 语言具有这么多运算符的原因之一是因为它提供了大量数组的单元运算。大量的运算符提供了极强的表达能力，但也使得 A 程序很难阅读，使人们认为 A 是一种最适于用来“一次性”程序设计的语言：虽然用 A 语言写起程序来很快，但是程序在使用过后就应该扔掉，因为程序难以维护。

**SNOBOL**

SNOBOL 语言诞生于 1962 年，由贝尔实验室（Bell Laboratories）的三个人 D．J．Farber、R．E．Griswold 和 I．P．Polonsky 设计的。它是专为文本处理而设计的。SNOBOL 语言的核心是一些用于字符串模式匹配的强大的运算符。

SNOBOL 语言早期的应用之一是编写文本编辑器。由于 SNOBOL 语言的动态特征，它比其他的语言慢，因此不再用于这样的程序了。不过 SNOBOL 语言仍然是一种有生命力且有支持的语言，在一些不同的应用领域中用于各种文本处理任务。

**CPL 语言**

1963 年英国剑桥大学（如图 5 所示）推出了 CPL（Combined Programming Langurage）语言。CPL 语言在 ALGOL 60 的基础上接近硬件一些，但规模比较大，难以实现。

1967 年英国剑桥大学的 Matin Richards 对 CPL 语言做了简化，推出了 BCPL（the Basic Combined Programming Language）语言。

1970 年美国贝尔实验室的 Ken Thompson 以 BCPL 语言为基础，又作了进一步的简化，设计出了很简单的而且很接近硬件的 B 语言（取 BCPL 的第一个字母），并用 B 语言写出了第一个 UNIX 操作系统。但 B 语言过于简单，功能有限。

1972 年至 1973 年间，贝尔实验室的 D.M.Ritchie 在 B 语言的基础上设计出了 C 语言（取 BCPL 的第二个字母）。C 语言即保持 BCPL 语言和 B 语言的优点（精练、接近硬件），又克服了他们的缺点（过于简单，数据无类型等）。

图 5 英国剑桥大学

**Simula**

1964 年，在挪威计算机中心诞生了 SIMULA I，然后不断的做改进最终于 1967 年，产生了 SIMULA 67。SIMULA 67 是对 Algol 60 的扩展，是第一个定义类结构的语言，这也被认为是面向对象的起源。

将命令式编程中的数据和数据的有关函数集成在一起，就形成了面向对象编程中的对象，而对象的类型就是类。将命令式编程中主程序调用子程序的从属关系，变为面向对象编程中对象之间互相发送消息的平等关系。

**PL/I**

PL/I 编程语言诞生于 1964 年。之前的所有语言都是领域特定语言，例如： 人工智能领域的 Lisp、科学计算领域的 Algol 60，商务应用领域的 COMBOL 等。

PL/I 是跨科学计算和商务应用领域的计算机语言。PL/I 是 Algol 60、Fortran IV、COBOL 的结合体。

**BASIC 语言**

BASIC 语言诞生于 1964 年， John Kemeny 和 Thomas Kurtz 创造了 BASIC，设计的主要目的是：使得计算机语言可以被非理工科生使用，因为当时理工科学生使用的 Fortran 和 Algol 60 很难被文科生掌握。这样的设计目的使得 BASIC 非常适合于初学者。BASIC 来自于 Fortran IV，语法受到 Algol 60 影响。BASIC 是第一个被广泛使用的分时处理语言。

BASIC 语言对行号和 goto 存在无节制滥用。

**JOSS**

JOSS 诞生于 1966 年。JOSS 是第一个交互式语言，它有很多方言，从而使分时系统成为实用系统。

**BCPL**

BCPL 语言(Basic Combined Programming Language)是 1967 年由剑桥大学的 Matin Richards 在同样由剑桥大学开发的 CPL 语言上改进而来。BCPL 最早被用做牛津大学的 OS6

操作系统上面的开发工具。后来通过美国贝尔实验室的改进和推广成为了 UNIX 上的常用开发语言。

BCPL 有些类似于 Fortran，也是典型的面向过程的高级语言。BCPL 的语法更加靠近机器本身，适合于开发精巧，高要求的应用程序，同时对编译器的要求也不高。BCPL 也是最早使用库函数封装基本输入输出的语言之一，这使得它跨平台的移植性很好。BCPL 的代码用小写字母书写，有别于同时代的 BASIC 和 PASCAL。BCPL 对于字符串的支持很差，这种缺点一直延续到了它的后代中，BCPL 的内存管理也很糟糕。BCPL 程序从一个主函数中开始，功能模块可以添加在各个函数中，函数支持嵌套和递归。

BCPL 本身并没有被使用太长时间，这是因为它的后代们实在太强大了。1970 年，贝尔实验室的 Ken Thompson 在 BCPL 的基础上改进出了 B 语言，用于书写 UNIX。这个名字取自 BCPL 中的第一个字母。B 语言使用的时间更短，因为三年后的 1973 年同样是贝尔实验室的 Dennis Ritchie 将 B 语言进一步改进，并且取了 BCPL 中的第二个字母将其命名为 C 语言。而 C 语言和 C++则在日后成为了最流行的高级语言。

**Logo**

Logo 语言诞生于 1968 年。在 1967 年，由美国麻省理工学院（MIT）佩帕特（Seymour Papert）教授指导下的一个研究小组在 LISP 语言基础上，专门为儿童研制开发了 Logo 编程语言。Logo 语言虽然结构简单、却有丰富的表达方式，体现了现代计算机科学许多最新概念。

LOGO 语言是一种交互式的结构化程序设计语言，它为人们提供了良好的编程环境；它是模块化的，便于程序的修改与扩充；它也是过程化的，包含了过程、参数、变量等重要概念，并允许递归调用；它有丰富的数据结构类型；有生动的图形处理功能。

**B 语言**

B 语言是贝尔实验室开发的一种通用的程序设计语言，它是于 1969 年前后由美国贝尔实验室的电脑科学家 Ken Thompson 在 Dennis Ritchie（如图 6 所示）的支持下设计出来。后来，Dennis Ritchie 以 B 语言为基础开发出 C 语言，C 语言是现在世界上最常用的编程语言之一。自从被 C 语言取代之后，B 语言几乎已遭弃置。

B 语言是一种无数据类型语言，所有的数据都和机器字相对应，并且程序员可以直接通过内存地址操作内存。B 语言最初在 DEC 的 PDP-7 和 PDP-11 小型机的早期 Unix 版本和 Honeywell 机器的 GCOS 系统上实现。

B 语言的无数据类型特性使得其在 PDP-11 机器上无法以比较优雅的方式访问字符类型，因而从 1971 年开始，Dennis Ritchie 开始对 B 语言进行改造，其中最重要的是引入了数据类型的概念，这些改造最终使 B 语言演化为了计算机发展历史上最为重要的编程语言——C 语言。

时至今日，除了在一些古老的大型机或是特殊的嵌入式平台上，B 语言已经芳踪难觅了。但 B 语言在编程语言发展历史上重要的承前启后的里程碑地位，却将永远为人们铭记。

B 语言是从 BCPL 系统中删减了汤普逊认为非必备的组件以便能运行在当时的小型计算机上而产生的。B 语言还包括了汤普逊的一些个人偏好（主要在一些特定的程序中减少非空格字符的数量）。

和 BCPL 以及 FORTH 类似，B 语言只有一种数据类型，计算机字。大部分操作将其作为整数对待（例如进行+、-、*、/操作），但其余操作将其作为一个复引用的内存地址。在许多方面 B 语言更像是一个早期版本的 C 语言，它还包括了一些库函数，其作用类似于 C 语言中的标准输入/输出函数库。

但 B 语言过于简单，功能有限。1972 年至 1973 年间，贝尔实验室的 Dennis Ritchie 在 B 语言的基础上设计出了 C 语言（取 BCPL 的第二个字母）。C 语言既保持了 BCPL 语言和 B 语言的优点（精练、接近硬件），又克服了他们的缺点（过于简单，数据无类型等）。1973 年初，C 语言的主体完成。Thompson 和 Ritchie 开始用它完全重写了 UNIX。随着 UNIX 的发展，C 语言自身也在不断地完善。

1977 年，Dennis M.Ritchie 发表了不依赖于具体机器系统的 C 语言编译文本《可移植的 C 语言编译程序》

图 6 图灵奖获得者：Ken Thompson（左）与 Dennis M. Ritchie（右）

## 20 世纪 70 年代编程语言发展

在 20 世纪 70 年代，计算机科学家发明了 Pascal，Forth， basic，C，Smalltalk，Prolog，ML， Scheme， SQL 等编程语言。

**Pascal**

Pascal 语言诞生于 1971 年，来自于 Algol 68。Niklaus Wirth 创造了 Pascal，一个过程式的语言。很快就有人开始声讨 Pascal，因为它使用了类似“x := x + y”这样的语法，而不是更为人熟知的类 C 语法“x = x + y”。而事实上当时 C 还没有被发明出来。

Pascal 重大影响来自于程序设计教学。它的出现取代了 Fortran 作为程序设计入门语言的地位。Pascal 在工业界也有广泛的使用。

**Forth**

Forth 语言诞生于 1970 年，它是 Charles Moore 发明的基于堆栈、交互式、具有简单性哲学思想的计算机编程语言和环境，已被证明特别适合于软件代码在千行数量级的中规模嵌入式系统应用，作为国外大公司竞争系统开发周期、生产成本和系统可靠性的秘密武器之一。

Forth 被广泛应用于天文、军事、航空、航天、工业自动化、图形、仪器仪表、工作站领域，并且研制了以 Forth 为体系结构的处理器芯片。

**C 语言**

C 语言诞生于 1972 年。贝尔实验室的 UNIX 操作系统，最初版本使用汇编语言写成。

1970 年在 UNIX 上实现的第一个高级语言叫做 B 语言，B 的目的是：用于开发 UNIX 系统程序，以及新版本的 UNIX 操作系统。B 以 BCPL 语言为基础。BCPL，诞生于 1967 剑桥大学，它的祖先是 1962 年诞生诞生于同一所大学的 CPL 语言。B 和汇编一样数据类型就是机器字，这极大的限制了 B 程序在不同位数的机器之间的迁移。

C 语言就是 B 的改进版本，同样以 BCPL 基础，并在此基础上引入了 Algol 68 的特点，使得 C 语言成为了一个可移植系统程序语言。C 语言语法中对操作符的大量灵活的使用，极大的影响了后来的一批程序语言。

C 语言由美国计算机科学家 Dennis Ritchie 发明。C 语言的发明，是计算机科学发展史上的一个伟大的创举，直接影响了后来的 IT 技术革命和互联网技术革命。时至今日，绝大多数的底层软件，包括操作系统（如 Windows, Linux, UNIX 等），数据库软件（如 MySQL），办公软件（如 OpenOffice），杀毒软件，网络通信协议，包括 JAVA 语言的实现，互联网，以及各种内核驱动等都是用 C 语言开发的。C 语言作为编程语言之王，是当之无愧的。

截止 2020 年 9 月，C 语言在 TIOBE 编程语言使用排行榜上依然占据 TOP1 的位置。

**Smalltalk**

Smalltalk 语言诞生于 1972 年，前身叫做 Flex 系统，最早是由 Alan Kay 提出的，然后逐渐有了 Smalltalk72，Smalltalk76。经过近 10 年的改进，Xerox 研究小组于 1980 年正式推出了 Smalltalk80 系统。

Smalltalk 的主要概念，如对象(object)、消息(message)和类(class)源于早期的 Simula 语言。Simula 仍然使用了面向过程与数据的 Algol 语言的基本数据结构和控制结构，然而，Smalltalk80 将所有的数据结构和控制结构都表示为对象和消息，类也可以用对象和消息来进行描述。Smalltalk80 实现了集成化的、交互式的程序设计环境，把面向对象的思想从语言级提高到程序员接口级。

1980 年，Alan Kay 创造了“面向对象”这个词。当被问到它的含义时，他说：“Smalltalk 程序本身就是对象。”当被问到对象是由什么组成时，他回答：“对象”。当再一次被问到这个问题时，他说“看，它从里到外都是对象。直到你抽出一只乌龟”。

Smalltalk 的设计从很大程度上受到了 Logo 的影响，它是第一个完全支持面向对象编程的语言。Smalltalk 源于 SIMULA 67，完善并定义了消息发送，和对象的继承关系。在 Smalltalk 中，所有数据都是对象：消息的发送者，消息的接受者，消息，消息的结果等。对象的类型是类。类中可以定义方法，属性等。

**Prolog**

Prolog 语言诞生于 1972 年，是逻辑式编程的鼻祖。所谓逻辑式编程，就是以形式逻辑的方式将计算过程传递给计算机。Alain Colmerauer 设计了逻辑编程语言 Prolog。他的目标是创造一个具有两岁小孩智商的程序语言。为了证明他成功达到了这个目标，他展示了一个 Prolog 程序，它对于每条查询都会机智地给出相同的回答：“No”。

**ML**

1973 年，Robin Milner 创造了 ML。由 ML 衍生而来的 SML 加上了一套形式语义的规范。其他 ML 家族的著名语言还包括 OCaml，F#，和 Visual Basic。

ML 是建立在 H-M（Hindley－Milner）类型推断的基础上的。Visual Basic 近年来吸收了函数式编程里的不少东西。最典型的是它具备和 ML 相似的类型推断。

ML 开创了以 Algol 作语法的函数式语言的先河。

**SQL**

SQL 语言 1974 年由 Boyce 和 Chamberlin 提出，并首先在 IBM 公司研制的关系数据库系统 SystemR 上实现。由于它具有功能丰富、使用方便灵活、语言简洁易学等突出的优点，深受计算机工业界和计算机用户的欢迎。1980 年 10 月，经美国国家标准局(ANSI)的数据库委员会 X3H2 批准，将 SQL 作为关系数据库语言的美国标准，同年公布了标准 SQL，此后不久，国际标准化组织(ISO)也作出了同样的决定。

SQL 从功能上可以分为 3 部分：数据定义、数据操纵和数据控制。

SQL 的核心部分相当于关系代数，但又具有关系代数所没有的许多特点，如聚集、数据库更新等。它是一个综合的、通用的、功能极强的关系数据库语言。

**Scheme**

Scheme 作为一种 Lisp 方言，诞生在 MIT，时间是 1975 年。Guy Steele 和 Gerald Sussman 创造了 Scheme。他们的工作导致了一系列以《Lambda 之究极（Lambda the Ultimate）》为标题开头的论文发表。Lambda 因为其概念相对难以理解而被大众所忽视，直到 Java 语言终于让它们变得广受欢迎。

Scheme 第一个引入词法作用域，延续体，闭包等概念。并将数据和函数等同对待。

Scheme 的简约优美使它博得了计算机语言皇后的桂冠，并与另一个以优雅著称的 Pascal 一起成为教学语言。

Scheme 是函数式语言，主要用于数学算法教学，Pascal 是命令式语言主要用于程序设计教学。

# 20 世纪 80 年代编程语言发展

在 20 世纪 80 年代，计算机科学家发明了 C++，R，Ada，Common Lisp，Matalab，EIFFEL，Objective-C，Self，Erlang，Perl，Tcl 等编程语言。

**C++**

C++ 诞生于 1980 年，起初被称为“C with Class”，即有类的 C 语言，1983 年 7 月正式更名为 C++。

1983 年， Bjarne Stroustrup（如图 7 所示）把他所听说过的一切都试图嫁接到 C 上，创造出了 C++。

C++是在 C 的基础上，以 SIMUAL 67 的方式对命令式编程的面向对象扩展。C++的成功在于完全兼容 C 的情况下支持了面向对象，从而使面向对象编程成为了命令式编程的一个子类，传统的命令式编程被叫做面向过程编程。

因为对 C 的完全兼容，ASNI 在 1989 年的标准化版本 C 89，合并了 C 和 C++，称为 C/C++。C/C++的最重要版本是 1999 年的 C 99。几乎吸收了当时被认为比较有效的计算机语言特性：面向过程编程、面向对象编程、泛型编程、自定义操作符重载等。

由于 C/C++吸收了太多的语言特性，使得 C/C++成为世界上最难精通的语言。

我们把 C++11 及之后的标准叫做现代 C++。C++11 为 C++98 发布后 13 年来的第一次重大修正：包含约 140 个新特性，以及对 C++03 约 600 个缺陷修正。脱胎换骨，“看起来像一门新语言”，性能也得到了极大的提升。

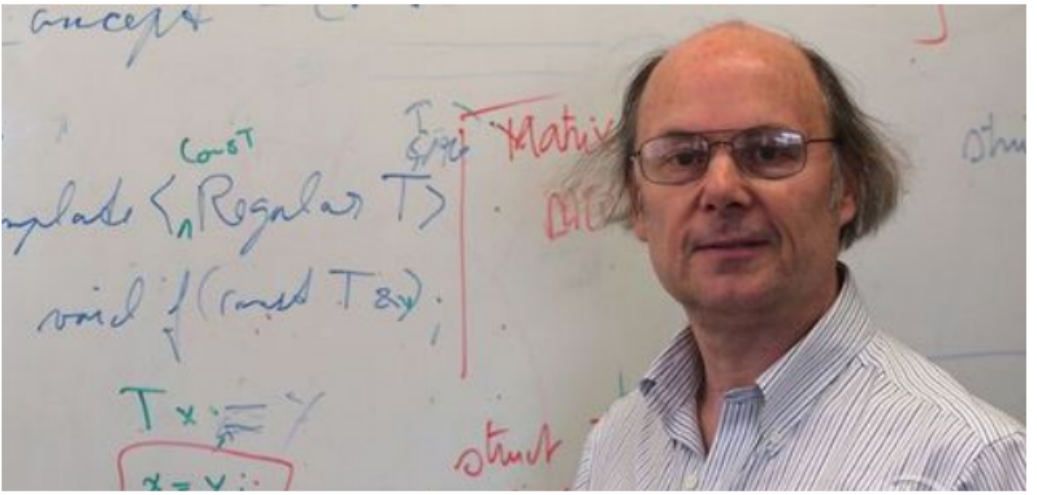


图 7 C++之父 Bjame stroustrup

C++程序过于复杂和庞大。另外由于消费电子产品所采用的嵌入式处理器芯片的种类繁杂，如何让编写的程序跨平台运行也是个难题。为了解决困难，Sun 公司研发人员并没有开发一种全新的语言，而是根据嵌入式软件的要求，对 C++进行了改造，去除了留在 C++的一些不太实用及影响安全的成分，并结合嵌入式系统的实时性要求，开发了一种称为 Oak 的面向对象语言，也就是后来长期统治互联网的编程语言 Java。

**R 语言**

R 语言在统计领域广泛使用。它诞生于 1980 年左右，是 S 语言的一个分支。可以认为 R 是 S 语言的一种实现。

S 语言是由 AT&T 贝尔实验室开发的一种用来进行数据探索、统计分析和作图的解释型语言。最初 S 语言的实现版本主要是 S-PLUS。S-PLUS 是一个商业软件，它基于 S 语言，并由 MathSoft 公司的统计科学部进一步完善。

后来，新西兰奥克兰大学的 Robert Gentleman 和 Ross Ihaka 及其他志愿人员开发了一个 R 系统。由“R 开发核心团队”负责开发。R 可以看作贝尔实验室（AT&T BellLaboratories）的 RickBecker，JohnChambers 和 AllanWilks 开发的 S 语言的一种实现。当然，S 语言也是 S-Plus 的基础。所以两者在程序语法上可以说是几乎一样的，可能只是在函数方面有细微差别，程序十分容易地就能移植到一程序中，而很多一的程序只要稍加修改也能运用于 R。

R 是用于统计分析、绘图的语言和操作环境。R 是属于 GNU 系统的一个自由、免费、源代码开放的软件，它是一个用于统计计算和统计制图的优秀工具。

**Ada 语言**

Ada 语言诞生于 1980 年，是嵌入式系统应用语言。Ada 83 是 1983 年的标准化版本。1995 年的，Ada 95 标准语言在 ARM 中定义。Ada 的最新版本是 Ada 2005。Ada 广泛的用于商务、国防航空和交通运输等领域。

欧洲航天局阿利亚纳五号火箭失败的一个普遍说法是因为 Ada 语言在编译过程的检查失败导致的。 将大的浮点数转换成整数是一种常见的程序错误来源。

1996 年 6 月 4 日，对于 Ariane 5 火箭的初次航行来说，这个错误产生了灾难性的后果。发射后仅仅 37 秒，火箭偏离它的飞行路径，解体并爆炸了。火箭上载有价值 5 亿美元的通信卫星。6 亿美元付之一炬。后来的调查显示，控制惯性导航系统的计算机向控制引擎喷嘴的计算机发送了一个无效数据。失事调查报告指出，火箭爆炸是因为：

During execution of a data conversion from 64-bit floating point to 16-bit signed integer

value, the floating point number which was converted had a value greater than what could be represented by a 16-bit signed integer. This resulted in an Operand Error.

翻译过来就是，Ada 程序在执行数据转换期间，将 64 位浮点数转换为 16 位有符号整数时，溢出了。

为了纪念世界上第一个程序员 Ada Lovelace，Jean Ichbiah 和美国国防部将此语言用 Ada 命名。Ada 曾经是美国国防部指定的嵌入式计算机系统唯一开发语言，在其研发上耗资巨大。虽然有充分的证据显示 Ada 的整型范围溢出检查失败导致欧空局的一枚 Ariane 5 运载火箭在发射过程中解体爆炸，不过美国国防部发言人对此表示：与我无关。

**Common Lisp**

Common Lisp 诞生于 1984 年。Common Lisp，缩写为 CL，是 Lisp 的众多方言之一，与 Scheme 合称现代两大 Lisp 方言，标准由 ANSI X3.226-1994 定义。它是为了标准化此前众多的 Lisp 分支而开发的，导致其语言规范和标准长达千页。它本身并不是一个具体的实现，而是各个 Lisp 实现所遵循的规范。它操作的数据是动态数据类型，但也可用类型声明来提高效率和增强安全性。

Common Lisp 被用于很多成功的商业系统，最著名的要数 Yahoo! 商店的站点。

Common Lisp 是一个现代的，多重范式的，高性能，可编译的标准化 ANSI 编程语言。相对于各种嵌入在特定产品中的 Lisp 方言，例如 Emacs Lisp 和 AutoLISP，Common Lisp 是一种通用用途的编程语言。不像很多早期的 Lisp，Common Lisp 同 Scheme 一样，变量有作用域。

Common Lisp 具有高度灵活性，对象化编程以及快速的框架能力提供优异的支持。同时它还提供强大的宏能力，使编程者在编码时定制自己的应用，并给编程者提供了高度灵活的运行环境，可在运行时修改和调试程序。

**MATLAB**

MATLAB 诞生于 1984 年，它和 Mathematica、Maple 并称为三大数学软件。它在数学类科技应用软件中在数值计算方面首屈一指。

MATLAB 可以进行矩阵运算、绘制函数和数据、实现算法、创建用户界面、连接其他编程语言的程序等。MATLAB 的基本数据单位是矩阵，它的指令表达式与数学、工程中常用的形式十分相似，故用 MATLAB 来解算问题要比用 C，FORTRAN 等语言完成相同的事情简捷得多，并且 MATLAB 也吸收了像 Maple 等软件的优点，使 MATLAB 成为一个强大的数学软件。

MATLAB 在新的版本中也加入了对 C，FORTRAN，C++，JAVA 的支持。

**Eiffel**

Eiffel 语言诞生于 1985 年，它是继 Smalltalk80 之后的另一个“纯”OOPL（面向对象编程语言）。这种语言是由 OOP 领域中著名的专家 Bertrand Meyer 等人 20 世纪 80 年代后期在 ISE 公司(Interactive Software Engineering Inc.)开发的，它的主要特点是全面的静态类型化、有大量的开发工具、支持多继承。

Eiffel 是由 Eiffiel 公司（现在被称为 ISE，Interactive Software Engineering）在 1985 年设计的。最初 ISE 在项目开发中，需要一种能体现现代软件工程学思想、现代的、面向对象的开发环境，但 ISE 认为当时没有这样的工具。因此，ISE 开发了 Eiffel 作为开发产品的内部工具。Eiffel 的名字来自工程师 Gastave Eiffel，他为 1889 年世界博览会设计了著名的艾菲尔铁塔，并在计划的工期和预算内完成了这个工程。ISE 希望用 Eiffel 所构建的软件工程项

目也能达到这样的目标。

**Objective-C**

Objective-C 诞生于 1983 年，布莱德·考克斯(Brad Cox)在其公司 Stepstone 发明 Objective-C。后来被苹果公司选中，和C++相似，也是对 C 的面向对象扩展。所不同的是 C++以 SIMUAL 67 的面向对象为基础，Objective-C 以 Smalltalk 的面向对象基础。

Object-C 一直伴随着苹果公司的操作系统一起发展，这两年随着 iOS 的开发被广泛使用。

**Self 语言**

Self 诞生于 1986 年。Self 源自于 Smalltalk。Self 对面向对象做了深入与研究，提出了“万物皆对象”的思想。它删除了面向对象概念中的类的概念，因为它认为类也是一种对象；消息仍然作为最基本操作；把对象的属性理解为获取或更改属性这两种方法，从而把属性的概念简化为方法；取消了变量和赋值，替代它们的是通过消息来读槽和写槽。

在 Self 中对象创建对象的方式是自我拷贝，所以叫做原型。Self 是原型面向对象语言的鼻祖，就像 SIMUAL 67 是类面向对象语的鼻祖。

**Erlang**

Erlang 编程语言是由爱立信公司于 1986 年开发的。Erlang 是一种面向并发运行环境的通用编程语言。最初目的是进行大型电信交换设备的软件开发，是一种适用于大规模并行处理环境的高可靠性编程语言。

随着多核处理器技术的日渐普及，以及互联网、云计算等技术的发展，该语言的应用范围也有逐渐扩大之势。

**Perl**

Perl 发布于 1987 年。Perl 起源于操作系统的控制台（shell）命令。用户将完成某个工作的所有命令保存在一个 sh 文件里，这样只要执行这个文件就可完成对应功能，省去每次都要输入所有命令的麻烦。sh 文件就叫做脚本。脚本只是一些命令行，发展到 Perl 时已经是一个相当成熟的语言了。

1987 年，Larry Wall 在电脑前打了个盹，他的脑门压到了电脑的键盘上，电脑显示器上出现的神秘字符串，受到这些神秘字符串的启发，他发明了 Perl 语言。

**TCL**

Tcl 诞生于 1988 年，作为一种脚本语言，它由 John Ousterhout 创建。 TCL 易学且功能强大。TCL 经常被用于快速原型开发，脚本编程，GUI 和测试等方面。

## 20 世纪 90 年代编程语言发展

在 20 世纪 90 年代，计算机科学家发明了 Haskell，Python，Ruby，Lua，Delphi，Java，PHP，Javascript，Ocaml，Rebol，D 等编程语言。

**Haskell**

Haskell 诞生于 1990 年，它是一种标准化的，通用纯函数式编程语言，有非限定性语义和强静态类型。Haskell 语言是 1990 年在编程语言 Miranda 的基础上标准化的，并且以 λ 演

算为基础发展而来。这也是为什么 Haskell 语言以希腊字母“λ”（Lambda）作为自己的标志。Miranda 诞生于 1986 年，部分基于 ML 语言。

它的命名源自美国逻辑学家哈斯凯尔·加里（Haskell Brooks Curry），他在数理逻辑方面上的工作使得函数式编程语言有了广泛的基础。在 Haskell 中，“函数是第一类对象”，“函数是一等公民”。作为一门函数编程语言，主要控制结构是函数。Haskell 具有“证明即程序、命题为类型”的特征。

Haskell 语言的最重要的两个应用是 GHC（Glasgow Haskell Compiler）和 Hugs（一个 Haskell 语言的解释器）。

**Visual Basic**

1991 年，Visual Basic 语言诞生。1991 年 4 月，微软公司推出了 Visual Basic 1.0 Windows 版本，这个连接编程语言和用户界面的进步被称为 Tripod（有些时候叫做 Ruby），最初的设计是由阿兰·库珀（Alan Cooper）完成的。在当时，它是第一个可视编程软件。

**Python**

Python 诞生于 1991 年，是一种跨平台的计算机程序设计语言，它是一个结合了解释性、编译性、互动性和面向对象的脚本语言。最初被设计用于编写自动化脚本(shell)，随着版本的不断更新和语言新功能的添加，越多被用于独立的、大型项目的开发。作者为了打发圣诞节的无趣，决心开发一个新的脚本解释程序，做为 ABC 语言的一种继承。ABC 是作者开发的教学语言。Python 和 Perl 用于相同领域，是面向过程与面向对象相结合的语言。

1991 年，荷兰程序员 Guido van Rossum 为了一次神秘的手术而进行了一次阿根廷之旅。回来后他带着一个巨大的颅疤，发明了 Python。

Python 可以应用于 Web 和 Internet 开发，科学计算和统计，人工智能，软件开发，后端开发，网络爬虫等。

Python 是人工智能和大数据开发的首选编程语言，具有丰富和强大的库，常被称为胶水语言，能把其他语言制作的模块(如 C/C++库)像胶水一样联结在一起，封装为 Python 扩展类库。Python 现已逐步在网络爬虫、数据分析、AI、机器学习、Web 开发、金融、运维、检验、安全等多个领域发展壮大。

深度学习 Keras 库之父 Chollet 指出：我爱 Python，Python 非常直观和优雅，但 Python 真正的杀手锏并不是语言本身，而是周围的生态系统和社区。无论你需要做什么，几乎都有一个 Python 库在做这件事，你不必花时间去做。

**Ruby**

Ruby 是一种纯面向对象解释式脚本语言，诞生于 1996 年，作者基于对 Perl 和 Python 的不满而设计的纯面向对象脚本语言。Ruby 基本等于 Smalltalk 加 Lisp。

1995 年，松本“Mad Matz”行弘创造出了 Ruby 语言，该语言后来被它的真正发明者 David Heinemeier Hansson 重新命名为 Ruby on Rails。

Ruby 被世界知名的黑客软件 MetaSploit 渗透测试工具选为官方的渗透编程语言。

**Lua**

Lua 是一种关联列表函数式面向对象脚本语言，诞生于 1993 年，设计目的是嵌入引用程序，成为其脚本。

Lua 以小巧性著称，基于 Self 和 Scheme，它是第一个以关联列表最为主要数据的函数式语言。

**Delphi**

Delphi (Object Pascal)诞生于 1995 年，是对 Pascal 的面向对象扩展，借鉴了 C/C++的经验，但没有像 C/C++后来那样的疯狂变化。

Delphi 是美国 Borland（宝兰）公司开发的工作在 Windows 平台下的开发工具，它的前身是在 DOS 下的产品 Borland Turbo Pascal，Turbo Pascal 使用的是 Pascal 语言。从 Turbo Pascal 5.5 版本开始，Borland 公司在传统 Pascal 的基础上加入了面向对象的功能，从而推出了 Delphi。

**Java**

1995 年，Sun 公司的 James Gosling（如图 8 所示）发明了一种被称为 Oak 的新的编程语言。

1995 年，互联网的蓬勃发展给了 Oak 机会。Sun 公司首先推出了可以嵌入网页并且可以随同网页在网络上传输的 Applet（Applet 是一种将小程序嵌入到网页中进行执行的技术），并将 Oak 更名为 Java（在申请注册商标时，发现 Oak 已经被人使用了，再想了一系列名字之后，最终，使用了提议者在喝一杯 Java 咖啡时无意提到的 Java 词语）。5 月 23 日，Sun 公司在 Sun world 会议上正式发布 Java。IBM、Apple、DEC、Adobe、HP、Oracle、Netscape 和微软等各大公司都纷纷停止了自己的相关开发项目，竞相购买了 Java 使用许可证，并为自己的产品开发了相应的 Java 平台。

1996 年 1 月，Sun 公司发布了 Java 的第一个开发工具包（JDK 1.0），这是 Java 发展历程中的重要里程碑，标志着 Java 成为一种独立的开发工具。

图 8 Java 之父 James Gosling

Java 是一种虚拟机面向对象语言，是一个相对繁冗的、带垃圾收集的、基于类的、静态类型的、单分派的面向对象语言，拥有单实现继承和多接口继承。在 Java 之前计算机编程语言要么由解释器解释执行源代码，叫做脚本语言；要么将源代码编译连接成实际的计算机指令后再由计算机执行，叫做静态语言。

Java 运行于 JM（Java 虚拟机）上，JM 和解释器的区别是：JM 会将 Java 源代码编译成字节码，然后然后再由 JM 解释执行。由于编译和执行过程是交错的，所以从外部来看就像是 JM 直接执行 Java 源代码一样。当然也可以，事先将 Java 源文件编译好，这样 JM 就可以直接执行了。

Java 1.0 是 C++的简化版本，保留了具有数据类型和类等一小半功能，删除了结构体，

泛型编程等一多半的功能。

Java 5.5 支持了泛型编程。

Java 是面向对象编程使用人数最多的语言。（C/C++的大部分人只使用 C 部分，即面向过程编程）。

2001 年， Anders Hejlsberg 发明了 C#。C#是一个相对繁冗的、带垃圾收集的、基于类的、静态类型的、单分派的面向对象语言，拥有单实现继承和多接口继承。

2003 年，一个叫 Martin Odersky 创造了 Scala，一种结合了面向对象和函数式编程的语言。

这两种语言都试图和 Java 竞争。

**PHP**

PHP 是一种 Web 服务器脚本语言。PHP 诞生于 1995 年。PHP 和 JavaScript 的语法相似，都是嵌入在 HTML 中，只不过一个在服务器端，一个在浏览器端。

1995 年，在家门口附近的一个意大利饭馆用餐时，Rasmus Lerdorf 意识到他吃的那盘意大利面正好是一个用来理解 WWW 万维网的极好模型，而所有的 Web 应用都应该仿照它们的媒介那样去做。在他的餐巾的背后，他设计出了著名的“可编程超链接 Pasta（Programmable Hyperlinked Pasta，PHP）”语言。PHP 的文档至今仍然保留在那片餐巾上。

PHP 最显著的特点就是：代码是可以直接嵌在 HTML 文档中的。

不知道从什么时候开始，也不知道由于什么原因，PHP 被程序员称为“世界上最好的编程语言”。以此完美的解决了由编程语言带来的无休止的优劣之争。

**Javascript**

Javascript 诞生于 1995 年，它是一种浏览器脚本语言。1995 年，网景公司发行的 Netscape Navigator 2.0，它的脚本语言就是 JavaScript。

JavaScript 和 Java 没有任何关系，它的前身是 ScriptEase。ScriptEase 的诞生于 1992 年，原名 C-minus-minus，是一个足够强大可以替代宏操作的脚本语言，同时保持与 C 足够的相似性，以便开发人员能很快学会。ScriptEase 基于 C 和 Self 语言，支持面向过程和原型面向对象编程。

JavaScript 后来由引入了 Lua 语言，支持以关联列表作为数据格式的函数式编程，这种关联列表就是著名的 JSON。

最后 JavaScript 是一个表面叫 Java 脚本，但是和 Java 没有任何关系；表面上是一个像 C 语言一样的面向过程语言，但实际上是一个原型面向对象语言；表面上你创建了一个对象，但其实只是一个关联列表；表面上你在定义变量和赋值，其实只是<键，值>对；表面上你声明函数，其实只是<键，lambda 表达式>对。表面上是一个命令式编程语言，但本质上是一个以关联列表为数据格式的函数式编程语言。

1995 年， Brendan Eich 读完了历史上所有在程序语言设计中曾经出现过的错误，自己又发明了一些更多的错误，然后用它们创造出了 LiveScript。之后，为了紧跟 Java 语言的潮流，它被重新命名为 JavaScript。

当时，网景公司管理层都是 Java 的信徒，坚信将来的 Java 会发展的好。因此网景公司决策：未来的网页脚本语言必须“看上去与 Java 足够类似”，但是比 java 简单，使得非专业的网页作者也能很快上手。于是，负责该脚本的设计者 Brendan Eich 花了 10 天的时间设计出了 JavaScript。

**OCaml**

OCaml 是一种支持面向对象的函数式语言。Caml 诞生于 1996 年，基于 ML 和 Haskell。OCaml 是 Caml 的面向对象版本，发布于 2006 年。

**Rebol**

Rebol 语言诞生于 1997 年，是针对互联网通讯设计的。REBOL 是一种高级解释语言，允许你访问和控制互联网的资源，而且它的便捷让你可以开始考虑把互联网当成你的个人操作系统。

**D 语言**

D 语言诞生于 1999 年，它是一种通用计算机程序语言，威力强大、功能丰富，支持多种编程范式，例如面向对象。

D 语言最初由 Digital Mars 公司就职的 Walter Bright 于 2001 年发布，意图改进 C++语言。目前最新 D 语言被简称为 D2。最主要的 D 语言的实现是 DMD。

D 语言源自 C/C++，借鉴了众多编程语言的特色和现代编译器技术，融会贯通了设计者丰富的实践经验，使之具备了非凡的威力：既有 C/C++ 语言的强大威力，又有 Python 和 Ruby 的开发效率。它集众多系统级编程所需的功能于一身，例如垃圾回收、手工内存操作、契约式设计、高级模板技术、内嵌汇编、内置单元测试、Mixin 风格多继承、类 Java 包管理机制、内置同步机制、内建基本运行时信息。

# 21 世纪编程语言发展

在 21 世纪截止本书完成时，计算机科学家发明了 C#，Visual Basic .NET，F#，Groovy，Scala，Clojure，Go，Julia，Dart，Kotlin，Rust，Swift 等编程语言。

**C#**

C#是微软公司在 2000 年 6 月发布的一种新的编程语言，主要由原 Borland 公司的首席研发设计师安德斯·海尔斯伯格（Anders Hejlsberg）主持开发，它是第一个面向组件的编程语言，其源码会编译成 msil 再运行。它借鉴了 Delphi 的一个特点，与 COM(组件对象模型)直接集成，并且新增了许多功能及语法，而且它是微软公司.NET windows 网络框架的主角。

C#最初有个更酷的名字，叫 COOL。微软从 1998 年 12 月开始了 COOL 项目，直到 2000 年 2 月，COOL 被正式更名为 C#。在 1998 年，Delphi 语言的设计者 Hejlsberg 带领着 Microsoft 公司的开发团队，开始了第一个版本 C#语言的设计。在 2000 年 9 月，国际信息和通信系统标准化组织为 C#语言定义了一个 Microsoft 公司建议的标准。最终 C#语言在 2001 年得以正式发布。

安德斯·海尔斯伯格在微软开发了 Visual J++ 1.0，很快的 Visual J++由 1.1 版本升级到 6.0 版。SUN 公司认为 Visual J++ 违反了 Java 开发平台的中立性，对微软提出了诉讼。2000 年 6 月 26 日微软在奥兰多举行的“职业开发人员技术大会”上，发表新的语言 C#。C#语言取代了 Visual J++，语言本身深受 Java、C 和 C++ 的影响。

C#是.NET 平台主语言，于 2000 年和.NET 开发平台一起由微软发布。.NET 开发平台包括，运行 IL 语言的虚拟机 CLR，基于 IL 语言的公共类库，用于编译产生 IL 语言程序的高级语言，以及支持这些高级语言开发的编译的基础开发环境。

由于 CLR 和 IL 公共类库作为 Window 操作系统的一部分捆绑发布，所以 IL 语言程序可以在 Window 操作系统上直接执行，就像机器指令一样。

CLR 运行 IL 和 JM 运行 Java 一样，只不过 CLR 将 IL 编译成机器指令，然后在由计算

机直接执行，而 JM 是将 Java 编译成字节码，然后由解释器解释执行。

C# 1.0 基于 Java 和 C++，以 Java 为基础将更多的 C++加入进来，并且吸收了一些 Delphi 和 Visual Basic 的特性。C# 2.0 引入泛型编程。C# 4.0 支持了 lambda 表达式，Linq 等。.NET 4.0 引入了动态类型，使得.NET 平台可以嵌入动态语言，比如：Lisp、Python、 Ruby 等。

**Visual Basic .NET**

2001 年，Visual Basic .NET 推出。Quick BASIC 由美国微软公司开发，1987 年推出，它是 BASIC 的继承者。Quick BASIC 是 DOS 下的开发语言，后来随着 Window 系统的推出加入了图像化设计所以新的版本改名为：Visual Basic。Visual Basic 是第一个支持可视化界面设计的语言。

1998 年发布的 Visual Basic 6.0 是一个应用非常广泛的版本。

2001 年，Visual Basic 迁移到了.NET 平台上，改名 Visual Basic .NET。Visual Basic .NET 对 BASIC 语言特性进行了重大改进。从此 Visual Basic .NET 伴随.NET 开发平台一起发展。

**F#**

F#语言诞生于 2002 年，它是由微软发展的为微软.NET 语言提供运行环境的程序设计语言。

F#基于 Ocaml 的，而 Ocaml 是基于 ML 函数程序设计语言的。这是一个用于显示.NET 在不同编程语言间互通的程序设计。自 2002 年开始研发，2005 年发布了第一个版本，2007 年底正式从研发专案转移至产品部门，并决定将 F#置入 Visual Studio 2010。

微软计划将 F#慢慢整合至.NET 平台并使 F#成为.NET 平台的计算辅助语言。众所周知，F#是一种函数型程序设计（FP，Functional Programming）语言。然而 F#对 IP（Imperative Programming）与 OOP（Object Oriented Programming）的支持几乎一样的出色。

**Groovy**

Groovy 语言诞生于 2003 年，它是 Apache 旗下的一门基于 JVM 平台的动态/敏捷编程语言，在语言的设计上它吸纳了 Python、Ruby 和 Smalltalk 语言的优秀特性，语法非常简练和优美，开发效率也非常高。

Groovy 可以与 Java 语言无缝对接，在写 Groovy 的时候如果忘记了语法可以直接按 Java 的语法继续写，也可以在 Java 中调用 Groovy 脚本，都可以很好的工作，这有效的降低了 Java 开发者学习 Groovy 的成本。

但 Groovy 并不会替代 Java，而是相辅相成、互补的关系，具体使用哪门语言这取决于要解决的问题和使用的场景。

**Scala**

Scala 语言诞生于 2003 年，它是一门多范式的编程语言，一种类似 java 的编程语言，设计初衷是实现可伸缩的语言、并集成面向对象编程和函数式编程的各种特性。Scala 可能是下一代 Java。它被特意设计成能够与 Java 和.NET 互操作。

Scala 和 Groovy 之间的核心区别在于前者是静态类型的。联邦理工学院洛桑（EPFL）的 Martin Odersky 于 2001 年基于 Funnel 的工作开始设计 Scala。Funnel 是把函数式编程思想和 Petri 网相结合的一种编程语言。Odersky 先前的工作是 Generic Java 和 javac（Sun Java 编译器）。Java 平台的 Scala 于 2004 年初发布。.NET 平台的 Scala 发布于 2004 年 6 月。该语言第二个版本，v2.0，发布于 2006 年 3 月。

2009 年 4 月，Twitter 宣布他们已经把大部分后端程序从 Ruby 迁移到 Scala，其余部分

也打算要迁移。此外， Wattzon 已经公开宣称，其整个平台都已经是基于 Scala 基础设施编写的。

**Clojure**

Clojure 语言诞生于 2007 年，它是一种运行在 Java 平台上的 Lisp 方言。

Lisp 是一种以表达性和功能强大著称的编程语言，但人们通常认为它不太适合应用于一般情况，而 Clojure 的出现彻底改变了这一现状。如今，在任何具备 Java 虚拟机的地方，您都可以利用 Lisp 的强大功能。

**Go 语言**

Go 语言（又称 Golang）诞生于 2009 年，它是 Google 的 Robert Griesemer，Rob Pike 及 Ken Thompson 开发的一种静态强类型、编译型语言。Go 语言语法与 C 相近，但功能上有：内存安全，GC（垃圾回收），结构形态及 CSP-style 并发计算。Go（又称 Golang）是 Google 开发的一种静态强类型、编译型、并发型，并具有垃圾回收功能的编程语言。

罗伯特·格瑞史莫（Robert Griesemer），罗勃·派克（Rob Pike）及肯·汤普逊（Ken Thompson）于 2007 年 9 月开始设计 Go，稍后 Ian Lance Taylor、Russ Cox 加入项目。Go 是基于 Inferno 操作系统所开发的。Go 于 2009 年 11 月正式宣布推出，成为开放源代码项目，并在 Linux 及 Mac OS X 平台上进行了实现，后来追加了 Windows 系统下的实现。在 2016 年，Go 被软件评价公司 TIOBE 选为“TIOBE 2016 年最佳语言”。 目前，Go 每半年发布一个二级版本（即从 a.x 升级到 a.y）

Go 的语法接近 C 语言，但对于变量的声明有所不同。Go 支持垃圾回收功能。Go 的并发模型是以东尼·霍尔的通信顺序进程（CSP）为基础，采取类似模型的其他语言包括 Occam 和 Limbo，但它也具有 Pi 运算的特征，比如通道传输。在 1.8 版本中开放插件（Plugin）的支持，这意味着现在能从 Go 中动态加载部分函数。

与 C++相比，Go 并不包括如枚举、异常处理、继承、泛型、断言、虚函数等功能，但增加了 切片(Slice) 型、并发、管道、垃圾回收、接口（Interface）等特性的语言级支持。Go 2.0 版本将支持泛型，对于断言的存在，则持负面态度，同时也为自己不提供类型继承来辩护。

不同于 Java，Go 内嵌了关联数组（也称为哈希表（Hashes）或字典（Dictionaries）），就像字符串类型一样。

Go 语言是 Java 的直接竞争者。

**Julia**

Julia 诞生于 2009 年，它是一个面向科学计算的高性能动态高级程序设计语言。其语法与其他科学计算语言相似。在许多情况下拥有能与编译型语言相媲美的性能。Julia 是个灵活的动态语言，适合科学和数值计算，性能可与传统静态类型语言媲美。

Julia 项目大约于 2009 年中开始。一群拥有各种语言丰富编程经验的 Matlab 高级用户，对现有的科学计算编程工具感到不满：这些软件对自己专长的领域表现得非常棒，但在其它领域却非常糟糕。他们想要的是这样一个开源的软件：要像 C 语言一般快速而有拥有如同 Ruby 的动态性；要具有 Lisp 般真正的同像性（Homoiconicity）而又有 Matlab 般熟悉的数学记号；要像 Python 般通用、像 R 般在统计分析上得心应手、像 Perl 般自然地处理字符串、像 Matlab 般具有强大的线性代数运算能力、像 shell 般胶水语言的能力，易于学习而又不让真正的黑客感到无聊；还有，它应该是交互式的，同时又是编译型的。

**Dart**

Dart 语言诞生于 2011 年，它是谷歌开发的计算机编程语言，后来被 Ecma (ECMA-408) 认定为标准。它被用于 Web、服务器、移动应用和物联网等领域的开发。它是宽松开源许可证（修改的 BSD 证书）下的开源软件。

Dart 是面向对象的、类定义的、单继承的语言。它的语法类似 C 语言，可以转译为 JavaScript，支持接口(interfaces)、混入(mixins)、抽象类(abstract classes)、具体化泛型(reified generics)、可选类型(optional typing)和 sound type system。

**Kotlin**

Kotlin 语言诞生于 2011 年。2011 年 7 月，JetBrains 推出 baiKotlin 项目，这是一个面向 duJVM 的新语言 ，它已被开发一年之久。JetBrains 负责人 Dmitry Jemerov 说，大多数语言没有他们正在寻找的特性，Scala 除外。但是，他指出了 Scala 的编译时间慢这一明显缺陷。Kotlin 的既定目标之一是像 Java 一样快速编译。2012 年 2 月，JetBrains 以 Apache 2 许可证开源此项目。

Jetbrains 希望这个新语言能够推动 IntelliJ IDEA 的销售。Kotlin v1.0 于 2016 年 2 月 15 日发布。这被认为是第一个官方稳定版本，并且 JetBrains 已准备从该版本开始的长期向后兼容性。在 Google I/O 2017 中，Google 宣布在 Android 上为 Kotlin 提供支持。

Kotlin 的设计目标包括：

创建一种兼容 Java 的语言

让它比 Java 更安全，能够静态检测常见的陷阱。如：引用空指针

让它比 Java 更简洁，通过支持 variable type inference，higher-order functions (closures)，extension functions，mixins and first-class delegation 等实现。

让它比最成熟的竞争对手 Scala 语言更加简单。

Kotlin 可以编译成 Java 字节码，也可以编译成 JavaScript，方便在没有 JVM 的设备上运行。除此之外 Kotlin 还可以编译成二进制代码直接运行在机器上（例如嵌入式设备或 iOS）。

Kotlin 已正式成为 Android 官方支持开发语言。

**Swift**

Swift 语言诞生于 2014 年，是苹果于 2014 年 WWDC 苹果开发者大会发布的新开发语言，可与 Objective-C 共同运行于 macOS 和 iOS 平台，用于搭建基于苹果平台的应用程序。

Swift 是一款易学易用的编程语言，而且它还是第一套具有与脚本语言同样的表现力和趣味性的系统编程语言。Swift 的设计以安全为出发点，以避免各种常见的编程错误类别。

Swift 结合了 C 和 Objective-C 的优点并且不受 C 兼容性的限制。Swift 采用安全的编程模式并添加了很多新特性，这将使编程更简单，更灵活，也更有趣。Swift 是基于成熟而且倍受喜爱的 Cocoa 和 Cocoa Touch 框架。

**Rust**

Rust 诞生于 2014 年。Rust 是一门系统编程语言，专注于安全，尤其是并发安全，支持函数式和命令式以及泛型等编程范式的多范式语言。Rust 在语法上和 C++类似，但是设计者想要在保证性能的同时提供更好的内存安全。

Rust 最初是由 Mozilla 研究院的 Graydon Hoare 设计创造，然后在 Dave Herman, Brendan Eich 以及很多其他人的贡献下逐步完善的。Rust 的设计者们通过在研发 Servo 网站浏览器布局引擎过程中积累的经验优化了 Rust 语言和 Rust 编译器。

Rust 编译器是在 MIT License 和 Apache License 2.0 双重协议声明下的免费开源软件。

Rust 已经连续四年（2016，2017，2018，2019）在 Stack Overflow 开发者调查的“最受喜爱编程语言”评选项目中折取桂冠。

Rust 是一门革命性的编程语言，它拥有和 C/C++同样的高性能，却又比 C/C++安全。它既无 GC 机制，又不用手动释放内存，还不会有各种溢出漏洞，Rust 是 C/C++，Go，Java 等编程语言的直接竞争者。

Rust 语言可以应用于命令行工具，Web 服务，嵌入式，系统程序等的项目开发。微软的云开发者代言人 Ryan Levick 曾经在 AllThingsOpen 虚拟会议上表示，微软正逐渐从 C/C++改用 Rust 来构建其基础设施软件。微软还鼓励其他软件行业巨头考虑采取同样的做法。

有关编程语言的进化史，我们就给大家讲到这里。从编程语言的历史发展过程中可以看出，许多编程语言的发展，都是基于已有的研究成果，每门编程语言的出现和进化，都有可能借鉴和吸收了其他语言的一些创新特性，比如面向对象特性，比如函数式编程，比如垃圾回收机制等等。

因此在选择编程语言的时候，我们不要陷入编程语言的优劣之争，选择适合自己和适合工作需求的编程语言就是最好的。具体来说：

如果你对信息安全有兴趣，那么你可能需要学习 C，汇编，Rust 等。

如果你对大数据和人工智能感兴趣，那么你可能需要学习 Python，C/C++等。

如果你对后台开发感兴趣，那么你可能需要学习 Java，Go，Rust 等。

如果你暂时不知道对什么感兴趣，那么就推荐从 C 语言开始学习吧，因为学会了 C 语言，再学其他编程语言就会非常容易。

最后，需要注意的是程序员往往不止要会一门编程语言，以信息安全为例子，很多安全技术要求安全工程师必须掌握 C，C++，Java，Python，SQL，Javascript，PHP 等语言。